

TMS320DM320
CPU and Peripherals
Vol - 1

Technical Reference Manual
Version 0.3

IMPORTANT NOTICE

Texas Instruments and its subsidiaries (TI) reserve the right to make changes to their products or to discontinue any product or service without notice, and advise customers to obtain the latest version of relevant information to verify, before placing orders, that information being relied on is current and complete. All products are sold subject to the terms and conditions of sale supplied at the time of order acknowledgment, including those pertaining to warranty, patent infringement, and limitation of liability.

TI warrants performance of its semiconductor products to the specifications applicable at the time of sale in accordance with TI's standard warranty. Testing and other quality control techniques are utilized to the extent TI deems necessary to support this warranty. Specific testing of all parameters of each device is not necessarily performed, except those mandated by government requirements.

Customers are responsible for their applications using TI components. In order to minimize risks associated with the customer's applications, adequate design and operating safeguards must be provided by the customer to minimize inherent or procedural hazards.

TI assumes no liability for applications assistance or customer product design. TI does not warrant or represent that any license, either express or implied, is granted under any patent right, copyright, mask work right, or other intellectual property right of TI covering or relating to any combination, machine, or process in which such semiconductor products or services might be or are used. TI's publication of information regarding any third party's products or services does not constitute TI's approval, warranty or endorsement thereof.

Mailing Address:

Texas Instruments
Post Office Box 655303
Dallas, Texas 75265

Copyright © 2003, Texas Instruments Incorporated

Document Revision History

Revision	Date	Notes
0.1	15 Dec, 2003	Draft version
0.2	5 Jan, 2004	Updated according to review comments
0.3	30 Jan, 2004	Updated after reviews

Read This First

About This Manual

This Technical Reference Manual (TRM) serves as a reference for the Texas Instruments TMS320DM320 low power, enhanced-architecture, dual-core digital signal processor (DSP). It is intended to assist hardware and software engineers in developing applications using this device. It describes both the cores of the chip and their coprocessors and peripherals — ARM926EJ microcontroller unit (MCU), the TMS320C54x DSP CPU, the Variable Length Coder and the Imaging Extension Coprocessor, DCT and Sequencer together with the memory and peripherals associated with core.

Throughout this document, the TMS320DM320 dual-core device is referred to as the DM320. Where the ARM926EJS MCU core is referred to separately, the alphanumeric designation is shortened to ARM.

Notational Conventions

This book uses the following conventions.

- Chip Support Library (CSL) library and macro notation is used throughout this book. Refer Appendix for description about this notation
- Program listings and program examples are shown in a special typeface. Here is a segment of a program listing:

```
OSD_FSET(VIDWINADH, V1AH, vwin1_start >> 16 );  
OSD_RSET(VINWIN1AL, vwin1_start & 0xFFFF );  
  
while ( MMCSF_FGET( DMASTAT1, RUNST ) == 1 );
```

- Instruction sets

The c5409A DSP module in DM320 can use either of two forms of the instruction set, viz. a mnemonic form or an algebraic form. This book uses the mnemonic form of the instruction set. For information about the mnemonic form of the instruction set, see TMS320C54x DSP Reference Set, Volume 2: Mnemonic Instruction Set. For information about the algebraic form of the instruction set, see TMS320C54x DSP Reference Set, Volume 3: Algebraic Instruction Set. These references are both listed in the section titled Related Documentation From Texas Instruments.

The ARM926J MCU module in DM320 can use either of the instruction set it supports viz. 16 bit Thumb mode and 32 bit ARM mode instruction sets. For information about the MCU's instruction set, see TMS470R1x User's Guide or ARM instruction manuals from ARM Ltd (www.arm.com).

Related DM320 Documentation From Texas Instruments

Refer the following documents for additional information about DM320

- DM320 Datasheet
- DM320 EVM User's Guide
- DM320 TRM Errata

Related Documentation

The following documents can be downloaded from <http://www.arm.com/>

- "ARM926EJ-S Product overview" gives an overview of ARM926EJ-S core from ARM.
- "ARM926EJ-S Technical reference manual" discusses about the various features and peripherals of ARM926EJ-S

The following documents provide related documentation for the TMS320DM320. To obtain a copy of any of these TI documents, call the Texas Instruments Literature Response Center at (800) 477-8924. When ordering, please identify the book by its title and literature number. Many of these documents are located on the Internet at <http://www.ti.com/>.

- TMS320C54x DSP Reference Set, Volume 1: CPU and Peripherals (literature number SPRU131F) provides information for developing hardware and software applications using the TMS320C54x
- TMS320C54x DSP Reference Set, Volume 2: Mnemonic Instruction Set (literature number SPRU172) describes the TMS320C54x digital signal processor mnemonic instructions individually. Also includes a summary of instruction set classes and cycles.
- TMS320C54x DSP Reference Set, Volume 3: Algebraic Instruction Set (literature number SPRU179) describes the TMS320C54x digital signal processor algebraic instructions individually. Also includes a summary of instruction set classes and cycles.
- TMS320C54x Assembly Language Tools User's Guide (literature number SPRU102) describes the assembly language tools (assembler, linker, and other tools used to develop assembly language code), assembler directives, macros, common object file format, and symbolic debugging directives for the C54x generation of devices.
- TMS320C54x C Source Debugger User's Guide (literature number SPRU099) tells you how to invoke the C54x emulator, evaluation module, and simulator versions of the C source debugger interface. This book discusses various aspects of the debugger interface, including window management, command entry, code execution, data management, and breakpoints. It also includes a tutorial that introduces basic debugger functionality.
- TMS320C54x Code Generation Tools Getting Started Guide (literature number SPRU147) describes how to install the TMS320C54x assembly language tools and the C compiler for the C54x devices. The installation

for MS-DOS., OS/2., SunOS., Solaris., and HP-UX. 9.0x systems is covered.

- ° TMS320C54x Optimizing C/C++ Compiler User's Guide (literature number SPRU103) describes the C54x C compiler. This C compiler accepts ANSI standard C source code and produces TMS320. assembly language source code for the C54x generation of devices.
- ° TMS320 Third-Party Support Reference Guide (literature number SPRU052) alphabetically lists over 100 third parties that provide various products that serve the family of TMS320 digital signal processors. A myriad of products and applications are offered—software and hardware development tools, speech recognition, image processing, noise cancellation, modems, etc.
- ° TMS320 DSP Development Support Reference Guide (literature number SPRU011) describes the TMS320_ DSP family of digital signal processors and the tools that support these devices. Included are code-generation tools (compilers, assemblers, linkers, etc.) and system integration and debug tools (simulators, emulators, evaluation modules, etc.). Also covered are available documentation, seminars, the university program, and factory repair and exchange.

Trademarks

TMS320C54x, C54x, TMS320, ICECrusher, and MicroStar BGA are trademarks of Texas Instruments.

ARM926EJ, ARM, Thumb, and Multi-ICE are registered trademarks of ARM Limited.

I²C is a registered trademark of PHILLIPS

Other trademarks are the property of their respective owners.

Table Of Contents

READ THIS FIRST	4
TABLE OF CONTENTS	7
1 INTRODUCTION	13
1.1 KEY FEATURES	14
1.2 PERFORMANCE	15
1.3 BLOCK DIAGRAM	16
1.4 FEATURES OF ARM SUBSYSTEM	17
1.5 FEATURES OF THE DSP SUBSYSTEM	19
1.6 FEATURES OF CO-PROCESSOR SUB-SYSTEM	20
1.7 FEATURES OF THE IMAGING PERIPHERAL BLOCK	22
1.8 FEATURES OF THE MEMORY TRAFFIC CONTROLLER BLOCK	24
1.9 DM320 SYSTEM DATA FLOWS	26
2 ARM SUBSYSTEM	29
2.1 INTRODUCTION	29
2.2 MEMORY MAP	30
2.3 BUS MASTERS AND SLAVES	32
2.4 INTERCONNECT BUS ACCESS	33
2.5 HIERARCHICAL SUB-SYSTEM	35
2.6 BOOT UP MODES	35
2.7 DATA ACCESS MODES	36
3 ARM CORE	38
3.1 INTRODUCTION	38
4 ARM BUS CONTROLLERS	51
4.1 INTRODUCTION	51
4.2 AIM RE-MAP	51
4.3 ENDIAN CONVERSION	53
4.4 SDRAM AND CS0 ADDRESS	55
4.5 PERIPHERAL AREA	56
4.6 BUS CONTROLLER REGISTER MAP (BUSC)	58
4.7 BUS CONTROLLER REGISTERS	58
4.8 AHB BUS CONTROLLER REGISTER MAP (AHBBUSC)	60
4.9 AHB BUS CONTROLLER REGISTERS	60
5 CLOCK CONTROLLER	63
5.1 INTRODUCTION	63
5.2 CLOCK OPERATIONS (SELECTION, DIVISION AND INVERSION)	67
5.3 SETTING PLL CLOCK OUTPUT FREQUENCY	68
5.4 PLL BYPASS MODE	69
5.5 MODULE CLOCK DISTRIBUTION	71
5.6 GIO CLOCK OUTPUT	71
5.7 PULSE WIDTH MODULATION	71
5.8 SLEEP AND POWER-DOWN MODES	73

5.9	CLOCK CONTROLLER REGISTER MAP (CLKC)	74
5.10	CLOCK CONTROLLER REGISTERS	75
6	INTERRUPT CONTROLLER	93
6.1	INTRODUCTION	93
6.2	INTERRUPT SOURCE LIST	94
6.3	INTC METHODOLOGY	95
6.4	CONFIGURABLE INTERRUPT ENTRY TABLE	97
6.5	EXTERNAL INTERRUPTS.....	99
6.6	INTERRUPT CONTROLLER REGISTER MAP (INTC)	100
6.7	INTERRUPT CONTROLLER REGISTERS	101
7	TIMERS	145
7.1	INTRODUCTION	145
7.2	CONFIGURING THE TIMERS	145
7.3	TIMER REGISTER MAP (TMRx).....	150
8	WATCH DOG TIMER (WDT)	158
8.1	INTRODUCTION	158
8.2	USING THE WATCH DOG TIMER	158
8.3	SETTING THE CONFIGURABLE DETECTION TIME.....	159
8.4	WDT OPERATION.....	160
8.5	WATCH DOG TIMER REGISTER MAP (WDT)	161
8.6	WATCH DOG TIMER REGISTERS	162
9	CCD CONTROLLER	164
9.1	FEATURES	164
9.2	CCDC INPUT SIGNAL INTERFACE	165
9.3	FRAME SETUP	166
9.4	CCDC PROCESSING.....	167
9.5	CCD INTERRUPTS	171
9.6	YCbCr SIGNAL PROCESSING	172
9.7	INPUT/OUTPUT PORT CONFIGURATION	173
9.8	CCIR-656 4:2:2 PARALLEL INTERFACE	173
9.9	CCDC REGISTER UPDATE TIMING.....	175
9.10	CCD CONTROLLER REGISTER MAP (CCDC).....	177
9.11	CCD CONTROLLER REGISTERS	178
10	PREVIEW ENGINE.....	206
10.1	INTRODUCTION	206
10.2	FUNCTIONAL DESCRIPTION.....	207
10.3	HISTOGRAM	216
10.4	PREVIEW ENGINE OPERATION GUIDE	217
10.5	FRACTIONAL NOTATION	229
10.6	PREVIEW ENGINE REGISTER MAP (PREV)	230
10.7	PREVIEW ENGINE REGISTERS	232
11	HARDWARE 3A	271
11.1	INTRODUCTION	271
11.2	FEATURES	271
11.3	INPUT DATA FOR H3A ENGINE	272
11.4	AUTO FOCUS (AF) ENGINE	272
11.5	AUTO EXPOSURE, AUTO WHITE-BALANCE ENGINE (AEWB)	278
11.6	HARDWARE 3A REGISTER MAP (H3A).....	283
11.7	HARDWARE 3A REGISTERS	284

12	OSD - ON-SCREEN DISPLAY	308
12.1	INTRODUCTION	308
12.2	FEATURES	308
12.3	OSD BLOCK DIAGRAM	309
12.4	BITMAP AND VIDEO WINDOWS	309
12.5	OSD WINDOW DATA STORAGE FORMAT	310
12.6	OSD CONFIGURATION AND CONTROL	312
12.7	BITMAP WINDOW	317
12.8	RECTANGULAR HARDWARE CURSOR	323
12.9	WINDOW EXPANSION (VGA TO NTSC/PAL CONVERSION)	324
12.10	OSD PING PONG BUFFER	325
12.11	ON SCREEN DISPLAY REGISTER MAP (OSD)	326
12.12	ON SCREEN DISPLAY REGISTERS	328
13	VIDEO ENCODER	369
13.1	INTRODUCTION	369
13.2	VIDEO ENCODER OPERATING MODES	371
13.3	FRONT-END DATA PROCESSING	372
13.4	NTSC/PAL VIDEO ENCODER	374
13.5	DIGITAL LCD CONTROLLER	379
13.6	INTERRUPTS	392
13.7	DAC CONTROL	392
13.8	VIDEO ENCODER REGISTER MAP (VENC)	393
13.9	VIDEO ENCODER INTERFACE REGISTERS	395
14	GENERAL PURPOSE INPUT OUTPUT (GIO)	442
14.1	INTRODUCTION	442
14.2	GIO PORT STATE CONTROL	442
14.3	EXTERNAL INTERRUPT SETTINGS	443
14.4	GIO FUNCTIONAL SETTINGS	443
14.5	MEMORY CARD DETECTION	450
14.6	GENERAL I/O REGISTER MAP (GIO)	451
14.7	GENERAL I/O REGISTERS	452
15	SERIAL PORT INTERFACE (SPI)	476
15.1	INTRODUCTION	476
15.2	SERIAL TRANSMIT / RECEIVE	477
15.3	SERIAL INTERFACE SIGNAL PINS	480
15.4	BAUD RATE	480
15.5	MASTER MODE SIF	480
15.6	SLAVE MODE SIF	481
15.7	TRANSMISSION AND RECEPTION BUFFERS	482
15.8	SERIAL PORT MODES	483
15.9	INTERRUPTS	483
15.10	TRANSFER USING DMA	483
15.11	PROGRAMMING PROCEDURE FOR SPI 0/1 (NON-DMA MODE)	485
15.12	PROGRAMMING PROCEDURE FOR SPI 0 (DMA MODE)	486
15.13	SERIAL PORT REGISTER MAPS (SPx)	487
15.14	SERIAL PORT REGISTERS	487
16	UART	492
16.1	INTRODUCTION	492
16.2	UART SIGNAL PINS	493
16.3	PROGRAMMING PROCEDURE FOR UART	494
16.4	UART REGISTER MAPS (UARTx)	494

16.5	UART REGISTERS	495
17	MMC/SD INTERFACE	502
17.1	INTRODUCTION	502
17.2	FEATURES	502
17.3	MMC CONTROLLER PINS	503
17.4	FUNCTION CLOCK AND MEMORY CLOCK	503
17.5	MMC CONTROLLER REGISTERS	504
17.6	MMC CONTROLLER INTERRUPTS	506
17.7	MMC DMA CONTROLLER	508
17.8	DATA FLOW IN THE DATA REGISTERS (MMCDRR AND MMCDXR)	509
17.9	MMC/SD MODE	510
17.10	SDIO CARD FUNCTION	528
17.11	MMC/SD CARD INTERFACE REGISTER MAP	529
17.12	MMC/SD CARD INTERFACE REGISTERS	530
18	USB OTG INTERFACE	569
18.1	INTRODUCTION	569
18.2	DEVICE CONFIGURATION	570
18.3	CONTROLLER INITIALIZATION	573
18.4	OTG SIGNALS	573
18.5	REGISTER DESCRIPTION	574
18.6	USB INTERRUPT CONTROL	574
18.7	USB BUS STATE	577
18.8	HANDLING CONTROL TRANSACTIONS	578
18.9	BULK / INTERRUPT / ISOCHRONOUS TRANSACTIONS	584
18.10	CONNECT / DISCONNECT	591
18.11	OTG SESSION REQUEST	592
18.12	HOST NEGOTIATION	593
18.13	VBus EVENTS	594
18.14	USB DMA CONTROLLER	595
18.15	USB REGISTER MAP	598
18.16	USB DMA CONTROLLER REGISTERS	634
19	I²C	648
19.1	INTRODUCTION	648
19.2	FEATURES	648
19.3	I ² C BUS CONFIGURATION	648
19.4	DATA TRANSFER SEQUENCE	650
19.5	ISSUING OF START, STOP AND RESTART CONDITIONS	650
19.6	DATA REGISTER FOR TRANSFER	651
19.7	ACKNOWLEDGEMENT BIT	651
19.8	TRIGGERING DATA TRANSFER	653
19.9	I ² C INTERRUPT	653
19.10	TRANSMIT (7BIT ADDRESS)	654
19.11	RECEIVE (7BIT ADDRESS)	655
19.12	MIXED TRANSFER (7BIT ADDRESS)	655
19.13	TRANSMIT (10BIT ADDRESS)	657
19.14	RECEIVE (10BIT ADDRESS)	658
19.15	MIXED TRANSFER (10BIT ADDRESS)	659
19.16	I2C REGISTER MAP (I2C)	661
19.17	I2C REGISTERS	662
20	MEMORY TRAFFIC CONTROLLER (MTC)	665
20.1	INTRODUCTION	665
20.2	SDRAM CONTROLLER	679

20.3	SDRAM INITIALIZATION	686
20.4	MTC DMA CONTROLLER	690
20.5	SDRAM & EXTERNAL MEMORY I/F PIN SHARING	692
20.6	EMIF REGISTER MAP	697
20.7	EXTERNAL MEMORY INTERFACE REGISTERS	698
20.8	SDRC REGISTER MAP	735
20.9	SDRAM CONTROLLER REGISTERS	736
21	SMARTMEDIA INTERFACE	761
21.1	INTRODUCTION	761
21.2	FEATURES	761
21.3	PROGRAMMING THE SMARTMEDIA	763
21.4	HIGH SPEED PROGRAMMING	768
22	COMPACT FLASH CARD CONTROLLER	771
22.1	INTRODUCTION	771
22.2	REFERENCES	771
22.3	MEMORY MAP	771
22.4	COMPACT FLASH INTERFACE	772
22.5	COMPACT FLASH STORAGE CARD	774
22.6	SOFTWARE INTERFACE	776
23	VLYNQ	786
23.1	INTRODUCTION	786
23.2	REFERENCES	787
23.3	OPERATION	787
23.4	ADDRESS TRANSLATION	789
23.5	INTERRUPTS	791
23.6	CLOCKING	792
23.7	FLOW CONTROL	792
23.8	REGISTER DESCRIPTION	792
24	DSP SUBSYSTEM	820
24.1	INTRODUCTION	820
24.2	DSP CORE	820
24.3	DSP SUBSYSTEM MEMORY	821
24.4	DSP INTERNAL MEMORY	823
24.5	DSP PERIPHERALS	827
24.6	DSP - POWER SAVING MODES	831
25	ARM-DSP COMMUNICATION	833
25.1	INTRODUCTION	833
25.2	HPI-16 MODE	834
25.3	MEMORY ACCESS	835
25.4	INTERRUPTS BETWEEN ARM AND DSP	835
25.5	DSP BOOT SEQUENCE	836
25.6	ARM-SIDE HPI CONTROL AND STATUS REGISTERS	837
25.7	DSP SUBSYSTEM - HPIB DMAC	839
25.8	DSP CONTROLLER (DSPC)	841
25.9	DSP CONTROLLER REGISTERS (ARM SIDE)	842
25.10	HOST PORT INTERFACE BRIDGE REGISTER MAP (HPIB)	844
25.11	HOST PORT INTERFACE BRIDGE REGISTERS	845
26	ARM BOOT-UP MODES	851
26.1	EXTERNAL MEMORY BOOT-UP MODE	852
26.2	SERIAL BOOT-UP MODE	852

26.3	EXTERNAL HOST EMIF BOOT-UP MODE	855
26.4	INTERNAL ROM BOOT-UP MODE	856
27	ARM POWER SAVING MODES	858
27.1	DISABLING MODULE CLOCKS	858
27.2	USB POWER SAVING MODES	858
27.3	SLEEP MODE	859
27.4	POWER DOWN MODE	860
27.5	ENTERING THE SDRAM INTO SELF REFRESH MODE	862
27.6	PLL POWER DOWN MODE	862
28	APPENDIX A	863
28.1	CHIP SUPPORT LIBRARY (CSL) MACRO NOTATION	863

1 Introduction

The DM320 is a highly integrated, programmable platform for the Digital still camera, Digital camcorder, PDAs and Music players. Designed to offer camera manufacturers the ability to produce affordable DSC products with high picture quality, the DM320 combines programmable image processing capability with a highly integrated imaging peripheral set. The chip's programmability comes from a DSP-based imaging co-processor that enables camera manufacturers to implement their own proprietary image processing algorithms in software. The DM320 also enables seamless interface to most additional external devices required for a complete digital camera implementation. The interface is flexible enough to support various types of CCD and CMOS sensors, signal conditioning circuits, power management, SDRAM, shutter, Iris and auto-focus motor controls. The DM320 will allow camera manufacturers to meet customer demands by fulfilling both image quality and low cost expectations required in the medium segment of digital still camera market.

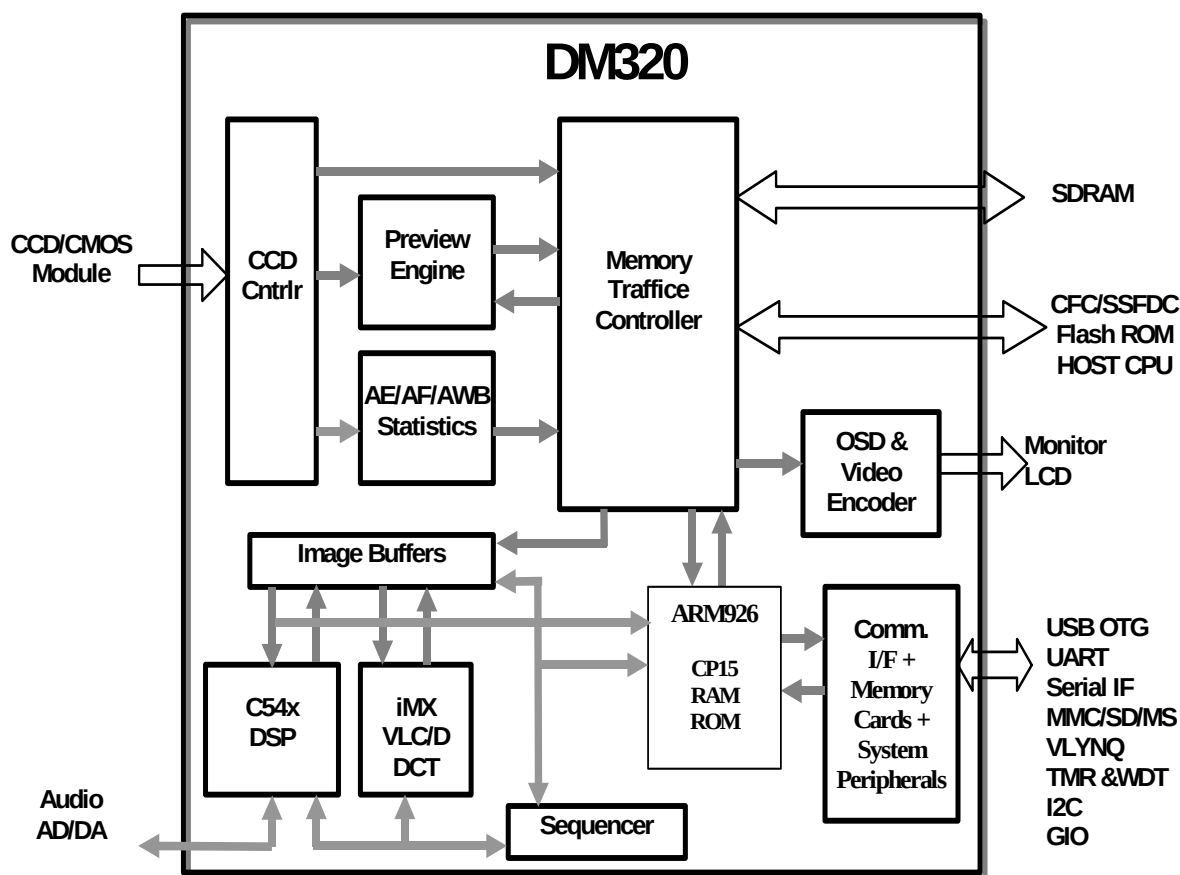


Figure 1: DM320 System Architecture

1.1 Key Features

- 306 pin 16x16 mm (0.8mm pitch) or 325 pin 12x12 mm (0.5 mm pitch) PBGA package
- Supports 108 MHz SDRAM up to 256 Mbytes with 32/16-bit interface *
- Supports up to 14-bit AFE (Analog Front End) and 4 field transfers
- Supports 8/16bit CCIR601/656 video format
- 180 MHz ARM926EJ 32 bit RISC microprocessor with 16KB I-cache, 8KB Data cache, MMU and Java accelerator *
- On-chip 8 KB ROM, 16 KB ARM internal memory
- 100MHz Texas Instruments C54X Digital Signal Processor (DSP) core with on-chip program/data memory (128Kbytes) *
- On-chip image processing buffers (24Kbytes)
- 180MHz programmable SIMD image processing engine (iMX) for programmable image processing with 8 MACs *
- DCT module to perform DCT and IDCT transforms.
- Variable Length Coding and Decoding coprocessor (VLC/VLD) with Quantization / Inverse Quantization (QIQ) for image/video compression and decompression.
- Sequencer for synchronizing the operation of Co-processors with ARM and DSP.
- ARM / DSP / Sequencer can control the operations of all co-processors and image buffers
- Preview engine for real-time live preview and digital zoom
- Histogram support in preview engine
- Hardware 3A module to compute the Auto exposure, Auto white-balance and Auto focus parameters
- Versatile On Screen Display (OSD) with multiple window support for video, bitmaps and cursor windows
- OSD supports different zoom factors
- 8bit/16 bit RGB and YUV digital interface for color LCD
- Seamless interface Compact Flash, Smart Media, MMC/SD, and Memory Stick cards
- Supports flash memory up to 16MB
- Supports 27MHz, 48MHz (USB) crystal input to generate all clocks
- Four Timers + One WDT, with precise shutter control feature
- Two UARTs with 32 byte FIFOs for receive and transmit

- 1-channel 10-bit DAC for NTSC/PAL composite video output
- 2-channel serial audio codec interface (McBSP)
- 3 SDRAM channels for DMA and a dedicated channel for Image buffer DMA
- MTC DMA available between SIF / AIM / Memory stick / MMC-SD / DSP DARAM and EMIF regions / SDRAM
- Two sets of SPI serial interface with master/slave mode
- 41 General-purpose IO pins available
- I2C bus master capability
- USB 2.0 Full speed dual role OTG controller, which supports USB device and limited USB host with DMA access
- 2 channel programmable PWM (pulse width modulation) outputs.
- Boot code downloadable using SIF (serial interface) or EMIF by an external host.
- ROM boot loader which can load application / bootloader from a NAND flash
- External Host interface capability to allow external processors to interface with and control DM320
- VLYNQ interface to connect to any VLYNQ enabled device

* Refer DM320 datasheet for exact frequency

1.2 Performance

- Supports encoding / decoding of a 7 MP image in one second
- Supports Motion JPEG compression/decompression video at VGA resolution and more than 30fps
- Supports MPEG4 I+P frame video at VGA resolution and more than 30fps

1.3 Block Diagram

The detailed DM320 Block Diagram is given below.

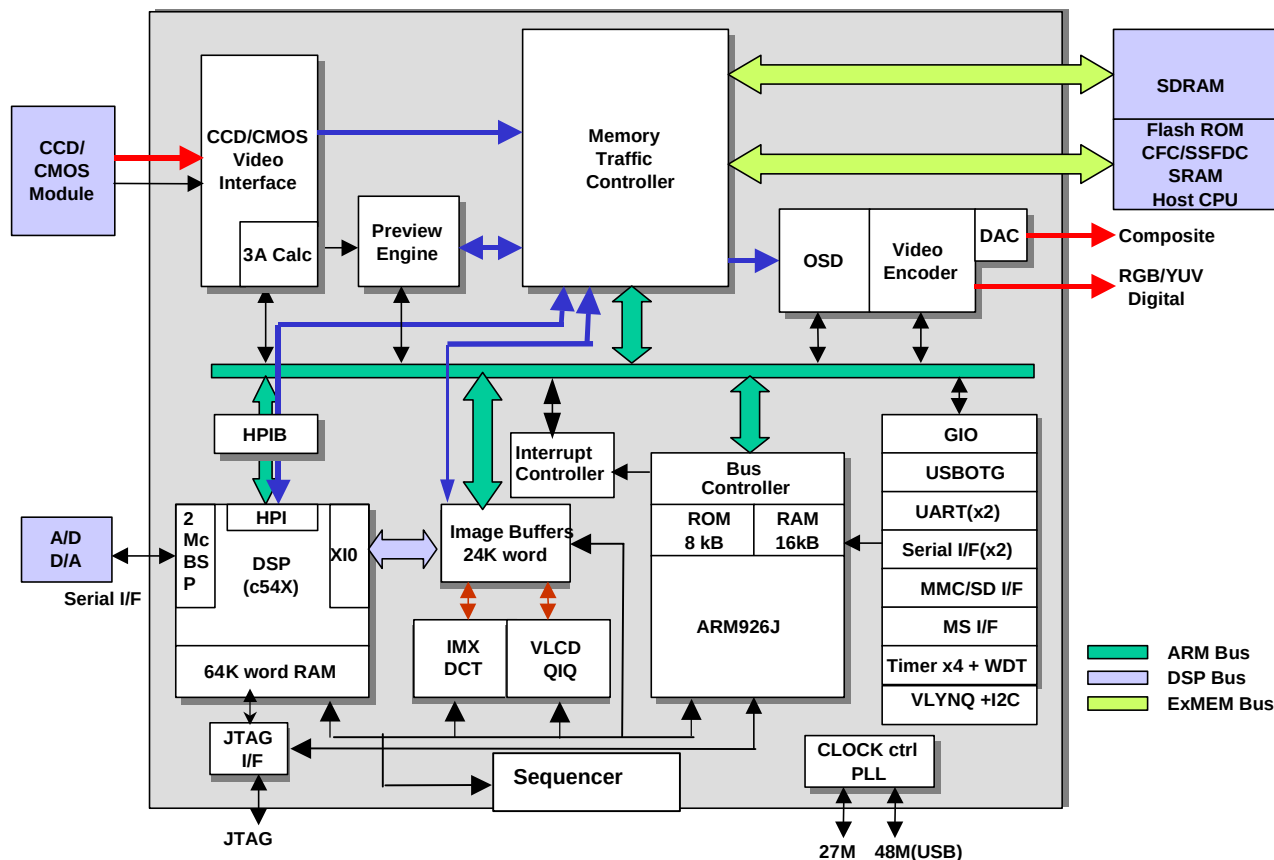


Figure 2: DM320 Block Diagram

DM320 can be divided into five sub-systems:

- The ARM subsystem and its peripherals
- The DSP subsystem
- The Imaging Co-processors and memories
- The imaging peripherals block
- The memory traffic controller block

The features of each of these blocks are given in the subsequent sections.

1.4 Features of ARM Subsystem

1.4.1 ARM926EJ Core

The ARM926EJ 32-bit RISC processor in the DM320 acts as the system controller. The ARM CPU performs general system control tasks such as system initialization, configuration, user interface and user command implementation. ARM926 can also be used for implementing audio and imaging algorithms.

The embedded ARM processor features are:

- 32-bit RISC processor
- Maximum operating frequency 180Mhz (Refer DM320 datasheet for exact value)
- 16-Kbyte built-in RAM
- 8 K-Byte built-in ROM
- 16KB I-cache, 8 KB D cache, 17 word write buffer and MMU
- 32/16-bit command sets
- Java accelerator
- Enhanced DSP instructions
- Little Endian Memory format
- 2 interrupt systems (FIQ, IRQ)
- JTAG-based emulation/debug capability

1.4.2 Memory traffic controller

The external memories and the SDRAM are accessed through the memory traffic controller. The external memory access is controlled by the external memory interface (EMIF) and the SDRAM access by the SDRAM controller (SDRC). Five different memory spaces exist for the EMIF of which two are general-purpose regions.

1.4.3 USB OTG Controller

The USB controller built into the DM320 conforms to USB Specification Revision 2.0 and USB OTG specification. It can operate as a full speed device or full / low speed host. The USB has a built-in transceiver, 4 KB RAM for FIFOs, programmable packet size, supports remote wakeup and DMA transfer to/from SDRAM / AIM / EMIF.

1.4.4 UART port

The DM320 supports two-channel start-stop synchronization serial ports (UART). The UARTs have separate 32-byte FIFOs for both the transmit and receive functions and interrupts can be generated on errors or various states of FIFO. The UARTs support 8 or 7 data bits; even, odd or no parity bit and 1 or 2 stop bits. Transmission and reception uses the ARM clock, and any bit rate can be set in integral multiples of 1/16 of the ARM clock.

1.4.5 Serial Ports

DM320 has two clock-synchronous serial interfaces. These are three-line serial interfaces consisting of clock, output data, and input data signals. Serial interface-0 supports SDRAM DMA transfers. Transmission is in 8-bit units, and each port has 8-bit buffers on each of the transmitting and receiving sides. Either LSB first or MSB first can be selected for transmitted/received data. An interrupt to the ARM is generated when 8-bit transmission (reception) is complete. It supports both master mode and slave mode of operation.

1.4.6 General Purpose Input Output Pins

DM320 has 41 general-purpose input/output pins (GPIO). Using the GPIO ports, the logic levels of the ports of an externally assigned device can be read and controlled. Many of the GPIO pins are multiplexed with other peripherals.

1.4.7 I2C Interface

DM320 includes a 2-line serial interface that can perform I2C bus protocol-compliant synchronous serial communication as bus master. This module has following features.

- Supports only single bus master
- Supports transmit, receive and mixed transfer using the slave address of 7bits or 10bits.
- Supports transfer rate of 100kbps or 400kbps.

1.4.8 MMC/SD Card Interface

MMCSDB interface supports native MMC protocol. The native protocol can use one bi-directional data line (for MMC) or four parallel data lines (for SD cards).

The MMC Controller includes the following features:

- Support for a Multimedia Card (an MMC) or a Secure Digital Memory Card (an SD card)
- Supports native MMC protocol
- A programmable frequency for the clock that controls the timing of transfers between the MMC Controller and the memory card.
- Support for DMA transfer between MMC Controller and SDRAM

1.4.9 Memory Stick Interface

Memory stick controller in DM320 supports serial mode of operation. There is a DMA channel between MS controller and SDRAM.

IMPORTANT: Customer needs to have a separate license from Sony for Memory Stick. DM320 Memory Stick Documentation will be provided only to customers who have Memory Stick license

1.4.10 External CPU interface

An external processor can gain access to DM320 internal peripherals and memories. In this mode, the EMIF bus is used to connect the external HOST with the DM320. DM320 ARM and DSP can still operate as usual in this mode. Dynamic WAIT control can be performed by the interface with the external host controller.

1.4.11 Timers

DM320 supports 4 general-purpose timers and 1 watchdog timer. They are configurable with settings for prescaler and count. 2 of the timers can also be used for precise control of the CCD shutter and exposure. Watch dog timer can output an interrupt, reset the system or send reset on a GIO pin.

1.4.12 Interrupt controller

The interrupt controller can handle up to 49 different interrupts from different peripherals and 16 external interrupts. All the interrupts can be individually enabled / disabled and the output can be selected as either an IRQ or FIQ interrupt.

1.4.13 ARM Boot-up Modes

DM320 provides four different ARM boot-up modes. These are

- CS0 boot-up mode
- Internal ROM boot-up mode
- Serial boot-up mode
- External host boot-up using EMIF pins

External host boot-up mode makes it possible to download an ARM boot code externally.

1.5 Features of the DSP subsystem

1.5.1 16-bit Fixed Point Digital Signal Processor (c5409)

The DSP subsystem has TMS320C5409 DSP core from Texas Instruments catalog DSP. The DSP subsystem includes TMS320C5409 DSP core from Texas Instruments, 32Kwords of Dual-Port SRAM for both program and data, and 16words of SARAM for program, 16Kwords of SARAM for data

The TI C54x core is a high-performance, low-power DSP core. The DSP software development tools for the C54x are very mature and fully compatible with the DM320 DSP subsystem. The embedded DSP processor features are:

- 16-bit fixed-point digital signal processor (based on the 'C5409)
- Maximum operating frequency 100 MHz (refer to DM320 Data sheet for actual value)
- 32K x 16-bit internal DARAM for program and data
- 16K x 16-bit internal SARAM for program
- 16K x 16-bit internal SARAM for data
- 16K x 16-bit internal ROM
- 2 Multi-Channel Buffered Serial Ports (McBSP)
- Host Port Interface that allows ARM access to DSP internal RAM
- JTAG-based emulation/debug

1.6 Features of Co-processor sub-system

DM320 co-processor sub-system has an imaging extension coprocessor (iMX), a discrete cosine transform (DCT) co-processor, variable length coding/decoding coprocessor (VLCD), sequencer, Image buffers and Co-processor DMA. The iMX accelerates many common image processing algorithms with regular processing – FIR filters, array scaling, array multiplication, etc. The iMX includes separate dedicated memories for program and coefficient data. DCT module can be used to do DCT and IDCT. The VLCD coprocessor, with quantization and inverse quantization module (QIQ), supports JPEG, MPEG1, H.263 and MPEG4 standards. The sequencer can be used to control DCT, iMX and VLCD co-processors. In addition, the coprocessors share three 4Kword Image Buffers as operand storage, which supports flexible 1-D/2-D SDRAM DMA transfers.

1.6.1 Hardware Accelerators (iMX/DCT/VLCD/Sequencer)

The Hardware Accelerators are designed to enhance the DM320's capability in image processing and other computation-intensive applications. ARM, DSP or sequencer can control the execution of co-processors and image buffers.

The iMX is a parallel MAC engine with flexible control and memory interface for extending the image processing performance of TI's programmable DSPs. The iMX works well in a shared memory configuration of DM320, so that regular block-based image processing tasks can be accelerated by iMX,

The hardware DCT module in DM320 performs 2-D 8x8 DCT and IDCT.

The sequencer is a micro-controller for synchronizing execution of iMX, VLCD, DCT and the co-processor DMA. It also can perform simple arithmetic functions to perform buffer management and loop control.

The co-processors can run at a maximum speed of 180Mhz. The sequencer runs at DSP frequency. (Refer DM320 datasheet for exact values)

1.6.1.1 Imaging Extensions (iMX)

The iMX is a parallel MAC engine for expanding image-processing performance. It has a flexible control, a memory interface, and eight MACs. The iMX is effective for fast computation of image processing in regular block units.

For digital cameras, the iMX can be used to accelerate:

- 2D FIR filter
- CFA interpolation filter
- Color space conversion
- Chroma down-sampling
- Edge enhancement
- False color suppression
- DCT (discrete cosine transform) and IDCT
- Table look-up

- Motion estimation

This list is not exhaustive and iMX can be programmed to perform many more functions.

1.6.1.2 DCT module

In image and video compression standards, DCT is the method to transform blocks of image in the spatial domain to the 2-D frequency domain. IDCT is numerically the inverse of DCT, and is used in the decompression flow to recover spatial data from the frequency domain data.

The standards use fixed size 8x8 2-D DCT and IDCT functions. Consequently, the hardware module in DM320 performs 8x8 2-D DCT and IDCT specified in JPEG, MPEG1,2,4 and H.263 standards. DCT requires 2 cycles per DCT coefficient for forward or reverse 2-D transforms.

1.6.1.3 Sequencer

Sequencer is a 16-bit microprocessor targeted for simple control, address calculation and loop control functions of the coprocessor subsystem. It is used in DM320 to coordinate among iMX, VLCD, DCT coprocessors and COP DMA along with C54x DSP and ARM9.

1.6.1.4 Variable Length Coder / Decoder

Variable Length Coder / Decoder consists of Quantizer / Inverse Quantizer Engine and Variable Length Coder / Decoder Engine

The Quantizer / Inverse Quantizer (Q/IQ) module is a high-performance, intelligent and highly programmable hardware accelerator for the quantization and inverse quantization of DCT coefficients. The Q/IQ is highly autonomous and is capable of carrying out almost all the steps of quantization and inverse quantization process with little or no help from the DSP. Q/IQ supports the quantization and inverse quantization of the standards listed below.

The VLCD coprocessor handles Huffman encode and decode operations for image compression algorithms. It operates with Huffman tables preloaded by DSP, via shared memory blocks. The VLCD is highly autonomous and is capable of carrying out all the steps of Huffman coding and decoding process with little or no help from the DSP. VLCD supports the Huffman encode/decode for the standards listed below.

- JPEG
- H.263
- MPEG 1
- MPEG 4
- MPEG2

1.7 Features of the Imaging Peripheral Block

1.7.1 CCD Controller

The CCD Controller captures and processes the physical data from the CCD sensor. Note, that the A/D converter and Timing Generator for the CCD sensor are external to the DM320. The CCD controller can provide synchronizing HD/VD signals to an external timing generator or can synchronize to externally generated HD/VD inputs from an external timing generator. The CCD Controller supports pixels clocks up to 100 MHz, imager sizes up to 4096x4096, and data widths from 8 to 14 bits. CCD controller also has interface to different sensors like Bayer pattern and Foveon sensors. The CCD Controller can also implement the following common image processing tasks:

- Optical black clamping
- DC offset
- Median filtering
- White balance gain/offset
- A-law compression (10-bit to 8-bit)
- Lowpass filtering
- Downsampling/culling

The CCD Controller can also be used to receive 8/16-bit YcbCr 4:2:2 as input. It can also receive REC656 compatible 8/10-bit YcbCr 4:2:2 as input.

1.7.2 Preview Engine

The DM320 Preview Engine is a hardware video processor module which generates an image data with YCbCr-4:2:2 format from CCD raw data. It is designed to provide a high quality video image for both the LCD preview function in a digital camera and for the source images for the video recording functions. In addition, the Preview Engine can process data from SDRAM as well as the CCD Controller input and can resize YUV 4:2:2 data stored in SDRAM. The Preview Engine supports the following image processing functions:

- Noise filter
- Digital gain
- White balance
- 2 Step Smoothing
- Seamless horizontal and vertical down sampling (x1/64~x1)
- Seamless horizontal and vertical zoom (x1~x4)*
- RGB2RGB blend matrix
- Programmable Gamma correction
- Chroma offset & suppression
- 1 shot preview

- Resize only mode
- Histogram

1.7.3 On-Screen Display (OSD)

The On-Screen Display module of the DM320 manages the display and blending of multiple windows containing video and bitmap data. This module features the following:

- Two Video windows (YUV 4:2:2)
- Two bitmap windows, or 1 bitmap window and 1 attribute window
- Blending of bitmap with video windows – 8 blending ratios
- One rectangular cursor window
- Background color of each window can be specified – background can be set to transparent
- Video-Window0 and Video-Window1 can display YCbCr data and OSD-Window0, OSD-Window1 can display bitmap and 16-bit RGB
- Bitmap windows can be 1,2,4, or 8 bit color depth
- Bitmap Color Look Up Table (CLUT) in ROM or user defined table in RAM
- Interlaced and non-interlaced modes are supported
- Built in VGA to NTSC/PAL conversion circuit (supports non square pixel)

1.7.4 Hardware 3A Engine

DM320 provides real-time 3A (AE/AWB/AF) statistics function. ARM uses the statistics to compute the optimal 3A control parameters.

AE/AWB statistics is calculated by using up to 192 windows. The windows are the same size and organized as an m-by-n matrix. The organization of the matrix, the size of the windows and the method of calculation are user-selectable.

AF statistics is calculated by using up to 96 windows. The position and the size of each window can be independently set by software.

1.7.5 Video Encoder Interface

The Video Encoder is used to display images and video on TV monitor and various LCD displays. The Video Encoder converts digital YCbCr signals to analog and digital video signals. The Video Encoder can generate composite NTSC or PAL signals, and 8/16 bit digital LCD interface signals. To generate these signals, the Video Encoder has digital to analog converter, a digital LCD interface, and a set of signals to perform timing and synchronization.

1.8 Features of the Memory Traffic Controller Block

1.8.1 SDRAM Controller

The SDRAM controller block acts as the primary interface between SDRAM and all functional blocks such as the processors (ARM, DSP), CCD Controller, OSD and DMA controller. This block supports SDRAM timing of a maximum of 100 MHz, and provides continuous data access with low overhead.

- Supports 16/64/128/256M-bit SDRAM
- Supports both 32-bit SDRAM or 16-bit SDRAM
- Maximum 108 MHz operation (refer to the data sheet for actual value)
- Access in word, half word, or byte units (ARM only)
- Mode setting, power down and SELF refresh commands
- Programmable refresh interval
- Column Access Strobe (CAS) latency 2 or 3 can be selected
- SDRAM size of up to 256 Mbytes can be used
- DMA transfer with peripheral modules
- SDRAM access priority is user configurable
- Pin sharing with CS0 or CS1-CS4 regions
- Supports Mobile SDRAM

1.8.2 External Memory Interface

The External Memory Interface (EMIF) provides a flexible interface circuit to external Flash memory (CS0), CompactFlash (CS1) and SmartMedia (CS2) interfaces and 2 other user configurable memory mapped areas (CS3, CS4). Each area has a separate chip select output terminal and DMA support

- Data width of 8 or 16 bits can be selected for each CS regions (CS0-4), except CS2.
- Using BUS_REQ and BUS_ACK signals, handshakes of open request and acknowledgment of bus rights with the external controller is possible.
- External wait can be applied by using the EM_WAIT terminal when accessing CS3 and CS4.
- IDLE can be inserted in write after read operation, and in access to other CS regions.
- All buses are in hi-impedance during reset and when set in bus open register from ARM.
- In CS0 boot-mode, after reset ARM jumps to offset address 0x0 of CS0.
- 8-bit/16bit access CF/CF+ cards

- Dynamic bus sizing (CF/CF+ IO mode) by the IOIS16 terminal
- Support for DMA transfer between External Memory Area and SDRAM
- External Host interface capability through external memory interface

1.9 DM320 System Data Flows

1.9.1 Preview Mode

In the preview mode, CCD or CMOS imager data comes in through the CCD controller to the preview engine. CCD controller has fault pixel correction, optical black clamping, and required timing control and interface. Preview engine contains a simplified image processing pipeline for AE/AWB, CFA interpolation, down sampling, and gamma correction. The CCD/CMOS sensor should be configured to draft mode output, and preview engine handles further down-sampling, if necessary, to produce LCD/video resolution. Preview output is written to SDRAM.

DM320 has hardwired 3A (AF/AE/AWB) calculator. Each result is stored in SDRAM, ARM can read its data and control AFE.

The display data is read out by the OSD module, and merged/blended with the OSD data. The display data is sent to video encoder (VENC) or LCD controller, which sends composite analog or digital LCD data to the external display device.

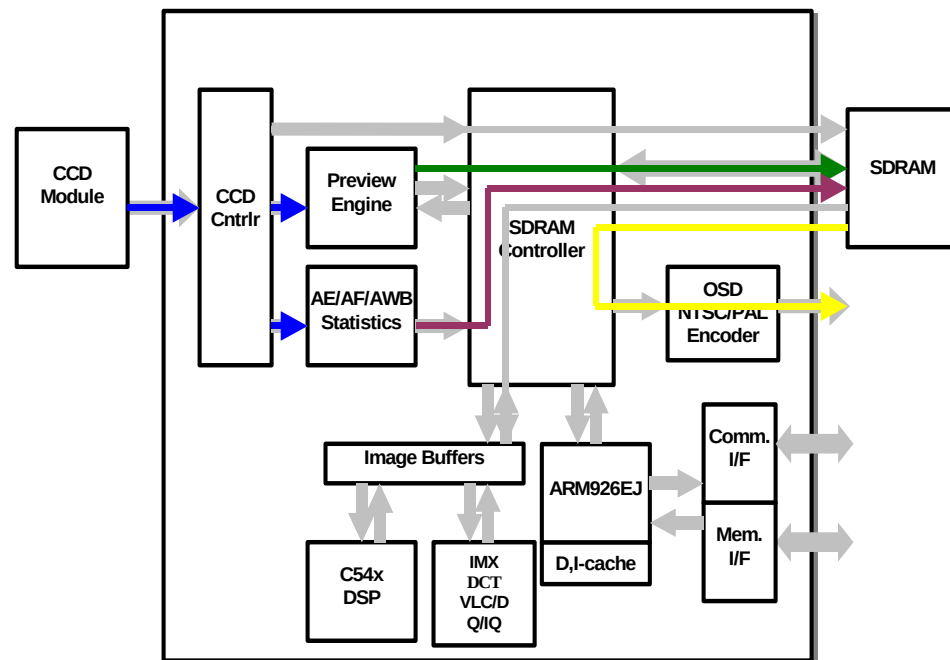


Figure 3: Preview Mode Data Flow

1.9.2 Still Picture Capture Mode

In the still picture capture mode, CCD or CMOS imager data comes in through the CCD controller and SDRAM controller directly to SDRAM first.

When there is a sufficient amount of data to start processing, which could be 20-24 lines with non-interlaced data and slightly over half-frame for interlaced data, ARM is notified via an interrupt signal from CCD controller. ARM then coordinates with iMX, VLCD, and DMA controller to transfer in the data from SDRAM into image buffer and process the data. The processing flow is software programmable and customer-specific. Typically compressed JPEG bitstream is the final output, and it's written back into SDRAM.

ARM then handles media storage card file system setup, sector header setup, and triggers DMA to write data from SDRAM into the card.

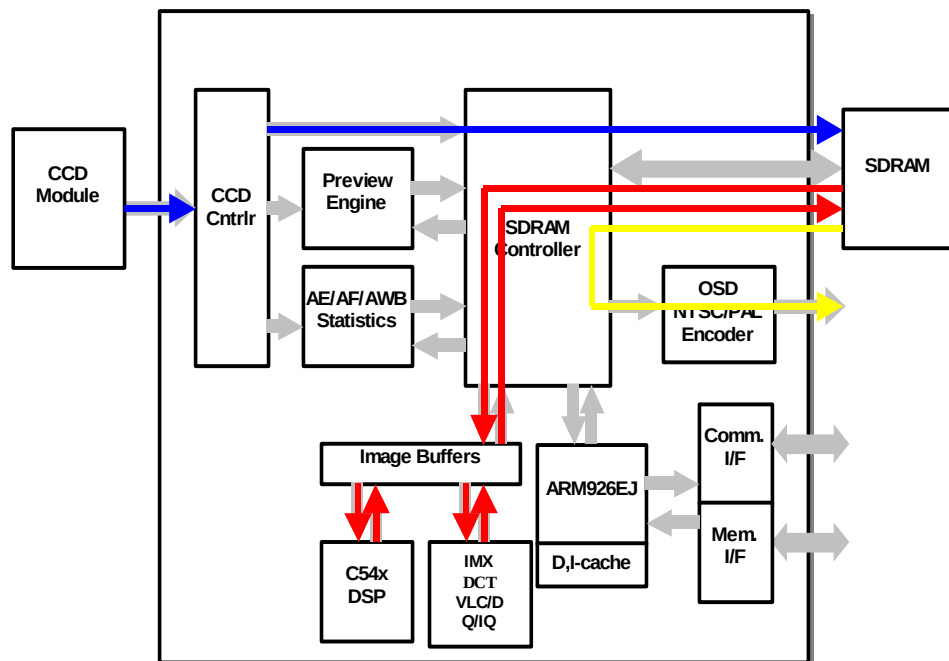


Figure 4: Still Picture Capture mode Data Flow

1.9.3 Video Capture Mode

Video capture is similar to still picture capture, except for the input stage. For video capture, we use Preview Engine to down sample the CCD/CMOS imager data to the video capture size (VGA, CIF, QCIF etc) before writing into SDRAM. This offloads much processing in the programmable engines and reduces power consumption due to processing and traffic.

Once sufficient amount of the video data is written into SDRAM, again ARM gets an interrupt from CCD controller and triggers the processing steps. MPEG-1 or MPEG-4 compression is performed with the coordination of ARM, iMX, VLCD and the DMA controller. The compressed MPEG bitstream is written back to SDRAM.

The video data produced by Preview Engine are also pulled out by OSD module to produce display data for composite analog video or for digital LCD interface.

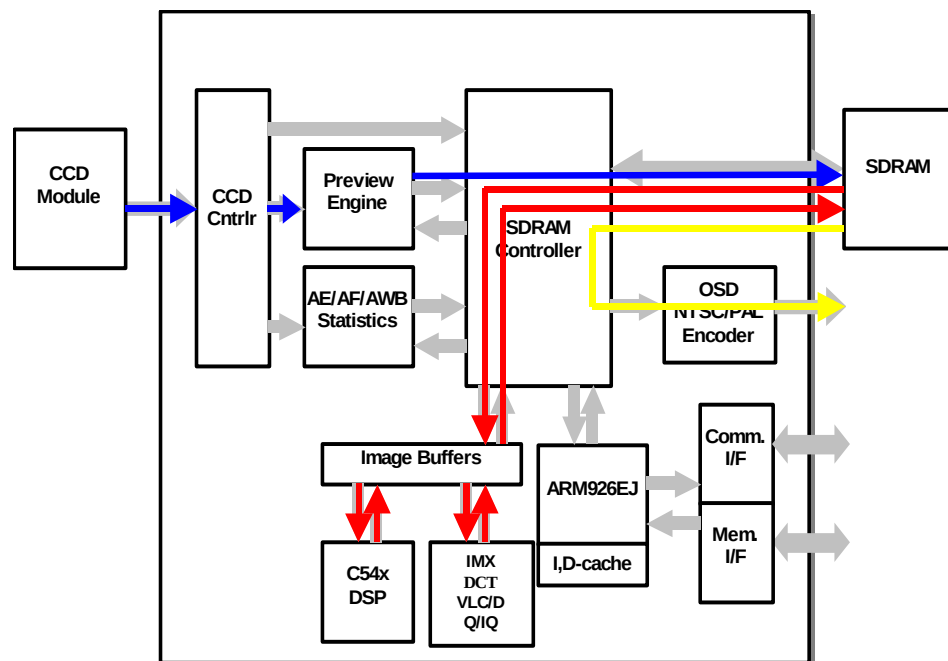


Figure 5: Video Capture Mode Data Flow

1.9.4 Still Picture and Video Playback

Still pictures or video clips can be played back on the digital camera. Typically the compressed data is first read out of the media card into SDRAM. Next the compressed data is processed and transported by ARM, coprocessors and DMA to uncompressed and, if necessary, resized into display resolution, and stored back into SDRAM.

The display data is then pulled out by OSD to supply VENC or LCD controller for analog composite video or digital LCD interface.

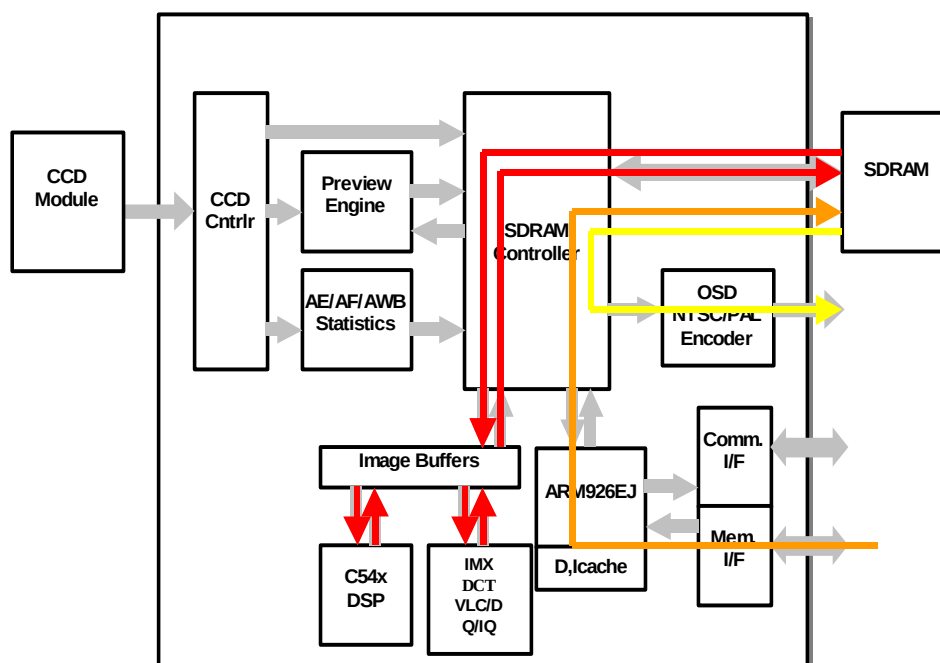


Figure 6: Play Back Mode Data Flow

2 ARM Subsystem

2.1 Introduction

The ARM subsystem consists of the ARM926EJ-S RISC processor core, Instruction cache, Data cache, MMU, internal RAM, internal ROM and peripherals. The ARM controls the various peripheral modules including the timers, Interrupt controller, CCD controller, on-screen display, USB, and serial ports. The user interface would run on this processor. ARM can also directly control the co-processors and would be used for implementing audio and video algorithms. The details of ARM subsystem are shown in Figure 7.

In the system, ARM is responsible for handling many system functions such as system-level initialization, configuration, user interface, user command execution, connectivity functions, interface to DSP and imaging co-processors, and overall system control. ARM handles these functions because it has a larger program memory space and better context switching capability, and is thus more suitable for complex, multi-tasking, and general-purpose control tasks compared to the DSP.

- Interconnect Bus Configuration - The Multi Bus Masters can access different slaves simultaneously.
- A Hierarchical Sub-system configuration - An Advanced Internal Memory (AIM). This Memory is used for ARM926's TCM and Device Internal memory.
- Support Boot-Up-Mode by change ARM926's Start Address
- Endian Conversion - For Read/Write Data at Master Access

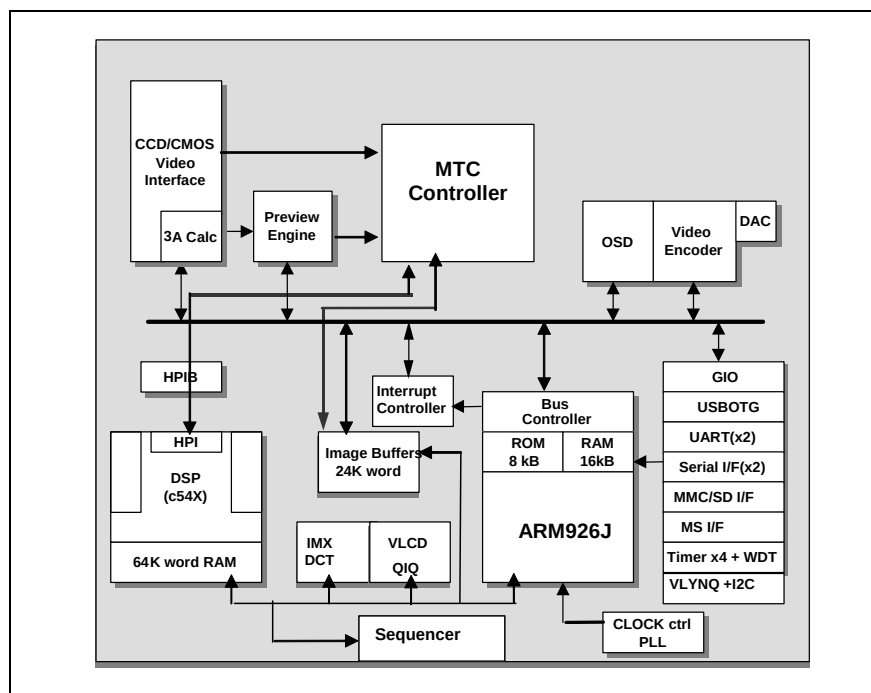


Figure 7: ARM Subsystem of DM320

2.2 Memory map

There are the following bus slaves in the 4G-Byte area (32-bit address).

- AIM
- Peripheral module registers
- DSP Memory
- Coprocessor Subsystem
- External Memory
- External SDRAM

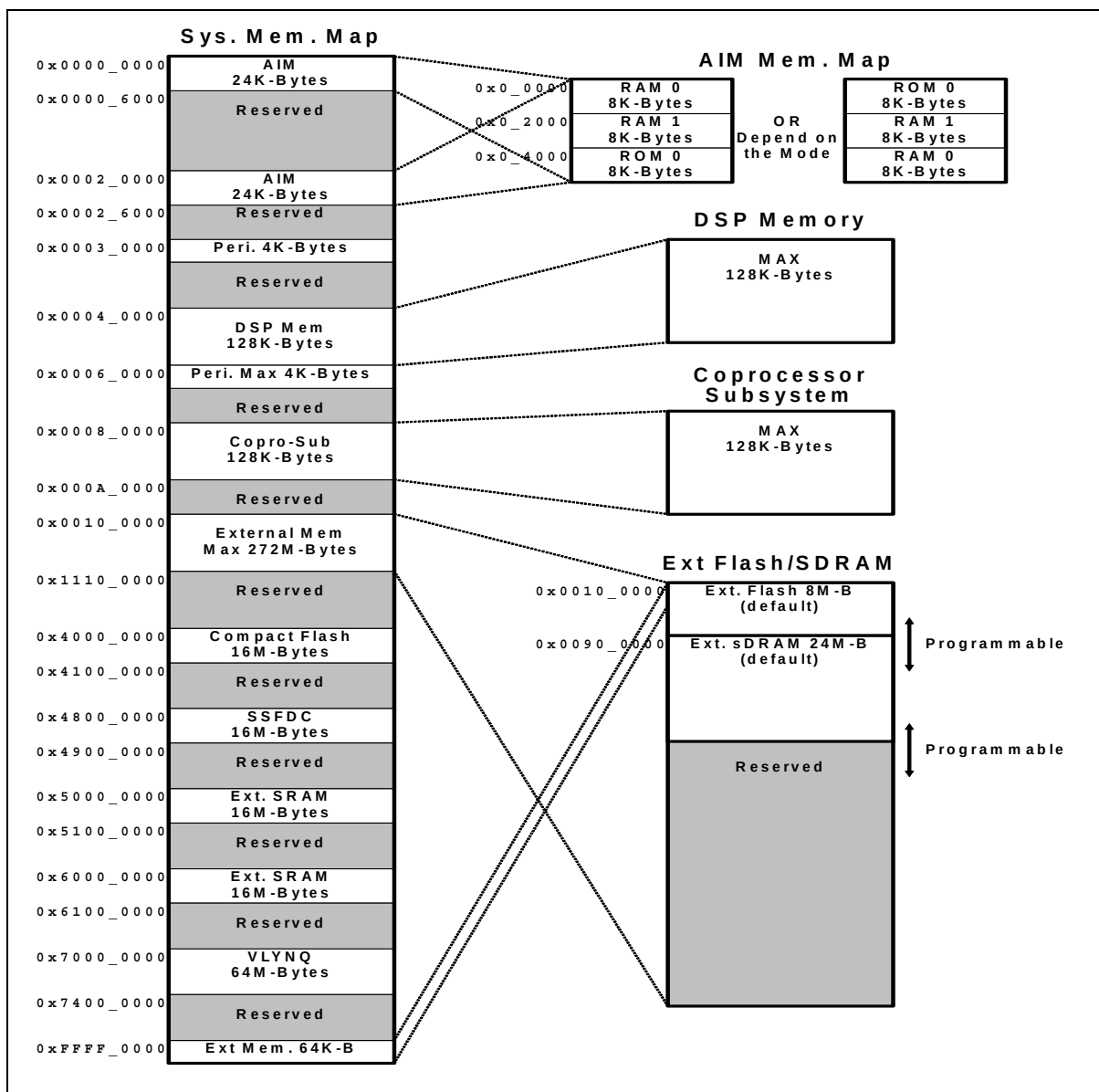


Figure 8: ARM Subsystem Memory Map

Table 1 shows each area's start address and maximum size. About an actual size, see each section's description.

Area	Start address (Hex)	Maximum Size (Hex)	Maximum Size (Decimal)
Instruction TCM	0x0000:0000	0x0000:6000	24 K-bytes
Data TCM	0x0002:0000	0x0002:6000	24 K-bytes
Peripherals	0x0003:0000	0x0001:0000	64 K-bytes
DSP memory	0x0004:0000	0x0002:0000	128 K-bytes
Peripherals	0x0006:0000	0x0002:0000	128 K-bytes
Co-processor Sub System	0x0008:0000	0x0002:0000	128 K-bytes
External Flash	0x0010:0000	0x0100:0000 (Programmable)	16 M-bytes (Default 8 MB)
External SDRAM	0x0090:0000 (Programmable)	0x1000:0000 (Programmable)	256 M-bytes (Default 24 MB)
External Compact flash	0x4000:0000	0x0100:0000	16 M-bytes
External SSFDC	0x4800:0000	0x0100:0000	16 M-bytes
External memory 0 (CS3)	0x5000:0000	0x0100:0000	16 M-bytes
External memory 1 (CS4)	0x6000:0000	0x0100:0000	16 M-bytes
VLYNQ Remote	0x7000:0000	0x0400:0000	64 M-bytes
USB	0x8000:0000	0x0400:0000	64 M-bytes

Table 1: ARM memory map

2.3 Bus Masters and Slaves

The following is ARM Subsystem's Master and Slave access summary.

		ARM Subsystem Slaves					
		AIM RAM	Peripheral	DSP Memory	Copro Subsystem	EMIF	sDRAM
Masters	ARM I-AHB	Available	Available	Available	Available	Available	Available
	ARM D-AHB	Available	Available	Available	Available	Available	Available
	EHIF	Available	Available	Available	Available	Available	Available
	MTC DMA	Available	N/A	N/A	Available	Available	Available
	VLYNQ	Available	N/A	N/A	Available	Available	Available
	USB DMA	Available	N/A	N/A	Available	Available	Available

Table 2: ARM Sub-system Master / Slave

2.4 Interconnect bus access

The Interconnect bus system supports different masters accessing different slaves simultaneously. When one bus-Master accesses a slave, other masters can access different slaves at the same time. If multiple masters try to access the same slave, the bus controller will sequence the access by adding wait states.

As the Figure 2, Interconnect bus allows the multi master can simultaneously access multi slave. In this case, the following access will be allow at the same time.

- ARM fetches the data of Instruction (ARM926's I-AHB read EMIF).
- ARM read or write a data to/from a peripheral module
- MTC's DMA read/write a data to/from a AIM
- EHIF read/write a data to/from SDRAM

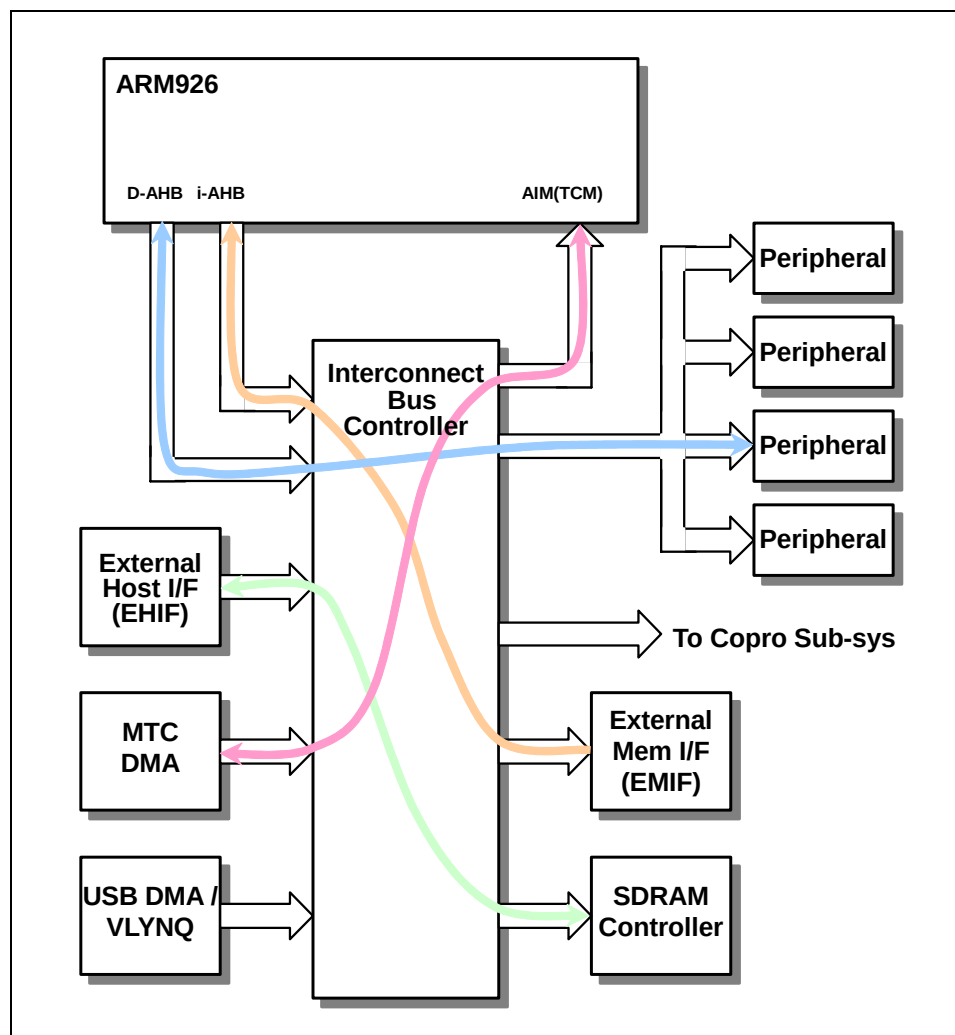


Figure 9: ARM Subsystem of DM320

2.5 Hierarchical Sub-system

There are two sub-systems in this device as following.

- ARM Sub-system (Primary Bus)
- Co-processor Sub-system (Image / video system)

ARM bus is the primary bus. Co-processor Sub-System is one of ARM Sub-system's slaves. ARM bus's Master can access Co-pro-bus's slave. However Co-processor bus masters can only access Co-processor bus slaves.

The different Bus masters and Bus slaves are given in Table 2.

Access Possibility	ARM Sub-sys. slave	Co-processor Sub-Sys. slave
ARM bus master	Yes	Yes
Co-processor bus master	No	Yes

Table 3: DM320 Subsystem access

2.6 Boot up modes

There are three different modes for boot-up. The signal states and reset vector addresses are as given in Table 4.

BTSEL [1:0]	Boot-up mode	ARM core's VINT signal	ARM core Start Address	Memory
00, 11	External Memory (CS0)	'1'	0xFFFF:0000	External Flash (Mirror)
01	External Host via AIM-SIF or EMIF	'0'	0x0000:0000	AIM-RAM0 of AIM
10	AIM ROM	'0'	0x0000:0000	AIM-ROM0 of AIM

Table 4: ARM boot-up modes and signal states

The area of 0xFFFF:0000 to 0xFFFF:FFFF is a mirror external memory (0x0010:0000 – 0x0010:FFFF) at CS0 region. This mirror function helps to boot-up in external memory mode, although the code resides in CS0 region.

Note: In external boot up mode, ARM core fetches the code from 0xFFFF:0000. Therefore, a vector at least should be generated as located at 0x0010:0000, executed at 0xFFFF:0000.

When ARM core start from 0x0000:0000, this area is mapped to AIM area. And AIM has a function to change the location of AIM-RAM0 and AIM-ROM0.

In addition to Internal ROM, external flash can also function as a boot-loader. Load the boot-loader code in a linear flash (size of which will depend upon the boot-loader). This boot-loader can then load the final application image from the NAND flash, similar to the internal ROM boot-loader. Refer Application note

Note: ARM core's VINTH signal along with the CP15 register setting decides the interrupt vector.

2.7 Data access modes

The memory, which is addressed by BUSC module, can be accessed as byte, half-word or word. During a half-word or a word access to the memory, the

address from ARM should be aligned to half-word or word respectively. Hence, when ARM accesses to memory in a half-word or a word data mode, BUSC won't use A [0] and A [1:0] respectively during address decoding. The 16-bit peripheral requires a half-word aligned address and the 32-bit peripheral requires a word-aligned address.

2.7.1 Advanced Internal Memory (AIM)

An AIM's key features are followings. Each feature is described below.

- AIM is one slave of ARM sub-system and is also TCM of ARM core
- AIM is divided into three for the efficiency of access of ARM TCM.
- Remap function to support boot-up and change ARM vector.

ARM in DM320 has 16 KB of internal RAM and 8 KB of internal ROM. This memory can be accessed in a single cycle (zero wait-state access). AIM is used as the TCM memory for ARM. The exception vector table for ARM should be placed in this memory depending upon the VINTH signal set in CP15 register. The entry table of interrupt controller also should be kept here for best performance.

When ARM boots up using internal ROM, the memory location starting from '0x0' should correspond to ROM and when the code is executing, the location starting '0x0' should be RAM, since it is the exception vector table base. The memory re-map is controlled through external pins and AHB bus controller register. Refer Bus controller chapter for more details about memory re-map.

TCM re-map functionality is required since the Interrupt vector table (IVT) is located at 0x0000:0000 for all the internal boot-modes. When booting from Internal ROM, ROM is mapped to 0x0000:0000. After the boot-loader or application is downloaded into IRAM / SDRAM, the TCM memory has to be re-mapped, so that RAM is present at 0x0000:0000 location. This facilitates the IVT to be loaded at address 0x0000:0000. The TCM re-map can be done using the AIMREMAP bit in BUSCTRL register of AHB bus controller. Details about TCM re-map feature are discussed in the bus controller chapter.

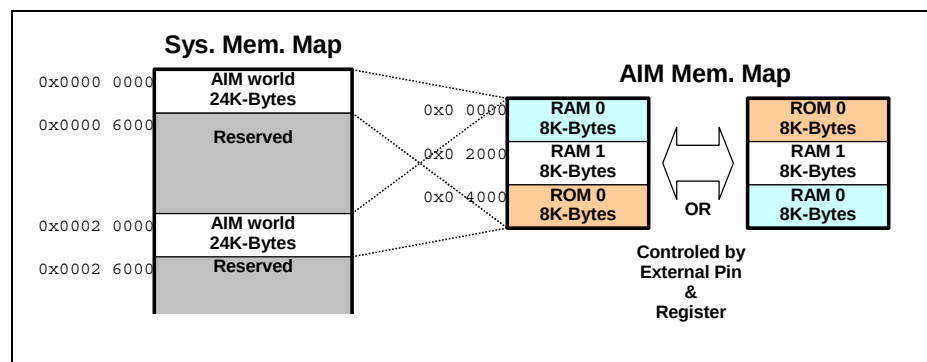


Figure 10: AIM Memory map

2.7.2 DSP memory

When the ARM accesses the DSP internal memory through the HPI bridge module (HPIB), one wait-state is inserted automatically. In addition to this, extra wait cycles will be inserted depending on the wait signal that is generated in HPIB module.

The DSP internal memory (32-K words of DARAM and 32-K words of SARAM) can be read and written by the ARM or an external host through the HPI-Bridge. Addresses 0x0-0x7FFFh of DSP DARAM, 0x8000-0xBFFFh of DSP SARAM (data) and 0xC000-0xFFFFh of DSP SARAM (PAGE1: program) correspond to addresses 0004:0000-0005:FFFF in the ARM memory map. Addresses 0x0-0x7FFFh of DSP DARAM, 0x8000-0xBFFFh of DSP SARAM (data) and 0xC000-0xFFFFh of DSP SARAM (PAGE1: program) correspond to addresses 0002:0000-0003:FFFF in the external host memory map.

The memory maps of the ARM and external HOST are shown in Table 5 and Table 6 respectively. When the endianness of the ARM and DSP do not match, EXCHG in HPIBCTL (0x00) can be used to swap the upper and lower 8-bits of data transferred between the ARM and DSP.

ARM address (byte)	DSP address (word)	DSP memory	Size (byte)
0x40000-0x4FFFF	0x0-0x7FFF	Data & Program (Page0)	64k
0x50000-0x57FFF	0x8000-0xBFFF	Data	32k
0x58000-0x5FFFF	0xC000-0xFFFF	Program (Page1)	32k

Table 5: ARM HPIB Memory Map

External Host address (byte)	DSP address (word)	DSP memory	Size (byte)
0x20000-0x2FFFF	0x0000-0x7FFF	Data & Program(Page0)	64k
0x30000-0x37FFF	0x8000-0xBFFF	Data	32k
0x38000-0x3FFFF	0xC000-0xFFFF	Program (Page1)	32k

Table 6: External HOST Memory Map

2.7.3 Coprocessor Subsystem

The co-processor system consists of image buffers, COP DMA, Sequencer, iMX, VLCD, DCT and the memories associated with sequencer, iMX and VLCD. To access the co-processor area, clock to coprocessor subsystem must be enabled.

2.7.4 External Memory

DM320 has a FLASH, SDRAM, memory cards and general-purpose memory interface. ARM can access SDRAM via the SDRAM controller module. It can access memory card or flash ROM via the external memory interface module. In access from ARM to external memory, a wait state is automatically inserted, and

its cycle varies depending up on SDRAM frequency and external memory access time. The SDRAM controller controls the SDRAM memory access. The dynamically allocated external memory, which is directly accessible by the ARM, can be divided into a maximum of 6 regions (SDRAM, and CS0 to CS4). SDRAM start and end addresses are configurable, thus allowing the size of Flash and SDRAM region to be changed.

3 ARM Core

3.1 Introduction

- 32-bit RISC processor
- Maximum operating frequency of 160 MHz (Refer DM320 datasheet for exact value)
- 16-KB built-in RAM
- 8-KB built-in ROM
- 16-KB Instruction cache
- 8-KB Data cache
- 17-byte write buffer
- MMU
- 32/16-bit instruction sets
- Little endian memory format
- Java accelerator
- Enhanced DSP instructions
- CP15 to control MMU, Cache etc

ARM926EJ-S includes new signal processing extensions to enhance 16-bit fixed-point performance using a single-cycle 32 x 16 multiply-accumulate (MAC) unit. It also includes the technology, which enables the direct execution of Java byte-codes. The ARM subsystem has 16 KB internal RAM and 8 KB internal ROM.

The ARM926EJ-S processor is a member of ARM9 family of general-purpose microprocessors. This processor is targeted at multi-tasking applications where full memory management, high performance, low die size and low power are all important.

The ARM926EJ-S processor supports the 32-bit ARM and 16 bit THUMB instruction sets, enabling the user to trade off between high performance and high code density. This includes features for efficient execution of Java byte codes, providing Java performance similar to Just in Time (JIT) Java interpreter, but without associated code overhead.

The ARM926EJ-S processor supports the ARM debug architecture and includes logic to assist in both hardware and software debug. The

ARM926EJ-S processor has a Harvard architecture and provides a complete high performance subsystem, including:

- An ARM926EJ -S integer core
- A Memory Management Unit (MMU)
- Separate instruction and data AMBA AHB bus interfaces
- Separate instruction and data TCM interfaces

The ARM926EJ-S processor implements ARM architecture version 5TEJ.

3.1.1 Operating states / modes

ARM can operate in two states: ARM (32 bit) mode and Thumb (16 bit) mode. ARM926EJ can be switched between ARM mode and Thumb mode using the BX instruction.

ARM can operate in the following modes:

- User mode (usr): Non-privileged mode, usual for execution of most application programs
- Fast interrupt mode (fiq): Fast interrupt processing
- Interrupt mode (irq): Normal interrupt processing
- Supervisor mode (svc): Protected mode of execution for operating systems
- Abort mode (abt): Mode of execution after a data abort or a pre-fetch abort
- System mode (sys): Privileged mode of execution for operating systems
- Undefined mode (und): Undefined mode is entered if an undefined instruction is executed

Privileged modes (system or supervisor) can be entered only from other privileged modes. To enter supervisor mode from user mode, generate a software interrupt (SWI). An IRQ interrupt will make the processor enter the IRQ mode and a FIQ interrupt will make the processor enter the FIQ mode.

Different Stacks need to be setup for different modes. The stack pointer (SP) automatically changes to the SP of the mode entered.

3.1.2 Processor status registers

Processor status register (PSR) controls the enabling and disabling of interrupts and setting the mode of operation of the processor. The bottom 8 bits PSR[7:0] are the control bits of the processor. PSR[27:8] are reserved bits and PSR[31:28] are status registers. The details of the control bits are given below.

- Bit 7 – I bit: Disable IRQ (I=1) or enable IRQ (I=0)
- Bit 6 – F bit: Disable FIQ (F=1) or enable FIQ (F=0)
- Bit 5 – T bit: Controls whether the processor is in thumb mode (T=1) or ARM mode (T=0)

- Bits 4:0 Mode: Controls the mode of operation of the processor
 - o PSR[4:0] = 10000 : User mode
 - o PSR[4:0] = 10001 : FIQ mode
 - o PSR[4:0] = 10010 : IRQ mode
 - o PSR[4:0] = 10011 : Supervisor mode
 - o PSR[4:0] = 10111 : Abort mode
 - o PSR[4:0] = 11011 : Undefined mode
 - o PSR[4:0] = 11111 : System mode

Status bits show the result of the most recent ALU operation. The details of status bits are given below.

- Bit 31 – N bit: Negative or less than
- Bit 30 – Z bit: Zero
- Bit 29 – C bit: Carry or borrow
- Bit 28 – V bit: Overflow or underflow

Note: Refer Chapter 2 – *Programmer's model* of the ARM926EJ-S TRM, downloadable from <http://www.arm.com/arm/TRMs> for more details.

3.1.3 Exception and Exception vectors

Exceptions arise when the normal flow of program has to be halted temporarily. The exceptions that occur in an ARM system are given below.

- Reset exception: processor reset
- FIQ interrupt: Fast interrupt
- IRQ interrupt: Normal interrupt
- Abort exception: Abort indicates that the current memory access could not be completed. The abort could be a pre-fetch abort or a data abort.
- SWI interrupt: Software interrupt is used to enter the supervisor mode.
- Undefined exception: An undefined exception occurs when the processor executes an undefined instruction.

The exceptions in the order of highest priority to lowest priority are: Reset, Data abort, FIQ, IRQ, Pre-fetch abort, undefined instruction and SWI. SWI and undefined instruction has the same priority. Depending upon the status of VINTH signal or the register setting in CP15, the vector table could be located at address 0x00000000 (VINTH = 0) or 0xFFFF0000 (VINTH = 1). The default vector table is given below in **Table 7**.

Vector offset address	Exception	Mode on entry	I bit state on entry	F bit state on entry
0x00000000	Reset	Supervisor	Set	Set
0x00000004	Undefined instruction	Undefined	Set	Unchanged
0x00000008	Software interrupt	Supervisor	Set	Unchanged
0x0000000C	Pre-fetch abort	Abort	Set	Unchanged
0x00000010	Data abort	Abort	Set	Unchanged
0x00000014	Reserved	-	-	-
0x00000018	IRQ	IRQ	Set	Unchanged
0x0000001C	FIQ	FIQ	Set	Set

Table 7: Exception Vector table for ARM

3.1.4 The 16-BIS/32-BIS Concept

The key idea behind 16-BIS is that of a super-reduced instruction set. Essentially the ARM926EJ processor has two instruction sets:

- ARM mode or 32-BIS: the standard 32-bit instruction set
- Thumb mode or 16-BIS: a 16-bit instruction set.

The 16-BIS 16-bit instruction length allows it to approach twice the density of standard 32-BIS code while retaining most of the 32-BIS's performance advantage over a traditional 16-bit processor using 16-bit registers. This is possible because 16-BIS code operates on the same 32-bit register set as 32-BIS code. 16-bit code is able to provide up to 65% of the code size of the 32-bit code, and 160% of the performance of an equivalent 32-BIS processor connected to a 16-bit memory system.

3.1.4.1 16-BIS/32-BIS Advantages

16-bit instructions operate with the standard 32-bit register configuration, allowing excellent interoperability between 32-BIS and 16-BIS states. Each 16-bit instruction has a corresponding 32-bit instruction with the same effect on the processor model. The major advantage of a 32-bit architecture over a 16-bit architecture is its ability to manipulate 32-bit integers with single instructions, and to address a large address space efficiently. When processing 32-bit data, a 16-bit architecture will take at least two instructions to perform the same task as a single 32-bit instruction. However, not all the code in a program will process 32-bit data (for example, code that performs character string handling), and some instructions, like branches, do not process any data at all. If a 16-bit architecture only has 16-bit instructions, and a 32-bit architecture only has 32-bit instructions, then overall, the 16-bit architecture will have better code density, and better than one half the performance of the 32-bit architecture. Clearly 32-bit performance comes at the cost of code density. The 16-bit instruction breaks this constraint by implementing a 16-bit instruction length on a 32-bit architecture, making the processing of 32-bit data efficient with a compact instruction coding. This provides far better performance than a 16-bit architecture, with better code density than a 32-bit architecture. The 16-BIS also has a major advantage over other 32-bit architectures with 16-bit instructions. This is the ability to switch back to full 32-bit code and execute at full speed. Thus critical loops for applications such as fast interrupts and DSP algorithms can be coded using the full 32-BIS, and linked with 16-BIS code. The overhead of switching from 16-bit code to 32-bit code is folded into sub-routine entry time. Various portions of a system can be optimized for speed or for code density by switching between 16-BIS and 32-BIS execution as appropriate.

3.1.5 Co-Processor 15 (CP15)

The system control coprocessor (CP15) is used to configure and control instruction and data caches, Tightly- Coupled Memories (TCMs), Memory Management Unit (MMU) and many system functions. The CP15 registers can only be accessed with MRC and MCR instructions by ARM in a privileged mode like supervisor or system mode.

Addresses in an ARM926EJ-S system

Three different types of address exist in an ARM926EJ-S system.

Domain	ARM9EJ-S	Caches and MMU	TCM and AMBA bus
Address type	Virtual Address (VA)	Modified Virtual Address (MVA)	Physical Address (PA)

Table 8: Different address types in ARM system

Given below is an example of the address manipulation that occurs when the ARM9EJ-S core requests an instruction.

- The VA of the instruction is issued by the ARM9EJ-S core.
- The VA is translated to the MVA. The Instruction Cache (Icache) and Memory Management Unit (MMU) detects the MVA.
- If the protection check carried out by the MMU on the MVA does not abort and the MVA tag is in the Icache, the instruction data is returned to the ARM9EJ-S core.
- If the protection check carried out by the MMU on the MVA does not abort, and the MVA tag is not in the cache, then the MMU translates the MVA to produce the PA.

Note: Please refer Chapter 2 - *Programmers model* of the ARM926EJ-S TRM downloadable from <http://www.arm.com/arm/TRMs> for more details.

3.1.5.1 Memory Management Unit

The ARM926EJ-S MMU provides virtual memory features required by operating systems such as SymbianOS, WindowCE and Linux. A single set of two level page tables stored in main memory is used to control the address translation, permission checks and memory region attributes for both data and instruction accesses. The MMU uses a single unified Translation Lookaside Buffer (TLB) to cache the information held in the page tables.

The MMU features are:

- Standard ARM architecture v4 and v5 MMU mapping sizes, domains and access protection scheme.
- Mapping sizes are 1 MB (sections), 64 KB (large pages), 4 KB (small pages) and 1 KB (tiny pages)
- Access permissions for large pages and small pages can be specified separately for each quarter of the page (subpage permissions)
- Hardware page table walks
- Invalidate entire TLB, using CP15 register 8

- Invalidate TLB entry, selected by MVA, using CP15 register 8
- Lockdown of TLB entries, using CP15 register 10

Example code for creating translation table and MMU initialization is given in section 3.1.8.

Note: Refer Chapter 3 - *Memory Management Unit* of the ARM926EJ-S TRM downloadable from <http://www.arm.com/arm/TRMs> for more details.

3.1.5.2 Caches and Write Buffer

The ARM926EJ-S processor includes:

- An instruction cache (Icache)
- A Data cache (Dcache)
- A write buffer

The size of the Data cache is 8 KB, Instruction Cache is 16 KB and Write buffer is 17 bytes.

The caches have the following features:

- The caches are virtual index, virtual tag, addressed using the Modified Virtual Address (MVA).
- The caches are four-way set associative, with a cache line length of eight words per line (32 byte per line) and with two dirty bits in the Dcache.
- The Dcache supports write-through and write-back (or copy back) cache operation, selected by memory region using the C and B bits in the MMU translation tables.
- The caches perform critical-word first cache refilling.
- Cache lockdown registers enable control over which cache ways are used for allocation on a linefill, providing a mechanism for both lockdown, and controlling cache pollution.
- The Dcache stores the Physical Address TAG (PA TAG) corresponding to each Dcache entry in the TAGRAM for use during the cache line write-backs, in addition to the Virtual Address TAG stored in the TAG RAM. This means that the MMU is not involved in Dcache write-back operations, removing the possibility of TLB misses related to the write-back address.
- Cache maintenance operations to provide efficient invalidation of:
 1. the entire Dcache or Icache
 2. regions of the Dcache or Icache
 3. regions of virtual memory

They also provide operations for efficient cleaning and invalidation of:

1. the entire Dcache
2. regions of the Dcache
3. regions of virtual memory

The write buffer is used for all writes to a noncachable bufferable region, write-through region and write misses to a write-back region. A separate buffer is incorporated in the Dcache for holding write-back for cache line evictions or cleaning of dirty cache lines.

The main write buffer has 16-word data buffer and a four-address buffer.

The Dcache write-back has eight data word entries and a single address entry.

The MCR drain write buffer enables both write buffers to be drained under software control.

The MCR wait for interrupt causes both write buffers to be drained and the ARM926EJ-S processor to be put into a low power state until an interrupt occurs.

Example code for cache and write buffer control is given in section 3.1.8.

Note: Refer Chapter 4 - *Caches and Write Buffer* of the ARM926EJ-S TRM, downloadable from <http://www.arm.com/arm/TRMs> for more details.

3.1.6 Tightly coupled memory

The ARM926EJ has a tightly coupled memory interface enabling separate instruction and data TCM to be interfaced to ARM. TCMs are meant for storing real-time and performance critical code. Access to TCM memories are fast and

DM320 supports both instruction TCM and data TCM. The instruction TCM is located at the memory 0x0000:0000 to 0x0000:5FFF. The data TCM is located at the memory 0x0002:0000 to 0x0002:5FFF.

The status of the TCM memory regions can be read from the TCM status register, which is CP15 register 0. The instruction for reading the TCM status is given below.

```
MRC p15, #0, Rd, c0, c0, #2 ; read TCM status register
```

Where Rd is any register where the status data is read into. The format of the data in TCM register is as given below.

31	17	16	15	1	0
SBZ/UNP				ITCM	

If DTCM bit is '0', Data TCM is not present and if '1' Data TCM is present. If ITCM bit is '0', Instruction TCM is not present and if '1' Instruction TCM is present.

ITCM and DTCM can be enabled and the addresses can be read or written using ITCM / DTCM region registers, which is the CP15 register 9. The instruction for reading and writing to the ITCM and DTCM are given below.

```
MRC p15, #0, Rd, c9, c0, #0 ; read DTCM region register
MCR p15, #0, Rd, c9, c0, #0 ; write DTCM region register
MRC p15, #0, Rd, c9, c0, #1 ; read ITCM region register
MCR p15, #0, Rd, c9, c0, #1 ; write ITCM region register
```

Where Rd is any register where the data is read or written into. The format of the data in TCM register is as given below.

31	12	11	6	5	1	0
ADDRESS			SBZ/UNP		SIZE	0 ENB

ITCM and DTCM can be enabled by writing '0' to the ENB bit and can be enabled by writing '1'. The physical address of the memory should be set to the ADDRESS field and the SIZE field reflects the size. The size encoding is given below in Table 9.

Binary code	Size
0000	0 KB / absent
0001,0010	Reserved
0011	4 KB
0100	8 KB
0101	16 KB
0110	32 KB
0111	64 KB
1000	128 KB
1001	256 KB
1010	512 KB
1011	1 MB
11xx	Reserved

Table 9: ITCM / DTCM size encoding

In DM320 size of ITCM and DTCM are 24 KB and the addresses are 0x0000:0000 – 0x0000:5FFF for ITCM and 0x0002:0000 – 0x0002:5FFF for DTCM.

Example code for TCM control is given in section 3.1.9

Note: Refer Chapter 5 – *Tightly-coupled memory interface* of the ARM926EJ-S TRM, downloadable from <http://www.arm.com/arm/TRMs> for more details.

3.1.7 Embedded Trace Macro Cell Support

To support real-time trace, the ARM926EJ-S processor provides an interface to enable connection of an Embedded Trace Macrocell (ETM).

The ETM consists of two parts:

Trace Port: A trace protocol has been developed to provide a real-time trace capability for processor cores that are deeply embedded in large ASIC designs. Because the ASIC normally includes significant amounts of on-chip memory, it is not possible to determine how the processor core is operating by only observing the pins of the ASIC. A trace port is required to understand the operation of the processor.

Triggering facilities: An extensible specification exists to specify the exact set of trigger resources required for a particular application. Resources include address and data comparators, counter, and sequencers.

The ETM is used to compress the trace information and export it through a narrow trace port. An external Trace Port Analyzer (TPA) is used to capture the trace information.

Note: Refer Chapter 10 – *Embedded Trace Macro-cell support* of the ARM926EJ-S TRM, downloadable from <http://www.arm.com/arm/TRMs> for more details.

3.1.8 Example Code for initializing MMU and Cache

```
.sect ".text"
.state32
.global _cacheMMUEnable

_cacheMMUEnable:
MRC p15, #0, R1, c1, c0, #0 ;Load CP15 Control Register 1
LDR R0, disableIDMMU
BIC R1, R1, R0 ;Disable D and I cache and MMU
MCR p15, #0, R1, c1, c0, #0 ;Store CP15 Control Register 1
MOV R0, #0
MCR p15, #0, R0, c7, c6, #0 ;Flush Data Cache

;Setup Domain3 as Manager for full access
;First Section is going to use Domain 3 as the domain control
;Read Domain Access Control Register
MRC p15, #0, R0, c3, c0, #0
ORR R0, R0, #0xC0 ;Domain3 as manager
MCR p15, #0, R0, c3, c0, #0
MRC p15, #0, R0, c3, c0, #0 ;Debug: Verify

LDR R0, translationTableBase
MCR p15, #0, R0, c2, c0, #0 ;Store TLB Base
LDR R1, firstSectionDescriptor
STR R1, [R0] ;First Entry of Level 1

LDR R0, translationTableBaseFlash
LDR R1, flashDesc ; Entry for SRAM/Flash
STR R1, [R0]
LDR R0, translationTableBaseSDRAM
LDR R1, sdramDesc ; Entry for SDRAM (1 MB -> 1 entries)
STR R1, [R0]

; flush TLBs
MOV R0, #0
NOP
NOP
MCR p15, #0, r0, c8, c5, #0
NOP
NOP
NOP
NOP
MCR p15, #0, r0, c8, c6, #0
NOP
NOP
NOP
NOP
NOP

;flush cache
MOV R0, #0
NOP
NOP
MCR p15, #0, r0, c7, c5, #0
NOP
NOP
NOP
NOP
MCR p15, #0, r0, c7, c6, #0
NOP
NOP
NOP
NOP
NOP

;enable the instruction cache and data cache
MRC p15, #0, r1, c1, c0, #0
MOV R0, #0x1000
ORR r1, r1, R0 ; enable IC
```

```

NOP
MCR p15, #0, r1, c1, c0, #0
NOP
NOP

MRC p15, #0, r1, c1, c0, #0
MOV R0, #13
ORR r1, r1, r0 ; enable DC, MMU and WB (WB - 8, DC - 4, MMU - 1)
NOP
MCR p15, #0, r1, c1, c0, #0
NOP
NOP
MRC p15, #0, r1, c1, c0, #0 ; verify
NOP
NOP

MOV PC, R14 ; Go back to where cacheMMUEnable() is called from

NOP
NOP

.align 4

translationTableBase .word 0x00F00000 ;Base for translation table and IRAM and control registers
translationTableBaseFLASH .word 0x00F00004 ;Base for SRAM / FLASH (starts at 0x00100000)
translationTableBaseSDRAM .word 0x00F00024 ;Base for SDRAM (starts at 0x00900000)

;Bit 1-0 Should be 10 for a section
;B Bit 2 Bufferable (B_mmu): indicates that data writes at this address
; are buffered if the write buffer is enabled.
;C Bit 3 Cacheable (C_mmu): indicates that data or instructions at this
; address will be placed in the cache if the cache is enabled Value = 1;
;Bit 4 Reserved, Should be written to as 1 for backward compatibility.
;Bits 8-5 Specify which one of the sixteen domains contains the primary access controls.
; Value = 3; (Using the 3rd Domain)
;Bit 9 Reserved, Should always be written to as 0.
;AP Bits 11-10 Specify the access permissions for this section
; Neglected as Domain 3 will be set as manager, value = 3
;Bits 19-12 Reserved, should always be written to as 0.
;Bits 31-20 Are the 12 MSBs of the address of the section in physical memory (Section Base Address).
; Value = 0x081

firstSectionDescriptor .word 0x00000C72 ; IRAM, DSP memory and control registers -non cacheable

; All regions below are cacheable

flashDesc .word 0x001000C7A ; Flash - I and D cacheable, Write through
sdramDesc .word 0x009000C7C ; SDRAM - I and D cacheable, Write back

disableIDMMU .word 0x0001005
enableDCacheMMU .word 0x0000005
disMMU .word 0x0000001
.end

```

3.1.9 Example Code for changing TCM addresses

```

MOV r0, #0 ; Disable
MCR p15, #0, r0, c9, c0, #0 ; Disable DTCM
NOP
NOP

MOV r0, #0x2019 ; Start address 0x20000, 32 KB size and enable
MCR p15, #0, r0, c9, c0, #0 ; Enable DTCM
NOP
NOP

```

4 ARM Bus Controllers

4.1 Introduction

- Supports Interconnect access
- Support Advanced Internal Memory (AIM) as used for ARM926's I/D-TCM and will be used as Sub-system's memory.
- Programmable SDRAM start address and end address
- The number of wait-state are inserted automatically according to the peripheral module accessed by ARM
- All peripheral module registers except USB and VLYNQ are accessed only by in 16-bit width access.
- Support for endian-conversion of data for accessing Coprocessor-subsystem and VLYNQ.
- To reduce latency and power consumption while accessing program memory, memory bus is separated from peripheral bus

4.2 AIM re-map

TCM re-map is required since the Interrupt vector table (IVT) is located at 0x0000:0000 for all the internal boot-modes. When booting from Internal ROM, ROM is mapped to 0x0000:0000. After the boot-loader or application is downloaded into IRAM / SDRAM, the TCM memory has to be re-mapped, so that RAM is present at 0x0000:0000 location. This facilitates the IVT to be loaded at address 0x0000:0000. The TCM re-map can be done using the AIMREMAP bit in

BUSCTRL register. The regions are mapped based on the BTSEL[1:0] signal state. Only address of IRAM0 and IROM changes with these. IRAM1 address remains constant at 0x0000:2000. The addresses are as shown in Table 10.

BTSEL [1:0]	AIMREMAP	Memory	Start Address
00 (CS0 mode)	0	IRAM0 IROM	0x0000:0000 0x0000:4000
00 (CS0 mode)	1	IROM IRAM0	0x0000:0000 0x0000:4000
01 (External boot mode)	0	IRAM0 IROM	0x0000:0000 0x0000:4000
01 (External boot mode)	1	IROM IRAM0	0x0000:0000 0x0000:4000
10 (Internal ROM boot)	0	IROM IRAM0	0x0000:0000 0x0000:4000
10 (Internal ROM boot)	1	IRAM0 IROM	0x0000:0000 0x0000:4000
11 (CS0 mode)	0	IRAM0 IROM	0x0000:0000 0x0000:4000
11 (CS0 mode)	1	IROM IRAM0	0x0000:0000 0x0000:4000

Table 10: AIM IRAM0-IROM address re-map

4.3 Endian Conversion

4.3.1 Using BUSC registers

The endianness of the data can be changed using ECR register. Writing data to be converted into ECR register allows the byte and bit endian converted data to be read in EBYTER and EBITR registers respectively. Figure 11 illustrates the endian

conversion.

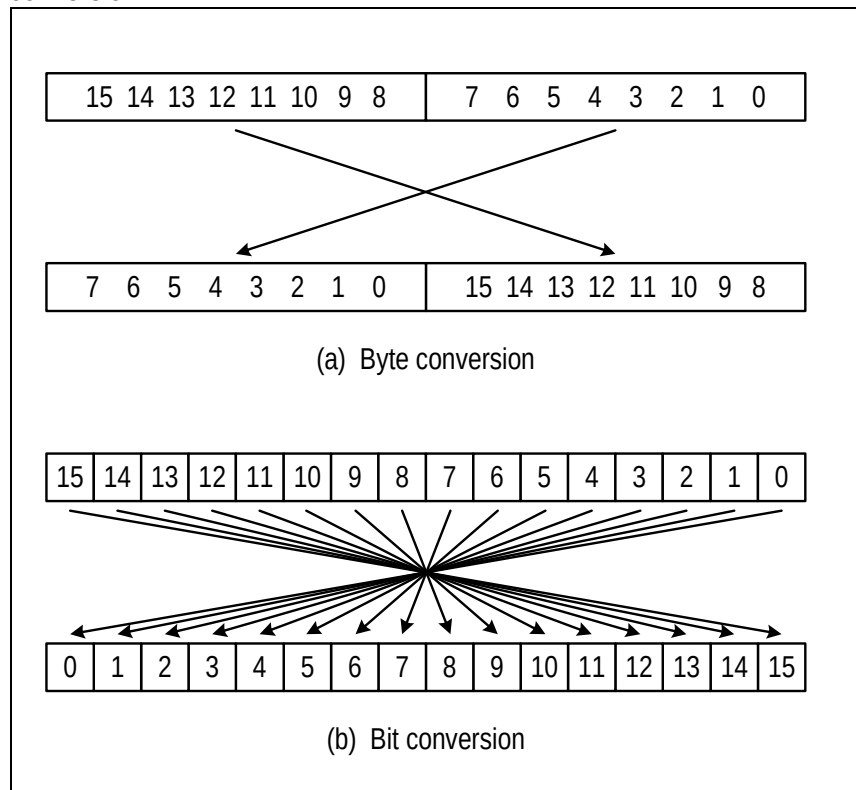


Figure 11: Endian Conversion

4.3.2 Endian Conversion for Co-processors and VLYNQ

Endian conversion is possible from ARM subsystem to Co-processor system. ARM system is 32-bit data width and Co-pro system is a 16-bit data width. So, it supports endian conversion of byte, half-word and word. Both little endian and big endian conversions are supported. The detail of endian conversion for co-processors is given in Table 10.

Size (In bytes)	Address (binary)	Endian conversion	Data after conversion
1	XX00	Little endian	xx D0
		Big endian	D0 xx
	XX01	Little endian	D1 xx
		Big endian	Xx D1
	XX10	Little endian	Xx D2
		Big endian	D2 xx
2	XX00	Little endian	D3 xx
		Big endian	xx D3
	XX01	Little endian	D1 D0
		Big endian	D0 D1
	XX10	Little endian	D3 D2
		Big endian	D2 D3
4	XX00	Little endian	D1 D0
		Big endian	D0 D1
	XX10	Little endian	D3 D2
		Big endian	D2 D3
		Little endian	D1 D0
		Big endian	D0 D1

Table 11: Endian conversion from ARM to Copro system

Note: X indicates it can be any value, Data before conversion is D3 D2 D1 D0

Endian conversion is also possible between ARM subsystem and VLYNQ master/slave. VLYNQ are 32-bit data width too. It supports endian conversion of byte, half-word and word. Both little endian and big endian conversions are supported. The details of endian conversion for VLYNQ master and slave is given in Table 12

Size (In bytes)	Address (binary)	Endian conversion	Data after conversion
1	XX00	Little endian	XX XX XX D0
		Big endian	D0 XX XX XX
	XX01	Little endian	XX XX D1 XX
		Big endian	XX D1 XX XX
	XX10	Little endian	XX D2 XX XX
		Big endian	XX XX D2 XX
2	XX00	Little endian	D3 XX XX XX
		Big endian	XX XX XX D3
	XX01	Little endian	XX XX D1 D0
		Big endian	D0 D1 XX XX
	XX10	Little endian	D3 D2 XX XX
		Big endian	XX XX D2 D3
4	XX00	Little endian	D3 D2 D1 D0
		Big endian	D0 D1 D2 D3

Table 12: Endian conversion for VLYNQ master and slave

Note: X indicates it can be any value, Data before conversion is D3 D2 D1 D0

4.4 SDRAM and CS0 address

The starting address and end address of SDRAM is configured using the SDRAMSA and SDRAMEA registers. The starting address of SDRAM automatically is the end address of CS0 region. Figure 12 illustrates this.

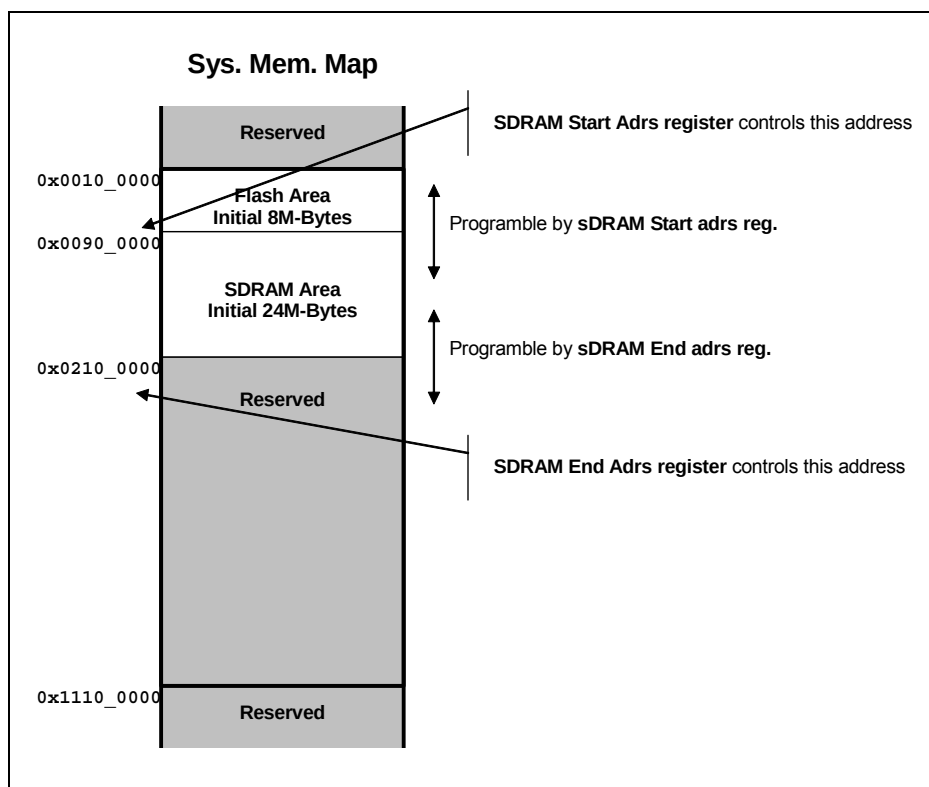


Figure 12: SDRAM and CS0 address control

4.5 Peripheral area

There are two peripherals register addresses as the following.

- Address 0x0003:xxxx - This area should be accessed by half word (16-bit)
- Address 0x0006:xxxx & 0x8000:xxxx - This area allow all access type (Byte/Half-word/Word).

Peripheral Area Name	Start address	End address	Access Cycle Number	Bus Width [bit]	Access Mode
Timer 0	0x0003:0000	0x0003:007F	1	16	Half-word
Timer 1	0x0003:0080	0x0003:00FF	1	16	Half-word
Timer 2	0x0003:0100	0x0003:017F	1	16	Half-word
Timer 3	0x0003:0180	0x0003:01FF	1	16	Half-word
Serial 0	0x0003:0200	0x0003:027F	1	16	Half-word
Serial 1	0x0003:0280	0x0003:02FF	1	16	Half-word
UART 0	0x0003:0300	0x0003:037F	2+n ^(*)	16	Half-word
UART 1	0x0003:0380	0x0003:03FF	2+n ^(*)	16	Half-word
WDT	0x0003:0400	0x0003:047F	1	16	Half-word
SD/MMC IF	0x0003:0480	0x0003:04FF	1+n ^(*)	16	Half-word
INTC	0x0003:0500	0x0003:057F	1	16	Half-word
GIO	0x0003:0580	0x0003:05FF	1	16	Half-word
DSP Controller	0x0003:0600	0x0003:067F	1	16	Half-word
OSD	0x0003:0680	0x0003:06FF	1	16	Half-word
CCD Controller	0x0003:0700	0x0003:077F	1	16	Half-word
(Reserved)	0x0003:0780	0x0003:07FF	1	16	Half-word
NTSC/PAL Encoder	0x0003:0800	0x0003:087F	1	16	Half-word

Peripheral Area Name	Start address	End address	Access Cycle Number	Bus Width [bit]	Access Mode
Clock Controller	0x0003:0880	0x0003:08FF	1	16	Half-word
Bus Controller	0x0003:0900	0x0003:097F	1	16	Half-word
SDRAM Controller	0x0003:0980	0x0003:09FF	1	16	Half-word
EMIF	0x0003:0A00	0x0003:0A7F	1	16	Half-word
Preview0	0x0003:0A80	0x0003:0AF F	1+n ^(*)	16	Half-word
Preview1	0x0003:0B00	0x0003:0B7F	1+n ^(*)	16	Half-word
3A	0x0003:0B80	0x0003:0BF F	1	16	Half-word
(Revered)	0x0003:0C00	0x0003:0C7 F	1	16	Half-word
Memory Stick	0x0003:0C80	0x0003:0CF F	1	16	Half-word
ATM	0x0003:0D00	0x0003:0D7 F	1	16	Half-word
I2C	0x0003:0D80	0x0003:0DF F	1	16	Half-word

(*) n: This is variable number depending on access condition.

Table 13 : 16-bit Peripheral Memory Space

Peripheral Name	Start address	End address
VLYNQ	0x0006:0300	0x0006:03FF
AHBBUSC	0x0006:0F00	0x0003:0FFF
USB	0x8000:0000	0x0003:00FF
USB DMA	0x8000:0200	0x8000:00FF

Table 14 : 32-bit Peripheral Memory Space

4.6 Bus Controller Register Map (BSC)

Offset	Address	Register	Description
0x00	0003:0900	ECR	Endian Conversion Register
0x01	0003:0902	EBYTER	Endian Byte Reverse Register
0x02	0003:0904	EBITR	Endian Bit Reverse Register
0x03	0003:0906	REVR	Device Revision Register

4.7 Bus Controller Registers

4.7.1 ECR

Endian conversion register

ECR	0003:0900		offset: 0x00	default: 0x0000												
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	ECR[15]	ECR[14]	ECR[13]	ECR[12]	ECR[11]	ECR[10]	ECR[9]	ECR[8]	ECR[7]	ECR[6]	ECR[5]	ECR[4]	ECR[3]	ECR[2]	ECR[1]	ECR[0]
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:0	ECR	0	R/W	<u>Endian conversion register</u> The endianness of data written to this register is converted and stored in the EBYTER and EBITR registers

4.7.2 EBYTER

Endian byte reverse register

EBYTER	0003:0902		offset: 0x02	default: 0x0000												
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	EBYTER[15]	EBYTER[14]	EBYTER[13]	EBYTER[12]	EBYTER[11]	EBYTER[10]	EBYTER[9]	EBYTER[8]	EBYTER[7]	EBYTER[6]	EBYTER[5]	EBYTER[4]	EBYTER[3]	EBYTER[2]	EBYTER[1]	EBYTER[0]
RW	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:0	EBYT	0	R	<u>Endian Byte Conversion register</u> Contains byte-swapped contents of data written to ECR register

4.7.3 EBTR

Endian Bit reverse register

EBTR	0003:0904			offset: 0x04			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	EBIT[15]	EBIT[14]	EBIT[13]	EBIT[12]	EBIT[11]	EBIT[10]	EBIT[9]	EBIT[8]	EBIT[7]	EBIT[6]	EBIT[5]	EBIT[4]	EBIT[3]	EBIT[2]	EBIT[1]	EBIT[0]
R/W	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:0	EBIT	0	R	<u>Endian Bit Conversion register</u> Contains bit-swapped contents of data written to ECR register

4.7.4 REVR

Device revision register

REVR	0003:0906			offset: 0x06			default: 0x0010									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	REV[7]	REV[6]	REV[5]	REV[4]	REV[3]	REV[2]	REV[1]	REV[0]
R/W	-	-	-	-	-	-	-	-	R	R	R	R	R	R	R	R
Default	-	-	-	-	-	-	-	-	0	0	0	1	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:8	RSV			Reserved
7:0	REV	16	R	<u>Device Revision number</u> 0x10 - ES1.0

4.8 AHB Bus Controller Register Map (AHBBUSC)

Offset	Address	Register	Description
0x00	0006:0F00	SDRAMSA	SDRAM start address
0x02	0006:0F08	SDRAMEA	SDRAM end address
0x04	0006:0F10	BUSCONTROL	Bus endianness control
0x06	0006:0F18	RSV1	Reserved
0x08	0006:0F20	RSV2	Reserved

4.9 AHB Bus Controller Registers

4.9.1 SDRAMSA

SDRAM Start address register

SDRAMSA		0006:0F00		offset: 0x00												default: 0x00900000	
Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	SDRAMSA[24]	SDRAMSA[23]	SDRAMSA[22]	SDRAMSA[21]	SDRAMSA[20]	SDRAMSA[19]	SDRAMSA[18]	SDRAMSA[17]	SDRAMSA[16]	
R/W	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R	R	R	R	
Default	-	-	-	-	-	-	-	0	1	0	0	1	0	0	0	0	

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	SDRAMSA[15]	SDRAMSA[14]	SDRAMSA[13]	SDRAMSA[12]	SDRAMSA[11]	SDRAMSA[10]	SDRAMSA[9]	SDRAMSA[8]	SDRAMSA[7]	SDRAMSA[6]	SDRAMSA[5]	SDRAMSA[4]	SDRAMSA[3]	SDRAMSA[2]	SDRAMSA[1]	SDRAMSA[0]
R/W	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
31:25	RSV			Reserved
24:0	SDRAMSA	0x00900000	R/W	<u>SDRAM Start address</u> Note: Bits 19:0 should be zero, Only bits 24:20 can be changed

4.9.2 SDRAMEA

SDRAM end address register

SDRAMEA				0006:0F04				offset: 0x04				default: 0x02100000				
Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Name	RSV	RSV	RSV	SDRAMEA[28]	SDRAMEA[27]	SDRAMEA[26]	SDRAMEA[25]	SDRAMEA[24]	SDRAMEA[23]	SDRAMEA[22]	SDRAMEA[21]	SDRAMEA[20]	SDRAMEA[19]	SDRAMEA[18]	SDRAMEA[17]	SDRAMEA[16]
R/W	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R	R	R	R
Default	-	-	-	0	0	0	1	0	0	0	0	1	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	SDRAMEA[15]	SDRAMEA[14]	SDRAMEA[13]	SDRAMEA[12]	SDRAMEA[11]	SDRAMEA[10]	SDRAMEA[9]	SDRAMEA[8]	SDRAMEA[7]	SDRAMEA[6]	SDRAMEA[5]	SDRAMEA[4]	SDRAMEA[3]	SDRAMEA[2]	SDRAMEA[1]	SDRAMEA[0]
R/W	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
31:29	RSV			Reserved
28:0	SDRAMEA	0x2100000	R/W	SDRAM End address Note: Bits 19:0 should be zero, Only bits 28:20 can be changed

4.9.3 BUSCTRL

Bus endianness control register

BUSCONTROL				0006:0F08				offset: 0x08				default: 0x00000000				
Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	AIMWAIT	OROMWAIT
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W	R/W
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	AIMREMAP	RSV	RSV	RSV	RSV	RSV	VLYNQSE	VLYNQME	COPE
R/W	-	-	-	-	-	-	-	R/W	-	-	-	-	-	R/W	R/W	R/W
Default	-	-	-	-	-	-	-	0	-	-	-	-	-	0	0	0

Bit	Name	Reset Value	R/W	Function
31:18	RSV			Reserved
17	AIMWAIT	0	R/W	<u>Insert 1 cycle to access all AIM area by TCM I/F</u> » 0: disable additional wait state for AIM « » 1: enable 1 additional wait state for AIM « Note: 1 additional cycle is inserted for both AIM RAM and ROM accesses
16	NOROMWAIT	0	R/W	<u>Do not insert 1 cycle to read AIMROM by TCM I/F</u> » 0: enable 1 wait state for AIMROM « » 1: disable 1 wait state for AIMROM « Note: This is wait is in addition to wait applied by the setting of the AIMWAIT field
15:9	RSV			Reserved
8	AIMREMAP	0	R/W	<u>AIM remap from boot condition selected by BTSEL[1:0]</u> » 0: No remap « » 1: Remap « NOTE: AIM remap should only be applied after ROM boot to remap AIM RAM to 0x0000 to allow user defined interrupt vector table. Setting this bit after normal boot will map ROM to 0x0000 and prevent user from defining ARM interrupt vector table. <u>Normal Boot and AIM/SIF boot:</u> RAM0: 0x0000-0x1FFF RAM1: 0x2000-0x3FFF ROM : 0x4000-0x5FFF <u>ROM/NAND boot:</u> ROM : 0x0000-0x1FFF RAM1: 0x2000-0x3FFF RAM0: 0x4000-0x5FFF
7:3	RSV			Reserved
2	VLYNQSE	0	R/W	<u>VLYNQ slave endianness</u> » 0: None - AHB (little) -> VLYNQ_S (little) « » 1: Swap - AHB (little) -> VLYNQ_S (big) «
1	VLYNQME	0	R/W	<u>VLYNQ master endianness</u> » 0: None - VLYNQ_M (little) -> AHB (little) « » 1: Swap - VLYNQ_M (big) -> AHB (little) «
0	COPE	0	R/W	<u>Co-processor endianness convert</u> » 0: None - AHB (little) -> Coproc (little) « » 1: Swap - AHB (little) -> Coproc (big) «

5 Clock Controller

Please refer to the latest DM320 datasheet for supplementary PLL / clock controller related information like maximum ARM, DSP, AXL, SDRAM frequencies.

5.1 Introduction

The Clock Controller (CLKC) module generates multiple clock frequencies from a single crystal / external clock input. This module generates an internal interim frequency as a multiple of the input frequency, then divides this frequency down to obtain the necessary clocks to drive the ARM, peripherals, coprocessors, SDRAM and DSP. The input frequency is user definable within the allowable range of 20-35MHz, however under normal conditions use 27MHz crystal to allow for accurate NTSC/PAL video output. The internal interim frequency is the direct output from the PLL and can be divided from 1 up to 32 ($\times 1$, $1/2$, $1/3$, ..., $1/32$) of the PLL output frequency. For USB there is a separate 48 MHz crystal input.

CLKC module has two built in analog PLLs. The user can set the frequencies of the ARM clock, SDRAM clock, Coprocessor and DSP clock using any of these PLLs and by setting the appropriate divider values. To reduce system power, each clock frequency can be slowed down or disabled.

The Clock Controller has the following external inputs:

- o **PCLK:** External pixel clock input
- o **SYSCLK:** External system clock input (optional)
- o **MXI:** Externally attached crystal oscillator input
- o **M48XI:** Externally attached 48MHz crystal oscillator input

In normal operation, a 27MHz crystal output signal is applied to MXI and a 48MHz signal is applied to M48XI. The CCD Controller (CCDC) pixel clock is connected to PCLK. The maximum PCLK frequency is 40MHz, since the CCDC operates at a maximum of 40MHz. The DM320 passes-through MXI and M48XI to output signals MXO and M48XO, respectively.

The Clock Controller generates the following clock signals for different modules or processors:

- o **CLK_ARM:** ARM clock for ARM core and its related modules
- o **CLK_DSP:** DSP clock for DSP core and its related modules
- o **CLK_AXL:** Accelerator clock for iMX, VLCD and Image Buffers
- o **CLK_SDR:** SDR clock for external SDRAM and its related modules
- o **CLK_AHB:** AHB clock for AHB bus controller, TMR, UART etc

- o CLK_CCD: CCD Controller clock
- o CLK_ENC Video Encoder clock
- o CLK_OSD OSD clock
- o CLK_USB: USB clock
- o CLK_VLYNQ: VLYNQ clock
- o CLK_MMC Multimedia Card / Secure Digital clock
- o CLK_MS Memory Stick clock

Clock controller block diagram is given in Figure 13. Please refer DM320 Data sheet for the frequency range supported by different modules.

PLLA and PLLB generate an intermediate clock output for use by the dividers for all modules. There is a multiplication (M) and division (N) unit in the PLLs. If F_{in} is the input frequency and $PLLf_{out}$ is the output frequency, the formula for calculating $PLLf_{out}$ is as given below:

$$PLLf_{out} = F_{in} * M / N.$$

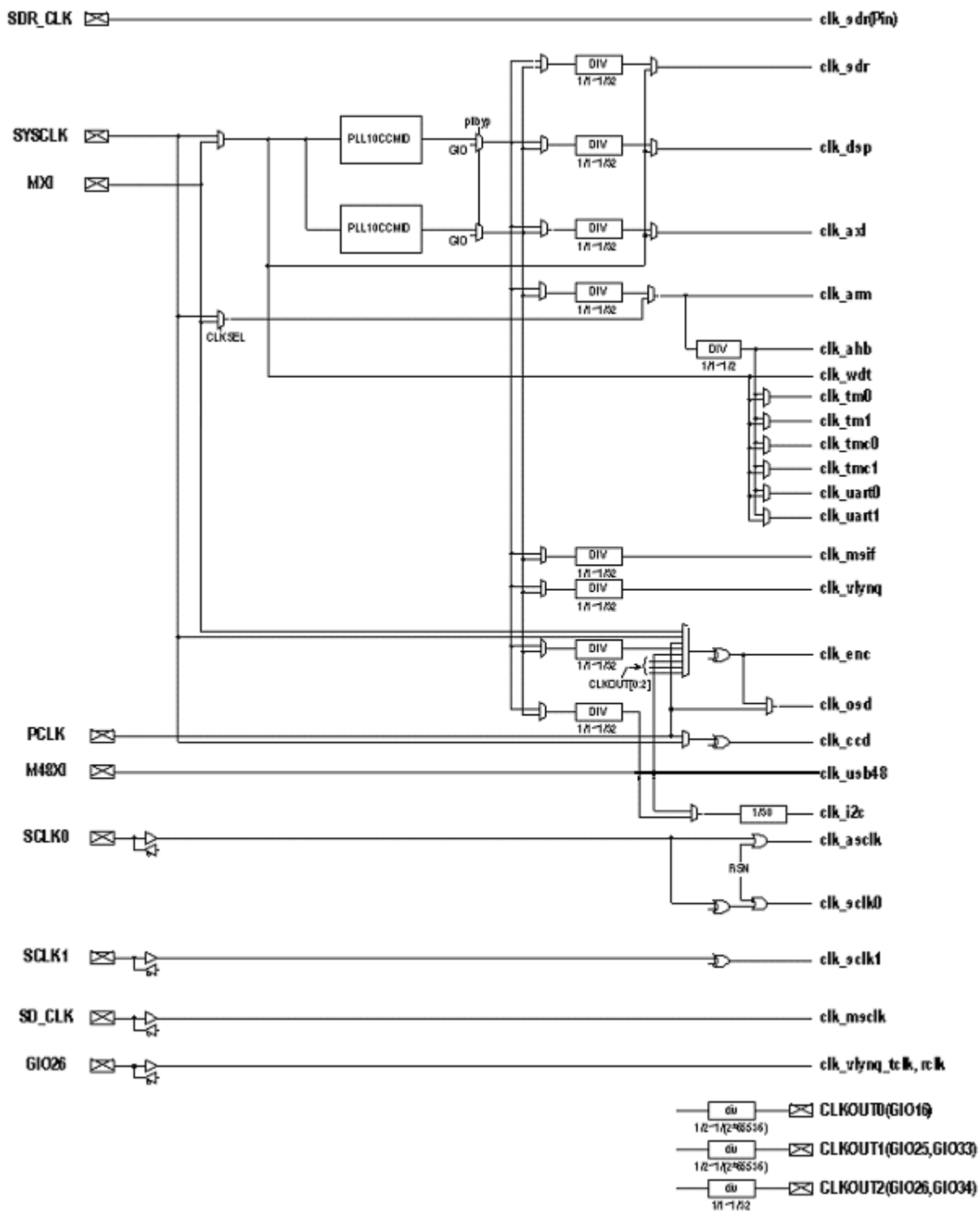
By default, the all the PLLs are bypassed during power-on reset, and either MXI or SYSCLK are used as the ARM clock based on the signal - CLKSEL. After the ARM is rebooted, the ARM can configure the clocks for various modules by setting respective registers in the clock controller module.

The clock source for many modules can be selected. Please refer registers SEL0, SEL1 and SEL2 for more details.

The clocks used by many different modules can be brought out through the three CLKOUT pins, which are multiplexed with GPIOs. Please refer the register OSEL for details about the clocks that can be brought out from the chip.

There is an individual clock enable / disable setting for all modules, which is useful for conservation of power. Any module which is not required to be running, can be shut down by turning off the clock. Refer registers MOD0, MOD1 and MOD2 for more details.

Figure 13: DM320 Clock controller block diagram



5.2 Clock operations (selection, division and inversion)

Input clock source can be selected on basis of select registers in clock controller. The input clock source and their relationship with different modules are shown in **Table 15**.

Module	Clock Selection
ARM clock selection during power-on and reset	The PLLs are bypassed during power-on and reset. After the ARM is rebooted, it can configure the clocks of various modules by setting respective registers in the clock controller module.
PLL input clock	Input clock to PLL can be either MXI or SYSCLK depending on the state of CLKSEL pin and register SEL0.
ARM Clock (after power-on reset)	After power-on reset input clock to ARM can be either PLLA output clock or PLLB output clock or PLL input clock (PLL bypass mode). Selection of PLLA or PLLB output is done via register SEL2. The clock can be further divided via register DIV0. PLL can be bypassed using register BYP.
DSP Clock	Input clock to DSP can be either PLLA output clock or PLLB output clock or PLL input clock (PLL bypass mode). Selection of PLLA or PLLB output is done via register SEL2. The clock can be further divided via register DIV2. PLL can be bypassed using register BYP.
SDRAM Clock	Input clock to SDRAM can be either PLLA output clock or PLLB output clock or PLL input clock (PLL bypass mode). Selection of PLLA or PLLB output is done via register SEL2. The clock can be further divided via register DIV1. PLL can be bypassed using register BYP.
AXL Clock	Input clock to AXL (iMX, VLCD, Image Buffers) can be either PLLA output clock or PLLB output clock or PLL input clock (PLL bypass mode). Selection of PLLA or PLLB output is done via register SEL2. The clock can be further divided via register DIV1. PLL can be bypassed using register BYP.

Module	Clock Selection
AHB Clock	AHB clock is derived from ARM clock and the divisor value can be configured in DIV0
UART, Timer Clock	Input clock to UART0-1, Timer 0-3 can be selected independently as either PLL input clock or AHB clock output via register SEL0
MS clock	Input clock to MS can be selected via register SEL0. The clock can be further divided via register DIV2.
Video Encoder clock	Input clock to video encoder module register SEL1. The clock can be further divided via register DIV3. The clock can be inverted via the register INV.
OSD clock	Input clock to OSD module can be either CLK_VENC or CLK_VENC / 2. This is selected via register SEL1
CCD Clock	Input Clock to CCD can be either PCLK or SYSCLK. This is selected via register SEL1. The clock can be inverted via the register INV.
I2C clock	Input clock to I2C can be selected via register SEL0
USB clock	Input clock to USB is based on M48XI.
VLYNQ clock	Input clock to VLYNQ can be selected via register SEL0. The clock can be further divided via register DIV4.
MMC clock	MMC clock can be further divided from the AHB clock using DIV4. The clock can be inverted via the register INV.
SIF clock	SIF0 and SIF1 clock can be inverted via the register INV.

Table 15: Input Clock Selection

5.3 Setting PLL Clock Output Frequency

The clock for ARM, DSP, AXL, SDRAM, VENC, VLYNQ, Memory stick and I2C can be generated from the PLLs. The output frequency of each of the two PLLs can be set separately. **Figure 14** shows the PLL structure. The PLL is a built-in analog PLL constructed of a charge pump, loop filter and VCO.

The ARM, DSP, AXL and SDRAM clock frequencies are setup by selecting the PLL, and setting the divider connected to the input of ARM, DSP, AXL, SDRAM module.

The PLL output frequency is determined by setting the division ratio of the counter N, which divides the input clock, and the counter M, which divides the PLL feedback output. The PLL output frequency is given by the following formula:

$$\text{PLLout} = [\text{Fin} * (\text{M}/\text{N})]$$

Where N = 1...4 and M = 1....31

Fin is the PLL input clock signal. M and N can be set in via the register PLLA - CLKC PLLA Configuration and register PLLB - CLKC PLLB Configuration.

The PLL output can be used as input to ARM, DSP, AXL, VENC, VLYNQ, Memory Stick, I2C and SDRAM module. The divider registers, DIV0-4, allows to further divide this PLL output clock without modifying the PLL configuration. This is advantageous because, modifying M and/or N forces the clock to switch to Fin while the PLL locks and also changes the frequency for all modules, which derives the clock from that PLL.

When the values of M and N are changed, the PLL is in an unstable state until the PLL output is stable (locked state). During this time, the clock is switched from the PLL output clock to Fin so that erroneous operation while PLL is unstable does not occur.

Important: User must wait for approximately 500us, since PLLs take around 500us to lock after a change has been made. Please refer to the datasheet for exact value of the delay required.

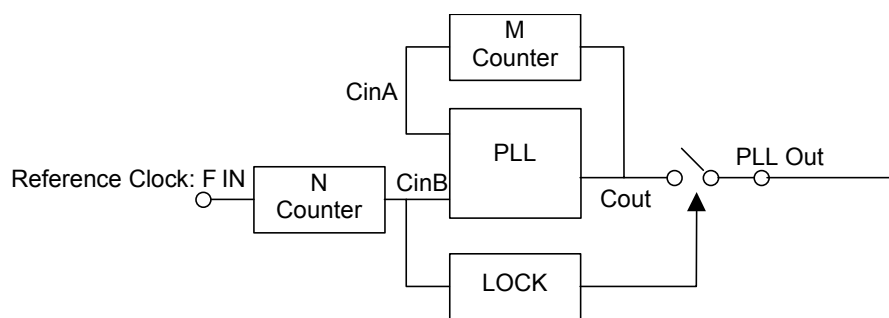


Figure 14: PLL Structure

5.4 PLL Bypass Mode

It is possible to bypass PLL for ARM, DSP, AXL, SDRAM clock by using register BYP. When PLLs are in bypass mode, the input clock to the PLL is directly connected to all the modules.

PLLs can be set to bypass mode, to save power, because of the lower clock frequency of operation. Bypass for ARM, DSP, AXL And SDRAM can be selected separately. This feature allows conditional bypass, depending upon the load on each system.

5.5 Module Clock distribution

Clock supply to individual modules can be turned ON or OFF using registers MOD0, MOD1 and MOD2. This is a very useful feature, as clocks to any unused module can be turned off, thus saving power.

Note: Image buffer, iMX, VLCD clocks can also be enabled from Co-processor clock controller, refer Co-processor clock controller DSP for more details.

5.6 GIO Clock Output

GIO16, GIO25, GIO26, GIO33 and GIO34 can be used as clock output signals. Clock source for these clock outputs can be selected separately via register OSEL. Before outputting the clock a divisor can be applied to the clock via registers O0DIV, O1DIV and O2DIV.

When using GIO16, GIO25, GIO26, GIO33 or GIO34 as clock output signal, the corresponding GIO must be set as output and appropriate function mode must be selected. Refer chapter on GIO for more details.

5.7 Pulse Width Modulation

GIO33 and GIO34 can be used as pulse width modulation (PWM) signals. PLL input clock is used as reference clock for PWM. PWM cycle width, high-level period can be set separately for GIO33 and GIO34. PWM cycle width is selected using PWM0C register for GIO33 and register PWM1C for GIO34. PWM high-level period is selected using register PWM0H for GIO33 and register PWM1H for GIO34.

When using GIO33 and GIO34 as PWM signal, the corresponding GIO must be set as output in register and appropriate function mode must be selected. Refer chapter on GIO for more details. Figure 15 shows the relationship between PWM cycle and PWM high-level signal

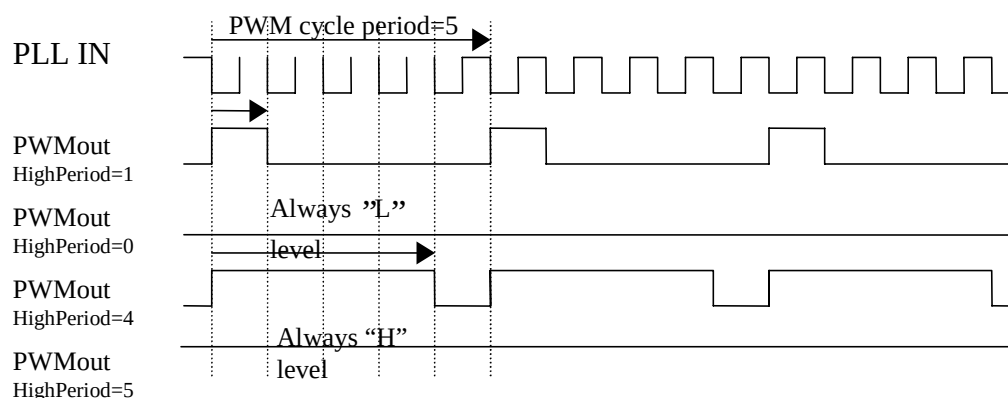


Figure 15: Pulse Width Modulation

5.8 Sleep and Power-down modes

DM320 has a good array of power saving features in addition to individual module clock enable / disable feature. PLLs and oscillators can be put to sleep or power-down mode.

In sleep mode all the clocks are turned off, but the PLLs are not powered down. The system can be woken up using GIO0 or GIO8. The wake up settings / selections must be done before going into the sleep mode. Refer register LPCTL0 for more details.

USB I/O also can be powered down. Refer register LPCTL1

In power-down mode, internal crystal oscillators are also powered down in addition to turning off the clocks of all modules. PWRDN pin is pulled high to achieve this after the PDMD bit in LPCTL1 register is set. The recovery is done automatically by pulling down the PWRDN pin. Refer LPCTL1 for more details.

Refer chapter on DM320 power saving for more details on power saving features.

5.9 Clock Controller Register Map (CLKC)

Address	Register	Description
00030880	PLLA	PLLA Configuration
00030882	PLLB	PLLB Configuration
00030884	SEL0	Input clock Source Selection #0
00030886	SEL1	Input clock Source Selection #1
00030888	SEL2	Input clock Source Selection #2
0003088A	DIV0	Clock Divisor Settings #0
0003088C	DIV1	Clock Divisor Settings #1
0003088E	DIV2	Clock Divisor Settings #2
00030890	DIV3	Clock Divisor Settings #3
00030892	DIV4	Clock Divisor Settings #4
00030894	BYP	Bypass control
00030896	INV	Inverse control
00030898	MOD0	Module Clock Enables #0
0003089A	MOD1	Module Clock Enables #1
0003089C	MOD2	Module Clock Enables #2
0003089E	LPCTL0	Low Power Control #0
000308A0	LPCTL1	Low Power Control #1
000308A2	OSEL	Output Clock Selector
000308A4	O0DIV	Output Clock #0 Divider
000308A6	O1DIV	Output Clock #1 Divider
000308A8	O2DIV	Output Clock #2 Divider
000308AA	PWM0C	PWM #0 Cycle Count
000308AC	PWM0H	PWM #0 High Period
000308AE	PWM1C	PWM #1 Cycle Count
000308B0	PWM1H	PWM #1 High Period

5.10 Clock Controller Registers

5.10.1 PLLA

CLKC PLLA Configuration register

PLLA	0003:0880			offset: 0x00			default: 0x1000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	TEST	PT	PB	PWRDN	RSV	POD[2]	POD[1]	POD[0]	PM[3]	PM[2]	PM[1]	PM[0]	RSV	PN[2]	PN[1]	PN[0]
R/W	R	R/W	R/W	R/W	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	-	R/W	R/W	R/W
Default	0	0	0	1	-	0	0	0	0	0	0	0	-	0	0	0

Bit	Name	Reset Value	R/W	Function
15	TEST	0	R	Test bit for PLLA
14	PT	0	R/W	Test bit for PLLA, set to 0
13	PB	0	R/W	Bypass Test bit for PLLA, set to 0
12	PWRDN	1	R/W	Powerdown for PLLA » 0: normal operation « » 1: powerdown «
11	RSV			Reserved
10:8	POD	0	R/W	Test bits for PLLA, set to 000
7:4	PM	0	R/W	PLLA Numerator $O/P \text{ freq} = Fin * (PM+1)/(PN+1)$
3	RSV			Reserved
2:0	PN	0	R/W	PLLA Denominator $O/P \text{ freq} = Fin * (PM+1)/(PN+1)$

5.10.2 PLLB

PLLB Configuration

PLLB	0003:0882			offset: 0x02												default: 0x1000		
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	TEST	PT	PB	PWRDN	RSV	POD[2]	POD[1]	POD[0]	PM[3]	PM[2]	PM[1]	PM[0]	RSV	PN[2]	PN[1]	PN[0]		
R/W	R	R/W	R/W	R/W	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	-	R/W	R/W	R/W		
Default	0	0	0	1	-	0	0	0	0	0	0	0	-	0	0	0		

Bit	Name	Reset Value	R/W	Function
15	TEST	0	R	Test bit for PLLB
14	PT	0	R/W	Test bit for PLLB, set to 0
13	PB	0	R/W	Bypass Test bit for PLLB, set to 0
12	PWRDN	1	R/W	Powerdown for PLLB » 0: normal operation « » 1: powerdown «
11	RSV			Reserved
10:8	POD	0	R/W	Test bits for PLLB, set to 000
7:4	PM	0	R/W	PLLB Numerator $O/P \text{ freq} = F_{in} * (PM+1)/(PN+1)$
3	RSV			Reserved
2:0	PN	0	R/W	PLLB Denominator $O/P \text{ freq} = F_{in} * (PM+1)/(PN+1)$

5.10.3 SEL0

Input clock Source Selection

SEL0	0003:0884				offset: 0x04				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	I2C	I2CPLL	RSV	VLYNQ	MS	UART1	UART0	CCDTMR1	CCDTMR0	TMR1	TMR0	CFINS
R/W	-	-	-	-	R/W	R/W	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	0	0	-	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:12	RSV			Reserved
11	I2C	0	R/W	<u>I2C Clock select</u> » 0: M48XI « » 1: PLLA/PLLB - PLL selected by I2CPLL « Note: Change only after turning I2C clock off
10	I2CPLL	0	R/W	<u>I2C PLL select</u> » 0: PLLA « » 1: PLLB « Note: Change only after turning I2C clock off
9	RSV			Reserved
8	VLYNQ	0	R/W	<u>VLYNQ Clock select</u> » 0: PLLA « » 1: PLLB « Note: Change only after turning VLYNQ clock off
7	MS	0	R/W	<u>MS Clock select</u> » 0: PLLA « » 1: PLLB « Note: Change only after turning MS clock off
6	UART1	0	R/W	<u>UART1 Clock select</u> » 0: AHB clock « » 1: PLLIN « Note: Change only after turning UART1 clock off
5	UART0	0	R/W	<u>UART0 Clock select</u> » 0: AHB clock « » 1: PLLIN « Note: Change only after turning UART0 clock off
4	CCDTMR1	0	R/W	<u>CCD Timer 1 Clock select</u> » 0: AHB clock « » 1: PLLIN « Note: Change only after turning CCD Timer 1 clock off
3	CCDTMR0	0	R/W	<u>CCD Timer 0 Clock select</u> » 0: AHB clock « » 1: PLLIN « Note: Change only after turning CCD Timer 0 clock off
2	TMR1	0	R/W	<u>Timer 1 Clock select</u> » 0: AHB clock « » 1: PLLIN « Note: Change only after turning Timer 1 clock off
1	TMR0	0	R/W	<u>Timer 0 Clock select</u> » 0: AHB clock « » 1: PLLIN « Note: Change only after turning Timer 0 clock off
0	CFINS	0	R/W	<u>PLL input clock select</u> If CLKSEL = L 0: MXI 1: SYSCLK If CLKSEL = H 0: SYSCLK 1: MXI Note: PLL setting should be done if values are altered

5.10.4 SEL1

PLL Clock Divisor Settings

SEL1	0003:0886			offset: 0x06		default: 0x0000										
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	OSD	RSV	RSV	RSV	CCD	RSV	RSV	RSV	VENCPLL	RSV	VENC[2]	VENC[1]	VENC[0]
R/W	-	-	-	R/W	-	-	-	R/W	-	-	-	R/W	-	R/W	R/W	R/W
Default	-	-	-	0	-	-	-	0	-	-	-	0	-	0	0	0

Bit	Name	Reset Value	R/W	Function
15:13	RSV			Reserved
12	OSD	0	R/W	OSD Clock select » 0: VENC clock / 2 « » 1: VENC clock « Note: Change only after turning OSD clock off
11:9	RSV			Reserved
8	CCD	0	R/W	CCD Clock select » 0: PCLK « » 1: SYSCLK « Note: Change only after turning CCD clock off
7:5	RSV			Reserved
4	VENCPLL	0	R/W	VENC PLL Select » 0: PLLA « » 1: PLLB « Note: Change only after turning VENC clock off
3	RSV			Reserved
2:0	VENC	0	R/W	VENC Clock select » 000: MXI « » 001: SYSCLK « » 010: PCLK « » 011: PLLA/PLLB « » 100: M48XI « » 101: GP clock output 0 (GI016) « » 110: GP clock output 1 (GI025, GI033) « » 111: GP clock output 2 (GI026, GI034) « Note: Change only after turning VENC clock off

5.10.5 SEL2

PLL Source Selection

SEL2 0003:0888 offset: 0x08 default: 0x0000

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	AXL	RSV	RSV	RSV	SDRAM	RSV	RSV	RSV	DSP	RSV	RSV	RSV	ARM
R/W	-	-	-	R/W	-	-	-	R/W	-	-	-	R/W	-	-	-	R/W
Default	-	-	-	0	-	-	-	0	-	-	-	0	-	-	-	0

Bit	Name	Reset Value	R/W	Function
15:13	RSV			Reserved
12	AXL	0	R/W	AXL clock select » 0: PLLA « » 1: PLLB «
11:9	RSV			Reserved
8	SDRAM	0	R/W	SDRAM clock select » 0: PLLA « » 1: PLLB «
7:5	RSV			Reserved
4	DSP	0	R/W	DSP Clock Select » 0: PLLA « » 1: PLLB «
3:1	RSV			Reserved
0	ARM	0	R/W	ARM Clock Select » 0: PLLA « » 1: PLLB «

5.10.6 DIV0

PLL Clock Divisor Settings

DIV0	0003:088A			offset: 0x0A			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	AHB	RSV	RSV	RSV	ARM[4]	ARM[3]	ARM[2]	ARM[1]	ARM[0]
R/W	-	-	-	-	-	-	-	R/W	-	-	-	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	-	-	0	-	-	-	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:9	RSV			Reserved
8	AHB	0	R/W	<u>AHB Dividing ratio</u> Divisor = AHB+1 Note- ARM clock is divided by the divisor
7:5	RSV			Reserved
4:0	ARM	0	R/W	<u>ARM Dividing ratio</u> Divisor = ARM[4:0]+1

5.10.7 DIV1

PLL Clock Divisor Settings

DIV1	0003:088C			offset: 0x0C			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	AXL[4]	AXL[3]	AXL[2]	AXL[1]	AXL[0]	RSV	RSV	RSV	SDRAM[4]	SDRAM[3]	SDRAM[2]	SDRAM[1]	SDRAM[0]
R/W	-	-	-	R/W	R/W	R/W	R/W	R/W	-	-	-	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	0	0	0	0	0	-	-	-	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:13	RSV			Reserved
12:8	AXL	0	R/W	Accelerator Clock Division Divisor = AXL[4:0] + 1
7:5	RSV			Reserved
4:0	SDRAM	0	R/W	SDRAM Clock Division Divisor = SDRAM[4:0] + 1

5.10.8 DIV2

Clock Divisor Settings

DIV2	0003:088E			offset: 0x0E			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	DSP[4]	DSP[3]	DSP[2]	DSP[1]	DSP[0]	RSV	RSV	RSV	MS[4]	MS[3]	MS[2]	MS[1]	MS[0]
R/W	-	-	-	R/W	R/W	R/W	R/W	R/W	-	-	-	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	0	0	0	0	0	-	-	-	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:13	RSV			Reserved
12:8	DSP	0	R/W	DSP clock division Divisor = DSP[4:0] + 1
7:5	RSV			Reserved
4:0	MS	0	R/W	MS clock division Divisor = MS[4:0] + 1

5.10.9 DIV3

Clock Divisor Settings

DIV3	0003:0890			offset: 0x10			default: 0x00FF									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	VENC[4]	VENC[3]	VENC[2]	VENC[1]	VENC[0]	MMC[7]	MMC[6]	MMC[5]	MMC[4]	MMC[3]	MMC[2]	MMC[1]	MMC[0]
R/W	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	0	0	0	0	0	1	1	1	1	1	1	1	1

Bit	Name	Reset Value	R/W	Function
15:13	RSV			Reserved
12:8	VENC	0	R/W	VENC Clock Division Divisor = VENC[4:0]+1
7:0	MMC	255	R/W	MMC Clock Division Divisor = MMC[7:0]+1 Note: AHB clock is divided for getting MMC clock

5.10.10 DIV4

Clock Divisor Settings

DIV4	0003:0892			offset: 0x12			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	VLYNQ[4]	VLYNQ[3]	VLYNQ[2]	VLYNQ[1]	VLYNQ[0]	RSV	RSV	RSV	I2C[4]	I2C[3]	I2C[2]	I2C[1]	I2C[0]
R/W	-	-	-	R/W	R/W	R/W	R/W	R/W	-	-	-	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	0	0	0	0	0	-	-	-	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:5	RSV			Reserved. Do not use
12:8	VLYNQ	0	R/W	VLYNQ clock division Divisor = VLYNQ[4:0] + 1
7:5	RSV			Reserved. Do not use
4:0	I2C	0	R/W	I2C clock division Divisor = I2C[4:0] + 1

5.10.11 BYP

Bypass settings

BYP	0003:0894			offset: 0x14			default: 0x1111									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	AXL	RSV	RSV	RSV	SDRAM	RSV	RSV	RSV	DSP	RSV	RSV	RSV	ARM
R/W	-	-	-	R/W	-	-	-	R/W	-	-	-	R/W	-	-	-	R/W
Default	-	-	-	1	-	-	-	1	-	-	-	1	-	-	-	1

Bit	Name	Reset Value	R/W	Function
15:13	RSV			Reserved
12	AXL	1	R/W	PLL Bypass for AXL » 0: Do not bypass « » 1: Bypass «
11:9	RSV			Reserved
8	SDRAM	1	R/W	PLL Bypass for SDRAM » 0: Do not bypass « » 1: Bypass «
7:5	RSV			Reserved
4	DSP	1	R/W	PLL Bypass for DSP » 0: Do not bypass « » 1: Bypass «
3:1	RSV			Reserved
0	ARM	1	R/W	PLL Bypass for ARM » 0: Do not bypass « » 1: Bypass «

5.10.12 INV

Inverse clock settings

INV	0003:0896				offset: 0x16				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	SIF1	SIF0	RSV	RSV	RSV	CCD	RSV	RSV	RSV	VENC	RSV	RSV	RSV	MMC
R/W	-	-	R/W	R/W	-	-	-	R/W	-	-	-	R/W	-	-	-	R/W
Default	-	-	0	0	-	-	-	0	-	-	-	0	-	-	-	0

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
13	SIF1	0	R/W	<u>Serial-1 interface clock inverse</u> » 0: No-inverse « » 1: Inverse « Note: Change only after turning clock off
12	SIF0	0	R/W	<u>Serial-0 interface clock inverse</u> » 0: No-inverse « » 1: Inverse « Note: Change only after turning clock off
11:9	RSV	0	R/W	Reserved
8	CCD	0	R/W	<u>CCD clock inverse</u> » 0: No-inverse « » 1: Inverse « Note: Change only after turning clock off
7:5	RSV			Reserved
4	VENC	0	R/W	<u>VENC clock inverse</u> » 0: No-inverse « » 1: Inverse « Note: Change only after turning clock off
3:1	RSV			Reserved
0	MMC	0	R/W	<u>MMC clock inverse</u> » 0: No-inverse « » 1: Inverse « Note: Change only after turning clock off

5.10.13 MOD0

Module Clock Enables #0

MOD0	0003:0898				offset: 0x18				default: 0x00F7							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	HPIB	DSP	EXTHOST	SDRAMC	EMIF	INTC	AIM	E2ICE	ETM	AHB	BUSC	ARM
R/W	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	0	0	0	0	1	1	1	1	0	1	1	1

Bit	Name	Reset Value	R/W	Function
15:12	RSV			Reserved
11	HPB	0	R/W	<u>HPIB clock enable</u> » 0: Disable « » 1: Enable «
10	DSP	0	R/W	<u>DSP clock enable</u> » 0: Disable (enabled 128 cycles after reset) « » 1: Enable «
9	EXTHOST	0	R/W	<u>External Host clock enable</u> » 0: Disable « » 1: Enable « Note- Default value is 1 in external host boot mode and 0 for other boot modes
8	SDRAMC	0	R/W	<u>SDRAM controller clock enable</u> » 0: Disable (Enable after 32 cycles after reset) « » 1: Enable « Note- Default value is 1 if FLSHSEL = 1, else 0
7	EMIF	1	R/W	<u>External memory interface clock enable</u> » 0: Disable « » 1: Enable «
6	INTC	1	R/W	<u>Interrupt controller clock enable</u> » 0: Disable « » 1: Enable «
5	AIM	1	R/W	<u>ARM internal memory clock enable</u> » 0: Disable « » 1: Enable «
4	E2ICE	1	R/W	<u>E2ICE clock enable</u> » 0: Disable « » 1: Enable «
3	ETM	0	R/W	<u>ETM clock enable</u> » 0: Disable « » 1: Enable «
2	AHB	1	R/W	<u>AHB clock enable</u> » 0: Disable « » 1: Enable «
1	BUSC	1	R/W	<u>Bus controller clock enable</u> » 0: Disable « » 1: Enable «
0	ARM	1	R/W	<u>ARM clock enable</u> » 0: Disable « » 1: Enable «

5.10.14 MOD1

Module Clock Enables #1

MOD1	0003:089A				offset: 0x1A				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	CPBUS	SEQ	DCT	IMGBUF	IMX	VLCD	DAC	VENC	OSD	PRV	H3A	CCDC
R/W	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:12	RSV			Reserved
11	CPBUS	0	R/W	Co-processor bus clock enable » 0: Disable « » 1: Enable «
10	SEQ	0	R/W	Sequencer clock enable » 0: Disable « » 1: Enable «
9	DCT	0	R/W	DCT clock enable » 0: Disable « » 1: Enable «
8	IMGBUF	0	R/W	Image buffer clock enable » 0: Disable « » 1: Enable «
7	IMX	0	R/W	iMX clock enable » 0: Disable « » 1: Enable «
6	VLCD	0	R/W	VLCD clock enable » 0: Disable « » 1: Enable «
5	DAC	0	R/W	DAC clock enable » 0: Disable « » 1: Enable «
4	VENC	0	R/W	VENC clock enable » 0: Disable « » 1: Enable «
3	OSD	0	R/W	OSD clock enable » 0: Disable « » 1: Enable «
2	PRV	0	R/W	Preview clock enable » 0: Disable « » 1: Enable «
1	H3A	0	R/W	H3A clock enable » 0: Disable « » 1: Enable «
0	CCDC	0	R/W	CCD Controller clock enable » 0: Disable « » 1: Enable «

5.10.15 MOD2

Module Clock Enables #2

MOD2	0003:089C			offset: 0x1C			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	TEST	MS	VLYNQ	I2C	MMC	SIF1	SIF0	UART1	UART0	USB	GIO	CCDTMR1	CCDTMR0	TMR1	TMR0	WDT
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15	TEST	0	R/W	Test bit
14	MS	0	R/W	<u>Memory stick clock enable</u> » 0: Disable « » 1: Enable «
13	VLYNQ	0	R/W	<u>VLYNQ clock enable</u> » 0: Disable « » 1: Enable «
12	I2C	0	R/W	<u>I2C clock enable</u> » 0: Disable « » 1: Enable «
11	MMC	0	R/W	<u>MMC clock enable</u> » 0: Disable « » 1: Enable «
10	SIF1	0	R/W	<u>Serial interface-1 clock enable</u> » 0: Disable « » 1: Enable «
9	SIF0	0	R/W	<u>Serial interface-0 clock enable</u> » 0: Disable « » 1: Enable «
8	UART1	0	R/W	<u>UART-1 clock enable</u> » 0: Disable « » 1: Enable «
7	UART0	0	R/W	<u>UART-0 clock enable</u> » 0: Disable « » 1: Enable «
6	USB	0	R/W	<u>USB clock enable</u> » 0: Disable « » 1: Enable «
5	GIO	0	R/W	<u>GIO clock enable</u> » 0: Disable « » 1: Enable «
4	CCDTMR1	0	R/W	<u>CCD Timer-1 (Timer-3) clock enable</u> » 0: Disable « » 1: Enable «
3	CCDTMR0	0	R/W	<u>CCD Timer-0 (Timer-2) clock enable</u> » 0: Disable « » 1: Enable «
2	TMR1	0	R/W	<u>Timer-1 clock enable</u> » 0: Disable « » 1: Enable «
1	TMR0	0	R/W	<u>Timer-0 clock enable</u> » 0: Disable « » 1: Enable «
0	WDT	0	R/W	<u>Watchdog Timer clock enable</u> » 0: Disable « » 1: Enable «

5.10.16 LPCTL0

Low Power Control #0

LPCTL0	0003:089E				offset: 0x1E				default: 0x0100							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	WUGIO8INV	WUGIO0INV	RSV	WULATCH	WUGIO8	WUGIO0	RSV	RSV	RSV	SLEEP	RSV	RSV	RSV	PLLPD
R/W	-	-	R/W	R/W	-	R/W	R/W	R/W	-	-	-	R/W	-	-	-	R/W
Default	-	-	0	0	-	0	0	1	-	-	-	0	-	-	-	0

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
13	WUGD8INV	0	R/W	<u>Wake up external interrupt signal GIO8 inverse</u> » 0: Non inverse « » 1: Inverse «
12	WUGD0INV	0	R/W	<u>Wake up external interrupt signal GIO0 inverse</u> » 0: Non inverse « » 1: Inverse «
11	RSV			Reserved
10	WULATCH	0	R/W	<u>Latch Wake up signal</u> » 0: Disable « » 1: Enable «
9	WUGD8	0	R/W	<u>Wake up external interrupt signal GIO8</u> » 0: Disable « » 1: Enable «
8	WUGD0	1	R/W	<u>Wake up external interrupt signal GIO0</u> » 0: Disable « » 1: Enable «
7:5	RSV			Reserved
4	SLEEP	0	R/W	<u>Sleep Mode for clocks</u> » 0: Normal « » 1: Sleep « Note: When 1 is written all clocks are turned off and it moves into sleep status. PLL is not powered down. Recovery is done by external interrupt selected by WUGIO0/WUGIO8
3:1	RSV			Reserved
0	PLLPD	0	R/W	<u>PLL Powerdown</u> » 0: Normal « » 1: PLL Powerdown « Note: When 1 is written PLL is powered down and all clocks turned off. Recovery is done by external interrupt selected by WUGIO0/WUGIO8

5.10.17 LPCTL1

Low Power Control #1

LPCTL1 0003:08A0 offset: 0x20 default: 0x0010

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	USBPDN	RSV	RSV	PDMD	OSC48
R/W	-	-	-	-	-	-	-	-	-	-	-	R/W	-	-	R/W	R/W
Default	-	-	-	-	-	-	-	-	-	-	-	1	-	-	0	0

Bit	Name	Reset Value	R/W	Function
15:5	RSV			<u>Reserved</u>
4	USBPDN	1	R/W	<u>USB I/O Buffer Powerdown</u> » 0: Normal « » 1: Powerdown «
3:2	RSV			<u>Reserved</u>
1	PDMD	0	R/W	<u>Powerdown mode</u> » 0: Normal « » 1: Auto powerdown mode « Note: When '0' internal crystal oscillators can be powered down by setting PWRDN pin high. When '1' internal crystal oscillator power down and moving to sleep status can be done by setting PWRDN pin high. Bring PWRDN to low to recover from sleep and oscillator powerdown.
0	OSC48	0	R/W	<u>M48XI crystal resonator enable</u> » 0: Enable « » 1: Disable «

5.10.18 OSEL

Output Clock Selector

OSEL	0003:08A2				offset: 0x22				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	O2SEL[3]	O2SEL[2]	O2SEL[1]	O2SEL[0]	O1SEL[3]	O1SEL[2]	O1SEL[1]	O1SEL[0]	O0SEL[3]	O0SEL[2]	O0SEL[1]	O0SEL[0]
R/W	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function											
15:12	RSV			Reserved											
11:8	02SEL	0	R/W	<u>General purpose clock output (GIO26, GIO34)</u> » 0000: ARM clock « » 0001: DSP clock « » 0010: SDR clock « » 0011: AXL clock « » 0100: PLLIN clock « » 0101: M48XI « » 0110: MS clock « » 0111: VLYNQ clock « » 1000: VENC clock « » 1001: SDR fast clock « » 1010: MS top clock « » 1011: VLYNQ top clock «											
				7:4	01SEL	0	R/W	<u>General purpose clock output (GIO25, GIO33)</u> » 0000: ARM clock « » 0001: DSP clock « » 0010: SDR clock « » 0011: AXL clock « » 0100: PLLIN clock « » 0101: M48XI « » 0110: MS clock « » 0111: VLYNQ clock « » 1000: VENC clock « » 1001: SDR fast clock « » 1010: MS top clock « » 1011: VLYNQ top clock «							
								3:0	00SEL	0	R/W	<u>General purpose clock output (GIO16)</u> » 0000: ARM clock « » 0001: DSP clock « » 0010: SDR clock « » 0011: AXL clock « » 0100: PLLIN clock « » 0101: M48XI « » 0110: MS clock « » 0111: VLYNQ clock « » 1000: VENC clock « » 1001: SDR fast clock « » 1010: MS top clock « » 1011: VLYNQ top clock «			

5.10.19 O0DIV

Output Clock #0 Divider

O0DIV	0003:08A4				offset: 0x24				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	O0DV[15]	O0DV[14]	O0DV[13]	O0DV[12]	O0DV[11]	O0DV[10]	O0DV[9]	O0DV[8]	O0DV[7]	O0DV[6]	O0DV[5]	O0DV[4]	O0DV[3]	O0DV[2]	O0DV[1]	O0DV[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:0	O0DV	0	R/W	Divisor = 2 x (Value +1) Note: Clock selected by O0SEL in OSEL will be divided

5.10.20 O1DIV

Output Clock #1 Divider

O1DIV	0003:08A6				offset: 0x26				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	O1DV[15]	O1DV[14]	O1DV[13]	O1DV[12]	O1DV[11]	O1DV[10]	O1DV[9]	O1DV[8]	O1DV[7]	O1DV[6]	O1DV[5]	O1DV[4]	O1DV[3]	O1DV[2]	O1DV[1]	O1DV[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:0	O1DV	0	R/W	Divisor = 2 x (Value +1) Note: Clock selected by O1SEL in OSEL will be divided

5.10.21 O2DIV

Output Clock #2 Divider

O2DIV	0003:08A8				offset: 0x28				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	O2DV[4]	O2DV[3]	O2DV[2]	O2DV[1]	O2DV[0]
R/W	-	-	-	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	-	-	-	-	-	-	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:5	RSV			Reserved
4:0	O2DV	0	R/W	Divisor = (Value +1) Note: Clock selected by O2SEL in OSEL will be divided

5.10.22 PWM0C

PWM #0 Cycle Count

PWM0C	0003:08AA			offset: 0x2A			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	WM0C[15]	WM0C[14]	WM0C[13]	WM0C[12]	WM0C[11]	WM0C[10]	WM0C[9]	WM0C[8]	WM0C[7]	WM0C[6]	WM0C[5]	WM0C[4]	WM0C[3]	WM0C[2]	WM0C[1]	WM0C[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:0	PWM0C	0	R/W	Cycle of PWM0 output Note: It is number of counts on PLL input clock

5.10.23 PWM0H

PWM #0 High Period

PWM0H	0003:08AC			offset: 0x2C			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	WM0H[15]	WM0H[14]	WM0H[13]	WM0H[12]	WM0H[11]	WM0H[10]	WM0H[9]	WM0H[8]	WM0H[7]	WM0H[6]	WM0H[5]	WM0H[4]	WM0H[3]	WM0H[2]	WM0H[1]	WM0H[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:0	PWM0H	0	R/W	High level period of PWM0 output Note: It is number of counts on PLL input clock

5.10.24 PWM1C

PWM #1 Cycle Count

PWM1C	0003:08AE			offset: 0x2E			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	WM1C[15]	WM1C[14]	WM1C[13]	WM1C[12]	WM1C[11]	WM1C[10]	WM1C[9]	WM1C[8]	WM1C[7]	WM1C[6]	WM1C[5]	WM1C[4]	WM1C[3]	WM1C[2]	WM1C[1]	WM1C[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:0	PWM1C	0	R/W	Cycle of PWM1 output Note: It is number of counts on PLL input clock

5.10.25 PWM1H

PWM #1 High Period

PWM1H	0003:08B0			offset: 0x30			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	WM1H[15]	WM1H[14]	WM1H[13]	WM1H[12]	WM1H[11]	WM1H[10]	WM1H[9]	WM1H[8]	WM1H[7]	WM1H[6]	WM1H[5]	WM1H[4]	WM1H[3]	WM1H[2]	WM1H[1]	WM1H[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:0	PWM1H	0	R/W	High level period of PWM1 output Note: It is number of counts on PLL input clock

6 Interrupt Controller

6.1 Introduction

The Interrupt Controller (INTC) has 45 interrupt channels - 29 internal peripheral interrupt channels, 16 external interrupt channels and a reserved interrupt.

Table 16 provides the interrupt source list.

The INTC has the following features:

- 45 interrupt channels (16 external channels)
- Interrupt mask for each channel
- Each interrupt channel can be mapped to a FIQ or IRQ type interrupt
- Configurable interrupt priority
- Configurable interrupt entry table, to reduce interrupt processing time

The ARM core supports two interrupt types: FIQ and IRQ. See the ARM926EJ Technical Reference Manual for detailed information about the ARM's FIQ and IRQ interrupts. Each interrupt channel can be mapped to a FIQ or IRQ type interrupt, and each channel can be enabled or disabled. The INTC supports user configurable interrupt-priority and interrupt entry address. Entry address minimizes the time spent for jumping to interrupt service routines (ISR). When an interrupt occurs the corresponding highest priority ISR's address is stored in the INTC's ENTRY register. The IRQ or FIQ interrupt routine can read the ENTRY register and jump to the corresponding ISR directly. Thus the ARM does not need to have a software dispatcher, to find out the asserted interrupt.

6.2 Interrupt Source List

Table 16 shows the various sources that can interrupt the ARM926, in the order of their default priorities. TMR0 has the default highest priority.

See respective sections for cause of each interrupt.

Table 16: Interrupt Source List

Interrupt ID	Tag	Name
0	INT_TMR0	Timer 0 Interrupt
1	INT_TMR1	Timer 1 Interrupt
2	INT_TMR2	Timer 2 Interrupt (CCD Timer 0)
3	INT_TMR3	Timer 3 Interrupt (CCD Timer 1)
4	INT_CCDVD0	CCD VD Interrupt #0
5	INT_CCDVD1	CCD VD Interrupt #1
6	INT_CCDWEN	CCD WEN Interrupt
7	INT_VENC	VENC Interrupt
8	INT_SER0	Serial I/F 0 Interrupt (with DMA)
9	INT_SER1	Serial I/F 1 Interrupt
10	INT_EXTHOST	External Host Interrupt
11	INT_IMGBUF	Image buffer
12	INT_UART0	UART 0 Interrupt
13	INT_UART1	UART 1 Interrupt
14	INT_USB0	USB Interrupt #0 (DMA)
15	INT_USB1	USB Interrupt #1 (Core)
16	INT_VLYNQ	VLYNQ interrupt
17	INT_MTC0	Memory Traffic Controller Interrupt #0 (DMA)
18	INT_MTC1	Memory Traffic Controller Interrupt #1 (CFC_RDY)
19	INT_SDMMCMS	SD / MMC or MS Host Interrupt
20	INT_SDIOMS	SDIO or MS DMA Interrupt
21	INT_GIO0	External Interrupt #0 (GIO0)
22	INT_GIO1	External Interrupt #1 (GIO1)
23	INT_GIO2	External Interrupt #2 (GIO2)
24	INT_GIO3	External Interrupt #3 (GIO3)
25	INT_GIO4	External Interrupt #4 (GIO4)
26	INT_GIO5	External Interrupt #5 (GIO5)
27	INT_GIO6	External Interrupt #6 (GIO6)
28	INT_GIO7	External Interrupt #7 (GIO7)
29	INT_GIO8	External Interrupt #8 (GIO8)
30	INT_GIO9	External Interrupt #9 (GIO9)
31	INT_GIO10	External Interrupt #10 (GIO10)
32	INT_GIO11	External Interrupt #11 (GIO11)
33	INT_GIO12	External Interrupt #12 (GIO12)
34	INT_GIO13	External Interrupt #13 (GIO13)
35	INT_GIO14	External Interrupt #14 (GIO14)
36	INT_GIO15	External Interrupt #15 (GIO15)
37	INT_PREV0	Preview Engine Interrupt #0 (Preview over)
38	INT_PREV1	Preview Engine Interrupt #1 (Preview histogram over)
39	INT_WDT	Watch Dog Timer Interrupt
40	INT_I2C	I2C Interrupt
41	INT_CLKC	Clock Controller Interrupt (Wake-up)
42	INT_E2ICE	Embedded ICE Interrupt
43	INT_ARMCOMRX	ARMCOMM Receive
44	INT_ARMCOMTX	ARMCOMM Transmit
45	INT_RSV	Reserved Interrupt

6.3 INTC Methodology

INTC methodology is described below and is illustrated in *Figure 16*.

- When an interrupt occurs the status is reflected in either the FIQn or IRQn registers depending upon the interrupt type selected.
- Interrupts are enabled or disabled (masked), by setting the EINTn register. Note: Even if an interrupt is masked, the status the interrupt is still reflected in the FIQn and IRQn registers.
- When an interrupt from any interrupt channel occurs, for which interrupt is enabled, IRQ or FIQ interrupt is generated to ARM926 core (depending on whether the interrupt channel is mapped to IRQ or FIQ interrupt). The ARM then branches to the IRQ or FIQ interrupt routine.
- The INTC generates the entry address of the pending interrupt with the highest priority and stores the entry address in the FIQENTRY or IRQENTRY register depending on whether the interrupt is mapped to IRQ or FIQ interrupt. The IRQ or FIQ ISR can then read the entry address and branch to the Interrupt service routine of the interrupt.

The entry address is equal to base address plus an offset, and the base address is set in the EABASEn registers. The offset is based on the priority of the interrupt. So if the priority of an interrupt is changed, the user should also change the offset for the interrupt in the entry table. Priority of the interrupt is configured in the INTPRIn registers.

All interrupt request signals from the peripherals need to be one system-clock wide pulse (active low), and they are stored into the registers of INTC once. To guarantee recognition of the interrupt request, the request signals must be synchronized with the rising edge of the system-clock.

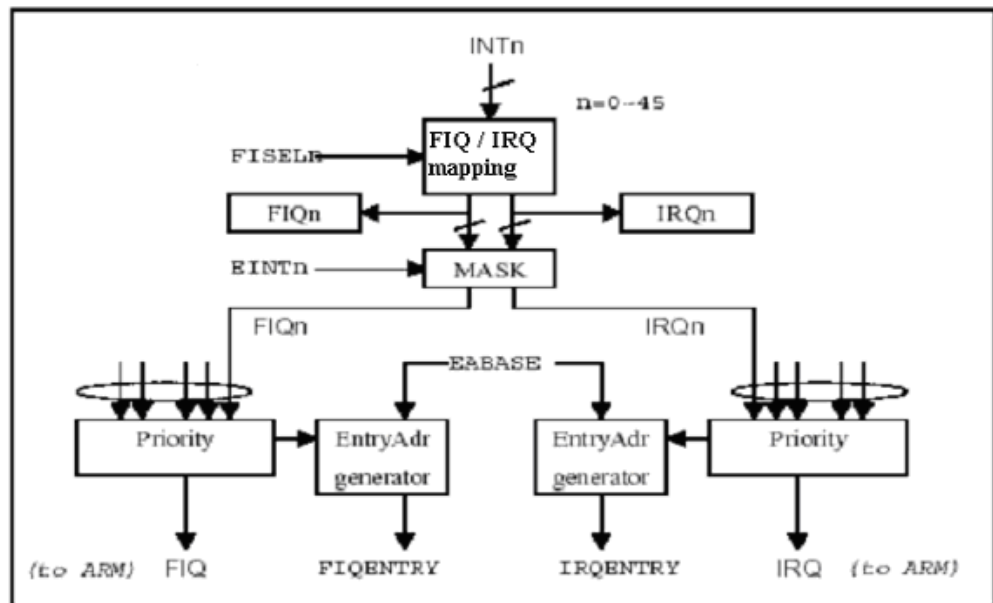


Figure 16: INTC Methodology

Table 17 summarizes the registers that are used to configure the interrupt controller. Refer to section on *Interrupt Controller Registers* for more details

Register	Description
FISELn	Select interrupt channel as FIQ or IRQ (See registers <i>FISEL0</i> , <i>FISEL1</i> and <i>FISEL2</i>)
EINTn	Enable or disable an interrupt channel Note: Even if an interrupt is masked (disabled), the status of each interrupt is reflected in the FIQn and IRQn registers (See <i>EINT0</i> , <i>EINT1</i> and <i>EINT2</i>)
EABASEn	Set base address and size of the entry table. (See <i>EABASE0</i> and <i>EABASE1</i>)
FIQENTRYn, IRQENTRYn,	Interrupt entry address of highest pending IRQ / FIQ interrupt (See <i>FIQENTRY0</i> , <i>FIQENTRY1</i> , <i>IRQENTRY0</i> and <i>IRQENTRY1</i>)
FIQENTLCKn, IRQENTLCKn	Locked interrupt entry address of highest pending IRQ / FIQ interrupt (See <i>FIQENTLCK0</i> , <i>FIQENTLCK1</i> , <i>IRQENTLCK0</i> and <i>IRQENTLCK1</i>)
INTRAW	Selects whether a masked interrupt is reflected in FIQENTRY or IRQENTRY (See INTRAW)
FIQn	Shows status of interrupts mapped to ARM's FIQ interrupt. Clear/acknowledge pending interrupt. When a bit in the register is read as '0', it indicates that the corresponding interrupt has occurred. '1', in a bit position indicates that the corresponding interrupt has NOT occurred. When the register is written, writing '0', in a bit position has no effect writing '1', in a bit position clears the pending interrupt (See <i>FIQ0</i> , <i>FIQ1</i> and <i>FIQ2</i>)

Register	Description
IRQn	Shows status of interrupts mapped to ARM's IRQ interrupt. Clear/acknowledge pending interrupt. When a bit in the register is read as '0', it indicates that the corresponding interrupt has occurred. '1', in a bit position indicates that the corresponding interrupt has NOT occurred. When the register is written, writing '0', in a bit position has no effect writing '1', in a bit position clears the pending interrupt (See <i>IRQ0</i> , <i>IRQ1</i> and <i>IRQ2</i>)
INTPRIn	Set priority for each interrupt channel. Highest priority interrupt entry address is reflected in FIQENTRYn or IRQENTRYn register. By default, Timer 0 (INT0) has priority and ARM COMMTX interrupt (INT44) has lowest priority (See interrupt priority registers INTPRI00 to INTPRI22)

Table 17: Interrupt Controller Register Summary

6.4 Configurable Interrupt Entry Table

Since a number of interrupts are mapped to IRQ or FIQ interrupt, the IRQ / FIQ interrupt handler has to find out the interrupt and jump to the corresponding ISR. The configurable interrupt entry table can be used to minimize the time spent in jumping to interrupt service routines.

FIQENTRYn/IRQENTRYn and **FIQENTLCKn/IRQENTLCKn** registers shows address of ISR for highest priority interrupt. Entry address is base address plus an offset, and the base address can be set by **EABASEn** register. Offset is decided by priority and size of each entry in the table. Size can be set as 4, 8, 16 or 32 byte.

The interrupt entry table is illustrated in **Figure 17**

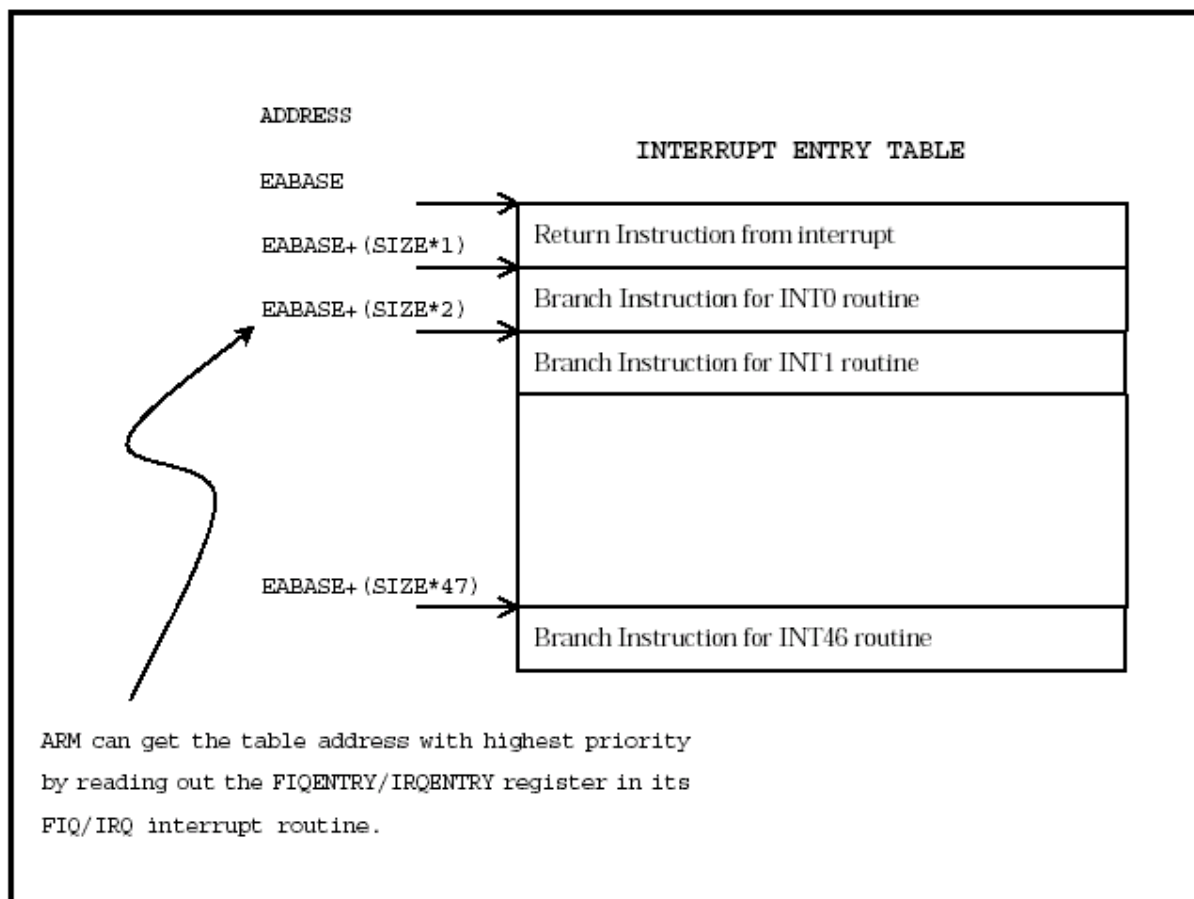


Figure 17: Interrupt Entry Table

$$\text{FIQ/IRQENTRY} = \text{EABASE} + (\text{SIZE} * \text{highest priority number})$$

The priority level of interrupt is programmable by setting INTPRIn register.

FISELn, EABASEn and INTPRIn registers should be set before occurrence of interrupt from any modules. If not, FIQENTRYn / IRQENTRYn registers does not guarantee correct values. It is recommended to enable INTC module by EINTn, only after all registers settings are done.

INTRAW register controls whether a masked interrupt is reflected in the FIQENTRYn / IRQENTRYn register or not. If this bit set to '1', FIQENTRYn / IRQENTRYn register will be updated even if the interrupt bit is masked.

When there is no interrupt, FIQENTRYn / IRQENTRYn return the content of EABASE. The structure of the entry table is shown in **Figure 17**.

Note: Priority number ranges from 1 to 45 and is termed as INT0 to INT45.

6.4.1 Initialization of the interrupt controller

- Clear FIQ and IRQ status registers
- Mask all interrupt channels by EINTn register
- Map each interrupt to FIQ or IRQ by FISELn register
- Setup the priority of each interrupt by priority configuration register (INTPRIn)
- Setup the interrupt entry table base register (EABASE)
- Enable the interrupts in interrupt controller (EINTn)
- Enable the FIQ and IRQ

6.4.2 Using the entry table to process interrupts

- ARM gets an FIQ/IRQ interrupt
- ARM jumps to common FIQ or IRQ vector (depending on whether an FIQ or IRQ type interrupt occurred). FIQ vector is at address 0x001C and IRQ vector is at address 0x0018
- IRQ / FIQ interrupt routine reads the FIQENTRYn / IRQENTRYn registers to get the interrupt service routine address, and then jumps to this address. This is the interrupt service routine corresponding to the highest priority interrupt among the interrupts occurred.
- The interrupt is cleared by setting the bit corresponding to the interrupt in FIQn / IRQn register
- Interrupt service routine is executed
- The control then returns to the IRQ / FIQ interrupt vector routine
- After this the state of program execution that existed prior to the interrupt handling, is restored.

6.5 External Interrupts

Interrupt IDs 21-36 (EXT0-EXT15) are external interrupts. These interrupts are shared with pins GIO0-15. To use any one of these interrupts, the shared GIO port must be setup as an interrupt port by setting the corresponding GIO registers (IRQPORT, IRQEDGE). Refer to the section on GIO for more information on configuring GIO port as interrupt ports.

The interrupt controller samples interrupt signals on the rising edge of the AHB clock. Therefore interrupt signal should be at least one AHB cycle long to be recognized.

6.6 Interrupt Controller Register Map (INTC)

Address	Register	Description
0003:0500	FIQ0	FIQ Interrupt Flag Register #0
0003:0502	FIQ1	FIQ Interrupt Flag Register #1
0003:0504	FIQ2	FIQ Interrupt Flag Register #2
0003:0508	RQ0	IRQ Interrupt Flag Register #0
0003:050A	RQ1	IRQ Interrupt Flag Register #1
0003:050C	RQ2	IRQ Interrupt Flag Register #2
0003:0510	FIQENTRY0	FIQ Entry Address Register #0
0003:0512	FIQENTRY1	FIQ Entry Address Register #1
0003:0514	FIQENTLCK0	FIQ Lock Entry Address Register #1
0003:0516	FIQENTLCK1	FIQ Lock Entry Address Register #1
0003:0518	RQENTRY0	IRQ Entry Address Register #0
0003:051A	RQENTRY1	IRQ Entry Address Register #1
0003:051C	RQENTLCK0	IRQ Lock Entry Address Register #1
0003:051E	RQENTLCK1	IRQ Lock Entry Address Register #1
0003:0520	FSEL0	FIQ select register #0
0003:0522	FSEL1	FIQ select register #1
0003:0524	FSEL2	FIQ select register #2
0003:0528	ENT0	Interrupt Enable Register #0
0003:052A	ENT1	Interrupt Enable Register #1
0003:052C	ENT2	Interrupt Enable Register #2
0003:0530	NTRAW	Interrupt Raw Register
0003:0538	EABASE0	Entry Table Base Address Register #0
0003:053A	EABASE1	Entry Table Base Address Register #1
0003:0540	NTPR0	Interrupt Priority Register #0
0003:0542	NTPR1	Interrupt Priority Register #1
0003:0544	NTPR2	Interrupt Priority Register #2
0003:0546	NTPR3	Interrupt Priority Register #3
0003:0548	NTPR4	Interrupt Priority Register #4
0003:054A	NTPR5	Interrupt Priority Register #5
0003:054C	NTPR6	Interrupt Priority Register #6
0003:054E	NTPR7	Interrupt Priority Register #7
0003:0550	NTPR8	Interrupt Priority Register #8
0003:0552	NTPR9	Interrupt Priority Register #9
0003:0554	NTPR10	Interrupt Priority Register #10
0003:0556	NTPR11	Interrupt Priority Register #11
0003:0558	NTPR12	Interrupt Priority Register #12
0003:055A	NTPR13	Interrupt Priority Register #13
0003:055C	NTPR14	Interrupt Priority Register #14
0003:055E	NTPR15	Interrupt Priority Register #15
0003:0560	NTPR16	Interrupt Priority Register #16
0003:0562	NTPR17	Interrupt Priority Register #17
0003:0564	NTPR18	Interrupt Priority Register #18
0003:0566	NTPR19	Interrupt Priority Register #19
0003:0568	NTPR20	Interrupt Priority Register #20
0003:056A	NTPR21	Interrupt Priority Register #21
0003:056C	NTPR22	Interrupt Priority Register #22

6.7 Interrupt Controller Registers

6.7.1 FIQ0

FIQ interrupt flag register #0

FIQ0	0003:0500			offset:	0x00	default: 0xFFFF										
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	USB1	USB0	UART1	UART0	IMGBUF	EXTHOST	SP1	SP0	VENC	CCDWEN	CCDVD1	CCDVD0	TMR3	TMR2	TMR1	TMR0
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

Bit	Name	Reset Value	R/W	Function
15	USB1	1	R/W	<u>USB1 Interrupt Flag (Generated by USB core)</u> » Rd 0: active « » Wr 1: cleared «
14	USB0	1	R/W	<u>USB0 Interrupt Flag (Generated by USB-DMA)</u> » Rd 0: active « » Wr 1: cleared «
13	UART1	1	R/W	<u>UART1 Interrupt Flag</u> » Rd 0: active « » Wr 1: cleared «
12	UART0	1	R/W	<u>UART0 Interrupt Flag</u> » Rd 0: active « » Wr 1: cleared «
11	IMGBUF	1	R/W	<u>Image Buffer Interrupt Flag</u> » Rd 0: active « » Wr 1: cleared «
10	EXTHOST	1	R/W	<u>External HOST Interrupt Flag</u> » Rd 0: active « » Wr 1: cleared «
9	SP1	1	R/W	<u>Serial Port 1 Interrupt Flag</u> » Rd 0: active « » Wr 1: cleared «
8	SP0	1	R/W	<u>Serial Port 0 Interrupt Flag</u> » Rd 0: active « » Wr 1: cleared «
7	VENC	1	R/W	<u>VENC Interrupt Flag (Falling edge of VSYNC)</u> » Rd 0: active « » Wr 1: cleared «
6	CCDWEN	1	R/W	<u>CCD WEN Interrupt (Falling edge of WEN port)</u> » Rd 0: active « » Wr 1: cleared «
5	CCDVD1	1	R/W	<u>CCD VD Interrupt #1 Flag</u> » Rd 0: active « » Wr 1: cleared «
4	CCDVD0	1	R/W	<u>CCD VD Interrupt #0 Flag</u> » Rd 0: active « » Wr 1: cleared «
3	TMR3	1	R/W	<u>Timer 3 Interrupt Flag</u> » Rd 0: active « » Wr 1: cleared «
2	TMR2	1	R/W	<u>Timer 2 Interrupt Flag</u> » Rd 0: active « » Wr 1: cleared «
1	TMR1	1	R/W	<u>Timer 1 Interrupt Flag</u> » Rd 0: active « » Wr 1: cleared «
0	TMR0	1	R/W	<u>Timer 0 Interrupt Flag</u> » Rd 0: active « » Wr 1: cleared «

6.7.2 FIQ1

FIQ interrupt flag register #1

FIQ1	0003:0502			offset: 0x02												default: 0xFFFF	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	EXT10	EXT9	EXT8	EXT7	EXT6	EXT5	EXT4	EXT3	EXT2	EXT1	EXT0	MMCSDFS	MMCSDFS	MTC1	MTC0	VLYNQ	
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	

Bit	Name	Reset Value	R/W	Function
15	EXT10	1	R/W	<u>External Interrupt #10 Flag</u> » Rd 0: active « » Wr 1: cleared «
14	EXT9	1	R/W	<u>External Interrupt #9 Flag</u> » Rd 0: active « » Wr 1: cleared «
13	EXT8	1	R/W	<u>External Interrupt #8 Flag</u> » Rd 0: active « » Wr 1: cleared «
12	EXT7	1	R/W	<u>External Interrupt #7 Flag</u> » Rd 0: active « » Wr 1: cleared «
11	EXT6	1	R/W	<u>External Interrupt #6 Flag</u> » Rd 0: active « » Wr 1: cleared «
10	EXT5	1	R/W	<u>External Interrupt #5 Flag</u> » Rd 0: active « » Wr 1: cleared «
9	EXT4	1	R/W	<u>External Interrupt #4 Flag</u> » Rd 0: active « » Wr 1: cleared «
8	EXT3	1	R/W	<u>External Interrupt #3 Flag</u> » Rd 0: active « » Wr 1: cleared «
7	EXT2	1	R/W	<u>External Interrupt #2 Flag</u> » Rd 0: active « » Wr 1: cleared «
6	EXT1	1	R/W	<u>External Interrupt #1 Flag</u> » Rd 0: active « » Wr 1: cleared «
5	EXT0	1	R/W	<u>External Interrupt #0 Flag</u> » Rd 0: active « » Wr 1: cleared «
4	MMCSDFS1	1	R/W	<u>SDIO / MS DMA Interrupt Flag</u> » Rd 0: active « » Wr 1: cleared «
3	MMCSDFS0	1	R/W	<u>MMCSDFS / MS Host Interrupt Flag</u> » Rd 0: active « » Wr 1: cleared «
2	MTC1	1	R/W	<u>Memory Traffic Controller CFC_RDY Interrupt Flag</u> » Rd 0: active « » Wr 1: cleared « Note: CF_RDY terminal transmit interrupt
1	MTC0	1	R/W	<u>Memory Traffic Controller DMA Interrupt Flag</u> » Rd 0: active « » Wr 1: cleared « Note: MTC DMA interrupt
0	VLYNQ	1	R/W	<u>VLYNQ Interrupt Flag</u> » Rd 0: active « » Wr 1: cleared «

6.7.3 FIQ2

FIQ interrupt flag register #2

FIQ2	0003:0504			offset: 0x04			default: 0x3FFF									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSVINT	ARMCMTX	ARMCOMRX	PREV1	PREV0	EXT15	EXT14	EXT13	EXT12	EXT11	0	EXT13	EXT12	EXT11
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	1	1	1	1	1	1	1	1	1	1	1	1	1	1

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
13	RSVINT	1	R/W	Reserved Interrupt Do not use
12	ARMCMTX	1	R/W	ARM COMM TX Interrupt Flag » Rd 0: active « » Wr 1: cleared «
11	ARMCOMRX	1	R/W	ARM COMM RX Interrupt Flag » Rd 0: active « » Wr 1: cleared «
10	E2IE	1	R/W	E2ICE Interrupt Flag » Rd 0: active « » Wr 1: cleared «
9	CLKC	1	R/W	Clock controller Interrupt Flag » Rd 0: active « » Wr 1: cleared « Note: Wake-up interrupt
8	I2C	1	R/W	I2C Interrupt Flag » Rd 0: active « » Wr 1: cleared «
7	WDT	1	R/W	Watch dog timer Interrupt Flag » Rd 0: active « » Wr 1: cleared «
6	PREV1	1	R/W	Preview 1 Interrupt Flag » Rd 0: active « » Wr 1: cleared « Note: Preview Histogram is completed
5	PREV0	1	R/W	Preview 0 Interrupt Flag » Rd 0: active « » Wr 1: cleared « Note: Preview is completed
4	EXT15	1	R/W	External Interrupt #15 Flag » Rd 0: active « » Wr 1: cleared «
3	EXT14	1	R/W	External Interrupt #14 Flag » Rd 0: active « » Wr 1: cleared «
2	EXT13	1	R/W	External Interrupt #13 Flag » Rd 0: active « » Wr 1: cleared «
1	EXT12	1	R/W	External Interrupt #12 Flag » Rd 0: active « » Wr 1: cleared «
0	EXT11	1	R/W	External Interrupt #11 Flag » Rd 0: active « » Wr 1: cleared «

6.7.4 IRQ0

IRQ interrupt flag register #0

IRQ0	0003:0508			offset: 0x08			default: 0xFFFF									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	USB1	USB0	UART1	UART0	IMGBUF	EXTHOST	SP1	SP0	VENC	CCDWEN	CCDVD1	CCDVD0	TMR3	TMR2	TMR1	TMR0
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

Bit	Name	Reset Value	R/W	Function
15	USB1	1	R/W	<u>USB1 Interrupt Flag (Generated by USB core)</u> » Rd 0: active « » Wr 1: cleared «
14	USB0	1	R/W	<u>USB0 Interrupt Flag (Generated by USB-DMA)</u> » Rd 0: active « » Wr 1: cleared «
13	UART1	1	R/W	<u>UART1 Interrupt Flag</u> » Rd 0: active « » Wr 1: cleared «
12	UART0	1	R/W	<u>UART0 Interrupt Flag</u> » Rd 0: active « » Wr 1: cleared «
11	IMGBUF	1	R/W	<u>Image Buffer Interrupt Flag</u> » Rd 0: active « » Wr 1: cleared «
10	EXTHOST	1	R/W	<u>External HOST Interrupt Flag</u> » Rd 0: active « » Wr 1: cleared «
9	SP1	1	R/W	<u>Serial Port 1 Interrupt Flag</u> » Rd 0: active « » Wr 1: cleared «
8	SP0	1	R/W	<u>Serial Port 0 Interrupt Flag</u> » Rd 0: active « » Wr 1: cleared «
7	VENC	1	R/W	<u>VENC Interrupt Flag (Falling edge of VSYNC)</u> » Rd 0: active « » Wr 1: cleared «
6	CCDWEN	1	R/W	<u>CCD WEN Interrupt (Falling edge of WEN port)</u> » Rd 0: active « » Wr 1: cleared «
5	CCDVD1	1	R/W	<u>CCD VD Interrupt #1 Flag</u> » Rd 0: active « » Wr 1: cleared «
4	CCDVD0	1	R/W	<u>CCD VD Interrupt #0 Flag</u> » Rd 0: active « » Wr 1: cleared «
3	TMR3	1	R/W	<u>Timer 3 Interrupt Flag</u> » Rd 0: active « » Wr 1: cleared «
2	TMR2	1	R/W	<u>Timer 2 Interrupt Flag</u> » Rd 0: active « » Wr 1: cleared «
1	TMR1	1	R/W	<u>Timer 1 Interrupt Flag</u> » Rd 0: active « » Wr 1: cleared «
0	TMR0	1	R/W	<u>Timer 0 Interrupt Flag</u> » Rd 0: active « » Wr 1: cleared «

6.7.5 IRQ1

IRQ interrupt flag register #1

IRQ1	0003:050A				offset: 0x0A				default: 0xFFFF							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	EXT10	EXT9	EXT8	EXT7	EXT6	EXT5	EXT4	EXT3	EXT2	EXT1	EXT0	MMCSDFS	MMCSDFS	MTC1	MTC0	VLNQ
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

Bit	Name	Reset Value	R/W	Function
15	EXT10	1	R/W	<u>External Interrupt #10 Flag</u> » Rd 0: active « » Wr 1: cleared «
14	EXT9	1	R/W	<u>External Interrupt #9 Flag</u> » Rd 0: active « » Wr 1: cleared «
13	EXT8	1	R/W	<u>External Interrupt #8 Flag</u> » Rd 0: active « » Wr 1: cleared «
12	EXT7	1	R/W	<u>External Interrupt #7 Flag</u> » Rd 0: active « » Wr 1: cleared «
11	EXT6	1	R/W	<u>External Interrupt #6 Flag</u> » Rd 0: active « » Wr 1: cleared «
10	EXT5	1	R/W	<u>External Interrupt #5 Flag</u> » Rd 0: active « » Wr 1: cleared «
9	EXT4	1	R/W	<u>External Interrupt #4 Flag</u> » Rd 0: active « » Wr 1: cleared «
8	EXT3	1	R/W	<u>External Interrupt #3 Flag</u> » Rd 0: active « » Wr 1: cleared «
7	EXT2	1	R/W	<u>External Interrupt #2 Flag</u> » Rd 0: active « » Wr 1: cleared «
6	EXT1	1	R/W	<u>External Interrupt #1 Flag</u> » Rd 0: active « » Wr 1: cleared «
5	EXT0	1	R/W	<u>External Interrupt #0 Flag</u> » Rd 0: active « » Wr 1: cleared «
4	MMCSDFS1	1	R/W	<u>SDIO / MS DMA Interrupt Flag</u> » Rd 0: active « » Wr 1: cleared «
3	MMCSDFS0	1	R/W	<u>MMCSDFS / MS Host Interrupt Flag</u> » Rd 0: active « » Wr 1: cleared «
2	MTC1	1	R/W	<u>Memory Traffic Controller CF_RDY Interrupt Flag</u> » Rd 0: active « » Wr 1: cleared « Note: CF_RDY terminal transmit interrupt
1	MTC0	1	R/W	<u>Memory Traffic Controller DMA Interrupt Flag</u> » Rd 0: active « » Wr 1: cleared « Note: MTC DMA interrupt
0	VLNQ	1	R/W	<u>VLNQ Interrupt Flag</u> » Rd 0: active « » Wr 1: cleared «

6.7.6 IRQ2

IRQ interrupt flag register #2

IRQ2	0003:050C			offset: 0x0C			default: 0x3FFF									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSVINT	ARMCOMTX	ARMCOMRX	E2ICE	CLKC	I2C	WDT	PREV1	PREV0	EXT15	EXT14	EXT13	EXT12	EXT11
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	1	1	1	1	1	1	1	1	1	1	1	1	1	1

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
13	RSVINT	1	R/W	<u>Reserved Interrupt</u> Do not use
12	ARMCOMTX	1	R/W	<u>ARM COMM TX Interrupt Flag</u> » Rd 0: active « » Wr 1: cleared «
11	ARMCOMRX	1	R/W	<u>ARM COMM RX Interrupt Flag</u> » Rd 0: active « » Wr 1: cleared «
10	E2ICE	1	R/W	<u>E2ICE Interrupt Flag</u> » Rd 0: active « » Wr 1: cleared «
9	CLKC	1	R/W	<u>Clock controller Interrupt Flag</u> » Rd 0: active « » Wr 1: cleared « Note: Wake-up interrupt
8	I2C	1	R/W	<u>I2C Interrupt Flag</u> » Rd 0: active « » Wr 1: cleared «
7	WDT	1	R/W	<u>Watch dog timer Interrupt Flag</u> » Rd 0: active « » Wr 1: cleared «
6	PREV1	1	R/W	<u>Preview 1 Interrupt Flag</u> » Rd 0: active « » Wr 1: cleared « Note: Preview Histogram is completed
5	PREV0	1	R/W	<u>Preview 0 Interrupt Flag</u> » Rd 0: active « » Wr 1: cleared « Note: Preview is completed
4	EXT15	1	R/W	<u>External Interrupt #15 Flag</u> » Rd 0: active « » Wr 1: cleared «
3	EXT14	1	R/W	<u>External Interrupt #14 Flag</u> » Rd 0: active « » Wr 1: cleared «
2	EXT13	1	R/W	<u>External Interrupt #13 Flag</u> » Rd 0: active « » Wr 1: cleared «
1	EXT12	1	R/W	<u>External Interrupt #12 Flag</u> » Rd 0: active « » Wr 1: cleared «
0	EXT11	1	R/W	<u>External Interrupt #11 Flag</u> » Rd 0: active « » Wr 1: cleared «

6.7.7 FIQENTRY0

FIQ entry address register #0

[illegible][illegible]

6.7.8 FIQENTRY1

FIQ entry address register #1

[illegible][illegible]

6.7.9 FIQENTLCK0

FIQ entry address lock register #0

FIQENTLCK0	0003:0514				offset: 0x14				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	FIQEL0[15]	FIQEL0[14]	FIQEL0[13]	FIQEL0[12]	FIQEL0[11]	FIQEL0[10]	FIQEL0[9]	FIQEL0[8]	FIQEL0[7]	FIQEL0[6]	FIQEL0[5]	FIQEL0[4]	FIQEL0[3]	FIQEL0[2]	FIQEL0[1]	FIQEL0[0]
R/W	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	c	R/W	Function
15:0	FIQEL0	0	R	FIQ Locked Entry Address #0 Low 16-bits of the FIQ entry address of the interrupt service routine whose priority level is the highest in valid/effective FIQ interrupt. When read FIQENTLCK0 and FIQENTLCK1 are locked, read again to release the lock

6.7.10 FIQENTLCK1

FIQ entry address lock register #1

[illegible][illegible]

6.7.11 IRQENTRY0

IRQ entry address register #0

[illegible][illegible]

6.7.13 IRQENTLCK0

IRQ entry address lock register #0

IRQENTLCK0	0003:051C			offset: 0x1C													default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	IRQEL0[15]	IRQEL0[14]	IRQEL0[13]	IRQEL0[12]	IRQEL0[11]	IRQEL0[10]	IRQEL0[9]	IRQEL0[8]	IRQEL0[7]	IRQEL0[6]	IRQEL0[5]	IRQEL0[4]	IRQEL0[3]	IRQEL0[2]	IRQEL0[1]	IRQEL0[0]		
R/W	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R		
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		

Bit	Name	Reset Value	R/W	Function
15:0	IRQEL0	0	R	IRQ Locked Entry Address #0 Low 16-bits of the IRQ entry address of the interrupt service routine whose priority level is the highest in valid/effective IRQ interrupt. When read IRQENTLCK0 and IRQENTLCK1 are locked, read again to release the lock

6.7.15 FISELO

FIQ select register #0

FISELO	0003:0520			offset: 0x20			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	USB1	USB0	UART1	UART0	IMGBUF	EXTHOST	SP1	SP0	VENC	CCDWEN	CCDVD1	CCDVD0	TMR3	TMR2	TMR1	TMR0
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15	USB1	0	R/W	<u>USB1 Interrupt Flag (Generated by USB core)</u> » 0: IRQ « » 1: FIQ «
14	USB0	0	R/W	<u>USB0 Interrupt Flag (Generated by USB-DMA)</u> » 0: IRQ « » 1: FIQ «
13	UART1	0	R/W	<u>UART1 Interrupt Flag</u> » 0: IRQ « » 1: FIQ «
12	UART0	0	R/W	<u>UART0 Interrupt Flag</u> » 0: IRQ « » 1: FIQ «
11	IMGBUF	0	R/W	<u>Image Buffer Interrupt Flag</u> » 0: IRQ « » 1: FIQ «
10	EXTHOST	0	R/W	<u>External HOST Interrupt Flag</u> » 0: IRQ « » 1: FIQ «
9	SP1	0	R/W	<u>Serial Port 1 Interrupt Flag</u> » 0: IRQ « » 1: FIQ «
8	SP0	0	R/W	<u>Serial Port 0 Interrupt Flag</u> » 0: IRQ « » 1: FIQ «
7	VENC	0	R/W	<u>VENC Interrupt Flag (Falling edge of VSYNC)</u> » 0: IRQ « » 1: FIQ «
6	CCDWEN	0	R/W	<u>CCD WEN Interrupt (Falling edge of WEN port)</u> » 0: IRQ « » 1: FIQ «
5	CCDVD1	0	R/W	<u>CCD VD Interrupt #1 Flag</u> » 0: IRQ « » 1: FIQ «
4	CCDVD0	0	R/W	<u>CCD VD Interrupt #0 Flag</u> » 0: IRQ « » 1: FIQ «
3	TMR3	0	R/W	<u>Timer 3 Interrupt Flag</u> » 0: IRQ « » 1: FIQ «
2	TMR2	0	R/W	<u>Timer 2 Interrupt Flag</u> » 0: IRQ « » 1: FIQ «
1	TMR1	0	R/W	<u>Timer 1 Interrupt Flag</u> » 0: IRQ « » 1: FIQ «
0	TMR0	0	R/W	<u>Timer 0 Interrupt Flag</u> » 0: IRQ « » 1: FIQ «

6.7.16 FISEL1

FIQ select register #1

FISEL1	0003:0522			offset:	0x22		default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	EXT10	EXT9	EXT8	EXT7	EXT6	EXT5	EXT4	EXT3	EXT2	EXT1	EXT0	MMCS1DMS	MMCS0DMS	MTC1	MTC0	VLYNQ
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15	EXT10	0	R/W	<u>External Interrupt #10 Flag</u> » 0: IRQ « » 1: FIQ «
14	EXT9	0	R/W	<u>External Interrupt #9 Flag</u> » 0: IRQ « » 1: FIQ «
13	EXT8	0	R/W	<u>External Interrupt #8 Flag</u> » 0: IRQ « » 1: FIQ «
12	EXT7	0	R/W	<u>External Interrupt #7 Flag</u> » 0: IRQ « » 1: FIQ «
11	EXT6	0	R/W	<u>External Interrupt #6 Flag</u> » 0: IRQ « » 1: FIQ «
10	EXT5	0	R/W	<u>External Interrupt #5 Flag</u> » 0: IRQ « » 1: FIQ «
9	EXT4	0	R/W	<u>External Interrupt #4 Flag</u> » 0: IRQ « » 1: FIQ «
8	EXT3	0	R/W	<u>External Interrupt #3 Flag</u> » 0: IRQ « » 1: FIQ «
7	EXT2	0	R/W	<u>External Interrupt #2 Flag</u> » 0: IRQ « » 1: FIQ «
6	EXT1	0	R/W	<u>External Interrupt #1 Flag</u> » 0: IRQ « » 1: FIQ «
5	EXT0	0	R/W	<u>External Interrupt #0 Flag</u> » 0: IRQ « » 1: FIQ «
4	MMCS1DMS	0	R/W	<u>SDIO / MS DMA Interrupt Flag</u> » 0: IRQ « » 1: FIQ «
3	MMCS0DMS	0	R/W	<u>MMCS0 / MS Host Interrupt Flag</u> » 0: IRQ « » 1: FIQ «
2	MTC1	0	R/W	<u>Memory Traffic Controller CF_RDY Interrupt Flag</u> » 0: IRQ « » 1: FIQ « Note: CF_RDY terminal transmit interrupt
1	MTC0	0	R/W	<u>Memory Traffic Controller DMA Interrupt Flag</u> » 0: IRQ « » 1: FIQ « Note: MTC DMA interrupt
0	VLYNQ	0	R/W	<u>VLYNQ Interrupt Flag</u> » 0: IRQ « » 1: FIQ «

6.7.17 FISEL2

FIQ select register #2

FISEL2	0003:0524			offset: 0x24								default: 0x0000				
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSVINT	ARMCMTX	ARMCOMRX	E2ICE	CLKC	I2C	WDT	PREV1	PREV0	EXT15	EXT14	EXT13	EXT12	EXT11
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
13	RSVINT	0	R/W	Reserved Interrupt Do not use
12	ARMCMTX	0	R/W	ARM COMM TX Interrupt Flag » 0: IRQ « » 1: FIQ «
11	ARMCOMRX	0	R/W	ARM COMM RX Interrupt Flag » 0: IRQ « » 1: FIQ «
10	E2ICE	0	R/W	E2ICE Interrupt Flag » 0: IRQ « » 1: FIQ «
9	CLKC	0	R/W	Clock controller Interrupt Flag » 0: IRQ « » 1: FIQ « Note: Wake-up interrupt
8	I2C	0	R/W	I2C Interrupt Flag » 0: IRQ « » 1: FIQ «
7	WDT	0	R/W	Watch dog timer Interrupt Flag » 0: IRQ « » 1: FIQ «
6	PREV1	0	R/W	Preview 1 Interrupt Flag » 0: IRQ « » 1: FIQ « Note: Preview Histogram is completed
5	PREV0	0	R/W	Preview 0 Interrupt Flag » 0: IRQ « » 1: FIQ « Note: Preview is completed
4	EXT15	0	R/W	External Interrupt #15 Flag » 0: IRQ « » 1: FIQ «
3	EXT14	0	R/W	External Interrupt #14 Flag » 0: IRQ « » 1: FIQ «
2	EXT13	0	R/W	External Interrupt #13 Flag » 0: IRQ « » 1: FIQ «
1	EXT12	0	R/W	External Interrupt #12 Flag » 0: IRQ « » 1: FIQ «
0	EXT11	0	R/W	External Interrupt #11 Flag » 0: IRQ « » 1: FIQ «

6.7.18 EINT0

Interrupt enable register #0

EINT0	0003:0528			offset: 0x28		default: 0x0000										
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	USB1	USB0	UART1	UART0	IMGBUF	EXTHOST	SP1	SP0	VENC	CCDWEN	CCDVD1	CCDVD0	TMR3	TMR2	TMR1	TMR0
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15	USB1	0	R/W	<u>USB1 Interrupt Flag (Generated by USB core)</u> » 0: Disable « » 1: Enable «
14	USB0	0	R/W	<u>USB0 Interrupt Flag (Generated by USB-DMA)</u> » 0: Disable « » 1: Enable «
13	UART1	0	R/W	<u>UART1 Interrupt Flag</u> » 0: Disable « » 1: Enable «
12	UART0	0	R/W	<u>UART0 Interrupt Flag</u> » 0: Disable « » 1: Enable «
11	IMGBUF	0	R/W	<u>Image Buffer Interrupt Flag</u> » 0: Disable « » 1: Enable «
10	EXTHOST	0	R/W	<u>External HOST Interrupt Flag</u> » 0: Disable « » 1: Enable «
9	SP1	0	R/W	<u>Serial Port 1 Interrupt Flag</u> » 0: Disable « » 1: Enable «
8	SP0	0	R/W	<u>Serial Port 0 Interrupt Flag</u> » 0: Disable « » 1: Enable «
7	VENC	0	R/W	<u>VENC Interrupt Flag (Falling edge of VSYNC)</u> » 0: Disable « » 1: Enable «
6	CCDWEN	0	R/W	<u>CCD WEN Interrupt (Falling edge of WEN port)</u> » 0: Disable « » 1: Enable «
5	CCDVD1	0	R/W	<u>CCD VD Interrupt #1 Flag</u> » 0: Disable « » 1: Enable «
4	CCDVD0	0	R/W	<u>CCD VD Interrupt #0 Flag</u> » 0: Disable « » 1: Enable «
3	TMR3	0	R/W	<u>Timer 3 Interrupt Flag</u> » 0: Disable « » 1: Enable «
2	TMR2	0	R/W	<u>Timer 2 Interrupt Flag</u> » 0: Disable « » 1: Enable «
1	TMR1	0	R/W	<u>Timer 1 Interrupt Flag</u> » 0: Disable « » 1: Enable «
0	TMR0	0	R/W	<u>Timer 0 Interrupt Flag</u> » 0: Disable « » 1: Enable «

6.7.19 EINT1

Interrupt enable register #1

EINT1	0003:052A			offset: 0x2A			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	EXT10	EXT9	EXT8	EXT7	EXT6	EXT5	EXT4	EXT3	EXT2	EXT1	EXT0	MMCSDFS	MMCSDFS	MTC1	MTC0	VLYNQ
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15	EXT10	0	R/W	<u>External Interrupt #10 Flag</u> » 0: Disable « » 1: Enable «
14	EXT9	0	R/W	<u>External Interrupt #9 Flag</u> » 0: Disable « » 1: Enable «
13	EXT8	0	R/W	<u>External Interrupt #8 Flag</u> » 0: Disable « » 1: Enable «
12	EXT7	0	R/W	<u>External Interrupt #7 Flag</u> » 0: Disable « » 1: Enable «
11	EXT6	0	R/W	<u>External Interrupt #6 Flag</u> » 0: Disable « » 1: Enable «
10	EXT5	0	R/W	<u>External Interrupt #5 Flag</u> » 0: Disable « » 1: Enable «
9	EXT4	0	R/W	<u>External Interrupt #4 Flag</u> » 0: Disable « » 1: Enable «
8	EXT3	0	R/W	<u>External Interrupt #3 Flag</u> » 0: Disable « » 1: Enable «
7	EXT2	0	R/W	<u>External Interrupt #2 Flag</u> » 0: Disable « » 1: Enable «
6	EXT1	0	R/W	<u>External Interrupt #1 Flag</u> » 0: Disable « » 1: Enable «
5	EXT0	0	R/W	<u>External Interrupt #0 Flag</u> » 0: Disable « » 1: Enable «
4	MMCSDFS1	0	R/W	<u>SD1 / MS DMA Interrupt Flag</u> » 0: Disable « » 1: Enable «
3	MMCSDFS0	0	R/W	<u>MMCSDFS / MS Host Interrupt Flag</u> » 0: Disable « » 1: Enable «
2	MTC1	0	R/W	<u>Memory Traffic Controller CF_RDY Interrupt Flag</u> » 0: Disable « » 1: Enable « Note: CF_RDY terminal transmit interrupt
1	MTC0	0	R/W	<u>Memory Traffic Controller DMA Interrupt Flag</u> » Rd 0: Disable « » Wr 1: Enable « Note: MTC DMA interrupt
0	VLYNQ	0	R/W	<u>VLYNQ Interrupt Flag</u> » 0: Disable « » 1: Enable «

6.7.20 EINT2

Interrupt enable register #2

EINT2	0003:052C			offset: 0x2C												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSVINT	ARMCOMTX	ARMCOMRX	E2ICE	CLKC	I2C	WDT	PREV1	PREV0	EXT15	EXT14	EXT13	EXT12	EXT11	
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	-	-	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
13	RSVINT	0	R/W	Reserved Interrupt Do not use
12	ARMCOMTX	0	R/W	ARM COMM TX Interrupt Flag » 0: Disable « » 1: Enable «
11	ARMCOMRX	0	R/W	ARM COMM RX Interrupt Flag » 0: Disable « » 1: Enable «
10	E2ICE	0	R/W	E2ICE Interrupt Flag » 0: Disable « » 1: Enable «
9	CLKC	0	R/W	Clock controller Interrupt Flag » 0: Disable « » 1: Enable « Note: Wake-up interrupt
8	I2C	0	R/W	I2C Interrupt Flag » 0: Disable « » 1: Enable «
7	WDT	0	R/W	Watch dog timer Interrupt Flag » 0: Disable « » 1: Enable «
6	PREV1	0	R/W	Preview 1 Interrupt Flag » 0: Disable « » 1: Enable « Note: Preview Histogram is completed
5	PREV0	0	R/W	Preview 0 Interrupt Flag » 0: Disable « » 1: Enable « Note: Preview is completed
4	EXT15	0	R/W	External Interrupt #15 Flag » 0: Disable « » 1: Enable «
3	EXT14	0	R/W	External Interrupt #14 Flag » 0: Disable « » 1: Enable «
2	EXT13	0	R/W	External Interrupt #13 Flag » 0: Disable « » 1: Enable «
1	EXT12	0	R/W	External Interrupt #12 Flag » 0: Disable « » 1: Enable «
0	EXT11	0	R/W	External Interrupt #11 Flag » 0: Disable « » 1: Enable «

6.7.21 INTRAW

Interrupt raw register

INTRAW	0003:0530			offset: 0x30												default: 0x0000		
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	DISSEL	IRAW1	IRAW0		
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W	R/W	R/W		
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	0	0	0		

Bit	Name	Reset Value	R/W	Function
15:3	RSV			Reserved
2	DISSEL	0	R/W	<u>Disable interrupt type</u> » 0: Interrupt can be disabled any time « » 1: Interrupt cannot be disabled after it occurs, unless cleared «
1	IRAW1	0	R/W	Set to see if the interrupt masked in IRQENTRY1, IRQENTRY0 register is reflected or not. » 0: Does not reflect « » 1: Reflects «
0	IRAW0	0	R/W	Set to see if the interrupt masked in the FIQENTRY0, FIQENTRY1 register is reflected or not. » 0: Does not reflect « » 1: Reflects «

6.7.24 INTPRI00

Interrupt priority register #0

INTPRI00	0003:0540			offset: 0x40			default: 0x0100									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	PRI01[5]	PRI01[4]	PRI01[3]	PRI01[2]	PRI01[1]	PRI01[0]	RSV	RSV	PRI00[5]	PRI00[4]	PRI00[3]	PRI00[2]	PRI00[1]	PRI00[0]
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	0	0	0	0	0	1	-	-	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
13:8	PRI01	1	R/W	Interrupt Factor with 2nd highest priority
7:6	RSV			Reserved
5:0	PRI00	0	R/W	Interrupt Factor with highest priority

6.7.25 INTPRI01

Interrupt priority register #1

INTPRI01	0003:0542			offset: 0x42												default: 0x0302	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	PRI03[5]	PRI03[4]	PRI03[3]	PRI03[2]	PRI03[1]	PRI03[0]	RSV	RSV	PRI02[5]	PRI02[4]	PRI02[3]	PRI02[2]	PRI02[1]	PRI02[0]	
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	
Default	-	-	0	0	0	0	1	1	-	-	0	0	0	0	1	0	

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
13:8	PRI03	3	R/W	Interrupt Factor with 4th highest priority
7:6	RSV			Reserved
5:0	PRI02	2	R/W	Interrupt Factor with 3rd highest priority

6.7.26 INTPRI02

Interrupt priority register #2

INTPRI02	0003:0544				offset: 0x44												default: 0x0504	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	PRI05[5]	PRI05[4]	PRI05[3]	PRI05[2]	PRI05[1]	PRI05[0]	RSV	RSV	PRI04[5]	PRI04[4]	PRI04[3]	PRI04[2]	PRI04[1]	PRI04[0]		
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W		
Default	-	-	0	0	0	1	0	1	-	-	0	0	0	1	0	0		

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
13:8	PRI05	5	R/W	Interrupt Factor with 6th highest priority
7:6	RSV			Reserved
5:0	PRI04	4	R/W	Interrupt Factor with 5th highest priority

6.7.27 INTPRI03

Interrupt priority register #3

INTPRI03	0003:0546			offset: 0x46												default: 0x0706		
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	PRI07[5]	PRI07[4]	PRI07[3]	PRI07[2]	PRI07[1]	PRI07[0]	RSV	RSV	PRI06[5]	PRI06[4]	PRI06[3]	PRI06[2]	PRI06[1]	PRI06[0]		
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W		
Default	-	-	0	0	0	1	1	1	-	-	0	0	0	1	1	0		

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
13:8	PRI07	7	R/W	Interrupt Factor with 8th highest priority
7:6	RSV			Reserved
5:0	PRI06	6	R/W	Interrupt Factor with 7th highest priority

6.7.28 INTPRI04

Interrupt priority register #4

INTPRI04	0003:0548			offset: 0x48													default: 0x0908	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	PRI09[5]	PRI09[4]	PRI09[3]	PRI09[2]	PRI09[1]	PRI09[0]	RSV	RSV	PRI08[5]	PRI08[4]	PRI08[3]	PRI08[2]	PRI08[1]	PRI08[0]		
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W		
Default	-	-	0	0	1	0	0	1	-	-	0	0	1	0	0	0		

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
13:8	PRI09	9	R/W	Interrupt Factor with 10th highest priority
7:6	RSV			Reserved
5:0	PRI08	8	R/W	Interrupt Factor with 9th highest priority

6.7.29 INTPRI05

Interrupt priority register #5

INTPRI05	0003:054A				offset: 0x4A				default: 0x0B0A							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	PRI11[5]	PRI11[4]	PRI11[3]	PRI11[2]	PRI11[1]	PRI11[0]	RSV	RSV	PRI10[5]	PRI10[4]	PRI10[3]	PRI10[2]	PRI10[1]	PRI10[0]
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	0	0	1	0	1	1	-	-	0	0	1	0	1	0

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
13:8	PRI11	11	R/W	Interrupt Factor with 12th highest priority
7:6	RSV			Reserved
5:0	PRI10	10	R/W	Interrupt Factor with 11th highest priority

6.7.30 INTPRI06

Interrupt priority register #6

INTPRI06	0003:054C			offset: 0x4C													default: 0x0D0C	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	PRI13[5]	PRI13[4]	PRI13[3]	PRI13[2]	PRI13[1]	PRI13[0]	RSV	RSV	PRI12[5]	PRI12[4]	PRI12[3]	PRI12[2]	PRI12[1]	PRI12[0]		
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W		
Default	-	-	0	0	1	1	0	1	-	-	0	0	1	1	0	0		

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
13:8	PRI13	13	R/W	Interrupt Factor with 14th highest priority
7:6	RSV			Reserved
5:0	PRI12	12	R/W	Interrupt Factor with 13th highest priority

6.7.31 INTPRI07

Interrupt priority register #7

INTPRI07	0003:054E			offset: 0x4E			default: 0x0F0E									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	PRI15[5]	PRI15[4]	PRI15[3]	PRI15[2]	PRI15[1]	PRI15[0]	RSV	RSV	PRI14[5]	PRI14[4]	PRI14[3]	PRI14[2]	PRI14[1]	PRI14[0]
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	0	0	1	1	1	1	-	-	0	0	1	1	1	0

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
13:8	PRI15	15	R/W	Interrupt Factor with 16th highest priority
7:6	RSV			Reserved
5:0	PRI14	14	R/W	Interrupt Factor with 15th highest priority

6.7.32 INTPRI08

Interrupt priority register #8

INTPRI08	0003:0550			offset: 0x50												default: 0x1110		
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	PRI17[5]	PRI17[4]	PRI17[3]	PRI17[2]	PRI17[1]	PRI17[0]	RSV	RSV	PRI16[5]	PRI16[4]	PRI16[3]	PRI16[2]	PRI16[1]	PRI16[0]		
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W		
Default	-	-	0	1	0	0	0	1	-	-	0	1	0	0	0	0		

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
13:8	PRI17	17	R/W	Interrupt Factor with 18th highest priority
7:6	RSV			Reserved
5:0	PRI16	16	R/W	Interrupt Factor with 17th highest priority

6.7.33 INTPRI09

Interrupt priority register #9

INTPRI09	0003:0552			offset: 0x52												default: 0x1312		
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	PRI19[5]	PRI19[4]	PRI19[3]	PRI19[2]	PRI19[1]	PRI19[0]	RSV	RSV	PRI18[5]	PRI18[4]	PRI18[3]	PRI18[2]	PRI18[1]	PRI18[0]		
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W		
Default	-	-	0	1	0	0	1	1	-	-	0	1	0	0	1	0		

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
13:8	PRI19	19	R/W	Interrupt Factor with 20th highest priority
7:6	RSV			Reserved
5:0	PRI18	18	R/W	Interrupt Factor with 19th highest priority

6.7.34 INTPRI10

Interrupt priority register #10

INTPRI10	0003:0554			offset: 0x54			default: 0x1514									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	PRI21[5]	PRI21[4]	PRI21[3]	PRI21[2]	PRI21[1]	PRI21[0]	RSV	RSV	PRI20[5]	PRI20[4]	PRI20[3]	PRI20[2]	PRI20[1]	PRI20[0]
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	0	1	0	1	0	1	-	-	0	1	0	1	0	0

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
13:8	PRI21	21	R/W	Interrupt Factor with 22nd highest priority
7:6	RSV			Reserved
5:0	PRI20	20	R/W	Interrupt Factor with 21st highest priority

6.7.35 INTPRI11

Interrupt priority register #11

INTPRI11	0003:0556			offset: 0x56													default: 0x1716	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	PRI23[5]	PRI23[4]	PRI23[3]	PRI23[2]	PRI23[1]	PRI23[0]	RSV	RSV	PRI22[5]	PRI22[4]	PRI22[3]	PRI22[2]	PRI22[1]	PRI22[0]		
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W		
Default	-	-	0	1	0	1	1	1	-	-	0	1	0	1	1	0		

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
13:8	PRI23	23	R/W	Interrupt Factor with 24th highest priority
7:6	RSV			Reserved
5:0	PRI22	22	R/W	Interrupt Factor with 23rd highest priority

6.7.36 INTPRI12

Interrupt priority register #12

INTPRI12	0003:0558			offset: 0x58													default: 0x1918		
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	PRI25[5]	PRI25[4]	PRI25[3]	PRI25[2]	PRI25[1]	PRI25[0]	RSV	RSV	PRI24[5]	PRI24[4]	PRI24[3]	PRI24[2]	PRI24[1]	PRI24[0]			
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W			
Default	-	-	0	1	1	0	0	1	-	-	0	1	1	0	0	0			

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
13:8	PRI25	25	R/W	Interrupt Factor with 26th highest priority
7:6	RSV			Reserved
5:0	PRI24	24	R/W	Interrupt Factor with 25th highest priority

6.7.37 INTPRI13

Interrupt priority register #13

INTPRI13	0003:055A			offset: 0x5A			default: 0x1B1A									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	PRI27[5]	PRI27[4]	PRI27[3]	PRI27[2]	PRI27[1]	PRI27[0]	RSV	RSV	PRI26[5]	PRI26[4]	PRI26[3]	PRI26[2]	PRI26[1]	PRI26[0]
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	0	1	1	0	1	1	-	-	0	1	1	0	1	0

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
13:8	PRI27	27	R/W	Interrupt Factor with 28th highest priority
7:6	RSV			Reserved
5:0	PRI26	26	R/W	Interrupt Factor with 27th highest priority

6.7.38 INTPRI14

Interrupt priority register #14

INTPRI14	0003:055C			offset: 0x5C			default: 0x1D1C									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	PRI29[5]	PRI29[4]	PRI29[3]	PRI29[2]	PRI29[1]	PRI29[0]	RSV	RSV	PRI28[5]	PRI28[4]	PRI28[3]	PRI28[2]	PRI28[1]	PRI28[0]
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	0	1	1	1	0	1	-	-	0	1	1	1	0	0

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
13:8	PRI29	29	R/W	Interrupt Factor with 30th highest priority
7:6	RSV			Reserved
5:0	PRI28	28	R/W	Interrupt Factor with 29th highest priority

6.7.39 INTPRI15

Interrupt priority register #15

INTPRI15	0003:055E				offset: 0x5E				default: 0x1F1E							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	PRI31[5]	PRI31[4]	PRI31[3]	PRI31[2]	PRI31[1]	PRI31[0]	RSV	RSV	PRI30[5]	PRI30[4]	PRI30[3]	PRI30[2]	PRI30[1]	PRI30[0]
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	0	1	1	1	1	1	-	-	0	1	1	1	1	0

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
13:8	PRI31	31	R/W	Interrupt Factor with 32nd highest priority
7:6	RSV			Reserved
5:0	PRI30	30	R/W	Interrupt Factor with 31st highest priority

6.7.40 INTPRI16

Interrupt priority register #16

INTPRI16	0003:0560			offset: 0x60													default: 0x2120		
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	PRI33[5]	PRI33[4]	PRI33[3]	PRI33[2]	PRI33[1]	PRI33[0]	RSV	RSV	PRI32[5]	PRI32[4]	PRI32[3]	PRI32[2]	PRI32[1]	PRI32[0]			
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W			
Default	-	-	1	0	0	0	0	1	-	-	1	0	0	0	0	0			

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
13:8	PRB3	33	R/W	Interrupt Factor with 34th highest priority
7:6	RSV			Reserved
5:0	PRB2	32	R/W	Interrupt Factor with 33rd highest priority

6.7.41 INTPRI17

Interrupt priority register #17

INTPRI17	0003:0562			offset: 0x62												default: 0x2322		
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	PRI35[5]	PRI35[4]	PRI35[3]	PRI35[2]	PRI35[1]	PRI35[0]	RSV	RSV	PRI34[5]	PRI34[4]	PRI34[3]	PRI34[2]	PRI34[1]	PRI34[0]		
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W		
Default	-	-	1	0	0	0	1	1	-	-	1	0	0	0	1	0		

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
13:8	PRI35	35	R/W	Interrupt Factor with 36th highest priority
7:6	RSV			Reserved
5:0	PRI34	34	R/W	Interrupt Factor with 35th highest priority

6.7.42 INTPRI18

Interrupt priority register #18

INTPRI18	0003:0564			offset: 0x64												default: 0x2524		
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	PRI37[5]	PRI37[4]	PRI37[3]	PRI37[2]	PRI37[1]	PRI37[0]	RSV	RSV	PRI36[5]	PRI36[4]	PRI36[3]	PRI36[2]	PRI36[1]	PRI36[0]		
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W		
Default	-	-	1	0	0	1	0	1	-	-	1	0	0	1	0	0		

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
13:8	PRI37	37	R/W	Interrupt Factor with 38th highest priority
7:6	RSV			Reserved
5:0	PRI36	36	R/W	Interrupt Factor with 37th highest priority

6.7.43 INTPRI19

Interrupt priority register #19

INTPRI19	0003:0566			offset: 0x66			default: 0x2726									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	PRI39[5]	PRI39[4]	PRI39[3]	PRI39[2]	PRI39[1]	PRI39[0]	RSV	RSV	PRI38[5]	PRI38[4]	PRI38[3]	PRI38[2]	PRI38[1]	PRI38[0]
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	1	0	0	1	1	1	-	-	1	0	0	1	1	0

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
13:8	PRI39	39	R/W	Interrupt Factor with 40th highest priority
7:6	RSV			Reserved
5:0	PRI38	38	R/W	Interrupt Factor with 39th highest priority

6.7.44 INTPRI20

Interrupt priority register #20

INTPRI20	0003:0568				offset: 0x68				default: 0x2928							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	PRI41[5]	PRI41[4]	PRI41[3]	PRI41[2]	PRI41[1]	PRI41[0]	RSV	RSV	PRI40[5]	PRI40[4]	PRI40[3]	PRI40[2]	PRI40[1]	PRI40[0]
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	1	0	1	0	0	1	-	-	1	0	1	0	0	0

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
13:8	PRI41	41	R/W	Interrupt Factor with 42nd highest priority
7:6	RSV			Reserved
5:0	PRI40	40	R/W	Interrupt Factor with 41st highest priority

6.7.45 INTPRI21

Interrupt priority register #21

INTPRI21	0003:056A			offset: 0x6A			default: 0x2B2A									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	PRI43[5]	PRI43[4]	PRI43[3]	PRI43[2]	PRI43[1]	PRI43[0]	RSV	RSV	PRI42[5]	PRI42[4]	PRI42[3]	PRI42[2]	PRI42[1]	PRI42[0]
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	1	0	1	0	1	1	-	-	1	0	1	0	1	0

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
13:8	PRI43	43	R/W	Interrupt Factor with 44th highest priority
7:6	RSV			Reserved
5:0	PRI42	42	R/W	Interrupt Factor with 43rd highest priority

6.7.46 INTPRI22

Interrupt priority register #22

INTPRI22	0003:056C			offset: 0x6C											default: 0x2D2C	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	PRI45[5]	PRI45[4]	PRI45[3]	PRI45[2]	PRI45[1]	PRI45[0]	RSV	RSV	PRI44[5]	PRI44[4]	PRI44[3]	PRI44[2]	PRI44[1]	PRI44[0]
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	1	0	1	1	0	1	-	-	1	0	1	1	0	0

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
13:8	PRI45	45	R/W	Interrupt Factor with lowest priority
7:6	RSV			Reserved
5:0	PRI44	44	R/W	Interrupt Factor with 45th highest priority

7 Timers

7.1 Introduction

Timers are used to get precise timing information. Timers can be used to generate precise clock ticks for operating systems, which schedules tasks based on ticks.

The TMS320DM320 ARM subsystem has four programmable timers. Each timer output is connected to a dedicated interrupt signal to the Interrupt Controller, as shown in Figure 18.

Timer 2 and Timer 3 also can function as “CCD Timers” for generating precise shutter and strobe signals to CCD Camera. TMR2 and TMR3 are also referred to as CCDTMR0 and CCDTMR1 respectively.

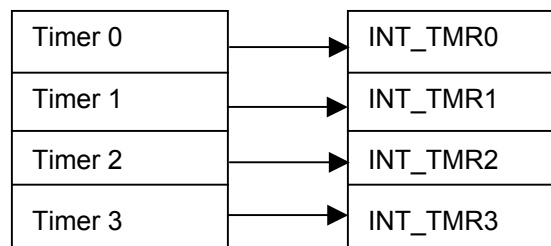


Figure 18: ARM timer interrupts

7.2 Configuring the Timers

Timer 0 and Timer 1 are identical in their functions. Timer 2 and Timer 3 (CCD Timer 0 and CCD Timer 1) are identical in their functions. In this chapter timers are referred as TMRx, where 'x' can take values from 0 to 3, indicating timer 0 to timer 3, respectively. Figure 19 shows the functional block diagram for timers.

Each of the general-purpose timers can run in one of two modes: one-shot mode or free-run mode.

Once the prescale and divider values of the timer are set and the timer is started, the prescale is decremented by one for every input clock cycle. Each time the prescale value reaches '0', the divider register is decremented by one and counter register incremented by one. Also the prescale register is reloaded with the programmed value. When the divider reaches zero, the timer generates an interrupt.

In one-shot mode, an interrupt occurs only once, when the timer divider reaches zero and then the timer must be explicitly re-triggered to begin the operation again.

In free-run mode, when the timer generates an interrupt, the divider value is automatically reloaded with the programmed value to start the count operation again.

The time interval between two consecutive interrupts is:

$$\text{Clock cycle} \times (\text{Timer pre scalar value} + 1) \times (\text{Timer count value} + 1)$$

Note: When both the timer prescale value and the divider value are set to “0”, the timer doesn’t run.

Either the AHB clock or PLL input clock can be selected as the clock source for the timer. Refer chapter on clock controller for more details. The actual count operation is equal to the value set in the bit field PRSCL (10-bits) of the register TMPRSCLx times the value set in the bit field DIV (16-bits) of the register TMDIVx. Frequency of a timer interrupt is given by the following equation:

$$\text{Timer Interrupt Frequency} = \frac{\text{Source Clock Frequency}}{(\text{PRSCL} + 1)(\text{DIV} + 1)}$$

The timer is automatically started once MODE field in TMMDx is set to free run mode. When a timer is set to one-shot mode, it is started by writing a “1” to the TRG bit in the register TMTRGx.

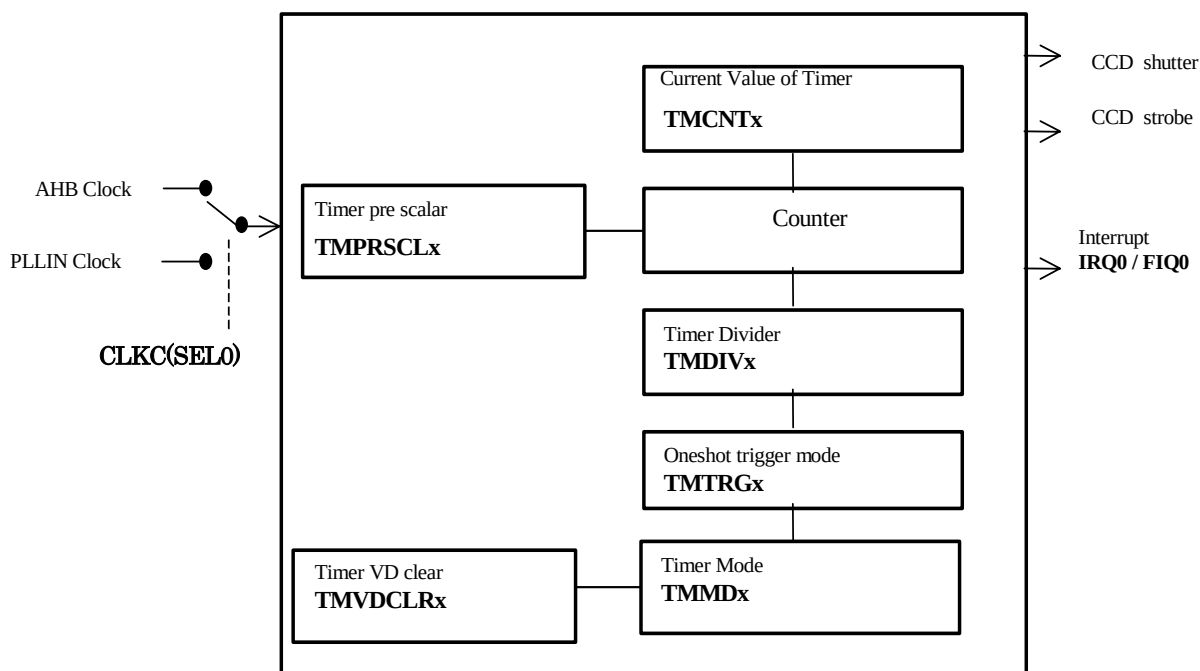


Figure 19: Function Block Diagram of Timer

Timer Operation

When the information about timing is required continuously (like system ticks for an OS), the timer should be configured for free-run mode. On the other hand if the information is required for only one instance (close the shutter of the camera after the required exposure time), the timer should be configured for one-shot mode.

The steps for setting up a timer is given below:

- o Set up the prescaler and divider values in TMPRSClX and TMDIVX registers
- o Select the mode of operation in TMMDX register
- o Set the TRG bit in TMTRGX register, if one-shot mode is selected
- o Stop the timer by setting the MODE field in TMMDX to '0', after getting the necessary timing information

Note: Use the TMCNTX register to get the count value at any point of time. The count value must be read before the timer is stopped explicitly, since stopping the timer clears all the registers.

7.2.1 CCD Timer Operation

Timer-2 and Timer-3 (CCDTMR0 and CCDTMR1) can also be used to precisely control the CCD camera. Shutter and strobe timing requirements can be programmed in the CCD-Timers and the signals can be output using GPIO35 or GPIO36, which will control the camera. The trigger for the timer can be set to automatic trigger, based on signals from CCD Controller or can be based on the TRG bit in TMTRGX register for the timers. For CCDC shutter / strobe functionalities select one-shot mode of operation of timer.

Shutter timer can be triggered on the positive edge of the VD signal from the CCD. The timing for the shutter timer is given in Figure 20.

Strobe timer can be triggered at the low level of the VD1 interrupt signal from the CCD. The timing for the strobe timer is given in Figure 21.

Shutter and Strobe timer modes can also be triggered using the register trigger or event trigger as described above.

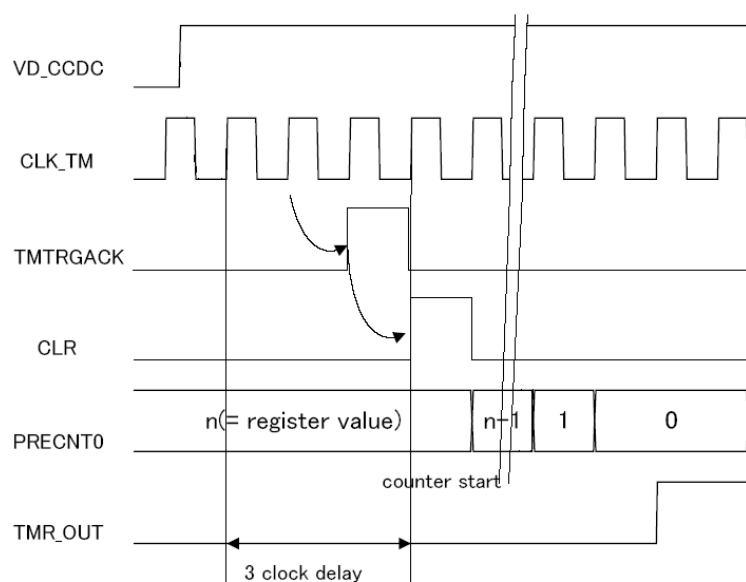


Figure 20: Timing diagram in shutter mode

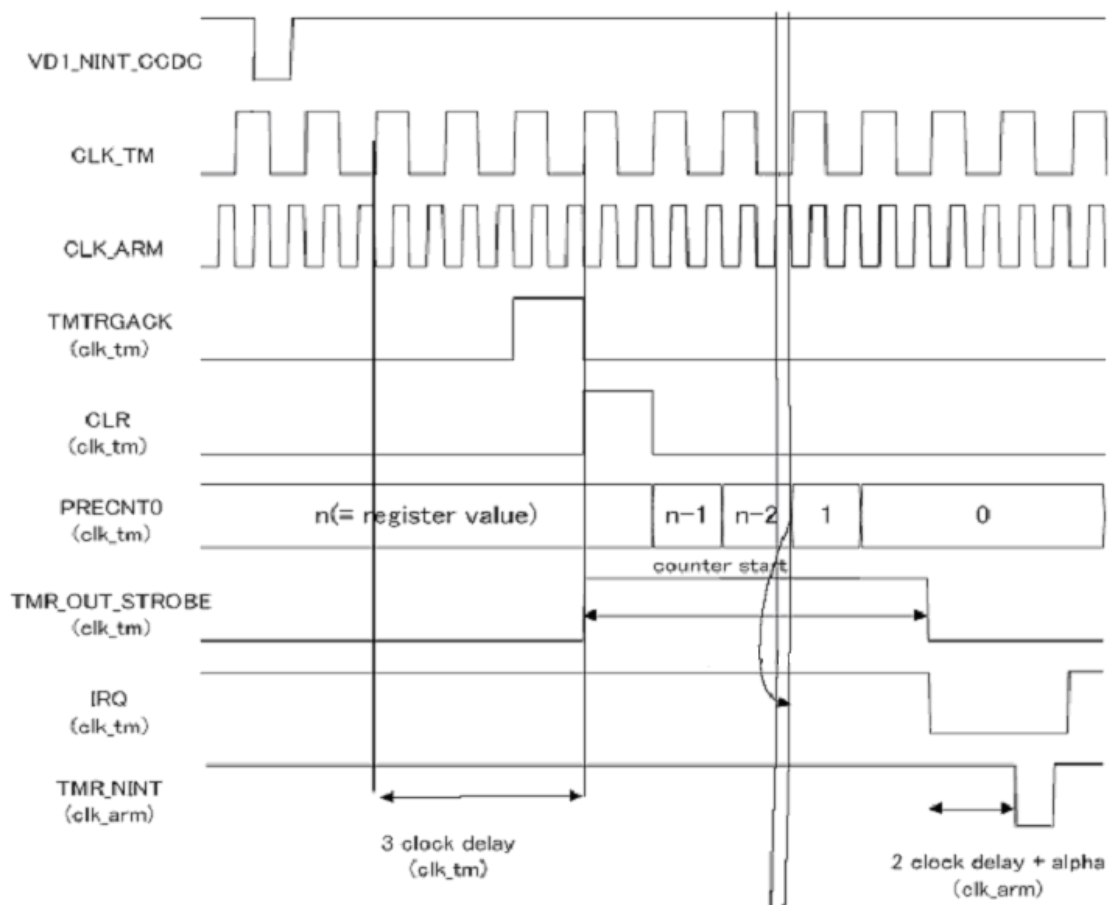


Figure 21: Timing diagram for Strobe mode

The steps for setting up a timer for CCD mode operation is given below:

- o Select CCD Timer operation for GIO35 or GIO36 in GIO FSEL register. Refer chapter on GIO for more details.
- o Set up the prescaler and divider values in TMPRSClX and TMDIVx registers
- o Set to one shot mode in TMMDx register
- o Select the CCD timer mode in VDMODE field of TMMDx register.
- o Select the polarity of the output signal in POL bit of TMMDx register.
- o Select the trigger (register or event trigger) in TRGSEL bit of TMMDx register.
- o Set the TRG bit in TMTRGx register after checking the status of the trigger ready in TRGRDY bit of TMMDx register, if register trigger mode is selected. If the TRGRDY bit is not set, the trigger signal will be ignored.

Note: CCD Controller (CCDC) configurations should be done along with CCD timer configurations.

7.3 Timer Register Map (TMRx)

7.3.1 Timer 0 Register Map

Address	Register	Description
0003:0000	TMMD0	Timer 0 Mode
0003:0002	TMRSV0	Reserved
0003:0004	TMPSCL0	Timer 0 Prescaler
0003:0006	TMDIV0	Timer 0 Divisor (count)
0003:0008	TMTRG0	Timer 0 One-Shot Trigger
0003:000A	TMCNT0	Timer 0 Count

7.3.2 Timer 1 Register Map

Address	Register	Description
0003:0080	TMMD1	Timer 1 Mode
0003:0082	TMRSV1	Reserved
0003:0084	TMPSCL1	Timer 1 Prescaler
0003:0086	TMDIV1	Timer 1 Divisor (count)
0003:0088	TMTRG1	Timer 1 One-Shot Trigger
0003:008A	TMCNT1	Timer 1 Count

7.3.3 Timer 2 (CCD Timer 0)

Address	Register	Description
0003:0100	TMMD2	Timer 2 Mode
0003:0102	TMVDCLR2	Timer 2 CCD control
0003:0104	TMPSCL2	Timer 2 Prescaler
0003:0106	TMDIV2	Timer 2 Divisor (count)
0003:0108	TMTRG2	Timer 2 One-Shot Trigger
0003:010A	TMCNT2	Timer 2 Count

7.3.4 Timer 3 (CCD Timer 1)

Address	Register	Description
0003:0180	TMMD3	Timer 3 Mode
0003:0182	TMVDCLR3	Timer 3 CCD control
0003:0184	TMPSCL3	Timer 3 Prescaler
0003:0186	TMDIV3	Timer 3 Divisor (count)
0003:0188	TMTRG3	Timer 3 One-Shot Trigger
0003:018A	TMCNT3	Timer 3 Count

Timer Registers

7.3.5 TMMDx

Timer Mode registers

Timer 0 and Timer 1 Mode Registers

TMMD0	0003:0000			offset: 0x00			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	MODE[1]	MODE[0]
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W	R/W
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0	0

TMMD1	0003:0080			offset: 0x00			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	MODE[1]	MODE[0]
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W	R/W
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0	0

Bit	Name	Reset Value	R/W	Function
15:2	RSV			Reserved
				Timer Mode » 00: stop « » 01: one shot « » 10: free run « » 11: n/a «
10	MODE	0	R/W	One Shot: Count is started when 1 is written in one shot trigger bit. When set value is reached, interrupt occurs and count operation is stopped. Free Run: Count is repeated up to set value. Each time set value is reached, interrupt occurs.

Timer 2 and Timer 3 Mode Registers

TMMD2	0003:0100			offset: 0x00		default: 0x0000										
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	OSIG	TRGRDY	TRGSEL	POL	VDMODE[1]	VDMODE[0]	RSV	RSV	RSV	RSV	RSV	RSV	MODE[1]	MODE[0]
R/W	-	-	R	R	R/W	R/W	R/W	R/W	-	-	-	-	-	-	R/W	R/W
Default	-	-	0	0	0	0	0	0	-	-	-	-	-	-	0	0

TMMD3	0003:0180			offset: 0x00												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	OSIG	TRGRDY	TRGSEL	POL	VDMODE[1]	VDMODE[0]	RSV	RSV	RSV	RSV	RSV	RSV	MODE[1]	MODE[0]	
R/W	-	-	R	R	R/W	R/W	R/W	R/W	-	-	-	-	-	-	R/W	R/W	
Default	-	-	0	0	0	0	0	0	-	-	-	-	-	-	0	0	

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
13	OSIG	0	R	Output signal from Timer
12	TRGRDY	0	R	Trigger ready » 0: Not ready for trigger « » 1: Ready for trigger « Note: When trigger is not ready, any trigger signal will be ignored
11	TRGSEL	0	R/W	Trigger select » 0: One shot trigger « » 1: VD in shutter mode or nint in strobe mode «
10	POL	0	R/W	Output signal polarity » 0: High active « » 1: Low active «
9:8	VDMODE	0	R/W	CCD timer Mode » 00: Reserved « » 01: Shutter mode « » 10: Strobe mode « » 11: Reserved « When selecting either Shutter / Strobe mode, select Timer mode as One Shot
7:2	RSV			Reserved
1:0	MODE	0	R/W	Timer Mode » 00: stop « » 01: one shot « » 10: free run « » 11: n/a « One Shot: Count is started when 1 is written in one shot trigger bit. When set value is reached, interrupt occurs and count operation is stopped. Free Run: Count is repeated up to set value. Each time set value is reached, interrupt occurs.

7.3.6 TMVDCLR

CCD Timer control registers

Timer 0 and Timer 1 Reserve Registers

TMRSV0	0003:0002			offset: 0x02												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	

TMRSV1	0003:0082			offset: 0x02												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	

Bit	Name	Reset Value	R/W	Function
15:0	RSV			Reserved

Timer 2 and Timer 3 CCD Timer control Registers

TMVDCLR2	0003:0102			offset: 0x02												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	SHUTCLR	
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W	
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0	

TMVDCLR3	0003:0182			offset: 0x02												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	SHUTCLR	
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W	
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0	

Bit	Name	Reset Value	R/W	Function
15:1	RSV			Reserved
0	SHUTCLR	0	R/W	When in Shutter mode Write: '0' to make GIO output '0' '1' Not possible Read: Always '0'

7.3.7 TMPRSCLx

Timer prescale registers

TMPRSCL0		0003:0004			offset: 0x04												default: 0x0100	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	RSV	RSV	RSV	RSV	PRSC[9]	PRSC[8]	PRSC[7]	PRSC[6]	PRSC[5]	PRSC[4]	PRSC[3]	PRSC[2]	PRSC[1]	PRSC[0]		
R/W	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W		
Default	-	-	-	-	-	-	0	1	0	0	0	0	0	0	0	0		

TMPRSCL1		0003:0084			offset: 0x04			default: 0x0100								
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	PRSC[9]	PRSC[8]	PRSC[7]	PRSC[6]	PRSC[5]	PRSC[4]	PRSC[3]	PRSC[2]	PRSC[1]	PRSC[0]
R/W	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	-	0	1	0	0	0	0	0	0	0	0

TMPRSCL2		0003:0104				offset: 0x04										default: 0x0100			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	RSV	RSV	RSV	RSV	PRSC[9]	PRSC[8]	PRSC[7]	PRSC[6]	PRSC[5]	PRSC[4]	PRSC[3]	PRSC[2]	PRSC[1]	PRSC[0]			
R/W	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W			
Default	-	-	-	-	-	-	0	1	0	0	0	0	0	0	0	0			

TMPRSCL3		0003:0184			offset: 0x04			default: 0x0100								
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	PRSC[9]	PRSC[8]	PRSC[7]	PRSC[6]	PRSC[5]	PRSC[4]	PRSC[3]	PRSC[2]	PRSC[1]	PRSC[0]
R/W	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	-	0	1	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:10	RSV			Reserved
9:0	PRSC	256	R/W	Timer Prescaler Divide Value Timer performs count operation with input clock frequency divided by (PRSC+1)

7.3.8 TMDIVx

Timer divider registers

TMDIV0	0003:0006			offset: 0x06			default: 0x1FFF									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	DIV[15]	DIV[14]	DIV[13]	DIV[12]	DIV[11]	DIV[10]	DIV[9]	DIV[8]	DIV[7]	DIV[6]	DIV[5]	DIV[4]	DIV[3]	DIV[2]	DIV[1]	DIV[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1

TMDIV1	0003:0086			offset: 0x06			default: 0x1FFF									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	DIV[15]	DIV[14]	DIV[13]	DIV[12]	DIV[11]	DIV[10]	DIV[9]	DIV[8]	DIV[7]	DIV[6]	DIV[5]	DIV[4]	DIV[3]	DIV[2]	DIV[1]	DIV[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1

TMDIV2	0003:0106			offset: 0x06			default: 0x1FFF									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	DIV[15]	DIV[14]	DIV[13]	DIV[12]	DIV[11]	DIV[10]	DIV[9]	DIV[8]	DIV[7]	DIV[6]	DIV[5]	DIV[4]	DIV[3]	DIV[2]	DIV[1]	DIV[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1

TMDIV3	0003:0186			offset: 0x06			default: 0x1FFF									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	DIV[15]	DIV[14]	DIV[13]	DIV[12]	DIV[11]	DIV[10]	DIV[9]	DIV[8]	DIV[7]	DIV[6]	DIV[5]	DIV[4]	DIV[3]	DIV[2]	DIV[1]	DIV[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1

Bit	Name	Reset Value	R/W	Function
15:0	DIV	8191	R/W	Timer Division ratio Timer count value is (PRSC1+1)x(DIV+1)

7.3.9 TMTRGx

Timer trigger registers

TMTRG0		0003:0008				offset: 0x08				default: 0x0000						
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	TRG
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	W
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0

TMTRG1		0003:0088			offset: 0x08			default: 0x0000								
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	TRG
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	W
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0

TMTRG2	0003:0108			offset: 0x08			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	TRG
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	W
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0

TMTRG3		0003:0188				offset: 0x08				default: 0x0000						
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	TRG
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	W
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0

Bit	Name	Reset Value	R/W	Function
15:1	RSV			Reserved
0	TRG	0	W	One Shot trigger Trigger bit when timer is set to one shot mode. When 1 is written, timer count begins.

7.3.10 TMCNTx

Timer count registers

TMCNT0		0003:000A				offset: 0x0A				default: 0x0000						
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	CNT[15]	CNT[14]	CNT[13]	CNT[12]	CNT[11]	CNT[10]	CNT[9]	CNT[8]	CNT[7]	CNT[6]	CNT[5]	CNT[4]	CNT[3]	CNT[2]	CNT[1]	CNT[0]
R/W	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

TMCNT1		0003:008A			offset: 0x0A												default: 0x0000		
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	CNT[15]	CNT[14]	CNT[13]	CNT[12]	CNT[11]	CNT[10]	CNT[9]	CNT[8]	CNT[7]	CNT[6]	CNT[5]	CNT[4]	CNT[3]	CNT[2]	CNT[1]	CNT[0]			
R/W	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R			
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			

TMCNT2	0003:010A			offset: 0x0A			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	CNT[15]	CNT[14]	CNT[13]	CNT[12]	CNT[11]	CNT[10]	CNT[9]	CNT[8]	CNT[7]	CNT[6]	CNT[5]	CNT[4]	CNT[3]	CNT[2]	CNT[1]	CNT[0]
R/W	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

TMCNT3		0003:018A				offset: 0x0A				default: 0x0000						
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	CNT[15]	CNT[14]	CNT[13]	CNT[12]	CNT[11]	CNT[10]	CNT[9]	CNT[8]	CNT[7]	CNT[6]	CNT[5]	CNT[4]	CNT[3]	CNT[2]	CNT[1]	CNT[0]
R/W	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:0	CNT	0	R	Current counter value

8 Watch Dog Timer (WDT)

8.1 Introduction

The Watch Dog Timer (WDT) is used to determine whether a system is running out-of-control and to initiate the system's recovery to a controlled state. If the WDT is not reset within a setting time, the WDT can automatically either interrupt or reset the ARM.

The setting time is as follows.

$\text{Clock cycle} \times (\text{Timer pre scaler value} + 1) \times (\text{Timer count value})$

The WDT uses the external clock connected to the MXI pin. In normal operation a 27MHz clock is connected to MXI.

8.2 Using the Watch Dog Timer

Figure 22 shows the functional block diagram of DM320 WDT.

The registers WDTPRSCL and WDTDIV are used to set the WDT's configurable detection time. The next section explains how to use WDTPRSCL and WDTDIV to set the configurable detection time.

The ENABLE field in the register WDTMD enables the WDT. The WDT produces a reset signal that is 32 clocks (external clock) wide. The action WDT should do, in case the WDT internal reset is not done within the prescribed time can be specified in the OPSEL field of the WDTMD.

The WDT declares that a system is running out of control when the WDT is not reset within a configurable period of time. Setting bit WDTRST of the WDTRST resets the WDT. Therefore, a normal system should be designed to set WDTRST within the configured detection time.

When the system fails to set WDTRST within the configured detection time, the WDT will automatically interrupt the ARM or reset the ARM. The WDT will interrupt the ARM when bit OPSEL of the register WDTMD is set to 0, and the WDT will reset the ARM when bit OPSEL is set to 1.

Through the ARM Interrupt Controller module, the WDT interrupt can be configured as an FIQ type interrupt or an IRQ type interrupt. Also this interrupt can be enabled and disabled through the Interrupt Controller. Refer to the section on the "Interrupt Controller" in this manual for information on the ARM interrupt controller.

The reset signal generated by GIO can be an ARM reset signal or an external reset on the GIO37. The width of the reset signal is 32 external clocks.

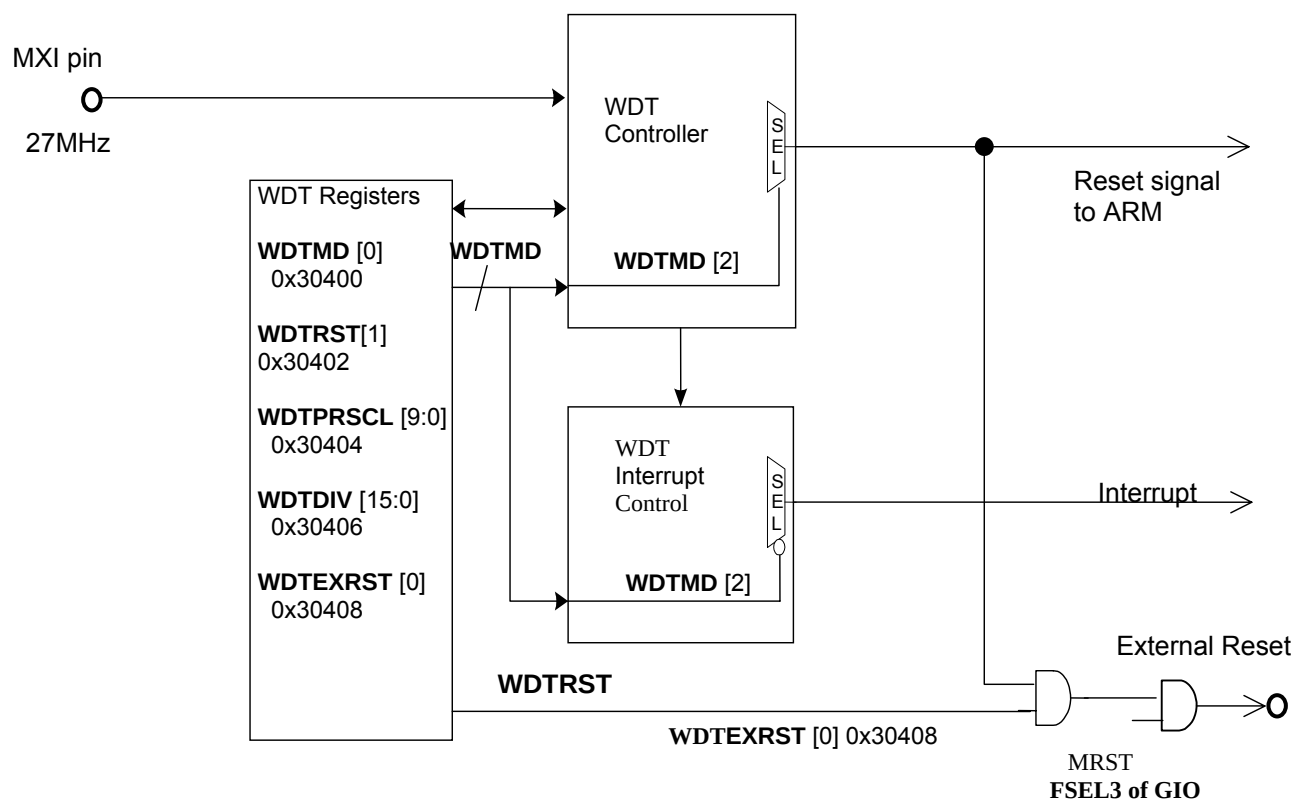


Figure 22: Watch Dog Timer Block Diagram

8.3 Setting the Configurable Detection Time

The WDT uses the external clock connected to the MXI pin. In normal operation a 27MHz clock is connected to MXI. Configure the detection time by setting a prescaler value in WDTPRSCL and a divisor value in WDTDIV. The detection time is as shown in the following equation (assuming a 27MHz clock connected to MXI). Setting WDTRST bit of the WDTRST register, will reset the watch dog timer.

$$\text{WDT Detection Time} = ((\text{WDTPRSCL} + 1) \times \text{WDTDIV}) / 27,000 \text{ ms}$$

Note: Don't set WDTDIV value to 0.

8.4 WDT operation

WDT is used to monitor if the system is going out of control. Before a critical operation is performed, the WDT is initialized and the outputs selected. The steps involved in using WDT are given below.

- o Set the prescaler and divider value for WDT in the WDTPRSC and WDTDIV registers respectively, according to the time the critical operation is going to take.
- o Select the output of the WDT operation – an interrupt or a reset signal, in the OPSEL bit of WDTMD register.
- o Select the reset signal output – external reset or ARM reset, if the output selected is reset. This is done by configuring the WERST bit in WDTXRST register.
- o To reset using external reset pin, configure GPIO37 as MRST using FSEL3 register in GPIO. Refer chapter on GPIO for more details.
- o Hook an interrupt service routine to WDT interrupt, if the output selected is interrupt.
- o If the control comes back issue a WDT reset by setting WDTRST bit in WDTRST register.
- o If the control doesn't come back, handle the exception in WDT ISR or reset the ARM or reset the external device.

8.5 Watch Dog Timer Register Map (WDT)

Address	Register	Description
0003:0400	WDTMD	Watch Dog Timer Mode
0003:0402	WDRST	Watch Dog Timer Reset
0003:0404	WDTPRSC	Watch Dog Timer Prescaler
0003:0406	WDTDIV	Watch Dog Timer Divisor (count)
0003:0408	WDTXRST	Watch Dog Timer External Reset

8.6 Watch Dog Timer Registers

8.6.1 WDTMD

Mode register

WDTMD	0003:0400				offset: 0x00				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	OPSEL	RSV	ENABLE
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W	-	R/W
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	0	-	0

Bit	Name	Reset Value	R/W	Function
15:3	RSV			Reserved
2	OPSEL	0	R/W	Watch dog timer output selection » 0: Generate WDT interrupt « » 1: Reset ARM «
1	RSV			Defines action taken if a write to WDTRST is not performed within specified time.
0	ENABLE	0	R/W	Watch Dog Timer enable » 0: disable « » 1: enable «

8.6.2 WDTRST

Reset register

WDTRST	0003:0402			offset: 0x02												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	WDTRST	
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	W	
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0	

Bit	Name	Reset Value	R/W	Function
15:1	RSV			Reserved
0	WDTRST	0	W	Watch Dog Timer Reset » 0: n/a « » 1: Reset Watch Dog Timer «

8.6.3 WDTPRCL

Pre-scale register

WDTPRCL	0003:0404			offset: 0x04		default: 0x0100										
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	PRSC[9]	PRSC[8]	PRSC[7]	PRSC[6]	PRSC[5]	PRSC[4]	PRSC[3]	PRSC[2]	PRSC[1]	PRSC[0]
R/W	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	-	0	1	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:10	RSV			Reserved
9:0	PRSC	256	R/W	Watch Dog Timer Prescaler Divide Value Timer performs count operation with 27 MHz clock divided by (PRSC+1)

8.6.4 WDTDIV

Divider register

WDTDIV	0003:0406			offset: 0x06												default: 0x1000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	DIV[15]	DIV[14]	DIV[13]	DIV[12]	DIV[11]	DIV[10]	DIV[9]	DIV[8]	DIV[7]	DIV[6]	DIV[5]	DIV[4]	DIV[3]	DIV[2]	DIV[1]	DIV[0]	
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:0	DIV	4096	R/W	Watch Dog Timer Timer Division ratio WDT count is (WDTPRCL+1)x(WDTDIV) Note: Don't set WDTDIV to 0x0

8.6.5 WDTEXRST

External reset register

WDTEXRST	0003:0408			offset: 0x08												default: 0x0001	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	WERST	
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W	
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	1	

Bit	Name	Reset Value	R/W	Function
15:1	RSV			Reserved
0	WERST	1	R/W	Watch Dog Timer External Reset External reset output pin (when GIO37 - MRST is set) » 0: External reset output pin is forced low « » 1: Reset signal to ARM output to Ext. Reset out pin « Note: Reset pulse width is 32-cycles of external clock input

9 CCD Controller

The CCD Controller captures and processes the raw data from the CCD sensor. The A/D converter and Timing Generator for CCD sensor are external to DM320 chip. The functions of CCD controller are output formatting, digital clamping, gain and offset control and decimation for physical image capturing.

9.1 Features

- o Generates timing signals VSYNC / HSYNC and field ID
- o Can synchronize to externally provided VSYNC / HSYNC signals
- o Supports progressive scan and interlaced CCD
- o Generates optical black clamping control signals
- o Programmable culling pattern
- o Maximal supported pixel clock is 100 MHz
- o Maximal supported CCD imager size 4096 X 4096
- o Digital clamping
- o Black level compensation
- o Median filtering
- o Gain and offset control
- o 10-bit to 8-bit A-law compression
- o Supports CCIR601/656 Digital YCbCr 4:2:2 (8/16bit) interface
- o Supports Bayer and Foveon input data format
- o Supports VGA read out mode
- o Supports up to 14-bit analog front end
- o Supports various image data format like Sony, Matsushita etc

Refer Figure 23 for details about CCDC processing blocks.

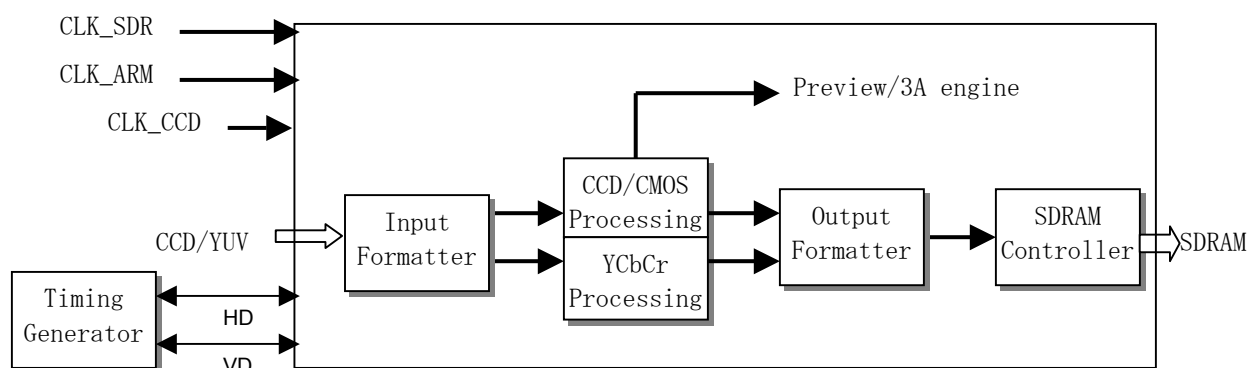


Figure 23: CCD Controller Block Diagram

9.2 CCDC Input Signal Interface

Timing and control of the interface is done using the HD, VD, field ID, pixel clock, and write enable signal. The CCD Controller accepts pixel data via the CI7-CI0, YI7-YI0 input pins. The definition of each pixel input is determined by the input format selected in the input formatter. The definition for signals in each mode is shown in Table 18. The interface consists of a set of signals used to transfer raw CCD data from an imager to the CCD Controller. Additionally, the CCD controller can be configured to operate in a mode that adheres to the CCIR 601/656 interface specification. The CCIR 601/656 specification provides a standard method, to transfer 8/16-bit, YCbCr-4:2:2 formatted video data. The pixel clock clocks data into the CCDC at a maximum rate of 100 MHz. When the CCDC is configured to write data to SDRAM, raw CCD data will be written to SDRAM. Details of the interface are given in Table 19.

Pin Name	CCD	16-bit YCbCr	8-bit YCbCr
CI7		Cb7,Cr7	Y7,Cb7,Cr7
CI6		Cb6,Cr6	Y6,Cb6,Cr6
CI5	C_DATA13	Cb5,Cr5	Y5,Cb5,Cr5
CI4	C_DATA12	Cb4,Cr4	Y4,Cb4,Cr4
CI3	C_DATA11	Cb3,Cr3	Y3,Cb3,Cr3
CI2	C_DATA10	Cb2,Cr2	Y2,Cb2,Cr2
CI1	C_DATA9	Cb1,Cr1	Y1,Cb1,Cr1
CI0	C_DATA8	Cb0,Cr0	Y0,Cb0,Cr0
YI7	C_DATA7	Y7	Y7,Cb7,Cr7
YI6	C_DATA6	Y6	Y6,Cb6,Cr6
YI5	C_DATA5	Y5	Y5,Cb5,Cr5
YI4	C_DATA4	Y4	Y4,Cb4,Cr4
YI3	C_DATA3	Y3	Y3,Cb3,Cr3
YI2	C_DATA2	Y2	Y2,Cb2,Cr2
YI1	C_DATA1	Y1	Y1,Cb1,Cr1
YI0	C_DATA0	Y0	Y0,Cb0,Cr0

Table 18: CCDC Data Input Formats

The CCDC supports 8 to 14-bit wide raw data signals. When the CCD has fewer than 14 data lines, connect the un-used data lines to ground.

CCDC input mode is selected using field IMPMOD in register *MODESET*

Name	I/O	Function
C_DATA[13]- C_DATA[0]	I	<u>Raw Image data loaded by CCD.</u> Bit width can be configured to 8 to 14 bits. The polarity of the input image data can be reversed.
C_VSYNC (VD)	I/O	<u>VSYNC. Vertical synch signal.</u> This signal can be configured as an input or an output. When configured as an input, the external CCD or CMOS sensor must supply the VD signal. When configured as an output the DM320 will supply the VD signal. The polarity of VD can be reversed.
C_HSYNC (HD)	I/O	<u>HSYNC. Horizontal synch signal.</u> This signal can be configured as an input or an output. When configured as an input, the external CCD or CMOS sensor must supply the HD signal. When configured as an output the DM320 will supply the HD signal. The polarity of HD can be reversed.
C_FIELD	I/O	<u>Field identification signal.</u> This signal can be configured as an input or an output. When configured as an input, the external CCD or CMOS sensor must supply the field identification signal. When configured as an output the DM320 will supply the field identification signal. When the field identification signal is set to be an input to the DM320, this signal can be configured to be latched by the VD signal. The polarity of the field identification signal can be reversed.
C_PCLK	I	<u>Pixel clock.</u> This signal is the pixel clock used to load image data into the CCDC. The Clock controller can configure to trigger on the rising or falling edge of the PCLK signal. The maximum pixel clock rate is 100MHz.

Table 19: CCDC Signal Interface

9.3 Frame Setup

ARM controls width, polarity, position and direction of internally generated signals. Figure 24 shows various CCDC register settings related to the frame setup. The shaded area is the physical imager size and the gray area is the valid data area. The image data in this valid data area will be processed and stored to external SDRAM. The vertical start position for even and odd fields can be configured independently. Various registers involved in the frame setup are described in Table 20.

The timing generator provides the use of external sync signals (HD/VD) or internally generated timing signals. The horizontal sampling start point is set in SPH register by using pixel number.

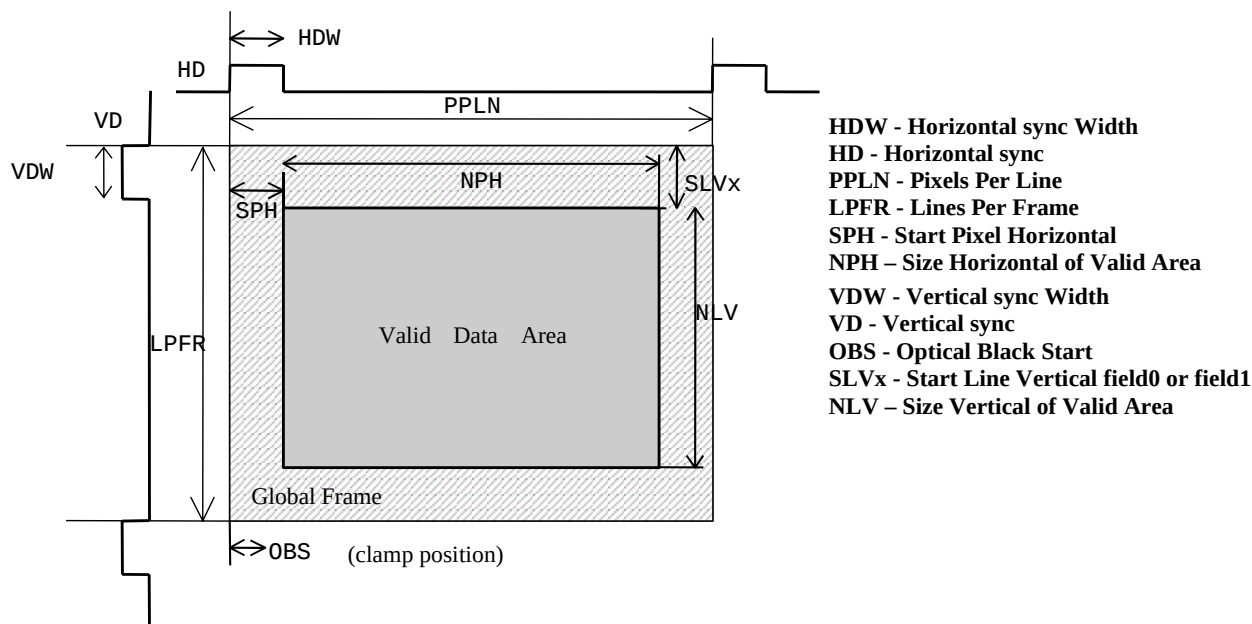


Figure 24 Frame setup

Register	Description
HDWIDTH	HSYNC pulse width
VDWIDTH	VSYNC pulse width
PPLN	Pixels per line
LPFR	Lines per frame
SPH	Start pixel horizontal
NPH	Number of pixels horizontal
SLV0	Start line vertical - field 0
SLV1	Start line vertical - field 1
NLV	Number of lines vertical

Table 20: Frame Setup Registers

All the above registers are latched by VD (see section 9.9)

9.4 CCDC Processing

As shown in Figure 25, the CCDC consists of several data processing modules that format and enhance raw CCD data. There are three destinations for the processed output data, SDRAM, H3A Engine and/or Preview Engine. The important functions of CCDC are given below:

- o Digital clamping
- o Data formatter
- o Black level compensation
- o Median Filtering
- o Gain and Offset control
- o A-Law compression
- o Output Formatter

Figure 2.

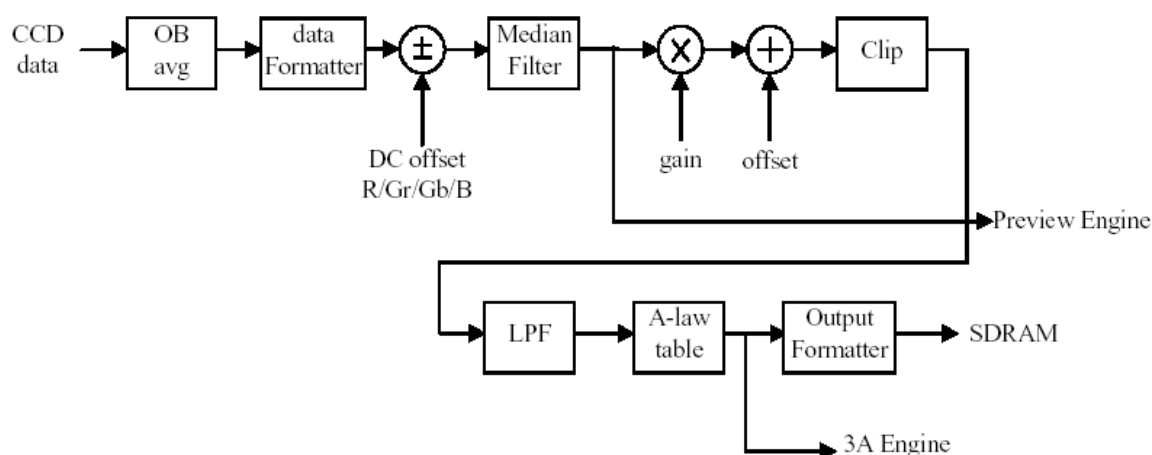


Figure 3.

Figure 4.

Figure 5.

Figure 25: CCD Raw data Processing blocks

9.4.1 Arrangement of CCD Raw Data

CCD controller can accept RGB raw data or complementary color raw data. The color pattern can be decided by the user using register *COLPTN*. The CCDC supports various sensors like Bayer, Foveon etc.



Existing Bayer format with R/Gr and Gb/B in alternate lines -
Horizontal distance between same colors is 2



R G B Foveon sensor with R, G, and B in same line -
Horizontal distance between same colors is 3

Figure 26: CCDC Color Patterns

9.4.2 Digital Clamp

The first module is the digital clamp block. The image data is latched at the rising or falling edge of the input pixel clock (PCLK). The averaging circuit takes an average of masked (black) pixel values from the image sensor and this value is subtracted from the image data. The user can control the position of the black pixels, the number of pixels (8 or 16) in each line averaged, and the number of lines (8 or 16) averaged.

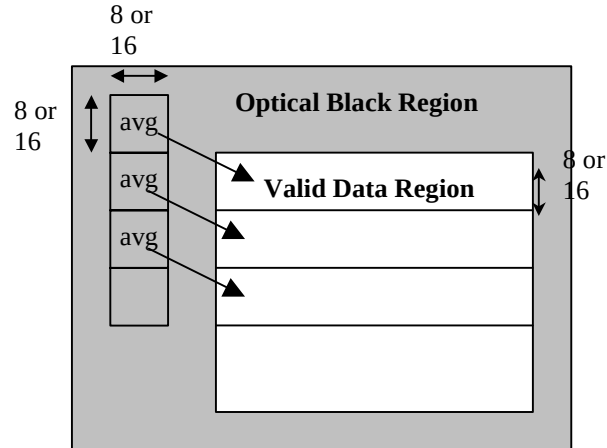


Figure 27: Digital Clamp Block Diagram

Alternately, the user can disable black clamp averaging and select a constant black value for subtraction, instead of using the calculated average value. CLAMP register is used to control the enable/disable and various parameters of clamping and DCSUB register is used to set the DC level to be subtracted from the CCD data.

9.4.3 Black Level Compensation

After the Digital Clamp is applied to the data, Black Level Compensation is applied to the R/Ye, Gr/Cy, Gb/G, and B/Mg signals. The offset applied to the current data sample is selected according to the even/odd phase and the color specified for each phase. Different black level compensation values can be set in the registers *BLKCMP0* and *BLKCMP1*.

9.4.4 Median Filter

The Median Filter's threshold is configurable and the Median Filter can be enabled or disabled. The threshold value for the median filter can be set in the register MEDFILT.

9.4.5 Gain, Offset, Clip and Low Pass Filter

A gain factor from 0 to 7.9921875 can be applied to the output of the Median Filter. After multiplying the data by the gain factor an offset can be added to the result. The offset can range from 0 to 1023. The resultant CCD data from gain control is clipped to signed 15bit data, and finally clipped to unsigned 14bit data after offset control.

The low-pass filter consists of a simple three-tap ($\frac{1}{4}$, $\frac{1}{2}$, $\frac{1}{4}$) filter. The low pass filter on/off can be set by bit LPF of MODESET register.

Different gains and offset can be set by registers RYE GAIN,GRCY GAIN,GBG GAIN,BMG GAIN , OFFSET. The output clipping value for the final stage of CCD processing is set by OUTCLIP register.

9.4.6 A-Law

A-Law function can be used to compress CCD data to reduce storage requirements in SDRAM.

When A-Law table compression is used, 10-bit data can be stored in SDRAM as 8-bit packed data, thus reducing SDRAM storage space requirement by 50%. The control of compression by A-law is set by register GAMMAWD.

Note: 10-bit raw data requires 16-bit space in SDRAM for storage.

9.4.6.1 Decimation

The final stage in CCDC raw data processing is a programmable decimation function along the horizontal and vertical directions.

Two registers control the horizontal and vertical decimation of CCD Data. The CULH register sets the horizontal culling for even and odd lines. The CULV register sets the culling along the vertical direction. The LSB of CULV represents the top (0th) line of the CCD, the MSB is the 7th line.

Figure 28 below is an example showing how CULH and CULV apply a decimation pattern to the data. The pixels colored in red are read out to SDRAM.

In this case CULH = 0x59C4 and CULV = 0x0066

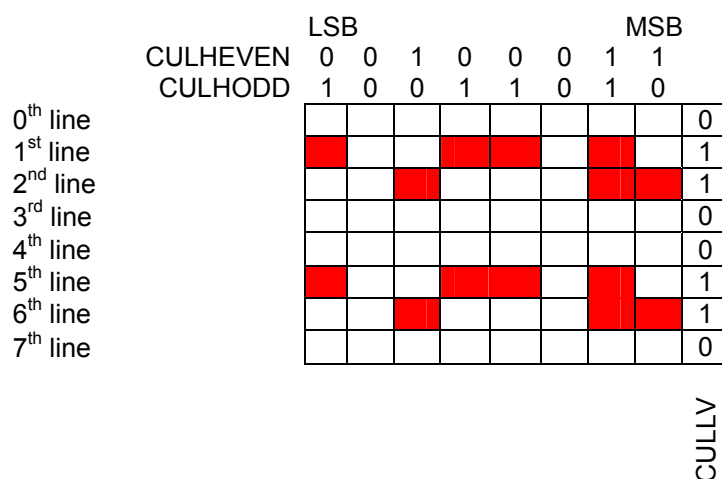


Figure 28: Example for Decimation Pattern

9.4.6.2 Output Format

The bits of data from each pixel are stored in the lower bits of a 16-bit SDRAM word, and the unused bit positions are filled with zeros. The SDRAM data format is shown in the Table 21.

	Lower word		Upper word	
	LSB(0)	MSB(15)	LSB(16)	MSB(31)
14bit	Pixel0	0	Pixel1	0
13bit	Pixel0	0	Pixel1	0
12bit	Pixel0	0	Pixel1	0
11bit	Pixel0	0	Pixel1	0
10bit	Pixel0	0	Pixel1	0
9bit	Pixel0	0	Pixel1	0
8bit	Pixel0	0	Pixel1	0
8bit pack	Pixel0	Pixel1	Pixel2	Pixel3

Table 21: SDRAM CCD Data Format

The processed CCD data is transferred to the SDRAM controller (SDRC) in units of 32 bytes. Data is transferred from the CCDC to the SDRC in 32-byte bursts.

Data is only written to SDRAM when WEN in SYNCEN is set to "1".

The output formatter can be configured for any image format by using SDRAM line offset register - HSIZE, and offset register - SDOFST.

Figure 29 shows how to construct a frame format in SDRAM.

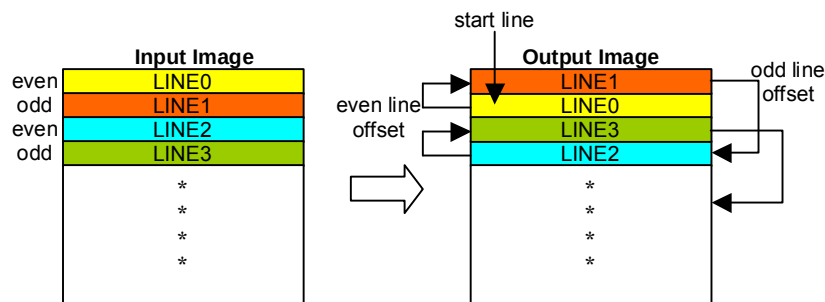


Figure 29: Frame Image format conversion

9.5 CCD Interrupts

The CCDC can generate two different interrupts VD0 and VD1.

9.5.1 Interrupts VD0 and VD1

As shown in the Figure 30, the VD0 and VD1 interrupts occur relative to the VD pulse. VD0 and VD1 will occur after receiving the number of horizontal lines (horizontal pulse signals) set in *VDINT0* and *VDINT1* respectively.

Figure 6.

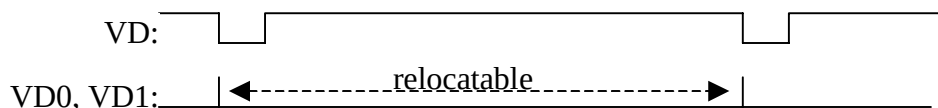


Figure 30: CCDC VD0, VD1 Interrupts

9.6 YCbCr signal Processing

Figure 31 shows YCbCr signal processing flow in the CCD controller block. The CCD controller accepts 4:2:2 sampled YCbCr input data. The luminance and color difference signals are each 8 bits, scaled 0 to 255. The color difference signals are multiplexed into one 8-bit bus beginning with a Cb sample. The Y and CbCr busses may be input parallel(16-bit mode) or may be time multiplexed and input as a single bus(8-bit mode). The single bus may also contain SAV and EAV video timing reference codes(ITU-R BT.656 mode). In CCIR656 mode, the CCD controller is controlled by the SAV(Start Active Video) time code in the 8-bit pixel data. All timing information is transmitted using a 4-byte synchronization pattern. A synchronization pattern is sent immediately before and after each line during active picture and retrace. DM320 decodes SAV code and is synchronized at this timing.

The 16-bit or 8-bit YCbCr data is stored in SDRAM as 4:2:2 format. Table 22 shows data format in SDRAM. Y data typically has a range of 16 to 235, however, it is possible to subtract a DC value to Y signal.

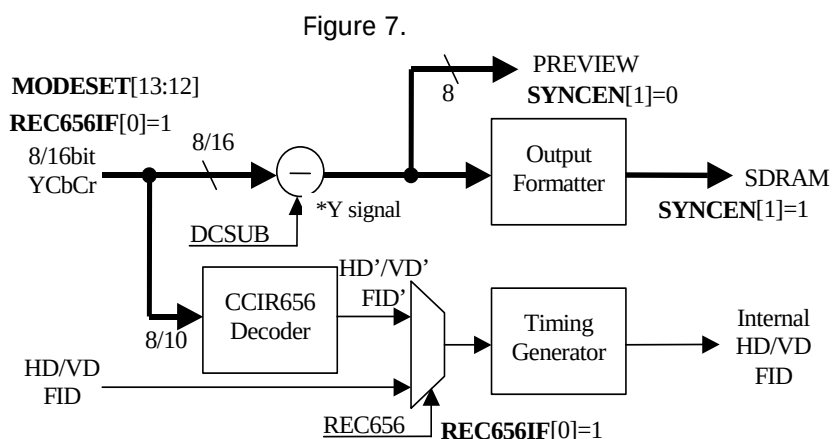


Figure 31: YCbCr signal processing

SDRAM Address	Lower word		Upper word	
	LSB(0)	MSB(15)	LSB(16)	MSB(31)
N	Cb0	Y0	Cr0	Y1
N+1	Cb2	Y2	Cr2	Y3
N+2	Cb4	Y4	Cr4	Y5

Table 22: SDRAM Data Format

9.7 Input/Output port configuration

DM320 supports several port configurations as shown in the Figure 32. Input / Output port configuration and byte swapping of input/output port or input data through is configurable by setting *CCDCFG* register.

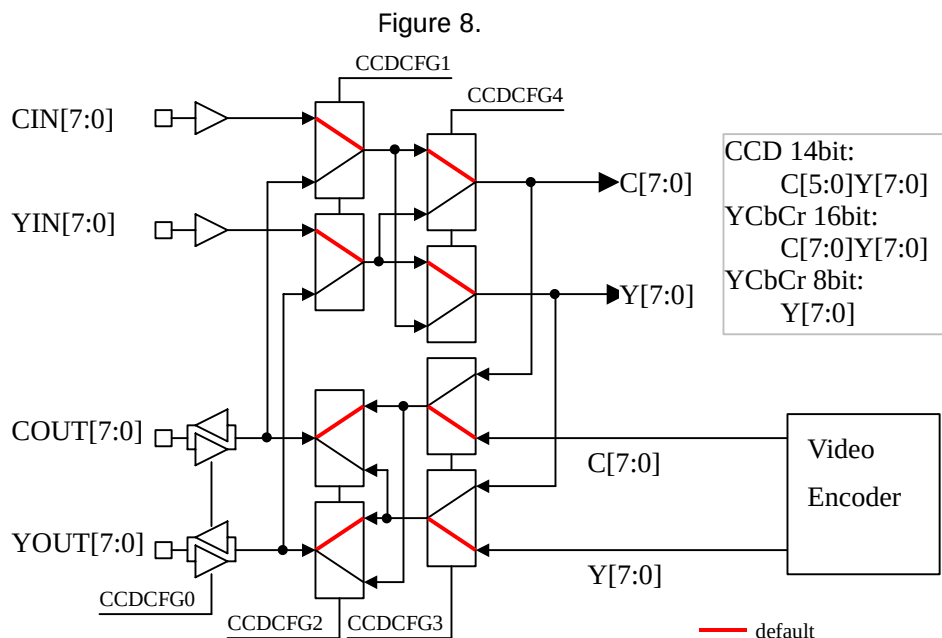


Figure 32: Video Port Configuration

9.8 CCIR-656 4:2:2 Parallel Interface

The CCIR-656 signal interface is shown in Figure 33. CCIR-656 is a specification that provides a method to transfer YCbCr-4:2:2 formatted, digital video data over an 8/10-bit wide interface. Data and timing codes are transferred over the same 8/10-bit interface.

To enable CCIR-656 mode, set R656E in REC656IF register. When in CCIR-656 mode, only the data lines and clock signal are connected between the external device and the CCDC. An NTSC/PAL decoder is an example of an external device that may be connected to the CCIR-656 interface.

Data lines Y7-Y0 are used for 8-bit YCbCr data and data lines C1-C0, Y7-Y0 are used for 10-bit YcbCr data. The video timing signals, HD, VD, and FIELD, are generated internally by the CCDC.

At the start and end of each video data block the device sends a unique timing reference code. The start code is called the Start of Active Video signal (SAV), and the end code is called the End of Active Video signal (EAV). The SAV and

EAV codes proceed and follow valid data as shown in Figure 33. HD, VD, and FIELD are generated internally by the CCDC based on the SAV and EAV codes. The delay between the end of the HD pulse and the start of valid data can be configured by setting SPH and the length of valid data can be configured by setting NPH.

Both timing reference signals, SAV and EAV, consist of a four word sequence in the following format: FF 00 00 XY, where FF 00 00 are a set preamble and the fourth word defines the field identification, the state of vertical field blanking, the state of horizontal line blanking, and error correction codes. The bit format of the fourth word is shown in Table 23 and the definitions for bits, F, V, and H, are given in Table 24. F, V, and H are used in place of the usual horizontal sync, vertical sync, and blank timing control signals. Bits P3, P2, P1, and P0 are error correction bits for F, V, and H. The relationship between F, V, and H and the error correction bits is given in Table 25. To enable error correction, set bit ECCFVH in *REC656IF*. The CCDC will automatically detect and apply error correction when ECCFVH is enabled.

Note: the CCIR-656 specification is for 525-line and 625-line, digital component video signals in compliance with CCIR Rec. 601. Please refer to CCIR document Rec. 656-1 for detailed information on the interface.

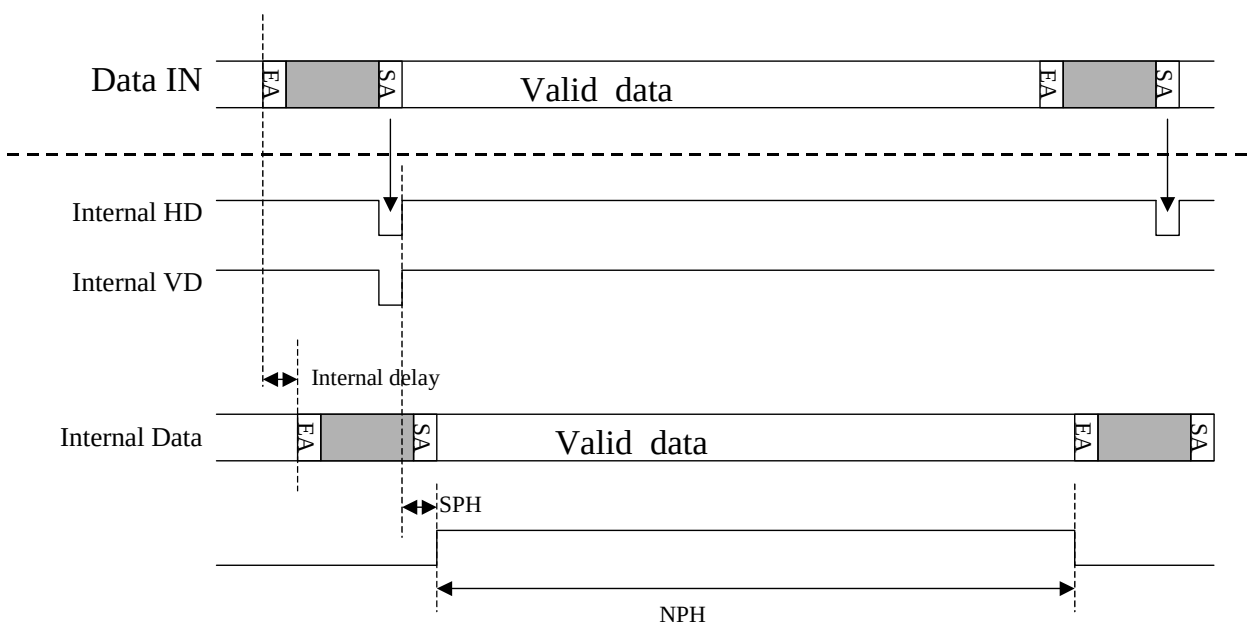


Figure 33: CCIR-656 Signal Interface

Data Bit Number	First Word	Second Word	Third Word	Fourth Word
9 (MSB)	1	0	0	1
8	1	0	0	F
7	1	0	0	V
6	1	0	0	H
5	1	0	0	P1
4	1	0	0	P2
3	1	0	0	P3
2	1	0	0	P4
1	1	0	0	0

Data Bit Number	First Word	Second Word	Third Word	Fourth Word
0	1	0	0	0

Table 23: Video Timing Reference Codes for SAV and EAV

Signal	Value	Command
F	0	Field 1
	1	Field 2
V	0	0
	1	Vertical blank
H	0	SAV
	1	EAV

Table 24: F, V, H Signal Descriptions

F	V	H	P3	P2	P1	P0
0	0	0	0	0	0	0
0	0	1	0	1	0	1
0	1	0	1	0	1	1
0	1	1	1	1	1	0
1	0	0	0	1	1	1
1	0	1	0	0	1	0
1	1	0	1	1	0	0
1	1	1	1	0	0	1

Table 25: F, V, H Error Correction Bits

When operating in CCIR-656 mode, data is stored in SDRAM according to the format shown in the Figure 34.

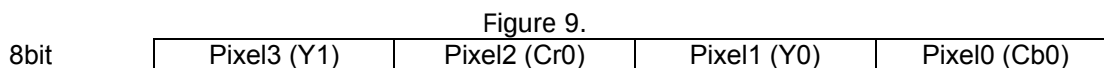


Figure 34: CCIR-656 mode data format in SDRAM

9.9 CCDC Register Update Timing

Many of the CCDC registers contain bits that are latched by the VD signal. Data written to a latched bit does not take affect until the next VD pulse is received. This allows registers that control the CCDC, such as the SDRAM write address location, HD width, VD width, etc. to be changed in between successive VD pulses without corrupting the current frame of CCD data. The CCDC registers, latched by the VD signal are, SYNCEN, LPF, bit 14 in MODESET, HDWIDTH, VDWIDTH, PPLN, LPFR, SPH, NPH, SLV0, SLV1, NLV, CULH, CULV, HSIZE, SDOFST, STADRH, STADRL, CLAMPEN bit in CLAMP and GAMMAWD.

Other CCDC registers are asynchronously set up with VD.

9.10 CCD Controller Register Map (CCDC)

Address	Register	Description
0003.0700	SYNCEN	Synchronization Enable
0003.0702	MODESET	Mode Setup
0003.0704	HDWIDTH	HD pulse width
0003.0706	VDWIDTH	VD pulse width
0003.0708	PPLN	Pixels per line
0003.070A	LPFR	Lines per frame
0003.070C	SPH	Start pixel horizontal
0003.070E	NPH	Number of pixels horizontal
0003.0710	SLV0	Start line vertical - field 0
0003.0712	SLV1	Start line vertical - field 1
0003.0714	NLV	Number of lines vertical
0003.0716	CULH	Culling - horizontal
0003.0718	CULV	Culling - vertical
0003.071A	HSIZE	Horizontal size
0003.071C	SDOFST	SDRAM Line Offset
0003.071E	STADRH	SDRAM Address #1 - high
0003.0720	STADRL	SDRAM Address #2 - low
0003.0722	CLAMP	CCD Data Clamping
0003.0724	DCSUB	DC Clamp
0003.0726	COLPTN	CCD Color Pattern
0003.0728	BLKCMP0	Black Compensation #1
0003.072A	BLKCMP1	Black Compensation #2
0003.072C	MEDFLT	CCD Median Filter
0003.072E	RYEGAIN	CCD Gain Adjustment
0003.0730	GRCYGAIN	CCD Gain Adjustment
0003.0732	GBGGAIN	CCD Gain Adjustment
0003.0734	BMGGAIN	CCD Gain Adjustment
0003.0736	OFFSET	CCD Offset Adjustment
0003.0738	OUTCLP	Output Clipping Value
0003.073A	VDINT0	VD Interrupt #0
0003.073C	VDINT1	VD Interrupt #1
0003.073E	RSV0	Reserved
0003.0740	GAMMAWD	Gamma Correction settings
0003.0742	REC656IF	CCIR 656 Control
0003.0744	CCDCFG	CCD Configuration
0003.0746	FMTCFG	CCD Formatter configuration
0003.0748	FMTSPH	CCD Formatter - Start pixel horiz
0003.074A	FMTLNH	CCD Formatter - number of pixels
0003.074C	FMTSLV	CCD Formatter - start line vertical
0003.074E	FMTLV	CCD Formatter - number of lines
0003.0750	FMTOFST	CCD Formatter - Offset
0003.0752	FMTRLN	CCD Formatter - Read out lines
0003.0754	FMTHCNT	CCD Formatter - HSYNC cycles
0003.0756	FMTPTNA	CCD Formatter - Culling patter A
0003.0758	FMTPTNB	CCD Formatter - Culling patter B

9.11 CCD Controller Registers

9.11.1 SYNCEN

CCDC Synchronization Enable

SYNCEN	0003:0700				offset: 0x00				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	WEN	VDHDEN
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W	R/W
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0	0

Bit	Name	Reset Value	R/W	Function
15:2	RSV			Reserved
1	WEN	0	R/W	<u>Data Write Enable</u> » 0: disable « » 1: enable « <i>Controls whether or not CCD raw data is written to SDRAM.</i> <i>*This bit is latched by VD.</i>
0	VDHDEN	0	R/W	<u>VD/HD Enable</u> » 0: disable « » 1: enable « <i>If VD/HD are defined as output, activates internal timing generator. If VD/HD are defined as inputs, activates internal timing generator to synchronize with VD/HD.</i>

9.11.2 MODESET

CCDC Mode Setup

MODESET	0003:0702				offset: 0x02				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	FLDSTAT	LPF	IMPMOD[1]	IMPMOD[0]	PACK8	DATSI[2]	DATSI[1]	DATSI[0]	FLDMODE	DATAPOL	EXWEN	FLDPOL	HDPOL	VDPOL	FLDOUT	VDHDOUT
R/W	R	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15	FLDSTAT	0	R	<u>Field Status</u> » 0: odd field « » 1: even field «
14	LPF	0	R/W	<u>3-tap Low-Pass (anti-aliasing) Filter</u> » 0: off « » 1: on «
13:12	IMPMOD	0	R/W	<u>Setting of data input mode</u> » 0: CCD RAW data « » 1: YCbCr 16-bit « » 2: YCbCr 8-bit « » 3: Reserved «
11	PACK8	0	R/W	<u>Pack to 8-bits/pixel</u> » 0: normal (16 bits/pixel) « » 1: pack to 8-bits/pixel «
10:8	DATSI	0	R/W	<u>CCD Data Width</u> » 000: 14-bits « » 001: 13-bits « » 010: 12-bits « » 011: 11-bits « » 100: 10-bits « » 101: 9-bits « » 110: 8-bits « » 111: Reserved «
7	FLDMODE	0	R/W	<u>Sensor Field Mode</u> » 0: non-interlaced (progressive) « » 1: interlaced «
6	DATAPOL	0	R/W	<u>CCD Data Polarity</u> » 0: normal (no change) « » 1: one's complement «
5	EXWEN	0	R/W	<u>External WEN Selection</u> » 0: do not use external WEN (Write Enable) « » 1: use external WEN (Write Enable) « To write CCD data to SDRAM
4	FLDPOL	0	R/W	<u>Field Indicator Polarity</u> » 0: positive « » 1: negative «
3	HDPOL	0	R/W	<u>HD Sync Polarity</u> » 0: positive « » 1: negative «
2	VDPOL	0	R/W	<u>VD Sync Polarity</u> » 0: positive « » 1: negative «
1	FLDOUT	0	R/W	<u>Field ID Direction</u> » 0: input « » 1: output «
0	VDHDOUT	0	R/W	<u>VD/HD Sync Direction</u> » 0: input « » 1: output «

9.11.3 HDWIDTH

CCDC HD pulse width

[illegible][illegible]

9.11.4 VDWIDTH

CCDC VD pulse width

[illegible][illegible]

9.11.5 PPLN

CCDC Pixels per line

PPLN	0003:0708			offset: 0x08												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	PPLN[15]	PPLN[14]	PPLN[13]	PPLN[12]	PPLN[11]	PPLN[10]	PPLN[9]	PPLN[8]	PPLN[7]	PPLN[6]	PPLN[5]	PPLN[4]	PPLN[3]	PPLN[2]	PPLN[1]	PPLN[0]	
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:0	PPLN	0	R/W	Pixels per line - number of pixel clock periods in one line HD period = PPLN+1 pixel clocks PPLN is not used when HD and VD are inputs, i.e when VDHDOUT in MODESET is cleared to '0'. *This bit field is latched by VD.

9.11.6 LPFR

CCDC Lines per frame

LPFR	0003:070A			offset: 0x0A			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	HLPFR[15]	HLPFR[14]	HLPFR[13]	HLPFR[12]	HLPFR[11]	HLPFR[10]	HLPFR[9]	HLPFR[8]	HLPFR[7]	HLPFR[6]	HLPFR[5]	HLPFR[4]	HLPFR[3]	HLPFR[2]	HLPFR[1]	HLPFR[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:0	HLPFR	0	R/W	Half lines per filed or frame - sets number of half lines per frame or field. VD period = (HLPFR+1)/2 lines LPFR is not used when HD and are inputs, i.e when VDHDOUT in MODESET is cleared to '0'. *This bit field is latched by VD.

9.11.7 SPH

CCDC Start pixel horizontal

[illegible][illegible]

9.11.8 NPH

CCDC Number of pixels horizontal

[illegible][illegible]

9.11.9 SLV0

CCDC Start line vertical - field 0

SLV0	0003:0710			offset:	0x10		default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	SLV0[14]	SLV0[13]	SLV0[12]	SLV0[11]	SLV0[10]	SLV0[9]	SLV0[8]	SLV0[7]	SLV0[6]	SLV0[5]	SLV0[4]	SLV0[3]	SLV0[2]	SLV0[1]	SLV0[0]
R/W	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15	RSV			Reserved
14:0	SLV0	0	R/W	Start Line, Vertical (Field 0) Sets line at which data output to SDRAM will begin, measured from the start of VD. <i>*This bit field is latched by VD.</i>

9.11.10 SLV1

CCDC Start line vertical - field 1

SLV1	0003:0712			offset:	0x12		default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	SLV1[14]	SLV1[13]	SLV1[12]	SLV1[11]	SLV1[10]	SLV1[9]	SLV1[8]	SLV1[7]	SLV1[6]	SLV1[5]	SLV1[4]	SLV1[3]	SLV1[2]	SLV1[1]	SLV1[0]
R/W	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15	RSV			Reserved
14:0	SLV1	0	R/W	Start Line, Vertical (Field 1) Sets line at which data output to SDRAM will begin, measured from the start of VD. <i>*This bit field is latched by VD.</i>

9.11.11 NLV

CCDC Number of lines vertical

NLV	0003:0714			offset:	0x14		default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	NLV[14]	NLV[13]	NLV[12]	NLV[11]	NLV[10]	NLV[9]	NLV[8]	NLV[7]	NLV[6]	NLV[5]	NLV[4]	NLV[3]	NLV[2]	NLV[1]	NLV[0]
R/W	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15	RSV			Reserved
14:0	NLV	0	R/W	Number of lines, vertical Sets number of vertical lines that will be output to SDRAM. The number of lines output to SDRAM = (NLV + 1). <i>*This bit field is latched by VD.</i>

9.11.12 CULH

CCDC Culling - horizontal

CULH	0003:0716				offset: 0x16				default: 0xFFFF							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	CULHEVN[7]	CULHEVN[6]	CULHEVN[5]	CULHEVN[4]	CULHEVN[3]	CULHEVN[2]	CULHEVN[1]	CULHEVN[0]	CULHODD[7]	CULHODD[6]	CULHODD[5]	CULHODD[4]	CULHODD[3]	CULHODD[2]	CULHODD[1]	CULHODD[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

Bit	Name	Reset Value	R/W	Function
15:8	CULHEVN	255	R/W	Horizontal Culling Pattern for Even Line when writing to SDRAM, 8-bit mask: 0: cull, 1:retain <i>LSB is first pixel, MSB is 8th pixel, then pattern repeats.</i> <i>*This bit field is latched by VD.</i>
7:0	CULHODD	255	R/W	Horizontal Culling Pattern for Odd Line when writing to SDRAM, 8-bit mask: 0: cull, 1:retain <i>LSB is first pixel, MSB is 8th pixel, then pattern repeats.</i> <i>*This bit field is latched by VD.</i>

9.11.13 CULV

CCDC Culling - vertical

CULV	0003:0718				offset: 0x18				default: 0x00FF							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	CULV[7]	CULV[6]	CULV[5]	CULV[4]	CULV[3]	CULV[2]	CULV[1]	CULV[0]
R/W	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	-	-	-	1	1	1	1	1	1	1	1

Bit	Name	Reset Value	R/W	Function
15:8	RSV			Reserved
7:0	CULV	255	R/W	Vertical Culling Pattern, 8-bit mask: 0: cull, 1:retain <i>LSB is first line, MSB is 8th line, then pattern repeats.</i> <i>*This bit field is latched by VD.</i>

9.11.14 HSIZE

CCDC Horizontal size

HSIZE	0003:071A				offset: 0x1A				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	LNOFST[11]	LNOFST[10]	LNOFST[9]	LNOFST[8]	LNOFST[7]	LNOFST[6]	LNOFST[5]	LNOFST[4]	LNOFST[3]	LNOFST[2]	LNOFST[1]	LNOFST[0]
R/W	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:12	RSV			Reserved
11:0	LNOFST	0	R/W	Address offset for each line. Sets size of line in SDRAM, units: 32 bytes Either 16 or 32 pixels depending on setting of PACK8 <i>*This bit field is latched by VD.</i>

9.11.15 SDOFST

CCDC SDRAM Line Offset

SDOFST	0003:071C			offset: 0x1C			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	FIINV	FOFST[1]	FOFST[0]	LOFST0[2]	LOFST0[1]	LOFST0[0]	LOFST1[2]	LOFST1[1]	LOFST1[0]	LOFST2[2]	LOFST2[1]	LOFST2[0]	LOFST3[2]	LOFST3[1]	LOFST3[0]
R/W	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15	RSV			Reserved
14	FIINV	0	R/W	Field identification signal inverse » 0: Non inverse « » 1: Inverse «
13:12	FOFST	0	R/W	Line offset value of odd field (FID = 1) » 00: +1 line « » 01: +2 line « » 10: +3 line « » 11: +4 line «
11:9	LOFST0	0	R/W	Line offset values of even line and even field (FID = 0) » 000: +1 line « » 001: +2 lines « » 010: +3 lines « » 011: +4 lines « » 100: -1 line « » 101: -2 lines « » 110: -3 lines « » 111: -4 lines «
8:6	LOFST1	0	R/W	Line offset values of odd line and even field (FID = 0) » 000: +1 line « » 001: +2 lines « » 010: +3 lines « » 011: +4 lines « » 100: -1 line « » 101: -2 lines « » 110: -3 lines « » 111: -4 lines «
5:3	LOFST2	0	R/W	Line offset values of even line and odd field (FID = 1) » 000: +1 line « » 001: +2 lines « » 010: +3 lines « » 011: +4 lines « » 100: -1 line « » 101: -2 lines « » 110: -3 lines « » 111: -4 lines «
2:0	LOFST3	0	R/W	Line offset values of odd line and odd field (FID = 1) » 000: +1 line « » 001: +2 lines « » 010: +3 lines « » 011: +4 lines « » 100: -1 line « » 101: -2 lines « » 110: -3 lines « » 111: -4 lines «

9.11.16 STADRH

CCDC SDRAM high address

STADRH	0003:071E			offset: 0x1E			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	ADRH[6]	ADRH[5]	ADRH[4]	ADRH[3]	ADRH[2]	ADRH[1]	ADRH[0]
R/W	-	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	-	-	-	-	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:7	RSV			Reserved
6:0	ADRH	0	R/W	Upper 7 bits of the SDRAM starting address for CCDC output The address is specified offset from the SDRAM base address in units of 32 bytes. <i>*This bit field is latched by VD.</i>

9.11.17 STADRL

CCDC SDRAM low address

STADRL	0003:0720			offset: 0x20			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	ADRL[15]	ADRL[14]	ADRL[13]	ADRL[12]	ADRL[11]	ADRL[10]	ADRL[9]	ADRL[8]	ADRL[7]	ADRL[6]	ADRL[5]	ADRL[4]	ADRL[3]	ADRL[2]	ADRL[1]	ADRL[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:0	ADRL	0	R/W	Lower 16 bits of the SDRAM starting address for CCDC output The address is specified offset from the SDRAM base address in units of 32 bytes. <i>*This bit field is latched by VD.</i>

9.11.18 CLAMP

CCD Data Clamping

CLAMP	0003:0722			offset: 0x22			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	CLAMPEN	OBSLEN	OBSLN	OBST[12]	OBST[11]	OBST[10]	OBST[9]	OBST[8]	OBST[7]	OBST[6]	OBST[5]	OBST[4]	OBST[3]	OBST[2]	OBST[1]	OBST[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15	CLAMPEN	0	R/W	Clamp Enable: » 0: disable « » 1: enable « Enables clamping of CCD data based on the calculated average of Optical Black Samples. *This bit is latched by VD.
14	OBSLEN	0	R/W	<u>Optical Black Sample Length:</u> » 0: 8 pixels « » 1: 16 pixels « Number of Optical Black Sample pixels per line to include in the average calculation.
13	OBSLN	0	R/W	<u>Optical Black Sample Lines:</u> » 0: 8 lines « » 1: 16 lines « Number of Optical Black Sample lines to include in the average calculation.
12:0	OBST	0	R/W	<u>Start Pixel of Optical Black Samples</u> Start pixel position of Optical Black Samples, specified from the start of HD in pixel clocks.

9.11.19 DCSUB

CCDC DC Clamp

[illegible][illegible]

9.11.20 COLPTN

CCD Color Pattern

COLPTN	0003:0726			offset: 0x26			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	CPELPC0[1]	CPELPC0[0]	CPELPC1[1]	CPELPC1[0]	CPELPC2[1]	CPELPC2[0]	CPELPC3[1]	CPELPC3[0]	CPOLPC0[1]	CPOLPC0[0]	CPOLPC1[1]	CPOLPC1[0]	CPOLPC2[1]	CPOLPC2[0]	CPOLPC3[1]	CPOLPC3[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:14	CPELPC0	0	R/W	Color Pattern for Even Line, Pixel counter = 0 » 00: R/Ye « » 01: Gr/Cy « » 10: Gb/G « » 11: B/Mg «
13:12	CPELPC1	0	R/W	Color Pattern for Even Line, Pixel counter = 1 » 00: R/Ye « » 01: Gr/Cy « » 10: Gb/G « » 11: B/Mg «
11:10	CPELPC2	0	R/W	Color Pattern for Even Line, Pixel counter = 2 » 00: R/Ye « » 01: Gr/Cy « » 10: Gb/G « » 11: B/Mg «
9:8	CPELPC3	0	R/W	Color Pattern for Even Line, Pixel counter = 3 » 00: R/Ye « » 01: Gr/Cy « » 10: Gb/G « » 11: B/Mg «
7:6	CPOLPC0	0	R/W	Color Pattern for Odd Line, Pixel counter = 0 » 00: R/Ye « » 01: Gr/Cy « » 10: Gb/G « » 11: B/Mg «
5:4	CPOLPC1	0	R/W	Color Pattern for Odd Line, Pixel counter = 1 » 00: R/Ye « » 01: Gr/Cy « » 10: Gb/G « » 11: B/Mg «
3:2	CPOLPC2	0	R/W	Color Pattern for Odd Line, Pixel counter = 2 » 00: R/Ye « » 01: Gr/Cy « » 10: Gb/G « » 11: B/Mg «
1:0	CPOLPC3	0	R/W	Color Pattern for Odd Line, Pixel counter = 3 » 00: R/Ye « » 01: Gr/Cy « » 10: Gb/G « » 11: B/Mg «

9.11.21 BLKCMP0

Black Compensation #0

[illegible][illegible]

9.11.22 BLKCMP1

Black Compensation #1

[illegible][illegible]

9.11.23 MEDFILT

CCD Median Filter

[illegible][illegible]

9.11.24 RYE GAIN

CCD RYE Gain Adjustment

RYEGAIN	0003:072E				offset: 0x2E		default: 0x0100									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	GAIN[10]	GAIN[9]	GAIN[8]	GAIN[7]	GAIN[6]	GAIN[5]	GAIN[4]	GAIN[3]	GAIN[2]	GAIN[1]	GAIN[0]
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	0	0	1	0	0	0	0	0	0	0	0

[illegible]

9.11.25 GRCY GAIN

CCD GRCY Gain Adjustment

GRCYGAIN	0003:0730	offset: 0x30	default: 0x0100
----------	-----------	--------------	-----------------

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	GAIN[10]	GAIN[9]	GAIN[8]	GAIN[7]	GAIN[6]	GAIN[5]	GAIN[4]	GAIN[3]	GAIN[2]	GAIN[1]	GAIN[0]
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	0	0	1	0	0	0	0	0	0	0	0

[illegible]

9.11.26 GBG GAIN

CCD GBG Gain Adjustment

GBGGAIN	0003:0732				offset: 0x32				default: 0x0100							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	GAIN[10]	GAIN[9]	GAIN[8]	GAIN[7]	GAIN[6]	GAIN[5]	GAIN[4]	GAIN[3]	GAIN[2]	GAIN[1]	GAIN[0]
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	0	0	1	0	0	0	0	0	0	0	0

[illegible]

9.11.27 BMG GAIN

CCD BMG Gain Adjustment

BMGGAIN	0003:0734			offset: 0x34			default: 0x0100									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	GAIN[10]	GAIN[9]	GAIN[8]	GAIN[7]	GAIN[6]	GAIN[5]	GAIN[4]	GAIN[3]	GAIN[2]	GAIN[1]	GAIN[0]
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	0	0	1	0	0	0	0	0	0	0	0

[illegible]

9.11.28 OFFSET

CCD Offset Adjustment

[illegible][illegible]

9.11.29 OUTCLIP

CCDC Output Clipping Value

[illegible][illegible]

9.11.30 VDINT0

CCDC VD Interrupt #0

[illegible][illegible]

9.11.31 VDINT1

CCDC VD Interrupt #1

[illegible][illegible]

9.11.32 GAMMAWD

CCDC CCD Gamma

GAMMAWD				0003:0740				offset: 0x40								default: 0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	RSV	RSV	MFIL1[1]	MFIL1[0]	MFIL2[1]	MFIL2[0]	RSV	RSV	CFAP	GWDI[2]	GWDI[1]	GWDI[0]	H3ATBL	CCDTBL			
R/W	-	-	-	-	R/W	R/W	R/W	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W			
Default	-	-	-	-	0	0	0	0	-	-	0	0	0	0	0	0			

Bit	Name	Reset Value	R/W	Function
15:12	RSV			Reserved
11:10	MFIL1	0	R/W	<u>Median Filter for Preview Engine</u> » 0: No Median filter « » 1: Average filter « » 2: Median filter « » 3: Reserved «
9:8	MFIL2	0	R/W	<u>Median Filter for 3A, CCD capture</u> » 0: No Median filter « » 1: Average filter « » 2: Median filter « » 3: Reserved «
7:6	RSV			Reserved
5	CFAP	0	R/W	<u>CFA Pattern</u> » 0: Mozaic « » 1: Stripe «
4:2	GWDI	0	R/W	<u>Gamma Width Input (A-LAW table)</u> » 000: bits 13-4 « » 001: bits 12-3 « » 010: bits 11-2 « » 011: bits 10-1 « » 100: bits 9-0 « Others: Reserved
1	H3ATBL	0	R/W	<u>Apply Gamma (A-LAW) to CCDC data sent to Hardware 3A</u> » 0: disable « » 1: enable «
0	CCDTBL	0	R/W	<u>Apply Gamma (A-LAW) to CCDC data saved to SDRAM</u> » 0: disable « » 1: enable «

9.11.33 REC656IF

CCDC REC 656 standard interface

REC656IF	0003:0742			offset: 0x42			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	ECCFVH	R656ON
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W	R/W
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0	0

Bit	Name	Reset Value	R/W	Function
15:2	RSV			Reserved
1	ECCFVH	0	R/W	<u>FVH Error Correction Enable</u> » 0: disable « » 1: enable «
0	R656ON	0	R/W	<u>REC656 Interface Enable</u> » 0: disable « » 1: enable «

9.11.34 CCDCFG

CCD Configuration

CCDCFG	0003:0744				offset: 0x44				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	VDLC	MSBINVO	MSBINVI	BSWD	Y8POS	EXTRG	TRGSEL	WENLOG	FIDMD[1]	FIDMD[0]	BW656	YCINSWP	RSV	YCOUTSWP	IDS	RSV
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	-	R/W	R/W	-
Default	0	0	0	0	0	0	0	0	0	0	0	0	-	0	0	-

Bit	Name	Reset Value	R/W	Function
15	VDLC	0	R/W	<u>Enable synchronizing function registers on VSYNC</u> » 0: Synchronized on VSYNC « » 1: Not Synchronized on VSYNC «
14	MSBINVO	0	R/W	<u>MSB of Chroma signal output inverted</u> » 0: normal « » 1: MSB inverted «
13	MSBINVI	0	R/W	<u>MSB of Chroma input signal stored to SDRAM inverted</u> » 0: normal « » 1: MSB inverted «
12	BSWD	0	R/W	<u>Byte Swap Data stored to SDRAM</u> » 0: normal « » 1: Swap Bytes «
11	Y8POS	0	R/W	<u>Location of Y signal when YCbCr 8bit data is input</u> » 0: even pixel « » 1: odd pixel «
10	EXTRG	0	R/W	<u>External Trigger</u> » 0: Disable « » 1: Enable «
9	TRGSEL	0	R/W	<u>Signal that initializes SDRAM address when EXTRG = 1</u> » 0: WEN bit (SYNCEN register) « » 1: FID input port «
8	WENLOG	0	R/W	<u>Specifies CCD valid area</u> » 0: Internal valid signal & WEN signal is ANDed logically « » 1: Internal valid signal & WEN signal is Ored logically «
7:6	FDMD	0	R/W	<u>Setting of FID detection function</u> » 00: FID signal is latched at the VSYNC timing « » 01: FID signal is not latched « » 10: FID signal is latched at edge of VD « » 11: FID signal is latched based on phase of VD and HD«
5	BW656	0	R/W	<u>The data width in CCIR656 input mode</u> » 0: 8-bits « » 1: 10-bits «
4	YCINSWP	0	R/W	<u>Y input (YIN[7:0]) and C input (CIN[7:0]) are swapped</u> » 0: YIN[7:0] = Y signal / CIN[7:0] = C signal « » 1: YIN[7:0] = C signal / CIN[7:0] = Y signal «
3	RSV			<u>Reserved</u>
2	YCOUTSWP	0	R/W	<u>Y output (YOUT[7:0]) and C output (COUT[7:0]) are swapped</u> » 0: YOUT[7:0] = Y signal / COUT[7:0] = C signal « » 1: YOUT[7:0] = C signal / COUT[7:0] = Y signal «
1	DS	0	R/W	<u>Selection of Y / C signal</u> » 0: YIN[7:0] & CIN[7:0] « » 1: YOUT[7:0] & COUT[7:0] «
0	RSV			<u>Reserved</u>

9.11.35 FMTCFG

CCDC FMT Configuration

FMTCFG	0003:0746		offset: 0x46		default: 0x0000											
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	MOFST4[1]	MOFST4[0]	MOFST3[1]	MOFST3[0]	MOFST2[1]	MOFST2[0]	MOFST1[1]	MOFST1[0]	LNUM[1]	LNUM[0]	PNUM[1]	PNUM[0]	LNALT	FMTEN
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
13:12	MOFST4	0	R/W	Offset value of write address at fourth line
11:10	MOFST3	0	R/W	Offset value of write address at third line
9:8	MOFST2	0	R/W	Offset value of write address at second line
7:6	MOFST1	0	R/W	Offset value of write address at first line
				<u>Number of lines</u>
5:4	LNUM	0	R/W	» 0: 1 line « » 1: 2 lines « » 2: 3 lines « » 3: 4 lines «
				<u>Interval of i/p data format (Number of pixels)</u>
3:2	PNUM	0	R/W	» 0: 1 pixel « » 1: 2 pixels « » 2: 3 pixels « » 3: 4 pixels «
				<u>Line Alternating mode</u>
1	LNALT	0	R/W	» 0: Normal mode « » 1: Line alternating mode «
				<u>CCD Formatter enable</u>
0	FMTEN	0	R/W	» 0: Off « » 1: On «

9.11.36 FMTSPH

CCDC FMT Start pixel horizontal

[illegible][illegible]

9.11.37 FMTLNH

CCDC FMT Numer of pixel horizontal

[illegible][illegible]

9.11.38 FMTSLV

CCDC FMT Start line vertical

[illegible][illegible]

9.11.39 FMTLNV

CCDC FMT Number line vertical

[illegible][illegible]

9.11.40 FMTOFST

CCDC FMT Offset adjustment

[illegible][illegible]

9.11.41 FMTRLEN

CCDC FMT Number of read-out lines

[illegible][illegible]

9.11.42 FMTHCNT

CCDC FMT Horizontal sync interval

[illegible][illegible]

9.11.43 FMTPTNA

CCDC FMT Culling pattern

[illegible][illegible]

9.11.44 FMTPTNB

CCDC FMT Culling pattern

FMTPTNB	0003:0758				offset: 0x58				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	CUL4OL[3]	CUL4OL[2]	CUL4OL[1]	CUL4OL[0]	CUL3OL[3]	CUL3OL[2]	CUL3OL[1]	CUL3OL[0]	CUL2OL[3]	CUL2OL[2]	CUL2OL[1]	CUL2OL[0]	CUL1OL[3]	CUL1OL[2]	CUL1OL[1]	CUL1OL[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:12	CUL4OL	0	R/W	Culling pattern for line - 4 in odd line
11:8	CUL3OL	0	R/W	Culling pattern for line - 3 in odd line
7:4	CUL2OL	0	R/W	Culling pattern for line - 2 in odd line
3:0	CUL1OL	0	R/W	Culling pattern for line - 1 in odd line

10 Preview Engine

10.1 Introduction

The Preview Engine is a programmable hardware video processing module that generates image data in YCbCr-4:2:2 format from raw CCD data. The Preview Engine can also be configured to operate in a resize only mode, which allows the YCbCr-4:2:2 format data to be resized without applying the processing of every module in the Preview Engine.

The Preview Engine has the following features:

- Supports Bayer, Sony VGA, and Foveon style patterns
- Dual data input ports (CCD controller and SDRAM controller)
- Noise filter with noise coring
- Digital gain
- White balance
- Two steps smoothing
- Horizontal and vertical seamless down sampling (x1/64~x1)
- Horizontal and vertical seamless zoom (x1~x4),
- Black adjustment
- RGB2RGB blending matrix
- Gamma correction
- Luminance offset, brightness, and contrast
- Chroma offset & suppression
- Programmable RGB-to-YCbCr conversion coefficients
- One-shot preview
- Inverse A-Law table
- Performs YCbCr-4:2:2 image resizing
- Support 16bit data bus SDRAM
- Histogram functionalities

Figure 35 shows the several modules that are used to process CCD raw data into YCbCr-4:2:2 data. These modules are described in detail in the later sections.

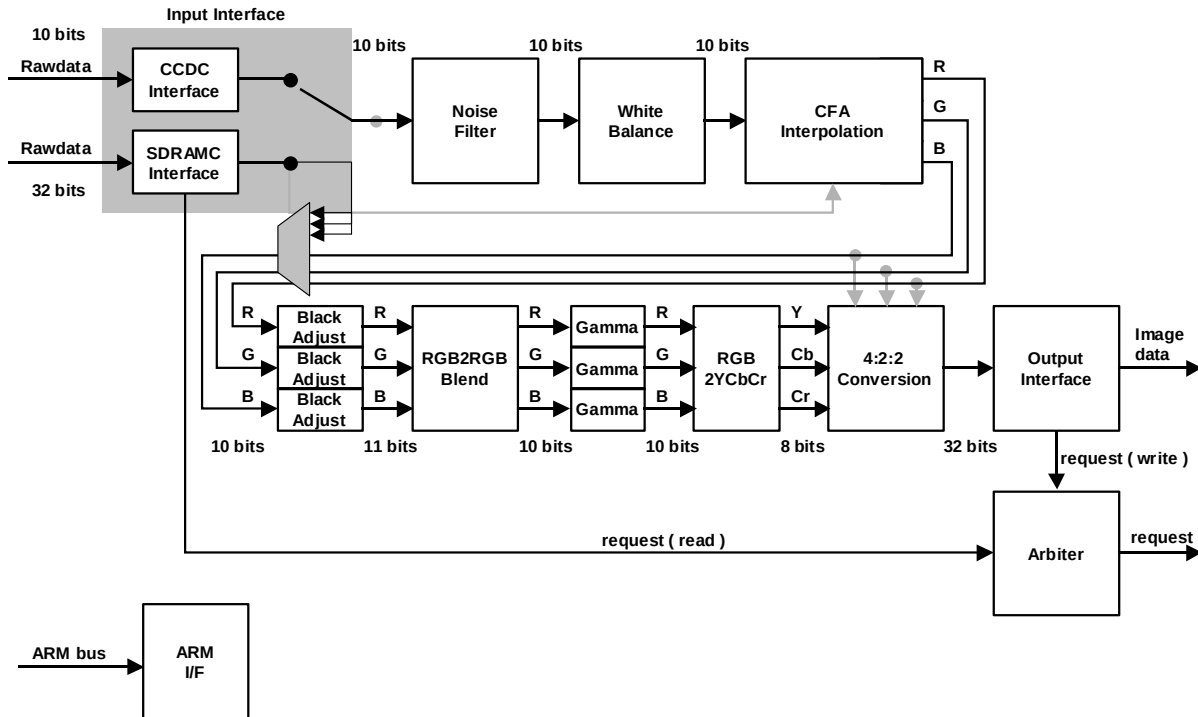


Figure 35: Block Diagram of the Preview Engine

10.2 Functional description

This section provides a detailed description of the modules that implement the Preview Engine's image pipeline. The module converts raw CCD data to YCbCr-4:2:2 data. The raw data input can come from the CCDC or from SDRAM. The YCbCr-4:2:2 output is always written to SDRAM.

See section 10.5 for description of fractional notation used in this chapter.

10.2.1 Color filter Array and Sensor Formats

A CCD sensor generates a color image by measuring the intensity of light that passes through a color filter mask. This color filter mask is called a Color Filter Array (CFA). The masking pattern of the CFA array as well as the filter color primaries vary between different manufactures. In digital camera applications, the filter color primaries are often components of the RGB or Complementary color space.

The Preview engine supports Sony and Foveon sensor formats in addition to the conventional Bayer pattern format. Figure 36 and Figure 37 shows the Bayer and Foveon pattern respectively. The configuration of the position of each RGB

or complementary color component is done by PVSET2 register according to the CFA pattern used by the CCD module.

Note: DM320 does not support Sony VGA sensors in SDRAM input mode.

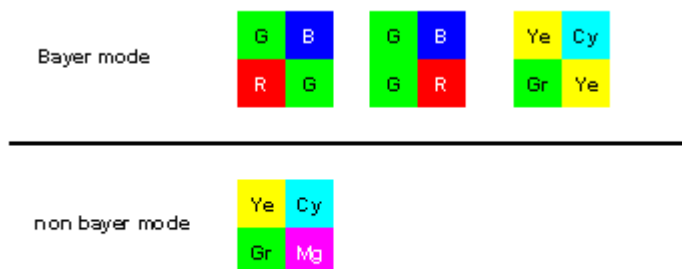


Figure 36: Conventional 2x2 CFA format



Figure 37: Foveon Sensor format

10.2.2 Noise Filter Module

The noise filter is the first image-processing step in the preview engine. It is useful to reduce an impulse noise present in the raw data from the CCD image sensor using a noise coring technique. When the NFEN bit in the NFILT register is 1, this noise filter is enabled and NFRT[1:0] bits in the NFILT register controls the gain of the high pass filter

10.2.3 White Balance Module

The preview engine white balance module has two gain adjusters, a digital gain adjuster and a white balance adjuster. In the digital gain adjuster, the raw data is multiplied by a fixed value gain regardless of the color pixel to be processed. This would affect the brightness of the image. The digital gain is configured in the DGAIN register. The format of DGAIN value is U10Q8.

In the white balance gain adjuster, the raw data is multiplied by a selected gain corresponding to the color of the processed pixel. While-balance parameters are configured in WBGAIN0 and WBGAIN1 registers. The format of WBGAINx registers is U8Q6.

Figure 38 shows the block diagram of the white balance module. It is composed of two multipliers for the digital gain and the white balance gain.

Note: The selection of the white balance gain depends on the type of sensor (conventional/converted Bayer or Foveon) and the color of the current pixel to be operated on.

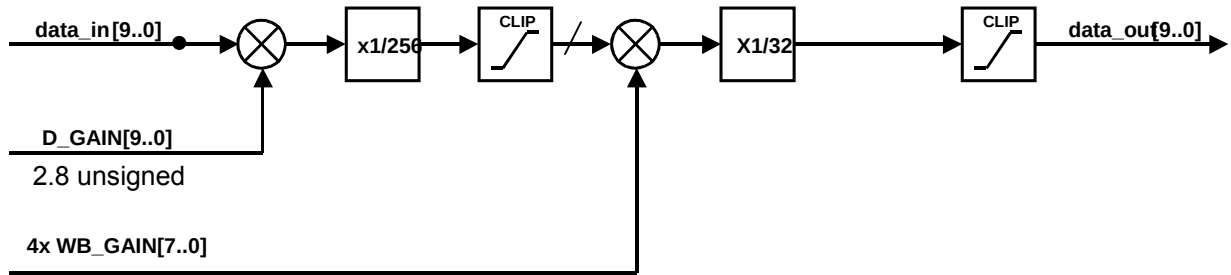


Figure 38: White Balance Module Block Diagram

10.2.4 CFA Interpolation

The Preview Engine's CFA interpolation modules perform functions such as, horizontal and vertical interpolation, smoothing, horizontal and vertical resizing, color adjustment and conversion from complementary to RGB color space. *Figure 39* shows a block diagram of the CFA interpolation module in normal mode. The dark lines shows the data path taken during normal mode.

When the Preview Engine is operating in resize only mode, some of the CFA interpolation modules are bypassed. In resize only mode, only smoothing and horizontal and vertical resizing are applied to the data. The data path taken in resize only mode is shown in shown by the dark lines in *Figure 40*.

The horizontal interpolation sub-module applies horizontal interpolation using a two tap or a five-tap interpolation filter.

A smoother sub-module performs a selectable low pass filter, which has 2 different levels of a bandwidth.

The horizontal re-sampler sub-module applies a seamless horizontal down-sampling or up-sampling (zoom) by x4 over-sampling.

The vertical interpolation and re-sampler sub-module performs a vertical interpolation and a seamless vertical down-sampling or up-sampling by x4 over-sampling using two line memories.

The color selector sub-module extracts the appropriate color from the 4 inputs of the vertical interpolation and re-sampler sub-module.

The color adjustment sub-module executes color adjustment on the interpolated data. This module is bypassed when the Preview Engine is in RGB CCD mode.

The comp2RGB conversion sub-module converts the data from the complementary color space to the RGB color space.

The output of the CFA interpolation module is RGB data regardless of a type of the color space used by the CCD imager.

In the case of the Foveon sensor, CFA interpolation is not necessary. Only the horizontal and vertical zoom circuitry in the CFA interpolation module should be active for the Foveon sensor. It is assumed that the Foveon sensor gives serial RGB data. Therefore, the RGB data needs to be buffered at the output of the CFA interpolation for feeding to the black adjustment module. In other words, all the stages following the CFA interpolation for the Foveon sensor will only

operate on one unique RGB data every three pixel clock cycles. To minimize power consumption, the buffer should transmit the same RGB pixel values for 3 consecutive pixel clock cycles. At the final output stage, the 2/3rd repetitive data need to be thrown away before writing to the SDRAM.

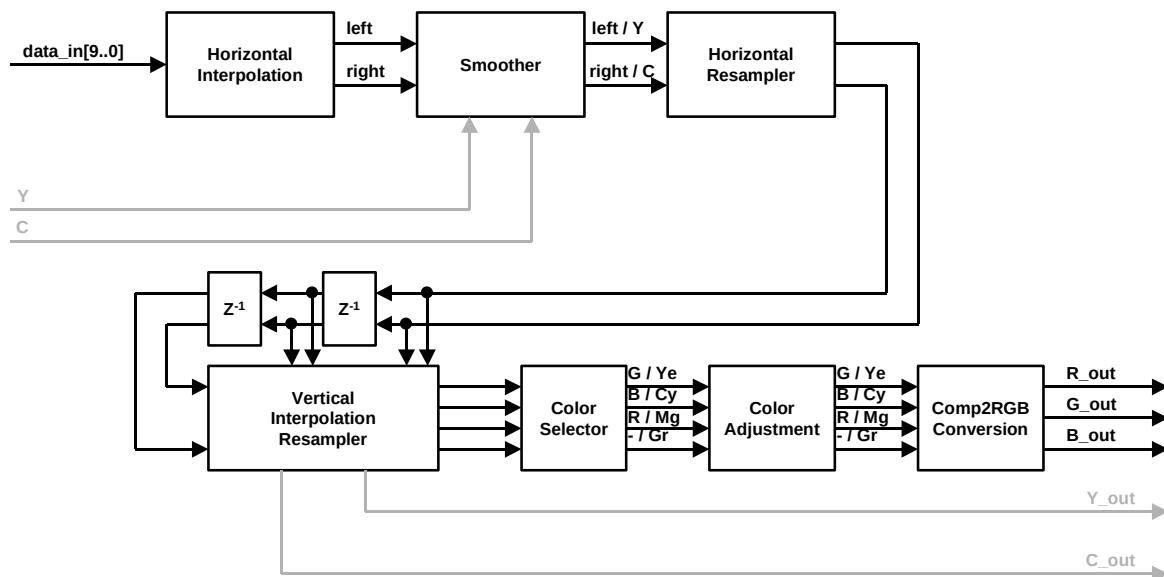


Figure 39: CFA Interpolation module (Normal Mode)

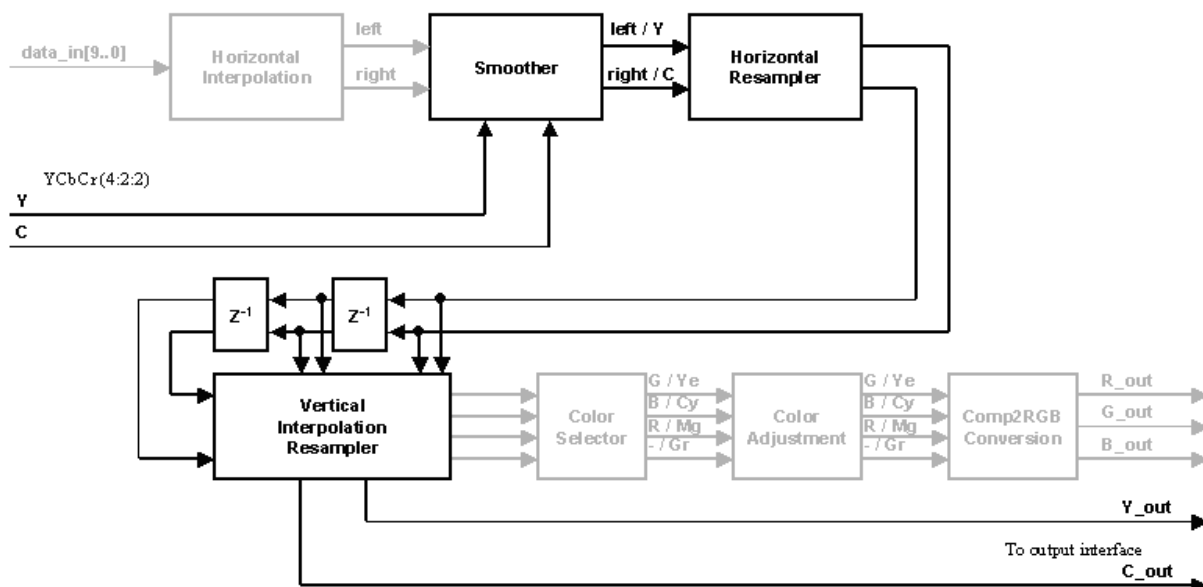


Figure 40 CFA Interpolation module (Resize only Mode)

10.2.4.1 Horizontal Interpolation

A horizontal interpolation sub-module performs a horizontal interpolation with a 2-tap or a 5-tap interpolation filter. The 2 taps interpolation filter uses an average of the next pixels on the left and right side of the pixel to be interpolated. On the other hand, the 5-tap interpolation filter utilizes the information of another color on the processed line so that a false color due to this interpolation process can be reduced effectively. Configure HINT bit in PVSET2 register for 2-tap or 5-tap interpolation.

10.2.4.2 Smoother

A smoother sub-module is an anti-aliasing filter that is composed of two kinds of FIR filters. By combining the two filters, one of two levels of horizontal low pass filter can be performed. The smoother sub-module (in Bayer mode) is composed of the two FIR filters and an output selector. The SMEN bit, SMLVL bit in the SMTH register determine the output.

10.2.4.3 Horizontal Re-sampler

A horizontal re-sampler sub-module performs a seamless horizontal down-sampling or up-sampling (zoom). This seamless re-sampling processing is done by a bi-linear interpolation of data from the smoother sub-module. *Figure 41* shows an example of this re-sampling process (x2 zoom in Bayer mode).

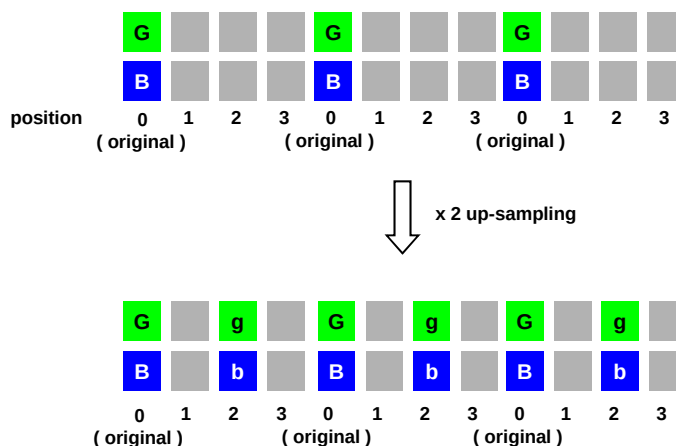


Figure 41: Example of the resampling (x2 up-sampling)

See section 10.4.3 for details about setting resizing registers.

10.2.4.4 Vertical Interpolation and Re-sampler

The vertical interpolation and re-sampler sub-module performs vertical interpolation and seamless vertical down-sampling or up-sampling.

Similar to horizontal resampler module, vertical resampling is done by a bi-linear interpolation of a data from the horizontal resampler sub-module and two line

memories. And this sub-module executes two different calculations depending on the color pattern of CCD image sensor. *Figure 36* shows an example of two types of the CFA formats.

If the CFA pattern is of the type that has one color component in both lines, then this sub-module operates in bayer mode. Otherwise, this sub-module operates in non-bayer mode. In bayer mode, the vertical interpolation and resampling processing is based on the main color (G or Ye). As shown in *Figure 42*, in bayer mode one main color (G or Ye in this figure) exists in every line, and the other colors are calculated based on the main color.

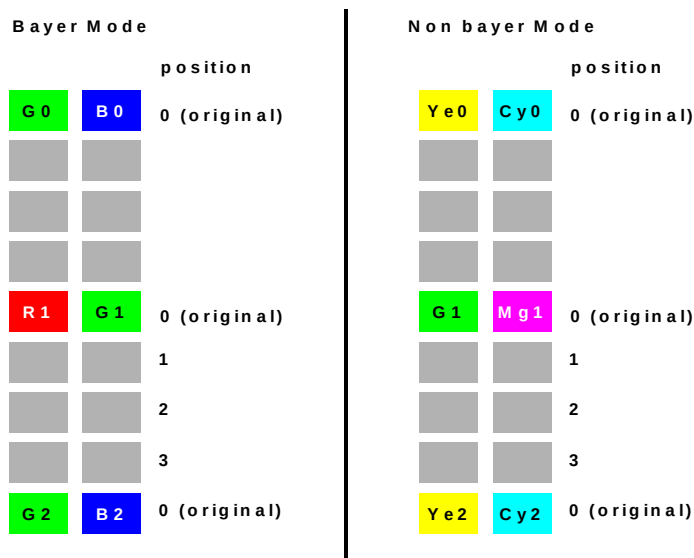


Figure 42: Comparison of inputs in Bayer mode and in non-Bayer mode

Note: Up-sampling is only possible when operating in SDRAM mode, i.e. when bit INMODE of PVSET1 register is set to 1.

10.2.4.5 Color Selector

The color selector sub-module extracts an appropriate color from the four input signals that the vertical interpolation and re-sampler sub-module generates. *Figure 43* shows a block diagram of the color selector sub-module. This extractor for each color selects an appropriate signal automatically using the sync signal and CFA [7:0] bits in the PVSET2 register.

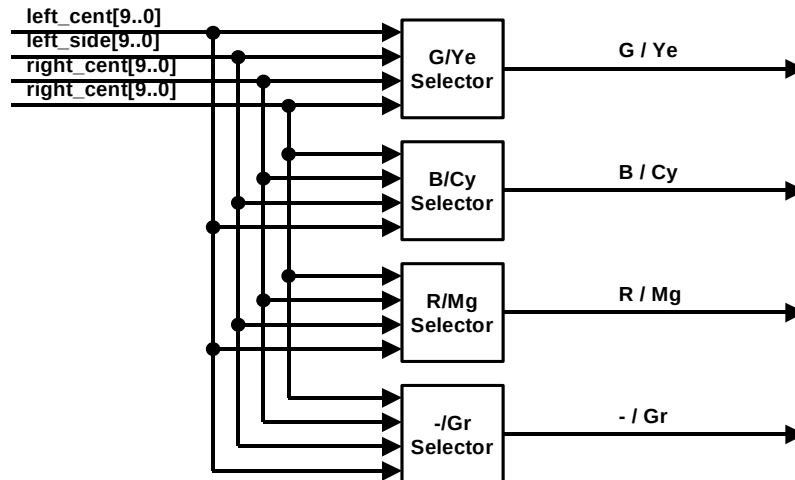


Figure 43: Block diagram of the color selector sub-module

10.2.4.6 Color Adjustment for the Complementary Color Space

The color adjustment sub-module performs the adjustment to interpolated data of 4 colors in the complementary color space in order to satisfy an equation of the following complementary color space. If a CCDMOD bit in the PVSET2 register is zero, the data bypasses this sub-module.

10.2.4.7 Comp2RGB conversion

The comp2RGB conversion sub-module converts the data from four colors in the complementary color space to the RGB color space. Especially for G, data from the interpolation process and data calculated in this module using the same formula in the complementary color space can be blended. If the CCDMOD bit is 0, the data bypass this module.

10.2.5 Black Adjustment

After CFA interpolation, black adjustment can be applied to each color component, R, G, and B. A different offset can be applied to each component. The color adjustment sub-module performs the following calculation on each color component, as shown in Figure 44.

$$\text{data_out} = \text{data_in} + \text{bl_offset}$$

Black adjustment offset is specified as 8-bit 2's complement value and is set using BLOFST0 and BLOFST1 registers.

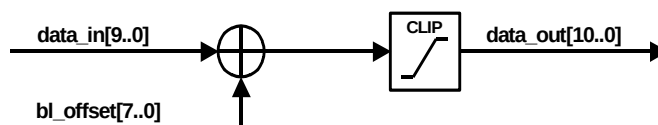


Figure 44: Block diagram of black adjuster module

10.2.6 RGB2RGB Blending Matrix

The RGB2RGB blending module transforms the RGB data generated by the CFA interpolation module using a 3x3 square matrix transformation in combination with an added offset.

The registers MTXGAIN0 - MTXGAIN8 and MTXOFST0 - MTXOFST2 are used for RGB2RGB blending. Each of the gains is 12-bit data with a range of -8 to +8 (with 8-bit fraction).

The precision of MTXGAINx register is S12Q8. Offset is specified as 10-bit 2's complement number.

10.2.7 Gamma Correction and Y-enhancing table

The gamma correction can be done in two ways. One is to use an embedded ROM table. The second is to perform a gamma correction independently for each color in the RGB color space by using a piece-wise linear interpolation. Each table has 128-entries, and each entry accommodates a 10-bit gain and 10-bit offset. The gamma correction module applies gamma correction to each color component, R, G and B. Set the bypass bits (BYPR, BYPG or BYPB) in GAMTSBYP register to bypass the gamma tables. The gamma tables can be bypassed for each color component, independently. In addition to Gamma tables for R, G and B, a table for non-linear enhancement of luminance component also can be setup.

From a software perspective each table has 256 entries of gain and offset. Gain and offset (10-bits each) needs to be setup separately. The address pointer increments by one for each write. All the four tables are linearly addressed. Thus there are 256 addresses in each table. The table address is set in TABLE_ADDR register and the data is written in TABLE_DATA register. Table 26 illustrates the addressing of Gamma and Y-enhancing table.

Figure 45 shows a block diagram of the gamma correction module. When the BYPASS bit is asserted, the input data is passed as it is (10-bit precision). When the ROM is selected, the output of the ROM is eight bits, and two zeros are added at the LSB side to make it 10-bit.

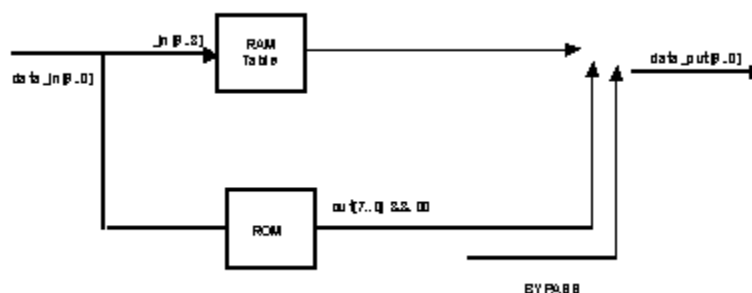


Figure 45: Gamma Correction Module Block Diagram

Gamma Table for Red	Address-0 (Gain)	Address-1 (Offset)
	...	...
	Address-254 (Gain)	Address-255 (Offset)
Gamma Table for Green	Address-256 (Gain)	Address-257 (Offset)
	...	...
	Address-510 (Gain)	Address-511 (Offset)
Gamma Table for Blue	Address-512 (Gain)	Address-513 (Offset)
	...	...
	Address-767 (Gain)	Address-768 (Offset)
Non-linear Y-enhancing table	Address-769 (Gain)	Address-770 (Offset)
	...	...
	Address-1022 (Gain)	Address-1023 (Offset)

Table 26: Gamma and Y-enhancing tables

10.2.8 RGB2YCbCr Conversion Module

The RGB2YCbCr conversion module has a 3x3 square matrix and converts the RGB color space of the image data into the YCbCr color space. In addition to the conversion matrix operation, offset, contrast, brightness and chroma suppression are performed in this module.

Register	Description
<i>CSC0 - CSC4</i>	Color Space Conversion Coefficient registers for RGB to Y, Cb, Cr conversion
<i>YOFST</i>	DC offset value for Y
<i>COFST</i>	DC offset value for Cb and Cr
<i>CNTBRT</i>	Contrast and brightness adjustment register for the Y data

Table 27: RGB to YCbCr conversion registers

The precision of CSCx register is S8Q6

In addition to this color space operation, contrast (scaling/multiplication) and brightness (offset/addition) adjustments can be made on the luminance data and pseudo-color suppression can be performed for chrominance data.

10.2.8.1 Pseudo-color suppression

The gain factor applied to Cb and Cr during pseudo-color suppression is determined using a configurable function mapping. The pseudo-color suppression can be enabled using CSUP0 register. The pseudo-color suppression threshold and gain can be set in the CSUP1 register. Use

CSUPTH field to set the threshold constant and use CSUPG field to set the gain constant. Precision of CSUPG is U8Q8

Cgain is suppressed smaller than one while luminance signal level is higher than CSUPTH.

10.2.9 Output Clipping Module

Before outputting data to the SDRAM, Preview Engine performs clipping on the data. The minimum and maximum threshold values are specified using *SETUPY* and *SETUPC* registers. By default, these values are zero, so user needs to set appropriate value in order to get output from the preview engine. For example 0xFF for maximum Y, Cb, Cr value and 0 for minimum Y, Cb, Cr value means no clipping of output data

This setting must be done for all modes of operation including normal CCD raw-data processing mode, one-shot mode, resize only mode, SDRAM input mode.

10.2.10 4:2:2 Conversion and Output Interface Module

The 4:2:2 conversion module converts the image data to YCbCr-4:2:2 format by discarding every other Cb and Cr component in the horizontal direction. After converting the data to YCbCr-4:2:2, the image data is transferred to the SDRAM controller (SDRC) in units of 32 bytes. The SDRC manages the transfer of the image data to SDRAM. The Preview Engine has an internal 16-bit by 16 FIFO that is used to buffer the data. Data is transferred from the FIFO to the SDRC in 32-byte bursts. *Figure 46* shows how the YCbCr image data is formatted in SDRAM

MSB				LSB			
31	24	23	16	15	8	7	0
Y1				Cr0			
				Y0			
				Cb0			

Figure 46: YCbCr-4:2:2 Format

10.3 Histogram

Histogram function supports the following

- Up to 4 regions (areas)
 - o Each region has separate on/off control
 - o Each region has its own start coordinate X/Y (12-bit) and horizontal/vertical sizes (12-bit)
 - o When the regions overlap, only one region is operated on (selected bin incremented)
- CFA data assumed (RGr lines and GbB lines interleaved). Foveon sensor is not supported, and data for each color go into separate set of bins
- Bins are counters, counting number of values in the range associated with the bins
- Per color, per region, there can be 32, 64, 128, or 256 bins
- Data values are first down-shifted then saturated for bin selection

10.3.1 Reset of Histogram RAM

The user is responsible for resetting the histogram RAM. This can be done by two ways.

- o Writing zeros to the RAM via software
- o If the CLR bit is set reading the memory will cause it to be reset after the read.

ARM reads and writes shall be blocked when the Start/Busy bit is 1

10.3.2 Illegal Setting Override

The histogram RAM is limited in size. Therefore the user can only set the number of bins as shown in Table 28

Regions	Bins allowed
1	256,128,64,32
1,2	128,64,32
1,2,3	64,32
1,2,3,4	64,32

Table 28: Regions and Bins for Histogram

10.3.3 Overflow Handling

If incrementing a histogram bin would cause the value to become greater than what the RAM word could hold, the value is saturated to the maximum value.

For eg, if the input data width is 10 bits (9 ... 0) and the data to be histogrammed is 8 bits wide. Therefore, if the input value is larger than the highest bin location, the result shall be clipped to the highest bin location. This allows data from above the bin range to be included in the upper most bin.

Example:

1 Region enabled, 256 bins per color

Shift = 0, Pixel value = 1000

Pixel value (1000) > Max bin index (255)

Therefore the down-shifted pixel value is clipped to max bin index, 255, and bin 255 is incremented. If bin 255 already holds the maximum value, this incrementing is saturated so that the maximum value remains in the bin.

10.3.4 Region priority

Up to 4 regions can be active at any time but a pixel is only binned into one of regions. The priority is Region 0 > Region 1 > Region 2 > Region 3.

10.4 Preview Engine Operation Guide

This section explains how to configure and use the Preview Engine.

Some Preview Engine register bits are latched by the VD signal. The VD signal is the vertical synch pulse generated by the CCDC or input to the CCDC from an

external CCD imager. Data written to these latched registers does not take effect until the VD pulse is received. This allows bits that control the Preview Engine, such as the Preview Engine enable bit and the SDRAM data source and destination bit field, to be changed in between successive VD pulses without corrupting the current image being processed by the Preview Engine.

In case of VD input mode (VDPOL and VDINT0 bits are set in CCDC module), preview engine registers are always latched on rising edge of the VD signal, irrespective of VD polarity, as shown in *Figure 47*.

When VD is output by CCDC, CCDC internally always generate VD signal with high polarity and before outputting the signal it inverts the VD based on VDPOL bit. Thus, when VD is output by CCDC, preview engine registers will always be latched at the start of VD.

Registers *PVEN*, *RSONLY* bit in *PVSET1* register, *RADRH*, *RADRL*, *WADRH*, *WADRL*, *HSTART*, *HSIZE*, *VSTART*, and *VSIZE* are latched by VD signal.

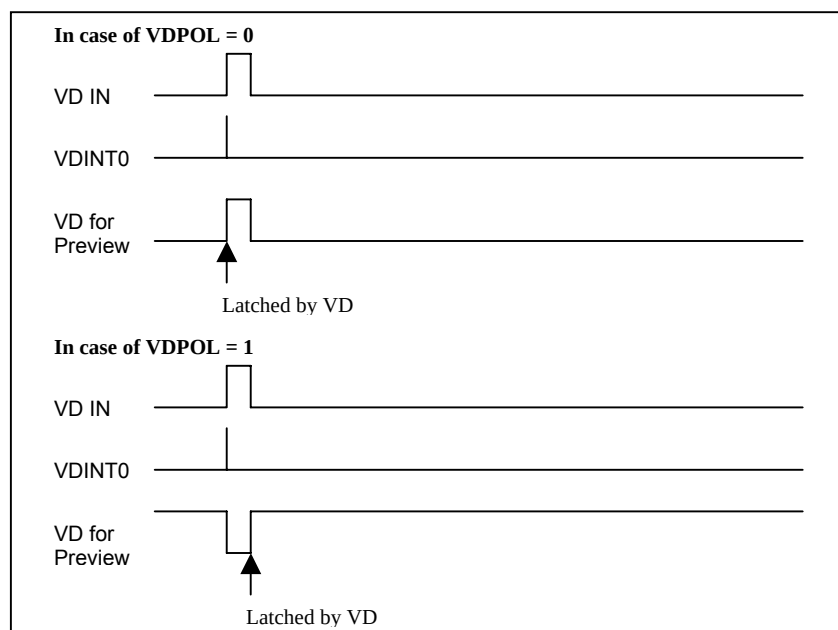


Figure 47: VD Polarity and Timing

10.4.1 Starting the Preview Engine

Setting PVEN bit of register PVEN activates the Preview Engine. Clearing this bit deactivates the Preview Engine. The status of the preview engine can be verified by reading PVEN bit. Also Preview Engine interrupt is generated when the preview engine changes from enabled state to disabled state. PVEN is latched by VD. PVEN timing is shown in *Figure 48*.

When PVEN bit is set, the Preview Engine becomes active at the next VD. When PVEN bit is cleared the Preview Engine becomes inactive after completing the last burst transfer of processed image data.

Furthermore when the Preview Engine becomes inactive the Preview Engine interrupt signal is issued. This interrupt always triggers after completing the last burst transfer of processed image data. Refer to chapter on "Interrupt Controller" for more information about this interrupt.

IMPORTANT: It is recommended that the preview engine is made inactive by clearing PVEN register when the following registers are being changed. PVSET1, HSRT, HSIZE, VSRT, VSIZE, SMTH, HRSZ, VRSZ

Preview Engine does not trigger Preview Engine interrupt after processing of each and every frame of data. It is triggered when

1. Preview Engine is disabled and last burst transfer of the last frame of processed image data is done
2. In one-shot mode, last burst transfer of the processed image data is done

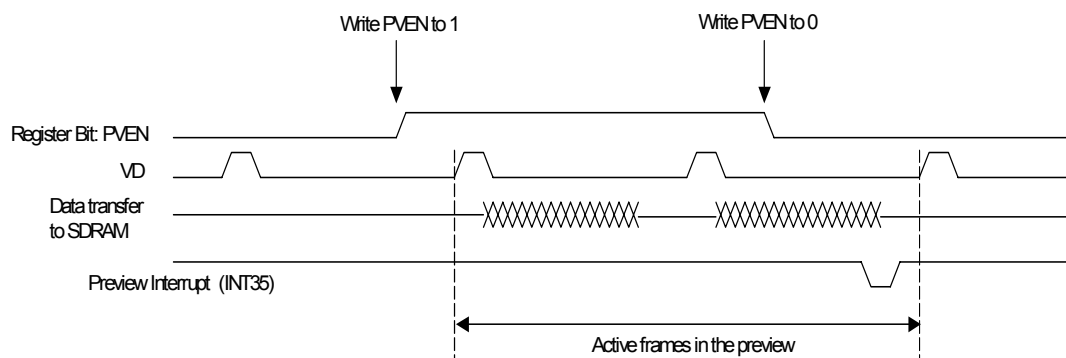


Figure 48: Preview Enable/disable timing

10.4.2 Preview Engine Input Modes

The preview engine has two selectable input ports, which are connected to the CCDC and the SDRAMC. There are some differences of available functions between the CCDC input mode and the SDRAMC input mode. Table 29, shows the main differences of available functions between the two input modes. Normal mode means RAW data input mode and Resize Only mode means YCbCr data input mode.

Functions for Bayer Sensor	CCDC input mode		SDRAMC input mode	
	Normal	Resize Only	Normal	Resize Only
Noise filter	Y	N	Y	N
Digital gain	Y	N	Y	N
White balance	Y	N	Y	N
Smoothing	Y	Y	Y	Y
Down sample	Y	Y	Y	Y
Zoom	N	N	Y	Y
RGB blend	Y	N	Y	N
Gamma	Y	N	Y	N
Chroma suppress	Y	N	Y	N
Chroma offset	Y	N	Y	N
1 shot preview	Y	Y	Y	Y
Histogram - Bayer	Y	N	Y	N
Black adjust	Y	N	Y	N

Table 29: Preview Engine: Input Mode Comparison for Bayer sensor

Functions for Foveon Sensor	CCDC input mode		SDRAMC input mode	
	Normal	Resize Only	Normal	Resize Only
Noise filter	Y	N	Y	N
Digital gain	Y	N	Y	N
White balance	Y	N	Y	N
Smoothing	Y	Y	Y	Y
Down sample	Y	Y	Y	Y
Zoom	N	N	Y	Y
RGB blend	Y	N	Y	N
Gamma	Y	N	Y	N
Chroma suppress	Y	N	Y	N
Chroma offset	Y	N	Y	N
1 shot preview	Y	Y	Y	Y
Histogram - Bayer	Y	N	Y	N
Black adjust	Y	N	Y	N

Table 30: Preview Engine: Input Mode Comparison for Foveon sensor

Functions for Sony VGA Sensor	CCDC input mode*	
	Normal	Resize Only
Noise filter	Y	N
Digital gain	Y	N
White balance	Y	N
Smoothing	Y	Y
Down sample	Y	Y
Zoom	N	N
RGB blend	Y	N
Gamma	Y	N
Chroma suppress	Y	N
Chroma offset	Y	N
1 shot preview	Y	Y
Histogram - Bayer	Y	N
Black adjust	Y	N

Table 31: Preview Engine: Input Mode Comparison for Sony VGA sensor

Note: Sony VGA sensor is not supported in SDRAM input mode

IMPORTANT: When Preview Engine is configured to take input from SDRAM (YCbCr input data during YCbCr resize or raw data input), in order for Preview Engine to operate the following conditions must be satisfied.

- Set the VD mode to internal VD mode
- SDRAM destination address must be specified. This is the location in SDRAM where the Preview Engine will write the processed image data.
- When operating in SDRAM input mode, the user must also specify an SDRAM input data address. This is the location in SDRAM where the Preview Engine will read the data to be processed.
- The SDRAM read/write address is specified as offset from start of SDRAM and in units of 32 bytes, since data is transferred between the Preview Engine and SDRAMC in bursts of 32 bytes.

For example suppose the SDRAM address 0x911000 is to be set as the Preview Engine write address, and 0x900000 is the SDRAM base Address, then the value that is to be programmed in the Preview Engine destination address registers (WADRL, WADRH) is calculated as follows:

$$(0x911000 - 0x900000)/32 = (0x11000)/32 = 0x880$$

- When Preview Engine is configured to take input from SDRAM, the register HSTART must be set with the pixel width of input data present in the SDRAM
- Furthermore, if the BSTAL bit is set to 1 in PVSET1 register, then the horizontal size of pixel data transferred to the SDRAMC will be aligned on a 32-byte boundary, i.e. if the horizontal size is not a multiple of 32-bytes, the excess data will not be transferred to the SDRAMC.
- MAXY, MINY fields in register SETUPY and SETUPC, must be set. These registers specify the clipping to be applied to the Y, Cb, Cr pixel data before it is output to SDRAM.

10.4.2.1 SDRAM Input mode - input pixel width selection

The preview engine has three selectable modes in SDRAM input mode.

- 10bits mode (8BITS = 0 and ALAW =0 and INMODE = 1 in PVSET1 register)
- 8bits mode (8BITS = 1 and ALAW =0 and INMODE = 1 in PVSET1 register)
- Inverse A-law mode (8BITS = 1 and ALAW =1 and INMODE = 1 in PVSET1 register).

NOTE: Inverse A-law mode is available only when preview engine is in SDRAM input mode. 10bits mode and 8bits mode is available in both SDRAM input mode as well as CCDC input mode

10bits mode

In 10bits mode, raw data must be stored in SDRAM as shown below,

31	24	23	16	15	8	7	0
x x x x	Pixel N+1 (25:16)				x x x x	Pixel N (9:0)	

X = invalid bits

8bits mode

In this mode each input raw data pixel has width of 8bits. The raw data stored in SDRAM as shown below

31	24	23	16	15	8	7	0
Pixel N+3				Pixel N+2		Pixel N+1	

Internally preview engine always processes raw data with 10 bits per pixel. When processing the 8-bit raw data, the preview engine expands the data from 8 bits to 10 bits by adding “00” in LSB side as shown in the *Figure 49*.

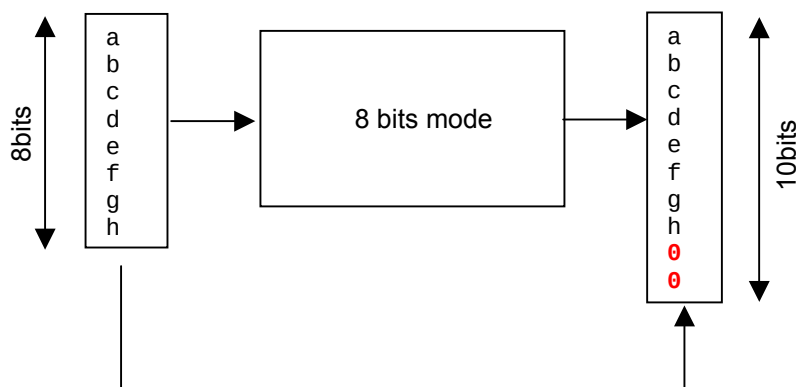


Figure 49: 8-bits mode

A-law Mode

In this mode each input raw data pixel has width of 8bits. The raw data stored in SDRAM as shown below

31	24	23	16	15	8	7	0
Pixel N+3	Pixel N+2	Pixel N+1	Pixel N				

Internally preview engine always processes raw data with 10 bits per pixel. When processing the 8-bit raw data, the preview engine expands the data from 8 bits to 10 bits by using inverse A-law table as shown in Figure 50. Refer to ITU-T G.711 specification for A-law/Inverse A-law characteristics

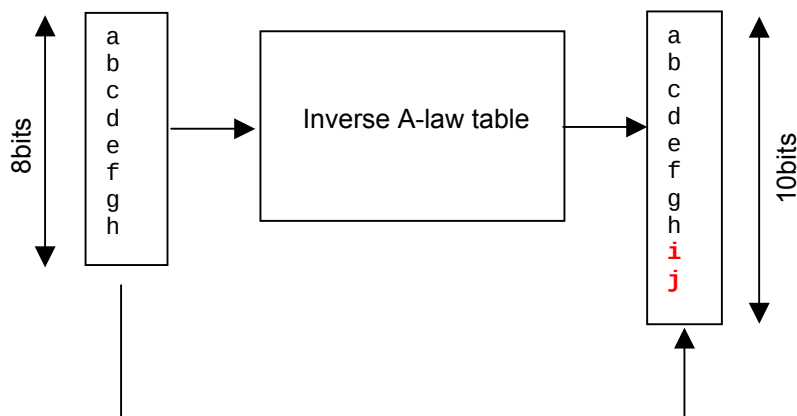


Figure 50: A-law mode

10.4.3 Defining Output Image Size

The size of the processed Preview Engine output image is determined by HSIZE,

VSIZE, HRSZ and VRSZ registers.

IMPORTANT: Up-sampling is only possible when operating in SDRAM mode, i.e. when bit INMODE of PVSET1 register is set to 1.

Depending on the values of the HRSZ and the VRSZ, the preview engine may send an extra burst (32-bytes) to SDRAM. It is recommended to have SDRAM work that is at least 32-bytes larger than the expected output image size.

Note: The maximum size of the number of lines output is 720 lines.

10.4.3.1 Burst Alignment

The Preview Engine always transfers data to the SDRC in bursts of 32-bytes. If BSTAL bit in PVSET1 register is cleared and the Preview Engine is configured such that number of horizontal pixels output (Hout) is not a multiple of 32-bytes (16 pixels), then the Preview Engine will transfer Hout in addition to some excess data in the final 32-byte transfer per horizontal line. It is recommended to reserve additional bytes of SDRAM for the excess data, in-order-to avoid unexpected overwriting of an important segment of SDRAM.

If BSTAL bit in PVSET1 register is set to 1 and the Preview Engine is configured such that Hout is a multiple of 32-bytes (16 pixels), the horizontal size of transferred data will be burst aligned, and excess pixel data will be discarded.

```
i.e, when BSTAL=0,
if(Hresizeout % 16 != 0)
    Hout = Hresizeout + 16 - Hresizeout % 16
else
    Hout = Hresizeout

and when BSTAL=1, Hout = Hresizeout - (Hresizeout % 16)
```

where, Hout, is number of pixels per line output to SDRAM and Hresizeout, is number of pixels per line after resizing. Note that, 1 pixel = 16-bits.

10.4.3.2 Output Image Size Calculation

The size of the output image from the preview engine is determined by the HSIZE, VSIZE, HRSZ and VRSZ registers as per the equations shown below.

$$H_{out} = \begin{cases} H_SIZE \times \frac{64}{H_RSZ} & (\text{if } H_RSZ \geq 64) \\ (H_SIZE - 1) \times \frac{64}{H_RSZ} & (\text{if } H_RSZ < 64) \end{cases}$$

$$V_{out} = V_SIZE \times \frac{64}{V_RSZ}$$

NOTE: When Preview Engine is configured to take input from SDRAM, the register HSTART must be set with the pixel width of input data present in the SDRAM as shown in Figure 51.

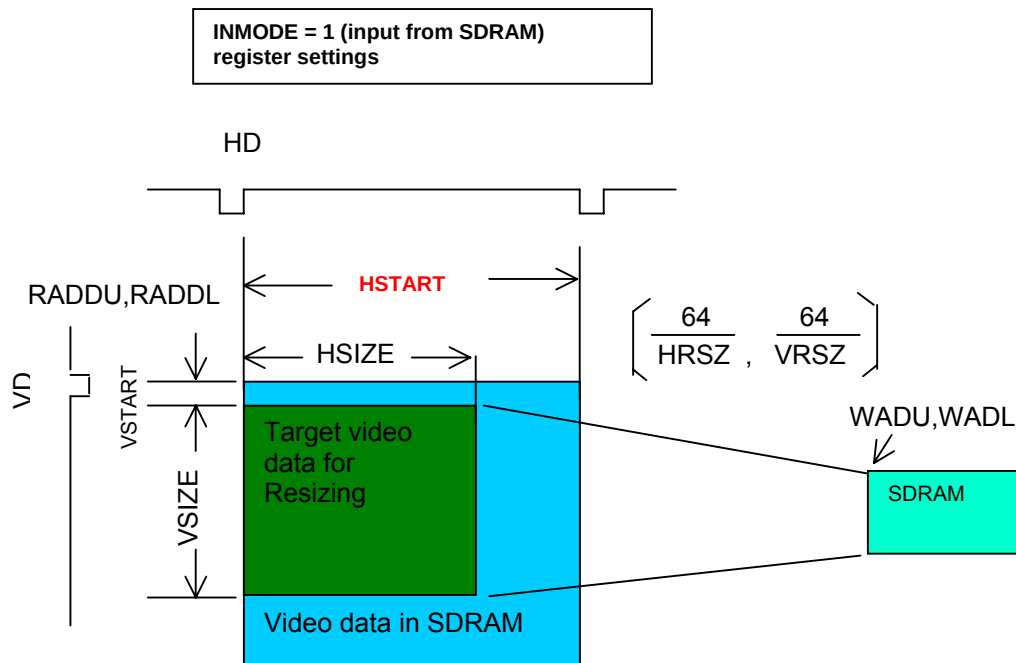


Figure 51: Preview Image parameters in SDRAM input mode

10.4.4 One-shot preview

When the PVOS bit in the PVSET1 register is set, the Preview Engine will process only one frame of data, and then automatically deactivate. Figure 52 shows signal timings in one-shot mode.

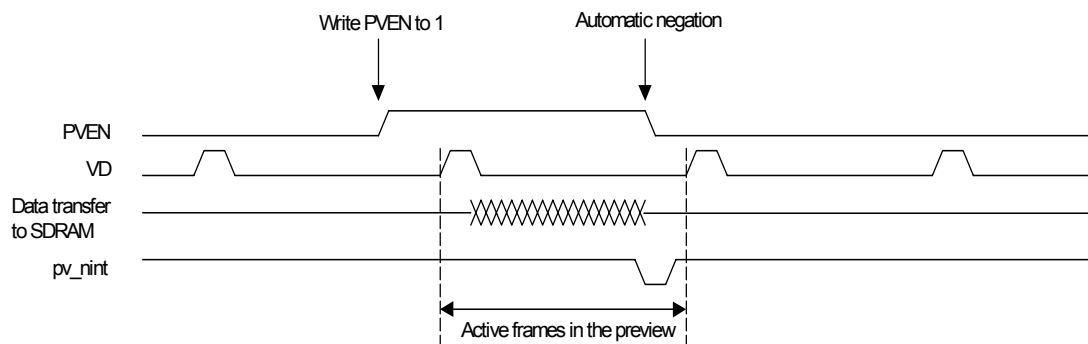


Figure 52: One-shot Mode Timings

10.4.5 Preview Engine Operation Examples

This section gives few examples on configuration and control of preview engine. Refer to “Appendix” for description about the CSL macro notation used in this section.

10.4.5.1 Example 1.

CCD input raw data size is 1536x240, 10-bits per pixel
 Required Preview engine output size is 720x240
 Preview Engine data output address is 0x97AB00

The sample code for this configuration is given in Figure 53

10.4.5.2 Example 2

SDRAM input raw data size is 1536x240, 10-bits per pixel
 Required Preview engine output size is 720x480
 Preview Engine data output address is 0x97AB00
 Preview Engine data input address is 0xA7AB00
 One-shot mode processing is required.

The sample code for this configuration is given in Figure 54


```

Uint16 input_width = 1536;
Uint16 input_height = 240;
Uint16 output_width = 720;
Uint16 output_height = 480;
Uint32 write_addr = 0x97Ab00;
Uint32 read_addr = 0xA7Ab00;
PREV_RSET(PVEN, 0); // disable Preview Engine
while(PREV_RGET(PVEN)); // wait till preview is disabled
PREV_FSET(PVSET1, 8BITS, 0); // 10 bits input mode
PREV_FSET(PVSET1, ALAW, 0); // Normal Mode
PREV_FSET(PVSET1, PVOS, 1); // Enable one-shot mode
PREV_FSET(PVSET1, RSONLY, 0); // Normal Mode—raw CCD data
PREV_FSET(PVSET1, INMODDE, 1); // SDRAM Input Mode
PREV_FSET(PVSET1, BSTAL, 1); // burst align mode
PREV_FSET(PVSET1, WEN, 0); // ignore WEN signal
write_addr = (write_addr - 0x900000)/32;
PREV_RSET(WADRH, write_addr >> 16);
PREV_RSET(WADRL, write_addr & 0xFFFF);
read_addr = (read_addr - 0x900000)/32;
PREV_RSET(RADRH, read_addr >> 16);
PREV_RSET(RADRL, read_addr & 0xFFFF);
PREV_RSET(HSTART, input_width); // horizontal size of data in SDRAM, units of pixels

PREV_RSET(VSTART, 0); // read data from 1st line in SDRAM

prev_hsize = input_width;
prev_vsize = input_height;
prev_hrsz = (prev_hsize * 16) / output_width;
prev_hsize = (prev_hrsz * output_width) / 16;
// since for horizontal upscaling Hout = (HSIZE-1)*16/HRSZ
if(output_width > input_width)
    prev_hsize++;
prev_vrsz = (prev_vsize * 16) / output_height;
prev_vsize = (prev_vrsz * output_height) / 16;
/** Since 1536/720 = 2.13333 resize ratio is not supported by preview engine, only 1530 pixels are
taken as input so that (1530*16)/34 = 720 pixels will be output */
PREV_RSET(HSIZE, prev_hsize); // HSIZE = 1530
PREV_RSET(HRSZ, prev_hrsz); // HRSZ = 34
PREV_RSET(VSIZE, prev_vsize); // VSIZE = 240
PREV_RSET(VRSZ, prev_vrsz); // VRSZ = 8
// setup other registers depending on CCD raw data properties

PREV_RSET(SETUPY, 0x00FF); // no clipping of Y, C output data
PREV_RSET(SETUPC, 0x00FF);
PREV_RSET(PVEN, 1); // enable Preview Engine, start one shot mode
/** wait for preview engine interrupt to indicate completion of one shot mode */

```

Figure 53: Sample code for Example 1

```

Uint16 input_width = 1536;
Uint16 input_height = 240;
Uint16 output_width = 720;
Uint16 output_height = 240;
Uint32 write_addr = 0x97Ab00;
PREV_RSET(PVEN, 0); // disable Preview Engine
while(PREV_RGET(PVEN)) // wait till preview is disabled ;
PREV_FSET(PVSET1, 8BITS, 0); // 10 bits input mode
PREV_FSET(PVSET1, ALAW, 0); // Normal Mode
PREV_FSET(PVSET1, PVOS, 0); // Normal Mode-continuous
PREV_FSET(PVSET1, RSONLY, 0); // Normal Mode-raw CCD data
PREV_FSET(PVSET1, INMODDE, 0); // CCDC Input Mode
PREV_FSET(PVSET1, BSTAL, 1); // burst align mode
PREV_FSET(PVSET1, WEN, 0); // ignore WEN signal
write_addr = (write_addr - 0x900000)/32;
PREV_RSET(WADRH, write_addr >> 16);
PREV_RSET(WADRL, write_addr & 0xFFFF);

PREV_RSET(HSTART, . . .); // depends on CCD module
PREV_RSET(VSTART, . . .); // depends on CCD module

prev_hsize = input_width;
prev_vsize = input_height;
prev_hrsz = ( prev_hsize * 16 )/output_width;
prev_hsize = ( prev_hrsz * output_width ) / 16;
// since for horizontal upscaling Hout = (HSIZE-1)*16/HRSZ
if(output_width > input_width )
    prev_hsize++;
prev_vrsz = ( prev_vsize* 16 ) / output_height;
prev_vsize = ( prev_vrsz * output_height ) / 16;
/**Since 1536/720 = 2.13333 resize ratio is not supported by preview engine, only 1530 pixels are
taken as input so that (1530*16)/34 = 720 pixels will be output***/
PREV_RSET(HSIZE, prev_hsize); // HSIZE = 1530
PREV_RSET(HRSZ, prev_hrsz); // HRSZ = 34
PREV_RSET(VSIZE, prev_vsize); // VSIZE = 240
PREV_RSET(VRSZ, prev_vrsz); // VRSZ = 16
// setup other registers depending on CCD raw data properties

// no clipping of Y, C output data
PREV_RSET(SETUPY, 0x00FF);
PREV_RSET(SETUPC, 0x00FF);

PREV_RSET(PVEN, 1); // enable Preview Engine

```

Figure 54: Sample code for Example 2

10.5 Fractional Notation

The fractional notation used in this chapter is as follows:

(S | U) x Q y

is used to denote a fractional numbers.

Here,

'x', represents the total number of bits in the real number including the fractional part

'y', represents the number of bits allocated to the fractional part. i.e **(x - y)** is the number of bits allocated to the integer part

'S', indicates that the number is signed, with negative numbers represented in 2's complement format

'U', indicates that the number is unsigned.

For example,

Precision: U10Q8

DGAIN[9:0]	Value
11 1111 1111	3.99609375 (3+255/256)
01 0000 0000	1.0
00 1000 0000	0.5
...	
00 0000 0001	0.00390625 (1/256)
00 0000 0000	0

U8Q5, means that the value can range from 0 to 7.96875

S8Q6, means that the value can range from -2 to 1.984375

10.6 Preview Engine Register Map (PREV)

Address	Register	Description
0003:0A80	PVEN	Preview Enable Register
0003:0A82	PVSET1	Preview Setup Register #1
0003:0A84	RADRH	SDRAM Read Address - high order bits
0003:0A86	RADRL	SDRAM Read Address - low order bits
0003:0A88	WADRH	SDRAM Write Address - high order bits
0003:0A8A	WADRL	SDRAM Write Address - low order bits
0003:0A8C	HSTART	Start pixel / Size horizontal
0003:0A8E	HSIZE	Number of pixels horizontal
0003:0A90	VSTART	Start line vertical
0003:0A92	VSIZE	Number of pixels vertical
0003:0A94	PVSET2	Preview Setup Register #2
0003:0A96	NFLT	Noise Filter Definition Register
0003:0A98	DGAIN	Digital Gain Register
0003:0A9A	WBGAIN0	White Balance Gain register #0
0003:0A9C	WBGAIN1	White Balance Gain register #1
0003:0A9E	SMTH	Smoothing Definition register
0003:0AA0	HRSZ	Horizontal Resize Register
0003:0AA2	VRSZ	Vertical Resize Register
0003:0AA4	BLOFST0	Black Level Offset Adjust #0
0003:0AA6	BLOFST1	Black Level Offset Adjust #1
0003:0AA8	MTXGAIN0	Matrix Gain register #0
0003:0AAA	MTXGAIN1	Matrix Gain register #1
0003:0AAC	MTXGAIN2	Matrix Gain register #2
0003:0AAE	MTXGAIN3	Matrix Gain register #3
0003:0AB0	MTXGAIN4	Matrix Gain register #4
0003:0AB2	MTXGAIN5	Matrix Gain register #5
0003:0AB4	MTXGAIN6	Matrix Gain register #6
0003:0AB6	MTXGAIN7	Matrix Gain register #7
0003:0AB8	MTXGAIN8	Matrix Gain register #8
0003:0ABA	MTXOFST0	R_offset register
0003:0ABC	MTXOFST1	G_offset register
0003:0ABE	MTXOFST2	B_offset register
0003:0AC0	GAMTBYP	GAMMA table bypass register
0003:0AC2	CSC0	Color Space Conversion Coefficient register #0
0003:0AC4	CSC1	Color Space Conversion Coefficient register #1
0003:0AC6	CSC2	Color Space Conversion Coefficient register #2
0003:0AC8	CSC3	Color Space Conversion Coefficient register #3
0003:0ACA	CSC4	Color Space Conversion Coefficient register #4
0003:0ACC	YOFST	Y offset register
0003:0ACE	COFST	Cb and Cr offset register
0003:0AD0	CNTBRT	Contrast and brightness adjustment register
0003:0AD2	CSUP0	Chroma Suppression Register #0
0003:0AD4	CSUP1	Chroma Suppression Register #1
0003:0AD6	SETUPY	Y max and min setup register
0003:0AD8	SETUPC	Cb and Cr max and min setup register
0003:0ADA	TABLE_ADDR	Address for Gamma table for luminance
0003:0ADC	TABLE_DATA	Data for Gamma table for luminance

0003:0ADE	HG_CTL	Histogram control
0003:0AE0	HG_R0_HSTART	Histogram Region 0 horizontal start location
0003:0AE2	HG_R0_HSIZE	Histogram Region 0 horizontal size
0003:0AE4	HG_R0_VSTART	Histogram Region 0 vertical start location
0003:0AE6	HG_R0_VSIZE	Histogram Region 0 vertical size
0003:0AE8	HG_R1_HSTART	Histogram Region 1 horizontal start location
0003:0AEA	HG_R1_HSIZE	Histogram Region 1 horizontal size
0003:0AEC	HG_R1_VSTART	Histogram Region 1 vertical start location
0003:0AEE	HG_R1_VSIZE	Histogram Region 1 vertical size
0003:0AF0	HG_R2_HSTART	Histogram Region 2 horizontal start location
0003:0AF2	HG_R2_HSIZE	Histogram Region 2 horizontal size
0003:0AF4	HG_R2_VSTART	Histogram Region 2 vertical start location
0003:0AF6	HG_R2_VSIZE	Histogram Region 2 vertical size
0003:0AF8	HG_R3_HSTART	Histogram Region 3 horizontal start location
0003:0AFA	HG_R3_HSIZE	Histogram Region 3 horizontal size
0003:0AFC	HG_R3_VSTART	Histogram Region 3 vertical start location
0003:0AFE	HG_R3_VSIZE	Histogram Region 3 vertical size
0003:0B00	HG_ADDR	Address for Histogram entry
0003:0B02	HG_DATA	Histogram data

10.7 Preview Engine Registers

10.7.1 PVEN

Preview Enable Register

PVEN	0003:0A80				offset: 0x00				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	PVEN
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0

Bit	Name	Reset Value	R/W	Function
15:1	RSV			Reserved
0	PVEN	0	R/W	<u>Preview Enable</u> » 0: disable « » 1: enable « <i>Turns Preview Engine on or off</i> <i>*This bit is latched by VD.</i>

10.7.2 PVSET1

Preview Setup Register #1

PVSET1	0003:0A82			offset: 0x02		default: 0x8080										
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	ATAWIDTH	ATAWIDTH	ATAWIDTH	INVD	SENSOR	SENSOR	SENSOR	SENSOR[1]	SENSOR[0]	PACK8	ALAW	PVOS	RSONLY	INMODE	BSTAL	WEN
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	1	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:13	DATAWIDTH	4	R/W	<u>Preview data width</u> » 0: 14 bit « » 1: 13 bit « » 2: 12 bit « » 3: 11 bit « » 4: 10 bit (default) « » 5: 9 bit « » 6: 8 bit « » 7: Reserved «
12	INVD	0	R/W	<u>VD generated internally for one-shot preview</u> » 0: No Internal VD « » 1: Internal VD generated «
11:7	SENSOR	1	R/W	<u>Type of sensor</u> » 00001: Bayer pattern sensor « » 00010: Sony VGA sensor « » 00100: Reserved « » 01000: Foveon sensor « » 10000: Reserved « Note: One bit of bit11/10/9/8/7 should set "1"
6	PACK8	0	R/W	<u>Data storage in SDRAM</u> » 0: 16-bit per pixel « » 1: 8-bit mode per pixel «
5	ALAW	0	R/W	<u>Inverse A-LAW mode</u> » 0: Normal mode « » 1: Inverse A-LAW mode « <i>*This bit is latched by VD.</i>
4	PVOS	0	R/W	<u>Preview One Shot</u> » 0: normal (continuous processing) « » 1: one frame only «
3	RSONLY	0	R/W	<u>Re-Size Only - Select type of input data</u> » 0: Normal (raw CCD data) « » 1: Resizing only mode (YCbCr-4:2:2 data input) « <i>*This bit is latched by VD.</i>
2	INMODE	0	R/W	<u>Input Mode</u> » 0: CCD Controller Interface « » 1: SDRAM Controller Interface «
1	BSTAL	0	R/W	<u>Burst Aligned</u> » 0: normal « » 1: burst aligned mode « <i>Apply a burst aligned transfer of output image to SDRAMC. If it is set to 1, the horizontal size is aligned to the burst length and surplus pixels are disregarded.</i>
0	WEN	0	R/W	<u>Write Enable</u> » 0: ignore « » 1: valid « <i>Select handling of a write enable signal from the CCDC. If set to 1 and the write enable signal is low, the preview engine is suspended in the frame.</i>

10.7.3 RADRH

SDRAM Read Address - high order bits

RADRH	0003:0A84			offset: 0x04												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	ADRH[6]	ADRH[5]	ADRH[4]	ADRH[3]	ADRH[2]	ADRH[1]	ADRH[0]	
R/W	-	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	-	-	-	-	-	-	-	-	-	0	0	0	0	0	0	0	

[illegible]

10.7.4 RADRL

SDRAM Read Address - low order bits

RADRL	0003:0A86			offset: 0x06			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	ADRL[15]	ADRL[14]	ADRL[13]	ADRL[12]	ADRL[11]	ADRL[10]	ADRL[9]	ADRL[8]	ADRL[7]	ADRL[6]	ADRL[5]	ADRL[4]	ADRL[3]	ADRL[2]	ADRL[1]	ADRL[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:0	ADRL	0	R/W	Specifies lower 16 bits of the SDRAM source address used when INMODE in HSTART = '1'. Note: The address is specified offset from the SDRAM base address in units of 32 bytes. *This bit is latched by VD.

10.7.5 WADRH

SDRAM Write Address - high order bits

WADRH	0003:0A88			offset: 0x08												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	ADRH[6]	ADRH[5]	ADRH[4]	ADRH[3]	ADRH[2]	ADRH[1]	ADRH[0]	
R/W	-	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	-	-	-	-	-	-	-	-	-	0	0	0	0	0	0	0	

[illegible]

10.7.6 WADRL

SDRAM Write Address - low order bits

WADRL	0003:0A8A			offset: 0x0A			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	ADRL[15]	ADRL[14]	ADRL[13]	ADRL[12]	ADRL[11]	ADRL[10]	ADRL[9]	ADRL[8]	ADRL[7]	ADRL[6]	ADRL[5]	ADRL[4]	ADRL[3]	ADRL[2]	ADRL[1]	ADRL[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:0	ADRL	0	R/W	Specifies lower 16 bits of the SDRAM destination address for Preview Engine output data. Note: The address is specified offset from the SDRAM base address in units of 32 bytes. *This bit is latched by VD.

10.7.7 HSTART

Start pixel horizontal (for CCD input) or size in SDRAM (for SDRAM input)

HSTART	0003:0A8C			offset: 0x0C			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	HSTRT[11]	HSTRT[10]	HSTRT[9]	HSTRT[8]	HSTRT[7]	HSTRT[6]	HSTRT[5]	HSTRT[4]	HSTRT[3]	HSTRT[2]	HSTRT[1]	HSTRT[0]
R/W	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:12	RSV			Reserved
11:0	HSTRT	0	R/W	If INMODE=0: - Sets horizontal start position of valid input image If INMODE=1: - Sets horizontal size of input image in SDRAM <i>*This bit is latched by VD.</i>

10.7.8 HSIZE

Number of pixels horizontal

HSIZE	0003:0A8E			offset: 0x0E			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	HSIZE[11]	HSIZE[10]	HSIZE[9]	HSIZE[8]	HSIZE[7]	HSIZE[6]	HSIZE[5]	HSIZE[4]	HSIZE[3]	HSIZE[2]	HSIZE[1]	HSIZE[0]
R/W	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:12	RSV			Reserved
11:0	HSIZE	0	R/W	Sets horizontal size of input image Note: The horizontal size of the output image must be lower than 720 pixels due to the size of implemented line memories. <i>*This bit is latched by VD.</i>

10.7.9 VSTART

Start line vertical

[illegible][illegible]

10.7.10 VSIZE

Number of pixels vertical

[illegible]

Bit	Name	Reset Value	R/W	Function
15:12	RSV			Reserved
11:0	VSIZE	0	R/W	Set vertical size of a valid input image <i>*This bit is latched by VD.</i>

10.7.11 PVSET2

Preview Setup Register #2

PVSET2	0003:0A94			offset: 0x14			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	HINT	VINT	VNFILT	CCDMOD	VDNINT	GBLND[2]	GBLND[1]	GBLND[0]	EPEL[1]	EPEL[0]	OPEL[1]	OPEL[0]	EPOL[1]	EPOL[0]	OPOL[1]	OPOL[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15	HNT	0	R/W	<u>Horizontal Process</u> » 0: Other peripheral color information is used (5 types)« » 1: Same color information is used (2 types) «
14	VNT	0	R/W	<u>Vertical Process</u> » 0: Other peripheral color information is used (5 types)« » 1: Same color information is used (2 types) «
13	VNFILT	0	R/W	<u>Vertical Noise Filter</u> » 0: disable « » 1: enable «
12	CCDMOD	0	R/W	<u>CCD Sensor Mode</u> » 0: RGB color space « » 1: Complementary color space «
11	VDNINT	0	R/W	<u>Interrupt every frame</u> » 0: Generate Interrupt once when preview is OFF « » 1: Generate Interrupt every frame «
10:8	GBLND	0	R/W	<u>Green Blend</u> In complementary CCD mode, adjust blending ratio between original green and a calculated green from the other colors. Goutput = rGorg + (1-r)(Ye+Cy-Mg) » 000: r=1 « » 001: r=3/4 « » 010: r=1/2 « » 011: r=1/4 « » 100: r=0 «
7:6	EPEL	0	R/W	Color Pattern for Even Pixel, Even Line » 00: Gb/Ye « » 01: B/Cy « » 10: R/Mg « » 11: Gr/Gr «
5:4	OPEL	0	R/W	Color Pattern for Odd Pixel, Even Line » 00: Gb/Ye « » 01: B/Cy « » 10: R/Mg « » 11: Gr/Gr «
3:2	EPOL	0	R/W	Color Pattern for Even Pixel, Odd Line » 00: Gb/Ye « » 01: B/Cy « » 10: R/Mg « » 11: Gr/Gr «
1:0	OPOL	0	R/W	Color Pattern for Odd Pixel, Odd Line » 00: Gb/Ye « » 01: B/Cy « » 10: R/Mg « » 11: Gr/Gr «

10.7.12 NFILT

Noise Filter Definition Register

NFILT	0003:0A96				offset: 0x16				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	THR[9]	THR[8]	THR[7]	THR[6]	THR[5]	THR[4]	THR[3]	THR[2]	THR[1]	THR[0]	RSV	NFEN	RSV	RSV	NFRT[1]	NFRT[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	-	R/W	-	-	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	-	0	-	-	0	0

Bit	Name	Reset Value	R/W	Function
15:5	THR	0	R/W	Threshold value for the noise filter Precision is U10Q0
4	RSV			Reserved
4	NFEN	0	R/W	Horizontal Noise Filter Enable » 0: disable « » 1: enable «
32	RSV			Reserved
10	NFRT	0	R/W	Strength of Horizontal Noise Filter » 00: weak « » 01: a little weak « » 10: a little strong « » 11: strong «

10.7.13 DGAIN

Digital Gain Register

DGAIN	0003:0A98				offset: 0x18				default: 0x0100							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	DGAIN[9]	DGAIN[8]	DGAIN[7]	DGAIN[6]	DGAIN[5]	DGAIN[4]	DGAIN[3]	DGAIN[2]	DGAIN[1]	DGAIN[0]
R/W	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	-	0	1	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:10	RSV			Reserved
9:0	DGAIN	256	R/W	Digital Gain Value Precision is U10Q8, i.e. Decimal point is between 7 and 8 Range is 0-3.99609375

10.7.14 WBGAIN0

White Balance Gain register #0

WBGAIN0	0003:0A9A			offset: 0x1A												default: 0x2020	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	WG00[7]	WG00[6]	WG00[5]	WG00[4]	WG00[3]	WG00[2]	WG00[1]	WG00[0]	WG01[7]	WG01[6]	WG01[5]	WG01[4]	WG01[3]	WG01[2]	WG01[1]	WG01[0]	
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	0	0	1	0	0	0	0	0	0	0	1	0	0	0	0	0	

[illegible]

10.7.15 WBGAIN1

White Balance Gain register #1

WBGAIN1	0003:0A9C			offset: 0x1C												default: 0x2020	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	WG02[7]	WG02[6]	WG02[5]	WG02[4]	WG02[3]	WG02[2]	WG02[1]	WG02[0]	WG03[7]	WG03[6]	WG03[5]	WG03[4]	WG03[3]	WG03[2]	WG03[1]	WG03[0]	
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	0	0	1	0	0	0	0	0	0	0	1	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:8	WG02	32	R/W	White Balance Gain for Gr/G Precision is U8Q5, I.e. decimal point is between 4 and 5
7:0	WG03	32	R/W	White Balance Gain for R/Mg Precision is U8Q5, I.e. decimal point is between 4 and 5

10.7.16 SMTH

Smoothing Definition register

[illegible][illegible]

10.7.17 HRSZ

Horizontal Resize Register

HRSZ	0003:0AA0			offset: 0x20			default: 0x0040									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	HRSZD[11]	HRSZD[10]	HRSZD[9]	HRSZD[8]	HRSZD[7]	HRSZD[6]	HRSZD[5]	HRSZD[4]	HRSZD[3]	HRSZD[2]	HRSZD[1]	HRSZD[0]
R/W	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	0	0	0	0	0	1	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:12	RSV			Reserved
11:0	HRSZD	64	R/W	Horizontal Resizing Parameter Horizontal Resize Ratio, R_H = 64/HRSZD Note: The horizontal size of the output image must be lower than 720 pixels due to the size of implemented line memories.

10.7.18 VRSZ

Vertical Resize Register

VRSZ	0003:0AA2				offset: 0x22												default: 0x0040			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
Name	RSV	RSV	RSV	RSV	VRSZD[11]	VRSZD[10]	VRSZD[9]	VRSZD[8]	VRSZD[7]	VRSZD[6]	VRSZD[5]	VRSZD[4]	VRSZD[3]	VRSZD[2]	VRSZD[1]	VRSZD[0]				
R/W	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W				
Default	-	-	-	-	0	0	0	0	0	1	0	0	0	0	0	0				

[illegible]

10.7.19 BLOFST0

Black Level Offset Adjust #0

[illegible][illegible]

10.7.20 BLOFST1

Black Level Offset Adjust #1

BLOFST1	0003:0AA6			offset: 0x26												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	BLB[7]	BLB[6]	BLB[5]	BLB[4]	BLB[3]	BLB[2]	BLB[1]	BLB[0]	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	-	-	-	-	-	-	-	-	
Default	0	0	0	0	0	0	0	0	-	-	-	-	-	-	-	-	

[illegible]

10.7.21 MTXGAIN0

Matrix Gain register #0

MTXGAIN0	0003:0AA8			offset: 0x28												default: 0x0100	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	MTXRR[11]	MTXRR[10]	MTXRR[9]	MTXRR[8]	MTXRR[7]	MTXRR[6]	MTXRR[5]	MTXRR[4]	MTXRR[3]	MTXRR[2]	MTXRR[1]	MTXRR[0]	
R/W	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	-	-	-	-	0	0	0	1	0	0	0	0	0	0	0	0	

[illegible]

10.7.22 MTXGAIN1

Matrix Gain register #1

[illegible][illegible]

10.7.23 MTXGAIN2

Matrix Gain register #2

[illegible][illegible]

10.7.24 MTXGAIN3

Matrix Gain register #3

[illegible][illegible]

10.7.25 MTXGAIN4

Matrix Gain register #4

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	MTXGG[11]	MTXGG[10]	MTXGG[9]	MTXGG[8]	MTXGG[7]	MTXGG[6]	MTXGG[5]	MTXGG[4]	MTXGG[3]	MTXGG[2]	MTXGG[1]	MTXGG[0]
R/W	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	0	0	0	1	0	0	0	0	0	0	0	0

[illegible]

10.7.26 MTXGAIN5

Matrix Gain register #5

[illegible][illegible]

10.7.27 MTXGAIN6

Matrix Gain register #6

[illegible][illegible]

10.7.28 MTXGAIN7

Matrix Gain register #7

[illegible][illegible]

10.7.29 MTXGAIN8

Matrix Gain register #8

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	MTXBB[11]	MTXBB[10]	MTXBB[9]	MTXBB[8]	MTXBB[7]	MTXBB[6]	MTXBB[5]	MTXBB[4]	MTXBB[3]	MTXBB[2]	MTXBB[1]	MTXBB[0]
R/W	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	0	0	0	1	0	0	0	0	0	0	0	0

[illegible]

10.7.30 MTXOFST0

R_offset register

[illegible][illegible]

10.7.31 MTXOFST1

G_offset register

[illegible][illegible]

10.7.32 MTXOFST2

B_offset register

MTXOFST2	0003:0ABE	offset: 0x3E	default: 0x0000
----------	-----------	--------------	-----------------

[illegible][illegible]

10.7.33 GAMTSBYP

GAMMA table bypass register

GAMTBYP	0003:0AC0			offset: 0x40												default: 0x0010	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	ROM	RSV	BYPR	BYPG	BYPB	
R/W	-	-	-	-	-	-	-	-	-	-	-	R/W	-	R/W	R/W	R/W	
Default	-	-	-	-	-	-	-	-	-	-	-	1	-	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:5	RSV			Reserved
4	ROM	1	R/W	<u>Gamma ROM Table Select</u> » 0: Piece-wise linear RAM table « » 1: ROM Table «
3	RSV			Reserved
2	BYPR	0	R/W	<u>Gamma RAM Table Bypass for Red</u> » 0: Use RAM gamma table for R « » 1: Bypass gamma table for R «
1	BYPG	0	R/W	<u>Gamma RAM Table Bypass for Green</u> » 0: Use RAM gamma table for G « » 1: Bypass gamma table for G «
0	BYPB	0	R/W	<u>Gamma RAM Table Bypass for Blue</u> » 0: Use RAM gamma table for B « » 1: Bypass gamma table for B «

10.7.34 CSC0

Color Space Conversion Coefficient register #0

CSC0	0003:0AC2			offset: 0x42													default: 0x1326		
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	CSCRY[7]	CSCRY[6]	CSCRY[5]	CSCRY[4]	CSCRY[3]	CSCRY[2]	CSCRY[1]	CSCRY[0]	CSCGY[7]	CSCGY[6]	CSCGY[5]	CSCGY[4]	CSCGY[3]	CSCGY[2]	CSCGY[1]	CSCGY[0]			
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W			
Default	0	0	0	1	0	0	1	1	0	0	1	0	0	1	1	0			

[illegible]

10.7.35 CSC1

Color Space Conversion Coefficient register #1

CSC1	0003:0AC4			offset: 0x44			default: 0x07F5									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	CSCBY[7]	CSCBY[6]	CSCBY[5]	CSCBY[4]	CSCBY[3]	CSCBY[2]	CSCBY[1]	CSCBY[0]	CSCRCB[7]	CSCRCB[6]	CSCRCB[5]	CSCRCB[4]	CSCRCB[3]	CSCRCB[2]	CSCRCB[1]	CSCRCB[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	1	1	1	1	1	1	1	0	1	0	1

Bit	Name	Reset Value	R/W	Function
15:8	CSCBY	7	R/W	Color Space Conversion Coefficient of B for computing Y Precision is S8Q6, I.e., $GAIN/(2^6)$, where GAIN is in 2's complement format
7:0	CSCRCB	245	R/W	Color Space Conversion Coefficient of R for computing Cb Precision is S8Q6, I.e., $GAIN/(2^6)$, where GAIN is in 2's complement format

10.7.36 CSC2

Color Space Conversion Coefficient register #2

CSC2	0003:0AC6			offset: 0x46			default: 0xEB20									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	CSCGCB[7]	CSCGCB[6]	CSCGCB[5]	CSCGCB[4]	CSCGCB[3]	CSCGCB[2]	CSCGCB[1]	CSCGCB[0]	CSCBCB[7]	CSCBCB[6]	CSCBCB[5]	CSCBCB[4]	CSCBCB[3]	CSCBCB[2]	CSCBCB[1]	CSCBCB[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	1	1	1	0	1	0	1	1	0	0	1	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:8	CSCGCB	235	R/W	Color Space Conversion Coefficient of G for computing Cb Precision is S8Q6, I.e., $GAIN/(2^6)$, where GAIN is in 2's complement format
7:0	CSCBCB	32	R/W	Color Space Conversion Coefficient of B for computing Cb Precision is S8Q6, I.e., $GAIN/(2^6)$, where GAIN is in 2's complement format

10.7.37 CSC3

Color Space Conversion Coefficient register #3

CSC3	0003:0AC8				offset: 0x48				default: 0x20E5							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	CSCRCR[7]	CSCRCR[6]	CSCRCR[5]	CSCRCR[4]	CSCRCR[3]	CSCRCR[2]	CSCRCR[1]	CSCRCR[0]	CSCGCR[7]	CSCGCR[6]	CSCGCR[5]	CSCGCR[4]	CSCGCR[3]	CSCGCR[2]	CSCGCR[1]	CSCGCR[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	1	0	0	0	0	0	1	1	1	0	0	1	0	1

Bit	Name	Reset Value	R/W	Function
15:8	CSCRCR	32	R/W	Color Space Conversion Coefficient of R for computing Cr Precision is S8Q6, I.e., $GAIN/(2^6)$, where GAIN is in 2's complement format
7:0	CSCGCR	229	R/W	Color Space Conversion Coefficient of G for computing Cr Precision is S8Q6, I.e., $GAIN/(2^6)$, where GAIN is in 2's complement format

10.7.38 CSC4

Color Space Conversion Coefficient register #4

CSC4	0003:0ACA				offset: 0x4A				default: 0xFB00							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	CSCBCR[7]	CSCBCR[6]	CSCBCR[5]	CSCBCR[4]	CSCBCR[3]	CSCBCR[2]	CSCBCR[1]	CSCBCR[0]	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	-	-	-	-	-	-	-	-
Default	1	1	1	1	1	0	1	1	-	-	-	-	-	-	-	-

Bit	Name	Reset Value	R/W	Function
15:8	CSCBCR	251	R/W	Color Space Conversion Coefficient of B for computing Cr Precision is S8Q6, I.e., $GAIN/(2^6)$, where GAIN is in 2's complement format
7:0	RSV			Reserved

10.7.39 YOFST

Y offset register

YOFST	0003:0ACC				offset: 0x4C				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	YOFST[7]	YOFST[6]	YOFST[5]	YOFST[4]	YOFST[3]	YOFST[2]	YOFST[1]	YOFST[0]
R/W	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	-	-	-	0	0	0	0	0	0	0	0

[illegible]

10.7.40 COFST

Cr and Cb offset register

[illegible][illegible]

10.7.43 CSUP1

Chroma suppression register #1

CSUP1	0003:0AD4				offset: 0x54				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	CSUPTH[7]	CSUPTH[6]	CSUPTH[5]	CSUPTH[4]	CSUPTH[3]	CSUPTH[2]	CSUPTH[1]	CSUPTH[0]	CSUPG[7]	CSUPG[6]	CSUPG[5]	CSUPG[4]	CSUPG[3]	CSUPG[2]	CSUPG[1]	CSUPG[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:8	CSUPTH	0	R/W	Chroma Suppression Threshold
7:0	CSUPG	0	R/W	Gain value for Chroma Suppression Function Precision is U8Q8 i.e (0 to 0.99609375)

10.7.44 SETUPY

Y max and min setup register

SETUPY	0003:0AD6				offset: 0x56				default: 0xFF00							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	MAXY[7]	MAXY[6]	MAXY[5]	MAXY[4]	MAXY[3]	MAXY[2]	MAXY[1]	MAXY[0]	MINY[7]	MINY[6]	MINY[5]	MINY[4]	MINY[3]	MINY[2]	MINY[1]	MINY[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:8	MAXY	255	R/W	Maximum Y value, values greater than set value are clipped to the set value <i>Note: MAXY must be set for all modes of operation of the Preview Engine, which includes raw data input from CCD, raw data input from SDRAM, YUV resize only mode, one-shot mode.</i>
7:0	MINY	0	R/W	Minimum Y value. Values less than set value are clipped to the set value <i>Note: MINY must be set for all modes of operation of the Preview Engine, which includes raw data input from CCD, raw data input from SDRAM, YUV resize only mode, one-shot mode.</i>

10.7.45 SETUPC

Cb and Cr max and min setup register

SETUPC	0003:0AD8			offset: 0x58			default: 0xFF00									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	MAXC[7]	MAXC[6]	MAXC[5]	MAXC[4]	MAXC[3]	MAXC[2]	MAXC[1]	MAXC[0]	MINC[7]	MINC[6]	MINC[5]	MINC[4]	MINC[3]	MINC[2]	MINC[1]	MINC[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:8	MAXC	255	R/W	Maximum Cb and Cr value, values greater than set value are clipped to the set value <i>Note: MAXC must be set for all modes of operation of the Preview Engine, which includes raw data input from CCD, raw data input from SDRAM, YUV resize only mode, one-shot mode.</i>
7:0	MINC	0	R/W	Minimum Cb and Cr value. Values less than set value are clipped to the set value <i>Note: MINC must be set for all modes of operation of the Preview Engine, which includes raw data input from CCD, raw data input from SDRAM, YUV resize only mode, one-shot mode.</i>

10.7.46 TABLE_ADDR

Address for Gamma table and non-linear Y table

TABLE_ADDR	0003:0ADA			offset: 0x5A			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	ADDR[9]	ADDR[8]	ADDR[7]	ADDR[6]	ADDR[5]	ADDR[4]	ADDR[3]	ADDR[2]	ADDR[1]	ADDR[0]
R/W	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:10	RSV			Reserved
9:0	ADDR	0	R/W	Address of gamma table and non linear Y enhancing table <i>Note: The address is auto incremented after every read / write in the TABLE_DATA register</i>

10.7.47 TABLE_DATA

Data for Gamma table and non-linear Y table

TABLE_DATA	0003:0ADC			offset: 0x5C			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	DATA[9]	DATA[8]	DATA[7]	DATA[6]	DATA[5]	DATA[4]	DATA[3]	DATA[2]	DATA[1]	DATA[0]
R/W	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:10	RSV			Reserved
9:0	DATA	0	R/W	Data of gamma table and non linear Y enhancing table Note: The data is copied to the location pointed by GAMMA_ADDR register. The address is auto incremented after every read / write in the TABLE_DATA register.

10.7.48 HG_CONTROL

Histogram control

HG_CTL	0003:0ADE				offset: 0x5E		default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	CLR	ALWAYS	INT_ENB	CFA	BINS[1]	BINS[0]	RGN_ENB[1]	RGN_ENB[0]	SHIFT[2]	SHIFT[1]	SHIFT[0]	TART_BUS
R/W	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:12	RSV			Reserved
11	CLR	0	R/W	<u>Clear Data after read</u> » 0: Enable clear when read « » 1: Disable clear when read «
10	ALWAYS	0	R/W	<u>Histogram always On</u> » 0: Disable (Start oneshot by writing 1 to START_BUSY) « » 1: Enable (Re-start automatically at beginning of each preview frame) «
9	INT_ENB	0	R/W	<u>Interrupt enable</u> » 0: Disable interrupt « » 1: Enable interrupt «
8	CFA	0	R/W	<u>CFA pattern</u> » 0: 2D (2 x 2) conventional patter « » 1: reserved «
7:6	BNS	0	R/W	<u>No of bins per histogram</u> » 0: 32 bins « » 1: 64 bins « » 2: 128 bins « » 3: 256 bins «
5:4	RGN_ENB	0	R/W	<u>Number of regions enabled</u>
3:1	SHIFT	0	R/W	No of bits to right shift data before accessing bins (0-7)
0	START_BUSY	0	R/W	Write When ALWAYS = 0, 1 to start histogram calculation Read 1 - Calculation under progress 0 - Calculation is done

10.7.51 HG_R0_VSTART

Histogram region 0 Vertical start location

[illegible][illegible]

10.7.52 HG_R0_VSIZE

Histogram region 0 vertical size

[illegible][illegible]

10.7.53 HG_R1_HSTART

Histogram region 1 horizontal start location

[illegible][illegible]

10.7.54 HG R1 HSIZE

Histogram region 1 horizontal size

[illegible][illegible]

10.7.55 HG_R1_VSTART

Histogram region 1 Vertical start location

[illegible][illegible]

10.7.56 HG_R1_VSIZE

Histogram region 1 vertical size

[illegible][illegible]

10.7.59 HG_R2_VSTART

Histogram region 2 Vertical start location

[illegible][illegible]

10.7.60 HG_R2_VSIZE

Histogram region 2 vertical size

[illegible][illegible]

10.7.61 HG_R3_HSTART

Histogram region 3 horizontal start location

[illegible][illegible]

10.7.62 HG_R3_HSIZE

Histogram region 3 horizontal size

[illegible][illegible]

10.7.63 HG_R3_VSTART

Histogram region 3 Vertical start location

[illegible][illegible]

10.7.64 HG_R3_VSIZE

Histogram region 3 vertical size

[illegible][illegible]

11 Hardware 3A

11.1 Introduction

DM320 provides real-time 3A (Auto Exposure / Auto White Balance / Auto Focus) statistics function. ARM can use these statistics to compute the optimal 3A control parameters.

AE/AWB statistics is calculated by using up to 96 windows. The windows are of the same size and organized as an m-by-n matrix. The organization of the matrix, the size of the windows and the method of calculation are user-selectable.

AF statistics is calculated by using up to 96 windows. The position and the size of each window can be independently configured.

11.2 Features

Auto Exposure / Auto White Balance:

- 96 windows organized as m-by-n matrix.
- Number of windows in horizontal direction is between 1 – 12 and in vertical direction is between 1 - 8
- Total number of pixels per window in horizontal direction is between 2 – 256 and in vertical direction is between 4 - 128
- Sampling step for calculation in horizontal and vertical direction is between 2 - 32. Total number of 2x2 blocks in a window is up to 256
- Outputs for each window are the accumulated R, Gr, Gb and B. The results are 16-bit integers stored in SDRAM readable by ARM

Auto Focus:

- Total number of windows is up to 96 and is organized as a m-by-n matrix
- The AF calculation can be performed only on green pixel data in the window
- Two biquad filters are used to filter the green or Y pixel data. The coefficients of both filters are 10-bit wide and can be set by software
- Number of windows in horizontal direction is between 1 – 12 and in vertical direction is between 1 - 8
- Total number of green pixels per window in horizontal direction is between 1 - 256 and in vertical direction is between 2 -128

Figure 55 shows the block diagram of Hardware 3A module in DM320.

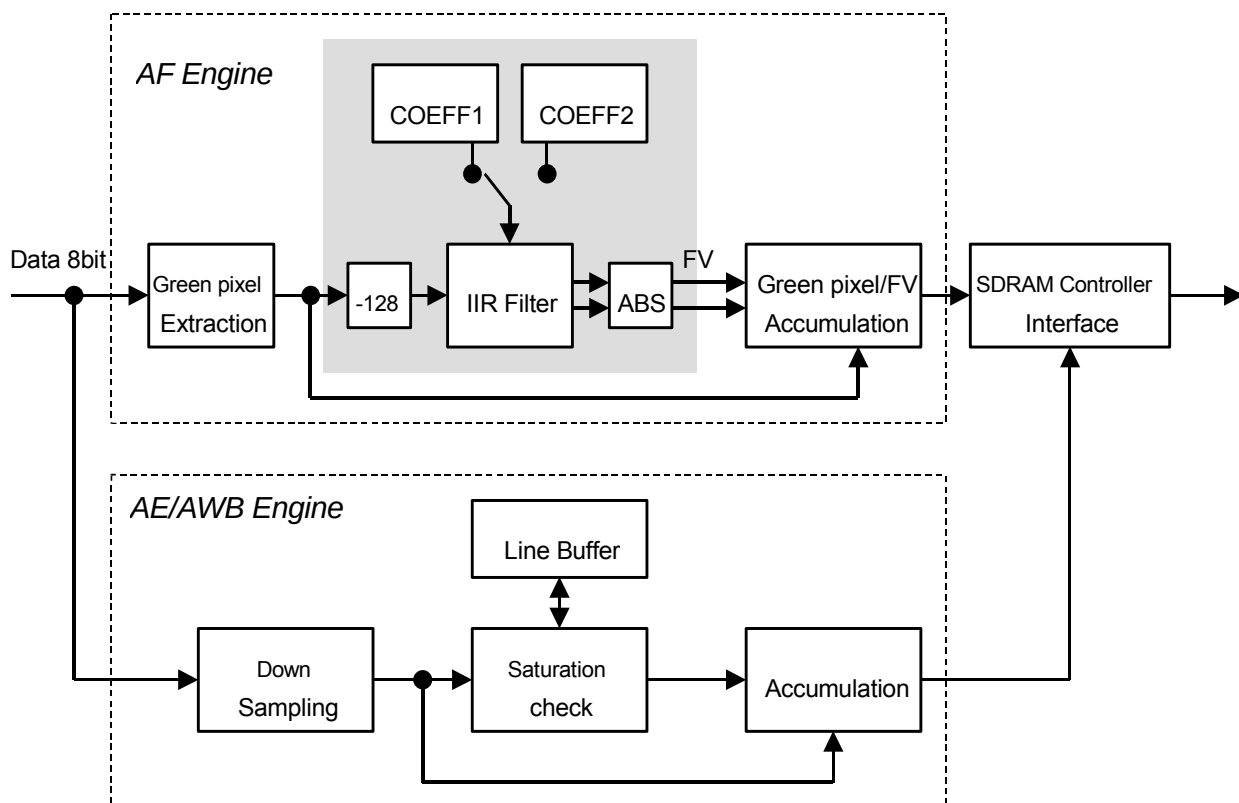


Figure 55 Block Diagram of 3A Hardware Engine

11.3 Input data for H3A Engine

The input signal to H3A engine is 8bit data selected with GWDI[2:0] of the CGAMMAWD register in CCDC.

Note: GWDI [2:0] selects 10-bit raw data, the higher 8-bits are taken as input by the H3A engine.

11.4 Auto Focus (AF) Engine

The input data to AF engine is the raw data output by the CCD Controller. The AF Engine extracts green pixel data from this raw data. The extracted green data are subtracted by 128, and fed to the filter block to calculate the Focus Value (FV), which is the absolute value of the IIR filter output. The maximum FV of each line in a Poxel is acquired if FV mode is set to 'Peak mode'. Value of the green pixels and either the FV or the maximum FV (of each line) are accumulated in the Poxel, and are stored in SDRAM.

The window configuration of the AF Engine is based on a frame formed by the extracted green pixels. Figure 56 illustrates the poxel configuration for AF engine. Table 32 illustrates the register fields used to configure AF Engine.

NOTE: Poxel in AF statistics is a window of green pixel data

Register Field	width	Description
AFEN (H3ACTRL)	1	AF Engine enable 0:disable 1:enable
GPOS(AFCTRL)	4	Green pixel position in 2x2 block Support 5/6/9/A and 1/2/4/8
PAXH (AFPAX1)	8	Paxel Width (PAXH+1) (1 – 256)
PAXV (AFPAX1)	7	Paxel Height (PAXV+1) (2 – 128)
PAXHC (AFPAX2)	4	Paxel Count for H direction (PAXHC+1) (1 – 12)
PAXVC(AFPAX2)	3	Paxel Count for V direction (PAXVC+1) (1 – 8)
PAXSH (AFPAX3)	12	Paxel start position (H) (1 – 4,095)
PAXSV (AFPAX4)	12	Paxel start position (V) (0 – 4,095)
IIRSH (AFIIRSH)	12	IIR filter start position (H) (0 – 4,095)
AFINCV(AFPAX5)	4	Line increments in a Paxel (AFINCV+1) (1-16)
FVMODE(AFCTRL)	1	Focus Value accumulation mode 0:Sum mode 1:Peak mode
COEFF1 (AFCOEFF10 to AFCOEFF110)	11x1 1	Coefficient table1 for IIR filter (decimal 6bit)
COEFF2 (AFCOEFF20 to AFCOEFF210)	11x1 1	Coefficient table2 for IIR filter (decimal 6bit)
AFSDRA (AFSDRA1, AFSDRA2)	23	SDRAM destination address to store the results of AF engine. It is automatically incremented until the two continuous buffers for the two successive frames become full.
AFSDRFLG (AFSDRFLG)	1	SDRAM buffer flag to indicate the buffer at which the results are stored.
AFERRFR(AFSDRFLG)	2	SDRAM access fail flag on previous buffer flag [0/1].
AFERR(AFSDRFLG)	1	SDRAM access fail flag. Clear by ARM write only.

Table 32: AF Engine Configuration Registers

Constraints for configuration of Auto focus engine are given below.

- PAXSH must be equal to or greater than (IIRSH + 1).
- Paxels cannot overlap the last pixel in a line. PAXSH, PAXH and PAXHC must be set accordingly.
- Paxels can overlap either the first or the last line in a frame, but not both of them. PAXSV, PAXV and PAXVC must be set accordingly.
- GPOS cannot pick continuous 2 data in a line. DM320 support 1/2/4/5/6/8/9/A values.
- PAXV=0 is NG. PAXH=0(or small value) might be problem in data transfer to SDRAM.

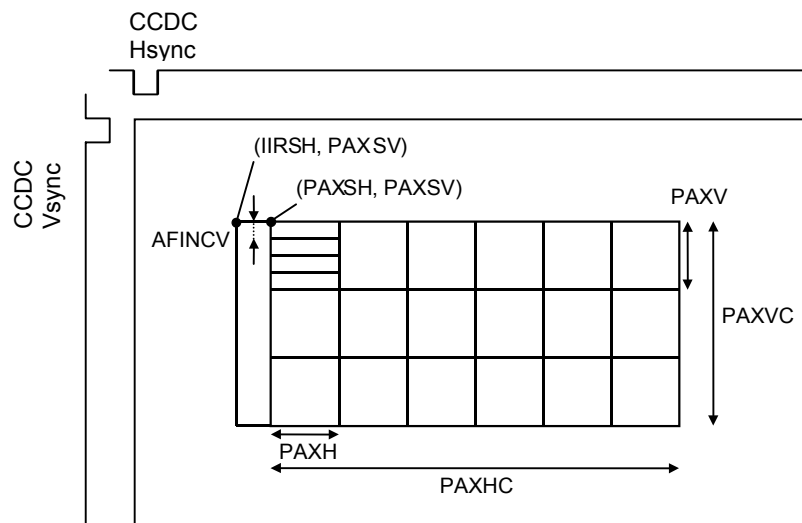


Figure 56: Paxel Configuration (green pixels only)

11.4.1 Green pixel position (GPOS)

The input signal is 8bit data, and the green pixel data is within a 2x2 window which is selected using GPOS[3:0] of AFCTRL register. Selectable green patterns are shown in the Figure 57.

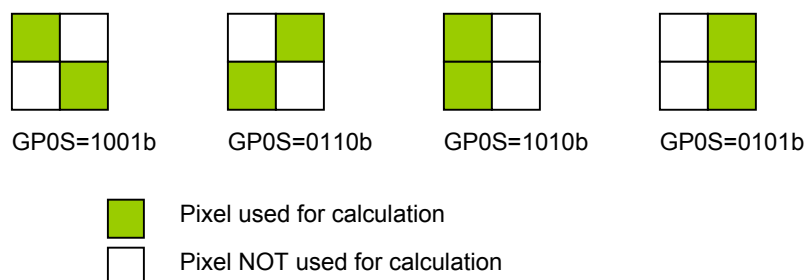


Figure 57: Sub-sampling Pattern

11.4.2 Paxel Setting

PAXH is used to set paxel width (horizontal), PAXV is used to set paxel height (vertical). Figure 58 below shows an example of setting PAXH, PAXV.

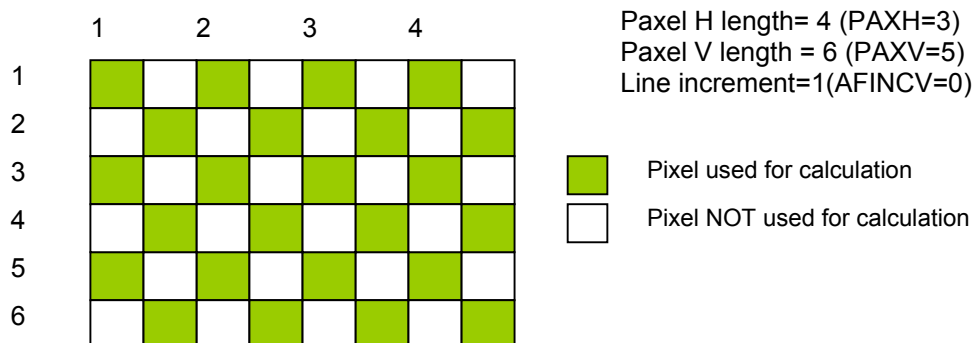


Figure 58: Paxel setting example

Note: Pixel width is number of green pixels in horizontal direction. See section 11.4.4 - Line increment also.

11.4.3 IIR Filter

There are two sets of programmable coefficient tables, AFCOEFF10 to AFCOEFF110 and AFCOEFF20 to AFCOEFF210. FV calculation is performed using both tables. Shift registers of the IIR filter are cleared to zero once in a line, at the next position of IIRSH in AFIIRSH register.

	Precision	Range
Filter Input	[Value of a green pixel (8bit)] – 128	-128 to 127
Coefficient	Signed 11bit (decimal 6 bit)	-16 to 15+63/64
Filter Output	Signed 12bit (decimal 4 bit)	-128 to 127+15/16

11.4.4 Line increment

Line increment within each paxel can be specified using AFINCV field in AFPAX5 register. Figure 59 shows an example of line increment within a paxel. IIR calculation is performed for only the lines selected using line increment.

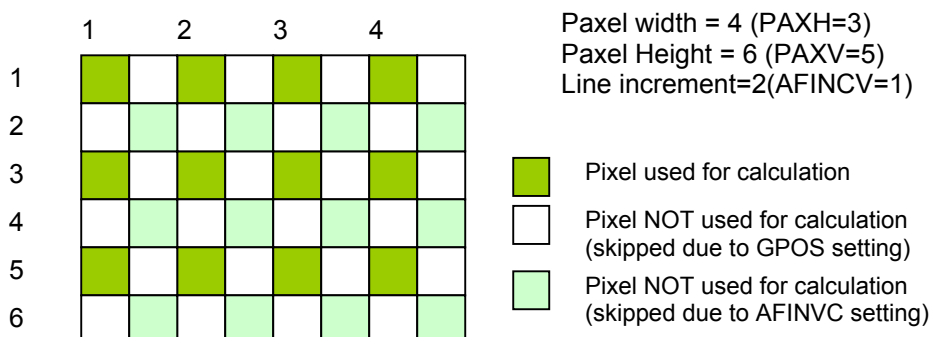


Figure 59: Line increment setting Example

11.4.5 AF Engine Output

1. Green Sum: For each paxel the green sum is output to SDRAM. This is the 32bit(23bit) in width.
2. FV sum: For each paxel the FV sum is output to SDRAM. This is 32bit(27bit) (decimal 4bit) x 2 / Peak-FV-value Sum.

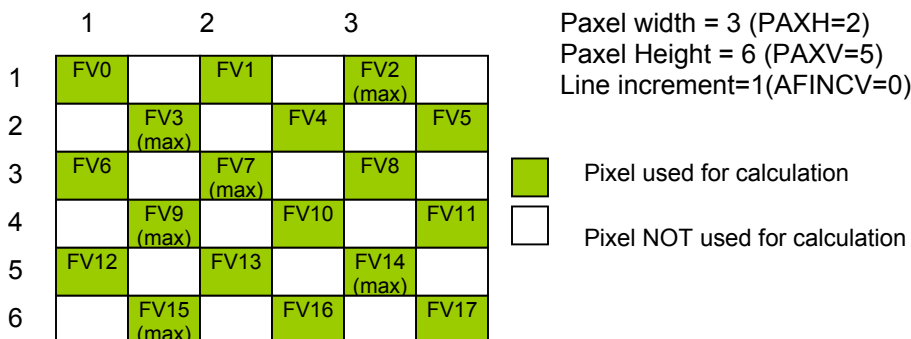


Figure 60: FV Accumulation Mode Example

Figure 60 shows an example of FV accumulation mode. Here FVn is FV value calculated by IIR filter using a coefficient table. FVn (max), is maximum FV value in a line within a paxel.

In sum mode, FV value output by AF engine for each paxel is $FV_{out} = FV_0 + FV_1 + FV_2 + \dots + FV_{17}$

In peak mode, FV value output by AF Engine for each paxel is sum of FVn max, i.e $FV_{out} = FV_2 + FV_3 + FV_7 + FV_9 + FV_{14} + FV_{15}$

11.4.5.1 Data Transfer to SDRAM

Results of the two adjacent Paxels in a row are gathered and transferred to the SDRAM at the same time, therefore, if PAXHC is set to 2 and PAXVC is set to 1, there will be 6 sets $[(PAXHC+1) \times (PAXVC+1)]$ of data transfer within an image as shown in Figure 61. The results of the first Paxel in an image are stored to the address specified in the AFSDRA1 and AFSDRA2 registers. The address is automatically incremented until the two continuous buffers for the two successive frames become full. SDRAM buffer flag AFSDRFLG in AFSDRFLG register indicates which buffer is used to store the results.

Data transfer might fail in special cases like small PAXH number, SDRAM access waiting or ccd-sdram clock issue. DM320 report fail-condition of SDRAM-transfer to error flag register. Once error happened, AFERR bit in AFSDRFLG register gets set. It is cleared by writing '0' to it. AFERRFR0 and AFERRFR1 bits in AFSDRFLG register store the previous error status and update on each v-sync timing.

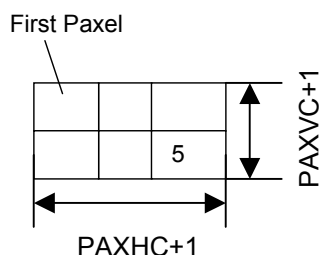


Figure 61 Exampe Paxel Configuration

In Table 33, the accumulation of green pixel values in the paxel is indicated as Green sum. The IIR filter output FV is indicated as FV sum-1, FV sum-2. These are the accumulated values using coefficient table 1 and 2 respectively. In addition, as AF result is output in sets of two paxels, when the number of horizontal paxel shown in Table 33 is odd number, dummy data is output for a paxel. The part in gray in Table 33 shows the dummy data output.

SDRAM address (byte address)	31	16	15	0	
AFSDRA	Green sum			(Paxel 0)	1 st transfer start
	FV sum -1 /Peak sum -1			(Paxel 0)	
	FV sum -2 /Peak sum -2			(Paxel 0)	
	Reserved				
AFSDRA + 32	Green sum			(Paxel 1)	1 st transfer end
	FV sum -1 /Peak sum -1			(Paxel 1)	
	FV sum -2 /Peak sum -2			(Paxel 1)	
	Reserved				
AFSDRA + 64	Green sum			(Paxel 2)	2 nd transfer start
	FV sum -1 /Peak sum -1			(Paxel 2)	
	FV sum -2 /Peak sum -2			(Paxel 2)	
	Reserved				
AFSDRA + 96	Green sum			(Paxel 1)	2 nd transfer end
	FV sum -1 /Peak sum -1			(Paxel 1)	
	FV sum -2 /Peak sum -2			(Paxel 1)	
	Reserved				
AFSDRA + 128	Green sum			(Paxel 3)	3 rd transfer start
	FV sum -1 /Peak sum -1			(Paxel 3)	
	FV sum -2 /Peak sum -2			(Paxel 3)	
	Reserved				
AFSDRA + 160	Green sum			(Paxel 4)	3 rd transfer end
	FV sum -1 /Peak sum -1			(Paxel 4)	
	FV sum -2 /Peak sum -2			(Paxel 4)	
	Reserved				
AFSDRA + 192	Green sum			(Paxel 5)	4 th transfer start
	FV sum -1 /Peak sum -1			(Paxel 5)	
	FV sum -2 /Peak sum -2			(Paxel 5)	
	Reserved				
AFSDRA + 224	Green sum			(Paxel 4)	4 th transfer end
	FV sum -1 /Peak sum -1			(Paxel 4)	
	FV sum -2 /Peak sum -2			(Paxel 4)	
	Reserved				

Table 33: AF SDRAM Output Format

11.4.5.1.1 Calculation of current SDRAM result buffer

Code below shows how to calculate the SDRAM buffer address where the AF results are stored for the previous frame.

```

Uint32 AF_getCurFrameDataAddress() {
    Uint32 dataAddr;

    dataAddr = H3A_FGET(AFSDRA1, ADRH);
    dataAddr = dataAddr << 16;
    dataAddr |= H3A_RGET(AFSDRA2);
    dataAddr *= 32;
    dataAddr += (Uint32)SDRAM_MEMORY_BASE;

    if (H3A_FGET(AFSDRFLG, AFSDRFLG)) {
        if ((H3A_FGET(AFPAX2, PAXHC) + 1) & 0x1) {
            dataAddr +=
                (H3A_FGET(AFPAX2, PAXHC) + 1 + 1) *
                H3A_FGET(AFPAX2, PAXVC) + 1) * 16;
        } else {
            dataAddr +=
                (H3A_FGET(AFPAX2, PAXHC) + 1) *
                (H3A_FGET(AFPAX2, PAXVC) + 1) * 16;
        }
    }
    return dataAddr;
}

```

11.5 Auto Exposure, Auto White-Balance Engine (AEWB)

The input data to AEWB engine is the raw data output by the CCD Controller. All color component pixels (R, Gr, Gb, B) are input to the AEWB engine. The pixels in the sub-sampled 2x2 blocks are accumulated in the window as AVE1. If any of the pixels in a 2x2 block are equal to or greater than the limit value AVE2LMT, in AEWCTRL register, the block is treated as a saturated block. Only the blocks, which are not saturating are counted into CNT2. The pixels, which are clipped to AVE2LMT in the sub-sampled 2x2 blocks are accumulated in the window as AVE2, and are stored in SDRAM with AVE1.

Register	width	Description
AEWEN (H3ACTRL)	1	AE/AWB Engine enable 0:disable 1:enable
WINH (AEWWIN1)	7	Window Width (even number) $((\text{WINH} + 1) \times 2)$ (2 – 256)
WINV (AEWWIN1)	6	Window Height (even number) $((\text{WINV} + 1) \times 2)$ (4 – 128)
WINHC (AEWWIN2)	4	Window Count for H direction $(\text{WINHC} + 1)$ (1 – 12)
WINVC (AEWWIN2)	3	Window Count for V direction $(\text{WINVC} + 1)$ (1 – 8)
WINSH (AEWWIN3)	11	X position of the Window start point $(\text{WINSH} \times 2)$ (even number)
WINSV (AEWWIN4)	11	Y position of the Window start point $(\text{WINSV} \times 2)$ (even number)
AEWINCH (AEWWIN5)	4	Sampling point increments (X) in a Window (even number) $((\text{AEWINCH} + 1) \times 2)$ (2-32)
AEWINCV (AEWWIN5)	4	Sampling point increments (Y) in a Window (even number) $((\text{AEWINCV} + 1) \times 2)$ (2-32)
AVE2LMT (AEWCTRL)	8	AVE2 limit value
AEWSDRA (AEWSDRA1 ,	23	SDRAM destination address to store the results of AE/AWB engine. It is automatically incremented until the two continuous buffers for the two
AEWSDRFLG (AEWSDRFLG)	1	SDRAM buffer flag to indicate the buffer at which the results are stored.
AEWERRFR (AEWSDRFLG)	2	SDRAM access fail flag on previous buffer flag [0/1].
AEWERR (AEWSDRFLG)	1	SDRAM access fail flag. Clear by ARM write only.

Table 34: AEWB Configuration Registers

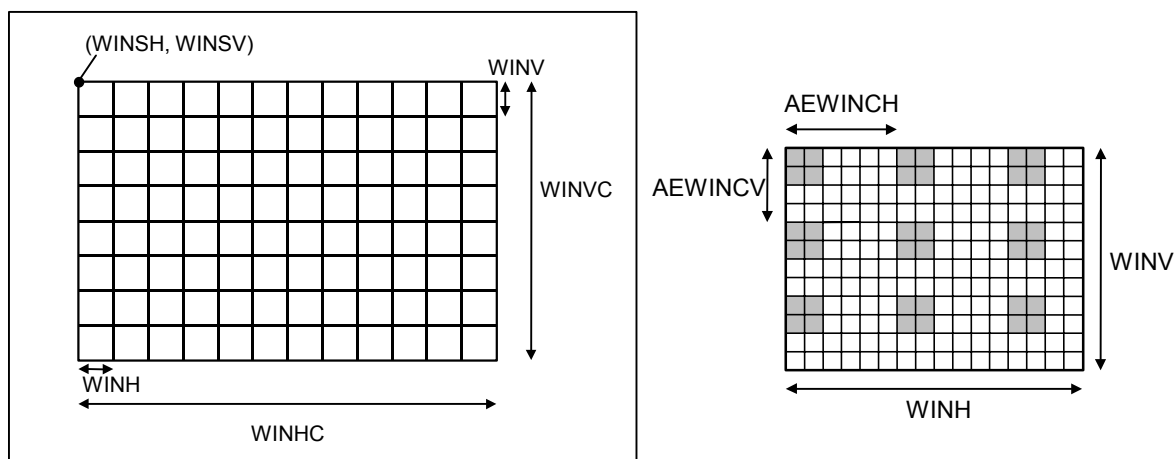


Figure 62: AEWB Window Configuration

Constraints for the configuration of AE/AWB parameters:

- The number of accumulated 2x2 blocks in a Window cannot exceed 256. WINH, WINV, AEWINCH and AEWINCV must be set accordingly.
- The number of sub-sampled pixels in a line cannot exceed 2816. WINH, WINHC and AEWINCH must be set accordingly. (Used 1408x16 line memory.)
- Windows can overlap either the first or the last line in a frame, but not both of them. WINSV, WINV and WINVC must be set accordingly.
- WINV=0 is NG. WINH=0(or small value) might be problem in data transfer to SDRAM.

11.5.1 AE/AWB Engine Output

1. AVE1[i][j] : 16bit (Array of accumulated pixels in the sub-sampled 2x2 blocks. Clipped to 65,535.)
2. AVE2[i][j] : 16bit (Clipped to 65,535. Array of accumulated pixels after clipped operation to AVE2LMT in the sub-sampled 2x2 blocks)
3. CNT2[i] : 16bit(9bit) (The number of the sub-sampled 2x2 blocks which are not saturating)
4. i: window number
5. j: pixel position in 2x2 block (0:upper left, 1:upper right, 2:lower left, 3:lower right)

Table 35 shows the output of AEWB module.

11.5.2 Data transfer to SDRAM

Results of each window are transferred to the SDRAM at the same time. Therefore, if WINHC is set to 1 and WINVC is set to 1, there will be 4 sets [(WINHC+1) x (WINVC+1)] of data transfer within an image as shown in Figure 63. The results of the first Window in an image are stored to the address specified in the AEWSRA1 and AEWSRA2 registers. The address is automatically incremented until the two continuous buffers for the two successive frames become full. SDRAM buffer flag AEWSRFLG in AEWSRFLG register indicates the buffer at which the results are stored.

Data transfer might fail in special case like small WINW number, SDRAM access waiting or ccd-sdram clock issue. DM320 report fail-condition of SDRAM-transfer to error flag register. Once error happened, AEWERR bit in AEWSRFLG register gets set. It is cleared by writing '0' to it. AEWERRFR0 and AEWERRFR1 bits in AEWSRFLG register store the previous error state and update on each v-sync timing.

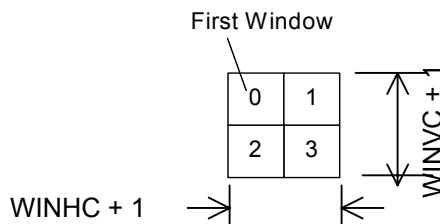


Figure 63: Example Window Configuration

NOTE: SDRAM Address is specified as offset from start of SDRAM in units of 32bytes

SDRAM address (byte address)	31	16	15	0	
AEWSDRA	AVE1[0][1]		AVE1[0][0]		1 st transfer start
	AVE1[0][3]		AVE1[0][2]		
	AVE2[0][1]		AVE2[0][0]		
	AVE2[0][3]		AVE2[0][2]		
	All zero		CNT2[0]		1 st transfer end
	All zero				
	All zero				
	All zero				
AEWSDRA + 32	AVE1[1][1]		AVE1[1][0]		2 nd transfer start
	AVE1[1][3]		AVE1[1][2]		
	AVE2[1][1]		AVE2[1][0]		
	AVE2[1][3]		AVE2[1][2]		
	All zero		CNT2[1]		2 nd transfer end
	All zero				
	All zero				
	All zero				
AEWSDRA + 64	AVE1[2][1]		AVE1[2][0]		3 rd transfer start
	AVE1[2][3]		AVE1[2][2]		
	AVE2[2][1]		AVE2[2][0]		
	AVE2[2][3]		AVE2[2][2]		
	All zero		CNT2[2]		3 rd transfer end
	All zero				
	All zero				
	All zero				
AEWSDRA + 96	AVE1[3][1]		AVE1[3][0]		4 th transfer start
	AVE1[3][3]		AVE1[3][2]		
	AVE2[3][1]		AVE2[3][0]		
	AVE2[3][3]		AVE2[3][2]		
	All zero		CNT2[3]		4 th transfer end
	All zero				
	All zero				
	All zero				

Table 35: AEWB SDRAM Data Format

11.5.3 Calculation of current SDRAM result buffer

Code below shows how to calculate the SDRAM buffer for AEWB results of the previous frame.

```
Uint32 AEWB_getCurFrameDataAddress()
{
    Uint32 dataAddr;
    dataAddr = H3A_FGET(AEWSDRA1, ADRH);
    dataAddr = dataAddr << 16;
    dataAddr |= H3A_RGET(AEWSDRA2);
    dataAddr *= 32;
    dataAddr += (Uint32)SDRAM_MEMORY_BASE;
    if (H3A_FGET(AEWSDRFLG, AEWSDRFLG)) {
        dataAddr +=
            (H3A_FGET(AEWWIN2, WINHC) + 1) *
            (H3A_FGET(AEWWIN2, WINVC) + 1) * 32;
    }
    return dataAddr;
}
```

NOTE: All registers in AF/AE/AWB Statistics Engine are latched by CCDC VD signal.

11.6 Hardware 3A register Map (H3A)

Address	Register	Description
0003 0B80	H3ACTRL	Hardware 3A Control Register
0003 0B82	AFCTRL	AF Engine Control Register
0003 0B84	AFPAX1	AF Engine Poxel Register #1
0003 0B86	AFPAX2	AF Engine Poxel Register #2
0003 0B88	AFPAX3	AF Engine Poxel Register #3
0003 0B8A	AFPAX4	AF Engine Poxel Register #4
0003 0B8C	AFIRSH	AF Engine IIR Shift register
0003 0B8E	AFPAX5	AF Engine Poxel Register #5
0003 0B90	AFSDRA1	AF Engine SDRAM Address (high)
0003 0B92	AFSDRA2	AF Engine SDRAM Address (low)
0003 0B94	AFSDRFLG	AF Engine SDRAM Buffer Flag
0003 0B96	AFCOEFF10	AF Engine IIR Filter Table #1, Coefficient #0
0003 0B98	AFCOEFF11	AF Engine IIR Filter Table #1, Coefficient #1
0003 0B9A	AFCOEFF12	AF Engine IIR Filter Table #1, Coefficient #2
0003 0B9C	AFCOEFF13	AF Engine IIR Filter Table #1, Coefficient #3
0003 0B9E	AFCOEFF14	AF Engine IIR Filter Table #1, Coefficient #4
0003 0BA0	AFCOEFF15	AF Engine IIR Filter Table #1, Coefficient #5
0003 0BA2	AFCOEFF16	AF Engine IIR Filter Table #1, Coefficient #6
0003 0BA4	AFCOEFF17	AF Engine IIR Filter Table #1, Coefficient #7
0003 0BA6	AFCOEFF18	AF Engine IIR Filter Table #1, Coefficient #8
0003 0BA8	AFCOEFF19	AF Engine IIR Filter Table #1, Coefficient #9
0003 0BAA	AFCOEFF110	AF Engine IIR Filter Table #1, Coefficient #10
0003 0BAC	AFCOEFF20	AF Engine IIR Filter Table #2, Coefficient #0
0003 0BAE	AFCOEFF21	AF Engine IIR Filter Table #2, Coefficient #1
0003 0BB0	AFCOEFF22	AF Engine IIR Filter Table #2, Coefficient #2
0003 0BB2	AFCOEFF23	AF Engine IIR Filter Table #2, Coefficient #3
0003 0BB4	AFCOEFF24	AF Engine IIR Filter Table #2, Coefficient #4
0003 0BB6	AFCOEFF25	AF Engine IIR Filter Table #2, Coefficient #5
0003 0BB8	AFCOEFF26	AF Engine IIR Filter Table #2, Coefficient #6
0003 0BBA	AFCOEFF27	AF Engine IIR Filter Table #2, Coefficient #7
0003 0BBC	AFCOEFF28	AF Engine IIR Filter Table #2, Coefficient #8
0003 0BBE	AFCOEFF29	AF Engine IIR Filter Table #2, Coefficient #9
0003 0BC0	AFCOEFF210	AF Engine IIR Filter Table #2, Coefficient #10
0003 0BC2	AEWCTRL	AE/AWB Engine Control Register
0003 0BC4	AEWIN1	AE/AWB Engine Window Register #1
0003 0BC6	AEWIN2	AE/AWB Engine Window Register #2
0003 0BC8	AEWIN3	AE/AWB Engine Window Register #3
0003 0BCA	AEWIN4	AE/AWB Engine Window Register #4
0003 0BCC	AEWIN5	AE/AWB Engine Window Register #5
0003 0BCE	AEWSDRA1	AE/AWB Engine SDRAM Address (high)
0003 0BD0	AEWSDRA2	AE/AWB Engine SDRAM Address (low)
0003 0BD2	AEWSDRFLG	AE/AWB Engine SDRAM Buffer Flag

11.7 Hardware 3A Registers

11.7.1 H3ACTRL

Hardware 3A Control Register

H3ACTRL	0003:0B80			offset: 0x00			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	AEWEN	RSV	RSV	RSV	AFEN
R/W	-	-	-	-	-	-	-	-	-	-	-	R/W	-	-	-	R/W
Default	-	-	-	-	-	-	-	-	-	-	-	0	-	-	-	0

Bit	Name	Reset Value	R/W	Function
15:5	RSV			Reserved
4	AEWEN	0	R/W	AE/AWB Engine enable. » 0: Disable « » 1: Enable «
3:1	RSV			Reserved
0	AFEN	0	R/W	AF Engine enable. » 0: Disable « » 1: Enable «

11.7.2 AFCTRL

AF Engine Control Register

AFCTRL	0003:0B82			offset: 0x02												default: 0x0000		
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	FVMODE	GPOSUL	GPOSUR	GPOSLL	GPOSLR		
R/W	-	-	-	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W		
Default	-	-	-	-	-	-	-	-	-	-	-	0	0	0	0	0		

Bit	Name	Reset Value	R/W	Function
15:5	RSV			Reserved
4	FVMODE	0	R/W	[AF Engine] Focus Value(FV) accumulation mode. » 0: Sum mode « » 1: Peak mode «
3	GPOSUL	0	R/W	[AF Engine] Green pixel extraction. upper left in 2x2 block (even pixel, even line). » 0: Other than green « » 1: Green «
2	GPOSUR	0	R/W	[AF Engine] Green pixel extraction. upper right in 2x2 block (odd pixel, even line). » 0: Other than green « » 1: Green «
1	GPOSLL	0	R/W	[AF Engine] Green pixel extraction. lower left in 2x2 block (even pixel, odd line). » 0: Other than green « » 1: Green «
0	GPOSLR	0	R/W	[AF Engine] Green pixel extraction. lower right in 2x2 block (odd pixel, odd line). » 0: Other than green « » 1: Green «

11.7.3 AFPAX1

AF Engine Paxel Register #1

[illegible][illegible]

11.7.4 AFPAX2

AF Engine Paxel Register #2

AFPAX2	0003:0B86				offset: 0x06				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	PAXVC[2]	PAXVC[1]	PAXVC[0]	RSV	RSV	RSV	RSV	PAXHC[3]	PAXHC[2]	PAXHC[1]	PAXHC[0]
R/W	-	-	-	-	-	R/W	R/W	R/W	-	-	-	-	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	0	0	0	-	-	-	-	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:8	PAXVC	0	R/W	[AF Engine] Paxel Count for V direction. Paxel Count is (PAXVC +1) and ranges: 1-8.
7:4	RSV			Reserved
3:0	PAXHC	0	R/W	[AF Engine] Paxel Count for H direction. Paxel Count is (PAXHC +1) and ranges: 1-12. Setting a value greater than 11 to PAXHC is not allowed.

11.7.5 AFPAX3

AF Engine Paxel Register #3

AFPAX3	0003:0B88				offset: 0x08				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	PAXSH[11]	PAXSH[10]	PAXSH[9]	PAXSH[8]	PAXSH[7]	PAXSH[6]	PAXSH[5]	PAXSH[4]	PAXSH[3]	PAXSH[2]	PAXSH[1]	PAXSH[0]
R/W	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:12	RSV			Reserved
11:0	PAXSH	0	R/W	[AF Engine] Paxel start position (H direction, green pixels). Range: 1-4,095. PAXSH must be equal to or greater than (IIRSH + 1).

11.7.6 AFPAX4

AF Engine Paxel Register #4

AFPAX4	0003:0B8A			offset: 0x0A		default: 0x0000										
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	PAXSV[11]	PAXSV[10]	PAXSV[9]	PAXSV[8]	PAXSV[7]	PAXSV[6]	PAXSV[5]	PAXSV[4]	PAXSV[3]	PAXSV[2]	PAXSV[1]	PAXSV[0]
R/W	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:12	RSV			Reserved
11:0	PAXSV	0	R/W	[AF Engine] Paxel start position (V direction, green lines). Range: 0-4,095.

11.7.7 AFIIRSH

AF Engine IIR Shift register

AFIIRSH	0003:0B8C			offset: 0x0C			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	IIRSH[11]	IIRSH[10]	IIRSH[9]	IIRSH[8]	IIRSH[7]	IIRSH[6]	IIRSH[5]	IIRSH[4]	IIRSH[3]	IIRSH[2]	IIRSH[1]	IIRSH[0]
R/W	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:12	RSV			Reserved
11:0	IIRSH	0	R/W	[AF Engine] IIR filter start position (H direction, green pixels). Range: 0-4,095. Shift registers of the IIR filter are cleared to zero at the next position of IIRSH.

11.7.8 AFPAX5

AF Engine Poxel Register #5

AFPAX5	0003:0B8E			offset: 0x0E				default: 0x0000								
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	AFINCV[3]	AFINCV[2]	AFINCV[1]	AFINCV[0]	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV
R/W	-	-	-	-	R/W	R/W	R/W	R/W	-	-	-	-	-	-	-	-
Default	-	-	-	-	0	0	0	0	-	-	-	-	-	-	-	-

Bit	Name	Reset Value	R/W	Function
15:12	RSV			Reserved
11:8	AFINCV	0	R/W	[AF Engine] Line increments in a Poxel (number of green lines). Increments are (AFINCV +1) and ranges: 1-16.
7:0	RSV			Reserved

11.7.9 AFSDRA1

AF Engine SDRAM Address (high)

AFSDRA1	0003:0B90			offset: 0x10		default: 0x0000										
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	ADRH[6]	ADRH[5]	ADRH[4]	ADRH[3]	ADRH[2]	ADRH[1]	ADRH[0]
R/W	-	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	-	-	-	-	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:7	RSV			Reserved
6:0	ADRH	0	R/W	[AF Engine] Upper 7 bits of SDRAM destination address to store the results of AF engine. There are two continuous buffers, and the results of the two successive frames are stored to the different buffers. Buffer size depends on PAXHC and PAXVC. AFSDRA is the start address of the first buffer, and the address is automatically incremented until data transfer for the two frame ends (address is 32 Byte unit) specified as offset from start of SDRAM.

11.7.10 AFSDRA2

AF Engine SDRAM Address (low)

AFSDRA2	0003:0B92			offset: 0x12												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	ADRL[15]	ADRL[14]	ADRL[13]	ADRL[12]	ADRL[11]	ADRL[10]	ADRL[9]	ADRL[8]	ADRL[7]	ADRL[6]	ADRL[5]	ADRL[4]	ADRL[3]	ADRL[2]	ADRL[1]	ADRL[0]	
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:0	ADRL	0	R/W	[AF Engine] Lower 16 bits of SDRAM destination address to store the results of AF engine. There are two continuous buffers, and the results of the two successive frames are stored to the different buffers. Buffer size depends on PAXHC and PAXVC. AFSDRA is the start address of the first buffer, and the address is automatically incremented until data transfer for the two frame ends (address is 32 Byte unit) specified as offset from start of SDRAM.

11.7.11 AFSDRFLG

AF Engine SDRAM Buffer Flag

AFSDRFLG	0003:0B94			offset: 0x14												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	AFERR	AFERRFR1	AFERRFR0	RSV	RSV	RSV	AFSDRFLG	
R/W	-	-	-	-	-	-	-	-	-	R/W	R	R	-	-	-	R	
Default	-	-	-	-	-	-	-	-	-	0	0	0	-	-	-	0	

Bit	Name	Reset Value	R/W	Function
15:7	RSV			Reserved
6	AFERR	0	R/W	SDRAM access fail flag, Clear by ARM write
5	AFERRFR1	0	R	SDRAM access fail on previous buffer flag 1
4	AFERRFR0	0	R	SDRAM access fail on previous buffer flag 0
3:1	RSV			Reserved
0	AFSDRFLG	0	R	SDRAM buffer flag to indicate the buffer at which the results of the last frame is stored. » 0: Results stored at first buffer starting from AFSDRA « » 1: Results are stored at the second buffer «

11.7.12 AFCOEFF10

AF Engine IIR Filter Table #1, Coefficient #0

AFCOEFF10	0003:0B96			offset: 0x16		default: 0x0000										
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	COEFF10[10]	COEFF10[9]	COEFF10[8]	COEFF10[7]	COEFF10[6]	COEFF10[5]	COEFF10[4]	COEFF10[3]	COEFF10[2]	COEFF10[1]	COEFF10[0]
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:0	COEFF10	0	R/W	[AF Engine] IIR filter Coefficient #0 <Table1>. Decimal 6 bit: $-16 \leq \text{coeff} \leq 15 + 63/64$

11.7.13 AFcoeff11

AF Engine IIR Filter Table #1, Coefficient #1

AFCOEFF11	0003:0B98			offset: 0x18			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	COEFF11[10]	COEFF11[9]	COEFF11[8]	COEFF11[7]	COEFF11[6]	COEFF11[5]	COEFF11[4]	COEFF11[3]	COEFF11[2]	COEFF11[1]	COEFF11[0]
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:0	COEFF11	0	R/W	[AF Engine] IIR filter Coefficient #1 <Table1>. Decimal 6 bit: $-16 \leq \text{coeff} \leq 15+63/64$

11.7.14 AFcoeff12

AF Engine IIR Filter Table #1, Coefficient #2

AFCOEFF12		0003:0B9A			offset: 0x1A		default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	COEFF12[10]	COEFF12[9]	COEFF12[8]	COEFF12[7]	COEFF12[6]	COEFF12[5]	COEFF12[4]	COEFF12[3]	COEFF12[2]	COEFF12[1]	COEFF12[0]
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:0	COEFF12	0	R/W	[AF Engine] IIR filter Coefficient #2 <Table1>. Decimal 6 bit: $-16 \leq \text{coeff} \leq 15+63/64$

11.7.15 AFcoeff13

AF Engine IIR Filter Table #1, Coefficient #3

AFCOEFF13		0003:0B9C			offset: 0x1C		default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	COEFF13[10]	COEFF13[9]	COEFF13[8]	COEFF13[7]	COEFF13[6]	COEFF13[5]	COEFF13[4]	COEFF13[3]	COEFF13[2]	COEFF13[1]	COEFF13[0]
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:0	COEFF13	0	R/W	[AF Engine] IIR filter Coefficient #3 <Table1>. Decimal 6 bit: $-16 \leq \text{coeff} \leq 15+63/64$

11.7.16 AFCOEFF14

AF Engine IIR Filter Table #1, Coefficient #4

AFCOEFF14	0003:0B9E			offset: 0x1E			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	COEFF14[10]	COEFF14[9]	COEFF14[8]	COEFF14[7]	COEFF14[6]	COEFF14[5]	COEFF14[4]	COEFF14[3]	COEFF14[2]	COEFF14[1]	COEFF14[0]
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:0	COEFF14	0	R/W	[AF Engine] IIR filter Coefficient #4 <Table1>. Decimal 6 bit: $-16 \leq \text{coeff} \leq 15 + 63/64$

11.7.17 AFCOEFF15

AF Engine IIR Filter Table #1, Coefficient #5

AFCOEFF15			0003:0BA0			offset: 0x20			default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	COEFF15[10]	COEFF15[9]	COEFF15[8]	COEFF15[7]	COEFF15[6]	COEFF15[5]	COEFF15[4]	COEFF15[3]	COEFF15[2]	COEFF15[1]	COEFF15[0]
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:0	COEFF15	0	R/W	[AF Engine] IIR filter Coefficient #5 <Table1>. Decimal 6 bit: $-16 \leq \text{coeff} \leq 15 + 63/64$

11.7.18 AFCOEFF16

AF Engine IIR Filter Table #1, Coefficient #6

AFCOEFF16			0003:0BA2			offset: 0x22		default: 0x0000								
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	COEFF16[10]	COEFF16[9]	COEFF16[8]	COEFF16[7]	COEFF16[6]	COEFF16[5]	COEFF16[4]	COEFF16[3]	COEFF16[2]	COEFF16[1]	COEFF16[0]
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:0	COEFF16	0	R/W	[AF Engine] IIR filter Coefficient #6 <Table1>. Decimal 6 bit: $-16 \leq \text{coeff} \leq 15 + 63/64$

11.7.19 AF_COEFF17

AF Engine IIR Filter Table #1, Coefficient #7

AFCOEFF17	0003:0BA4			offset: 0x24		default: 0x0000										
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	COEFF17[10]	COEFF17[9]	COEFF17[8]	COEFF17[7]	COEFF17[6]	COEFF17[5]	COEFF17[4]	COEFF17[3]	COEFF17[2]	COEFF17[1]	COEFF17[0]
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:0	COEFF17	0	R/W	[AF Engine] IIR filter Coefficient #7 <Table1>. Decimal 6 bit: $-16 \leq \text{coeff} \leq 15 + 63/64$

11.7.20 AF_COEFF18

AF Engine IIR Filter Table #1, Coefficient #8

AFCOEFF18	0003:0BA6			offset: 0x26		default: 0x0000										
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	COEFF18[10]	COEFF18[9]	COEFF18[8]	COEFF18[7]	COEFF18[6]	COEFF18[5]	COEFF18[4]	COEFF18[3]	COEFF18[2]	COEFF18[1]	COEFF18[0]
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:0	COEFF18	0	R/W	[AF Engine] IIR filter Coefficient #8 <Table1>. Decimal 6 bit: $-16 \leq \text{coeff} \leq 15 + 63/64$

11.7.21 AF_COEFF19

AF Engine IIR Filter Table #1, Coefficient #9

AFCOEFF19	0003:0BA8			offset: 0x28		default: 0x0000										
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	COEFF19[1]	COEFF19[9]	COEFF19[8]	COEFF19[7]	COEFF19[6]	COEFF19[5]	COEFF19[4]	COEFF19[3]	COEFF19[2]	COEFF19[1]	COEFF19[0]
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:0	COEFF19	0	R/W	[AF Engine] IIR filter Coefficient #9 <Table1>. Decimal 6 bit: $-16 \leq \text{coeff} \leq 15 + 63/64$

11.7.22 AFCEFF110

AF Engine IIR Filter Table #1, Coefficient #10

AFCEFF110		0003:0BAA			offset: 0x2A		default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	COEFF110[10]	COEFF110[9]	COEFF110[8]	COEFF110[7]	COEFF110[6]	COEFF110[5]	COEFF110[4]	COEFF110[3]	COEFF110[2]	COEFF110[1]	COEFF110[0]
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:0	COEFF110	0	R/W	[AF Engine] IIR filter Coefficient #10 <Table1>. Decimal 6 bit: $-16 \leq \text{coeff} \leq 15 + 63/64$

11.7.23 AFCEFF20

AF Engine IIR Filter Table #2, Coefficient #0

AFCEFF20		0003:0BAC			offset: 0x2C												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	RSV	RSV	RSV	COEFF20[10]	COEFF20[9]	COEFF20[8]	COEFF20[7]	COEFF20[6]	COEFF20[5]	COEFF20[4]	COEFF20[3]	COEFF20[2]	COEFF20[1]	COEFF20[0]		
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W		
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0		

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:0	COEFF20	0	R/W	[AF Engine] IIR filter Coefficient #0 <Table2>. Decimal 6 bit: $-16 \leq \text{coeff} \leq 15 + 63/64$

11.7.24 AFCEFF21

AF Engine IIR Filter Table #2, Coefficient #1

AFCOEFF21		0003:0BAE			offset: 0x2E		default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	COEFF21[10]	COEFF21[9]	COEFF21[8]	COEFF21[7]	COEFF21[6]	COEFF21[5]	COEFF21[4]	COEFF21[3]	COEFF21[2]	COEFF21[1]	COEFF21[0]
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:0	COEFF21	0	R/W	[AF Engine] IIR filter Coefficient #1 <Table2>. Decimal 6 bit: $-16 \leq \text{coeff} \leq 15 + 63/64$

11.7.25 AFCEFF22

AF Engine IIR Filter Table #2, Coefficient #2

AFCEFF22		0003:0BB0			offset: 0x30		default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	COEFF22[10]	COEFF22[9]	COEFF22[8]	COEFF22[7]	COEFF22[6]	COEFF22[5]	COEFF22[4]	COEFF22[3]	COEFF22[2]	COEFF22[1]	COEFF22[0]
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:0	COEFF22	0	R/W	[AF Engine] IIR filter Coefficient #2 <Table2>. Decimal 6 bit: $-16 \leq \text{coeff} \leq 15 + 63/64$

11.7.26 AFCEFF23

AF Engine IIR Filter Table #2, Coefficient #3

AFCEFF23		0003:0BB2			offset: 0x32		default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	COEFF23[10]	COEFF23[9]	COEFF23[8]	COEFF23[7]	COEFF23[6]	COEFF23[5]	COEFF23[4]	COEFF23[3]	COEFF23[2]	COEFF23[1]	COEFF23[0]
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:0	COEFF23	0	R/W	[AF Engine] IIR filter Coefficient #3 <Table2>. Decimal 6 bit: $-16 \leq \text{coeff} \leq 15 + 63/64$

11.7.27 AFCEFF24

AF Engine IIR Filter Table #2, Coefficient #4

AFCOEFF24	0003:0BB4			offset: 0x34												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	COEFF24[1]	COEFF24[9]	COEFF24[8]	COEFF24[7]	COEFF24[6]	COEFF24[5]	COEFF24[4]	COEFF24[3]	COEFF24[2]	COEFF24[1]	COEFF24[0]	
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:0	COEFF24	0	R/W	[AF Engine] IIR filter Coefficient #4 <Table2>. Decimal 6 bit: $-16 \leq \text{coeff} \leq 15 + 63/64$

11.7.28 AFCEFF25

AF Engine IIR Filter Table #2, Coefficient #5

AFCEFF25		0003:0BB6			offset: 0x36		default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	COEFF25[10]	COEFF25[9]	COEFF25[8]	COEFF25[7]	COEFF25[6]	COEFF25[5]	COEFF25[4]	COEFF25[3]	COEFF25[2]	COEFF25[1]	COEFF25[0]
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:0	COEFF25	0	R/W	[AF Engine] IIR filter Coefficient #5 <Table2>. Decimal 6 bit: $-16 \leq \text{coeff} \leq 15+63/64$

11.7.29 AFCEFF26

AF Engine IIR Filter Table #2, Coefficient #6

AFCEFF26		0003:0BB8			offset: 0x38		default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	COEFF26[10]	COEFF26[9]	COEFF26[8]	COEFF26[7]	COEFF26[6]	COEFF26[5]	COEFF26[4]	COEFF26[3]	COEFF26[2]	COEFF26[1]	COEFF26[0]
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:0	COEFF26	0	R/W	[AF Engine] IIR filter Coefficient #6 <Table2>. Decimal 6 bit: $-16 \leq \text{coeff} \leq 15+63/64$

11.7.30 AFCEFF27

AF Engine IIR Filter Table #2, Coefficient #7

AFCEFF27		0003:0BBA			offset: 0x3A		default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	COEFF27[10]	COEFF27[9]	COEFF27[8]	COEFF27[7]	COEFF27[6]	COEFF27[5]	COEFF27[4]	COEFF27[3]	COEFF27[2]	COEFF27[1]	COEFF27[0]
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:0	COEFF27	0	R/W	[AF Engine] IIR filter Coefficient #7 <Table2>. Decimal 6 bit: $-16 \leq \text{coeff} \leq 15+63/64$

11.7.31 AFCEFF28

AF Engine IIR Filter Table #2, Coefficient #8

AFCEFF28		0003:0BBC			offset: 0x3C												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	RSV	RSV	RSV	COEFF28[1]	COEFF28[9]	COEFF28[8]	COEFF28[7]	COEFF28[6]	COEFF28[5]	COEFF28[4]	COEFF28[3]	COEFF28[2]	COEFF28[1]	COEFF28[0]		
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W		
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0		

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:0	COEFF28	0	R/W	[AF Engine] IIR filter Coefficient #8 <Table2>. Decimal 6 bit: $-16 \leq \text{coeff} \leq 15 + 63/64$

11.7.32 AFCEFF29

AF Engine IIR Filter Table #2, Coefficient #9

AFCEFF29		0003:0BBE			offset: 0x3E												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	RSV	RSV	RSV	COEFF29[1]	COEFF29[9]	COEFF29[8]	COEFF29[7]	COEFF29[6]	COEFF29[5]	COEFF29[4]	COEFF29[3]	COEFF29[2]	COEFF29[1]	COEFF29[0]		
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W		
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0		

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:0	COEFF29	0	R/W	[AF Engine] IIR filter Coefficient #9 <Table2>. Decimal 6 bit: $-16 \leq \text{coeff} \leq 15 + 63/64$

11.7.33 AFCEFF210

AF Engine IIR Filter Table #2, Coefficient #10

AFCEFF210		0003:0BC0			offset: 0x40												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	RSV	RSV	RSV	COEFF210[1]	COEFF210[9]	COEFF210[8]	COEFF210[7]	COEFF210[6]	COEFF210[5]	COEFF210[4]	COEFF210[3]	COEFF210[2]	COEFF210[1]	COEFF210[0]		
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W		
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0		

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:0	COEFF210	0	R/W	[AF Engine] IIR filter Coefficient #10 <Table2>. Decimal 6 bit: $-16 \leq \text{coeff} \leq 15 + 63/64$

11.7.34 AEWCTRL

AE/AWB Engine Control Register

AEWCTRL	0003:0BC2			offset: 0x42		default: 0x0000										
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	AVE2LMT[7]	AVE2LMT[6]	AVE2LMT[5]	AVE2LMT[4]	AVE2LMT[3]	AVE2LMT[2]	AVE2LMT[1]	AVE2LMT[0]
R/W	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	-	-	-	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:8	RSV			Reserved
7:0	AVE2LMT	0	R/W	[AE/AWB Engine] Saturation limit. Note: If any of the pixels in a 2x2 block are equal to or greater than AVE2LMT, the block is treated as a saturated data, and not counted into CNT2. If a sub-sampled 2x2 block is saturating, the pixels equal to or greater than AVE2LMT are clipped to AVE2LMT, and accumulated to AVE2.

11.7.35 AEWWIN1

AE/AWB Engine Window Register #1

AEWWIN1	0003:0BC4			offset: 0x44			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	WINV[5]	WINV[4]	WINV[3]	WINV[2]	WINV[1]	WINV[0]	RSV	WINH[6]	WINH[5]	WINH[4]	WINH[3]	WINH[2]	WINH[1]	WINH[0]
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	0	0	0	0	0	0	-	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
13:8	WINV	0	R/W	[AE/AWB Engine] Window Height (even number). Window Height is ((WINV +1) x2) ranges: 4-128
7	RSV			Reserved
6:0	WINH	0	R/W	[AE/AWB Engine] Window Width (even number). Window Width is ((WINH +1) x2) and ranges: 2-256

11.7.36 AEWWIN2

AE/AWB Engine Window Register #2

AEWWIN2		0003:0BC6			offset: 0x46												default: 0x0000		
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	RSV	RSV	RSV	WINVC[2]	WINVC[1]	WINVC[0]	RSV	RSV	RSV	RSV	WINHC[3]	WINHC[2]	WINHC[1]	WINHC[0]			
R/W	-	-	-	-	-	R/W	R/W	R/W	-	-	-	-	R/W	R/W	R/W	R/W			
Default	-	-	-	-	-	0	0	0	-	-	-	-	0	0	0	0			

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:8	WINVC	0	R/W	[AE/AWB Engine] Window Count for V direction. Window Count is (WINVC +1) and ranges: 1-8.
7:4	RSV			Reserved
3:0	WINHC	0	R/W	[AE/AWB Engine] Window Count for H direction. Window Count is (WINHC +1) and ranges: 1-12. Setting a value greater than 11 to WINHC is not allowed.

11.7.37 AEWWIN3

AE/AWB Engine Window Register #3

AEWWIN3		0003:0BC8			offset: 0x48												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	RSV	RSV	RSV	WINSH[10]	WINSH[9]	WINSH[8]	WINSH[7]	WINSH[6]	WINSH[5]	WINSH[4]	WINSH[3]	WINSH[2]	WINSH[1]	WINSH[0]		
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W		
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0		

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:0	WINSH	0	R/W	[AE/AWB Engine] Window start position (H direction, even number). Window start position is (WINSH x2) and ranges: 0-4,094

AE/AWB Engine Window Register #4

[illegible]

11.7.39 AEWWIN5

AE/AWB Engine Window Register #5

AEWWIN5	0003:0BCC			offset: 0x4C		default: 0x0000										
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	AEWINCV[3]	AEWINCV[2]	AEWINCV[1]	AEWINCV[0]	RSV	RSV	RSV	RSV	AEWINCH[3]	AEWINCH[2]	AEWINCH[1]	AEWINCH[0]
R/W	-	-	-	-	R/W	R/W	R/W	R/W	-	-	-	-	R/W	R/W	R/W	R/W
Default	-	-	-	-	0	0	0	0	-	-	-	-	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:12	RSV			Reserved
11:8	AEWINCV	0	R/W	[AE/AWB Engine] Sampling point increments in a window (V direction, even number). Increments are ((AEWINCV +1) x2) and ranges: 2-32
7:4	RSV			Reserved
3:0	AEWINCH	0	R/W	[AE/AWB Engine] Sampling point increments in a window (H direction, even number). Increments are ((AEWINCH +1) x2) and ranges: 2-32

11.7.40 AEWSRA1

AE/AWB Engine SDRAM Address (high)

AEWSRA1	0003:0BCE			offset: 0x4E												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	ADRH[6]	ADRH[5]	ADRH[4]	ADRH[3]	ADRH[2]	ADRH[1]	ADRH[0]	
R/W	-	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	-	-	-	-	-	-	-	-	-	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:7	RSV			Reserved
6:0	ADRH	0	R/W	[AE/AWB Engine] Upper 7 bits of SDRAM destination address to store the results of AE/AWB Engine. There are two continuous buffers, and the results of the two successive frames are stored to the different buffers. Buffer size depends on WINHC and WINVC. AEWSRA is the start address of the first buffer, and the address is automatically incremented until data transfer for the two frame ends (address is 32 Byte unit) specified as offset from start of SDRAM.

11.7.41 AEWSRA2

AE/AWB Engine SDRAM Address (low)

AEWSDRA2		0003:0BD0			offset: 0x50												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	ADRL[15]	ADRL[14]	ADRL[13]	ADRL[12]	ADRL[11]	ADRL[10]	ADRL[9]	ADRL[8]	ADRL[7]	ADRL[6]	ADRL[5]	ADRL[4]	ADRL[3]	ADRL[2]	ADRL[1]	ADRL[0]		
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W		
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		

Bit	Name	Reset Value	R/W	Function
15:0	ADRL	0	R/W	[AE/AWB Engine] Lower 16 bits of SDRAM destination address to store the results of AE/AWB Engine. There are two continuous buffers, and the results of the two successive frames are stored to the different buffers. Buffer size depends on WINHC and WINVC. AEWSRA is the start address of the first buffer, and the address is automatically incremented until data transfer for the two frame ends (address is 32 Byte unit) specified as offset from start of SDRAM.

11.7.42 AEWSDRFLG

AE/AWB Engine SDRAM Buffer Flag

AEWSDRFLG	0003:0BD2			offset: 0x52			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	AEWERR	AEWERRFR1	AEWERRFR0	RSV	RSV	RSV	AEWSDRFLG
R/W	-	-	-	-	-	-	-	-	-	R/W	R	R	-	-	-	R
Default	-	-	-	-	-	-	-	-	-	0	0	0	-	-	-	0

Bit	Name	Reset Value	R/W	Function
15:7	RSV			Reserved
6	AEWERR	0	R/W	SDRAM access fail flag, Clear by ARM write
5	AEWERRFR1	0	R	SDRAM access fail flag on previous buffer flag 1
4	AEWERRFR0	0	R	SDRAM access fail flag on previous buffer flag 0
3:1	RSV			Reserved
0	AEWSDRFLG	0	R	[AE/AWB Engine] SDRAM buffer flag to indicate the buffer at which the results of the last frame are stored. » 0: Results stored at first buffer starting AEWSDR « » 1: Results are stored at the second buffer «

12 OSD - On-Screen Display

12.1 Introduction

The OSD graphic accelerator is responsible for managing OSD data from different OSD windows and blending it with the video. It reads OSD data from SDRAM, and outputs it to the on-chip Video Encoder. After being configured and activated by the ARM, the OSD reads OSD data and mixes it with the video output. ARM is responsible for configuration and control of OSD operations and writing of the OSD data to the SDRAM.

12.2 Features

- o Two Video Windows
- o One bitmap Window + Attribute window, or two bitmap windows
- o One of the OSD window can be configured as RGB16 window
- o One rectangular Cursor Window
- o Color Palette ROM/RAM selection
- o Programmable Color Palette
- o All Windows can be blended and displayed
- o Video Window Zoom (x2,x4,H/V)
- o Video Window Field/Frame selection
- o Bitmap Window Zoom (x2,x4,H/V)
- o Bitmap Window Field/Frame selection
- o Resize of VGA to NTSC/PAL
- o Back Ground color selection
- o Selectable bitmap width of 1,2,4,8-bit
- o Blending of Video and OSD data (8-step)
- o 16-LUT registers for OSD Window
- o Cursor Display with Width and color selectable
- o Ping Pong Buffer Control with H/W Signal
- o Polarity Selection of Field Signal
- o Configurable CbCr order

12.3 OSD Block Diagram

The Figure 64 shows the OSD block diagram.

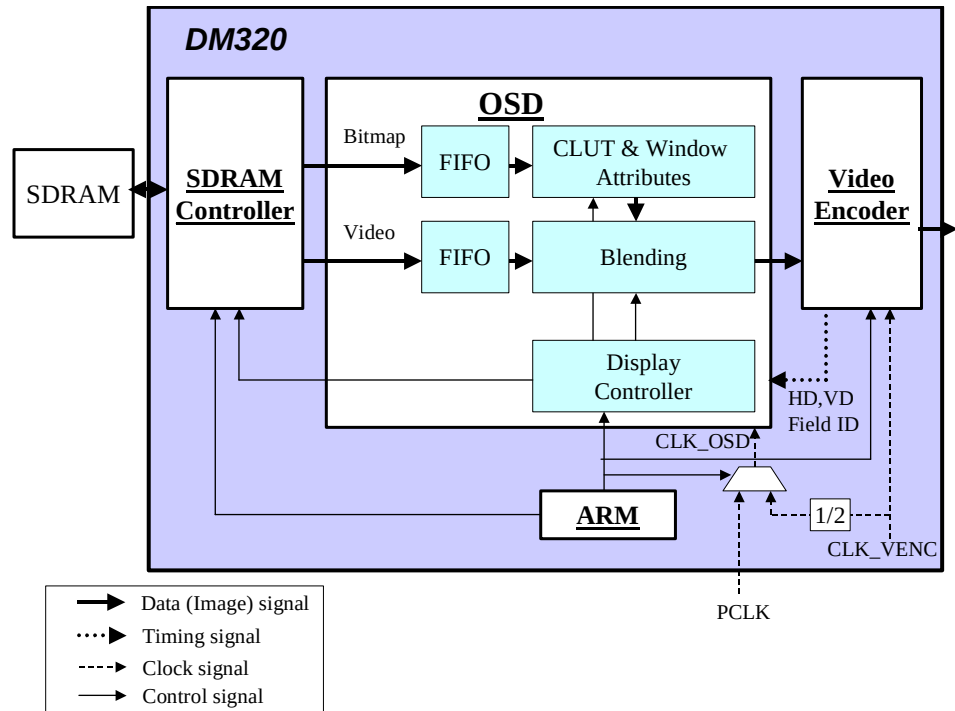


Figure 64: OSD Block Diagram

12.4 Bitmap and Video Windows

OSD supports the following windows:

- OSD Video Window 0 (Main Video Window)– displays YCbCr 4:2:2 video data
- OSD Video Window 1 – displays YCbCr 4:2:2 video data
- OSD Bitmap Window 0 – displays 1,2,4,8-bit bitmap data using color lookup table or RGB16 data
- OSD Bitmap Window 1 – displays 1,2,4,8-bit bitmap data using color lookup table / RGB16 data or OSD Attribute Window – for pixel level blending of OSD Bitmap Window 0 and video window 0,1
- Rectangular Hardware cursor

Register	Description
<i>VIDWINMD</i>	Controls the display, zoom and on/off control of Video windows
<i>OSDWIN0MD</i>	Controls the display, zoom blending, and on/off control of OSD window 0
<i>OSDWIN1MD</i>	Controls the display, zoom blending, and on/off control of OSD window 1
<i>OSDATRMD</i>	Controls the blinking, display, zoom, on/off control of Attribute window
<i>RECTCUR</i>	Controls the size and on/off control of rectangular cursor window

If overlapping occurs between different windows, the color of each displayed pixel corresponds to the color of the window with the highest priority. The priority order is fixed. When OSD bitmap window 1 is configured to be in attribute mode, its priority does not change. Figure 65 illustrates the priority of different windows. Video Window 0 and 1 should not overlap with the OSD window if the window is set to RGB 16 mode.

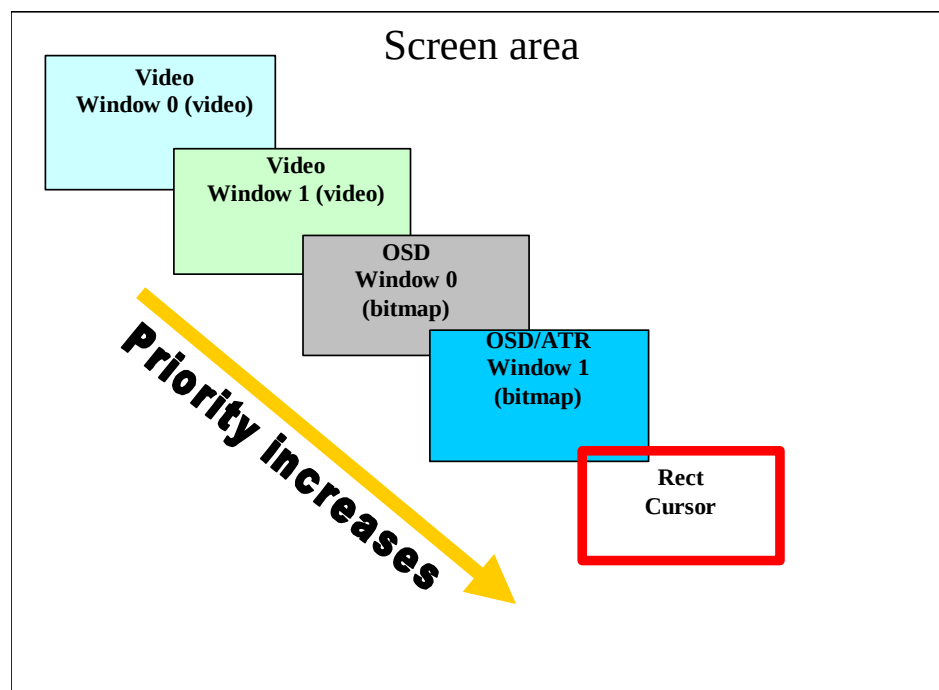


Figure 65: OSD Window Priority

12.5 OSD Window Data Storage Format

In the bitmap window, each pixel can be 1, 2, 4, or 8 bits wide. In the YCbCr 4:2:2 window, each Y, Cb, Cr component takes 8-bit, and the components are

arranged according to 4:2:2 interleaved format (Cb-Y-Cr-Y...). In case RGB graphics data needs to be displayed using OSD, the application should perform software conversion to Y,Cr,Cb before storing it. The OSD data is always packed into 32-bit words and left justified. Starting from the upper left corner of the OSD window, all data will be packed into adjacent 32-bit words. Figure 66 shows data format of video, bitmap data in SDRAM.

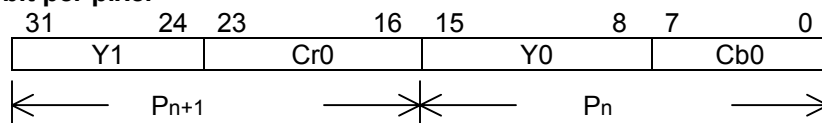
(a) Left top

Pixel arrangement in the display

P0	P1	P2	P3	P4	P5	P6	P7...
----	----	----	----	----	----	----	-------

(b) Video Data (Video Window 0, 1)

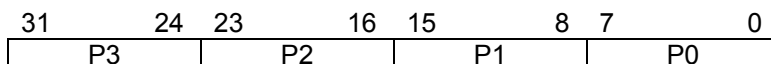
16-bit per pixel

**SDRAM format**

Address	31	16	0
N	P1	P0	
N+1	P3	P2	
N+2	P5	P4	
...	...	...	

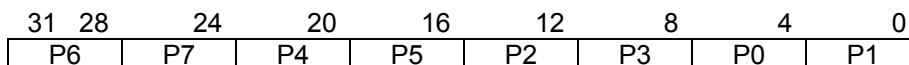
(c) Bitmap Data (OSD Bitmap Window 0, 1)

8-bit per pixel

**SDRAM format**

Address	31	24	16	8	0
N	P3	P2	P1	P0	
N+1	P7	P6	P5	P4	
N+2	P11	P10	P9	P8	
...	...	...	...	...	

4-bit per pixel

**SDRAM format**

Address	31	28	24	20	16	12	8	4	0
N	P6	P7	P4	P5	P2	P3	P0	P1	
N+1	P14	P15	P12	P13	P10	P11	P8	P9	
N+2	P22	P23	P20	P21	P18	P19	P16	P17	
N+3									

2-bit per pixel

31	30	28	26	24	22	20	18	16	14	12	10	8	6	4	2	0
P12	P13	P14	P15	P8	P9	P10	P11	P4	P5	P6	P7	P0	P1	P2	P3	

SDRAM format

[illegible]

1-bit per pixel

[illegible]

SDRAM format

Address	31												0							
N	P24	P25	P26~P3										P4	P5	P6	P7				
N+1	P56	P57	P58~P35										P36	P37	P38	P39				
N+2	P88	P89	P90~P67										P68	P69	P70	P71				
M	M1	M2	M3	M4	M5	M6	M7	M8	M9	M10	M11	M12	M13	M14	M15					

(d) RGB 16bit data for OSD window



SDRAM Format

Address	31	26	20	15	10	4
N	P1 (Red)	P1 (Green)	P1 (Blue)	P0 (Red)	P0 (Green)	P0 (Blue)
N+1	P3 (Red)	P3 (Green)	P3 (Blue)	P2 (Red)	P2 (Green)	P2 (Blue)
N+2	P5 (Red)	P5 (Green)	P5 (Blue)	P4 (Red)	P4 (Green)	P4 (Blue)
...	...	...	...	...	...	...

Figure 66 OSD SDRAM Data Format

12.6 OSD Configuration and Control

Many of the OSD registers contain bits that are latched by the VD signal, which is the vertical sync pulse generated by the DM320's Video Encoder module. Data written to a latched bit does not take affect until the VD pulse is received. This allows registers that control the OSD, such as the SDRAM data address, display window size, display zoom configuration, etc. to be changed in between successive VD pulses without corrupting the current display. In the OSD register descriptions, bit fields marked with (*) are latched by the VD signal.

12.6.1 SDRAM location and offset

The location of data stored in SDRAM is defined by several memory-mapped registers (see *Table 36*). The SDRAM addresses are specified in units of 32 bytes, since data is transferred from SDRAM to the OSD hardware in bursts of 32 bytes. SDRAM address is specified as offset from start of SDRAM region. SDRAM address must be 32byte aligned.

For example, let the video window 0 video data be located at absolute SDRAM address 0x97AB00. The address register would be set as shown in the code below.

```

Uint32 window_addr;

window_addr = (0x97AB00 - 0x900000)/32;

OSD_FSET( VIDWINADH, V0AH, window_addr >> 16);
OSD_RSET( VIDWIN0ADL, window_addr & 0xFFFF);

```

SDRAM address location register	Window
VIDWINADH VIDWIN0ADL	Video Window 0 low and high address
VIDWINADH VIDWIN1ADL	Video Window 1 low and high address
OSDWINADH OSDWIN0ADL	OSD Bitmap Window 0 low and high address
OSDWINADH OSDWIN1ADL	OSD Bitmap Window 1 / Attribute Window low and high address

Table 36: SDRAM address location registers

The offset registers allow a cropped segment of an image to be easily displayed. An offset register specifies the distance, in SDRAM, from the start of a displayed line to the start of the next displayed line. The offset is in units of 32 bytes. Thus width of data stored in SDRAM must be multiple of 32bytes. In case, the width of data in SDRAM is not multiple of 32bytes, then padding must be done to make it multiple of 32bytes.

Table 37 shows the registers for controlling the SDRAM offset for video and bitmap windows.

For example, for OSD bitmap window 0, with 4-bit pixel width, If the width of the bitmap is “size” pixels. Since for 4-bit bitmap, two pixels is one byte. Hence width of data stored in SDRAM would be “size / 2” bytes and offset register would be set as shown in the code below.

```

Uint32 addr = size / 2;

OSD_RSET(OSDWIN0OFST, addr/32 );

```

SDRAM offset register	Window
VIDWIN0OFST	Video Window 0 SDRAM offset register
VIDWIN1OFST	Video Window 1 SDRAM offset register
OSDWIN0OFST	OSD Bitmap Window 0 SDRAM offset register
OSDWIN1OFST	OSD Bitmap Window 1 / Attribute Window SDRAM offset register

Table 37: SDRAM offset registers

Note: The size restriction is only applicable for bitmap windows.

12.6.2 Windows positioning

The windows use a common reference pixel (base pixel). The position of this pixel is determined from the beginning of the Video Encoder HD and the beginning of the Video Encoder VD signal.

For each window (video or bitmap) and for the cursor, the location of the upper left corner can be specified. The display size of the window needs to be specified as well. The relationship of the display positions of each window is shown in Figure 67.

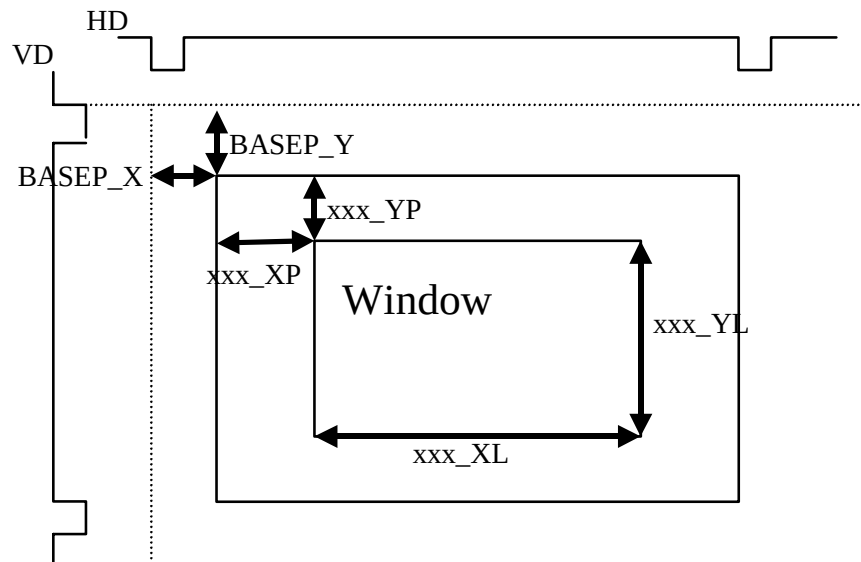


Figure 67: Window Setup

Window start position is specified with respect to the BASE_X, BASE_Y position (see BASEPX and BASEPY registers). Window start position in X-direction and window width is specified in units of pixels. Window start position in Y-direction and window height is specified in units of field lines. The Table 38 shows the register used for window positioning.

Window positioning registers	Window
VIDWIN0XP VIDWIN0YP VIDWIN0XL VIDWIN0YL	Video Window 0 start position and size registers
VIDWIN1XP VIDWIN1YP VIDWIN1XL VIDWIN1YL	Video Window 1 start position and size registers
OSDWIN0XP OSDWIN0YP OSDWIN0XL OSDWIN0YL	OSD Bitmap Window 0 start position and size registers
OSDWIN1XP OSDWIN1YP OSDWIN1XL OSDWIN1YL	OSD Bitmap Window 1 / Attribute Window start position and size registers
CURXP CURYP CURXL CURYL	Hardware cursor start position and size registers

Table 38: Window positioning registers

12.6.3 Window Mode

12.6.3.1 Field / Frame Mode

Each video and bitmap window has two display modes: field mode and frame mode. The video encoder reads data from the video, bitmap windows with the help of two fields, even field and odd field. When the window mode is field mode, each line is duplicated in both even field as well as odd field. When window mode is frame mode, the even field reads every even line and the odd field reads every odd line. This relationship is shown in the Figure 68 and Figure 69.

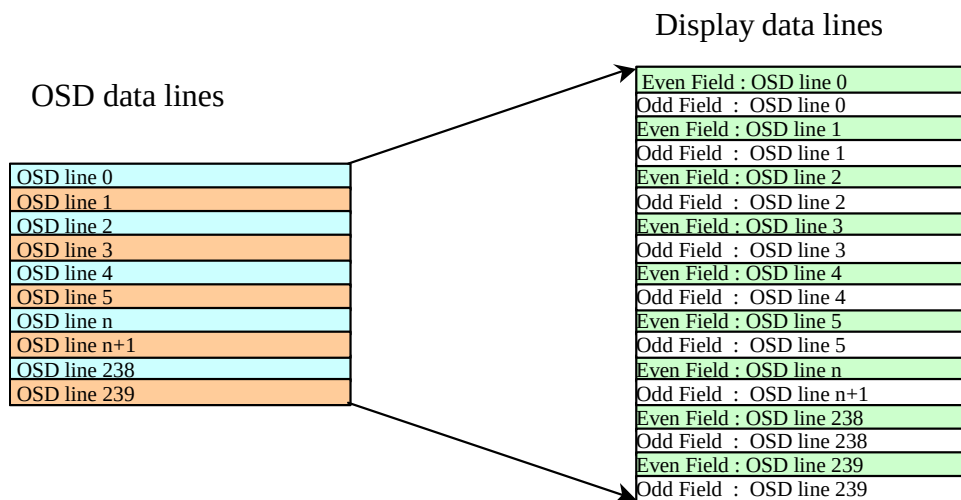


Figure 68: Window Field Mode

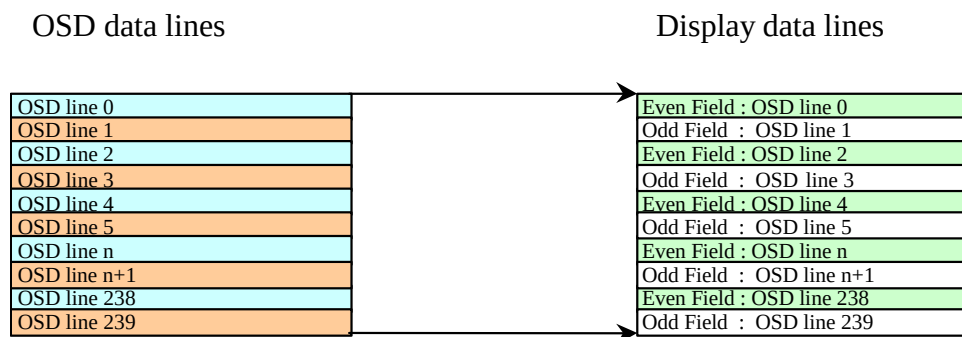


Figure 69: Window Frame Mode

12.6.3.2 Window Zooming

The video windows and OSD Bitmap windows can be zoomed along their horizontal and vertical directions by a factor of 2 or 4. *Figure 70* shows the zoom process and the parameters that must be setup to execute a zoom. Below are the steps involved in this procedure,

- o Set the starting SDRAM address of the area desired to be magnified, offset, zoom factor and the display window size. The OSD will take data, starting from the start address, to generate a magnified image that fits into the display window.
- o The offset is always the number of bursts in one horizontal line of SDRAM pixel data.
- o Set the display position to the desired position of the window. Set the display height and display width to the desired magnified height and width.
- o When zoom is enabled SDRAM data starting from the SDRAM start position will be magnified to fit into the display area specified by the display height and display width. For example, if the horizontal and vertical directions are set to 2x zoom and the display width and height are set 640x480, then a 320x240 block of data starting from the SDRAM start position will be magnified to 640x480 in the display window.

All of the registers used in the zoom process, are latched by the VD signal. So they can be safely updated any time, without any distortion.

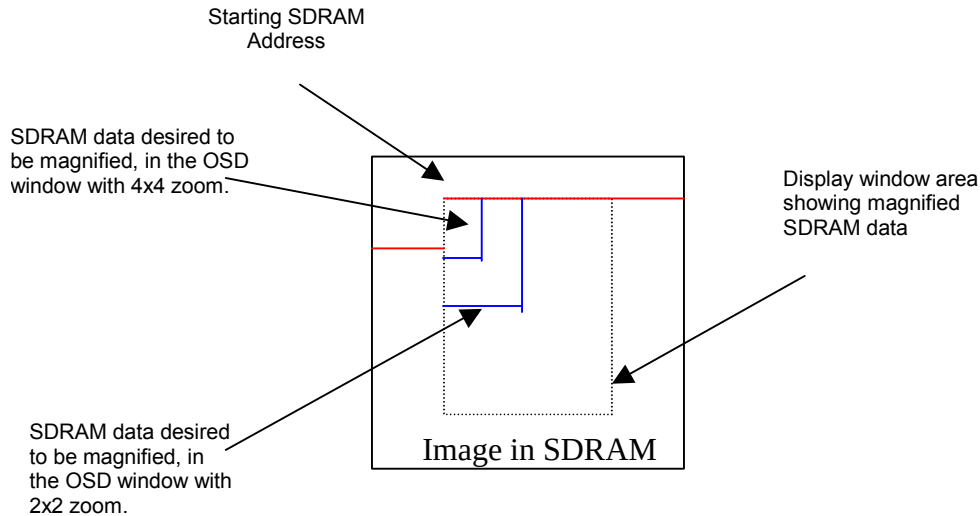


Figure 70: OSD Zoom Process

12.7 Bitmap Window

Bitmap window allows the user to display graphics and icons on the display unit. Bitmap Window 0 can only be used as a bitmap window. Bitmap Window 1 can be used as either bitmap window or attribute window. These windows can also display RGB16 data. The bitmap window uses a color lookup table (CLUT) in order to display a color for a given bitmap pixel value. A total of 256 colors are available. The maximum width of bitmap pixel is thus 8-bit. However 1,2,4-bit bitmaps are also supported. See section 12.5 for SDRAM storage format of bitmap data.

12.7.1 RGB16 data

In addition to bitmaps OSD bitmap windows can display RGB16 data also. Only one of the OSD bitmap windows can be set to display RGB16 data. Video Window 0 and 1 should not overlap with the OSD window if the window is set to RGB 16 mode.

12.7.2 Color Lookup Table

The user has the option to select either the ROM CLUT or can configure a RAM color look table.

12.7.2.1 ROM color look up table

The ROM CLUT is embedded into the internal ROM in OSD.

The ROM color look-up table values are given in Figure 71.

The color look-up chart is illustrated in Figure 72.

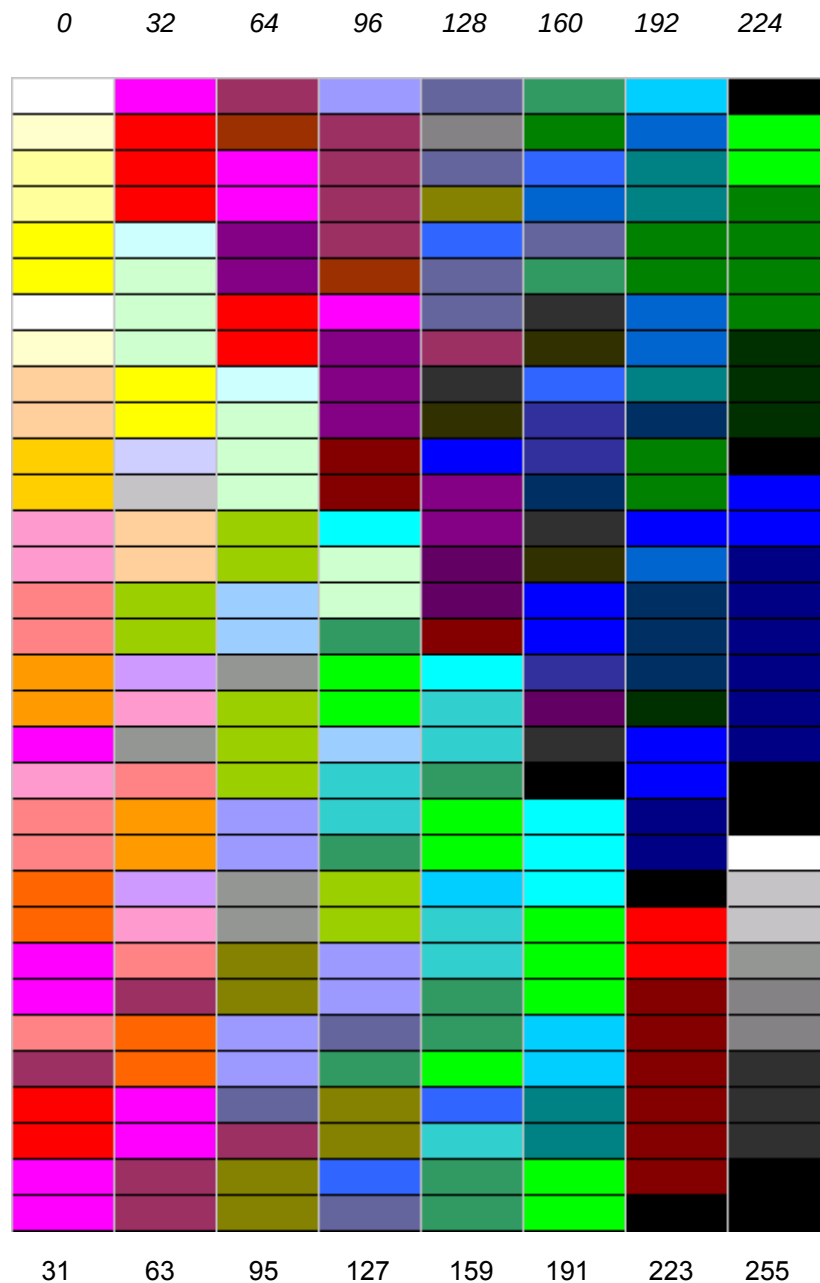


Figure 72: ROM CLUT Color Chart

12.7.2.2 RAM Color Lookup Table

A RAM CLUT can be used instead of the default ROM CLUT, which has to be written by the user into the internal OSD RAM memory.

To setup the OSD RAM CLUT, following steps are required:

- o Wait for the CPBSY bit of register *MISCCTL* to be zero
- o Write the address and the value of Cr into the CLUTRAMYCB register. The address is the offset address in the OSD RAM for 1 set of Y, Cb and Cr values.
- o Write Y and Cb value into the *MISCCTL* register
- o Repeat the previous steps until the RAM table is loaded completely.

Note: After a device reset, the color values of the RAM CLUT are undetermined.

Sample code to write to RAM CLUT is given below.

```
extern Uint8 lutY[255];           //LUT entries for Y (0-255)
extern Uint8 lutCb[255];         //LUT entries for Cb (0-255)
extern Uint8 lutCr[255];         //LUT entries for Cr (0-255)
Uint8 i;
for(i=0; i< 255; i++) {
    while(OSD_FGET( MISCCTL, CPBSY) );
    OSD_RSET(CLUTRAMCR, ((Uint16)Cr[i] << 8) | i;
    // Write the LUT address and Cr value
    OSD_RSET(CLUTRAMYCB, ((Uint16)Y[i] << 8) | (Uint16)Cb[i];
    // Write LUT values from Y, Cb arrays
}
```

12.7.2.3 Selecting RAM/ROM CLUT

Selection between RAM and ROM CLUT can be done separately for OSD Bitmap Window 0, OSD Bitmap Window 1. For OSD Bitmap Window 0 configure CLUTS0 bit in register *OSDWIN0MD*. For OSD Bitmap Window 1 configure CLUTS1 bit in register *OSDWIN1MD*.

12.7.2.4 1,2,4-bit Bitmap and CLUT

When the bitmap size is 1,2 or 4-bit, the registers *W0BMP01* to *W0BMPEF* or *W1BMP01* to *W1BMPEF* can be used to map any of the 256 colors to a bitmap value. For example, for 1-bit bitmap data, any of the 256 colors in the CLUT can be mapped to bit value "0" and any of the 256 colors can be mapped to bit value "1". This mapping can be specified separately for each OSD Bitmap window. The *Table: 39* shows the register, which can be used to select the color for a bitmap value.

Register : Field (x = 0,1 for OSD Bitmap Window 0,1	<u>4-bit bitmap</u> Color corresponding	<u>2-bit bitmap</u> Color corresponding	<u>1-bit bitmap</u> Color corresponding to bitmap value
---	---	---	---

respectively)	to bitmap value	to bitmap value	
WxBMP01 : PAL00	0	0	0
WxBMP01 : PAL01	1	-	-
WxBMP23 : PAL02	2	-	-
WxBMP23 : PAL03	3	-	-
WxBMP45 : PAL04	4	-	-
WxBMP45 : PAL05	5	1	-
WxBMP67 : PAL06	6	-	-
WxBMP67 : PAL07	7	-	-
WxBMP89 : PAL08	8	-	-
WxBMP89 : PAL09	9	-	-
WxBMPAB : PAL10	10	2	-
WxBMPAB : PAL11	11	-	-
WxBMPCD : PAL12	12	-	-
WxBMPCD : PAL13	13	-	-
WxBMPEF : PAL14	14	-	-
WxBMPEF : PAL15	15	3	1

Table: 39 1,2,4-bit Bitmap Color Selection

12.7.3 Blending and Transparency

OSD supports transparency mode with bitmap windows. If transparency is enabled, then any pixel on the bitmap display that has a value of 0 will allow video to be displayed. In this case, blending factor would be applied before displaying the video. Essentially, 0 valued pixels are considered transparent, and the background color (after blending) will show through the bitmap. Background color selection and Cb/Cr format selection can be done in the *OSDMODE* register.

OSD also supports color blending at the pixel level. This feature is available for the bitmap windows only. If the window color blending is enabled, the amount of blending of each pixel is determined by the blending factor. The different blending factors are shown in Table 40.

Blending factor and transparency is configured using *OSDWIN0MD* and *OSDWIN1MD* registers for OSD Bitmap window 0 and 1 respectively.

Transparency	Blending Factor	OSD Window contribution	Video contribution
OFF	0	0	1
	1	1/8	7/8
	2	2/8	6/8
	3	3/8	5/8
	4	4/8	4/8
	5	5/8	3/8
	6	6/8	2/8
	7	1	0
ON	If pixel value = 0		
	0	0	1
	1	1/8	7/8
	2	2/8	6/8
	3	3/8	5/8
	4	4/8	4/8
	5	5/8	3/8
	6	6/8	2/8
	7	1	0
	If pixel value != 0		
	X	1	0

Table 40: Bending and Transparency in OSD

12.7.4 Attribute Window

OSD bitmap window 1 can be configured as an attribute window. The attribute window allows blending and blinking of individual pixels in the display windows. In particular the attribute window allows to:

- Blend pixels from OSD Window 0 with Video Window 0 or Video Window 1
- Blink pixels in OSD Window 0

The SDRAM start address, position, and size of the attribute window needs to be configured. However, the data in SDRAM is interpreted as shown in Figure 73. Bit3 specifies whether or not to blink the corresponding pixel and Bit2-Bit0 specifies the blending factor to be applied to the corresponding pixel. The blending factors are defined in Table 40.

Bit 3	Bit 2	Bit 1	Bit 0
0 - No blink	Blend Factor		
1 - Blink			

Figure 73: Attribute Data Format

As shown in Figure 74 the attribute window applies blending and blinking to OSD window 0 depending on the position of the attribute window.

Attribute window is configured using register *OSDATRMD*

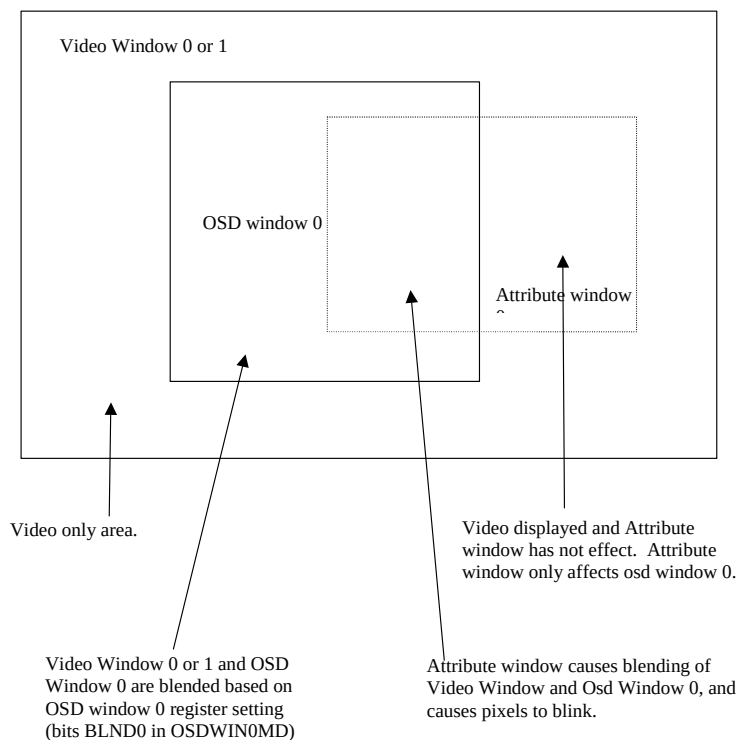


Figure 74: The Attribute Window's Relationship to the Other Windows

12.8 Rectangular Hardware Cursor

The rectangular hardware cursor always appears on top of other OSD Windows. The cursor's size, color, horizontal and vertical thickness can be specified. This cursor can also be configured to use the ROM or RAM CLUT.

Figure 75 shows an example usage of the rectangle cursor. Yellow Line shows the rectangle cursor and ICON1-4 are displayed by using OSD window 0 or 1.

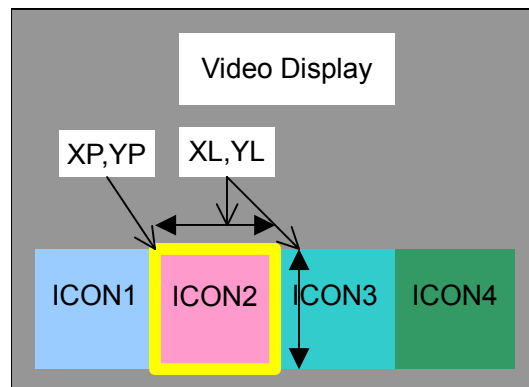


Figure 75: Rectangular Cursor

Register	Description
<i>RECTCUR</i>	Controls the size and enable / disable rectangular cursor.
<i>CURXP</i>	Sets the horizontal display start position
<i>CURYP</i>	Sets the vertical display start position
<i>CURXL</i>	Sets the horizontal display width in pixels.
<i>CURYL</i>	Sets the vertical display height in pixels/lines

12.9 Window Expansion (VGA to NTSC/PAL Conversion)

OSD supports conversion from VGA size image to NTSC/PAL image during display of video windows, bitmap windows. *Table 41* shows the expansion ratios supported.

	Horizontal	Vertical
Video Window	9/8	6/5
Bitmap Window	--	6/5

Table 41: Window Expansion

For Video Windows horizontal 9/8 expansion is enabled by VHRSZ bit and vertical 6/5 expansion is enabled by VVRSZ bit of *OSDMODE* register. When either of them is enabled, the Expansion Filter can be applied by EF bit of the same register.

For Bitmap Window, horizontal 9/8 expansion is enabled by OHRSZ bit and vertical 6/5 expansion is enabled by OVRSZ bit of *OSDMODE* register.

12.10 OSD Ping Pong Buffer

The main video window (video window 0) supports ping-pong buffers. Initially the SDRAM source address for OSD Video window 0 can be set to the SDRAM address specified in *VIDWIN0ADH* and *VIDWIN0ADL* registers, and when the ping pong buffer toggle bit is cleared the SDRAM source address switches to the address specified in *PPVWIN0ADH* and *PPVWIN0ADL* registers. Ping Pong buffer switching is done by configuring PPSW bit in *MISCCTL* register. The buffer switching is illustrated Figure 76.

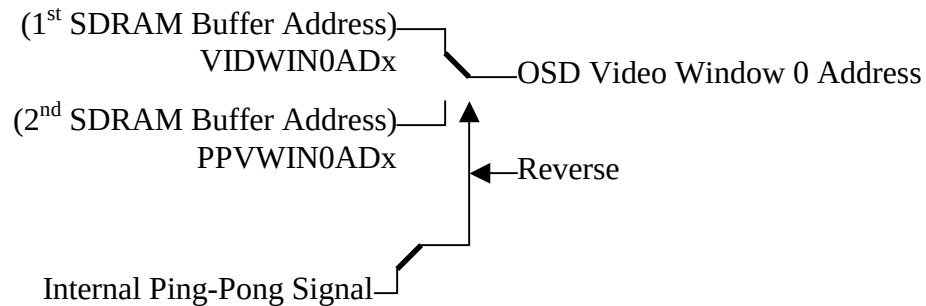


Figure 76: Ping-Pong buffers for the main video window (OSD Video Window 0)

12.11 On Screen Display Register Map (OSD)

Address	Register	Description
0003:0680	OSDMODE	OSD Mode Setup
0003:0682	VIDWINMD	Video Window Mode Setup
0003:0684	OSDWIN0MD	OSD Window 0 Mode Setup
0003:0686	OSDWIN1MD	OSD Window 1 Mode Setup
0003:0686	OSDATRMD	OSD Attribute Window Mode Setup
0003:0688	RECTCUR	Rectangular Cursor Setup
0003:068A	RSV0	Reserved
0003:068C	VIDWIN0OFST	Video Window 0 Offset
0003:068E	VIDWIN1OFST	Video Window 1 Offset
0003:0690	OSDWIN0OFST	OSD Window 0 Offset
0003:0692	OSDWIN1OFST	OSD Window 1 Offset
0003:0694	VIDWINADH	Video Window 0/1 Address - High
0003:0696	VIDWIN0ADL	Video Window 0 Address - Low
0003:0698	VIDWIN1ADL	Video Window 1 Address - Low
0003:069A	OSDWINADH	OSD Window 0/1 Address - High
0003:069C	OSDWIN0ADL	OSD Window 0 Address - Low
0003:069E	OSDWIN1ADL	OSD Window 1 Address - Low
0003:06A0	BASEPX	Base Pixel X
0003:06A2	BASEPY	Base Pixel Y
0003:06A4	VIDWIN0XP	Video Window 0 X-Position
0003:06A6	VIDWIN0YP	Video Window 0 Y-Position
0003:06A8	VIDWIN0XL	Video Window 0 X-Size
0003:06AA	VIDWIN0YL	Video Window 0 Y-Size
0003:06AC	VIDWIN1XP	Video Window 1 X-Position
0003:06AE	VIDWIN1YP	Video Window 1 Y-Position
0003:06B0	VIDWIN1XL	Video Window 1 X-Size
0003:06B2	VIDWIN1YL	Video Window 1 Y-Size

0003:06B4	OSDWIN0XP	OSD Bitmap Window 0 X-Position
0003:06B6	OSDWIN0YP	OSD Bitmap Window 0 Y-Position
0003:06B8	OSDWIN0XL	OSD Bitmap Window 0 X-Size
0003:06BA	OSDWIN0YL	OSD Bitmap Window 0 Y-Size
0003:06BC	OSDWIN1XP	OSD Bitmap Window 1 X-Position
0003:06BE	OSDWIN1YP	OSD Bitmap Window 1 Y-Position
0003:06C0	OSDWIN1XL	OSD Bitmap Window 1 X-Size
0003:06C2	OSDWIN1YL	OSD Bitmap Window 1 Y-Size
0003:06C4	CURXP	Rectangular Cursor Window X-Position
0003:06C6	CURYP	Rectangular Cursor Window Y-Position
0003:06C8	CURXL	Rectangular Cursor Window X-Size
0003:06CA	CURYL	Rectangular Cursor Window Y-Size
0003:06CC	RSV1	Reserved
0003:06CE	RSV2	Reserved
0003:06D0	W0BMP01	Window 0 Bitmap Value to Palette Map 0/1
0003:06D2	W0BMP23	Window 0 Bitmap Value to Palette Map 2/3
0003:06D4	W0BMP45	Window 0 Bitmap Value to Palette Map 4/5
0003:06D6	W0BMP67	Window 0 Bitmap Value to Palette Map 6/7
0003:06D8	W0BMP89	Window 0 Bitmap Value to Palette Map 8/9
0003:06DA	W0BMPAB	Window 0 Bitmap Value to Palette Map A/B
0003:06DC	W0BMPCD	Window 0 Bitmap Value to Palette Map C/D
0003:06DE	W0BMPEF	Window 0 Bitmap Value to Palette Map E/F
0003:06E0	W1BMP01	Window 1 Bitmap Value to Palette Map 0/1
0003:06E2	W1BMP23	Window 1 Bitmap Value to Palette Map 2/3
0003:06E4	W1BMP45	Window 1 Bitmap Value to Palette Map 4/5
0003:06E6	W1BMP67	Window 1 Bitmap Value to Palette Map 6/7
0003:06E8	W1BMP89	Window 1 Bitmap Value to Palette Map 8/9
0003:06EA	W1BMPAB	Window 1 Bitmap Value to Palette Map A/B
0003:06EC	W1BMPCD	Window 1 Bitmap Value to Palette Map C/D
0003:06EE	W1BMPEF	Window 1 Bitmap Value to Palette Map E/F
0003:06F0	RSV3	Reserved
0003:06F2	RSV4	Reserved
0003:06F4	MISCCTL	Miscellaneous Control
0003:06F6	CLUTRAMYCB	CLUT RAM Y/Cb Setup
0003:06F8	CLUTRAMCR	CLUT RAM Cr/Mapping Setup
0003:06FA	RSV5	Reserved
0003:06FC	PPVWIN0ADH	Ping-Pong Video Window 0 Address (High)
0003:06FE	PPVWIN0ADL	Ping-Pong Video Window 0 Address (Low)

12.12 On Screen Display Registers

12.12.1 OSDMODE

OSD Mode Setup

OSDMODE	0003:0680		offset: 0x00		default: 0x0000											
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	CS	OVRSZ	OHRSZ	EF	VVRSZ	VHRSZ	FSINV	CLUT	CABG[7]	CABG[6]	CABG[5]	CABG[4]	CABG[3]	CABG[2]	CABG[1]	CABG[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15	CS	0	R/W	<u>Cb/Cr or Cr/Cb format</u> » 0: Cb/Cr « » 1: Cr/Cb «
14	OVRSZ	0	R/W	<u>OSD Window Vertical Expansion Enable</u> » 0: x 1 « » 1: x 6/5 « When enabled, the bitmap window images are stretched by a factor of 6/5 in the vertical direction. Results in proper vertical sizing to display a normal VGA (640x480) image on a PAL television (720x576); i.e., 480 x 6/5=576
13	OHRSZ	0	R/W	<u>OSD Window Horizontal Expansion Enable</u> » 0: x 1 « » 1: x 9/8 « When enabled, the bitmap window images are stretched by a factor of 9/8 in the horizontal direction. Results in proper horizontal sizing to display a normal VGA (640x480) image on a PAL / NTSC television (720); i.e., 640 x 9/8=720
12	EF	0	R/W	<u>Expansion Filter Enable</u> » 0: Off « » 1: On « Valid when either VVRSZ or VHRSZ is on, or video window zoom is set. Caution is required when using the filter since expansion filter memory can only correspond to 720 horizontal pixels. <i>*This bit is latched by VD.</i>
11	VVRSZ	0	R/W	<u>Video Window Vertical Expansion Enable</u> » 0: x 1 « » 1: x 6/5 « When enabled, the video window images are stretched by a factor of 6/5 in the vertical direction. Results in proper vertical sizing to display a normal VGA (640x480) image on a PAL television (720x576); i.e., 480 x 6/5=576 <i>*This bit is latched by VD.</i>

				<u>Video Window Horizontal Expansion Enable</u> » 0: x 1 « » 1: x 9/8 «
10	VHSZ	0	R/W	When enabled, the video window images are stretched by a factor of 9/8 in the horizontal direction. Results in proper horizontal sizing to display a normal VGA (640x480) image on an NTSC/PAL television (720x576); i.e., $640 \times 9/8 = 720$ <i>*This bit is latched by VD.</i>
9	FSIV	0	R/W	<u>Field signal inversion</u> » 0: Uninverted « » 1: Inverted « <i>*This bit is latched by VD.</i>
8	CLUT	0	R/W	<u>CLUT selection</u> Selects look-up table » 0: ROM « » 1: RAM « <i>*This bit is latched by VD.</i>
70	CABG	0	R/W	<u>Background Color CLUT</u> Specifies image display background color by CLUT address. In parts that do not display image, color specified by this register is displayed.

12.12.2 VIDWINMD

Video Window Mode Setup

VIDWINMD	0003:0682			offset: 0x02												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	VFINV	V1EFC	VHZ1[1]	VHZ1[0]	VVZ1[1]	VVZ1[0]	VFF1	ACT1	RSV	V0EFC	VHZ0[1]	VHZ0[0]	VVZ0[1]	VVZ0[0]	VFF0	ACT0	
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	0	0	0	0	0	0	0	0	-	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15	VFINV	0	R/W	Video window 0/1 Expansion filter co-efficient Inverse » 0: Normal « » 1: Inversed « Note: If V1EFC or V0EFC is set, this bit is valid
14	V1EFC	0	R/W	Video window 1 Expansion filter co-efficient » 0: same co-efficients for field-0 and field-1 « » 1: different co-efficients for field-0 and field-1 « Valid when vertical expansion and expansion filter is set.
13:12	VHZ1	0	R/W	Video window1 horizontal direction zoom » 00: x1 « » 01: x2 « » 10: x4 « » 11: Reserved (same as '00') « *This bit is latched by VD.
11:10	VZ1	0	R/W	Video window1 vertical direction zoom » 00: x1 « » 01: x2 « » 10: x4 « » 11: Reserved (same as '00') « *This bit is latched by VD.
9	VFF1	0	R/W	Video window1 display mode » 0: Field mode « » 1: Frame mode « *This bit is latched by VD.
8	ACT1	0	R/W	Sets image display on/off video window1 » 0: Off « » 1: On « *This bit is latched by VD.
7	RSV			Reserved
6	V0EFC	0	R/W	Video window 0 Expansion filter co-efficient » 0: same co-efficients for field-0 and field-1 « » 1: different co-efficients for field-0 and field-1 « Valid when vertical expansion and expansion filter is set.
5:4	VHZ0	0	R/W	Video window0 horizontal direction zoom » 00: x1 « » 01: x2 « » 10: x4 « » 11: Reserved (same as '00') « *This bit is latched by VD.
3:2	VZ0	0	R/W	Video window0 vertical direction zoom » 0: x1 « » 1: x2 « » 0: x4 « » 1: Reserved (same as '00') « *This bit is latched by VD.
1	VFF0	0	R/W	Video window0 display mode » 0: Field mode « » 1: Frame mode « *This bit is latched by VD.
0	ACT0	0	R/W	Sets image display on/off video window0 » 0: Off « » 1: On « *This bit is latched by VD.

12.12.3 OSDWIN0MD

OSD Window 0 Mode Setup

OSDWIN0MD		0003:0684		offset:		0x04										default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	ATN0E	RGB0E	CLUTS0	OHZ0[1]	OHZ0[0]	OVZ0[1]	OVZ0[0]	BMW0[1]	BMW0[0]	BLND0[2]	BLND0[1]	BLND0[0]	TE0	OFF0	OACT0	
R/W	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	-	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15	RSV			Reserved
14	ATN0E	0	R/W	<u>Attenuation Enable for REC601</u> » 0: Normal (Y: 0-255, Cr: 0-255, Cb: 0-255) « » 1: Attenuated (Y: 16-235, Cr: 16-240, Cb: 16-240) «
13	RGB0E	0	R/W	<u>RGB input for window 0 Enable</u> » 0: bitmap input « » 1: 16-bit RGB mode « Only one OSD window can be configured for RGB input mode
12	CLUTS0	0	R/W	<u>CLUT select</u> » 0: ROM-look-up table « » 1: RAM-look-up table « *This bit is latched by VD.
11:10	OHZ0	0	R/W	<u>OSD Window0 Horizontal Zoom</u> » 00: x1 « » 01: x2 « » 10: x4 « » 11: Reserved (same as '00') « *This bit is latched by VD.
9:8	OVZ0	0	R/W	<u>OSD Window0 Vertical Zoom</u> » 00: x1 « » 01: x2 « » 10: x4 « » 11: Reserved (same as '00') « *This bit is latched by VD.
7:6	BMW0	0	R/W	<u>Bitmap bit width for OSD window0</u> » 00: 1 « » 01: 2 « » 10: 4 « » 11: 8 « *This bit is latched by VD, Valid for bitmap input mode
5:3	BLND0	0	R/W	<u>Blending Ratio for OSD Window0 and Video Window 0/1</u> OSD Video » 000: 0 1 « » 001: 1/8 7/8 « » 010: 2/8 6/8 « » 011: 3/8 5/8 « » 100: 4/8 4/8 « » 101: 5/8 3/8 « » 110: 6/8 2/8 « » 111: 1 0 « *This bit is latched by VD.
2	TE0	0	R/W	<u>Transparency Enable for OSD Window0</u> » 0: disable « » 1: enable « In Bitmap mode, When transparency is on, blending is only performed for pixels whose bitmap value is 0. In RGB mode, if transparency is on, the pixels with RGB as 0x0000 is blended with video windows. The blending is done as per the BLND0 register configuration. *This bit is latched by VD.
1	OFF0	0	R/W	<u>OSD Window0 Display Mode</u> » 0: Field mode « » 1: Frame mode « *This bit is latched by VD.
0	OACT0	0	R/W	<u>OSD Window0 Active (displayed)</u> » 0: Off « » 1: On « *This bit is latched by VD.

12.12.4 OSDWIN1MD

OSD Window 1 Mode Setup

OSDWIN1MD	0003:0686			offset: 0x06			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	OASW	ATN1E	RGB1E	CLUTS1	OHZ1[1]	OHZ1[0]	OVZ1[1]	OVZ1[0]	BMW1[1]	BMW1[0]	BLND1[2]	BLND1[1]	BLND1[0]	TE1	OFF1	OACT1
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15	OASW	0	R/W	<u>OSD Window1 Attribute Mode Enable</u> » 0: OSD Window1 « » 1: Attribute window « *This bit is latched by VD.
14	ATN1E	0	R/W	<u>Attenuation Enable for REC601</u> » 0: Normal (Y: 0-255, Cr: 0-255, Cb: 0-255) « » 1: Attenuated (Y: 16-235, Cr: 16-240, Cb: 16-240) «
13	RGB1E	0	R/W	<u>RGB input for window 1 Enable</u> » 0: bitmap input « » 1: 16-bit RGB mode « Only one OSD window can be configured for RGB input mode
12	CLUTS1	0	R/W	<u>CLUT Select OASW=0</u> » 0: ROM-look-up table « » 1: RAM-look-up table «
11:10	OHZ1	0	R/W	<u>OSD Window1 Horizontal Zoom OASW=0</u> » 00: x1 « » 01: x2 « » 10: x4 « » 11: Reserved « *This bit is latched by VD.
9:8	OVZ1	0	R/W	<u>OSD Window1 Vertical Zoom OASW=0</u> » 00: x1 « » 01: x2 « » 10: x4 « » 11: Reserved « *This bit is latched by VD.
7:6	BMW1	0	R/W	<u>Bitmap Bit Width OASW=0</u> Bitmap bit width for OSD Window1 » 00: 1-bit « » 01: 2-bits « » 10: 4-bits « » 11: 8-bits « *This bit is latched by VD, Valid for bitmap input mode
5:3	BLND1	0	R/W	<u>Blending Ratio for OSD Window1(OASW=0) & Video Window 0/1</u> OSD Video » 000: 0 1 « » 001: 1/8 7/8 « » 010: 2/8 6/8 « » 011: 3/8 5/8 « » 100: 4/8 4/8 « » 101: 5/8 3/8 « » 110: 6/8 2/8 « » 111: 1 0 « *This bit is latched by VD.
2	TE1	0	R/W	<u>Transparency Enable for OSD Window0 OASW=0</u> » 0: disable « » 1: enable « In Bitmap mode, When transparency is on, blending is only performed for pixels whose bitmap value is 0. In RGB mode, if transparency is on, the pixels with RGB as 0x0000 is blended with video windows. The blending is done as per the BLND1 register configuration. *This bit is latched by VD.
1	OFF1	0	R/W	<u>OSD Window0 Display Mode OASW=0</u> » 0: Field mode « » 1: Frame mode « *This bit is latched by VD.
0	OACT1	0	R/W	<u>OSD Window0 Active (displayed) OASW=0</u> » 0: Off « » 1: On « *This bit is latched by VD.

12.12.5 OSDATRMD

OSD Attribute Window Mode Setup

OSDATRMD	0003:0686			offset: 0x06												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	OASW	RSV	RSV	NA1	OHZA[1]	OHZA[0]	OVZA[1]	OVZA[0]	BLNKINT[1]	BLNKINT[0]	NA2[2]	NA2[1]	NA2[0]	NA3	OFFA	BLNK	
R/W	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	0	-	-	0	0	0	0	0	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15	OASW	0	R/W	<u>OSD Window1 Attribute Mode Enable</u> » 0: OSD Window1 « » 1: Attribute window « Note: This bit enables attribute mode for OSD Window1. <i>*This bit is latched by VD.</i>
14:13	RSV			Reserved
12	NA1	0	R/W	Not Applicable in Attribute Window OASW=1 Not Used, Please set to 0 <i>*This bit is latched by VD.</i>
11:10	OHZA	0	R/W	<u>OSD Attribute Window Horizontal Zoom</u> OASW=1 » 00: x1 « » 01: x2 « » 10: x4 « » 11: Reserved (same as '00') « <i>*This bit is latched by VD.</i>
9:8	OVZA	0	R/W	<u>OSD Window1 or Attribute Window Vertical Zoom</u> OASW=1 » 00: x1 « » 01: x2 « » 10: x4 « » 11: Reserved (same as '00') « <i>*This bit is latched by VD.</i>
7:6	BLNKINT	0	R/W	<u>Blinking Interval</u> OASW=1 » 00: 1-unit « » 01: 2-units « » 10: 3-units « » 11: 4-units « Specifies the blinking interval of the attribute window in units of 8 VD pulses <i>*This bit is latched by VD.</i>
5:3	NA2	0	R/W	Not Applicable in Attribute Window OASW=1 Not Used, Please set to 0 <i>*This bit is latched by VD.</i>
2	NA3	0	R/W	Not Applicable in Attribute Window OASW=1 Not Used, Please set to 0 <i>*This bit is latched by VD.</i>
1	OFFA	0	R/W	<u>OSD Attribute Window Display Mode</u> OASW=1 » 0: Field mode « » 1: Frame mode « <i>*This bit is latched by VD.</i>
0	BLNK	0	R/W	<u>OSD Attribute Window Blink Enable</u> OASW=1 » 0: disable « » 1: enable « <i>*This bit is latched by VD.</i>

12.12.6 RECTCUR

Rectangular Cursor Setup

RECTCUR	0003:0688				offset: 0x08				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RCAD[7]	RCAD[6]	RCAD[5]	RCAD[4]	RCAD[3]	RCAD[2]	RCAD[1]	RCAD[0]	CLUTSR	RCHW[2]	RCHW[1]	RCHW[0]	RCVW[2]	RCVW[1]	RCVW[0]	RCACT
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:8	RCAD	0	R/W	Rectangular Cursor Color Palette Address CLUT Select
7	CLUTSR	0	R/W	» 0: ROM-look-up table « » 1: RAM-look-up table « <i>*This bit is latched by VD.</i>
6:4	RCHW	0	R/W	Rectangular Cursor Horizontal Line Width Width is 4xRCHW, with 0 interpreted as 1 pixel Range is: 1,4,8,16,20,24,28 pixels <i>*This bit is latched by VD.</i>
3:1	RCVW	0	R/W	Rectangular Cursor Vertical Line Width Width is 2xRCVW, with 0 interpreted as 1 line Range is: 1,2,4,6,8,10,12,14 lines <i>*This bit is latched by VD.</i>
0	RCACT	0	R/W	Rectangular Cursor Active (displayed) » 0: Off « » 1: On « <i>*This bit is latched by VD.</i>

12.12.7 VIDWIN0OFST

Video Window 0 Offset

VIDWIN0OFST		0003:068C			offset: 0x0C												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	V0LO[8]	V0LO[7]	V0LO[6]	V0LO[5]	V0LO[4]	V0LO[3]	V0LO[2]	V0LO[1]	V0LO[0]		
R/W	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W		
Default	-	-	-	-	-	-	-	0	0	0	0	0	0	0	0	0		

Bit	Name	Reset Value	R/W	Function
15:9	RSV			Reserved
8:0	V0LO	0	R/W	<u>Video Window 0 Line Offset</u> Number of burst transfers (32-bytes) in a horizontal line. Video data format is YCbYCr, or 32-bits per 2 pixels, which gives 16-pixels/burst: Line width in pixels/16 i.e., $720/16 = 45$ (0x2D) <i>Note: If line width setting for the window result in a non-integer value, round the value up to the next larger integer and organize the data in SDRAM so that each line begins on a burst boundary.</i> <i>*This bit is latched by VD.</i>

12.12.9 OSDWIN0OFST**OSD Window 0 Offset**

OSDWIN0OFST				0003:0690		offset: 0x10				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	O0LO[8]	O0LO[7]	O0LO[6]	O0LO[5]	O0LO[4]	O0LO[3]	O0LO[2]	O0LO[1]	O0LO[0]	
R/W	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	-	-	-	-	-	-	-	0	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:9	RSV			Reserved
8:0	O0LO	0	R/W	<u>OSD Window 0 Line Offset</u> Number of burst transfers (32-bytes) in a horizontal line. This depends on OSD window bit depth: Line width in pixels * bitdepth / 256-bits/burst i.e., $64 * 8 / 256 = 2$ <i>Note: If line width and bitdepth settings for the window result in a non-integer value, round the value up to the next larger integer and organize the data in SDRAM so that each line begins on a burst boundary.</i> <i>*This bit is latched by VD.</i>

12.12.11 VIDWINADH**Video Window 0/1 Address - High**

VIDWINADH		0003:0694		offset: 0x14												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	V1AH[6]	V1AH[5]	V1AH[4]	V1AH[3]	V1AH[2]	V1AH[1]	V1AH[0]	RSV	V0AH[6]	V0AH[5]	V0AH[4]	V0AH[3]	V0AH[2]	V0AH[1]	V0AH[0]	
R/W	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	-	0	0	0	0	0	0	0	-	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15	RSV			Reserved
14:8	V1AH	0	R/W	<u>Video Window 1 SDRAM Source Address - High</u> 7 MSBs of SDRAM source address The SDRAM source address is specified offset from the SDRAM base address in units of 32 bytes. <i>*This bit is latched by VD.</i>
7	RSV			Reserved
6:0	V0AH	0	R/W	<u>Video Window 0 SDRAM Source Address - High</u> 7 MSBs of SDRAM source address The SDRAM source address is specified offset from the SDRAM base address in units of 32 bytes. <i>*This bit is latched by VD.</i>

12.12.12 **VIDWIN0ADL**

Video Window 0 Address - Low

[illegible][illegible]

12.12.13 **VIDWIN1ADL**

Video Window 1 Address - Low

[illegible][illegible]

12.12.14 OSDWINADH**OSD Window 0/1 Address - High**

OSDWINADH	0003:069A			offset: 0x1A													default: 0x0000		
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	O1AH[6]	O1AH[5]	O1AH[4]	O1AH[3]	O1AH[2]	O1AH[1]	O1AH[0]	RSV	O0AH[6]	O0AH[5]	O0AH[4]	O0AH[3]	O0AH[2]	O0AH[1]	O0AH[0]			
R/W	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W			
Default	-	0	0	0	0	0	0	0	-	0	0	0	0	0	0	0			

Bit	Name	Reset Value	R/W	Function
15	RSV			Reserved
14:8	O1AH	0	R/W	<u>OSD Window 1 / Attribute Window SDRAM Source Address - High</u> 7 MSBs of SDRAM source address The SDRAM source address is specified offset from the SDRAM base address in units of 32 bytes. <i>*This bit is latched by VD.</i>
7	RSV			Reserved
6:0	O0AH	0	R/W	<u>OSD Window 0 SDRAM Source Address - High</u> 7 MSBs of SDRAM source address The SDRAM source address is specified offset from the SDRAM base address in units of 32 bytes. <i>*This bit is latched by VD.</i>

12.12.16 OSDWIN1ADL

OSD Window 1 Address - Low

OSDWIN1ADL	0003:069E		offset: 0x1E												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	O1AL[15]	O1AL[14]	O1AL[13]	O1AL[12]	O1AL[11]	O1AL[10]	O1AL[9]	O1AL[8]	O1AL[7]	O1AL[6]	O1AL[5]	O1AL[4]	O1AL[3]	O1AL[2]	O1AL[1]	O1AL[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15	O1AL	0	R/W	<u>OSD Window 1 / Attribute Window SDRAM Source Address - Low</u> 16 LSBs of SDRAM source address (specified in 32-byte units) The SDRAM source address is specified offset from the SDRAM base address in units of 32 bytes. <i>*This bit is latched by VD.</i>

12.12.18 **BASEPY**

Base Pixel Y

BASEPY	0003:06A2				offset:		0x22				default: 0x0000					
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	BPY[8]	BPY[7]	BPY[6]	BPY[5]	BPY[4]	BPY[3]	BPY[2]	BPY[1]	BPY[0]
R/W	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	-	-	0	0	0	0	0	0	0	0	0

[illegible]

12.12.19 **VIDWIN0XP**

Video Window 0 X-Position

[illegible][illegible]

12.12.20 **VIDWIN0YP**

Video Window 0 Y-Position

WIDWIN0YP				0003:06A6		offset:		0x26								default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	V0Y[8]	V0Y[7]	V0Y[6]	V0Y[5]	V0Y[4]	V0Y[3]	V0Y[2]	V0Y[1]	V0Y[0]	
R/W	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	-	-	-	-	-	-	-	0	0	0	0	0	0	0	0	0	

[illegible]

12.12.21 **VIDWIN0XL**

Video Window 0 X-Size

[illegible][illegible]

12.12.22 VIDWIN0YL

Video Window 0 Y-Size

[illegible][illegible]

12.12.23 **VIDWIN1XP**

Video Window 1 X-Position

[illegible][illegible]

12.12.24 **VIDWIN1YP**

Video Window 1 Y-Position

VIDWIN1YP		0003:06AE			offset:		0x2E							default: 0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	V1Y[8]	V1Y[7]	V1Y[6]	V1Y[5]	V1Y[4]	V1Y[3]	V1Y[2]	V1Y[1]	V1Y[0]	
R/W	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	-	-	-	-	-	-	-	0	0	0	0	0	0	0	0	0	

[illegible]

12.12.25 VIDWIN1XL

Video Window 1 X-Size

VIDWIN1XL	0003:06B0				offset: 0x30		default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	V1W[11]	V1W[10]	V1W[9]	V1W[8]	V1W[7]	V1W[6]	V1W[5]	V1W[4]	V1W[3]	V1W[2]	V1W[1]	V1W[0]
R/W	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:12	RSV			Reserved
11:0	V1W	0	R/W	<u>Video Window 1 X-Width</u> Horizontal display width in pixels <i>*This bit is latched by VD.</i>

12.12.26 VIDWIN1YL

Video Window 1 Y-Size

VIDWIN1YL	0003:06B2				offset: 0x32		default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	V1H[10]	V1H[9]	V1H[8]	V1H[7]	V1H[6]	V1H[5]	V1H[4]	V1H[3]	V1H[2]	V1H[1]	V1H[0]
R/W	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
10:0	V1H	0	R/W	<u>Video Window 1 Y-Height</u> Vertical display height in lines <i>*This bit is latched by VD.</i>

12.12.27 OSDWIN0XP

OSD Bitmap Window 0 X-Position

[illegible][illegible]

12.12.28 **OSDWIN0YP**

OSD Bitmap Window 0 Y-Position

OSDWIN0YP	0003:06B6			offset: 0x36													default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	W0Y[8]	W0Y[7]	W0Y[6]	W0Y[5]	W0Y[4]	W0Y[3]	W0Y[2]	W0Y[1]	W0Y[0]		
R/W	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W		
Default	-	-	-	-	-	-	-	0	0	0	0	0	0	0	0	0		

[illegible]

12.12.29 OSDWIN0XL

OSD Bitmap Window 0 X-Size

[illegible][illegible]

12.12.30 OSDWIN0YL

OSD Bitmap Window 0 Y-Size

[illegible][illegible]

12.12.31 OSDWIN1XP

OSD Bitmap Window 1 X-Position

[illegible][illegible]

12.12.32 **OSDWIN1YP**

OSD Bitmap Window 1 Y-Position

OSDWIN1YP		0003:06BE			offset: 0x3E								default: 0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	W1Y[8]	W1Y[7]	W1Y[6]	W1Y[5]	W1Y[4]	W1Y[3]	W1Y[2]	W1Y[1]	W1Y[0]
R/W	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	-	-	0	0	0	0	0	0	0	0	0

[illegible]

12.12.33 **OSDWIN1XL**

OSD Bitmap Window 1 X-Size

[illegible][illegible]

12.12.34 **OSDWIN1YL**

OSD Bitmap Window 1 Y-Size

[illegible][illegible]

12.12.35 **CURXP**

Rectangular Cursor Window X-Position

CURXP 0003:06C4 offset: 0x44 default: 0x0000

[illegible][illegible]

12.12.36 **CURYP**

Rectangular Cursor Window Y-Position

CURYP	0003:06C6	offset: 0x46	default: 0x0000
-------	-----------	--------------	-----------------

[illegible][illegible]

12.12.37 **CURXL**

Rectangular Cursor Window X-Size

[illegible][illegible]

12.12.38 CURL

Rectangular Cursor Window Y-Size

[illegible][illegible]

12.12.39 **W0BMP01**

Window 0 Bitmap Value to Palette Map 0/1

[illegible][illegible]

12.12.40 W0BMP23

Window 0 Bitmap Value to Palette Map 2/3

[illegible][illegible]

12.12.41 **W0BMP45**

Window 0 Bitmap Value to Palette Map 4/5

[illegible][illegible]

12.12.42 **W0BMP67**

Window 0 Bitmap Value to Palette Map 6/7

[illegible][illegible]

12.12.43 W0BMP89

Window 0 Bitmap Value to Palette Map 8/9

[illegible][illegible]

12.12.44 **W0BMPAB**

Window 0 Bitmap Value to Palette Map A/B

[illegible][illegible]

12.12.45 **W0BMPCD**

Window 0 Bitmap Value to Palette Map C/D

[illegible][illegible]

12.12.46 **W0BMPEF**

Window 0 Bitmap Value to Palette Map E/F

[illegible][illegible]

12.12.49 **W1BMP45**

Window 1 Bitmap Value to Palette Map 4/5

[illegible][illegible]

12.12.50 **W1BMP67**

Window 1 Bitmap Value to Palette Map 6/7

[illegible][illegible]

12.12.51 **W1BMP89**

Window 1 Bitmap Value to Palette Map 8/9

[illegible][illegible]

12.12.52 **W1BMPAB**

Window 1 Bitmap Value to Palette Map A/B

[illegible][illegible]

12.12.53 **W1BMPCD**

Window 1 Bitmap Value to Palette Map C/D

[illegible][illegible]

12.12.54 **W1BMPEF**

Window 1 Bitmap Value to Palette Map E/F

[illegible][illegible]

12.12.55 MISCCTL

Miscellaneous Control

MISCCTL	0003:06F4			offset: 0x74													default: 0x0000		
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	CPBSY	PPSW	PPRV	RSV			
R/W	-	-	-	-	-	-	-	-	-	-	-	-	R	R/W	R/W	-			
Default	-	-	-	-	-	-	-	-	-	-	-	-	0	0	0	-			

Bit	Name	Reset Value	R/W	Function
15#	RSV			Reserved
3	CPBSY	0	R	<u>CLUT Write Busy</u> Used when writing CLUT data to RAM. » 0: Not busy, okay to write « » 1: Busy, do not write «
2	PPSW	0	R/W	<u>Ping-pong buffer toggle select</u> When PPRV=0: 0: Use SDRAM address in VIDWINADH / VIDWIN0ADL 1: Use SDRAM address in PPVWINAD0H / PPVWIN0ADL When PPRV=1: 0: Use SDRAM address in PPVIDWIN0H / PPVIDWIN0L 1: Use SDRAM address in VIDWINADH / VIDWIN0ADL
1	PPRV	0	R/W	<u>Ping-pong buffer Reverse</u> Inverts polarity of internal select signal. Affects bit PPSW.
0	RSV	0	R/W	Reserved, please set to 0.

12.12.57 CLUTRAMCR**CLUT RAM Cr/Mapping Setup**

CLUTRAMCR	0003:06F8			offset: 0x78			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	CR[7]	CR[6]	CR[5]	CR[4]	CR[3]	CR[2]	CR[1]	CR[0]	CADDR[7]	CADDR[6]	CADDR[5]	CADDR[4]	CADDR[3]	CADDR[2]	CADDR[1]	CADDR[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:8	CR	0	R/W	Write Cr data into built-in CLUT-RAM. Note: It is necessary to write into CLUTRAMYCB before CLUTRAMCR
7:0	CADDR	0	R/W	<u>CLUT Write Palette Address</u> When updating CLUT-RAM: - verify CPBSY=0 in MISCCTL and - write CLUTRAMYCB before CLUTRAMCR.

12.12.59 PPVWIN0ADL

Ping-Pong Video Window 0 Address (Low)

PPVWIN0ADL		0003:06FE		offset: 0x7E												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	V0PAL[15]	V0PAL[14]	V0PAL[13]	V0PAL[12]	V0PAL[11]	V0PAL[10]	V0PAL[9]	V0PAL[8]	V0PAL[7]	V0PAL[6]	V0PAL[5]	V0PAL[4]	V0PAL[3]	V0PAL[2]	V0PAL[1]	V0PAL[0]	
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	

Bit	Name	Reset Value	R/W	Function
150	V0PAL	0	R/W	<u>Ping-pong Video Window 0 Address (Low)</u> 16 LSBs of SDRAM source address (specified in 32-byte units) <i>*This bit is latched by VD.</i>

13 Video Encoder

13.1 Introduction

The Video Encoder is used to display images and video on TV monitor and various LCD displays. The Video Encoder converts digital YCbCr signals to analog and digital video signals. The Video Encoder can generate composite NTSC or PAL signals, and 8-bit digital LCD interface signals. To generate these signals, the Video Encoder has digital to analog converter, a digital LCD interface, and a set of signals to perform timing and synchronization. The block diagram for Video Encoder is shown in the *Figure 77*.

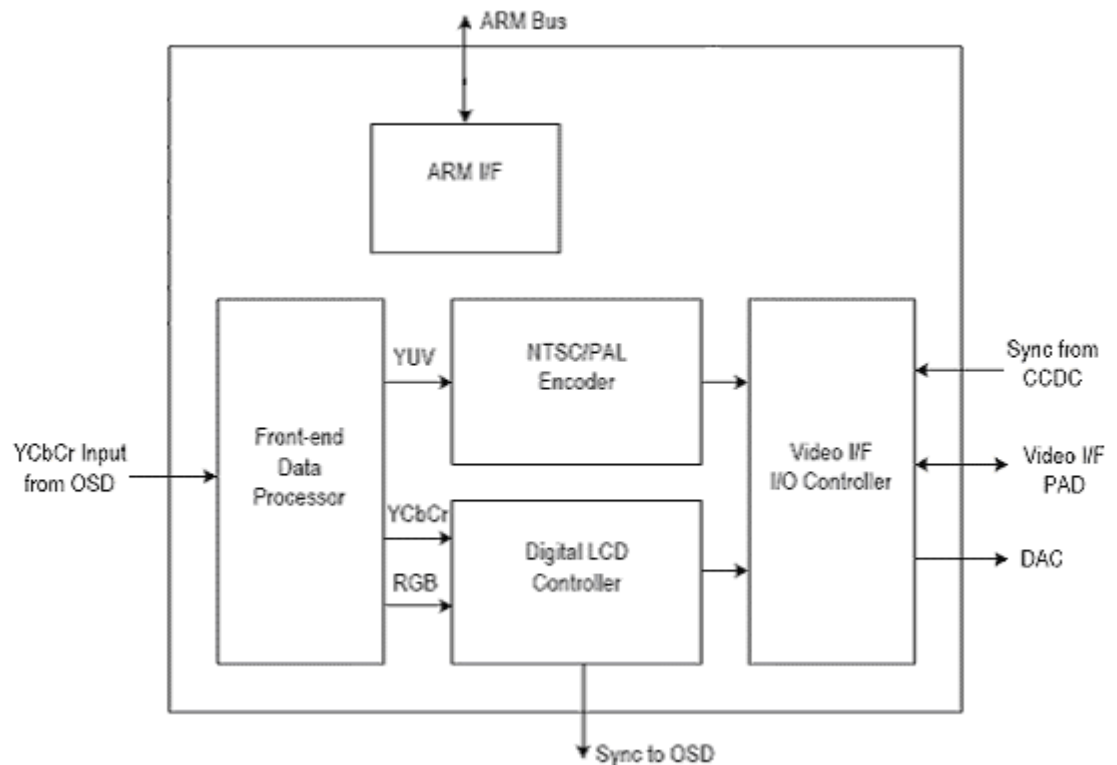


Figure 77: Video Encoder Block Diagram

13.1.1 Composite Analog Video Encoder features (NTSC/PAL)

- Master Clock Input - 27MHz
- 10-bit D/A Output Interface with output control
- NTSC/PAL Output Selectable
- Video Attribute Insertion (NTSC – Line 20, PAL – Line 23)
- Interlace/Non-Interlace selected output
- Setup Selection for NTSC
- VSYNC/ HSYNC Output
- Selectable VSYNC/Composite SYNC Output
- 16-235/0-255 Input Amplitude Selectable
- Video Output Blanking
- Chrominance Low Pass Filter 1.5MHz/3MHz
- Internal Color Bar Generation (100/75%)
- Closed caption support

13.1.2 Digital RGB/YUV Interface features

- Programmable DCLK out for Digital-RGB Output
- Supports various output formats
 - YCbCr 16 bit
 - YcbCr 8 bit
 - ITU-R BT.656
 - RGB 18-bit
 - RGB 8-bit
 - DisplayTech QVGA
 - STN
- Digital Low Pass Filter for RGB Output
- Digital VSYNC/HSYNC output
- Programmable Timing generators
- Built-in timing generators for EPSON/CASIO LCD panels
- Master-slave operation
- Internal Color Bar Generation (100/75%)

13.2 Video Encoder Operating Modes

13.2.1 Video Mode

The DM320 VENC supports the video modes discussed in following sections.

13.2.1.1 NTSC/PAL standard mode

DM320 VENC operates as the NTSC/PAL encoder. 525/60(NTSC) or 625/50 (PAL) formats are supported. For NTSC/PAL CVBS output, the VENC should be operated in this mode. The digital output for the LCD is also available simultaneously. The NTPLS field of VMOD register selects NTSC/PAL formats.

13.2.1.2 Nonstandard mode

This is a dedicated mode for digital LCD. In this mode the VENC operates at any given timing by register setting. To select non-standard mode, set 1 to the VMD field of register VMOD.

13.2.2 Synchronous Mode

The DM320 VENC supports two synchronous modes: master and slave. This can be used in above two video modes.

13.2.2.1 Master mode

This operates in synchronization with horizontal / vertical sync signals generated in built-in sync signal generator.

13.2.2.2 Slave mode

This operates in synchronization with sync signals input from outside. Set SLAVE bit of VMOD register to '1' when slave mode is used. When EXSYNC bit of SYNCTL register is '0', the external inputs from the HSYNC/VSNC pins are used as external sync signals. When EXSYNC = '1', the sync signals from the CCDC are used as external sync signals. This provides synchronizing the CCD timing and the video output timing. The polarity of external sync input is active high by default. It is possible to invert by EXHIV, EXVIV of SYNCTL. The field ID can be applied in the following combination. The external field input mode is enabled by setting EXFEN bit in SYNCTL register.

EXFEN	EXSYNC	Field ID Mode
0	-	Use field ID generated internally
1	0	Field ID is fixed to '0'
	1	Use field ID generated in the CCDC.

13.3 Front-end Data Processing

The front-end data processor receives the YCbCr pixel data from the OSD module and converts it into the digital YCbCr, YUV and RGB representation.

Figure 78 shows the block diagram of the front-end data processor. The 8 bit YCbCr pixel data from the OSD passes through the low-pass filter (pre-filter) at the very beginning of the data path. Then it is multiplexed with the internal color bar. The level attenuator has a role to compress the signal level with 0-255 into the ITU-R BT.601 specified level (Y:16-235, C:16-240). Each data path (digital YCbCr, digital RGB and YUV) has its own attenuator. After attenuation, for TV encoder path, the appropriate gain is applied to the attenuated Y, Cb and Cr. For the digital RGB data path, the RGB conversion matrix is applied. For the digital YCbCr, the attenuated YCbCr is used. Each data path is properly adjusted for the delay timing to match the output latency.

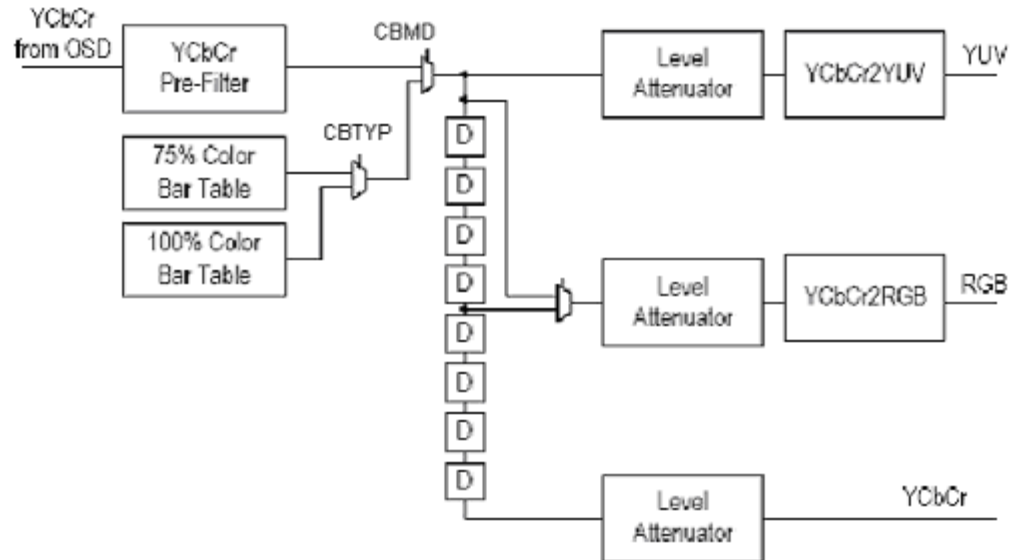


Figure 78: Front-end Data Processor Block Diagram

13.3.1 YCbCr Pre-filter

The pre-filter structure resides at the beginning of the data path. Each component (Y, Cb and Cr) has its own filter. The filter length can be programmed to 2 or 3 tap by PFLTY bit for Y and PFLTC bit for C in the VDPRO register. The sampling rate can be chosen as CLK_ENC or CLK_ENC/2 by PFLTR bit of VDPRO register.

13.3.2 Level Attenuator

The VENC assumes the two possibilities for the level of the input video data. One is the data conforming to the ITU-RBT.601 specified level, namely Y is ranged between 16-235 and C is between 16-240. Another is the data with the range of 0-255 for Y, C. The attenuator compresses the data with the range of 0-255 into the ITU-R BT.601 specified level. User can independently choose whether or not to apply the attenuation by bits ATCMP, ATYCC and ATRGB of VDPRO register for each data path.

13.3.3 YUV Conversion

The proper gain is applied to attenuated YCbCr data and then it is converted into YUV signals for CVBS generation. When STUP bit of VDPRO register is '1', 7.5% setup offset is added for the output. The STUP is effective both for NTSC or PAL. However for PAL mode setting STUP to 1 causes illegal output level. The gains are different in each component and each video mode (NTSC, NTSC with setup or PAL).

The BLNK bit of VMOD register is the blanking enable. When this is set to 1, CVBS output is blanked without regard to input video signals. There is a multiplexer for replacing the input data by blanking level in ITU-R BT.601.

13.3.4 RGB Conversion

VENC supports YCbCr to RGB conversion. The conversion matrixes are selectable by the RGBMAT bit of VDPRO register. Since the converted RGB may become negative due to finite precision arithmetic operation, zero level clipping is applied. The offset specified by OFST field of RGBCLP register is added followed by the upper level clipping. The level for clipping is set by UCLIP field of RGBCLP register. The output RGB is 8 bits in length.

13.4 NTSC/PAL Video Encoder

Figure 79 shows the block diagram of the NTSC/PAL video encoder block. The NTSC/PAL encoder supports PAL-M/N on trial in addition to NTSC and general PAL.

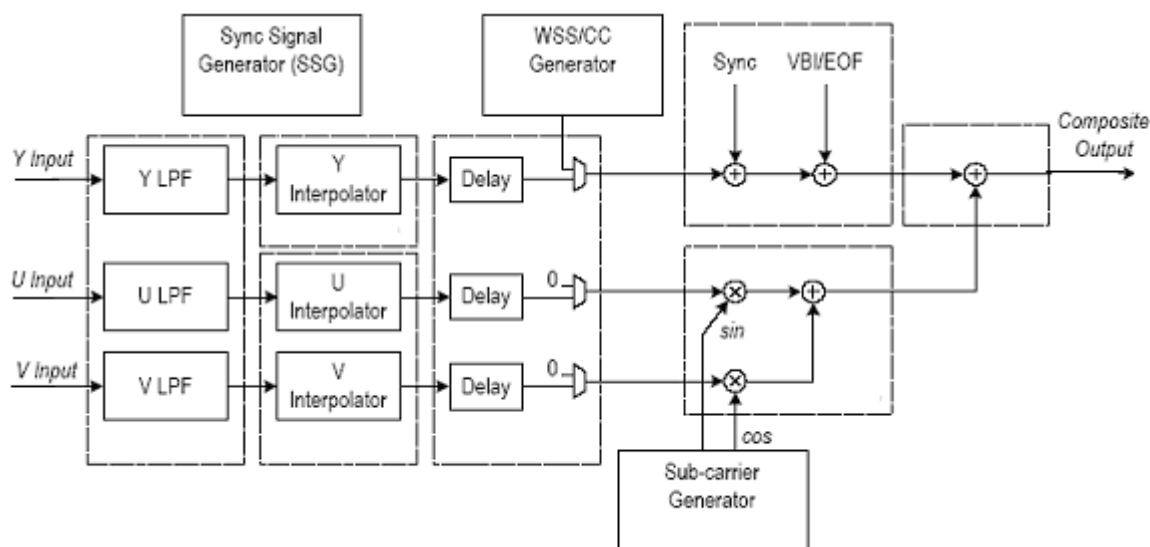


Figure 79 NTSC/PAL Encoder block diagram

13.4.1 Luma Signal Processing

The Y signal from the front end data processor is applied to x2 interpolation after an optional low-pass filter. The interpolation filter is 11 taps filter. The interpolation is disabled by default and can be enabled by setting bit YUPS of VDPRO register. When the OSD is in same clock mode, the interpolation is disabled by force since pixel rate is already 27MHz.

13.4.2 Chroma Signal Processing

The chroma signal is also applied to the x2 interpolation. The interpolation filter is 7 tap filter. The chroma upsampling is disabled by default and can be enabled by bit CUPS in VDPRO register.

13.4.3 CVBS Generation

The upsampled Y signal and upsampled chroma signals (U and V) after the luma and chroma signal processing modules , are mixed together in the YC mixer the mixed signal is output to the DAC. Setting bit VIE in VMOD register to 0 can force DAC input to L level regardless the output of YC mixer.

13.4.4 Y/C Delay

The delays on Y and C signals are aligned at those signals are mixed at the YC mixer module. The bit YCDLY of VDPRO register can adjust the delay of the Y component within -4 to 3 ENC clock.

13.4.5 Sub-carrier Generation

The video encoder generates the sub-carrier by internal DDS (Direct Digital Synthesizer). The phase resolution of DDS is $(1/1024)*360^\circ$. SC-H (Sub-carrier to Horizontal) phase can be controlled by user. First the initial phase value (0 to 1023) has to be written to the field SCSD of SCPROG register and then bit SCPRG of VDPRO register has to set to 1. Writing 1 to this bit automatically updates the sub-carrier phase. Update is occurred at the line 9 in the color field 1 for both NTSC and PAL. Writing 0 put back to the preset phase in same manner when writing 1. By default, preset value by which SC-H phase will close to 0° is applied.

Mode	Preset Value
NTSC	60
PAL	375
PAL-M	375
PAL-N	375

Figure 80 Sub-carrier Initial Phase Default Value

The frequency of sub-carrier is calculated by 3 parameters that are SRATE, SCCP and SCCN as follows,

$$f_{sc} = \frac{1}{1024} \times \left(SRATE - \frac{SCCN}{SCCP - SCCN} \right) \times 27MHz$$

The default number of these parameters are defined as in Figure 81.

Mode	SRATE	SCCN	SCCP
NTSC	135	-25	33-25
PAL	168	-2516	16875-2516
PAL-M	135	-87	143-87
PAL-N	135	-14391	16875-14391

Figure 81 Sub-carrier Parameter Default Value

13.4.6 Video Attribute Insertion

The video encoder has capability to insert video information into the vertical blanking period. For example, the video encoder can insert a video attribute which indicates the proper aspect ratio to the video receiver.

For NTSC mode, the video encoder can insert 14-bit video information on line 20 of every field to conform to the EIAJ CPX-1024 Video Aspect Ratio ID specification. Attribute information should be set using the ATR1 and ATR0 registers. The ATR2 register should be set with the 6-bit CRC data. Bit 7 of the ATR2 register enables attribute insertion.

For PAL mode, the video encoder can insert 14-bit video information (WSS) on line 20 of every frame to conform to the ETS 300-294 Wide Screen Signaling specification. Attribute information should be set in the ATR1 register and bits 5:0 of the ATR2 register. Bit 7 of the ATR2 register enables attribute insertion.

In NTSC and PAL encoding modes, data in the ATR1 and ATR0 registers are transferred to internal circuitry when the ATR2 register is set. For this reason, the ATR2 register should be set last.

13.4.7 Closed Caption

The DM320 VENC supports closed caption encoding (DSC former devices did not support). Closed caption data is transmitted on the line 21 of the odd field and the line 284 of the even field. It is possible to specify the fields on which closed captioning is enabled by CAPF of CAPCTL register. The data should be written to CAPDO or CAPDE registers for odd or even field respectively. It is required to load the data at least 1 line before. When data is written to CAPDO or CAPDE, the bit CAOST or CAEST of VSTAT register is changed to 1. This bit is automatically cleared to 0 when a caption data transmission is completed on the line 21 in odd field or the line 284 in even field. When the caption data registers CAPDO or CAPDE are not updated before the caption data transmission timing for the corresponding field, the ASCII code specified by CADF of CAPCTL register is automatically transmitted for closed caption data. The width of every data register is 7bit and the parity bit is automatically calculated by hardware.

13.4.8 Horizontal Timing

The timing such as location of horizontal sync pulses, color burst position and active video position is automatically calculated by hardware. Figure 82 and Figure 83 show horizontal timing characteristics for NTSC and PAL respectively. Figure 84 shows the parameter of the timing chart. Note that every duration between steps is two ENC clocks.

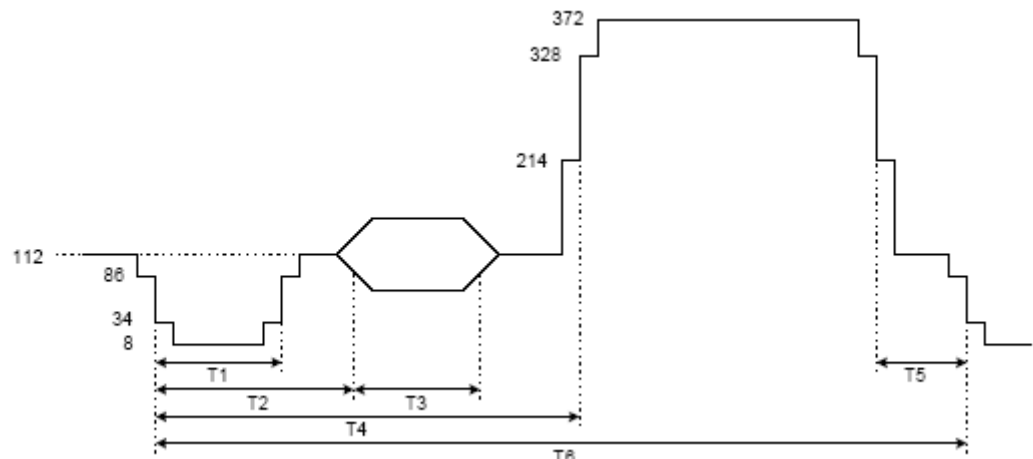


Figure 82 NTSC Horizontal Timing

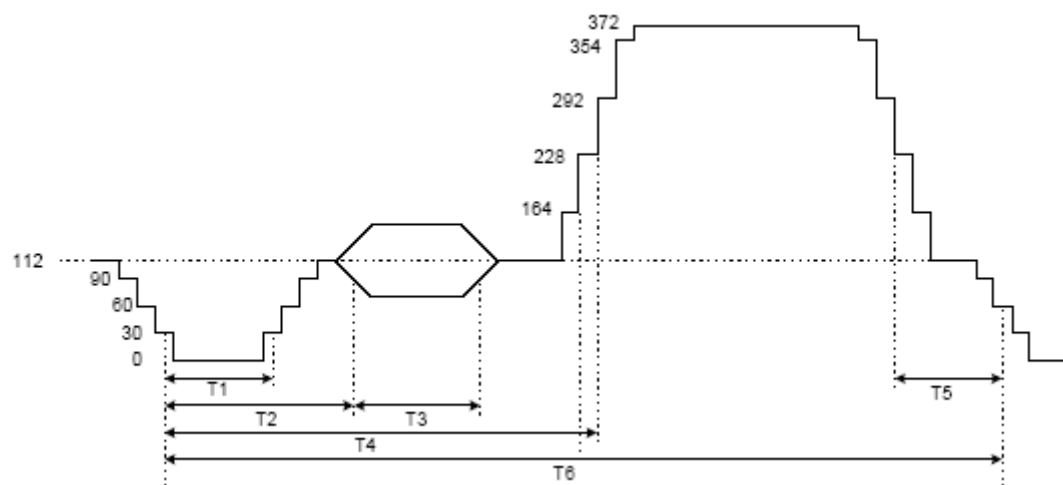


Figure 83 PAL Horizontal Timing

Parameter	Item	NTSC	PAL
T1	Horizontal Sync Pulse Width	126	128
T2	H Ref to Burst Start	147	155
T3	Burst	67	61
T4	H ref to H Blanking End	257	276
T5	Front porch	41	46
T6	1H	1716	1728

* The unit is ENC clock.

Figure 84 Horizontal Timing Parameters

13.4.9 Vertical Timing

The vertical timing is also controlled by hardware automatically for each mode (NTSC or PAL). Serration and equalization pulses are generated for appropriate lines. Color burst is automatically disabled on appropriate lines.

Figure 85 and Figure 86 shows the vertical timing characteristics for each NTSC and PAL modes. The signals appeared in these figures are generated in SSG (sync signal generator) block in the module.

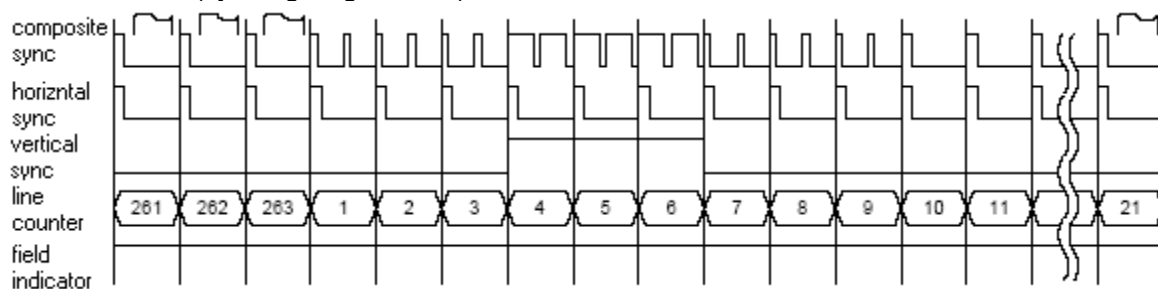


Figure 85 Non-Interlace NTSC Vertical Timing

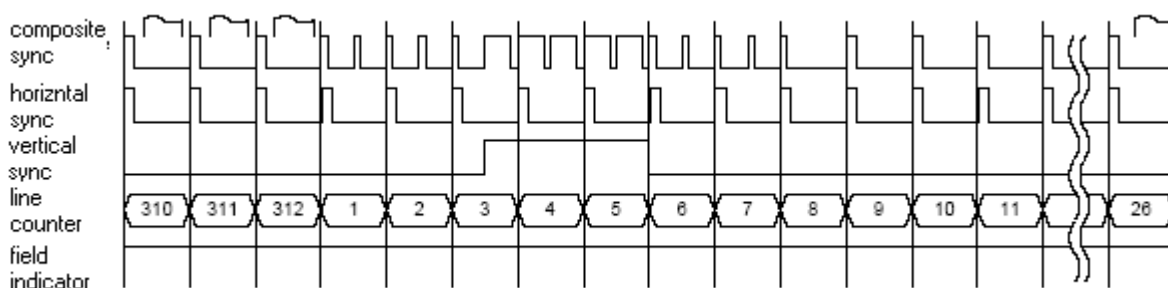


Figure 86 Non-Interlace PAL Vertical Timing

13.4.10 Internal Color Bar

The NTSC/PAL encoder can internally generate color bar by itself. Setting 1 to the CBAE bit of VMOD register enables internal color bar generator. The CBTY bit of VMOD register switches the saturation of the color bar. Set 0 for 75%, 1 for 100%.

13.5 Digital LCD Controller

Figure 87 shows the block diagram of the digital LCD controller block.

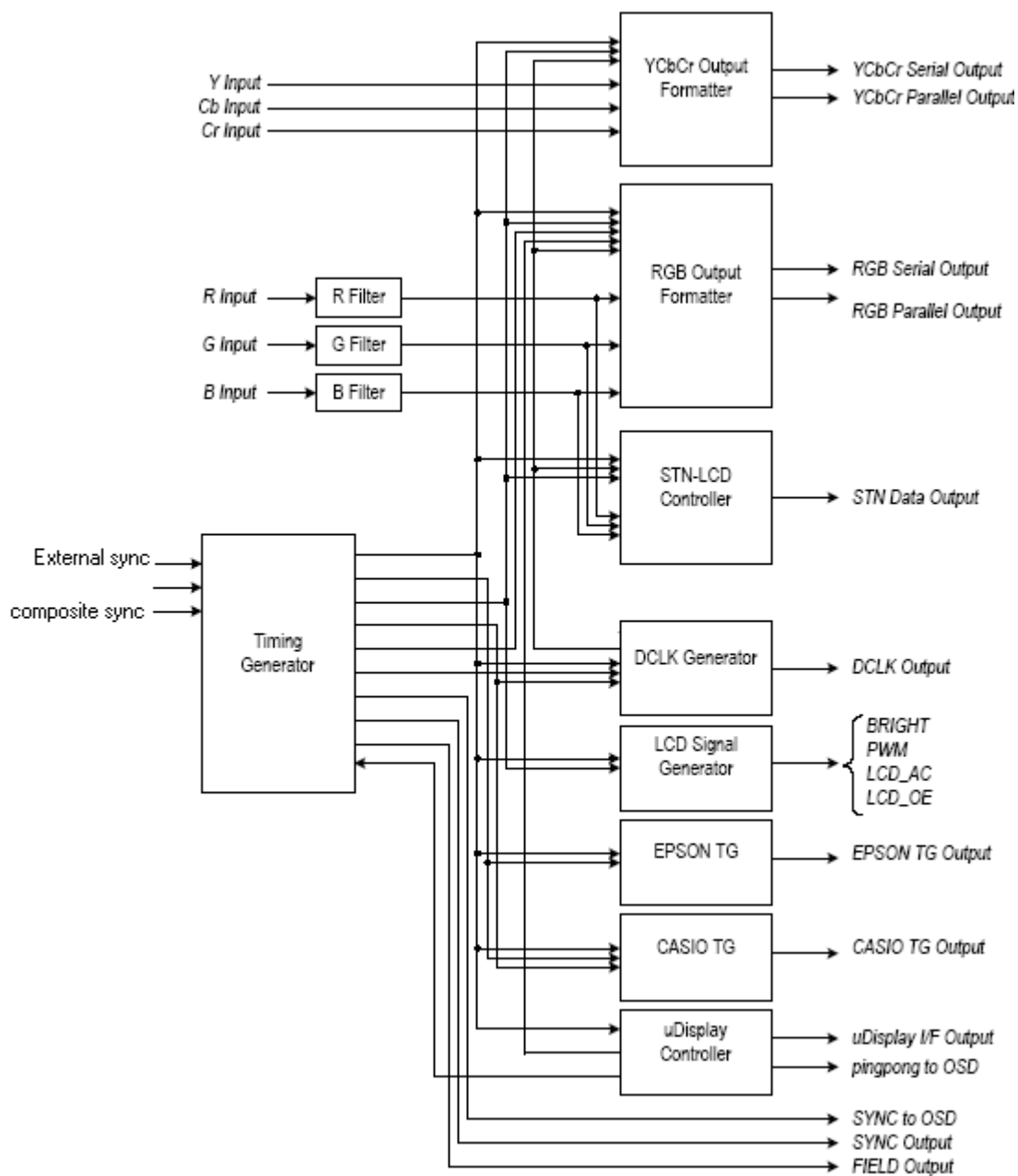


Figure 87 Digital LCD Controller Block Diagram

13.5.1 Digital Video Output Mode

DM320 video encoder supports 8 digital output modes. The mode can be selected by VDMD field of VMOD register. Figure 88 describes the various

modes of the video encoder. The output pins of video encoder acts differently in different video modes as shown in Figure 89.

VDMD	Mode	Description
0	YCC16	To output 16-bit YCbCr(4:2:2) in ITU-R BT.601 format.
1	YCC8	To output 8-bit YCbCr(4:2:2). This supports ITU-R BT.656 output.
2	RGB666	To output 18-bit RGB(6:6:6).
3	RGB8	To output signals for standard LCD.
4	EPSON	To output signals for LCD manufactured by EPSON.
5	CASIO	To output signals for LCD manufactured by CASIO.
6	UDISP	For DisplayTech QVGA (uDisplay)
7	STN	STN-LCD

Figure 88 Digital Video Output Mode

Pin Name	YCC16	YCC8	RGB666	RGB8	EPSON	CASIO	UDISP	STN
YOUT7	Y7	Data7	R7	Data7	Data5	Data5	Data7	Data7
YOUT6	Y6	Data6	R6	Data6	Data4	Data4	Data6	Data6
YOUT5	Y5	Data5	R5	Data5	Data3	Data3	Data5	Data5
YOUT4	Y4	Data4	R4	Data4	Data2	Data2	Data4	Data4
YOUT3	Y3	Data3	R3	Data3	Data1	Data1	Data3	Data3
YOUT2	Y2	Data2	G7	Data2	Data0	Data0	Data2	Data2
YOUT1	Y1	Data1	G6	Data1	GCP	STBYB	Data1	Data1
YOUT0	Y0	Data0	G5	Data0	LP	RIT	Data0	Data0
COU7	C7	LCD_AC	G4	LCD_AC	RES	CP	LCD_AC	LCD_AC
COU6	C6	LCD_OE	G3	LCD_OE	XINH	STH	DVALID	LCD_OE
COU5	C5	BRIGHT	G2	BRIGHT	YSCL	STB	BRIGHT	BRIGHT
COU4	C4	PWM	B7	PWM	YSCLD	GPCK	PWM	PWM
COU3	C3	CSYNC	B6	CSYNC	FRYS	GRES	UDISP_FS	Data11
COU2	C2	0	B5	0	FRYP	POL	0	Data10
COU1	C1	0	B4	0	FRX	0	BALANCE	Data9
COU0	C0	0	B3	0	DY	GSRT	FULL	Data8
GIO34	N/A	N/A	R2	N/A	N/A	N/A	N/A	N/A
GIO38	N/A	N/A	B2	EXTLINE_ID	N/A	N/A	N/A	N/A
GIO39	LCD_OE/FIELD	LCD_OE/FIELD	LCD_OE/FIELD	LCD_OE/FIELD	LCD_OE/FIELD	LCD_OE/FIELD	LCD_OE/FIELD	LCD_OE/FIELD

Figure 89 Output Signals in Each Video Output Mode

13.5.2 Timing Generator

The digital LCD controller provides a programmable timing generator. User can generate any non-standard timing other than NTSC/PAL format and use it as basic timing for LCD control. Figure 90 shows the block diagram of the timing generator.

Non-Std Sync Generator creates non-standard horizontal and vertical sync signals as specified in HINT and VINT registers. For NTSC/PAL standard timing, SSG(sync signal generator) can create the timing signals. SSG is a sub-module in the NTSC/PAL encoder block. The sync signals from non-standard and standard sync generators are multiplexed then used as the fundamental sync signal.

In addition, rising edge detected pulse and counter signals are also generated as the fundamental timing signal. Only these signals manage all timing in the digital LCD controller. Sync generator also produces OSD sync signals. The OSD sync signals are advanced of 8 pixel clock in order to cancel the data path delay which occurs in the front-end data processing. The OSD sync signals are active low while any others are active high signals because the OSD module requires active low signals.

“Valid Generator” generates the data valid signal that is the basis of the LCD_OE output. The assertion timing can be configured by HSTART, VSTART registers and valid duration can be programmed by HVALID, VVALID registers.

“Sync Pulse Generator” takes a role to produce sync signal output to chip outside. HSPLS and VSPLS registers program the pulse width of the output sync signals. User can delay the output sync signals by HSDLY and VSDLY registers without affecting the timing of the internal sync signals. The standard sync signals can also be delayed by these registers.

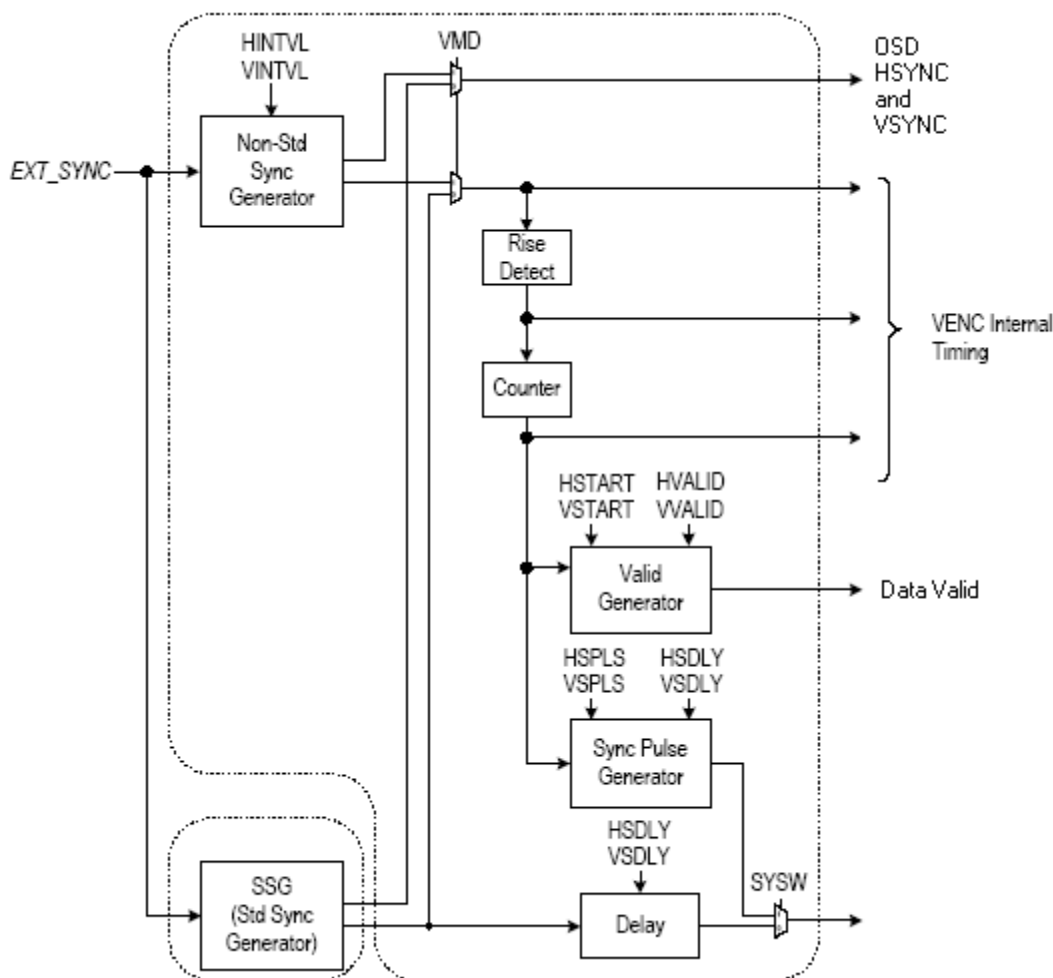


Figure 90 Timing Generator Functional Diagram

Figure 91 shows the timing diagram of basic timing signals generated by timing generator (HSYNC, VSYNC and LCD_OE). In standard mode operation, horizontal and vertical intervals are fixed to NTSC/PAL timing.

Figure 91 shows the basic timing signals diagram and Figure 92 lists the timing generator related registers.

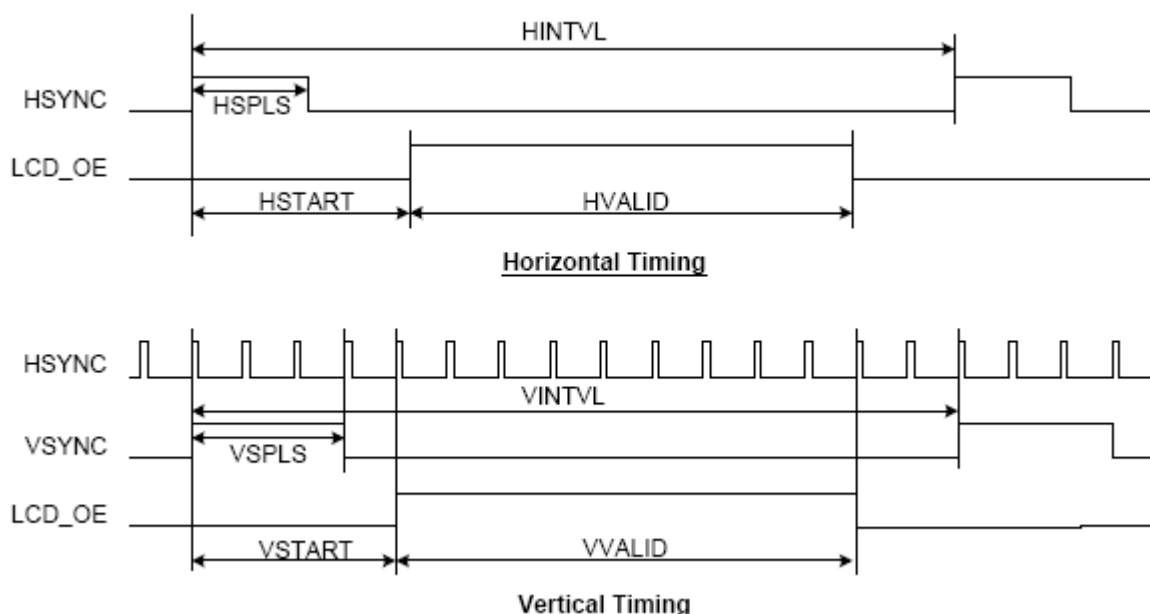


Figure 91 Timing Diagram of Basic Timing Signals

Register	Offset	Description	Unit
HSPLS[12-0]	0x04	HSYNC pulse width.	clk_enc
VSPLS[12-0]	0x05	VSYSN pulse width.	line
HINTVL[12-0]	0x06	HSYNC interval.	clk_enc
HSTART[12-0]	0x07	Horizontal data valid start position.	clk_enc
HVALID[12-0]	0x08	Horizontal data valid duration.	clk_enc
VINTVL[12-0]	0x09	VSYSN interval.	line
VSTART[12-0]	0x0A	Vertical data valid start position.	line
VVALID[12-0]	0x0B	Vertical data valid duration.	line
HSDLY[12-0]	0x0C	HSYNC delay.	clk_enc
VSDLY[12-0]	0x0D	VSYSN delay.	clk_enc

Figure 92 Timing Generator Related Registers

13.5.3 DCLK Generator

The LCD controller can generate dot clock called DCLK that is fed to LCD panels. The generated DCLK is output from VCLK pin. Frequency, waveform and valid duration of the DCLK can be programmed by register settings with various options. The digital data is output in synchronization to the rising edge of DCLK.

13.5.3.1 Pattern Register

The DCLKPTN register is provided for DCLK waveform configuration. User can configure various waveform for DCLK up to within 64 cycles period. The register is 64bit length mapped onto four 16bit registers (DCLKPTN0-DCLKPTN3). The effective pattern length can be specified in DCKPW of DCLKCTL register.

Moreover, another pattern register with same structure (DCLKPTN0A-DCLKPTN3A) is optionally provided. This enables user to switch the waveform by every certain line. Figure 93 shows DCLK pattern register configuration scheme.

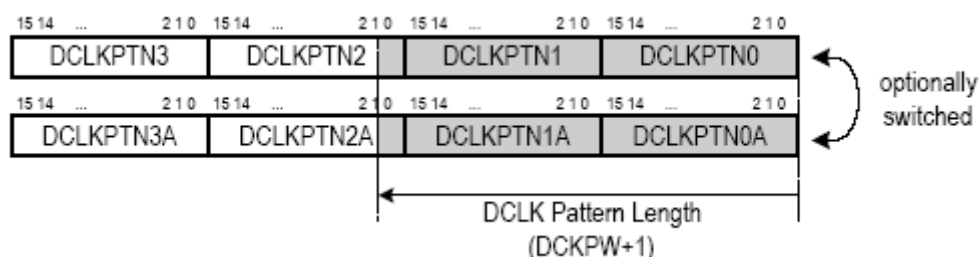
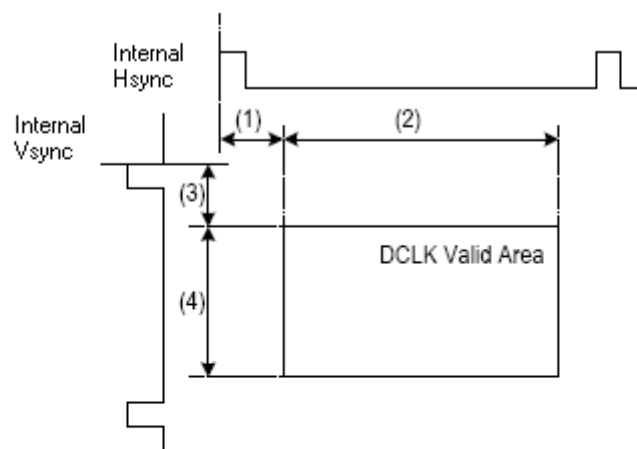


Figure 93 DCLK Pattern Register Configuration

13.5.3.2 Masking

It is possible to mask the DCLK signal in horizontal and vertical. As shown in Figure 94, the valid start position in the horizontal direction is set relative to HSYNC, and the length of valid data in the horizontal position is set relative to the horizontal start position of valid data. The horizontal resolution is ENC clock.

The DCKME field of DCLKCTL register can activate DCLK masking. Regarding horizontal start position, two sets of registers are provided as well as pattern register.



Mark	Register	Offset	Description	Unit
(1)	DHS[12-0] DHSA[12-0]	0x22 0x23	Horizontal DCLK mask start position.	clk_enc
(2)	DHV[12-0]	0x24	Horizontal DCLK mask range.	clk_enc
(3)	DVS[12-0]	0x25	Vertical DCLK mask start position.	line
(4)	DVV[12-0]	0x26	Vertical DCLK mask range.	line

Figure 94 DCLK Masking

13.5.4 Half-rate Mode

It is possible to divide the DCLK by two. The dividing can be applied to the internal DCLK or the output DCLK. When bit DCKOH of DCLKCTL register is 1, only the DCLK output is divided by two. Since the internal DCLK is not divided, the RGB data rate is not still changed. Thus, this mode can be used to connect to the LCD that captures the data using both edges of DCLK. On the other hand, when bit DCKIH of DCLKCTL register is 1, the internal DCLK is divided by two. When output dividing is not enabled, two clocks can be output per one data. Thus this mode can be used to connect to the LCD that requires double clock frequency of data rate.

13.5.5 Line Control

The DCLK controller provides two kinds of DCLK waveform alteration by line. The culling line ID which controls RGB data output sequence also affects DCLK waveform alteration.

When bit DCKCLP of LINECTL register is 1, DCLK pattern can be switched according to the culling line ID. The DCLK pattern on each line is specified by the DCLKPTN and DCLKPTNA registers.

When bit DCKCLI of LINECTL register is 1, DCLK polarity is inverted on the line whose line ID is the culling line ID set by the CULLLINE register.

Both DCKCLP and DCKCLI can be set to 1 simultaneously.

13.5.6 DCLK Output

VCLK output attributes, such as output enable, polarity and clock output on/off, can be controlled by VIOCTL register. Moreover, the DCLK output offset can be controlled in 0.5 ENC clock unit.

13.5.7 RGB Output Formatter

13.5.7.1 RGB Filter

The RGB data from the front-end data processor can be applied to low-pass filter. The LCD controller has 3 RGB filter instances for each component R, G, B. Each component is 8bit width and filtered output is also 8bit. The filter can be a 3 tap or 7 tap filter. These can be selected by the field DFLTS of RGBCTL register. The sampling clock can also be chosen from ENC clock or its divided clock by the bit DFLTR of RGBCTL register. Setting by DFLTS and DFLTR commonly affects three components (R, G, B).

13.5.8 YCbCr Output Formatter

YCbCr output formatter manages YCbCr data output in YCC16 and YCC8 mode.

In YCC16 mode, Y signal is output to YOUT7-0 ports at every DCLK rising edge, while Cb and Cr are alternately appeared onto COUT7-0 ports. The order of chroma output is controlled by field YCP of YCCCTL register.

In YCC8 mode, each component of YCbCr is alternately output from YOUT7-0 ports. The default output order is Cb-YCr-Y and can be modified by YCP registers. The data output is enabled only when data valid is asserted as well as RGB both in YCC16 and YCC8 mode.

ITU-R BT.656 format output is optionally available in YCC8 mode. Setting bit R656 of YCCCTL register to 1 activates this mode. In this mode, the YCbCr output timing and output order is fixed by hardware in order to conform to the standard and they cannot be altered by user. Note that this mode operates correctly only when the pixel clock frequency is half of ENC CLK.

13.5.9 Line ID Control

Line ID is the line identification flag altered at hsync and reset by vsync. This flag is used for RGB rotation order selector or DCLK waveform alteration. Normally line ID is toggled at every hsync. In addition to this normal behavior, the LCD controller provides culling line ID feature. This feature enables to use line ID toggled by specified line interval. User can also set line ID toggle position within the interval for even and odd field respectively.

The generated culling line ID affects the RGB rotation order when bit RGBCL in LINECTL register is 1. In this mode, the XORed signal of the actual line ID and the culling line ID will operate as the ID for the RGB rotation order. In addition to RGB order, DCLK waveform can be controlled by culling line ID. When DCKCLP is 1, effective DCLK pattern register is switched by the culling line ID. The DCLKPTN register is selected for culling line ID = 0 and DCLKPTNA for culling line ID = 1.

DCLK inversion feature is also provided when DCKCLI=1. In this mode, the created DCLK waveform is inverted for the culling line ID=1.

13.5.10 5/6 Line Culling

The digital video output can be vertically culled of 5/6. Setting the bit VCL56 of LINECTL register to 1, can activate the culling. When in this mode, one line of video output is discarded every six lines. The VENC asserts sync for the OSD and reads data from the OSD but ignores it for output for the culled line. Hsync output and LCD_OE assertion are also disabled in the culled line. The line position to be culled can be controlled by the VCLRD and VCLID of LINECTL register. Culling is enabled on the line where the internal culling counter value is equal to VCLID. The internal culling counter is incremented at hsync and reset at vsync. The reset value can be 0 or pseudo-random value which can be selected by the VCLRD.

13.5.11 Output Hold

The LCD controller provides the video output hold function. The controller can automatically stop the operation of the timing generator at when current line or field transmission is completed. During the hold mode, reading data from the OSD is suspended and the output of sync signals and video data is also suspended. Hold function is available only for digital video output in non-standard mode.

Setting the bit HLDL of LINECTL register to 1 can bring the controller into the line hold mode. Once this register

is set, the controller automatically turns into the hold mode when current line transmission is completed. Similarly, setting the bit HLDF of LINECTL register to 1 can activate the field hold mode. After this register is set, the timing

generator is suspended at when current field transmission is completed. Setting 0 to these registers can restart the timing generator. Figure 95 and Figure 96 shows the behavior of the output hold mode.

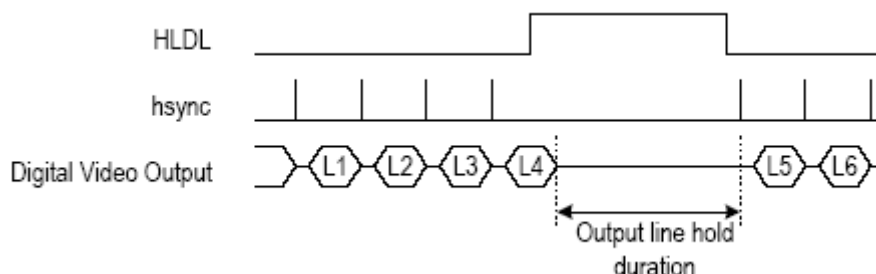


Figure 95 Output Line Hold Mode

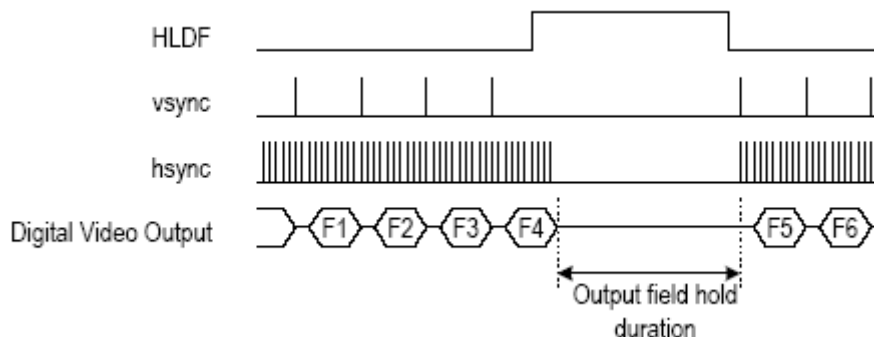


Figure 96 Output Field Hold Mode

13.5.12 GCP/FRC RAM Table

GCP/FRC table RAM of 64x16 bits is implemented by flops and it is used for the GCP pattern for EPSON D-TFD LCD panel or FRC table for STN-LCD. This RAM can be easily accessed through, RAMADR register and RAMPORT. The target RAM address (0-63) has to be specified in the RAMADR register, to access the RAMPORT register which returns the contents of the specified RAM address. Writing to it overwrites the specified RAM address with the written data. Accessing the RAMPORT automatically increments the RAMADR. The RAM contents are initialized to 0 at the reset.

13.5.13 LCD Signal Generation

BRIGHT pulse: For using BRIGHT pulse, bit BRE of LCDCTL register has to be set to 1. Polarity of this signal can be inverted by BRP in LCDCTL register. The unit is ENC clock.

LCD_AC: AC switching signal is toggled in specified field. To use LCD_AC, bit ACE of LCDCTL register has to be set to 1. The units of ACTH and ACTF are ENC clock and line respectively.

Pulse width modulation (PWM) signal: For using PWM, bit PWME of LCDCTL register has to be set to 1. Polarity can be inverted by bit PWMP in LCDCTL register. This is a free-run signal and not synchronized to any sync signals. The unit is ENC clock.

Figure 97 , Figure 98 and Figure 99 shows the timing for the BRIGHT , LCD_AC and PWM signals respectively.

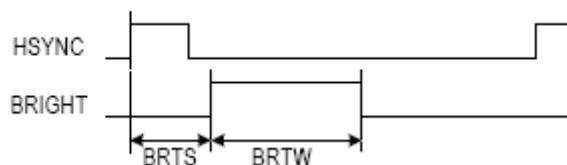


Figure 97 BRIGHT Signal Timing

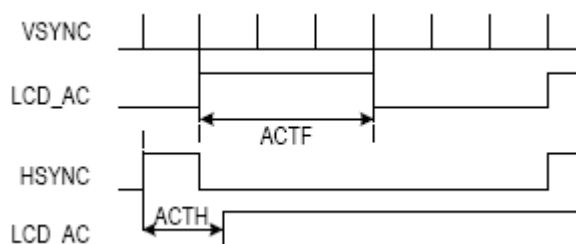


Figure 98 LCD_AC Signal Timing

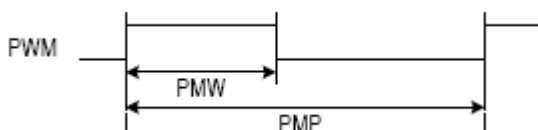


Figure 99 PWM Signal Timing

13.5.14 EPSON LCD Interface

In EPSON mode, the internal timing generator for EPSON LCD outputs timing signals. These signals can directly connect to EPSON D-TFD LCD panels. The waveform of generated timing signals is shown in Figure 100 and Figure 101. For EPSON TG signal assignment, please refer to Figure 89.

The number of low pulse of YSCL output can be one or two. One pulse is used for 1/2 reset drive type LCD, while two pulses mode is used for 1/2 select drive LCD. They can be selected by bit YSCLS of EPSON_LCDCTL register.

XINH signal has static DC output by default and its level can be set by bit XINHL of EPSON_LCDCTL register. When bit XINHS of EPSON_LCDCTL is set to 1, XINH will be controlled by internal TG. DCLK can be XSCL and it should be programmed by user appropriately. There is an option to reset vertical line counter earlier in the specified field. This function is called "SYS" (EPSON's

term). SYS can be activated by setting bit SYSE of EPSON_LCDCTL register. The effective field is set by bit SYSFID of EPSON_LCDCTL .

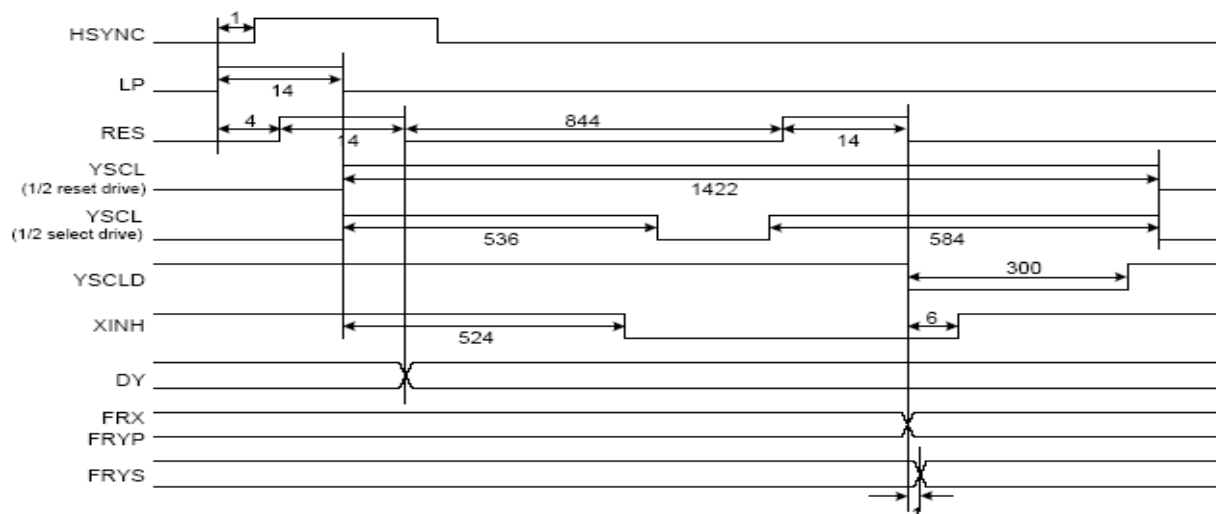


Figure 100 EPSON TG Horizontal Timing

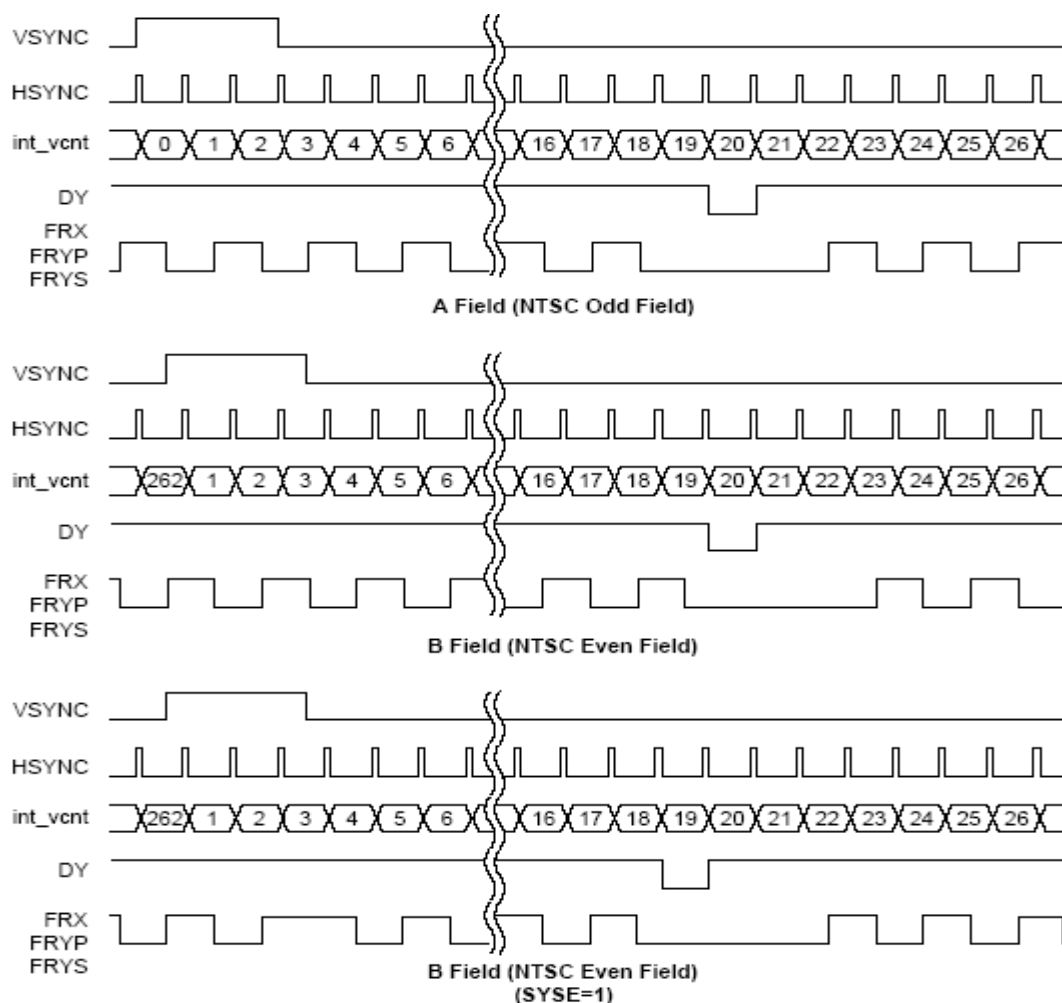


Figure 101 EPSON TG Vertical Timing

13.5.15 CASIO LCD Interface

CASIO LCD panels with serial RGB 6bit interface require the special timing signals. DM320 digital LCD controller can generate them by internal TG. The waveform is shown in Figure 102 and Figure 103.

Line number where a GSRT pulse is asserted is determined by VSTART register setting. Horizontal timing of GSRT can be controlled by bit CDM of CASIO_LCDCTL. When CDM is 0, GSRT rises during GPCK is H. This sets the display mode to "Normal Mode". When CDM is 1, GSRT rises before GPCK rising edge. This turns the display mode to "Vertical Reverse Mode".

POL is toggled every line, and it is reset to the current field ID at the line specified in VSTART register. GRES is forced to L level when bit CGRES of CASIO_LCDCTL register is set to 1.

RIT port can be controlled by bit CRIT of CASIO_LCDCTL register. Update of RIT output is synchronized to the rising edge of GPCK. STBYB is controlled by bit CSTB of CASIO_LCDCTL register. The value of this bit directly output onto STBYB port. CP clock signal for charge pump is also available in CASIO mode. TG creates two clock pulses with duty 50% for every line. When CSTB is 0, CP output is automatically fixed H level.

To use CASIO panel in PAL mode, it is required to apply 5/6 line culling. When culling is applied, the appropriate lines are culled, GPCK and GRES pulses are masked on these lines. Also toggling of POL is disabled at these lines.

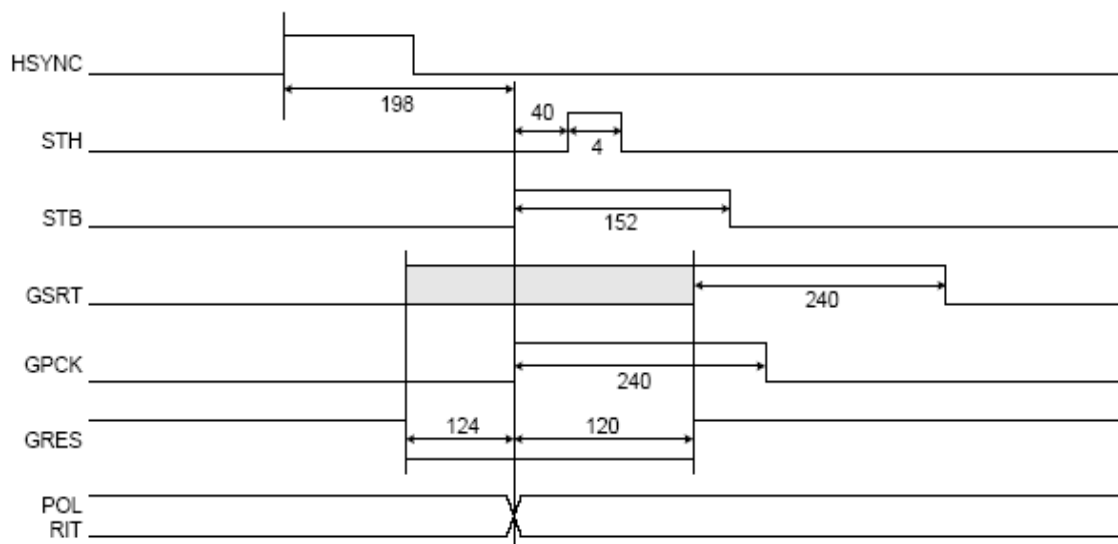


Figure 102 CASIO TG Horizontal Timing

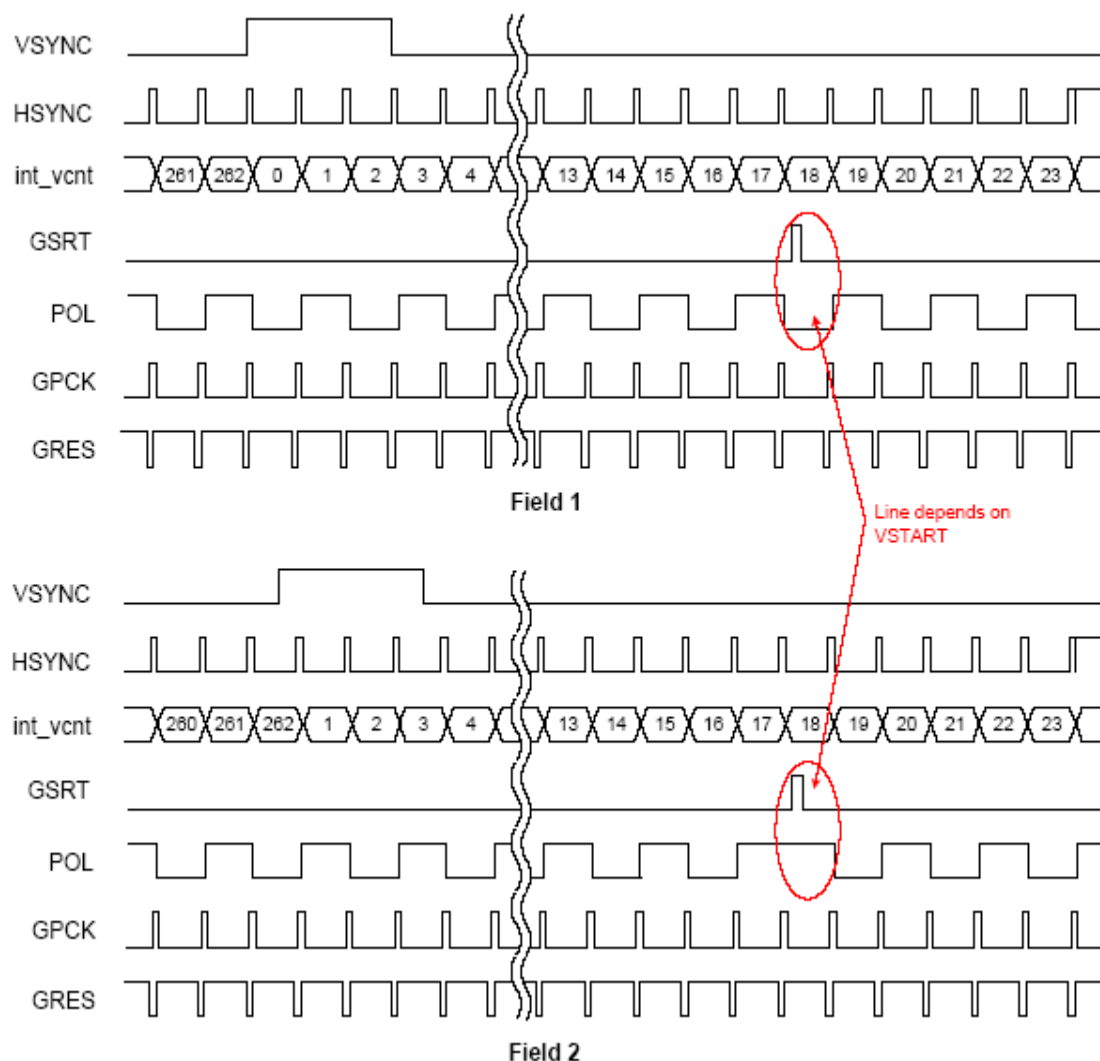


Figure 103 CASIO TG Vertical Timing

13.5.16 uDisplay (UDISP) Interface

DM320 LCD controller supports DisplayTech QVGA (uDisplay) interface. This is a new function in DM320. The features of uDisplay I/F are

1. Single color output during one field. RGB alteration occurs every field.
2. Monitor FULL status flag that indicates uDisplay cannot accept data, then stop data transmission while FULL is asserted.
3. Generate a FRAME_START pulse every three field.
4. Generate pingpong flag for OSD to synchronize the OSD video buffer to "field" in UDISP controller.

In UDISP mode, COUT0 and COUT1 pins are forced to input and used for FULL and BALANCE inputs respectively. Their status can be monitored in UDFUL and UDBAL bits of VSTAT register.

User can change the pulse width of FRAME_START signal. Bit FSW of UDISP_LCDCTL register represents the pulse width of FRAME_START in ENC clock unit. Since FSW is set to zero by default, it should be set the appropriate number before displaying the image.

13.5.17 STN LCD Interface

STN-LCD interface is also introduced in DM320. In STN mode, FRC (Frame Rate Control) is used for multiple tone representation. User is required to configure 16x16 FRC table. The 4 bit address corresponds to the tone of RGB data (MSB 4bit is used). The 16bit data represents dot pattern for a tone. It is shifted from MSB to LSB and circulates in period of 16 frame.

13.6 Interrupts

An interrupt pulse is asserted at every field. The internal vsync pulse triggers the interrupt assertion. Please note that it is synchronized to the internal sync timing, but not to output delayed sync or OSD sync. Figure 104 shows the detail interrupt generation.

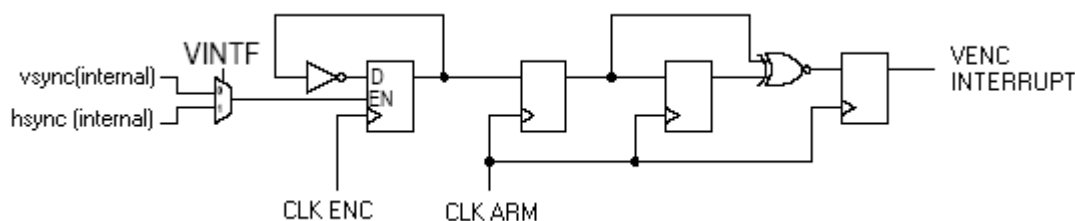


Figure 104 Interrupt Generation Circuit

13.7 DAC Control

There are several registers that control the embedded DAC macro (DAA15). To power down DAC itself, bit DAPD of VMOD register should be set to 1. By default, this bit is set to 1, so user has to write 0 before using analog output. To get DC output from DAOUT pin, DC output mode is useful. Setting DADC bit of DACTST register to 1 switches DAC digital input from normal video signal to value of DALVL bit in DACTST register. Bit DAOE of DACTST register can control the OE signal of DAA15. When set to 1, DAOUT output is fixed to L level.

13.8 Video Encoder Register Map (VENC)

Address	Register	Description
0003:0800	VMOD	Video Mode
0003:0802	VDCTL	Video interface I/O control
0003:0804	VDPRO	Video data processing
0003:0806	SYNCTL	Sync control
0003:0808	HSPLS	Horizontal Sync pulse width
0003:080A	VSPLS	Vertical Sync pulse width
0003:080C	HINT	Horizontal interval
0003:080E	HSTART	Horizontal valid data start position
0003:0810	HVALD	Horizontal data valid range
0003:0812	VINT	Vertical interval
0003:0814	VSTART	Vertical valid data start position
0003:0816	WALD	Vertical data valid range
0003:0818	HSDLY	Horizontal sync delay
0003:081A	VSDLY	Vertical sync delay
0003:081C	YCCCTL	YCbCr control
0003:081E	RGBCTL	RGB control
0003:0820	RGBCLP	RGB level clipping
0003:0822	LINECTL	Line Id control
0003:0824	CULLINE	Culling line control
0003:0826	LCDOUT	LCD output signal control
0003:0828	BRTS	Brightness start position signal control
0003:082A	BRTW	Brightness width signal control
0003:082C	ACCTL	LCD_AC signal control
0003:082E	PWMP	PWM start position signal control
0003:0830	PWMW	PWM width signal control
0003:0832	DCLKCTL	DCLK control
0003:0834	DCLKPTN0	DCLK pattern 0
0003:0836	DCLKPTN1	DCLK pattern 1
0003:0838	DCLKPTN2	DCLK pattern 2
0003:083A	DCLKPTN3	DCLK pattern 3
0003:083C	DCLKPTN0A	DCLK auxiliary pattern 0
0003:083E	DCLKPTN1A	DCLK auxiliary pattern 1
0003:0840	DCLKPTN2A	DCLK auxiliary pattern 2
0003:0842	DCLKPTN3A	DCLK auxiliary pattern 3
0003:0844	DCLKHS	Horizontal DCLK mask start
0003:0846	DCLKHSA	Horizontal auxiliary DCLK mask start
0003:0848	DCLKHR	Horizontal DCLK mask range
0003:084A	DCLKVS	Vertical DCLK mask start
0003:084C	DCLKVR	Vertical DCLK mask range
0003:084E	CAPCTL	Closed caption control
0003:0850	CAPDO	Closed caption odd field data
0003:0852	CAPDE	Closed caption even field data
0003:0854	ATR0	Video Attribute Data #0

0003:0856	ATR1	Video Attribute Data #1
0003:0858	ATR2	Video Attribute Data #2
0003:085A	EPSON_LCDCTL	EPSON LCD Control
0003:085A	CASIO_LCDCTL	CASIO LCD Control
0003:085A	UDISP_LCDCTL	uDisplay LCD control
0003:085A	STN_LCDCTL	STN LCD control
0003:085C	VSTAT	Video status
0003:085E	RAMADR	GCP/FRC table RAM address
0003:0860	RAMPOR	GCP/FRC table RAM data port
0003:0862	DACST	DAC Test
0003:0864	YCOLVL	YOUT and COUT levels
0003:0866	SCPROG	Sub-carrier programming

13.9 Video Encoder Interface Registers

13.9.1 VMOD

Video Mode Setup Register

VMOD	0003:0800				offset:	0x00				default: 0x0004						
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	VDMD[2]	VDMD[1]	VDMD[0]	RSV	ITLC	CBTYP	CBMD	NTPLS[1]	NTPLS[0]	SLAVE	VMD	BLNK	DACPD	VIE	VENC
R/W	-	R/W	R/W	R/W	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	0	0	0	-	0	0	0	0	0	0	0	0	1	0	0

Bit	Name	Reset Value	R/W	Function
15	RSV			<u>Reserved</u>
14:12	VDMD	0	R/W	<u>Digital video output mode</u> » 0: YCbCr parallel 16 bit « » 1: YCbCr serial 8 bit « » 2: RGB666 parallel 18 bit « » 3: RGB8 serial 8 bit « » 4: EPS0N « » 5: CASIO « » 6: uDISP (QVGA uDisplay) « » 7: STN (STN-LCD) «
11	RSV			<u>Reserved</u>
10	ITLC	0	R/W	<u>Scan Mode</u> » 0: Interlace « » 1: Non-interlace « Note: This setting is effective in standard mode (VMD=0)
9	CBTYP	0	R/W	<u>Color Bar Type</u> » 0: 75% « » 1: 100% «
8	CBMD	0	R/W	<u>Color bar mode</u> » 0: Normal output « » 1: Color bar output «
7:6	NTPLS	0	R/W	<u>NTSC / PAL mode select</u> » 0: NTSC « » 1: PAL « » 2: Reserved « » 3: Reserved «
5	SLAVE	0	R/W	<u>Master-slave select</u> .Set "1" to operate this module in slave mode in synchronization with external sync signal or CCDC sync signal » 0: Master mode « » 1: Slave mode «
4	VMD	0	R/W	<u>Video timing</u> » 0: NTSC/PAL timing « » 1: Not NTSC/PAL timing «
3	BLNK	0	R/W	<u>Blanking enable</u> » 0: Normal « » 1: Force Blanking «
2	DACPD	1	R/W	<u>DAC Powerdown</u> » 0: Normal mode« » 1: Powerdown mode «
1	VE	0	R/W	<u>Composite output enable</u> » 0: Fixed L level output « » 1: Normal composite output «
0	VENC	0	R/W	<u>Video Encoder Enable</u> » 0: Disable « » 1: Enable «

13.9.2 VIOCTL

Video I/O Control Register

VIDCTL	0003:0802				offset: 0x02										default: 0x1001	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	VCLKP	VCLKE	VCLKZ	RSV	RSV	RSV	RSV	RSV	RSV	DOMD[1]	DOMD[0]	RSV	YCDC	INPTRU	YCDIR
R/W	-	R/W	R/W	R/W	-	-	-	-	-	-	R/W	R/W	-	R/W	R/W	R/W
Default	-	0	0	1	-	-	-	-	-	-	0	0	-	0	0	1

Bit	Name	Reset Value	R/W	Function
15	RSV			<u>Reserved</u>
14	VCLKP	0	R/W	<u>VCLK output polarity</u> » 0: Non-inverse « » 1: Inverse «
13	VCLKE	0	R/W	<u>VCLK output enable</u> » 0: Off « » 1: On « Note: Setting 1 outputs DCLK from VCLK pin. When 0 VCLKP setting is still available
12	VCLKZ	1	R/W	<u>VCLK pin output enable</u> » 0: Output « » 1: High Impedence «
11:6	RSV			<u>Reserved</u>
5:4	DOMD	0	R/W	<u>Digital data output mode</u> » 0: Normal output « » 1: Inverse output « » 2: L level output « » 3: H level output «
3	RSV			<u>Reserved</u>
2	YCDC	0	R/W	<u>YOUT/COUT pin DC output mode</u> » 0: Normal output « » 1: DC level output «
1	INPTRU	0	R/W	<u>YOUT / COUT pin output mode</u> » 0: Digital video output « » 1: YIN / CIN input through «
0	YCDIR	1	R/W	<u>YOUT / COUT pin I/O control</u> » 0: Output « » 1: Input « Note: YOUT/COUT pin is used as YC data input pin for CCDC. Set '0' for digital video output

13.9.3 VDPRO

Video Data Processing Register

VDPRO	0003:0804				offset: 0x04				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	PFLTC[1]	PFLTC[0]	PFLTY[1]	PFLTY[0]	PFLTR	YCDLY[2]	YCDLY[1]	YCDLY[0]	RGBMAT	ATRGB	ATYCC	ATCOM	STUP	CRCUT	CUPS	YUPS
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:14	PFLTC	0	R/W	<u>C Prefilter select</u> » 0: No filter « » 1: 1+1 « » 2: 1+2+1 « » 3: Reserved « Note: When PFLTR and PFLTY are '1', set PFLTY to 1 tp adjust delay between Y and C.
13:12	PFLTY	0	R/W	<u>Y Prefilter select</u> » 0: No filter « » 1: 1+1 « » 2: 1+2+1 « » 3: Reserved «
11	PFLTR	0	R/W	<u>Prefilter sampling frequency</u> » 0: ENC clock / 2 « » 1: ENC clock «
10:8	YCDLY	0	R/W	<u>Delay adjutsment of Y signal in composite signal This is set by ENC clock</u> » 0: 0 « » 1: +1 « » 2: +2 « » 3: +3 « » 4: -4 « » 5: -3 « » 6: -2 « » 7: -1 « Note: The value is 2's complement
7	RGBMAT	0	R/W	<u>RGB conversion matrix (YCbCr - RGB)</u> 0: 1/512 R 596 0 816 Y-16 G = 596 -199 -414 Cb-128 B 596 1029 0 Cr-128 1: 1/512 R 512 0 718 Y G = 512 -176 -366 Cb-128 B 512 907 0 Cr-128
6	ATRGB	0	R/W	<u>Input video Attenuation control for RGB</u> » 0: No Attenuation « » 1: 0-255 -> REC601 specified level
5	ATYCC	0	R/W	<u>Input video Attenuation control for YCbCr</u> » 0: No Attenuation « » 1: 0-255 -> REC601 specified level
4	ATCOM	0	R/W	<u>Input video data damping control (for composite output)</u> » 0: No Attenuation « » 1: 0-255 -> REC601 specified level
3	STUP	0	R/W	<u>Setup level at NTSC output</u> » 0: 0% « » 1: 7.5% «
2	CRCUT	0	R/W	<u>Chroma signal low pass filter select</u> » 0: 1.5 MHz cut-off « » 1: 3 MHz cut-off «
1	CUPS	0	R/W	<u>Chroma signal up sampling enable</u> » 0: Off « » 1: On «
0	YUPS	0	R/W	<u>Y signal up sampling enable</u> » 0: Off « » 1: On «

13.9.4 SYNCTL

Sync Control Register

SYNCTL			0003:0806			offset: 0x06			default: 0x0001							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	EXFEN	EXFIV	EXCCD	EXVIV	EXHIV	CSP	CSE	SYSW	VSYNCS	VPL	HPL	SYE	SYDIR
R/W	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	0	0	0	0	0	0	0	0	0	0	0	0	1

Bit	Name	Reset Value	R/W	Function
15:13	RSV			<u>Reserved</u>
12	EXFEN	0	R/W	<u>External field input mode</u> Effective in slave operation (SLAVE=1). When this bit is at "1" and "EXCCD=1", the field signal generated in CCDC module is used as external field input, and at "1" and "EXCCD=0", the field is fixed "0". When this is set at "0", field ID generated internally is used » 0: Internal field mode « » 1: External field mode «
11	EXFIV	0	R/W	<u>External field input inversion, effective in slave operation</u> » 0: Non-inverse « » 1: Inverse «
10	EXCCD	0	R/W	<u>External sync select, effective in slave operation</u> » 0: HSYNC/VSYNC pin « » 1: CCD sync signal «
9	EXVIV	0	R/W	<u>External vertical sync input polarity</u> » 0: Active H « » 1: Active L «
8	EXHIV	0	R/W	<u>External horizontal sync input polarity</u> » 0: Active H « » 1: Active L «
7	CSP	0	R/W	<u>Composite sync output polarity</u> » 0: Active H « » 1: Active L « Note: Specifies polarity of composite sync output from COUT3 pin in YCC8 or RGB8 mode
6	CSE	0	R/W	<u>Composite sync output enable</u> » 0: Off « » 1: On « Note: Output control of composite sync signal from COUT3 pin in YCC8 or RGB8 mode. Writing 1 activates output, and writing 0 outputs inactive level determined by CSP
5	SYSW	0	R/W	<u>Output sync select</u> » 0: Normal « » 1: Sync pulse width processing mode« Note: Applicable to Standard mode only
4	VSYNCS	0	R/W	<u>VSYNC pin output signal select</u> » 0: Vertical sync signal « » 1: composite sync signal «
3	VPL	0	R/W	<u>Vertical sync output polarity</u> » 0: Active H « » 1: Active L «
2	HPL	0	R/W	<u>Horizontal sync output polarity</u> » 0: Active H « » 1: Active L «
1	SYE	0	R/W	<u>Sync output enable</u> » 0: Off « » 1: On «
0	SYDIR	1	R/W	<u>Horizontal / Vertical pin I/O control</u> » 0: Output « » 1: Input «

13.9.5 HSPLS

Horizontal Sync Pulse Width Register

[illegible][illegible]

13.9.6 VSPLS

Vertical Sync Pulse Width Register

[illegible][illegible]

13.9.7 HINT

Horizontal Interval Register

[illegible][illegible]

13.9.8 HSTART

Horizontal Valid Data Start Position

[illegible][illegible]

13.9.9 HVALID

Horizontal Data Valid Range

[illegible][illegible]

13.9.10 VINT

Vertical Interval

[illegible][illegible]

13.9.11 VSTART

Vertical Data Valid Start Position

[illegible][illegible]

13.9.12 VVALID

Vertical Valid Data Range

[illegible][illegible]

13.9.13 HSDLY

Horizontal Sync Delay Register

HSDLY	0003:0818			offset: 0x18												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	HSDLY[12]	HSDLY[11]	HSDLY[10]	HSDLY[9]	HSDLY[8]	HSDLY[7]	HSDLY[6]	HSDLY[5]	HSDLY[4]	HSDLY[3]	HSDLY[2]	HSDLY[1]	HSDLY[0]	
R/W	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	-	-	-	0	0	0	0	0	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:13	RSV			Reserved
12:0	HSDLY	0	R/W	Output delay of horizontal sync signal. This can delay horizontal sync output from HSYNC pin by ENC clock

13.9.14 VSDLY

Vertical Sync Delay Register

[illegible][illegible]

13.9.15 YCCCTL

YcbCr Control Register

YCCCTL	0003:081C			offset: 0x1C		default: 0x0000										
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	YCP[1]	YCP[0]	RSV	R656
R/W	-	-	-	-	-	-	-	-	-	-	-	-	R/W	R/W	-	R/W
Default	-	-	-	-	-	-	-	-	-	-	-	-	0	0	-	0

Bit	Name	Reset Value	R/W	Function
15:4	RSV			Reserved
3:2	YCP	0	R/W	<u>YC output order</u> YCC16 mode » 0: CbCr « » 1: CrCb « » 2: Reserved « » 3: Reserved « YCC8 mode » 0: Cb-Y-Cr-Y « » 1: Y-Cr-Y-Cb « » 2: Cr-Y-Cb-Y « » 3: Y-Cb-Y-Cr «
1	RSV			Reserved
0	R656	0	R/W	REC656 mode » 0: Normal « » 1: REC656 mode « Note: This is effective in YCC8 mode

13.9.16 RGBCTL

RGB Control Register

RGBCTL	0003:081E				offset: 0x1E				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	IRONM	DFLTR	DFLTS[1]	DFLTS[0]	RSV	RGBEF[2]	RGBEF[1]	RGBEF[0]	RSV	RGBOF[2]	RGBOF[1]	RGBOF[0]
R/W	-	-	-	-	R/W	R/W	R/W	R/W	-	R/W	R/W	R/W	-	R/W	R/W	R/W
Default	-	-	-	-	0	0	0	0	-	0	0	0	-	0	0	0

Bit	Name	Reset Value	R/W	Function
15:12	RSV			Reserved
11	IRONM	0	R/W	<u>Iron-man type RGB output</u> » 0: Normal « » 1: Iron-man type «
10	DFLTR	0	R/W	<u>RGB LPF sampling frequency</u> » 0: ENC clock / 2 « » 1: ENC clock « Note: Effective in all digital modes with RGB output
9:8	DFLTS	0	R/W	<u>RGB LPF select</u> » 0: No filter « » 1: 1+2+1 « » 2: 1+2+3+4+3+2+1 « » 3: Reserved « Note: Effective in all digital modes with RGB output
7	RSV			Reserved
6:4	RGBEF	0	R/W	<u>RGB output order (Line Id = 1)</u> » 0: R0-G1-B2 « » 1: R0-B1-G2 « » 2: G0-R1-B2 « » 3: G0-B1-R2 « » 4: B0-R1-G2 « » 5: B0-G1-R2 « » 6: Reserved « » 7: Reserved « Note: Effective in RGB8, EPSON and CASIO modes
3	RSV			Reserved
2:0	RGBOF	0	R/W	<u>RGB output order (Line Id = 0)</u> » 0: R0-G1-B2 « » 1: R0-B1-G2 « » 2: G0-R1-B2 « » 3: G0-B1-R2 « » 4: B0-R1-G2 « » 5: B0-G1-R2 « » 6: Reserved « » 7: Reserved « Note: Effective in RGB8, EPSON and CASIO modes

13.9.17 RGBCLP

RGB Level Clipping Register

RGBCLP	0003:0820				offset: 0x20				default: 0xFF00							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	UCLIP[7]	UCLIP[6]	UCLIP[5]	UCLIP[4]	UCLIP[3]	UCLIP[2]	UCLIP[1]	UCLIP[0]	OFST[7]	OFST[6]	OFST[5]	OFST[4]	OFST[3]	OFST[2]	OFST[1]	OFST[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:8	UCLP	255	R/W	Upper clip level for RGB output Note: Effective in all digital output modes with RGB output. Clipping is done following offset addition
7:0	OFST	0	R/W	Offset level for RGB output Note: Effective in all digital output modes with RGB output. Offset specified here can be added to RGB converted from YCbCr

13.9.18 LINECTL

Line Id Control Register

LINECTL	0003:0822			offset: 0x22			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	EXIDP	EXIDE	RSV	RSV	RSV	VCLID[2]	VCLID[1]	VCLID[0]	VCLRD	VCL56	HLDF	HLDL	LINID	DCKCLP	DCKCLI	RGBCL
R/W	R/W	R/W	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	-	-	-	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15	EXDP	0	R/W	External Line Id signal polarity » 0: Non-inverse « » 1: Inverse «
14	EXDE	0	R/W	External Line Id signal input mode » 0: Internal Id line « » 1: External Id Line «
13:11	RSV			Reserved
10:8	VCLD	0	R/W	Vertical Culling line position Effective when VCL56 = 1. No culling when VCLID > 5 Note: This setting is effective if VCL56 = 1
7	VCLRD	0	R/W	Vertical Culling counter reset mode » 0: Reset to 0 « » 1: Reset to Random value «
6	VCL56	0	R/W	Digital output vertical culling of 5/6 » 0: No culling « » 1: 5/6 culling «
5	HLDF	0	R/W	Digital output Field Hold » 0: Normal « » 1: Output hold « Note: Effective in non-standard mode
4	HLDL	0	R/W	Digital output Line Hold » 0: Normal « » 1: Output hold « Note: Effective in non-standard mode
3	LIND	0	R/W	Start line Id control in even field » 0: Line Id=0 « » 1: Line Id=1 «
2	DCKCLP	0	R/W	DCLK pattern switching by culling line Id » 0: Off « » 1: On «
1	DCKCLI	0	R/W	DCLK polarity inversion by culling line Id » 0: Off « » 1: On «
0	RGBCL	0	R/W	RGB output order switching by culling line Id » 0: Off « » 1: On «

13.9.19 CULLLINE

Culling Line control

CULLLINE	0003:0824				offset: 0x24				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	CLOF[3]	CLOF[2]	CLOF[1]	CLOF[0]	CLEF[3]	CLEF[2]	CLEF[1]	CLEF[0]	RSV	RSV	RSV	RSV	CULI[3]	CULI[2]	CULI[1]	CULI[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	-	-	-	-	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	-	-	-	-	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:12	CLOF	0	R/W	Culling line Id toggle position (Odd field) Specify toggle line numbers of culling line ID in odd field
11:8	CLEF	0	R/W	Culling line Id toggle position (Even field) Specify the culling line ID toggle position in even field
7:4	RSV			Reserved
3:0	CULI	0	R/W	Culling line Id inversion interval This is set by line. The interval is represented by CULI+1

13.9.20 LCDOUT

LCD Output Signal Control Register

LCDOUT	0003:0826			offset: 0x26		default: 0x0000										
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	FIDS	FIDP	PWMP	PWME	ACE	BRP	BRE	OEP	OEE
R/W	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	-	-	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:9	RSV			<u>Reserved</u>
8	FDS	0	R/W	<u>GI039 output signal select</u> » 0: Field Id « » 1: LCD_OE « Note: Select GI039 signal using FSEL3[13:12] = 01
7	FDP	0	R/W	<u>Field Id output polarity</u> » 0: Non-inverse « » 1: Inverse «
6	PWMP	0	R/W	<u>PWM output pulse polarity</u> » 0: Active H « » 1: Active L «
5	PWME	0	R/W	<u>PWM output control, effective when digital output mode with PWM output is selected</u> » 0: Off « » 1: On «
4	ACE	0	R/W	<u>LCD AC output control</u> » 0: Off « » 1: On «
3	BRP	0	R/W	<u>Bright output polarity</u> » 0: Active H « » 1: Active L «
2	BRE	0	R/W	<u>Bright output control</u> » 0: Off « » 1: On «
1	OEP	0	R/W	<u>LCD OE output polarity</u> » 0: Active H « » 1: Active L «
0	OEE	0	R/W	<u>LCD OE output control</u> » 0: Off « » 1: On «

13.9.21 BRTS

Brightness Start Position Signal Control Register

[illegible][illegible]

13.9.22 BRTW

Brightness Width Signal Control Register

[illegible][illegible]

13.9.23 ACCTL

LCD_AC signal Control Register

[illegible][illegible]

13.9.25 PWMW

Pulse Width Modulator Start Width Signal Control Register

[illegible][illegible]

13.9.26 DCLKCTL

DCLK Control Register

DCLKCTL	0003:0832				offset: 0x32				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	DOFST[1]	DOFST[0]	DCKEC	DCKME	DCKOH	DCKIH	RSV	RSV	DCKPW[5]	DCKPW[4]	DCKPW[3]	DCKPW[2]	DCKPW[1]	DCKPW[0]
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	0	0	0	0	0	0	-	-	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:11	RSV			Reserved
13:12	DOFST	0	R/W	<u>DCLK output offset</u> 0: 0 1: -0.5 2: +0.5 3: +1.0 To adjust the DCLK delay from VCLK pin by ENC clock.
11	DCKEC	0	R/W	<u>DCLK Pattern mode</u> » 0: Level « » 1: Enable « When 0, the specified value in DCLKPTN (or DCLKPTNA) register becomes the clock level of DCLK. When 1, DCLKPTN works as the clock enable for ENC clock
10	DCKME	0	R/W	<u>DCLK mask control</u> » 0: Off « » 1: On « Note: Setting '1' means mask is on and setting '0' means mask is off
9	DCKOH	0	R/W	<u>DCLK output dividing control</u> » 0: Divide by 1 « » 1: Divide by 2«
8	DCKIH	0	R/W	Internal DCLK dividing control » 0: Divide by 1 « » 1: Divide by 2«
7:6	RSV			Reserved
5:0	DCKPW	0	R/W	DCLK pattern valid bit width Note: Set the width of valid bits among all 64 bits in the DCLKPTN0-3 registers

13.9.28 DCLKPTN1

DCLK Pattern 1 Register

[illegible][illegible]

13.9.29 DCLKPTN2

DCLK Pattern 2 Register

[illegible][illegible]

13.9.30 DCLKPTN3

DCLK Pattern 3 Register

[illegible][illegible]

13.9.31 DCLKPTN0A

DCLK Auxiliary Pattern 0 Register

[illegible][illegible]

13.9.32 DCLKPTN1A

DCLK Auxiliary Pattern 1 Register

DCLKPTN1A	0003:083E		offset: 0x3E		default: 0x0000											
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	CPTN1A[15]	CPTN1A[14]	CPTN1A[13]	CPTN1A[12]	CPTN1A[11]	CPTN1A[10]	CPTN1A[9]	CPTN1A[8]	CPTN1A[7]	CPTN1A[6]	CPTN1A[5]	CPTN1A[4]	CPTN1A[3]	CPTN1A[2]	CPTN1A[1]	CPTN1A[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15	DCPTN1A	0	R/W	DCLK pattern (DCPTA31-DCPTA16)auxiliary The specified bit pattern is output in resolution of ENC clock units

13.9.33 DCLKPTN2A

DCLK Auxiliary Pattern 2 Register

DCLKPTN2A	0003:0840			offset: 0x40			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	CPTN2A[15]	CPTN2A[14]	CPTN2A[13]	CPTN2A[12]	CPTN2A[11]	CPTN2A[10]	CPTN2A[9]	CPTN2A[8]	CPTN2A[7]	CPTN2A[6]	CPTN2A[5]	CPTN2A[4]	CPTN2A[3]	CPTN2A[2]	CPTN2A[1]	CPTN2A[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15	DCPTN2A	0	R/W	DCLK pattern (DCPTA47-DCPTA32)auxiliary The specified bit pattern is output in resolution of ENC clock units

13.9.35 DCLKHS

Horizontal DCLK Mask Start Position

[illegible][illegible]

13.9.36 DCLKHSA

Horizontal auxiliary DCLK Mask Start Position

[illegible][illegible]

13.9.37 DCLKHR

Horizontal DCLK Mask Start Range

[illegible][illegible]

13.9.38 DCLKVS

Vertical Mask start position

[illegible][illegible]

13.9.39 DCLKVR

Vertical DCLK Mask Start Range

[illegible][illegible]

13.9.40 CAPCTL

Closed caption control register

CAPCTL	0003:084E			offset: 0x4E											default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	CADF[6]	CADF[5]	CADF[4]	CADF[3]	CADF[2]	CADF[1]	CADF[0]	RSV	RSV	RSV	RSV	RSV	RSV	CAPF[1]	CAPF[0]
R/W	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	-	-	-	-	-	-	R/W	R/W
Default	-	0	0	0	0	0	0	0	-	-	-	-	-	-	0	0

Bit	Name	Reset Value	R/W	Function
15	RSV			Reserved
14:8	CADF	0	R/W	Closed caption default data register
7:2	RSV			Reserved
1:0	CAPF	0	R/W	Closed caption field select, Specify the fields on which closed captioning is enabled » 0: No data output « » 1: Even field « » 2: Odd field « » 3: Both fields «

13.9.41 CAPDO

Closed caption odd field data

CAPDO	0003:0850			offset: 0x50													default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	CADO0[6]	CADO0[5]	CADO0[4]	CADO0[3]	CADO0[2]	CADO0[1]	CADO0[0]	RSV	CADO1[6]	CADO1[5]	CADO1[4]	CADO1[3]	CADO1[2]	CADO1[1]	CADO1[0]		
R/W	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W		
Default	-	0	0	0	0	0	0	0	-	0	0	0	0	0	0	0		

Bit	Name	Reset Value	R/W	Function
15	RSV			Reserved
14:8	CADO0	0	R/W	Closed caption data odd field (CADO06-CADO00) Specify the ASCII code for the 1st byte to be transmitted in odd field
7	RSV			Reserved
6:0	CADO1	0	R/W	Closed caption data odd field (CADO16-CADO10) Specify the ASCII code for the 2nd byte to be transmitted in odd field

13.9.42 CAPDE

Closed caption even field data

CAPDE	0003:0852			offset: 0x52			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	CADE0[6]	CADE0[5]	CADE0[4]	CADE0[3]	CADE0[2]	CADE0[1]	CADE0[0]	RSV	CADE1[6]	CADE1[5]	CADE1[4]	CADE1[3]	CADE1[2]	CADE1[1]	CADE1[0]
R/W	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	0	0	0	0	0	0	0	-	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15	RSV			Reserved
14:8	CADE0	0	R/W	Closed caption data even field (CADE06-CADE00) Specify the ASCII code for the 1st byte to be transmitted in even field
7	RSV			Reserved
6:0	CADE1	0	R/W	Closed caption data even field (CADE16-CADE10) Specify the ASCII code for the 2nd byte to be transmitted in even field

13.9.46 EPSON_LCDCTL

EPSON LCD Control Register

EPSON_LCDCTL	0003:085A	offset:	0x5A	default: 0x0000												
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	YSCLS	XINHS	XINHL	SYSFID	SYSSE	GCPS[2]	GCPS[1]	GCPS[0]
R/W	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	-	-	-	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:8	RSV			Reserved
7	YSCLS	0	R/W	<u>Low pulse number of YSCL</u> » 0: one pulse (1/2 reset drive) « » 1: two pulses (1/2 select drive) «
6	XINHS	0	R/W	<u>XINH Signal select</u> » 0: DC output « » 1: TG controlled «
5	XINHL	0	R/W	<u>XINH DC level</u> » 0: L level output « » 1: H level output «
4	SYSFID	0	R/W	<u>SYS function applied field ID select</u> <u>SYS function enable</u> » 0: Off « » 1: On « Note: setting '1 resets the vertical counter one line earlier in the field selected by SYSFID
2:0	GCPS	0	R/W	<u>GCP pattern select</u> » 0: ROM (Type 0) « » 1: ROM (Type 1) « » 2: ROM (Type 2) « » 3: ROM (Type 3) « » 4: RAM « » 5-7: Reserved « Note: When RAM is used, data written in RAM address 0-63 is shifted out by one bit from MSB to LSB and output as GCP pulse

13.9.47 CASIO_LCDCTL

CASIO LCD Control Register

CASIO_LCDCTL	0003:085A			offset: 0x5A												default: 0x0000		
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	CRIT	CSTB	CGRES	CDM		
R/W	-	-	-	-	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W		
Default	-	-	-	-	-	-	-	-	-	-	-	-	0	0	0	0		

Bit	Name	Reset Value	R/W	Function
15:4	RSV			Reserved
3	CRIT	0	R/W	<u>RII port control</u> » 0: L level output « » 1: H level output « Note: Setting is enabled at the rising edge of horizontal sync signal
2	CSTB	0	R/W	<u>STBYB port control</u> » 0: L level output « » 1: H level output « Note: CP output is fixed at H level when 0
1	CGRES	0	R/W	<u>GRES port control</u> » 0: Normal « » 1: Fixed L level «
0	CDM	0	R/W	<u>Display mode</u> » 0: Normal « » 1: Vertical reverse mode«

13.9.49 STN_LCDCTL

STN LCD Control Register SCPROG – Sub-carrier programming Register

STN	LCDCTL	0003:085A	offset: 0x5A	default: 0x0000

[illegible][illegible]

13.9.50 VSTAT

Video Status Register

VSTAT	0003:085C			offset: 0x5C			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	CAEST	CAOST	RSV	RSV	RSV	FIDST	VSST	HSST	UDBAL	UDFUL
R/W	-	-	-	-	-	-	R	R	-	-	-	R	R	R	R	R
Default	-	-	-	-	-	-	0	0	-	-	-	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:10	RSV			<u>Reserved</u>
9	CAEST	0	R	<u>Closed caption status (even field).</u> » 0: Ready « » 1: Data is being input « Note: The bit automatically becomes '0', when transmission is done
8	CAOST	0	R	<u>Closed caption status (odd field).</u> » 0: Ready « » 1: Data is being input « Note: The bit automatically becomes '0', when transmission is done
7:5	RSV			<u>Reserved</u>
4	FIDST	0	R	<u>Field ID monitor</u>
3	VSST	0	R	<u>VSYNC monitor</u>
2	HSST	0	R	<u>HSYNC monitor</u>
1	UDBAL	0	R	<u>uDisplay 'Balance' signal monitor</u>
0	UDFUL	0	R	<u>uDisplay 'Full' signal monitor</u>

13.9.51 RAMADR

GCP / FRP RAM table address

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RAMADR[5]	RAMADR[4]	RAMADR[3]	RAMADR[2]	RAMADR[1]	RAMADR[0]
R/W	-	-	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	-	-	-	-	-	0	0	0	0	0	0

[illegible]

13.9.52 RAMPORT

GCP / FRP RAM Data Port

RAMPORT	0003:0860			offset: 0x60			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RAMPORT[15]	RAMPORT[14]	RAMPORT[13]	RAMPORT[12]	RAMPORT[11]	RAMPORT[10]	RAMPORT[9]	RAMPORT[8]	RAMPORT[7]	RAMPORT[6]	RAMPORT[5]	RAMPORT[4]	RAMPORT[3]	RAMPORT[2]	RAMPORT[1]	RAMPORT[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:0	RAMPORT	0	R/W	RAM data port At reading, the data in the address specified in the RAMA register can be read. At writing, the data is written to the address specified in the RAMA register. The RAMA is automatically incremented every access to the RAMP register in both write and read cases

13.9.53 DACTST

DAC Test Register

DACTST	0003:0862				offset: 0x62		default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	DACOE	DACDC	DALVL[9]	DALVL[8]	DALVL[7]	DALVL[6]	DALVL[5]	DALVL[4]	DALVL[3]	DALVL[2]	DALVL[1]	DALVL[0]
R/W	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:12	RSV			Reserved
11	DACOE	0	R/W	DAC cell OE control » 0: Normal « » 1: DAOUT L level output«
10	DACDC	0	R/W	DAC DC output mode » 0: Normal « » 1: DC output mode « Note: Setting "1" converts the value written in the DALVL register to DAC and directly outputs from DAOUT
9:0	DALVL	0	R/W	DC level control

13.9.55 SCPROG

Sub-carrier programming Register

SCPROG	0003:0866			offset: 0x66		default: 0x0000										
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	SCPRG	RSV	RSV	SCSD[9]	SCSD[8]	SCSD[7]	SCSD[6]	SCSD[5]	SCSD[4]	SCSD[3]	SCSD[2]	SCSD[1]	SCSD[0]
R/W	-	-	-	R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	0	-	-	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:13	RSV			Reserved
12	SCPRG	0	R/W	Sub-carrier phase program mode. When 1, the initial phase degree of sub-carrier can be programmed to the specified value in SCSD register. The sub-carrier phase is immediately updated when CPU writes this register. When 0, internal preset value is used for sub-carrier phase » 0: Fixed by Hardware « » 1: Program mode «
11:10	RSV			Reserved
9:0	SCSD	0	R/W	Sub-carrier initial phase value The degree can be specified by SCSD/1024*360.

14 General Purpose Input Output (GIO)

14.1 Introduction

DM320 has 41 General-Purpose I/O ports (GIO0-GIO40). GIO ports serve two purposes. It provides users a general-purpose programmable interface that provides a convenient way of performing simple control / data transfer to external devices. The other purpose is to provide additional programmable interface signals to external / internal devices. With these additional signals, DM320 is able to increase the performance and the functionality of the overall system. Since the GIO ports can't perform both functions at the same time, it is up to the user to trade off between general-purpose programming I/Os and programmable feature for performance enhanced functions.

14.2 GIO Port State Control

This section describes the registers that enable the GIO ports to be used as regular I/O ports, i.e. how to read the current state and how to set the state of each GIO port.

The port direction of each GIO can be set in the *DIR0*, *DIR1* and *DIR2*. The direction can be set as input or output. To allow an external device to drive a particular GIO port, set the port direction as input. The port is in a high impedance state when set as input, and then an external device can drive the port. To allow a particular GIO port to drive an external device, set the port direction as output.

The GIO Bit Set registers *BITSET0*, *BITSET1* and *BITSET2* are used to set any of the 41 bits configured as an output. The GIO Bit Clear registers *BITCLR0*, *BITCLR1* and *BITCLR2* are used to clear the state of any of the 41 GIO ports configured as an output. The GIO Bit Set registers or GIO Bit Clear registers can be used to read the status of the GIO ports, irrespective of whether they are in input or output mode.

GIO chatter control registers allow for de-bouncing of the external input signals. Chatter control for each GIO port can be enabled or disabled using registers *CHAT0*, *CHAT1* and *CHAT2*. User should set the chatter width (NCH) in *NCHAT*. When chatter is enabled, the GIO port imports the input data only if the input is stable for more than (NCH+1) x AHB clocks.

When the GIO port is set in the input mode, the bit can be inverted while being read. The GIO Inverse registers *INV0*, *INV1* and *INV2* allows any of the 41 GIO ports to be inverted. When the inverse bit corresponding to a GIO port is set, the value read in the *BITSETx* or *BITCLRx* register, will be inverse of the actual level at the pin.

Note1: Write '1' at the corresponding position to set a GIO in *BITSETx* register or to clear a GIO in *BITCLRx* register.

Note2: GIO Inverse settings are only applicable if the GIO port is configured as an input port.

14.3 External Interrupt Settings

Sixteen GIOs (GIO0 - GIO15) can be used as external ARM interrupt ports. These interrupt signals are supplied to the interrupt controller. Please refer to the section on the Interrupt Controller for information on how to configure the interrupt controller when using these interrupts.

To use a particular GIO port as an external interrupt, set the appropriate bit in register *IRQPORT* and set the GIO port direction to input. To trigger the interrupt on the rising edge of the external input signal, set the appropriate bit in the GIO Inversion register to 1. When a GIO port's inversion bit is set to 0, the interrupt will trigger on the falling edge of the signal. It is also possible to trigger on any edge of the external signal by setting the corresponding bit in *IRQEDGE*.

When using the GIO ports for external interrupts, the width of external signal should be at least 1 AHB clock cycle.

Even when GIO ports are configured as external interrupts, the corresponding bits in the Bit Set and Bit Clear registers are updated to reflect the port status.

14.4 GIO Functional Settings

GIO ports (GIO0-GIO40) can also be used as programmable function ports instead of using them as general-purpose input/output ports. Dedicated ports can be selected by setting the corresponding bits in the *FSEL0*, *FSEL1*, *FSEL2* and *FSEL3* registers. GIO ports and signals can be configured as shown in *Table 42*.

GIO pins resets to non-inverted, general-purpose input mode after a CPU reset.

Table 42: GIO port selection output

GIO Port Number	Functionality		FSEL register	Input / Output	Comments
	Value	Alternate functionality			
GIO0	00	GIO	FSEL0[1:0]	I/O	Normal GIO / External Interrupt
	01	GIO	FSEL0[1:0]	I/O	Normal GIO
	10	RSV	FSEL0[1:0]	NA	Reserved
	11	RSV	FSEL0[1:0]	NA	Reserved
GIO1	00	GIO	FSEL0[1:0]	I/O	Normal GIO / External Interrupt
	01	GIO	FSEL0[1:0]	I/O	Normal GIO
	10	RSV	FSEL0[1:0]	NA	Reserved
	11	RSV	FSEL0[1:0]	NA	Reserved
GIO2	00	GIO	FSEL0[1:0]	I/O	Normal GIO / External Interrupt
	01	GIO	FSEL0[1:0]	I/O	Normal GIO
	10	RSV	FSEL0[1:0]	NA	Reserved
	11	RSV	FSEL0[1:0]	NA	Reserved
GIO3	00	GIO	FSEL0[1:0]	I/O	Normal GIO / External Interrupt
	01	GIO	FSEL0[1:0]	I/O	Normal GIO
	10	RSV	FSEL0[1:0]	NA	Reserved
	11	RSV	FSEL0[1:0]	NA	Reserved
GIO4	00	GIO	FSEL0[1:0]	I/O	Normal GIO / External Interrupt
	01	GIO	FSEL0[1:0]	I/O	Normal GIO
	10	RSV	FSEL0[1:0]	NA	Reserved
	11	RSV	FSEL0[1:0]	NA	Reserved
GIO5	00	GIO	FSEL0[1:0]	I/O	Normal GIO / External Interrupt
	01	GIO	FSEL0[1:0]	I/O	Normal GIO
	10	RSV	FSEL0[1:0]	NA	Reserved

	11	GIO	FSEL0[1:0]	NA	Reserved
GIO6	00	GIO	FSEL0[1:0]	I/O	Normal GIO / External Interrupt
	01	GIO	FSEL0[1:0]	I/O	Normal GIO
	10	RSV	FSEL0[1:0]	NA	Reserved
	11	RSV	FSEL0[1:0]	NA	Reserved
GIO7	00	GIO	FSEL0[1:0]	I/O	Normal GIO / External Interrupt
	01	GIO	FSEL0[1:0]	I/O	Normal GIO
	10	RSV	FSEL0[1:0]	NA	Reserved
	11	GIO	FSEL0[1:0]	NA	Reserved
GIO8	00	GIO	FSEL0[1:0]	I/O	Normal GIO / External Interrupt
	01	GIO	FSEL0[1:0]	I/O	Normal GIO
	10	RSV	FSEL0[1:0]	NA	Reserved
	11	RSV	FSEL0[1:0]	NA	Reserved
GIO9	00	GIO	FSEL0[1:0]	I/O	Normal GIO / External Interrupt
	01	DSPXF	FSEL0[1:0]	O	DSP XF Signal
	10	RSV	FSEL0[1:0]	NA	Reserved
	11	RSV	FSEL0[1:0]	NA	Reserved
GIO10	00	GIO	FSEL0[3:2]	I/O	Normal GIO / External Interrupt
	01	DSP_BDX1	FSEL0[3:2]	O	McBSP-1 BDX signal
	10	RSV	FSEL0[3:2]	NA	Reserved
	11	RSV	FSEL0[3:2]	NA	Reserved
GIO11	00	GIO	FSEL0[5:4]	I/O	Normal GIO / External Interrupt
	01	DSP_CLKX1	FSEL0[5:4]	I/O	McBSP-1 BCLKX signal
	10	RSV	FSEL0[5:4]	NA	Reserved
	11	RSV	FSEL0[5:4]	NA	Reserved
GIO12	00	GIO	FSEL0[7:6]	I/O	Normal GIO / External Interrupt
	01	DSP_BFSX1	FSEL0[7:6]	I/O	McBSP-1 BFSX signal

	10	RSV	FSEL0[7:6]	NA	Reserved
	11	RSV	FSEL0[7:6]	NA	Reserved
GIO13	00	GIO	FSEL0[9:8]	I/O	Normal GIO / External Interrupt
	01	DSP_BDR1	FSEL0[9:8]	I	McBSP-1 BDR signal
	10	RSV	FSEL0[9:8]	NA	Reserved
	11	RSV	FSEL0[9:8]	NA	Reserved
GIO14	00	GIO	FSEL0[11:10]	I/O	Normal GIO / External Interrupt
	01	DSP_BCLKR1	FSEL0[11:10]	I/O	McBSP 1 BCLKR signal
	10	RSV	FSEL0[11:10]	NA	Reserved
	11	RSV	FSEL0[11:10]	NA	Reserved
GIO15	00	GIO	FSEL0[13:12]	I/O	Normal GIO / External Interrupt
	01	DSP_BFSR1	FSEL0[13:12]	I/O	McBSP-1 BFSR signal
	10	RSV	FSEL0[13:12]	NA	Reserved
	11	RSV	FSEL0[13:12]	NA	Reserved
GIO16	00	GIO	FSEL0[15:14]	I/O	Normal GIO / External Interrupt
	01	DSP_BCLKS1	FSEL0[15:14]	I	McBSP-1 BCLKS signal
	10	RSV	FSEL0[15:14]	NA	Reserved
	11	CLKOUT0	FSEL0[15:14]	O	Clock out 0 –12 MHz
GIO17	00	GIO	FSEL1[1:0]	I/O	Normal GIO
	01	VBUSCHARGE	FSEL1[1:0]	O	USB VBUSCHARGE signal
	10	VLYNQ_RXD0	FSEL1[1:0]	I	VLYNQ RXD0 signal
	11	GIO	FSEL1[1:0]	I/O	Normal GIO
GIO18	00	GIO	FSEL1[3:2]	I/O	Normal GIO
	01	VBUSEN	FSEL1[3:2]	O	USB VBUSEN signal
	10	VLYNQ_RXD1	FSEL1[3:2]	I	VLYNQ RXD1 signal
	11	GIO	FSEL1[3:2]	I/O	Normal GIO
GIO19	00	GIO	FSEL1[5:4]	I/O	Normal GIO

	01	VBUSVAL	FSEL1[5:4]	I	USB VBUSVAL signal
	10	VLYNQ_RXD2	FSEL1[5:4]	I	VLYNQ RXD2 signal
	11	GIO	FSEL1[5:4]	I/O	Normal GIO
GIO20	00	GIO	FSEL1[7:6]	I/O	Normal GIO
	01	VBUSSES	FSEL1[7:6]	I	USB VBUSSES signal
	10	VLYNQ_RXD3	FSEL1[7:6]	I	VLYNQ RXD3 signal
	11	GIO	FSEL1[7:6]	I/O	Normal GIO
GIO21	00	GIO	FSEL1[9:8]	I/O	Normal GIO
	01	VBUSLO	FSEL1[9:8]	I	USB VBUSLO signal
	10	VLYNQ_TXD0	FSEL1[9:8]	O	VLYNQ TXD0 signal
	11	GIO	FSEL1[9:8]	I/O	Normal GIO
GIO22	00	GIO	FSEL1[11:10]	I/O	Normal GIO
	01	CID	FSEL1[11:10]	I	USB CID signal
	10	VLYNQ_TXD1	FSEL1[11:10]	O	VLYNQ TXD1 signal
	11	GIO	FSEL1[11:10]	I/O	Normal GIO
GIO23	00	GIO	FSEL1[13:12]	I/O	Normal GIO
	01	PUCON	FSEL1[13:12]	O	USB PUCON signal
	10	VLYNQ_TXD2	FSEL1[13:12]	O	VLYNQ TXD2 signal
	11	GIO	FSEL1[13:12]	I/O	Normal GIO
GIO24	00	GIO	FSEL1[15:14]	I/O	Normal GIO
	01	PDCON	FSEL1[15:14]	O	USB PDCON signal
	10	VLYNQ_TXD3	FSEL1[15:14]	O	VLYNQ TXD3 signal
	11	GIO	FSEL1[15:14]	I/O	Normal GIO
GIO25	00	GIO	FSEL2[1:0]	I/O	Normal GIO
	01	IDEN	FSEL2[1:0]	O	USB IDEN signal
	10	VLYNQ_SCRUN	FSEL2[1:0]	I/O	VLYNQ SCRUN signal
	11	CLKOUT1A	FSEL2[1:0]	O	Clock out 1A
GIO26	00	GIO	FSEL2[3:2]	I/O	Normal GIO
	01	GIO	FSEL2[3:2]	I/O	Normal GIO
	10	VLYNQ_CLOCK	FSEL2[3:2]	I/O	VLYNQ CLOCK signal

	11	CLKOUT2A	FSEL2[3:2]	O	Clock out 2A
GIO27	00	GIO	FSEL2[5:4]	I/O	Normal GIO
	01	UART_RXD1	FSEL2[5:4]	I	UART-1 RXD signal
	10	RSV	FSEL2[5:4]	NA	Reserved
	11	RSV	FSEL2[5:4]	NA	Reserved
GIO28	00	GIO	FSEL2[7:6]	I/O	Normal GIO
	01	UART_TXD1	FSEL2[7:6]	O	UART-1 TXD signal
	10	RSV	FSEL2[7:6]	NA	Reserved
	11	RSV	FSEL2[7:6]	NA	Reserved
GIO29	00	GIO	FSEL2[9:8]	I/O	Normal GIO
	01	GIO	FSEL2[9:8]	I/O	Normal GIO
	10	SIF_EN	FSEL2[9:8]	I/O	SIF Enable
	11	RSV	FSEL2[9:8]	NA	Reserved
GIO30	00	GIO	FSEL2[11:10]	I/O	Normal GIO
	01	I2C_CLK	FSEL2[11:10]	O	I2C Clock
	10	SIF_CLK	FSEL2[11:10]	I/O	SIF Clock
	11	RSV	FSEL2[11:10]	NA	Reserved
GIO31	00	GIO	FSEL2[13:12]	I/O	Normal GIO
	01	I2C_DATA	FSEL2[13:12]	I/O	I2C Data
	10	SIF_DI	FSEL2[13:12]	I	SIF Data In
	11	RSV	FSEL2[13:12]	NA	Reserved
GIO32	00	GIO	FSEL2[15:14]	I/O	Normal GIO
	01	GIO	FSEL2[15:14]	I/O	Normal GIO
	10	SIF_DO	FSEL2[15:14]	O	SIF Data Out
	11	RSV	FSEL2[15:14]	NA	Reserved
GIO33	00	GIO	FSEL3[1:0]	I/O	Normal GIO
	01	PWM0	FSEL3[1:0]	O	PWM0 signal
	10	RSV	FSEL3[1:0]	NA	Reserved
	11	CLKOUT1B	FSEL3[1:0]	O	Clock Out 1B
GIO34	00	GIO	FSEL3[3:2]	I/O	Normal GIO

	01	PWM1	FSEL3[3:2]	O	PWM1 signal
	10	R2	FSEL3[3:2]	O	R2 signal
	11	CLKOUT2B	FSEL3[3:2]	O	Clock Out 2B
GIO35	00	GIO	FSEL3[5:4]	I/O	Normal GIO
	01	TMRC0	FSEL3[5:4]	O	CCD Timer 0 output
	10	RSV	FSEL3[5:4]	NA	Reserved
	11	RSV	FSEL3[5:4]	NA	Reserved
GIO36	00	GIO	FSEL3[7:6]	I/O	Normal GIO
	01	TMRC1	FSEL3[7:6]	O	CCD Timer 1 output
	10	RSV	FSEL3[7:6]	NA	Reserved
	11	RSV	FSEL3[7:6]	NA	Reserved
GIO37	00	GIO	FSEL3[9:8]	I/O	Normal GIO
	01	MIRQ	FSEL3[9:8]	O	MIRQ pin
	10	RSV	FSEL3[9:8]	NA	Reserved
	11	MRST	FSEL3[9:8]	O	MRST signal
GIO38	00	GIO	FSEL3[11:10]	I/O	Normal GIO
	01	EXT_LINE_ID	FSEL3[11:10]	I	External Line ID
	10	B2	FSEL3[11:10]	O	B2 signal
	11	RSV	FSEL3[11:10]	NA	Reserved
GIO39	00	GIO	FSEL3[13:12]	I/O	Normal GIO
	01	FIELD_VENC	FSEL3[13:12]	O	Field VENC signal
	10	RSV	FSEL3[13:12]	NA	Reserved
	11	RSV	FSEL3[13:12]	NA	Reserved
GIO40	00	GIO	FSEL3[15:14]	I/O	Normal GIO
	01	C_WEN	FSEL3[15:14]	I	C_WEN signal
	10	RSV	FSEL3[15:14]	NA	Reserved
	11	RSV	FSEL3[15:14]	NA	Reserved

ETM_CLK bit in FSEL4 register is used to invert the settings of the ETM_TRACE_CLK signal.

14.5 Memory Card detection

Memory card detection in DM320 has two functions:

- o Generate an interrupt when a card is plugged in / pulled out
- o Wake-up DM320, if a card is plugged in / pulled out during stand-by state

GIO8 can be used as the card detect signal. When a card is plugged in the value of GIO8 is high and when a card is pulled out the value is low. GIO0 is used as the wake-up signal. The timing diagram when a card is inserted or removed during standby is given in Figure 105.

Steps for card detection:

- o Enable card detection and select appropriate polarity in CARD_SET register
- o Select wakeup features in clock controller registers for GIO0, refer clock controller for more details.
- o Read the status from CARD_ST register or configure GIO8 to generate an interrupt.

Note: Card detect signal from the memory card connector should be connected to GIO8 for this functionality. GIO8 should be set in Normal GPIO / External interrupt mode in FSEL0 register.

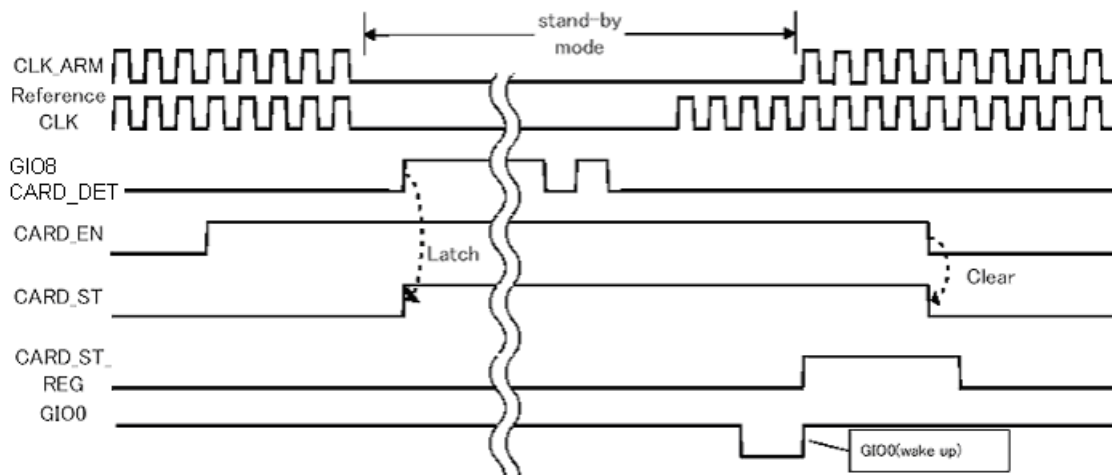


Figure 105: DM320 card detection timing diagram

14.6 General I/O Register Map (GIO)

Address	Register	Description
0003 0580	DR0	GIO Direction Register 0
0003 0582	DR1	GIO Direction Register 1
0003 0584	DR2	GIO Direction Register 2
0003 0586	INV0	GIO Inversion Register 0
0003 0588	INV1	GIO Inversion Register 1
0003 058A	INV2	GIO Inversion Register 2
0003 058C	BITSET0	GIO Bit Set Register 0
0003 058E	BITSET1	GIO Bit Set Register 1
0003 0590	BITSET2	GIO Bit Set Register 2
0003 0592	BITCLR0	GIO Bit Clear Register 0
0003 0594	BITCLR1	GIO Bit Clear Register 1
0003 0596	BITCLR2	GIO Bit Clear Register 2
0003 0598	RQPORT	GIO IRQ Port Setting Register
0003 059A	RQEDGE	GIO IRQ Edge Setting Register
0003 059C	CHAT0	GIO Chatter Setting Register 0
0003 059E	CHAT1	GIO Chatter Setting Register 1
0003 05A0	CHAT2	GIO Chatter Setting Register 2
0003 05A2	NCHAT	GIO Chatter Value Register
0003 05A4	FSEL0	GIO Function Select Register 0
0003 05A6	FSEL1	GIO Function Select Register 1
0003 05A8	FSEL2	GIO Function Select Register 2
0003 05AA	FSEL3	GIO Function Select Register 3
0003 05AC	FSEL4	Function Select register 4
0003 05AE	CARD_SET	Set Card features
0003 05B0	CARD_ST	Card status

14.7 General I/O Registers

14.7.1 DIR0

Bit direction register #0

DIR0	0003:0580			offset: 0x00			default: 0xFFFF									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	GIO15DIR	GIO14DIR	GIO13DIR	GIO12DIR	GIO11DIR	GIO10DIR	GIO09DIR	GIO08DIR	GIO07DIR	GIO06DIR	GIO05DIR	GIO04DIR	GIO03DIR	GIO02DIR	GIO01DIR	GIO00DIR
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

Bit	Name	Reset Value	R/W	Function
15	GD15DR	1	R/W	GIO15 Port Direction; » 0: output « » 1: input «
14	GD14DR	1	R/W	GIO14 Port Direction; » 0: output « » 1: input «
13	GD13DR	1	R/W	GIO13 Port Direction; » 0: output « » 1: input «
12	GD12DR	1	R/W	GIO12 Port Direction; » 0: output « » 1: input «
11	GD11DR	1	R/W	GIO11 Port Direction; » 0: output « » 1: input «
10	GD10DR	1	R/W	GIO10 Port Direction; » 0: output « » 1: input «
9	GD09DR	1	R/W	GIO09 Port Direction; » 0: output « » 1: input «
8	GD08DR	1	R/W	GIO08 Port Direction; » 0: output « » 1: input «
7	GD07DR	1	R/W	GIO07 Port Direction; » 0: output « » 1: input «
6	GD06DR	1	R/W	GIO06 Port Direction; » 0: output « » 1: input «
5	GD05DR	1	R/W	GIO05 Port Direction; » 0: output « » 1: input «
4	GD04DR	1	R/W	GIO04 Port Direction; » 0: output « » 1: input «
3	GD03DR	1	R/W	GIO03 Port Direction; » 0: output « » 1: input «
2	GD02DR	1	R/W	GIO02 Port Direction; » 0: output « » 1: input «
1	GD01DR	1	R/W	GIO01 Port Direction; » 0: output « » 1: input «
0	GD00DR	1	R/W	GIO00 Port Direction; » 0: output « » 1: input «

14.7.2 DIR1

Bit direction register #1

DIR1	0003:0582			offset: 0x02												default: 0xFFFF	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	GIO31DIR	GIO30DIR	GIO29DIR	GIO28DIR	GIO27DIR	GIO26DIR	GIO25DIR	GIO24DIR	GIO23DIR	GIO22DIR	GIO21DIR	GIO20DIR	GIO19DIR	GIO18DIR	GIO17DIR	GIO16DIR	
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	

Bit	Name	Reset Value	R/W	Function
15	GD31DR	1	R/W	GIO31 Port Direction; » 0: output « » 1: input «
14	GD30DR	1	R/W	GIO30 Port Direction; » 0: output « » 1: input «
13	GD29DR	1	R/W	GIO29 Port Direction; » 0: output « » 1: input «
12	GD28DR	1	R/W	GIO28 Port Direction; » 0: output « » 1: input «
11	GD27DR	1	R/W	GIO27 Port Direction; » 0: output « » 1: input «
10	GD26DR	1	R/W	GIO26 Port Direction; » 0: output « » 1: input «
9	GD25DR	1	R/W	GIO25 Port Direction; » 0: output « » 1: input «
8	GD24DR	1	R/W	GIO24 Port Direction; » 0: output « » 1: input «
7	GD23DR	1	R/W	GIO23 Port Direction; » 0: output « » 1: input «
6	GD22DR	1	R/W	GIO22 Port Direction; » 0: output « » 1: input «
5	GD21DR	1	R/W	GIO21 Port Direction; » 0: output « » 1: input «
4	GD20DR	1	R/W	GIO20 Port Direction; » 0: output « » 1: input «
3	GD19DR	1	R/W	GIO19 Port Direction; » 0: output « » 1: input «
2	GD18DR	1	R/W	GIO18 Port Direction; » 0: output « » 1: input «
1	GD17DR	1	R/W	GIO17 Port Direction; » 0: output « » 1: input «
0	GD16DR	1	R/W	GIO16 Port Direction; » 0: output « » 1: input «

14.7.3 DIR2

Bit direction register #2

DIR2	0003:0584			offset: 0x04			default: 0x01FF									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	GIO40DIR	GIO39DIR	GIO38DIR	GIO37DIR	GIO36DIR	GIO35DIR	GIO34DIR	GIO33DIR	GIO32DIR
R/W	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	-	-	1	1	1	1	1	1	1	1	1

Bit	Name	Reset Value	R/W	Function
159	RSV			Reserved
8	GD40DIR	1	R/W	GIO40 Port Direction; » 0: output « » 1: input «
7	GD39DIR	1	R/W	GIO39 Port Direction; » 0: output « » 1: input «
6	GD38DIR	1	R/W	GIO38 Port Direction; » 0: output « » 1: input «
5	GD37DIR	1	R/W	GIO37 Port Direction; » 0: output « » 1: input «
4	GD36DIR	1	R/W	GIO36 Port Direction; » 0: output « » 1: input «
3	GD35DIR	1	R/W	GIO35 Port Direction; » 0: output « » 1: input «
2	GD34DIR	1	R/W	GIO34 Port Direction; » 0: output « » 1: input «
1	GD33DIR	1	R/W	GIO33 Port Direction; » 0: output « » 1: input «
0	GD32DIR	1	R/W	GIO32 Port Direction; » 0: output « » 1: input «

14.7.4 INV0

Bit Inversion register #0

INV0	0003:0586			offset: 0x06			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	GIO15INV	GIO14INV	GIO13INV	GIO12INV	GIO11INV	GIO10INV	GIO09INV	GIO08INV	GIO07INV	GIO06INV	GIO05INV	GIO04INV	GIO03INV	GIO02INV	GIO01INV	GIO00INV
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15	GD15INV	0	R/W	GIO15 Port Inversion; » 0: normal « » 1: inverted «
14	GD14INV	0	R/W	GIO14 Port Inversion; » 0: normal « » 1: inverted «
13	GD13INV	0	R/W	GIO13 Port Inversion; » 0: normal « » 1: inverted «
12	GD12INV	0	R/W	GIO12 Port Inversion; » 0: normal « » 1: inverted «
11	GD11INV	0	R/W	GIO11 Port Inversion; » 0: normal « » 1: inverted «
10	GD10INV	0	R/W	GIO10 Port Inversion; » 0: normal « » 1: inverted «
9	GD09INV	0	R/W	GIO09 Port Inversion; » 0: normal « » 1: inverted «
8	GD08INV	0	R/W	GIO08 Port Inversion; » 0: normal « » 1: inverted «
7	GD07INV	0	R/W	GIO07 Port Inversion; » 0: normal « » 1: inverted «
6	GD06INV	0	R/W	GIO06 Port Inversion; » 0: normal « » 1: inverted «
5	GD05INV	0	R/W	GIO05 Port Inversion; » 0: normal « » 1: inverted «
4	GD04INV	0	R/W	GIO04 Port Inversion; » 0: normal « » 1: inverted «
3	GD03INV	0	R/W	GIO03 Port Inversion; » 0: normal « » 1: inverted «
2	GD02INV	0	R/W	GIO02 Port Inversion; » 0: normal « » 1: inverted «
1	GD01INV	0	R/W	GIO01 Port Inversion; » 0: normal « » 1: inverted «
0	GD00INV	0	R/W	GIO00 Port Inversion; » 0: normal « » 1: inverted «

14.7.5 INV1

Bit Inversion register #1

INV1	0003:0588			offset: 0x08			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	GIO31INV	GIO30INV	GIO29INV	GIO28INV	GIO27INV	GIO26INV	GIO25INV	GIO24INV	GIO23INV	GIO22INV	GIO21INV	GIO20INV	GIO19INV	GIO18INV	GIO17INV	GIO16INV
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15	GD31INV	0	R/W	GIO31 Port Inversion; » 0: normal « » 1: inverted «
14	GD30INV	0	R/W	GIO30 Port Inversion; » 0: normal « » 1: inverted «
13	GD29INV	0	R/W	GIO29 Port Inversion; » 0: normal « » 1: inverted «
12	GD28INV	0	R/W	GIO28 Port Inversion; » 0: normal « » 1: inverted «
11	GD27INV	0	R/W	GIO27 Port Inversion; » 0: normal « » 1: inverted «
10	GD26INV	0	R/W	GIO26 Port Inversion; » 0: normal « » 1: inverted «
9	GD25INV	0	R/W	GIO25 Port Inversion; » 0: normal « » 1: inverted «
8	GD24INV	0	R/W	GIO24 Port Inversion; » 0: normal « » 1: inverted «
7	GD23INV	0	R/W	GIO24 Port Inversion; » 0: normal « » 1: inverted «
6	GD22INV	0	R/W	GIO24 Port Inversion; » 0: normal « » 1: inverted «
5	GD21INV	0	R/W	GIO24 Port Inversion; » 0: normal « » 1: inverted «
4	GD20INV	0	R/W	GIO24 Port Inversion; » 0: normal « » 1: inverted «
3	GD19INV	0	R/W	GIO24 Port Inversion; » 0: normal « » 1: inverted «
2	GD18INV	0	R/W	GIO24 Port Inversion; » 0: normal « » 1: inverted «
1	GD17INV	0	R/W	GIO24 Port Inversion; » 0: normal « » 1: inverted «
0	GD16INV	0	R/W	GIO24 Port Inversion; » 0: normal « » 1: inverted «

14.7.6 INV2

Bit Inversion register #2

INV2	0003:058A				offset: 0x0A				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	GIO40INV	GIO39INV	GIO38INV	GIO37INV	GIO36INV	GIO35INV	GIO34INV	GIO33INV	GIO32INV
R/W	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	-	-	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
152	RSV			Reserved
8	GD40INV	0	R/W	GI024 Port Inversion; » 0: normal « » 1: inverted «
7	GD39INV	0	R/W	GI024 Port Inversion; » 0: normal « » 1: inverted «
6	GD38INV	0	R/W	GI024 Port Inversion; » 0: normal « » 1: inverted «
5	GD37INV	0	R/W	GI024 Port Inversion; » 0: normal « » 1: inverted «
4	GD36INV	0	R/W	GI024 Port Inversion; » 0: normal « » 1: inverted «
3	GD35INV	0	R/W	GI024 Port Inversion; » 0: normal « » 1: inverted «
2	GD34INV	0	R/W	GI024 Port Inversion; » 0: normal « » 1: inverted «
1	GD33INV	0	R/W	GI024 Port Inversion; » 0: normal « » 1: inverted «
0	GD32INV	0	R/W	GI024 Port Inversion; » 0: normal « » 1: inverted «

14.7.7 BITSET0

Bit set register #0

BITSET0	0003:058C			offset: 0x0C			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	GIO15SET	GIO14SET	GIO13SET	GIO12SET	GIO11SET	GIO10SET	GIO09SET	GIO08SET	GIO07SET	GIO06SET	GIO05SET	GIO04SET	GIO03SET	GIO02SET	GIO01SET	GIO00SET
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15	GD15SET	0	R/W	GIO15 Bit Set Wr 0: no effect Wr 1: DIR=IN, no effect DIR=OUT & INV=NORMAL, set HIGH, DIR=OUT & INV=INVERT, set LOW Rd : read current state, using INV
14	GD14SET	0	R/W	GIO14 Bit Set
13	GD13SET	0	R/W	GIO13 Bit Set
12	GD12SET	0	R/W	GIO12 Bit Set
11	GD11SET	0	R/W	GIO11 Bit Set
10	GD10SET	0	R/W	GIO10 Bit Set
9	GD09SET	0	R/W	GIO09 Bit Set
8	GD08SET	0	R/W	GIO08 Bit Set
7	GD07SET	0	R/W	GIO07 Bit Set
6	GD06SET	0	R/W	GIO06 Bit Set
5	GD05SET	0	R/W	GIO05 Bit Set
4	GD04SET	0	R/W	GIO04 Bit Set
3	GD03SET	0	R/W	GIO03 Bit Set
2	GD02SET	0	R/W	GIO02 Bit Set
1	GD01SET	0	R/W	GIO01 Bit Set
0	GD00SET	0	R/W	GIO00 Bit Set

14.7.8 BITSET1

Bit set register #1

BITSET1		0003:058E			offset: 0x0E			default: 0x0000								
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	GIO31SET	GIO30SET	GIO29SET	GIO28SET	GIO27SET	GIO26SET	GIO25SET	GIO24SET	GIO23SET	GIO22SET	GIO21SET	GIO20SET	GIO19SET	GIO18SET	GIO17SET	GIO16SET
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15	GD31SET	0	R/W	GIO31 Bit Set Wr 0: no effect Wr 1: DIR=IN, no effect DIR=OUT & INV=NORMAL, set HIGH, DIR=OUT & INV=INVERT, set LOW Rd : read current state, using INV
14	GD30SET	0	R/W	GIO30 Bit Set
13	GD29SET	0	R/W	GIO29 Bit Set
12	GD28SET	0	R/W	GIO28 Bit Set
11	GD27SET	0	R/W	GIO27 Bit Set
10	GD26SET	0	R/W	GIO26 Bit Set
9	GD25SET	0	R/W	GIO25 Bit Set
8	GD24SET	0	R/W	GIO24 Bit Set
7	GD23SET	0	R/W	GIO23 Bit Set
6	GD22SET	0	R/W	GIO22 Bit Set
5	GD21SET	0	R/W	GIO21 Bit Set
4	GD20SET	0	R/W	GIO20 Bit Set
3	GD19SET	0	R/W	GIO19 Bit Set
2	GD18SET	0	R/W	GIO18 Bit Set
1	GD17SET	0	R/W	GIO17 Bit Set
0	GD16SET	0	R/W	GIO16 Bit Set

14.7.9 BITSET2

Bit set register #2

BITSET2	0003:0590			offset: 0x10			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	GIO40SET	GIO39SET	GIO38SET	GIO37SET	GIO36SET	GIO35SET	GIO34SET	GIO33SET	GIO32SET
R/W	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	-	-	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:9	RSV			Reserved
8	GD40SET	0	R/W	GIO40 Bit Set
7	GD39SET	0	R/W	GIO39 Bit Set
6	GD38SET	0	R/W	GIO38 Bit Set
5	GD37SET	0	R/W	GIO37 Bit Set
4	GD36SET	0	R/W	GIO36 Bit Set
3	GD35SET	0	R/W	GIO35 Bit Set
2	GD34SET	0	R/W	GIO34 Bit Set
1	GD33SET	0	R/W	GIO33 Bit Set
0	GD32SET	0	R/W	GIO32 Bit Set

14.7.10 BITCLR0

Bit clear register #0

BITCLR0		0003:0592			offset: 0x12												default: 0x0000		
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	GIO15CLR	GIO14CLR	GIO13CLR	GIO12CLR	GIO11CLR	GIO10CLR	GIO09CLR	GIO08CLR	GIO07CLR	GIO06CLR	GIO05CLR	GIO04CLR	GIO03CLR	GIO02CLR	GIO01CLR	GIO00CLR			
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W			
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			

Bit	Name	Reset Value	R/W	Function
15	GD15CLR	0	R/W	GIO15 Bit Clr Wr 0: no effect Wr 1: DIR=IN, no effect DIR=OUT & (INV=NORMAL or INV=INVERT), set LOW Rd : DIR=IN, read current state, using INV DIR=OUT, read current state, without using INV
14	GD14CLR	0	R/W	GIO14 Bit Clr
13	GD13CLR	0	R/W	GIO13 Bit Clr
12	GD12CLR	0	R/W	GIO12 Bit Clr
11	GD11CLR	0	R/W	GIO11 Bit Clr
10	GD10CLR	0	R/W	GIO10 Bit Clr
9	GD09CLR	0	R/W	GIO09 Bit Clr
8	GD08CLR	0	R/W	GIO08 Bit Clr
7	GD07CLR	0	R/W	GIO07 Bit Clr
6	GD06CLR	0	R/W	GIO06 Bit Clr
5	GD05CLR	0	R/W	GIO05 Bit Clr
4	GD04CLR	0	R/W	GIO04 Bit Clr
3	GD03CLR	0	R/W	GIO03 Bit Clr
2	GD02CLR	0	R/W	GIO02 Bit Clr
1	GD01CLR	0	R/W	GIO01 Bit Clr
0	GD00CLR	0	R/W	GIO00 Bit Clr

14.7.11 BITCLR1

Bit clear register #1

BITCLR1	0003:0594			offset: 0x14			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	GIO31CLR	GIO30CLR	GIO29CLR	GIO28CLR	GIO27CLR	GIO26CLR	GIO25CLR	GIO24CLR	GIO23CLR	GIO22CLR	GIO21CLR	GIO20CLR	GIO19CLR	GIO18CLR	GIO17CLR	GIO16CLR
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15	GD31CLR	0	R/W	GIO31 Bit Clr Wr 0: no effect Wr 1: DIR=IN, no effect DIR=OUT & (INV=NORMAL or INV=INVERT), set LOW Rd : DIR=IN, read current state, using INV DIR=OUT, read current state, without using INV
14	GD30CLR	0	R/W	GIO30 Bit Clr
13	GD29CLR	0	R/W	GIO29 Bit Clr
12	GD28CLR	0	R/W	GIO28 Bit Clr
11	GD27CLR	0	R/W	GIO27 Bit Clr
10	GD26CLR	0	R/W	GIO26 Bit Clr
9	GD25CLR	0	R/W	GIO25 Bit Clr
8	GD24CLR	0	R/W	GIO24 Bit Clr
7	GD23CLR	0	R/W	GIO23 Bit Clr
6	GD22CLR	0	R/W	GIO22 Bit Clr
5	GD21CLR	0	R/W	GIO21 Bit Clr
4	GD20CLR	0	R/W	GIO20 Bit Clr
3	GD19CLR	0	R/W	GIO19 Bit Clr
2	GD18CLR	0	R/W	GIO18 Bit Clr
1	GD17CLR	0	R/W	GIO17 Bit Clr
0	GD16CLR	0	R/W	GIO16 Bit Clr

14.7.12 BITCLR2

Bit clear register #2

BITCLR2		0003:0596			offset: 0x16			default: 0x0000								
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	GIO40CLR	GIO39CLR	GIO38CLR	GIO37CLR	GIO36CLR	GIO35CLR	GIO34CLR	GIO33CLR	GIO32CLR
R/W	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	-	-	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
159	RSV			Reserved
8	GD40CLR	0	R/W	GIO40 Bit Clr
7	GD39CLR	0	R/W	GIO39 Bit Clr
6	GD38CLR	0	R/W	GIO38 Bit Clr
5	GD37CLR	0	R/W	GIO37 Bit Clr
4	GD36CLR	0	R/W	GIO36 Bit Clr
3	GD35CLR	0	R/W	GIO35 Bit Clr
2	GD34CLR	0	R/W	GIO34 Bit Clr
1	GD33CLR	0	R/W	GIO33 Bit Clr
0	GD32CLR	0	R/W	GIO32 Bit Clr

14.7.13 IRQPORT

IRQ port setting register

IRQPORT	0003:0598				offset: 0x18				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	GIRQ15	GIRQ14	GIRQ13	GIRQ12	GIRQ11	GIRQ10	GIRQ9	GIRQ8	GIRQ7	GIRQ6	GIRQ5	GIRQ4	GIRQ3	GIRQ2	GIRQ1	GIRQ0
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15	GIRQ15	0	R/W	Enable External IRQ; » 0: normal « » 1: external IRQ «
14	GIRQ14	0	R/W	Enable External IRQ; » 0: normal « » 1: external IRQ «
13	GIRQ13	0	R/W	Enable External IRQ; » 0: normal « » 1: external IRQ «
12	GIRQ12	0	R/W	Enable External IRQ; » 0: normal « » 1: external IRQ «
11	GIRQ11	0	R/W	Enable External IRQ; » 0: normal « » 1: external IRQ «
10	GIRQ10	0	R/W	Enable External IRQ; » 0: normal « » 1: external IRQ «
9	GIRQ9	0	R/W	Enable External IRQ; » 0: normal « » 1: external IRQ «
8	GIRQ8	0	R/W	Enable External IRQ; » 0: normal « » 1: external IRQ «
7	GIRQ7	0	R/W	Enable External IRQ; » 0: normal « » 1: external IRQ «
6	GIRQ6	0	R/W	Enable External IRQ; » 0: normal « » 1: external IRQ «
5	GIRQ5	0	R/W	Enable External IRQ; » 0: normal « » 1: external IRQ «
4	GIRQ4	0	R/W	Enable External IRQ; » 0: normal « » 1: external IRQ «
3	GIRQ3	0	R/W	Enable External IRQ; » 0: normal « » 1: external IRQ «
2	GIRQ2	0	R/W	Enable External IRQ; » 0: normal « » 1: external IRQ «
1	GIRQ1	0	R/W	Enable External IRQ; » 0: normal « » 1: external IRQ «
0	GIRQ0	0	R/W	Enable External IRQ; » 0: normal « » 1: external IRQ «

14.7.14 IRQEDGE

IRQ edge setting register

IRQEDGE	0003:059A				offset: 0x1A				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	GIO15IRE	GIO14IRE	GIO13IRE	GIO12IRE	GIO11IRE	GIO10IRE	GIO9IRE	GIO8IRE	GIO7IRE	GIO6IRE	GIO5IRE	GIO4IRE	GIO3IRE	GIO2IRE	GIO1IRE	GIO0IRE
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15	GD15RE	0	R/W	Edge detection » 0: falling (rising if INV is set) « » 1: any edge (falling and rising)«
14	GD14RE	0	R/W	Edge detection » 0: falling (rising if INV is set) « » 1: any edge (falling and rising)«
13	GD13RE	0	R/W	Edge detection » 0: falling (rising if INV is set) « » 1: any edge (falling and rising)«
12	GD12RE	0	R/W	Edge detection » 0: falling (rising if INV is set) « » 1: any edge (falling and rising)«
11	GD11RE	0	R/W	Edge detection » 0: falling (rising if INV is set) « » 1: any edge (falling and rising)«
10	GD10RE	0	R/W	Edge detection » 0: falling (rising if INV is set) « » 1: any edge (falling and rising)«
9	GD9RE	0	R/W	Edge detection » 0: falling (rising if INV is set) « » 1: any edge (falling and rising)«
8	GD8RE	0	R/W	Edge detection » 0: falling (rising if INV is set) « » 1: any edge (falling and rising)«
7	GD7RE	0	R/W	Edge detection » 0: falling (rising if INV is set) « » 1: any edge (falling and rising)«
6	GD6RE	0	R/W	Edge detection » 0: falling (rising if INV is set) « » 1: any edge (falling and rising)«
5	GD5RE	0	R/W	Edge detection » 0: falling (rising if INV is set) « » 1: any edge (falling and rising)«
4	GD4RE	0	R/W	Edge detection » 0: falling (rising if INV is set) « » 1: any edge (falling and rising)«
3	GD3RE	0	R/W	Edge detection » 0: falling (rising if INV is set) « » 1: any edge (falling and rising)«
2	GD2RE	0	R/W	Edge detection » 0: falling (rising if INV is set) « » 1: any edge (falling and rising)«
1	GD1RE	0	R/W	Edge detection » 0: falling (rising if INV is set) « » 1: any edge (falling and rising)«
0	GD0RE	0	R/W	Edge detection » 0: falling (rising if INV is set) « » 1: any edge (falling and rising)«

14.7.15 CHAT0

Chatter setting register #0

CHAT0	0003:059C				offset: 0x1C				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	GIO15CHA	GIO14CHA	GIO13CHA	GIO12CHA	GIO11CHA	GIO10CHA	GIO9CHA	GIO8CHA	GIO7CHA	GIO6CHA	GIO5CHA	GIO4CHA	GIO3CHA	GIO2CHA	GIO1CHA	GIO0CHA
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15	GD15CHA	0	R/W	0: Normal mode 1: Chattering invalid (time set in NCHAT register)
14	GD14CHA	0	R/W	0: Normal mode 1: Chattering invalid (time set in NCHAT register)
13	GD13CHA	0	R/W	0: Normal mode 1: Chattering invalid (time set in NCHAT register)
12	GD12CHA	0	R/W	0: Normal mode 1: Chattering invalid (time set in NCHAT register)
11	GD11CHA	0	R/W	0: Normal mode 1: Chattering invalid (time set in NCHAT register)
10	GD10CHA	0	R/W	0: Normal mode 1: Chattering invalid (time set in NCHAT register)
9	GD9CHA	0	R/W	0: Normal mode 1: Chattering invalid (time set in NCHAT register)
8	GD8CHA	0	R/W	0: Normal mode 1: Chattering invalid (time set in NCHAT register)
7	GD7CHA	0	R/W	0: Normal mode 1: Chattering invalid (time set in NCHAT register)
6	GD6CHA	0	R/W	0: Normal mode 1: Chattering invalid (time set in NCHAT register)
5	GD5CHA	0	R/W	0: Normal mode 1: Chattering invalid (time set in NCHAT register)
4	GD4CHA	0	R/W	0: Normal mode 1: Chattering invalid (time set in NCHAT register)
3	GD3CHA	0	R/W	0: Normal mode 1: Chattering invalid (time set in NCHAT register)
2	GD2CHA	0	R/W	0: Normal mode 1: Chattering invalid (time set in NCHAT register)
1	GD1CHA	0	R/W	0: Normal mode 1: Chattering invalid (time set in NCHAT register)
0	GD0CHA	0	R/W	0: Normal mode 1: Chattering invalid (time set in NCHAT register)

14.7.16 CHAT1

Chatter setting register #1

CHAT1	0003:059E				offset: 0x1E				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	GIO31CHA	GIO30CHA	GIO29CHA	GIO28CHA	GIO27CHA	GIO26CHA	GIO25CHA	GIO24CHA	GIO23CHA	GIO22CHA	GIO21CHA	GIO20CHA	GIO19CHA	GIO18CHA	GIO17CHA	GIO16CHA
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15	GD31CHA	0	R/W	0: Normal mode 1: Chattering invalid (time set in NCHAT register)
14	GD30CHA	0	R/W	0: Normal mode 1: Chattering invalid (time set in NCHAT register)
13	GD29CHA	0	R/W	0: Normal mode 1: Chattering invalid (time set in NCHAT register)
12	GD28CHA	0	R/W	0: Normal mode 1: Chattering invalid (time set in NCHAT register)
11	GD27CHA	0	R/W	0: Normal mode 1: Chattering invalid (time set in NCHAT register)
10	GD26CHA	0	R/W	0: Normal mode 1: Chattering invalid (time set in NCHAT register)
9	GD25CHA	0	R/W	0: Normal mode 1: Chattering invalid (time set in NCHAT register)
8	GD24CHA	0	R/W	0: Normal mode 1: Chattering invalid (time set in NCHAT register)
7	GD23CHA	0	R/W	0: Normal mode 1: Chattering invalid (time set in NCHAT register)
6	GD22CHA	0	R/W	0: Normal mode 1: Chattering invalid (time set in NCHAT register)
5	GD21CHA	0	R/W	0: Normal mode 1: Chattering invalid (time set in NCHAT register)
4	GD20CHA	0	R/W	0: Normal mode 1: Chattering invalid (time set in NCHAT register)
3	GD19CHA	0	R/W	0: Normal mode 1: Chattering invalid (time set in NCHAT register)
2	GD18CHA	0	R/W	0: Normal mode 1: Chattering invalid (time set in NCHAT register)
1	GD17CHA	0	R/W	0: Normal mode 1: Chattering invalid (time set in NCHAT register)
0	GD16CHA	0	R/W	0: Normal mode 1: Chattering invalid (time set in NCHAT register)

14.7.17 CHAT2

Chatter setting register #2

CHAT2	0003:05A0				offset: 0x20				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	GIO40CHA	GIO39CHA	GIO38CHA	GIO37CHA	GIO36CHA	GIO35CHA	GIO34CHA	GIO33CHA	GIO32CHA
R/W	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	-	-	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:9	RSV			Reserved
8	GD40CHA	0	R/W	0: Normal mode 1: Chattering invalid (time set in NCHAT register)
7	GD39CHA	0	R/W	0: Normal mode 1: Chattering invalid (time set in NCHAT register)
6	GD38CHA	0	R/W	0: Normal mode 1: Chattering invalid (time set in NCHAT register)
5	GD37CHA	0	R/W	0: Normal mode 1: Chattering invalid (time set in NCHAT register)
4	GD36CHA	0	R/W	0: Normal mode 1: Chattering invalid (time set in NCHAT register)
3	GD35CHA	0	R/W	0: Normal mode 1: Chattering invalid (time set in NCHAT register)
2	GD34CHA	0	R/W	0: Normal mode 1: Chattering invalid (time set in NCHAT register)
1	GD33CHA	0	R/W	0: Normal mode 1: Chattering invalid (time set in NCHAT register)
0	GD32CHA	0	R/W	0: Normal mode 1: Chattering invalid (time set in NCHAT register)

14.7.18 NCHAT

Chatter value register

NCHAT	0003:05A2				offset: 0x22				default: 0x0008							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	NCH[3]	NCH[2]	NCH[1]	NCH[0]
R/W	-	-	-	-	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	-	-	-	-	-	-	-	1	0	0	0

Bit	Name	Reset Value	R/W	Function
15:4	RSV			Reserved
3:0	NCH	8	R/W	Chattering invalid width

14.7.19 FSEL0

Function select register #0

FSEL0	0003:05A4				offset: 0x24				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	GIO16[1]	GIO16[0]	GIO15[1]	GIO15[0]	GIO14[1]	GIO14[0]	GIO13[1]	GIO13[0]	GIO12[1]	GIO12[0]	GIO11[1]	GIO11[0]	GIO10[1]	GIO10[0]	GIO0_9[1]	GIO0_9[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:14	GD16	0	R/W	<u>Pin multiplexing (GIO16)</u> » 0: Normal (GIO16) « » 1: DSP_BCLKS1 (GIO16) « » 2: Reserved « » 3: CLKOUT0 (GIO16) «
13:12	GD15	0	R/W	<u>Pin multiplexing (GIO15)</u> » 0: Normal (GIO15) « » 1: DSP_BFSR1 (GIO15) « » 2: Reserved « » 3: Reserved «
11:10	GD14	0	R/W	<u>Pin multiplexing (GIO14)</u> » 0: Normal (GIO14) « » 1: DSP_BCLKR1 (GIO14) « » 2: Reserved « » 3: Reserved «
9:8	GD13	0	R/W	<u>Pin multiplexing (GIO13)</u> » 0: Normal (GIO13) « » 1: DSP_BDR1 (GIO13) « » 2: Reserved « » 3: Reserved «
7:6	GD12	0	R/W	<u>Pin multiplexing (GIO12)</u> » 0: Normal (GIO12) « » 1: DSP_BFSX1 (GIO12) « » 2: Reserved « » 3: Reserved «
5:4	GD11	0	R/W	<u>Pin multiplexing (GIO11)</u> » 0: Normal (GIO11) « » 1: DSP_BCLKX1 (GIO11) « » 2: Reserved « » 3: Reserved «
3:2	GD10	0	R/W	<u>Pin multiplexing (GIO10)</u> » 0: Normal (GIO10) « » 1: DSP_BDX1 (GIO10) « » 2: Reserved « » 3: Reserved «
1:0	GD0_9	0	R/W	<u>Pin multiplexing (GIO0-9)</u> » 0: Normal (GIO0-9) « » 1: DSP_XF (GIO09) « » 2: Reserved « » 3: Reserved «

14.7.20 FSEL1

Function select register #1

FSEL1	0003:05A6				offset: 0x26				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	GIO24[1]	GIO24[0]	GIO23[1]	GIO23[0]	GIO22[1]	GIO22[0]	GIO21[1]	GIO21[0]	GIO20[1]	GIO20[0]	GIO19[1]	GIO19[0]	GIO18[1]	GIO18[0]	GIO17[1]	GIO17[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:14	GD24	0	R/W	<u>Pin multiplexing (GIO24)</u> » 0: Normal (GIO24) « » 1: PDCON (GIO24) « » 2: VLYNQ_TXD3 (GIO24) « » 3: Normal (GIO24) «
13:12	GD23	0	R/W	<u>Pin multiplexing (GIO23)</u> » 0: Normal (GIO23) « » 1: PUCON (GIO23) « » 2: VLYNQ_TXD2 (GIO23) « » 3: Normal (GIO23) «
11:10	GD22	0	R/W	<u>Pin multiplexing (GIO22)</u> » 0: Normal (GIO22) « » 1: CID (GIO22) « » 2: VLYNQ_TXD1 (GIO22) « » 3: Normal (GIO22) «
9:8	GD21	0	R/W	<u>Pin multiplexing (GIO21)</u> » 0: Normal (GIO21) « » 1: VBUSLO (GIO21) « » 2: VLYNQ_TXD0 (GIO21) « » 3: Normal (GIO21) «
7:6	GD20	0	R/W	<u>Pin multiplexing (GIO20)</u> » 0: Normal (GIO20) « » 1: VBUSSES (GIO20) « » 2: VLYNQ_RXD3 (GIO20) « » 3: Normal (GIO20) «
5:4	GD19	0	R/W	<u>Pin multiplexing (GIO19)</u> » 0: Normal (GIO19) « » 1: VBUSVAL (GIO19) « » 2: VLYNQ_RXD2 (GIO19) « » 3: Normal (GIO19) «
3:2	GD18	0	R/W	<u>Pin multiplexing (GIO18)</u> » 0: Normal (GIO18) « » 1: VBUSEN (GIO18) « » 2: VLYNQ_RXD1 (GIO18) « » 3: Normal (GIO18) «
1:0	GD17	0	R/W	<u>Pin multiplexing (GIO17)</u> » 0: Normal (GIO17) « » 1: VBUSCHARGE (GIO17) « » 2: VLYNQ_RXD0 (GIO17) « » 3: Normal (GIO17) «

14.7.21 FSEL2

Function select register #2

FSEL2	0003:05A8				offset: 0x28				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	GIO32[1]	GIO32[0]	GIO31[1]	GIO31[0]	GIO30[1]	GIO30[0]	GIO29[1]	GIO29[0]	GIO28[1]	GIO28[0]	GIO27[1]	GIO27[0]	GIO26[1]	GIO26[0]	GIO25[1]	GIO25[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:14	GD32	0	R/W	<u>Pin multiplexing (GIO32)</u> » 0: Normal (GIO32) « » 1: Normal (GIO32) « » 2: SIF_D0 (GIO32) « » 3: Reserved «
13:12	GD31	0	R/W	<u>Pin multiplexing (GIO31)</u> » 0: Normal (GIO31) « » 1: I2C_DATA (GIO31) « » 2: SIF_DI (GIO31) « » 3: Reserved «
11:10	GD30	0	R/W	<u>Pin multiplexing (GIO30)</u> » 0: Normal (GIO30) « » 1: I2C_CLK (GIO30) « » 2: SIF_CLK (GIO30) « » 3: Reserved «
9:8	GD29	0	R/W	<u>Pin multiplexing (GIO29)</u> » 0: Normal (GIO29) « » 1: Normal (GIO29) « » 2: SIF_EN (GIO29) « » 3: Reserved «
7:6	GD28	0	R/W	<u>Pin multiplexing (GIO28)</u> » 0: Normal (GIO28) « » 1: UART_TXD1 (GIO28) « » 2: Reserved « » 3: Reserved «
5:4	GD27	0	R/W	<u>Pin multiplexing (GIO27)</u> » 0: Normal (GIO27) « » 1: UART_RXD1 (GIO27) « » 2: Reserved « » 3: Reserved «
3:2	GD26	0	R/W	<u>Pin multiplexing (GIO26)</u> » 0: Normal (GIO26) « » 1: Normal (GIO26) « » 2: VLYNQ_CLOCK (GIO26) « » 3: CLKOUT2A (GIO26) «
1:0	GD25	0	R/W	<u>Pin multiplexing (GIO25)</u> » 0: Normal (GIO25) « » 1: IDEN (GIO25) « » 2: VLYNQ_SCRUN (GIO25) « » 3: CLKOUT1A (GIO25) «

14.7.22 FSEL3

Function select register #3

FSEL3	0003:05AA				offset: 0x2A				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	GIO40[1]	GIO40[0]	GIO39[1]	GIO39[0]	GIO38[1]	GIO38[0]	GIO37[1]	GIO37[0]	GIO36[1]	GIO36[0]	GIO35[1]	GIO35[0]	GIO34[1]	GIO34[0]	GIO33[1]	GIO33[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:14	GD40	0	R/W	<u>Pin multiplexing (GIO40)</u> » 0: Normal (GIO40) « » 1: C_WEN (GIO40) « » 2: Reserved « » 3: Reserved «
13:12	GD39	0	R/W	<u>Pin multiplexing (GIO39)</u> » 0: Normal (GIO39) « » 1: FIELD_VENC (GIO39) « » 2: Reserved « » 3: Reserved «
11:10	GD38	0	R/W	<u>Pin multiplexing (GIO38)</u> » 0: Normal (GIO38) « » 1: EXT_LINE_ID (GIO38) « » 2: B2 (GIO38) « » 3: Reserved «
9:8	GD37	0	R/W	<u>Pin multiplexing (GIO37)</u> » 0: Normal (GIO37) « » 1: MIRQ (GIO37) « » 2: Reserved « » 3: MRST (GIO37) «
7:6	GD36	0	R/W	<u>Pin multiplexing (GIO36)</u> » 0: Normal (GIO36) « » 1: TMRC1 (GIO36) « » 2: Reserved « » 3: Reserved «
5:4	GD35	0	R/W	<u>Pin multiplexing (GIO35)</u> » 0: Normal (GIO35) « » 1: TMRC0 (GIO35) « » 2: Reserved « » 3: Reserved «
3:2	GD34	0	R/W	<u>Pin multiplexing (GIO34)</u> » 0: Normal (GIO34) « » 1: PWM1 (GIO34) « » 2: R2 (GIO34) « » 3: CLKOUT2B (GIO34) «
1:0	GD33	0	R/W	<u>Pin multiplexing (GIO33)</u> » 0: Normal (GIO33) « » 1: PWM0 (GIO33) « » 2: Reserved « » 3: CLKOUT1B (GIO33) «

14.7.23 FSEL4

Function select register

FSEL4	0003:05AC			offset: 0x2C		default: 0x0000										
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	ETM_CLK
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0

Bit	Name	Reset Value	R/W	Function
15:1	RSV			Reserved
0	ETM_CLK	0	R/W	ETM Trace clock inverse » 0: ETM Trace clock « » 1: Inverse ETM Trace clock « » 2: Reserved « » 3: Reserved «

14.7.24 CARD_SET

Card settings register

CARD_SET	0003:05AE				offset: 0x2E				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	ST_POL	ENB
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W	R/W
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0	0

Bit	Name	Reset Value	R/W	Function
152	RSV			Reserved
1	ST_POL	0	R/W	<u>Card status polarity</u> » 0: No inversion « » 1: inversion « Note: This is valid only if ENB is 0 (Enabled)
0	ENB	0	R/W	<u>Card Detect Enable</u> » 0: Disable « » 1: Enable «

15 Serial Port Interface (SPI)

15.1 Introduction

DM320 has a two clock synchronized serial port interfaces (SIF or SPI). The transfer unit is always 8 bits, and it is a simple three-line serial interface consisting of a clock line, output data line, and input data line. Serial Port 0 has a DMA transfer function with SDRAM to transfer large amount of data. DM320 SPI has master mode and slave mode, which is selectable. The SPI can supply the serial clock when it is used in a master mode. The SPI also can be synchronized by the external serial clock input when it is used in a slave mode. Block diagram of SPI is shown in Figure 106.

Both of the serial ports and the DMA channel are controlled by ARM registers. Refer Serial Port 0 for a list of control registers for serial port 0, and Serial Port 1 for a list of control registers for serial port 1.

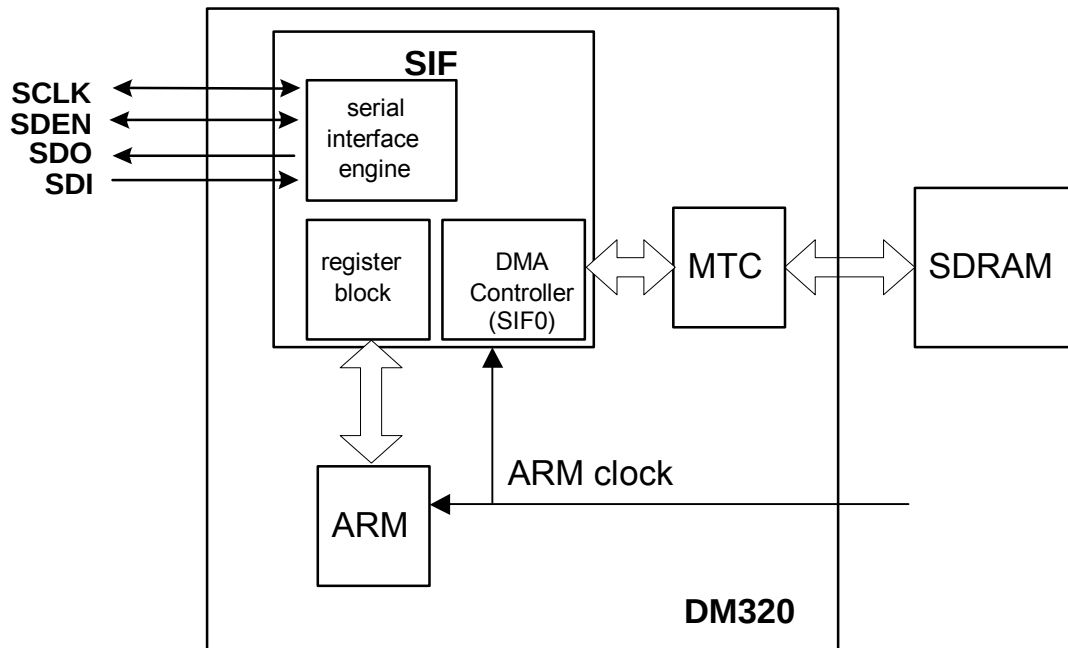


Figure 106: DM320 Serial Port Interface

15.2 Serial Transmit / Receive

15.2.1 Transfer procedure

1. Writing the transmit data to the transmit register (8bits) starts the transmit/receive data. The transfer flag XMIT is asserted.
2. The data set in the transmit register TXDATA is output from the SDO port in synchronization with the falling edge of serial clock output (SCLK).
3. During receive, the data from the SDI is captured to receive buffer with the rising edge of SCLK.
4. Upon the completion of data transmit/receive of 8bits, a transfer complete interrupt is generated. The transfer flag is cleared automatically.
5. The receive data can be read from the RXDATA register.

The programming details for SPI are discussed in 15.11.

15.2.2 Other features

- ☐ For transmit or receive data, MSB first or LSB first options are selectable.
- ☐ Bit rate can be set in the BR register. The calculation is as follows.

$$\text{Bit Rate} = \text{ARM Clock} / ((\text{BR} + 1) \times 2)$$
- ☐ The polarity of clock during non-transfer is selectable.
- ☐ Both Master and Slave mode are selectable. While the transfer is in progress, the mode cannot be changed.
- ☐ It is in slave mode by default.
- ☐ Clock controller can set invert clock mode. Refer to chapter on Clock Controller for more details on invert clock registers.
- ☐ In invert clock mode, the falling edge of serial clock latches receive data, and the rising edge of serial clock output transmit data.

Table 43 shows the different possible clock mode selections.

Mode	SCLKM	Clock mode	
Master	0	Default	Strobe data at rise edge of SCLK., output at fall edge. In non-transfer period SCLK is HI.
Master	1	Default	Strobe data at rise edge of SCLK., output at fall edge. In non-transfer period SCLK is LOW.
Master	0	Invert	Strobe data at fall edge of SCLK., output at rise edge. In non-transfer period SCLK is LOW.
Master	1	Invert	Strobe data at fall edge of SCLK., output at rise edge. In non-transfer period SCLK is HI.
Slave	-	Default	Strobe data at rise edge of SCLK., output at fall edge.
Slave	-	Invert	Strobe data at rise edge of SCLK., output at fall edge.

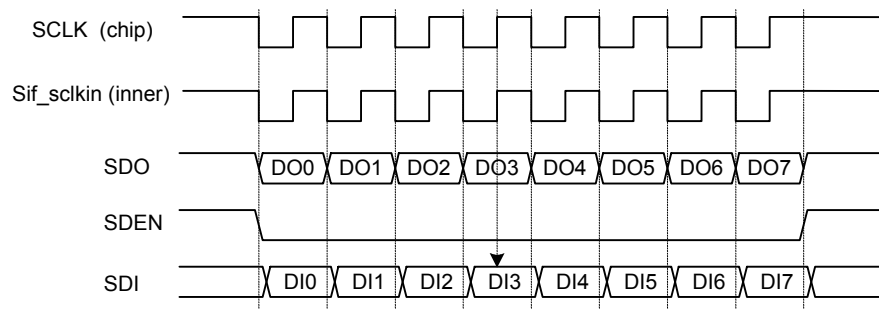
Table 43: Clock Mode Selection Table

15.2.3 Timing

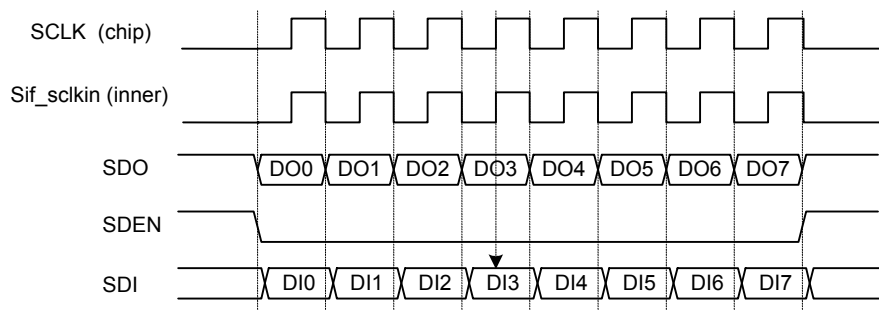
Master mode

Figure 107 Timing Diagram (master)

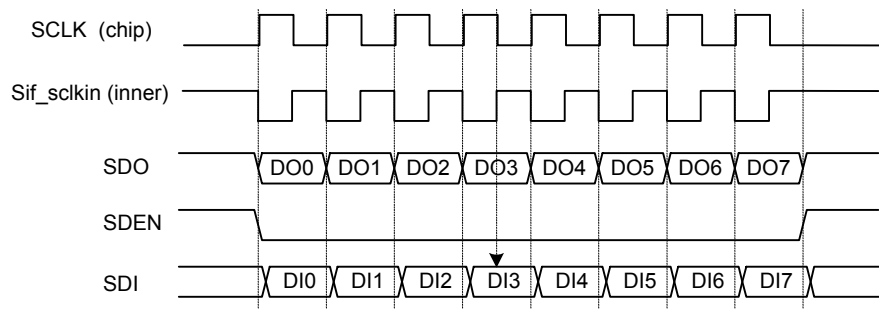
SCLKM = 0, clock default



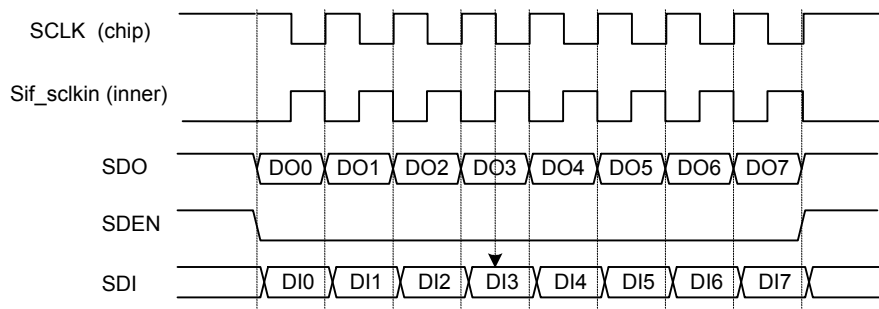
SCLKM = 1, clock default



SCLKM = 0, clock inv mode



SCLKM = 1, clock inv mode



slave mode

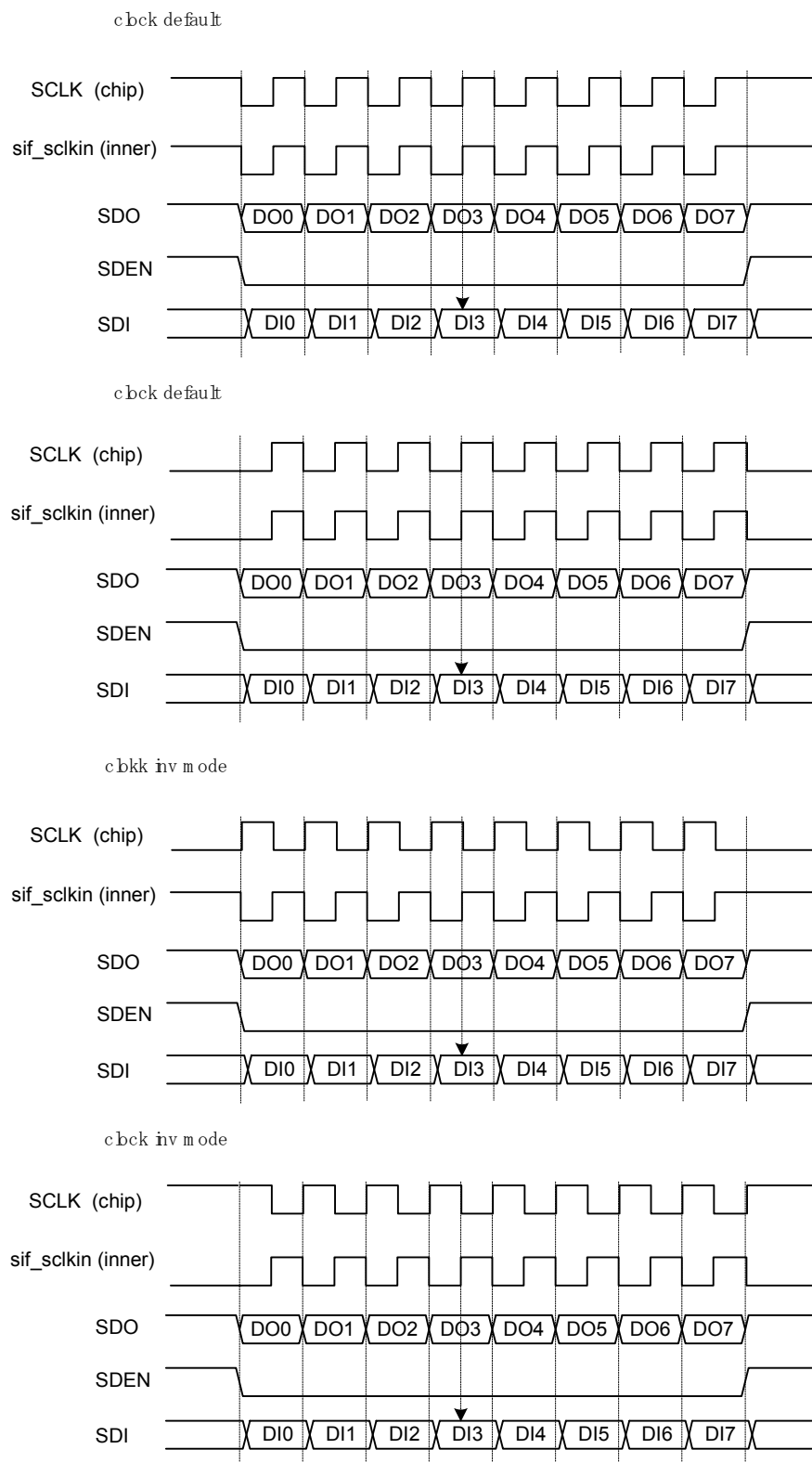


Figure 108 Timing Diagram (Slave)

15.3 Serial Interface Signal Pins

The SPI-0 uses dedicated pins for external interface. SPI-1 on the other hand use multiplexed GIO pins for external interface. Table 44 shows the pins used by the two Serial Interface ports. When SPI-1 is used, set appropriate GIO pins for usage in serial interface using the register FSEL2 in the “GIO module”. Refer the chapter on GIO for more details.

Serial Interface Signal	SPI 0	SPI 1
Serial Data Input	SDI	GIO31
Serial Data Output	SDO	GIO32
Serial Data Enable	SDEN	GIO29
Serial Clock	SCLK	GIO30

Table 44: Serial Interface Signal Pins

15.4 Baud Rate

Baud rate is set in the RATE field of SIOMODEx register. The baud rate is defined with the following formula:

$$\text{Baud Rate} = (\text{ARM clock}) / ((\text{RATE} + 1) \times 2)$$

Where, RATE is the value set in SIOMODEx register.

Note that the baud rate settings are meaningful only in master mode. In slave mode baud rate settings are invalid.

15.5 Master Mode SIF

In master mode, the serial port outputs both the Serial Clock (SCLK) and Serial Enable (SDEN) signals. Writing “0” to the bit MSSEL in SIOMODEx configures the SPI in master mode.

15.5.1 Data Transfer Procedure

This section describes the data transfer procedure in master mode.

- Write “1” in bit SIOEN of SIOENx register.
- During transmit, write data byte to be transferred to TXDATAx register. This initiates the serial data transmission one bit at a time from SDO port in synchronization with the falling edge of serial clock output (SCLK).
- During receive, the data is read from SDI to RXDATAx register one bit at a time in synchronization with the rising edge of SCLK.
- The transfer flag XMIT in RXDATAx register is asserted at the first rising edge of SCLK after data transfer is initiated.

- The transfer flag XMIT is cleared automatically at the eighth rising edge of SCLK from the start of transfer.
- Upon the completion of data transfer (transmit/receive) of 8 bits, SDEN signal is asserted “High” and a transfer complete interrupt is generated.
- The receive data can be read from the RXDATAx. When transfer is continued without reading the receive data, the RXDATA register is overwritten.

15.5.2 Data Transfer Timing Diagram

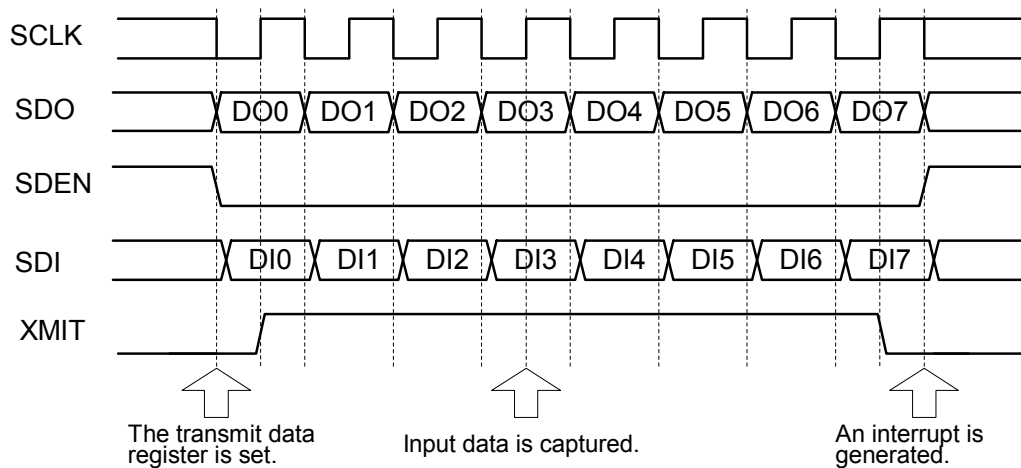


Figure 109: Timing Diagram for Master Mode Data Transfer

15.6 Slave Mode SIF

In slave mode, the serial port expects both the Serial Clock (SCLK) and Serial Enable (SDEN) signals as inputs. Writing “1” to the bit MSSEL in SIOMODEx configures the SPI in slave mode. By default, both SPI-0 and SPI-1 are configured in slave mode.

15.6.1 Data Transfer Procedure

This section describes the data transfer procedure in slave mode.

- Write “1” in bit SIOEN of SIOENx register
- During transmit, write data byte to be transferred to TXDATAx register. The first bit (Bit-0 in case of LSB first, and Bit-7 in case of MSB first) of the data set in TXDATAx is output to SDO port in synchronization with the falling edge of serial clock output (SCLK).
- When SDEN is asserted “Low”, the transfer flag XMIT is asserted at the first rising edge of SCLK.

- Remaining 7 bits of the data set in the TXDATAx register is output to SDO port in synchronization with the falling edge of serial clock output (SCLK).
- During receive, the data is read from SDI to RXDATAx register one bit at a time in synchronization with the rising edge of SCLK.
- The transfer flag XMIT is cleared automatically and a transfer complete interrupt is generated at the eighth rising edge of SCLK from the start of transfer.
- The receive data can be read from the RXDATAx register. When transfer is continued without reading the receive data, the RXDATA register is overwritten.

15.6.2 Data Transfer Timing Diagram

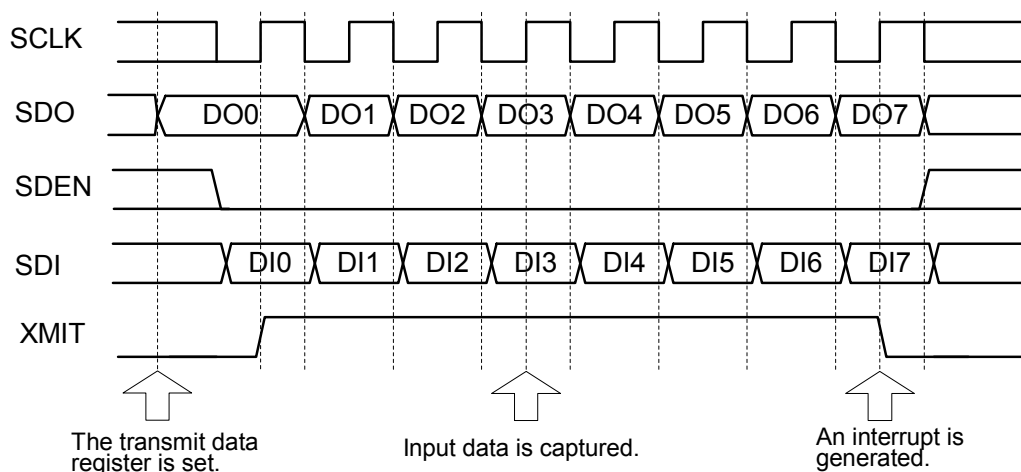


Figure 110: Timing Diagram for Slave Mode Data Transfer

15.7 Transmission and Reception Buffers

Each serial port has a 8-bit buffer for transmit and another 8-bit buffer for receiving data. These buffers correspond to the TXDATAx and RXDATAx registers. The data set in the transmission buffer is output in synchronization with the falling edge of SCLK. The XMIT bit of RXDATAx register can be used to detect if a transmission or reception is in progress or not. When writing into the transmit data buffer, the transfer flag XMIT is automatically set. The XMIT bit is cleared automatically at the end of the transmission or reception completion.

Check the status of XMIT before writing into the transmit register. If a transmission is in progress, the piece of data written into TXDATAx register will be ignored and lost. When a reception is in progress (XMIT=1), the data read from RXDATAx register would always be the previous one. Any new piece of data that arrives into the serial port will update the RXDATAx. If the previous data hasn't been read before, it will be lost.

15.8 Serial Port Modes

In master mode of operation, the serial clock mode bit (SCLKM) of the SIOMODEx register can be used to set the polarity of the clock pin when the serial port is in idle mode. By default, this bit is '0' and the SCLK clock signal will stay high when no data transfer is in progress.

The SIOEN bit in SIOENx register is used to enable or disable the serial port. When this bit is set to 0 (default value), the serial port is disabled. If a transmission or reception is in progress, writing to this bit will take effect only after the current transfer has been completed.

When the serial port is disabled, any new data written into TXDATAx register does not initiate any new transmission, any new data received is ignored and the RXDATAx register keeps its previous value. In disabled mode, the pins SDO and SDEN output the logic level '1'. The status of the SCLK pin is dependent on the SCLKM bit of SIOMODEx register.

The MSB bit of the SIOMODEx register is used to choose between the LSB or the MSB of the data, to be placed first. This bit affects the transmission and the reception.

15.9 Interrupts

When a transmission or reception is completed (DMA (only for SPI 0) as well as non-DMA mode), an interrupt can be sent to the ARM. The interrupts INT8 and INT9 are connected to the serial port 0 and serial port 1, respectively. The interrupt can be enabled or disabled by setting registers in the interrupt controller. Refer to chapter on Interrupt Controller for more details.

15.10 Transfer Using DMA

SPI 0 is connected to SDRAM by a dedicated bus, and DMA data transfer to and from SDRAM is possible without going through ARM.

The SDRAM controller has three DMA channels, which are used to perform DMAs involving SDRAM as source or destination. A DMA channel between SDRAM and Serial I/F 0 must be selected prior to triggering Serial I/F 0 DMA. The DMA channel is selected using the DMASEL register (0x309C6) in MTC. Also the DMA channel SDRAM access priority can be changed using the SDPTY1-12 (0x309AA – 0x309C0) registers. Refer to "SDRAM priority access" and "SDRAM DMA channels" in chapter on Memory Traffic Controller (MTC) for more details.

ARM executes DMA transfer by configuring the DMA controller in the serial I/F module. The SDRAM source or destination address, total number of bytes to be transferred and the transfer direction are set. When DMA transfer is complete, an interrupt is generated to ARM (INT8). The number of bytes to be transferred using DMA has to be at least 4 bytes. So, the number of bytes transmitted over the serial port line has to be a multiple of 4. Endian conversion is possible at the time of DMA transfer. There are four conversion modes, as shown in *Figure 111*. The endian conversion is configured in the field PRM[1:0] of *DMAMODE0* register.

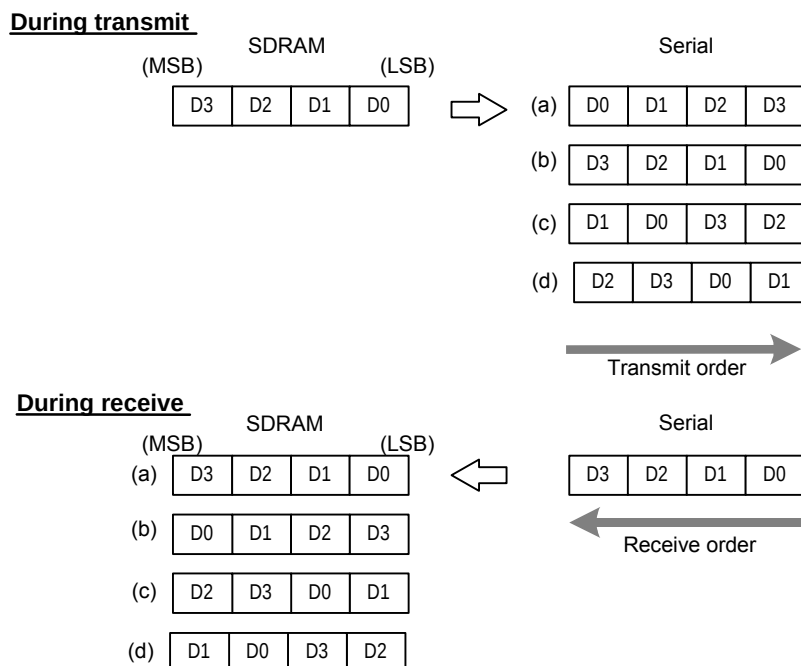


Figure 111: Endian Conversions During Transmission and Reception

15.10.1 DMA transfer procedure

Below is an example of DMA transfer procedure:

- DMA channel is selected between serial interface and SDRAM by setting SEL1 or SEL2 or SEL3 field in DMASEL (0x309C6) register of MTC.
- The serial interface is set to DMA mode. SDRAM start address and the numbers of transfer bytes are set.
NOTE: SDRAM address must be Word aligned and transfer byte must be multiple of 4 bytes
- DMA transfer starts by setting the start bit (TRG) in the register DMATRG0. With DMA starts; data is read from SDRAM, 4 bytes at a time. The data is written in the SPI transfer register 1 byte at a time. The data transmitted is synchronized with the falling edge of the serial port clock.
- In master as well as slave mode, serial transfer is suspended during SDRAM access. That is serial transfer is suspended for 1 ARM clock after every byte transfer. In slave mode, wide enough time-interval should be left to ensure proper SDRAM access.
- The next 4 bytes are read from the SDRAM and transmitted.
- When the requested amount of bytes to be sent has been reached, an interrupt occurs.

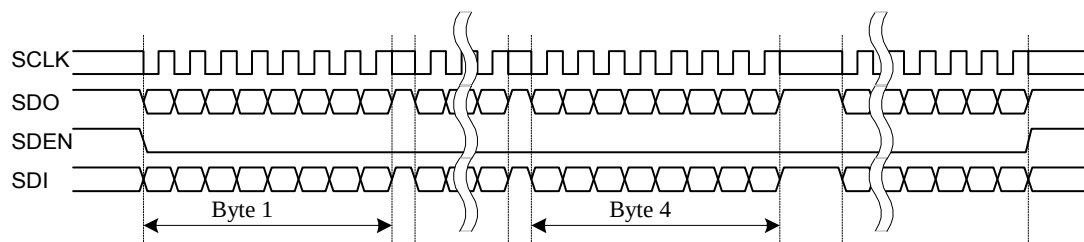


Figure 112: Timing chart at time of DMA transfer

Because DMA transfer request occurs every 4 bytes, the serial transfer can look discontinued.

15.10.2 Endianness conversion during DMA serial transmit

The read data of 4 bytes from the SDRAM are described as D3 D2 D1 D0 (MSB=D3, LSB=D0). The data is transmitted from serial port buffer in the order as shown in Table 45 (leftmost byte transmitted first).

PRM[1:0] in DMAMODE0 register	Byte order in Serial Transmit Buffer
0	D0 D1 D2 D3
1	D3 D2 D1 D0
2	D1 D0 D3 D2
3	D2 D3 D0 D1

Table 45: Endianness Conversion during Transmit

15.10.3 Endianness conversion during DMA serial receive

The data is received in the serial port buffer as D0 D1 D2 D3, D0 received first. It is transmitted to SDRAM as shown in

PRM[1:0] in DMAMODE0 register	Data in SDRAM
0	D3 D2 D1 D0
1	D0 D1 D2 D3
2	D2 D3 D0 D1
3	D1 D0 D3 D2

Table 46: Endianness Conversion during Receive

15.11 Programming Procedure for SPI 0/1 (Non-DMA Mode)

1. Configure the SIOMODEx Register.
 - Set the Transfer Rate in RATE [7:0] field.
 - Set the SPI in Master Mode->'0' or Slave Mode->'1' in MSSEL bit.

- Set the Bit-order i.e., LSB->'0' or MSB->'1' first in MSB field.
 - Set the polarity of clock during non-transfer by writing appropriate value in SCLKM field.
2. Configure the SIOENx Register.
Write '1' in SIOEN field.
 3. Write data to be transferred in TXDATAx register in transmission mode and read data from RXDATAx register in reception mode.
 4. Check for XMIT flag in RXDATAx register to see the completion status of transmission or reception.

15.12 Programming Procedure for SPI 0 (DMA Mode)

1. Configure the SIOMODEx Register.
 - Set the Transfer Rate in RATE [7:0] field.
 - Set the SPI in Master Mode->'0' or Slave Mode ->'1' in MSSEL field.
 - Set the Bit-order i.e., LSB->'0' or MSB->'1' first in MSB field.
 - Set the polarity of clock during non-transfer by writing appropriate value in SCLKM field.
 - Set DMA mode by writing '1' in SELSDEN field.
2. Configure the SIOENx Register.
Write '1' in SIOEN field.
3. Configure the DMASTADL0 and DMASTADH0 registers.
Set the SDRAM starting address in DMASTADL0 and DMASTADH0 register. Start address should be offset from SDRAM base address (0x00900000). The address should be word aligned.
4. Configure the DMA Channel and Burst Size in MTC-DMASEL Register.
Refer Chapter MTC for DMASEL register description.
5. Configure DMAMODE0 Register.
Configure the DMA endian by writing appropriate bits in *PRM1-PRM0* field.
Set the DMA transfer direction by writing appropriate value in *DIR* field
Set the DMA transfer count in *TXSZ10-TXSZ0* field. The count should be word aligned.
6. Configure theDMATRGO register to kick-off the DMA transfer.
Write '1' in TRG field to kick-off the DMA transfer.
7. Check the DMA completion either by Poll-mode or Interrupt mode method.

15.13 Serial Port Register Maps (SPx)

15.13.1 Serial Port 0

Address	Register	Description
0003:0200	TXDATA0	SP0 Transmit Data Register
0003:0202	RXDATA0	SP0 Receive Data Register
0003:0204	SIOEN0	SP0 Transmit Enable
0003:0206	SIOMODE0	SP0 Mode
0003:0208	DMATRG0	SP0 DMA Trigger
0003:020A	DMAMODE0	SP0 DMA Mode
0003:020C	DMASTADL0	SP0 DMA SDRAM Start Address - low
0003:020E	DMASTADH0	SP0 DMA SDRAM Start Address - high
0003:0210	DMASTAT0	SP0 DMA Status

15.13.2 Serial Port 1

Address	Register	Description
0003:0280	TXDATA1	SP1 Transmit Data Register
0003:0282	RXDATA1	SP1 Receive Data Register
0003:0284	SIOEN1	SP1 Transmit Enable
0003:0286	SIOMODE1	SP1 Mode

15.14 Serial Port Registers

15.14.1 TXDATAx

SP0 and SP1 Transmit Data Registers

TXDATA0	0003:0200	offset:	0x00	default: 0x0000												
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	TXD[7]	TXD[6]	TXD[5]	TXD[4]	TXD[3]	TXD[2]	TXD[1]	TXD[0]
R/W	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	-	-	-	0	0	0	0	0	0	0	0

TXDATA1	0003:0280	offset:	0x00	default: 0x0000												
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	TXD[7]	TXD[6]	TXD[5]	TXD[4]	TXD[3]	TXD[2]	TXD[1]	TXD[0]
R/W	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	-	-	-	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:8	RSV			Reserved
7:0	TXD	0	R/W	Transmit Data register When data is written to this register, transmission is started. Data output is performed on the falling edge of the clock. NOTE: - SIOEN must be set to 1 to enable writes to this register - Cannot write to this register when RXDATA:XMIT is 1

15.14.2 RXDATAx

SP0 and SP1 Receive Data Registers

RXDATA0	0003:0202			offset: 0x02												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	XMIT	RXD[7]	RXD[6]	RXD[5]	RXD[4]	RXD[3]	RXD[2]	RXD[1]	RXD[0]	
R/W	-	-	-	-	-	-	-	R	R	R	R	R	R	R	R	R	
Default	-	-	-	-	-	-	-	0	0	0	0	0	0	0	0	0	

RXDATA1	0003:0282			offset: 0x02			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	XMIT	RXD[7]	RXD[6]	RXD[5]	RXD[4]	RXD[3]	RXD[2]	RXD[1]	RXD[0]
R/W	-	-	-	-	-	-	-	R	R	R	R	R	R	R	R	R
Default	-	-	-	-	-	-	-	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:9	RSV			Reserved
8	XMIT	0	R	Send/Receive flag » 0: Send/Receive complete « » 1: Send/Receive in progress «
7:0	RXD	0	R	Received Data register After waiting for the send/receive to complete, received data can be read from this register. Data is sampled on the rising edge of the clock.

15.14.3 SIOENx

SP0 and SP1 Transmit Enable Registers

SIOEN0	0003:0204			offset: 0x04												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	SIOEN	
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W	
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0	

SIOEN1	0003:0284			offset: 0x04												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	SIOEN	
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W	
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0	

Bit	Name	Reset Value	R/W	Function
15:1	RSV			Reserved
0	SDEN	0	R/W	Transmission enable register When this bit is set, transmission can be started by writing data into TXDATAx. When this bit is cleared, transmission is prohibited, and data cannot be written into TXDATAx.

15.14.4 SIOMODEx

SP0 and SP1 Mode Registers

SIOMODE0	0003:0206			offset:	0x06										default: 0x0100		
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	SELSDEN	SLVEN	SIOCLR	SCLKM	MSB	MSSEL	RATE[7]	RATE[6]	RATE[5]	RATE[4]	RATE[3]	RATE[2]	RATE[1]	RATE[0]	
R/W	-	-	R/W	R	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	-	-	0	-	0	0	0	1	0	0	0	0	0	0	0	0	

SIOMODE1	0003:0286			offset: 0x06													default: 0x0100	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	SELSDEN	SLVEN	SIOCLR	SCLKM	MSB	MSSEL	RATE[7]	RATE[6]	RATE[5]	RATE[4]	RATE[3]	RATE[2]	RATE[1]	RATE[0]		
R/W	-	-	R/W	R	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W		
Default	-	-	0	-	0	0	0	1	0	0	0	0	0	0	0	0		

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
13	SELSDEN	0	R/W	SDEN mode selector in DMA mode » 0: SDEN is low in DMA mode « » 1: SDEN toggles as in normal serial mode «
12	SLVEN	X	R	<u>External Enable</u> Note: Indicates status of SDEN0
11	SIOCLR	0	R/W	<u>Serial I/O Port Clear</u> Resets Serial Port, all registers settings are retained
10	SCLKM	0	R/W	Serial Clock Mode Sets level of clock when transmission is idle » 0: serial clock level = 1 when idle « » 1: serial clock level = 0 when idle «
9	MSB	0	R/W	<u>Determines bit order of transmitted or received data</u> » 0: LSB first « » 1: MSB first «
8	MSSEL	1	R/W	<u>Master/Slave Selection</u> » 0: Master « » 1: Slave «
7:0	RATE	0	R/W	Transfer Bit Rate The transfer rate is given by Bit rate = (ARM clock) / (2*(RATE+1))

15.14.5 DMATRGO

SP0 DMA Trigger Registers

DMATRGO	0003:0208			offset:	0x08										default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	TRG
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	W
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0

Bit	Name	Reset Value	R/W	Function
15:1	RSV			Reserved
0	TRG	0	W	<u>DMA Transfer Start bit</u> When set to 1, DMA transfer is started. After transfer ends, bit is automatically cleared to 0. Writing 0 has no meaning. If set to 1 during transfer, it has no meaning.

15.14.6 DMAMODE0

SP0 DMA Mode Registers

DMAMODE0	0003:020A			offset:	0x0A										default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	PRM[1]	PRM[0]	DIR	TRSZ[10]	TRSZ[9]	TRSZ[8]	TRSZ[7]	TRSZ[6]	TRSZ[5]	TRSZ[4]	TRSZ[3]	TRSZ[2]	TRSZ[1]	TRSZ[0]
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
13:12	PRM	0	R/W	<u>DMA transfer byte order</u> » 00: D3D2D1D0 -> D3D2D1D0 « » 01: D3D2D1D0 -> D0D1D2D3 « » 10: D3D2D1D0 -> D2D3D0D1 « » 11: D3D2D1D0 -> D1D0D3D2 «
11	DIR	0	R/W	<u>DMA transfer direction</u> » 0: SIF -> SDRAM « » 1: SDRAM -> SIF «
10:0	TRSZ	0	R/W	<u>Transfer size in bytes</u> 4 byte alignment is required. i.e., bit 0 and bit 1 are forced to 0

15.14.7 DMASTADL0

SP0 DMA SDRAM Start Address Low Registers

DMASTADL0		0003:020C			offset: 0x0C		default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	STADL[15]	STADL[14]	STADL[13]	STADL[12]	STADL[11]	STADL[10]	STADL[9]	STADL[8]	STADL[7]	STADL[6]	STADL[5]	STADL[4]	STADL[3]	STADL[2]	STADL[1]	STADL[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:0	STADL	0	R/W	Lower 16 bits of DMA start address of SDRAM. 4-byte aligned address only can be specified (bits 0,1 are 0) NOTE: address specified as offset from start of SDRAM

15.14.8 DMASTADH0

SP0 DMA SDRAM Start Address High Registers

DMASTADH0		0003:020E			offset: 0x0E		default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	STADH[11]	STADH[10]	STADH[9]	STADH[8]	STADH[7]	STADH[6]	STADH[5]	STADH[4]	STADH[3]	STADH[2]	STADH[1]	STADH[0]
R/W	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:12	RSV			Reserved
11:0	STADH	0	R/W	Upper 12 bits of DMA start address of SDRAM NOTE: address specified as offset from start of SDRAM

15.14.9 DMASTAT0

SP0 DMA Status Registers

DMASTAT0		0003:0210			offset: 0x10		default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RUN	RSV	RSV	RSV	RSV	REST[10]	REST[9]	REST[8]	REST[7]	REST[6]	REST[5]	REST[4]	REST[3]	REST[2]	REST[1]	REST[0]
R/W	R	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15	RUN	0	R	DMA transfer flag » 0: Transfer done « » 1: Transfer in progress «
14:11	RSV			Reserved
10:0	REST	0	R/W	Remaining number of DMA transfer bytes

16 UART

16.1 Introduction

DM320 supports two start-stop synchronization serial ports (UART). A 32-byte FIFO is built in for each of the transmitting and receiving channels of both the ports. Interrupts can be generated due to errors or various states of FIFO. Data width can be 8 or 7 bits. Even, odd or no parity can be selected. Stop bits can be configured to be 1 or 2 bits. Transmission/reception uses the AHB clock or the PLLIN clock. The maximum baud rate achievable is 1/16 of the input clock.

Figure 113 shows the DM320-UART block diagram

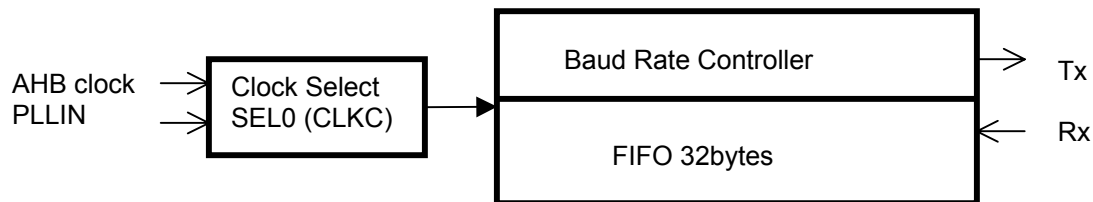


Figure 113 UART block diagram

The UART module detects parity errors and overrun errors. Interrupt to ARM is generated when any of the following events occur:

- Amount of data in receiving-side FIFO exceeds the configurable trigger value.
- Amount of data in transmitting-side FIFO falls below the configurable trigger value.
- UART word (start + data + parity + stop) received has an error.
- The timeout period is reached.

The speed of UART ports is determined by BRSRx. The UART module is clocked by AHB clock (or PLLIN) divided by 16.

$$\text{UARTspeed} = \text{Reference Clock} / (16 * (\text{BRSRx} + 1))$$

Where, Reference Clock = AHB clock or PLL input clock (refer “Clock Controller” chapter for details)

Each UART has two 32-byte FIFOs, one for transmits, and one for receive. However, there is only one register to access them. The DTRRx is used to

access transmit and receive FIFOs. When this register is read, the oldest data stored in reception FIFO is returned. When data is written into it, it is stored into the transmission FIFO and then transferred. Make sure that the transmit FIFO is not full before writing into this register, otherwise the last FIFO entry is overwritten. When reading this register, the bits 12 to 8 indicates if the word read is valid or not. The data is valid only when bits 11 to 8 are equal to '0' (i.e., Bit 11-> Break flag, Bit 10->Framing error, Bit 9-> Overrun flag, and Bit 9->Parity error flag) and bit 12 (Receive word valid flag) is equal to '1'.

The configuration of the UART is defined by the MSRx. The PSB and PEB bits select the parity used during a communication. When enabled, a parity bit is appended at the end of the data. The bit CLS selects the size of the word transmitted or received. When only 7 bits are used, DTR [7] bit of the DTRRx is ignored during a write and always reads 0 during a read.

The RFTIE bit enables the interrupt to be generated when the data size in reception FIFO is above the trigger level.

The TFTIE bit enables the interrupt to be generated when the data size in transmission FIFO is below trigger level.

The REIE bit enables the interrupt to be generated when a received word has parity error, overrun, frame error or break.

The TOIC [1:0] bits are used to configure the interval for timeout interrupt. If at least one word of received data is stored in reception FIFO, and if no new data has been received until the timeout interval, interrupt is generated. ARM can read the data in receive FIFO on this interrupt, even though the data level is less than FIFO trigger level. This is useful when there is lot of idle time between words received.

16.2 UART Signal Pins

UART-0 uses dedicated pins for external interface, UART-1 on the other hand uses multiplexed GIO pins. Table 47 shows the pins used by the two UART ports. When UART-1 is used, set appropriate GIO pins for usage in UART interface using the register FSEL2 (0003:05A8h) in the GIO module. Refer the chapter on GIO for more details.

Table 47 UART Interface Signal Pins

UartI Interface Signal	UART 0	UART 1
Uart Data Input	RXD	GIO27
Uart Data Output	TXD	GIO28

16.3 Programming procedure for UART

1. Configure the BRSR_x register to set the UART Speed.
2. Configure the MSR_x register to set Character length, stop bit length, parity type, parity enable/disable and enable the interrupts if required.
3. Configure the TFCR_x register to set the Tx FIFO trigger level.
4. Configure the RFCR_x register to set the Rx FIFO trigger level.
5. Write the data to be transferred into DTRR_x register.
6. Read the data from Rx FIFO via DTRR_x register.

16.4 UART Register Maps (UART_x)

16.4.1 UART 0

Address	Register	Description
0003:0300	DTRR0	UART0 Data Transmit/Receive Register
0003:0302	BRSR0	UART0 Bit Rate Set Register
0003:0304	MSR0	UART0 Mode Setup Register
0003:0306	RFCR0	UART0 Receive FIFO Control Register
0003:0308	TFCR0	UART0 Transmit FIFO Control Register
0003:030A	LCR0	UART0 Line Control Register
0003:030C	SR0	UART0 Status Register

16.4.2 UART 1

Address	Register	Description
0003:0380	DTRR1	UART1 Data Transmit/Receive Register
0003:0382	BRSR1	UART1 Bit Rate Set Register
0003:0384	MSR1	UART1 Mode Setup Register
0003:0386	RFCR1	UART1 Receive FIFO Control Register
0003:0388	TFCR1	UART1 Transmit FIFO Control Register
0003:038A	LCR1	UART1 Line Control Register
0003:038C	SR1	UART1 Status Register

16.5 UART Registers

16.5.1 DTRRx

UART Data Transmit/Receive Registers

DTRR0	0003:0300			offset:	0x00		default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RVF	BF	FE	ORF	PEF	DTR[7]	DTR[6]	DTR[5]	DTR[4]	DTR[3]	DTR[2]	DTR[1]	DTR[0]
R/W	-	-	-	R	R	R	R	R	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	0	0	0	0	0	0	0	0	0	0	0	0	0

DTRR1	0003:0380			offset:	0x00		default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RVF	BF	FE	ORF	PEF	DTR[7]	DTR[6]	DTR[5]	DTR[4]	DTR[3]	DTR[2]	DTR[1]	DTR[0]
R/W	-	-	-	R	R	R	R	R	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:13	RSV			Reserved
12	RVF	0	R	<u>Received Word Valid Flag</u> » 0: Received word invalid « » 1: Received word valid «
11	BF	0	R	<u>Break Flag</u> » 0: No break « » 1: detected a break «
10	FE	0	R	<u>Framing Error</u> » 0: No Framing Error « » 1: There is framing error «
9	ORF	0	R	<u>Over Run Flag</u> » 0: No Overrun « » 1: Overrun «
8	PEF	0	R	<u>Parity Error Flag</u> » 0: No parity error « » 1: Parity error «
7:0	DTR	0	R/W	<u>Data Transmit/Receive</u> On a read, the oldest received byte stored in Receive FIFO is read along with the corresponding status bits and the byte read is deleted from the Recieve FIFO. On a write, the data is stored in the transmission FIFO. Before writing, it is necessary to confirm that the transmission FIFO is not full.

16.5.2 BRSR_x

UART Bit Rate Set Registers

BRSR0	0003:0302			offset: 0x02			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	BRSR[15]	BRSR[14]	BRSR[13]	BRSR[12]	BRSR[11]	BRSR[10]	BRSR[9]	BRSR[8]	BRSR[7]	BRSR[6]	BRSR[5]	BRSR[4]	BRSR[3]	BRSR[2]	BRSR[1]	BRSR[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

BRSR1	0003:0382			offset: 0x02			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	BRSR[15]	BRSR[14]	BRSR[13]	BRSR[12]	BRSR[11]	BRSR[10]	BRSR[9]	BRSR[8]	BRSR[7]	BRSR[6]	BRSR[5]	BRSR[4]	BRSR[3]	BRSR[2]	BRSR[1]	BRSR[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15	BRSR	0	R/W	<u>Bit Rate Set Register</u> Sets data transfer bit rate: $\text{BRSR} = \{ \text{CLK} / (16 * \text{bitrate}) \} - 1$ $\text{UARTbitrate} = \text{CLK} / ((16 * (\text{BRSR} + 1)))$ Note: The clock source should be selected in SEL0 register of CLKC module

16.5.3 MSRx

UART Mode Setup Registers

MSR0	0003:0304				offset:	0x04				default: 0x0000						
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RFTIE	TFTIE	REIE	RSV	TOIC[1]	TOIC[0]	RSV	RSV	RSV	RSV	RSV	PEB	PSB	SBLS	RSV	CLS
R/W	R/W	R/W	R/W	-	R/W	R/W	-	-	-	-	-	R/W	R/W	R/W	-	R/W
Default	0	0	0	-	0	0	-	-	-	-	-	0	0	0	-	0

MSR1	0003:0384			offset: 0x04												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RFTIE	TFTIE	REIE	RSV	TOIC[1]	TOIC[0]	RSV	RSV	RSV	RSV	RSV	PEB	PSB	SBLS	RSV	CLS	
R/W	R/W	R/W	R/W	-	R/W	R/W	-	-	-	-	-	R/W	R/W	R/W	-	R/W	
Default	0	0	0	-	0	0	-	-	-	-	-	0	0	0	-	0	

Bit	Name	Reset Value	R/W	Function
15	RFTIE	0	R/W	<u>Receive FIFO Trigger Interrupt Enable</u> » 0: disabled « » 1: enabled « Receive Interrupt set when amount of data in Receive FIFO is above trigger level
14	TFTIE	0	R/W	<u>Transmit FIFO Trigger Interrupt Enable</u> » 0: disabled « » 1: enabled « Transmit Interrupt set when amount of data in Transmit FIFO falls below trigger level
13	REIE	0	R/W	<u>Receive Error Interrupt Enable</u> » 0: disabled « » 1: enabled « Receive Error Interrupt is set if - any received word that has a parity error, - there is a FIFO overrun, - there is a frame error, or - a break exists in the reception FIFO.
12	RSV			Reserved
11:10	TOIC	0	R/W	<u>Time Out Interrupt Control</u> Timeout Interrupt period: » 00: disabled « » 01: 3 words « » 10: 7 words « » 11: 15 words « Note: Timeout Interrupt is set in case where at least one word of received data has been stored in reception FIFO and new data has not been received even though the set time has elapsed. This interrupt can be used to provide a trigger for received data loading to ARM in the case where the received data has not been accumulated in the reception FIFO up to the reception FIFO trigger level at the end of data transfer.
9:5	RSV			Reserved
4	PEB	0	R/W	<u>Parity Enable Bit</u> » 0: disable (no parity) « » 1: enable (parity bit added to end of data byte) «
3	PSB	0	R/W	<u>Parity Selection Bit</u> » 0: even parity « » 1: odd parity «
2	SBLS	0	R/W	<u>Stop Bit Length Selection</u> » 0: 1 stop bit « » 1: 2 stop bits «
1	RSV			Reserved
0	CLS	0	R/W	<u>Character Length Selection</u> » 0: 8-bits « » 1: 7-bits « Specifies the data size

16.5.4 RFCRx

UART Receive FIFO Control Registers

RFCR0	0003:0306			offset: 0x06												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RFCB	RDEF	RSV	RSV	RSV	RTL[2]	RTL[1]	RTL[0]	RSV	RSV	RWC[5]	RWC[4]	RWC[3]	RWC[2]	RWC[1]	RWC[0]	
R/W	R/W	R/W	-	-	-	R/W	R/W	R/W	-	-	R	R	R	R	R	R	
Default	0	0	-	-	-	0	0	0	-	-	0	0	0	0	0	0	

RFCR1	0003:0386			offset: 0x06												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RFCB	RDEF	RSV	RSV	RSV	RTL[2]	RTL[1]	RTL[0]	RSV	RSV	RWC[5]	RWC[4]	RWC[3]	RWC[2]	RWC[1]	RWC[0]	
R/W	R/W	R/W	-	-	-	R/W	R/W	R/W	-	-	R	R	R	R	R	R	
Default	0	0	-	-	-	0	0	0	-	-	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15	RFCB	0	R/W	<u>Receive FIFO Clear Bit</u> » 0: no action « » 1: Receive FIFO is cleared, set to 0 after clear «
14	RDEF	0	R/W	<u>Receive Data Error Flag</u> Set to 1 if data that causes receive error is in FIFO Cleared to 0 if no data in FIFO causes a receive error This bit is the same as bit 9 of the SR register. When 1 is written to this bit, the receive block is reset and then the bit is cleared.
13:11	RSV			Reserved
10:8	RTL	0	R/W	<u>Receive Trigger Level</u> » 000: 1 word « » 001: 4 words « » 010: 8 words « » 011: 16 words « » 100: 24 words « » 101: 32 words « » 110: N/A « » 111: N/A « Sets Receive FIFO Trigger Interrupt level
7:6	RSV			Reserved
5:0	RWC	0	R	<u>Receive Word Count</u> Number of words in Receive FIFO

16.5.5 TFCRx

UART Transmit FIFO Control Registers

TFCR0	0003:0308			offset: 0x08													default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	TFCB	RSV	RSV	RSV	RSV	TTL[2]	TTL[1]	TTL[0]	RSV	RSV	TWC[5]	TWC[4]	TWC[3]	TWC[2]	TWC[1]	TWC[0]		
R/W	R/W	-	-	-	-	R/W	R/W	R/W	-	-	R	R	R	R	R	R		
Default	0	-	-	-	-	0	0	0	-	-	0	0	0	0	0	0		

TFCR1	0003:0388			offset: 0x08													default: 0x0000		
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	TFCB	RSV	RSV	RSV	RSV	TTL[2]	TTL[1]	TTL[0]	RSV	RSV	TWC[5]	TWC[4]	TWC[3]	TWC[2]	TWC[1]	TWC[0]			
R/W	R/W	-	-	-	-	R/W	R/W	R/W	-	-	R	R	R	R	R	R			
Default	0	-	-	-	-	0	0	0	-	-	0	0	0	0	0	0			

Bit	Name	Reset Value	R/W	Function
15	TFCB	0	R/W	<u>Transmit FIFO Clear Bit</u> » 0: no action « » 1: Transmit FIFO is cleared, set to 0 after clear «
14:11	RSV			Reserved
10:8	TTL	0	R/W	<u>Transmit Trigger Level</u> » 000: 1 word « » 001: 4 words « » 010: 8 words « » 011: 16 words « » 100: 24 words « » 101: 32 words « » 110: N/A « » 111: N/A « Sets Transmit FIFO Trigger Interrupt level
7:6	RSV			Reserved
5:0	TWC	0	R	<u>Transmit Word Count</u> Number of words in Transmit FIFO

16.5.6 LCRx

UART Line Control Registers

LCR0		0003:030A			offset: 0x0A												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	UTST	RSV	RSV	RSV	RSV	RSV	BOC	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV		
R/W	-	R/W	-	-	-	-	-	R/W	-	-	-	-	-	-	-	-		
Default	-	0	-	-	-	-	-	0	-	-	-	-	-	-	-	-		

LCR1	0003:038A			offset: 0x0A												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	UTST	RSV	RSV	RSV	RSV	RSV	BOC	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	
R/W	-	R/W	-	-	-	-	-	R/W	-	-	-	-	-	-	-	-	
Default	-	0	-	-	-	-	-	0	-	-	-	-	-	-	-	-	

Bit	Name	Reset Value	R/W	Function
15	RSV			Reserved
14	UTST	0	R/W	Test mode setting - set to 0 for normal operation
13	RSV			Reserved
8	BOC	0	R/W	Break Output Control Controls the TX output: » 0: Normal data output « » 1: Transmit output is forced to 0 «
7	RSV			Reserved

16.5.7 SRx

UART Status Registers

SR0	0003:030C			offset: 0x0C			default: 0x0403									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RFTI	TFTI	RFER	TOIF	RSV	RSV	RSV	RSV	RSV	RFNEF	TFEF	TREF
R/W	-	-	-	-	R	R	R	R	-	-	-	-	-	R	R	R
Default	-	-	-	-	0	1	0	0	-	-	-	-	-	0	1	1

SR1	0003:038C			offset: 0x0C			default: 0x0403									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RFTI	TFTI	RFER	TOIF	RSV	RSV	RSV	RSV	RSV	RFNEF	TFEF	TREF
R/W	-	-	-	-	R	R	R	R	-	-	-	-	-	R	R	R
Default	-	-	-	-	0	1	0	0	-	-	-	-	-	0	1	1

Bit	Name	Reset Value	R/W	Function
15:12	RSV			Reserved
11	RFTI	0	R	<u>Receive FIFO Trigger Indicator</u> » 0: amount of data is below trigger level « » 1: amount of data is above trigger level «
10	TFTI	1	R	<u>Transmit FIFO Trigger Indicator</u> » 0: amount of data is above trigger level « » 1: amount of data is below trigger level «
9	RFER	0	R	<u>Receive Data Error Flag</u> Set to 1 if data that causes receive error is in FIFO Cleared to 0 if no data in FIFO causes a receive error This bit is the same as bit 14 of the RFCR register.
8	TOIF	0	R	<u>Time Out Interrupt Flag</u> » 0: no timeout interrupt « » 1: timeout interrupt asserted «
7:3	RSV			Reserved
2	RFNEF	0	R	<u>Receive FIFO Not Empty Flag</u> » 0: No data in Receive FIFO « » 1: Data in Receive FIFO «
1	TFEF	1	R	<u>Transmit FIFO Empty Flag</u> » 0: Data in Transmit FIFO « » 1: No data in Transmit FIFO «
0	TREF	1	R	<u>Transmit Register Empty Flag</u> » 0: Data in either Transmit register or FIFO « » 1: No data in either Transmit register or FIFO «

17 MMC/SD Interface

17.1 Introduction

The MMC Controller transfers data between the ARM and DMA controller on one side and MMC/SD card on the other side. (Refer to Figure 114). The ARM and DMA controller can read/write the data in the card by accessing the registers in the MMC Controller. (Refer MMC/SD Card Interface Register Map)

The communication between the MMC Controller and memory card(s) is performed by the MMC/SD protocol. The MMC/SD protocol uses one bi-directional data line in the case of MMC and SD. In the case of SD, there is an additional option of using four parallel data lines.

Note: SPI protocol is not supported in MMC controller on DM320.

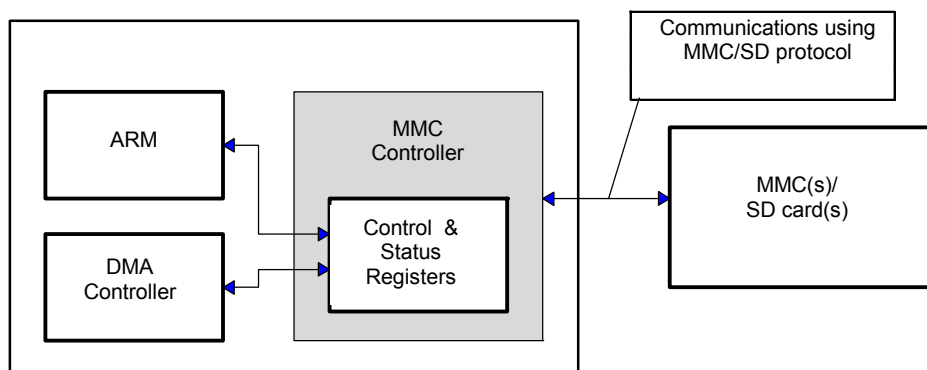


Figure 114: Role of the MMC Controller

17.2 Features

MMC Controller has the following features.

- Supports a MultiMedia Card (MMC) / Secure Digital Memory Card (SD card).
- The ability to use the MMC/SD protocol.
- Software-oriented implementation for future extensibility.
- A programmable frequency of the clock that controls the timing of transfers between the MMC controller and memory card.
- Supports DMA transfer between the MMC controller and SDRAM.
- Supports SDIO card, both I/O only and combo (I/O as well as memory) variants.

17.3 MMC Controller Pins

Table 48 shows the pins of the MMC Controller used in each mode.

The MMC/SD protocol uses the clock, command (two-way communication between the MMC Controller and memory card) and data (one line for MMC, 4 lines for SD card) pins.

Pin	Type [*]	Function	
		MMC Communications	SD Communications
CLK	O	Clock line	Clock line
CMD	I/O	Command line	Command line
DAT0	I/O	Data line 0	Data line 0
DAT1	I/O	(Not used)	Data line 1
DAT2	I/O	(Not used)	Data line 2
DAT3	I/O	(Not used)	Data line 3

^{*} I = Input to the MMC Controller; O = Output from the MMC Controller

Table 48: MMC Controller Pins

17.4 Function Clock and Memory Clock

There are two clocks, the function clock and memory clock, in the MMC module and each clock must be set with the proper frequency.

The function clock frequency determines the operation speed of the MMC Controller. This function clock is generated by dividing the AHB clock by programmable clock divider in the Clock Controller Module (CLKC). The divide-down value is set by the value in the MMC field of the CLKC's DIV3 register. The function clock is computed as per the following equation.

$$\text{Function clock frequency} = \text{AHB clock frequency} / (\text{MMC} + 1)$$

Note: Function clock frequency should be at most half of the AHB clock. So, MMC field in DIV3 register should atleast be set to 1.

The memory clock appears on the CLK pin of MMC controller interface. This clock controls the timing of communication between the MMC Controller and the connected memory card. Memory clock is generated by dividing the function clock in the MMC Controller (Refer to Figure 115). The divide-down value is set in CLKRT field of the MMCCLK register. Memory clock is computed as per the following equation.

$$\text{Memory clock frequency} = \text{Function clock frequency} / (2 * (\text{CLKRT} + 1))$$

Refer to the chapter on Clock Controller for more details on CLKC registers.

The following restrictions must be observed when the function clock and memory clock are determined.

- o Ensure that MMC bit in the INV register of CLKC module is set to '1' (as default setting)
- o Set the operating frequencies of the AHB, SDRAM and memory clock frequency to adhere to the following equation, when DMA transfer is used for the data transfer between MMC/SD and the SDRAM.

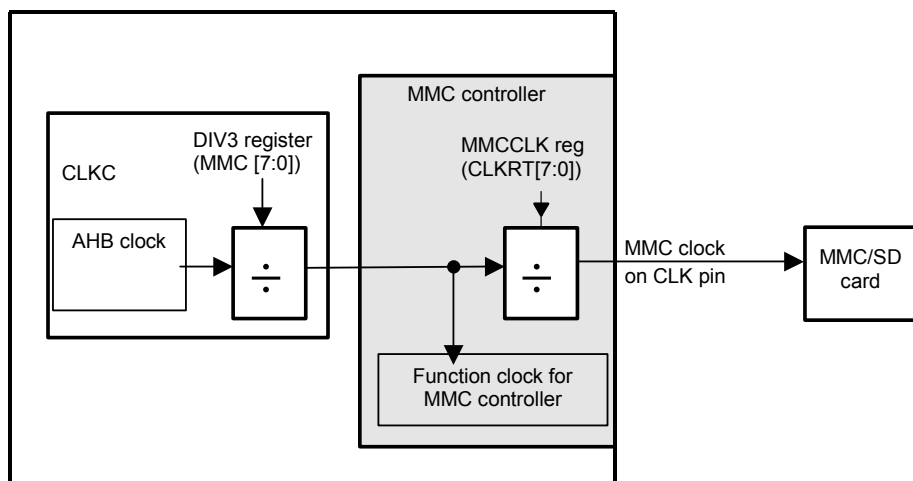


Figure 115: Clocking Diagram for the MMC Controller

17.5 MMC Controller Registers

17.5.1 Command Registers (MMCCMD, MMCARGH and MMCARGL)

The argument and command index are included in the command packet. Set upper 16 bits and lower 16 bits of the argument to the MMCARGH and MMCARGL registers, respectively. Subsequently, writing the command index to the CMD field in the MMCCMD register sends the command packet on the command line. In addition, set the other bits to the appropriate values in the MMCCMD register depending on the type of command. Table 49 gives the bit descriptions in the MMCCMD register.

Bit	Name	Description
15	DCLR	Setting '1' clears the shadow buffer for data transmit/receive. Set this bit when issuing a command to write data to the card using DMA.
14	INITCK	Setting '1' sends 80 dummy clocks before sending the command packet. This is an optional setting depending on the card used.
13	WDATX	Specify if a command to be executed is accompanied by the data transfer. '0': No data transfer '1': With data transfer
12	STRMTP	When a command with a data transfer is executed, specify the transfer is either a block transfer or a stream transfer. '0': Block transfer '1': Stream transfer (Only for MMC)
11	DTRW	When a command with a data transfer is executed, set the direction of the data transfer. '0': Reading the data from the card '1': Writing the data to the card
10-9	RSPFMT1- RSPFMT0	Specify the type of the expected command response. "00": No command response "01": R1/R4/R5/R6-type command response (48 bits length) "10": R2-type command response (136 bits length) "11": R3-type command response (48 bits length without CRC)
8	BSYEXP	Specify if the command to be executed requires the BUSY signal detect. '0': BUSY signal detect is not required. '1': BUSY signal detect is required.
7	PPLEN	Specify the type of the output buffer of command lines. '0': Nonuse of push-pull drivers '1': Use of push-pull drivers
6	RSV	Reserved bit.
5-0	CMD5-CMD0	Specify the command index.

Table 49: Descriptions of the MMCCMD register bits

17.5.2 Command Response Registers (MMCCIDX and MMCRSP0-MMCRSP7)

After MMC controller issues a command, the card drives the response. The data rows of the response vary depending on the protocol mode and the type of the response.

In the MMC/SD protocol mode, one byte at the top of the response data row is stored into the MMCCIDX register. The succeeding data rows are stored into the MMCRSP7 register to MMCRSP0 register in sequence.

The following tables show how response data rows are stored into the command response registers.

Register Name	Bit position of command response	Description
MMCCIDX	[47-40]	Command index of MMC/SD command sent
MMCRSP7	[39-24]	Upper 16 bits of R1/R3/R4/R5/R6 response
MMCRSP6	[23-8]	Lower 16 bits of R1/R3/R4/R5/R6 response
MMCRSP5-0	-	No assignment

Table 50: Data assignment for R1/ R3/ R4/ R5/ R6 response in MMC mode

Register Name	Bit position of command response	Description
MMCCIDX	[135-128]	Command index of MMC/SD command sent
MMCRSP7	[127-112]	Bits [127 : 112] of R2 response
MMCRSP6	[111-96]	Bits [111 : 96] of R2 response
MMCRSP5	[95-80]	Bits [95 : 80] of R2 response
MMCRSP4	[79-64]	Bits [79 : 64] of R2 response
MMCRSP3	[63-48]	Bits [63 : 48] of R2 response
MMCRSP2	[47-32]	Bits [47 : 32] of R2 response
MMCRSP1	[31-16]	Bits [31 : 16] of R2 response
MMCRSP0	[15-0]	Bits [15 : 0] of R2 response

Table 51: Data assignment for R2 response in MMC mode

17.6 MMC Controller Interrupts

The MMC Controller can generate the interrupt requests as described in Table 52. When an interrupt event occurs, its flag bit is set in the MMCST0 register. If enable bits corresponding to each flag are set in the MMCIE register, an interrupt request is generated. All such requests are multiplexed to a single MMC interrupt request for the ARM, as shown in Figure 116.

The MMC interrupt is one of the maskable ARM interrupts (INT19). The ISR for the MMC interrupt can determine the event that caused the interrupt by checking the bits in the MMCST0 register. When MMCST0 register is read, all the bits are

automatically cleared except DXRDY bit and DRRDY bit. DXRDY bit is cleared only when transmit buffer is full. DRRDY bit is cleared when there is no data in the receive buffer.

Interrupt Request	Interrupt Event
DATEDINT	An edge was detected on the DAT3 pin.
DRRDYINT	MMCDRR is ready to be read (data received).
DXRDYINT	MMCDXR is ready to transmit new data (data transmitted).
ERRINT	A data error token was received in the SPI mode (Not applicable).
CRCRSINT	A CRC error was detected in a response from the memory card.
CRCRDINT	A CRC error was detected in the data read from the memory card.
CRCWRINT	A CRC error was detected in the data written to the memory card.
TOUTRSINT	A time-out occurred while the MMC Controller was waiting for a response to a command.
TOUTRDINT	A time-out occurred while the MMC Controller was waiting for the data from the memory card.
RSPDNEINT	For a command that requires a response: The MMC Controller has received the response without a CRC error. For a command that does not require a response: The MMC Controller has finished sending the command.
BSYDNEINT	The memory card stops or is no longer sending a busy signal when the MMC Controller is expecting a busy signal.
DATDNEINT	For read operations: The MMC Controller has received data without a CRC error. For write operations: The MMC Controller has finished sending data.

Table 52: Descriptions of the MMC Interrupt Requests

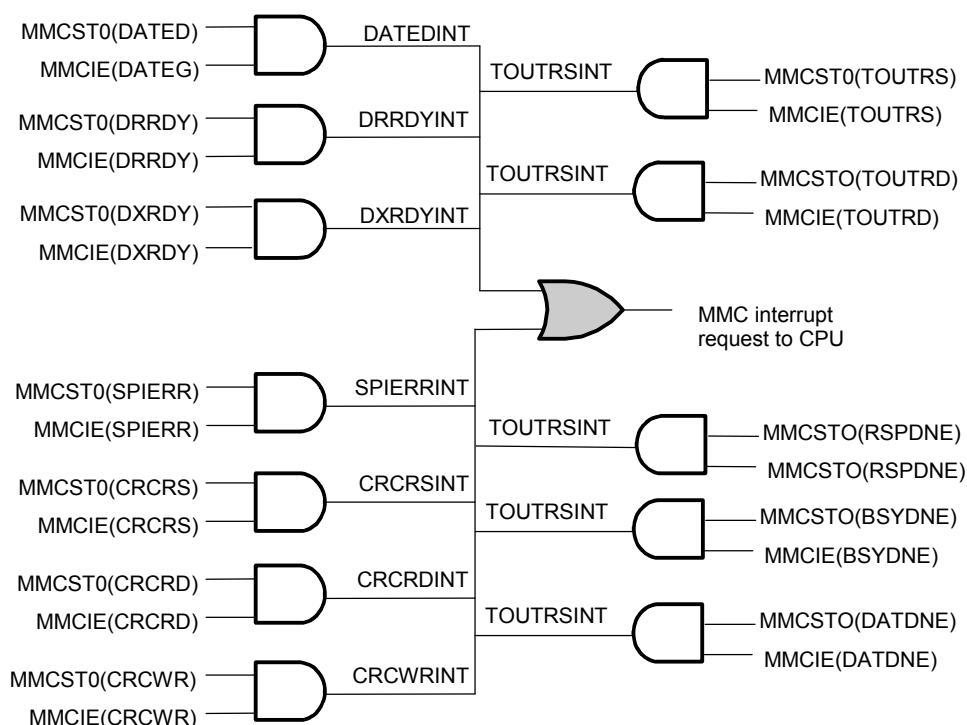


Figure 116: MMC Controller Interrupts

17.7 MMC DMA Controller

There is a DMA channel for transferring the data between the MMC Controller and SDRAM.

The MTC, SIF, MS, MMC/SD and HPIB modules in DM320 share three SDRAM DMA channels. Consequently, only up to three modules can use the channels simultaneously. The setting of the modules that use the DMA channel is performed in the SEL1, SEL2 and SEL3 fields of the DMASEL register of SDRAM Controller. To enable MMC DMA, set 0x3 to SEL1 or SEL2 or SEL3 field. The burst size of the DMA transfer can also be selected. There are three DMA burst sizes supported. The DMA burst size can be selected, by setting 0, 1, or 2 in the BURST1, BURST2 or BURST3 fields, corresponding to the channel number. For instance, to select a burst size of 32 bytes in DMA using channel 2, set 0x3 in SEL2 and 0x2 in BURST2 of DMASEL register. Refer to chapter on SDRAM Controller for more details.

The SDRAM transfer start address or the destination address is set in the MMC Controller's DMAAD0 and DMAAD1 registers. The DMA registers in MMC Controller also allow the user to suspend DMA transfers and to read transfer status information, such as the number of bytes transferred. If DATDNEINT is enabled, by setting EDATDNE bit in MMCIE register, then the ARM is interrupted (INT19) on DMA transfer completion.

Note: When DMA is used, the destination/ source SDRAM address and the size of data transfer must be 4bytes aligned.

17.8 Data Flow in the Data Registers (MMCDRR and MMCDXR)

The ARM or DMA controller can read 16 bits at a time from MMCDRR register and write 16 bits at a time to the MMCDXR register. However, the memory card is 8-bit device; it transmits or receives one byte at a time. Figure 117 and Figure 118 show how the data-size difference is handled via the data registers.

When MMCDRR register is filled with two bytes of data, the MMC Controller generates a data receive ready (DRRDY) event. If odd number of bytes of data is received, the last data byte is stored in the lower byte of MMCDRR register and then a DRRDY event is generated (Refer Figure 117).

During transmit, MMCDXR register is loaded with 2 bytes of data. When the second data byte is transmitted from MMCDXR register, a data transmit ready (DXRDY) event is generated. In case, odd number of bytes is transferred, the last byte is written right aligned to MMCDXR register (Refer to Figure 118). The transmission of the last byte data causes a DXRDY event.

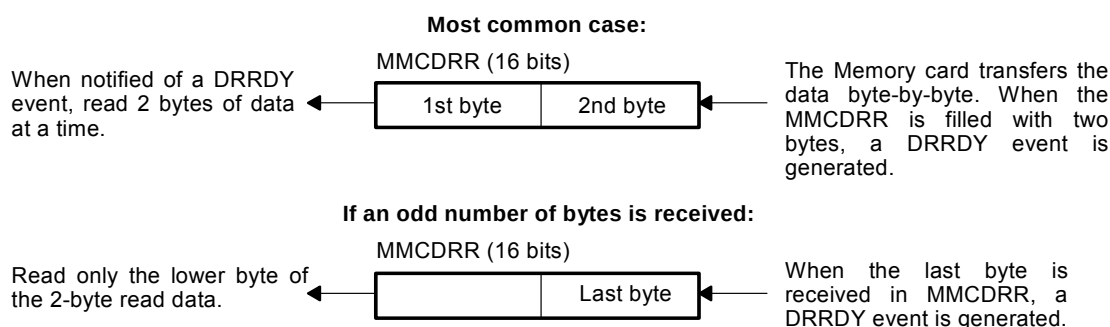


Figure 117: Data Flow in the Data Receive Register (MMCDRR)

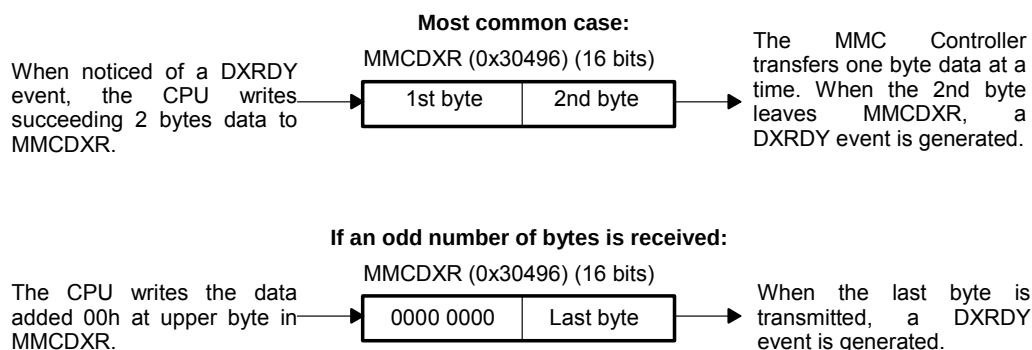


Figure 118: Data Flow in the Data Receive Register (MMCDRR)

17.9 MMC/SD Mode

The interface of the MMC Controller uses an MMC/SD protocol.

17.9.1 MMC/SD Mode Interface

Figure 119 summarizes the MMC/SD mode interface.

In the MMC/SD mode, the MMC Controller supports one or more MultiMediaCards (MMCs) / Secure Digital Memory Cards (SD cards). When multiple cards are connected, the MMC Controller selects one by using identification broadcast on the data line. The following MMC Controller pins are used:

- CMD: This pin is used for two-way communication between the connected card and the MMC Controller. The MMC Controller transmits commands to the card and the memory card drives responses to the commands on this pin.
- DAT0 or DAT [3:0]: MMC uses only one data line (DAT0) and SD Card uses one or four data lines. The number of DAT pins (the data bus width) can be set with the WIDTH field in the MMCCTL register (Refer to Figure 120).
- CLK: This pin provides the clock to the memory card from the Controller.

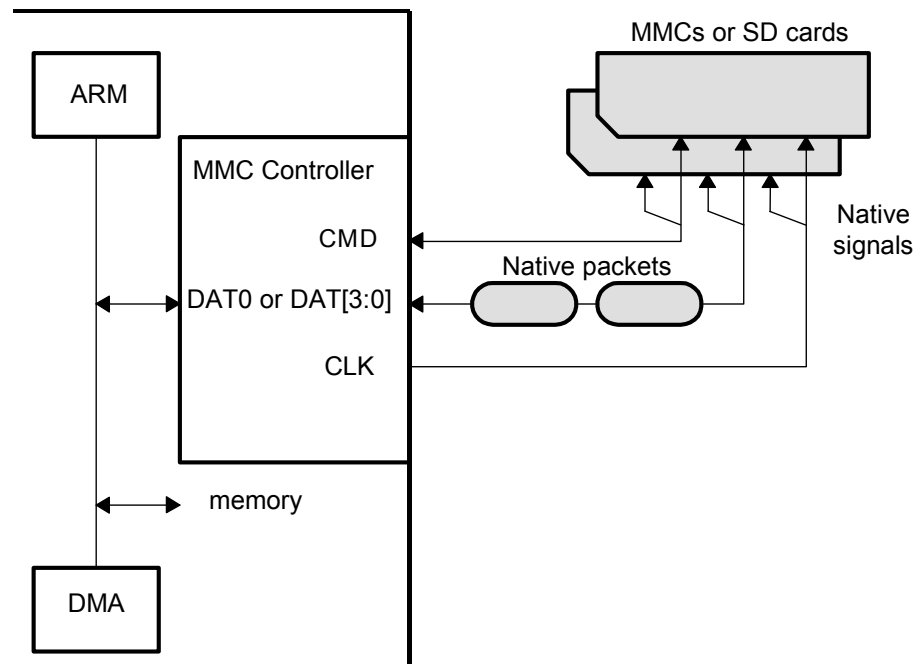


Figure 119: MMC/SD Mode Interface

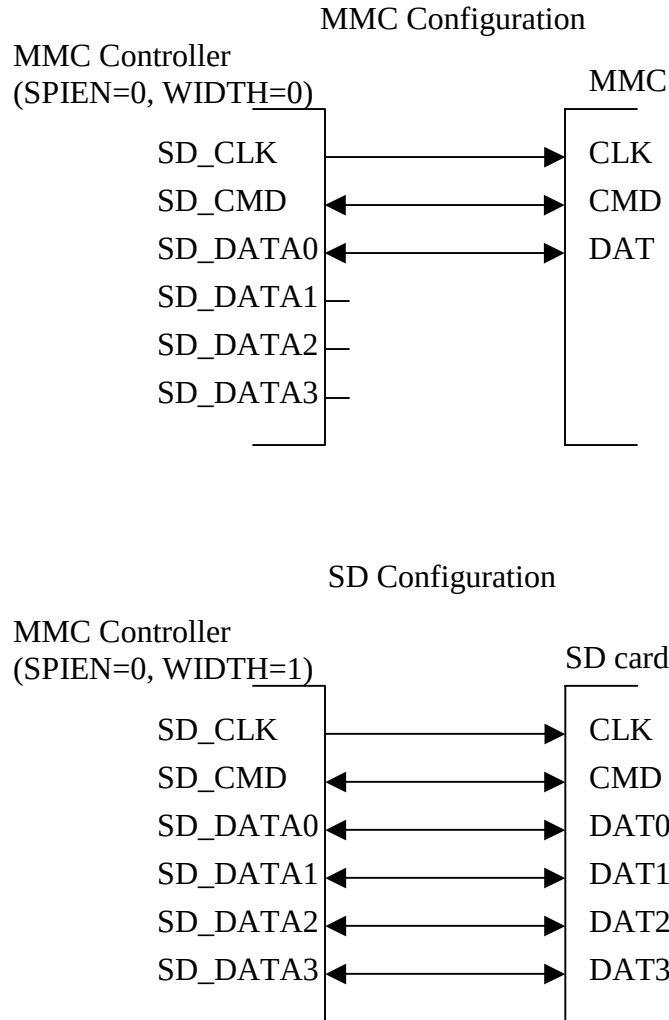


Figure 120: MMC Configuration Versus SD Configuration

17.9.2 MMC/SD Mode Write Sequence

Figure 121 and Table 53 show the signal activity when the MMC Controller is in the MMC/SD mode and is writing data to a memory card.

The same block length must be defined in the MMC Controller and in the card before initiating data write. In a successful write sequence:

- The Controller sends a write command to the card.
- The card receives the command and sends responses to the command.
- The Controller sends a block of data to the card.
- The card sends the CRC status to the Controller.
- The card sends low BUSY bit until all the data has been programmed into the flash memory inside the card.

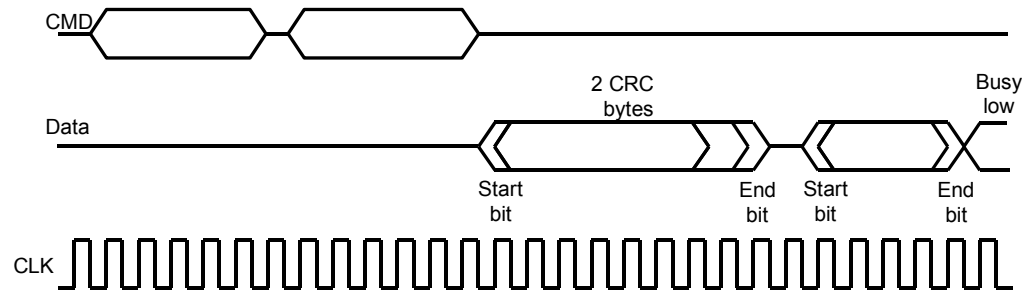


Figure 121: MMC/SD Mode Write Sequence

Portion of the Sequence	Description
WR CMD	Write command: A 6-byte WRITE_BLOCK command token is sent from the ARM to the card.
CMD RSP	Command response: The card sends a 6-byte response of type R1 to acknowledge the WRITE_BLOCK to the ARM.
DAT BLK	Data block: The ARM writes a block of data to the card. The data content is preceded by one start bit and is followed by two CRC bytes and one end bit.
CRC STAT	CRC status: The card sends a one byte CRC status information, which indicates to the ARM whether the data has been accepted by the card or rejected due to a CRC error. The CRC status information is preceded by one start bit and is followed by one end bit.
BSY	BUSY bit: The CRC status information is followed by a continuous stream of low busy bits until all of the data has been programmed into the flash memory on the card.

Table 53: MMC/SD Mode Write Sequence

17.9.3 MMC/SD Mode Read Sequence

Figure 122 and Table 54 show the signal activity when the MMC Controller is in the MMC/SD mode and is reading data from a memory card.

The same block length must be defined in the MMC Controller and in the card before initiating data read. In a successful read sequence:

- o The Controller sends a read command to the card.
- o The card drives responses to the command.
- o The card sends a block of data to the ARM.

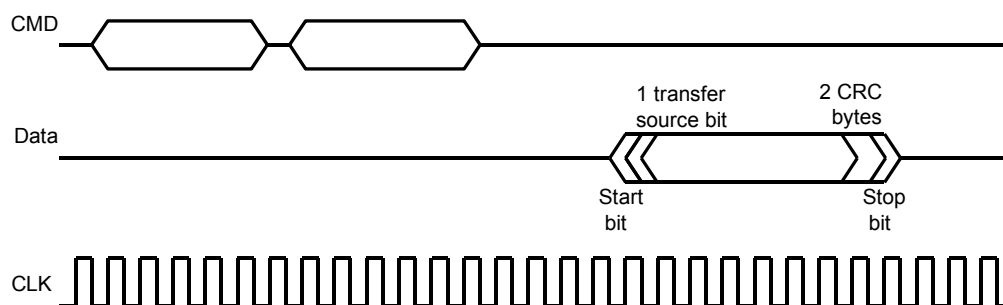


Figure 122: MMC/SD Mode Read Sequence

Portion of the Sequence	Description
RD CMD	Read command: A 6-byte READ_SINGLE_BLOCK command token is sent from the ARM to the card.
CMD RSP	Command response: The card sends a response of type R1 to acknowledge the READ_SINGLE_BLOCK command to the ARM.
DAT BLK	Data block: The card sends a block of data to the ARM. The data content is preceded by a start bit and is followed by two CRC byte and an end bit.

Table 54: MMC/SD Mode Read Sequence

17.9.4 Card Identification Operation

In the MMC/SD mode, before the MMC Controller can start data transfers from memory cards, it has to first identify how many cards are present on the bus and configure them.

For each card that responds to the ALL_SEND_CID command, the controller reads that card's unique card identification address (CID) and then assigns it a relative card address (RCA). This address is much shorter than the CID and is used by the controller to identify the card.

Only one card completes the response to ALL_SEND_CID at any one time. The absence of any response to ALL_SEND_CID indicates that all cards have been identified and configured.

The procedure for a card identification operation is as follows (see also Figure 123):

1. Use MMCCMD register to send the GO_IDLE_STATE command to the cards. This puts all cards in the idle state.
2. Use MMCCMD register to send the ALL_SEND_CID command to the cards. This tells all cards to identify themselves.
3. Wait for a card to respond. If a card responds, go to step 4). Otherwise, stop.
4. Read the CID from the response registers MMCRSP7 - MMCRSP0, and assign a relative address to the card by sending the SET_RELATIVE_ADDR command.

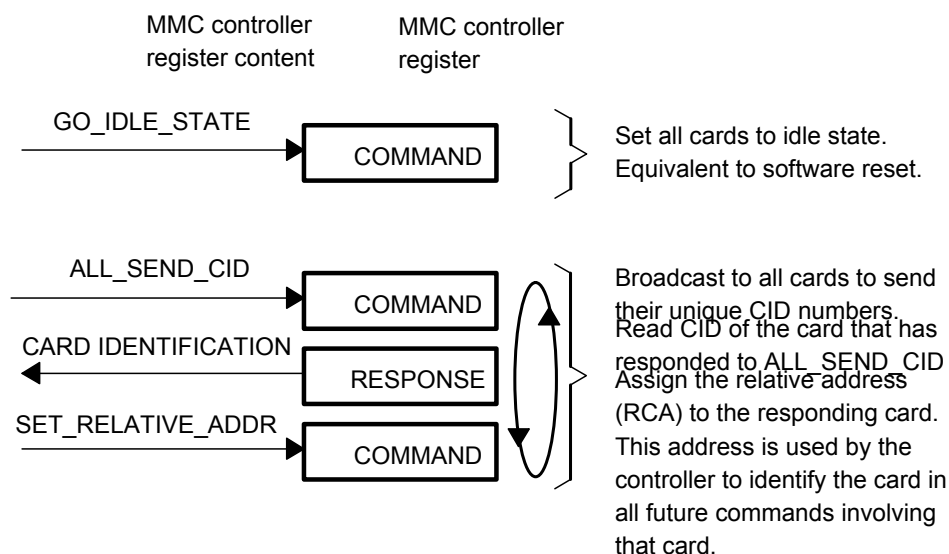


Figure 123: Card Identification (MMC/SD Mode)

17.9.5 MMC/SD Mode Single-Block Write Operation

In the MMC/SD mode, to write a single block of data to a memory card, use the following procedure. Figure 124 illustrates this procedure with some additional details. Note that the same block length must be defined in the MMC Controller and in the card. On the Controller side, set the size of one block (in bytes) to the `MMCBLEN` register. Sending `SET_BLOCK_LEN` command sets the block length in the card. Set the number of blocks to transfer as '1' in the `MMCNBLK` register.

1. Write the card's relative address to the argument registers. Load the upper address to the `MMCARGH` register and the lower address to the `MMCARGL` register.
2. Use `MMCCMD` register to send a `SELECT/ DESELECT_CARD` command. This selects the addressed card and deselects the others.
3. Write the destination (memory card) start address to the argument registers. Load the upper address to the `MMCARGH` register and the lower address to the `MMCARGL` register.
4. Send the `WRITE_BLOCK` command to the card.
5. Write the first two data bytes to `MMCDXR` register.
6. Use `MMCST0` register to check for errors. If all of the data has not been written, go to step 7. If all of the data has been written, go to step 8.

7. Write the next byte of the data block to MMCDXR register, and go to step 6.
8. Wait for the BSYDNE and DATDNE flags in MMCST0 register. This completes writing to the card. Also, check whether CRCWR flag is set, to check whether a CRC error occurred during the write.

When using DMA transfer:

When using DMA transfer in Single-Block Write operation, use the following steps.

1. Set the block size (in bytes) in the MMCBLLEN register and the number of blocks to be transferred in MMCNBLK register.
2. Write the card's relative address to the argument registers. Load the upper address to the MMCARGH register and the lower address to the MMCARGL register.
3. Use MMCCMD register to send a SELECT/ DESELECT_CARD command. This selects the addressed card and deselects the others.
4. Write the destination (memory card) start address to the argument registers. Load the upper address to the MMCARGH register and the lower address to the MMCARGL register.
5. Select the DMA channel and the DMA burst size by setting the appropriate values in DMASEL register of the SDR interface.
6. Set DCLR bit in MMCCMD register and send the WRITE_BLOCK command to the card.
7. Set the transfer source SDRAM relative address (offset from SDRAM base address) of the data to the DMAAD0 register and DMAAD1 register. The address must be aligned in 4bytes units.
8. Set '1' to both DMAE and DIR bits in the DMAMODE register.
9. Set '1' to TRG bit in the DMATRIG and trigger DMA.
10. Check the DMA completion bit RUNST, of the DMASTAT1 register and see whether DMA transfer is completed.

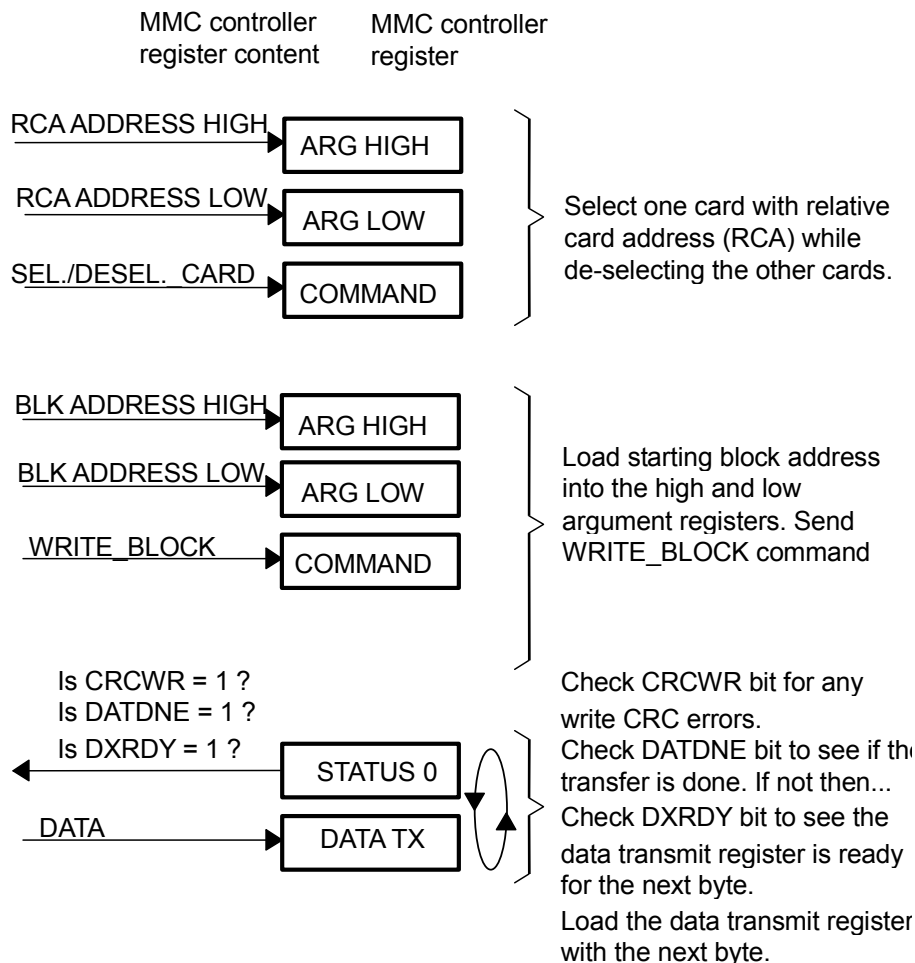


Figure 124: MMC/SD Mode Single-Block Write Operation

17.9.6 MMC/SD Mode Single-Block Read Operation

In the MMC/SD mode, to read a single block of data from a memory card, use the following procedure. Figure 125 illustrates this procedure with some additional details. Note that the same block length must be defined in the MMC Controller and in the card. On the Controller side, set the size of one block (in bytes) to the **MMCBLEN** register. Sending **SET_BLOCK_LEN** command sets the block length in the card. Set the number of blocks to transfer as '1' in the **MMCNBLK** register.

The following sequence of steps is required for performing a single-block read operation:

1. Write the card's relative address to the argument registers. Load the upper address to the **MMCARGH** register and the lower address to the **MMCARGL** register.

2. Use MMCCMD register to send a SELECT/ DESELECT_CARD command. This selects the addressed card and deselects the others.
3. Set the block size (in bytes) in the MMCBLLEN register and the number of blocks to be transferred in MMCNBLK register.
4. Write the source (memory card) start address to the argument registers. Load the upper address to the MMCARGH register and the lower address to the MMCARGL register.
5. Send a READ_SINGLE_BLOCK command using the MMCCMD register.
6. Use the MMCST0 register to check for errors and to determine whether a word/byte of data is available to be read. The DRRDY bit of MMCST0 register is set when a new data is available.
7. When DRRDY is set, read data from the MMCDRR register. If all of the data has not been read, go to Step 6. If all of the data has been read, go to Step 8.
8. Check whether the CRCRD bit of MMCST0 register is set indicating a CRC error. If there is no CRC error, check whether the DATDNE bit of the MMCST0 register is set, signifying the successful completion of the read operation.

When using DMA transfer:

When using DMA transfer in Single-Block Write operation, use the following steps.

1. Set the block size (in bytes) in the MMCBLLEN register and the number of blocks to be transferred in MMCNBLK register.
2. Write the card's relative address to the argument registers. Load the upper address to the MMCARGH register and the lower address to the MMCARGL register.
3. Use MMCCMD register to send a SELECT/ DESELECT_CARD command. This selects the addressed card and deselects the others.
4. Write the source (memory card) start address to the argument registers. Load the upper address to the MMCARGH register and the lower address to the MMCARGL register.
5. Select the DMA channel and the DMA burst size by setting the appropriate values in DMASEL register of the SDR interface.
6. Set DCLR bit in MMCCMD register and send the READ_SINGLE_BLOCK command to the card.
7. Set the transfer destination SDRAM relative address (offset to SDRAM base address) of the data to the DMAAD0 register and DMAAD1 register. The address must be aligned in 4bytes units.
8. Set '1' to DMAE bit and '0' to DIR bit in the DMAMODE register.

9. Set '1' to TRG bit in the DMATRG and trigger DMA.
10. Check the DMA completion bit RUNST, of the DMASTAT1 register and see whether DMA transfer is completed.

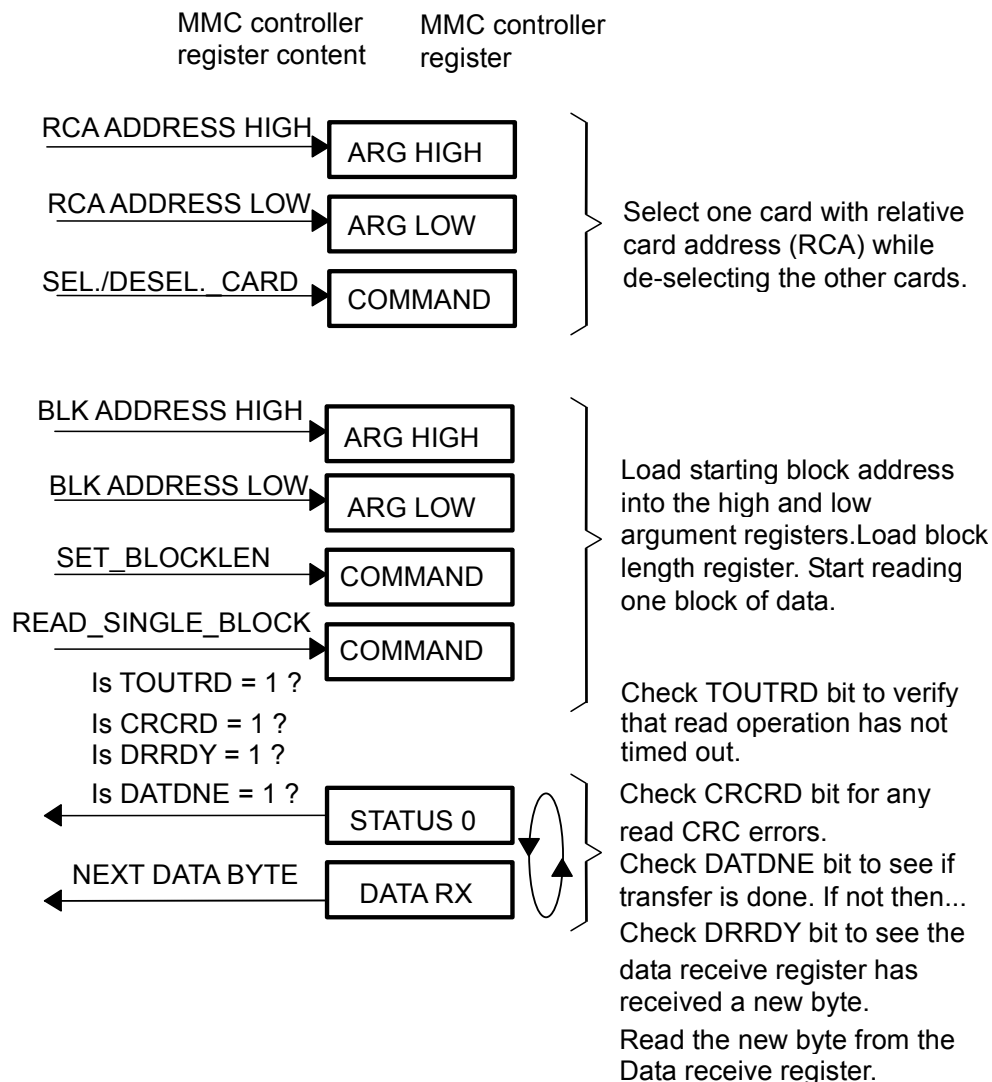


Figure 125: MMC/SD Mode Single-Block Read Operation

17.9.7 MMC/SD Mode Multiple-Block Write Operation

In the MMC/SD mode, multiple blocks can be written to a MMC/SD card using the sequence of steps enumerated below. The procedure is also illustrated in Figure 126 with some additional details. The same block length must be defined in the MMC Controller and in the card.

Note: The procedure in this section uses a STOP_TRANSMISSION command to end the block transfer. A multiple-block operation can terminate itself if the exact number of blocks to be transferred, is specified in the MMCNBLK register. In addition, setting 0 to this register can continue to transfer until a STOP_TRANSMISSION command is issued.

1. Set the block size (in bytes) in the MMCBLEN register
2. Write the card's relative address to the argument registers. Load the upper address to the MMCARGH register and the lower address to the MMCARGL register.
3. Use MMCCMD register to send a SELECT/ DESELECT_CARD command. This selects the addressed card and deselects the others.
4. Write the destination (memory card) start address to the argument registers. Load the upper address to the MMCARGH register and the lower address to the MMCARGL register.
5. Send the WRITE_MULTIPLE_BLOCK command to the card using the MMCCMD register.
6. Write the first two data bytes to MMCDXR register.
7. Use MMCST0 register to check for errors. If all of the data has not been written, go to step 8. If all of the data has been written, go to step 9.
8. Write the next byte of the data block to MMCDXR register, and go to step 7.
9. Send the STOP_TRANSMISSION command to the card using the MMCCMD register.
10. Wait for the BSYDNE and DATDNE flags in MMCST0 register, which completes writing to the card. Also, check whether CRCWR flag is set, to check whether a CRC error occurred during the write.

When using DMA transfer:

When using DMA transfer in Single-Block Write operation, use the following steps.

1. Set the block size (in bytes) in the MMCBLEN register. Set the number of blocks to be written in MMCNBLK register.
2. Write the card's relative address to the argument registers. Load the upper address to the MMCARGH register and the lower address to the MMCARGL register.
3. Use MMCCMD register to send a SELECT/ DESELECT_CARD command. This selects the addressed card and deselects the others.

4. Write the destination (memory card) start address to the argument registers. Load the upper address to the MMCARGH register and the lower address to the MMCARGL register.
5. Select the DMA channel and the DMA burst size by setting the appropriate values in DMASEL register of the SDRC interface.
6. Set DCLR bit in MMCCMD register and send the WRITE_MULTIPLE_BLOCK command to the card.
7. Set the transfer source SDRAM relative address (offset to the SDRAM base address) of the data to the DMAAD0 register and DMAAD1 register. The address must be aligned in 4bytes units.
8. Set '1' to both DMAE and DIR bits in the DMAMODE register.
9. Set '1' to TRG bit in the DMATRIG and trigger DMA.
10. Check the DMA completion bit RUNST, of the DMASTAT1 register and see whether DMA transfer is completed.

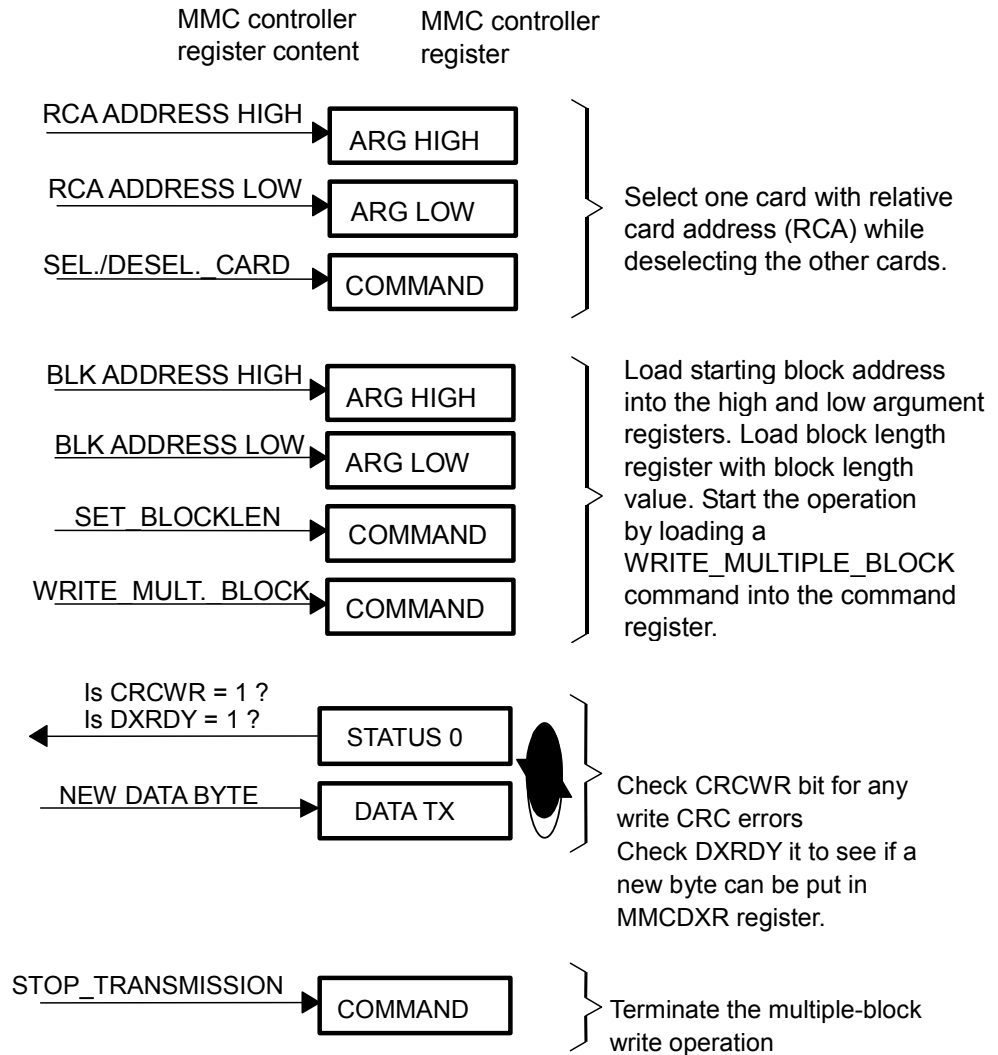


Figure 126: MMC/SD Multi-Block Write Operation

17.9.8 MMC/SD Mode Multiple-Block Read Operation

In the MMC/SD mode, multiple blocks of data can be read from a MMC/SD card, using the sequence of steps enumerated below. The procedure is also illustrated in Figure 127 with some additional details. The same block length must be defined in the MMC Controller and in the card.

Note: The procedure in this section uses a STOP_TRANSMISSION command to end the block transfer. A multiple-block operation can terminate itself if the exact number of blocks to be transferred, is specified in the MMCNBLK register. In addition, setting 0 to this register can continue to transfer until a STOP_TRANSMISSION command is issued.

1. Write the card's relative address to the argument registers. Load the upper address to the MMCARGH register and the lower address to the MMCARGL register.

2. Use MMCCMD register to send a SELECT/ DESELECT_CARD command. This selects the addressed card and deselects the others.
3. Set the block size (in bytes) in the MMCBLLEN register and the number of blocks to be transferred in MMCNBLK register.
4. Write the destination start address to the argument registers. Load the upper address to the MMCARGH register and the lower address to the MMCARGL register.
5. Send a READ_MULTIPLE_BLOCK command using the MMCCMD register.
6. Use the MMCST0 register to check for errors and to determine whether a word/byte of data is available to be read. The DRRDY bit of MMCST0 register is set when a new data is available.
7. When DRRDY is set, read data from the MMCDRR register. If all of the data has still not been read, go to Step 6. If all of the data has been read, go to Step 8.
8. Send the STOP_TRANSMISSION command to the card using the MMCCMD register.
9. Check whether the CRCRD bit of MMCST0 register is set indicating a CRC error. If there is no CRC error, check whether the DATDNE bit of the MMCST0 register is set, signifying the successful completion of the read operation.

When using DMA transfer:

When using DMA transfer in Single-Block Write operation, use the following steps.

1. Set the block size (in bytes) in the MMCBLLEN register and the number of blocks to be transferred in MMCNBLK register.
2. Write the card's relative address to the argument registers. Load the upper address to the MMCARGH register and the lower address to the MMCARGL register.
3. Use MMCCMD register to send a SELECT/DESELECT_CARD command. This selects the addressed card and deselects the others.
4. Write the source (memory card) start address to the argument registers. Load the upper address to the MMCARGH register and the lower address to the MMCARGL register.
5. Select the DMA channel and the DMA burst size by setting the appropriate values in DMASEL register of the SDRC interface.
6. Set DCLR bit in MMCCMD register and send the READ_SINGLE_BLOCK command to the card.

7. Set the transfer destination SDRAM relative address (offset to SDRAM base address) of the data to the DMAAD0 register and DMAAD1 register. The address must be aligned in 4bytes units.
8. Set '1' to DMAE bit and '0' to DIR bit in the DMAMODE register.
9. Set '1' to TRG bit in the DMATRIG and trigger DMA.
10. Check the DMA completion bit RUNST, of the DMASTAT1 register and see whether DMA transfer is completed.
11. Send the STOP_TRANSMISSION command to the card using the MMCCMD register.

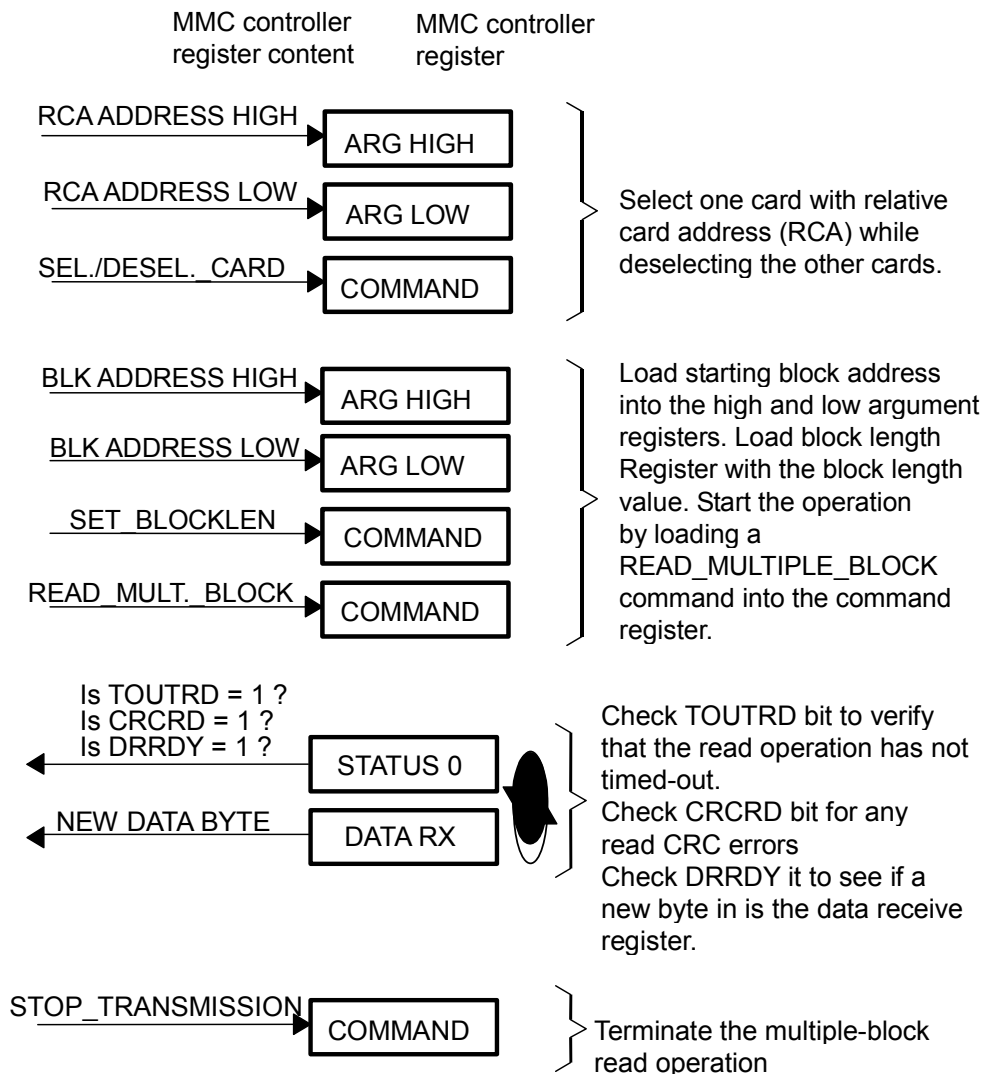


Figure 127: MMC/SD Mode Multiple-Block Read Operation

17.9.9 MMC/SD Mode Initialization

The general procedure for initializing the MMC Controller is given in the following steps. Details about the registers or register bit fields to be configured in the MMC/SD mode are in the subsequent subsections.

1. Place the MMC Controller in its reset state by setting '1' to CMDRST bit and DATRST bit in the MMCCTL register. After the reset, other bits in MMCCTL register can be set.
2. Write the required values to other registers to complete the MMC Controller configuration.
3. Clear CMDRST bit and DATRST bit to release the MMC Controller from its reset state. It is recommended not to rewrite the values written to the other bits of MMCCTL register in step 1.
4. Enable the CLK pin so that the memory clock is sent to the memory card. Enable the CLK pin by setting CLEN bit in MMCCLK register.

17.9.9.1 Initializing the MMC Control Register (MMCCTL)

When operating the MMC Controller in the MMC/SD mode, the bit fields named in the MMCCTL register affect the operation of the controller. The subsections that follow explain the procedure for initialization of each of MMCCTL register's bit fields.

The DATED[1:0] bits in MMCCTL register enable or disable edge detection on the DAT3 pin. If edge detection is enabled and an edge is detected, the DATED flag bit of MMCST0 register is set. In addition, if EDATED bit = 1 in MMCIE register, an interrupt request is generated.

In the MMC/SD mode, the MMC Controller must know the width of the data bus for the memory card that is connected. The WIDTH bit of MMCCTL register is used to specify whether the MMC/SD card operates with a one-bit or four-bit data bus. If an MMC is connected, only a 1-bit data bus (WIDTH = 0) can be used. If an SD card is connected, either a 1-bit (WIDTH = 0) or a 4-bit data bus (WIDTH = 1) can be configured.

To place the MMC Controller in its reset state and disable it, set the CMDRST and DATRST bits of MMCCTL register. The first step of the MMC Controller initialization process is to disable both the command and data logic. After initialization is complete, but prior to enabling of the CLK pin, enable the MMC Controller by clearing the CMDRST and DATRST bits.

17.9.9.2 Initializing the Clock Controller Registers (DIV3 & MMCCLK)

The function clock is generated in the Clock Controller Module and configured by setting MMC field of the CLKC's DIV3 register. The programmable clock divider in the CLKC module divides down the AHB clock to produce the function clock. The function clock is given by the following equation.

$$\text{Function clock frequency} = \text{AHB clock frequency} / (\text{MMC} + 1)$$

A clock divider in the MMC Controller divides down the function clock to produce the memory clock. Load the divider value into the CLKRT field of MMCCLK register. The resulting frequency is:

$$\text{Memory clock frequency} = \text{Function clock frequency} / (2 * (\text{CLKRT} + 1))$$

The CLEN bit of MMCCLK register determines whether the memory clock appears on the CLK pin anytime. If this bit is set as '0', the memory clock is not provided.

For details of the clock setting, refer section 17.4.

17.9.9.3 Initializing the Interrupt Mask Register (MMCIE)

The bits in MMCIE register individually enable or disable the interrupt requests. Set the relevant bits in MMCIE register to enable the associated interrupts. Clear the relevant bits in MMCIE register to disable the associated interrupts. Disable the interrupts that are not used in the MMC/SD mode.

17.9.9.4 Initializing the Time-Out Registers (MMCTOR and MMCTOD)

A time-out period for MMC/SD command responses can be specified using MMCTOR register. Similarly, a time-out period for reading data from MMC/SD can be specified using MMCTOD register.

When the MMC Controller sends a command to a memory card, it often must wait for a response. The controller can wait indefinitely or for up to 255 memory clock cycles. If the MMCTOR register is loaded with 0, the controller will wait for a response indefinitely. If a nonzero value is loaded into MMCTOR register, the controller stops waiting after the specified number of cycles and then sets automatically a response time-out state to MMCST0 register. If the associated interrupt request is enabled, the MMC Controller also sends an interrupt request to the ARM.

When the MMC Controller requests data from a memory card, it can wait indefinitely for that data, or it can stop waiting after a programmable number of cycles. If the MMCTOD register is loaded with 0, the controller will wait indefinitely. If a nonzero value is loaded into MMCTOD register, the controller will wait for the specified number of memory clock cycles and then record a data-read time-out state in MMCST0 register. If the associated interrupt request is enabled, the controller also sends an interrupt request to the ARM.

17.9.9.5 Initializing the Data Block Registers (MMCBLEN and MMCNBLK)

The number of bytes in a data block must be specified for block read and write operations in MMC/SD. The number of blocks must also be specified in case of multiple block transfers. The block length is specified in MMCBLEN register and the number of blocks is specified in MMCNBLK register.

In MMCBLEN register, the size for each block of data transferred between the MMC Controller and a memory card must be specified. The valid size depends on the type of read/ write operation. A length of 0 bytes is prohibited.

For multiple-block transfers, the number of blocks of data, to be transferred between the MMC Controller and a memory card, must be specified. Loading 0 into MMCNBLK register specifies an infinite number of blocks to be transferred. When MMCNBLK = 0, the MMC Controller will continue to transfer data blocks until the transfer is stopped with a STOP_TRANSMISSION command. If a specific number of blocks are to be transferred, load MMCNBLK register with a value from 1 through 65535.

17.9.9.6 Initializing SDIO Specific Registers (SDIOCTL and SDIOIEN)

If the card is in the SDIO mode, the SDIO specific registers of the MMC/SD interface need to be initialized. These can be set during the initialization phase.

If the SDIO card is required to support Read Wait operation, set the RDWTRQ bit of the SDIOCTL register to '1'. If the Read Wait operation is to be triggered by CRC errors, set the RDWTCR bit of SDIOCTL register to '1'.

If the SDIO card is to support interrupt mode operation, set the IOINTEN bit of the SDIOIEN register to '1'. If the onset of Read Wait operation is to trigger an interrupt, set the RWSEN bit of the SDIOIEN register to '1'.

17.9.10 Monitoring Activity in the MMC/SD Mode

This section describes registers and specific register bits that can be used to obtain the status of the MMC Controller in the MMC/SD mode. The status of the MMC Controller can be determined by reading the bits in MMCST0 register and MMCST1 register.

17.9.10.1 Detecting Edges and Level Changes on the DAT3 Pin

Detecting edges - The MMC Controller sets the DATED flag of MMCST0 register if DAT3 edge detection is enabled and the edge is detected. DAT3 edge detection is enabled when DATED[1:0] are nonzero in MMCCTL register. If the interrupt request is enabled by setting EDATED bit in MMCIE register, the ARM can also be notified of the DAT3 by an interrupt.

Detecting level changes - DAT3ST bit in the MMCST1 register tracks the signal level on the DAT3 pin.

17.9.10.2 Determining Whether New Data is Available in MMCDRR

The MMC Controller sets the DRRDY flag of MMCST0 register when new data is received in the MMCDRR register. If the interrupt request is enabled by setting EDRRDY bit in MMCIE register, the ARM can also be notified of the event by an interrupt.

17.9.10.3 Verifying that MMCDXR is Ready to Accept New Data

The MMC Controller sets the DXRDY flag of MMCST0 register when data is transmitted from the MMCDXR register. If the interrupt request is enabled by setting EDXRDY bit in MMCIE register, the ARM can also be notified of the event by an interrupt.

17.9.10.4 Checking for CRC Errors

The MMC Controller sets CRCRS/CRCRD/CRCWR bits in MMCST0 register when there are CRC errors in command response / data read / data write. If the interrupt request is enabled by setting ECRCRS/ECRCRD/ECRCWR bit in MMCIE register, the ARM can also be notified of the CRC error by an interrupt.

17.9.10.5 Checking for Time-Out Events

The MMC Controller sets TOUTRS/TOUTRD bits in MMCST0 register when there is command response/data read time-out event. If the interrupt request is enabled by setting ETOUTRS/ETOUTRD bit in MMCIE register, the ARM can also be notified of the event by an interrupt.

17.9.10.6 Determining When a Response/Command is Done

The MMC Controller sets the RSPDNE bit of MMCST0 register when the response is done. RSPDNE bit is also set when the command is done, in the case of commands that do not require a response. If the interrupt request is enabled by setting ERSPDNE bit in MMCIE register, the ARM can also be notified of the done condition by an interrupt.

17.9.10.7 Determining Whether the Memory Card is Busy

The card sends a busy signal to the MMC controller either waiting for an R1b-type response or data transfer completion. The MMC Controller has two flags to indicate whether the memory card is sending a busy signal. The two flags are complements of each other:

- BSYDNE bit of MMCST0 register is set if the card did not send earlier or currently not sending a 'busy signal' when the Controller is expecting a busy signal (BSYEXP = 1 in MMCCMD register). The busy interrupt can be enabled by setting EBSYDNE bit in MMCIE register.
- BUSY bit in MMCST1 register is set when a busy signal is received from the card.

17.9.10.8 Determining Whether a Data Transfer is done

The MMC Controller sets the DATDNE bit when all the bytes of a data transfer have been transmitted/ received. If the interrupt request is enabled by setting EDATDNE bit in MMCIE register, the ARM can also be notified of the time-out event by an interrupt.

17.9.10.9 Checking For a Data Transmit Empty Condition

A data value sent from MMC controller is passed from MMCDXR register to the data transmit shift register during transmission. Then this value is passed from this shift register to the memory card one bit at a time. The DXEMP bit in MMCST1 register indicates when this shift register is empty.

Typically, DXEMP bit in MMCST1 register is not used to control data transfers, instead it is being checked during recovery from an error condition. There is no interrupt associated with this bit.

17.9.10.10 Checking for a Data Receive Full Condition

During reception, the shift register accepts a data value one bit at a time from the MMC/SD card. Then the value is passed from this shift register to the MMCDRR register. The DRFUL bit in MMCST1 register indicates when this shift register is full and no new bits can be shifted in from the memory card.

Typically, DRFUL bit in MMCST1 register is not used to control data transfers, instead it is checked during recovery from an error condition. There is no interrupt associated with this bit.

17.9.10.11 *Checking the Status of the CLK Pin*

To determine whether the memory clock has been stopped on the CLK pin, CLKSTP bit in MMCST1 register is to be read.

17.9.10.12 *Getting the Remaining Block Count During a Multiple-Block Transfer*

During a transfer of multiple data blocks, MMCNBLC register value specifies the number of blocks, yet to be transferred. This register is read-only.

17.10 SDIO Card Function

To support the SDIO Card, the following features are included additionally to the typical MMC/SD card controller

- Read Wait Interrupt
- SDIO Card Interrupt

These interrupts are connected to the Interrupt Controller module through INT20. Refer to chapter on Interrupt Controller for more details.

Refer to section 17.9.9.6 for configuring SDIO mode registers.

17.11 MMC/SD Card Interface Register Map

0x00	0003:0480	MMCTL	MMC Control Register
0x01	0003:0482	MMCLK	MMC memory clock Control Register
0x02	0003:0484	MMCST0	MMC Status Register #0
0x03	0003:0486	MMCST1	MMC Status Register #1
0x04	0003:0488	MMCE	MMC Interrupt Enable Register
0x05	0003:048A	MMCTOR	MMC Response Time-Out Register
0x06	0003:048C	MMCTOD	MMC Data Read Time-Out Register
0x07	0003:048E	MMCBLEN	MMC Block Length Register
0x0A	0003:0494	MMCNBLK	MMC Number of Blocks Register
0x0B	0003:0496	MMCNBLC	MMC Number of Blocks Counter
0x0A	0003:0494	MMCDRR	MMC Data Receive Register
0x0B	0003:0496	MMCDXR	MMC Data Transmit Register
0x0E	0003:049C	MMCCMD	MMC Command Register
0x0F	0003:049E	MMCARGL	MMC Argument Register Low
0x0E	0003:049C	MMCARGH	MMC Argument Register High
0x0F	0003:049E	MMCRSP0	MMC Response Register #0
0x10	0003:04A0	MMCRSP1	MMC Response Register #1
0x11	0003:04A2	MMCRSP2	MMC Response Register #2
0x12	0003:04A4	MMCRSP3	MMC Response Register #3
0x13	0003:04A6	MMCRSP4	MMC Response Register #4
0x14	0003:04A8	MMCRSP5	MMC Response Register #5
0x15	0003:04AA	MMCRSP6	MMC Response Register #6
0x16	0003:04AC	MMCRSP7	MMC Response Register #7
0x17	0003:04AE	MMCDRSP	MMC SPI Data Response Register
0x18	0003:04B0	MMCETOK	MMC SPI Error Token Register
0x19	0003:04B2	MMCCDX	MMC Command Index Register
0x1A	0003:04B4	MMCKC	MMC Clock start phase control Register
0x1B	0003:04B6	MMCTORC	MMC Response Time-Out count Register
0x1C	0003:04B8	MMCTODC	MMC Data Read Time-Out count Register
0x1D	0003:04BA	MMCBLNC	MMC Block length counter Register
0x1E	0003:04BC	DMATRG	DMA Trigger Register
0x1F	0003:04BE	DMAMODE	DMA Mode Register
0x20	0003:04C0	DMAAD0	DMA Address Low
0x21	0003:04C2	DMAAD1	DMA Address High
0x22	0003:04C4	DMASTAT0	DMA Status Register #0
0x2C	0003:04D8	DMASTAT1	DMA Status Register #1
0x2D	0003:04DA	DMATOR	DMA Time-Out Register
0x25	0003:04CA	SDICTL	SDIO Control Register
0x26	0003:04CC	SDIST0	SDIO Status Register #0
0x27	0003:04CE	SDIEN	SDIO Interrupt Enable Register
0x28	0003:04D0	SDIST	SDIO Interrupt Status Register

17.12 MMC/SD Card Interface Registers

17.12.1 MMCCTL

MMC Control Register

MMCCTL	0003:0480		offset: 0x00		default: 0x0000											
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	PERMDX	PERMDR	RSV	DATED[1]	DATED[0]	RSV	RSV	RSV	DATRST	0	0
R/W	-	-	-	-	-	R/W	R/W	-	R/W	R/W	-	-	-	R/W	R/W	R/W
Default	-	-	-	-	-	0	0	-	0	0	-	-	-	0	-	-

Bit	Name	Reset Value	R/W	Function
15:11	RSV			<u>Reserved</u>
10	PERMDX	0	R/W	<u>Endian select Enable when writing</u> » 0: Little Endian « » 1: Big Endian «
9	PERMDR	0	R/W	<u>Endian select Enable when reading</u> » 0: Little Endian « » 1: Big Endian «
8	RSV			<u>Reserved</u>
7:6	DATED	0	R/W	<u>DAT3 Edge Detection Select Bits</u> » 00: DAT3 edge detection is disabled « » 01: DAT3 rising edge detection is enabled « » 10: DAT3 falling edge detection is enabled « » 11: DAT3 dual edge detection is enabled «
5:3	RSV			<u>Reserved</u>
2	WDTH	0	R/W	<u>Data Bus Width (only in native MMC mode)</u> » 0: Data bus has 1 bit (only DAT0 is used) « » 1: Data bus has 4 bits (DAT0-DAT3 are used) « Note: This is applicable to SD cards only
1	CMDRST	0	R/W	<u>CMD (command) Logic Reset</u> » 0: CMD line portion is enabled « » 1: CMD line portion is disabled and in reset state «
0	DATRST	0	R/W	<u>DAT (data) Logic Reset</u> » 0: DAT line portion is enabled « » 1: DAT line portion is disabled and in reset state «

17.12.2 MMCCLK

MMC Clock Register

MMCCLK	0003:0482			offset: 0x02			default: 0x00FF									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	CLEN	CLKRT[7]	CLKRT[6]	CLKRT[5]	CLKRT[4]	CLKRT[3]	CLKRT[2]	CLKRT[1]	CLKRT[0]
R/W	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	-	-	0	1	1	1	1	1	1	1	1

Bit	Name	Reset Value	R/W	Function
15:9	RSV			Reserved
8	CLEN	0	R/W	<u>Clock Enable</u> » 0: Disable MMC Clock Output (MMC_CLK), set output low « » 1: Enable MMC Clock Output (MMC_CLK) «
7:0	CLKRT	255	R/W	<u>MMC Clock Rate</u> Divider rate for clock: $\text{MMC Clock} = (\text{Function Clock}) / (2 * (\text{CLKRT} + 1))$ The Function Clock rate is set in the MMCFCLK register. The MMC clock is output on the pin named MMC_CLK. Note: MMC/SD Clock Freq $\geq 1 / ((\text{ARM Clock cycles} \times 3) + \text{SDRAM Clock cycles} \times 11)$

17.12.3 MMCST0

MMC Status Register #0

MMCST0	0003:0484			offset: 0x04			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	DATED	CRCRD	CRCWR	TOUTRS	TOUTRD	RSPDNE	BSYDNE	DATDNE	0	RSPDNE	BSYDNE	DATDNE
R/W	-	-	-	-	R	R	R	R/W	R	R	R	R	R	R	R	R
Default	-	-	-	-	0	0	-	-	-	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:12	RSV			Reserved
11	DATED	0	R	<u>DAT3 Edge Detect bit</u> » 0: No DAT3 edge detected « » 1: DAT3 edge has been detected «
10	DRRDY	0	R	<u>Data Receive Ready Bit</u> » 0: MMCDRR not ready « » 1: MMCDRR ready - new data has arrived « Data and can be read by the CPU or by the DMA controller. DRRDY is cleared to 0 when the DAT logic is reset (DATRST=1), when a command is sent with data receive/transmit clear (DCLR=1), or when data is read from MMCDRR
9	DXRDY	1	R	<u>Data Transmit Ready Bit</u> » 0: MMCDXR not ready « » 1: MMCDXR ready - data in MMCDXR has been transmitted « MMCDXR can accept new data from the CPU or from the DMA controller. DXRDY is cleared to 0 when the DAT logic is reset (DATRST=1), when a command is sent with data receive/transmit clear (DCLR=1), or when data is written to MMCDXR.
8	RSV			Reserved
7	CRCRS	0	R	<u>Response CRC Error Bit</u> » 0: No CRC error is detected « » 1: CRC error has been detected «
6	CRCRD	0	R	<u>Read-Data CRC Error Bit</u> » 0: No CRC error is detected « » 1: CRC error has been detected «
5	CRCWR	0	R	<u>Write-Data CRC Error Bit</u> » 0: No CRC error is detected « » 1: CRC error has been detected «
4	TOUTRS	0	R	<u>Response Time-Out Bit</u> » 0: No time-out event « » 1: A time-out event has occurred while the MMC controller was waiting for a response to a command «
3	TOUTRD	0	R	<u>Read-Data Time-Out Bit</u> 0: No time-out event « 1: A time-out event has occurred while the MMC controller was waiting for data «
2	RSPDNE	0	R	<u>Command/Response Done Bit</u> If the command requires a response: 0: Response not received 1: Response received with no CRC error If no response required: 0: Command not sent 1: Command has been sent
1	BSYDNE	0	R	<u>Busy Done Bit</u> » 0: The memory card is busy « » 1: The memory card is not busy « The busy done bit is used for commands with R1b response (CMD38) to indicate card is busy. BSYDNE is set to indicate card is no longer busy.
0	DATDNE	0	R	<u>Data Transfer Done Bit</u> When reading from memory card with CPU or DMA 0: Read operation not done 1: Data received with no CRC error When writing to memory card with CPU or DMA: 0: Write operation not done 1: Data transmitted

17.12.4 MMCST1

MMC Status Register #1

MMCST1	0003:0486			offset: 0x06			default: 0x0002									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	DAT3ST	DRFUL	DXEMP	CLKSTP	BUSY
R/W	-	-	-	-	-	-	-	-	-	-	-	R	R	R	R	R
Default	-	-	-	-	-	-	-	-	-	-	-	0	0	0	1	0

Bit	Name	Reset Value	R/W	Function
15:5	RSV			<u>Reserved</u>
4	DAT3ST	0	R	<u>DAT3 Status Bit</u> » 0: Low signal level on DAT3 pin « » 1: High signal level on DAT3 pin «
3	DRFUL	0	R	<u>DRRDY Data Receive Full</u> » 0: MMCDRR (data receive register) is not full « » 1: MMCDRR (data receive register) is full «
2	DXEMP	0	R	<u>DXRDY Data Transmit Empty</u> » 0: MMCDXR (data transmit register) is not empty « » 1: MMCDXR (data transmit register) is empty «
1	CLKSTP	1	R	<u>Clock Stop Status Bit</u> » 0: CLK is active « » 1: CLK is held low « Note: Possible reasons are Manual stop (CLKEN=0), data receive full condition, or data transmit empty condition.
0	BUSY	0	R	<u>Busy Bit</u> » 0: No busy signal detected from memory card « » 1: Busy signal detected (memory card is busy) «

17.12.5 MMCIE

MMC Interrupt Enable Register

MMCIE	0003:0488				offset: 0x08				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	EDATED	EDRRDY	EDXRDY	RSV	ECRCRS	ECRCRD	ECRCWR	ETOUTRS	ETOUTRD	ERSPDNE	EBSYDNE	EDATDNE
R/W	-	-	-	-	R/W	R/W	R/W	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	0	0	0	-	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:12	RSV			Reserved
11	EDATED	0	R/W	<u>DAT3 Edge Detect Interrupt Enable</u> » 0: DAT3 edge detect interrupt is disabled « » 1: DAT3 edge detect interrupt is enabled «
10	EDRRDY	0	R/W	<u>Data Receive Ready Interrupt Enable</u> » 0: Data receive ready interrupt is disabled « » 1: Data receive ready interrupt is enabled «
9	EDXRDY	0	R/W	<u>Data Transmit Ready Interrupt Enable</u> » 0: Data transmit ready interrupt is disabled « » 1: Data transmit ready interrupt is enabled «
8	RSV			Reserved
7	ECRCRS	0	R/W	<u>Response CRC Error Interrupt Enable</u> » 0: Response CRC error interrupt is disabled « » 1: Response CRC error interrupt is enabled «
6	ECRCRD	0	R/W	<u>Read-Data CRC Error Interrupt Enable</u> » 0: Read-data CRC error interrupt is disabled « » 1: Read-data CRC error interrupt is enabled «
5	ECRCWR	0	R/W	<u>Write-Data CRC Error Interrupt Enable</u> » 0: Write-data CRC error interrupt is disabled « » 1: Write-data CRC error interrupt is enabled «
4	ETOUTRS	0	R/W	<u>Response Time-Out Interrupt Enable</u> » 0: Response time-out interrupt is disabled « » 1: Response time-out interrupt is enabled «
3	ETOUTRD	0	R/W	<u>Read-Data Time-Out Interrupt Enable</u> » 0: Read-data time-out interrupt is disabled « » 1: Read-data time-out interrupt is enabled «
2	ERSPDNE	0	R/W	<u>Response/Command Done Interrupt Enable</u> » 0: Response/command done interrupt is disabled « » 1: Response/command done interrupt is enabled «
1	EBSYDNE	0	R/W	<u>Busy Done Interrupt Enable</u> » 0: Busy done interrupt is disabled « » 1: Busy done interrupt is enabled «
0	EDATDNE	0	R/W	<u>Data Transfer Done Interrupt Enable</u> » 0: Data transfer done interrupt is disabled « » 1: Data transfer done interrupt is enabled «

17.12.6 MMCTOR

MMC Response Time-Out Register

MMCTOR	0003:048A			offset: 0x0A												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	TOD[4]	TOD[3]	TOD[2]	TOD[1]	TOD[0]	TOR[7]	TOR[6]	TOR[5]	TOR[4]	TOR[3]	TOR[2]	TOR[1]	TOR[0]	
R/W	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	-	-	-	0	0	0	0	0	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:13	RSV			Reserved
128	TOD	0	R/W	Upper 5 bits for timeout Count For Data Read (in MMC mode) 0x0000: no time-out 0x0001: 1 CLK clock 0x0002: 2 CLK clocks 0x1FFFFFF: 2097151 CLK clocks Note: Please add TOD15-0 from MMCTOD register also for calculations
7:0	TOR	0	R/W	Time-Out Count For Response (in MMC mode) 0000 0000: no time-out 0000 0001: 1 CLK clock 0000 0010: 2 CLK clocks 1111 1111 : 255 CLK clocks

17.12.7 MMCTOD

MMC Data Read Time-Out Register

MMCTOD	0003:048C			offset: 0x0C			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	TOD[15]	TOD[14]	TOD[13]	TOD[12]	TOD[11]	TOD[10]	TOD[9]	TOD[8]	TOD[7]	TOD[6]	TOD[5]	TOD[4]	TOD[3]	TOD[2]	TOD[1]	TOD[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
150	TOD	0	R/W	<u>Lower 16 bits for timeout Count For Data Read (in MMC mode)</u> 0x0000: no time-out 0x0001: 1 CLK clock 0x0002: 2 CLK clocks 0x1FFFFFF: 2097151 CLK clocks * Please add TOD20-16 from MMCTOR register also for calculations

17.12.8 MMCBLEN

MMC Block Length Register

MCMCBLN	0003:048E				offset: 0x0E						default: 0x0200					
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	BLEN[11]	BLEN[10]	BLEN[9]	BLEN[8]	BLEN[7]	BLEN[6]	BLEN[5]	BLEN[4]	BLEN[3]	BLEN[2]	BLEN[1]	BLEN[0]
R/W	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	0	0	1	0	0	0	0	0	0	0	0	0

[illegible]

17.12.9 MMCNBLK

MMC Number of Blocks Register

[illegible][illegible]

17.12.10 **MMCNBLC**

MMC Number of Blocks Counter

[illegible][illegible]

17.12.11 MMCDRR**MMC Data Receive Register**

MMCDRR	0003:0494			offset: 0x14													default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	DRR[15]	DRR[14]	DRR[13]	DRR[12]	DRR[11]	DRR[10]	DRR[9]	DRR[8]	DRR[7]	DRR[6]	DRR[5]	DRR[4]	DRR[3]	DRR[2]	DRR[1]	DRR[0]		
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W		
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		

Bit	Name	Reset Value	R/W	Function
15:0	DRR	0	R/W	<u>Data Receive Register</u> Used for reading of receive data from card when block read. If an odd byte number of a data block is set up by MMCBLN, last byte is received to DRR[7:0].

17.12.12 MMCDXR

MMC Data Transmit Register

MMCDXR	0003:0496			offset: 0x16												default: 0x0000		
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	DXR[15]	DXR[14]	DXR[13]	DXR[12]	DXR[11]	DXR[10]	DXR[9]	DXR[8]	DXR[7]	DXR[6]	DXR[5]	DXR[4]	DXR[3]	DXR[2]	DXR[1]	DXR[0]		
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W		
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		

Bit	Name	Reset Value	R/W	Function
15:0	DXR	0	R/W	<u>Data Transmit Register</u> Used for writing of transmit data to card when block write. If an odd byte number of a data block is set up by MMCBLLEN, last byte is transmitted from DXR[15:8].

17.12.13 MMCCMD

MMC Command Register

MMCCMD	0003:0498			offset: 0x18			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	DCLR	INITCK	DATA	STREAM	WRITE	RSPFMT[1]	RSPFMT[0]	BSYEXP	PPLN	RSV	CMD[5]	CMD[4]	CMD[3]	CMD[2]	CMD[1]	CMD[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	-	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	-	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15	DCLR	0	R/W	<u>Data Receive/Transmit Clear</u> » 0: Do not clear data receive ready (DRRDY) and data transmit ready (DXRDY) bits « » 1: Clear DRRDY and DXRDY «
14	INITCK	0	R/W	<u>Initialization Clock Cycles</u> » 0: Do not insert initialization clock cycles « » 1: Insert initialization clock cycles « Insert 80 CLK cycles before sending the command specified in the CMD field. These dummy clock cycles are required for resetting a card after power on.
13	DATA	0	R/W	<u>Data Transfer Indicator</u> » 0: No data transfer « » 1: A data transfer associated with the command «
12	STREAM	0	R/W	<u>Stream Enable</u> » 0: If DATA = 1, the transfer is a block transfer « » 1: If DATA = 1, the transfer is a stream transfer «
11	WRITE	0	R/W	<u>Write Enable</u> » 0: If DATA = 1, the data transfer is a read operation « » 1: If DATA = 1, the data transfer is a write operation «
10:9	RSPFMT	0	R/W	<u>Response Format (expected type of response to command)</u> 00: No response 01: R1/R4/R5/R6 Native mode: 48 bits with CRC 10: R2 Native mode: 136 bits with CRC 11: R3 Native mode: 48 bits with no CRC
8	BSYEXP	0	R/W	<u>Busy Expected</u> » 0: No busy signal expected « » 1: Busy signal expected « If an R1b (R1 with busy) response is expected, set RSPFMT = 01 and BSYEXP = 1
7	PPLEN	0	R/W	<u>Push-Pull Enable</u> » 0: Open-drain mode « » 1: Push-Pull mode «
6	RSV			<u>Reserved</u>
5:0	CMD	0	R/W	<u>Command Index</u> The command to be sent to the memory card CMD line. Note: Writing to any bit in MMCCMD causes the MMC controller to send the command. Thus, the argument registers (ARGH and ARG L) have to be loaded properly before a write to MMCCMD. The CMD field specifies the type of command to be sent. The other fields in MMCCMD define the operation (command, response, additional activity) for the MMC controller. The contents of MMCCMD are retained until the next time MMCCMD is written. Note: Command packet sent by controller - [47] start bit - [46] transmission bit MMCCMD[5:0] [45-40] command index MMCARGH [39-24] argument MMCARG L [23-8] argument - [7-1] CRC7 - [0] end bit

17.12.14 MMCARGL

MMC Argument Register Low

[illegible]

Bit	Name	Reset Value	R/W	Function
150	ARGL	0	R/W	<u>Argument Register, Low</u> Lower 16-bits of the command argument Note: ARGH and ARGL are used for specifying the arguments to be sent with the command specified in CMD. Writing to MMCCMD causes the MMC controller to send a command; You must load ARGH and ARGL before writing the command to MMCCMD. Also, make sure you do not modify the argument registers while they are being used for an operation. The contents of the argument registers are retained until next time this register is written Note: Command packet sent by controller - [47] start bit - [46] transmission bit MMCCMD[5:0] [45-40] command index MMCARGH [39-24] argument MMCARGL [23-8] argument - [7-1] CRC7 - [0] end bit

17.12.15 **MMCARGH**

MMC Argument Register High

[illegible][illegible]

17.12.16 MMCRSP0**MMC Response Register #0**

MMCRSP0	0003:049E			offset: 0x1E													default: 0x0000		
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSP0[15]	RSP0[14]	RSP0[13]	RSP0[12]	RSP0[11]	RSP0[10]	RSP0[9]	RSP0[8]	RSP0[7]	RSP0[6]	RSP0[5]	RSP0[4]	RSP0[3]	RSP0[2]	RSP0[1]	RSP0[0]			
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W			
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			

Bit	Name	Reset Value	R/W	Function
				<u>Response Register 0</u>
				Note:
				R1 or R3(R4,R5 or R6) response in SD/MMC mode
				MMCCIDX [47-40]
				MMCRSP7 [39-24]
				MMCRSP6 [23-8]
				MMCRSP5-MMCRSP0
				R2 response in SD/MMC mode
150	RSP0	0	R/W	MMCCIDX [135-128]
				MMCRSP7 [127-112]
				MMCRSP6 [111-96]
				MMCRSP5 [95-80]
				MMCRSP4 [79-64]
				MMCRSP3 [63-48]
				MMCRSP2 [47-32]
				MMCRSP1 [31-16]
				MMCRSP0 [15-0]

17.12.17 MMCRSP1

MMC Response Register #1

MMCRSP1	0003:04A0			offset: 0x20			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSP1[15]	RSP1[14]	RSP1[13]	RSP1[12]	RSP1[11]	RSP1[10]	RSP1[9]	RSP1[8]	RSP1[7]	RSP1[6]	RSP1[5]	RSP1[4]	RSP1[3]	RSP1[2]	RSP1[1]	RSP1[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
150	RSP1	0	R/W	<u>Response Register 1</u>
				Note:
				R1 or R3(R4,R5 or R6) response in SD/MMC mode
				MMCCIDX [47-40]
				MMCRSP7 [39-24]
				MMCRSP6 [23-8]
				MMCRSP5-MMCRSP0
				R2 response in SD/MMC mode
				MMCCIDX [135-128]
				MMCRSP7 [127-112]
				MMCRSP6 [111-96]
				MMCRSP5 [95-80]
				MMCRSP4 [79-64]
				MMCRSP3 [63-48]
				MMCRSP2 [47-32]
				MMCRSP1 [31-16]
				MMCRSP0 [15-0]

17.12.18 MMSRSP2**MMC Response Register #2**

MMSRSP2	0003:04A2			offset: 0x22			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSP2[15]	RSP2[14]	RSP2[13]	RSP2[12]	RSP2[11]	RSP2[10]	RSP2[9]	RSP2[8]	RSP2[7]	RSP2[6]	RSP2[5]	RSP2[4]	RSP2[3]	RSP2[2]	RSP2[1]	RSP2[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
				<u>Response Register 2</u>
				Note:
				R1 or R3(R4,R5 or R6) response in SD/MMC mode
				MMCCIDX [47-40]
				MMCRSP7 [39-24]
				MMCRSP6 [23-8]
				MMCRSP5-MMCRSP0
				R2 response in SD/MMC mode
150	RSP2	0	R/W	MMCCIDX [135-128]
				MMCRSP7 [127-112]
				MMCRSP6 [111-96]
				MMCRSP5 [95-80]
				MMCRSP4 [79-64]
				MMCRSP3 [63-48]
				MMCRSP2 [47-32]
				MMCRSP1 [31-16]
				MMCRSP0 [15-0]

17.12.19 MMCRSP3**MMC Response Register #3**

MMCRSP3	0003:04A4			offset: 0x24			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSP3[15]	RSP3[14]	RSP3[13]	RSP3[12]	RSP3[11]	RSP3[10]	RSP3[9]	RSP3[8]	RSP3[7]	RSP3[6]	RSP3[5]	RSP3[4]	RSP3[3]	RSP3[2]	RSP3[1]	RSP3[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
150	RSP3	0	R/W	<u>Response Register 3</u>
				Note:
				R1 or R3(R4,R5 or R6) response in SD/MMC mode
				MMCCIDX [47-40]
				MMCRSP7 [39-24]
				MMCRSP6 [23-8]
				MMCRSP5-MMCRSP0
				R2 response in SD/MMC mode
				MMCCIDX [135-128]
				MMCRSP7 [127-112]
				MMCRSP6 [111-96]
				MMCRSP5 [95-80]
				MMCRSP4 [79-64]
				MMCRSP3 [63-48]
				MMCRSP2 [47-32]
				MMCRSP1 [31-16]
				MMCRSP0 [15-0]

17.12.20 MMCRSP4**MMC Response Register #4**

MMCRSP4	0003:04A6			offset: 0x26													default: 0x0000		
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSP4[15]	RSP4[14]	RSP4[13]	RSP4[12]	RSP4[11]	RSP4[10]	RSP4[9]	RSP4[8]	RSP4[7]	RSP4[6]	RSP4[5]	RSP4[4]	RSP4[3]	RSP4[2]	RSP4[1]	RSP4[0]			
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W			
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			

Bit	Name	Reset Value	R/W	Function
15:0	RSP4	0	R/W	<u>Response Register 4</u>
				Note:
				R1 or R3(R4,R5 or R6) response in SD/MMC mode
				MMCCIDX [47-40]
				MMCRSP7 [39-24]
				MMCRSP6 [23-8]
				MMCRSP5-MMCRSP0
				R2 response in SD/MMC mode
				MMCCIDX [135-128]
				MMCRSP7 [127-112]
				MMCRSP6 [111-96]
				MMCRSP5 [95-80]
				MMCRSP4 [79-64]
				MMCRSP3 [63-48]
				MMCRSP2 [47-32]
				MMCRSP1 [31-16]
				MMCRSP0 [15-0]

17.12.21 MMCRSP5

MMC Response Register #5

MMCRSP5	0003:04A8			offset: 0x28			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSP5[15]	RSP5[14]	RSP5[13]	RSP5[12]	RSP5[11]	RSP5[10]	RSP5[9]	RSP5[8]	RSP5[7]	RSP5[6]	RSP5[5]	RSP5[4]	RSP5[3]	RSP5[2]	RSP5[1]	RSP5[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
150	RSP5	0	R/W	<u>Response Register 5</u>
				Note:
				R1 or R3(R4,R5 or R6) response in SD/MMC mode
				MMCCIDX [47-40]
				MMCRSP7 [39-24]
				MMCRSP6 [23-8]
				MMCRSP5-MMCRSP0
				R2 response in SD/MMC mode
				MMCCIDX [135-128]
				MMCRSP7 [127-112]
				MMCRSP6 [111-96]
				MMCRSP5 [95-80]
				MMCRSP4 [79-64]
				MMCRSP3 [63-48]
				MMCRSP2 [47-32]
				MMCRSP1 [31-16]
				MMCRSP0 [15-0]

17.12.22 MMCRSP6**MMC Response Register #6**

MMCRSP6	0003:04AA			offset: 0x2A													default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSP6[15]	RSP6[14]	RSP6[13]	RSP6[12]	RSP6[11]	RSP6[10]	RSP6[9]	RSP6[8]	RSP6[7]	RSP6[6]	RSP6[5]	RSP6[4]	RSP6[3]	RSP6[2]	RSP6[1]	RSP6[0]		
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W		
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		

Bit	Name	Reset Value	R/W	Function
15:0	RSP6	0	R/W	<u>Response Register 6</u>
				Note:
				R1 or R3(R4,R5 or R6) response in SD/MMC mode
				MMCCIDX [47-40]
				MMCRSP7 [39-24]
				MMCRSP6 [23-8]
				MMCRSP5-MMCRSP0
				R2 response in SD/MMC mode
				MMCCIDX [135-128]
				MMCRSP7 [127-112]
				MMCRSP6 [111-96]
				MMCRSP5 [95-80]
				MMCRSP4 [79-64]
				MMCRSP3 [63-48]
				MMCRSP2 [47-32]
				MMCRSP1 [31-16]
				MMCRSP0 [15-0]

17.12.23 MMCRSP7

MMC Response Register #7

MMCRSP7	0003:04AC			offset: 0x2C			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSP7[15]	RSP7[14]	RSP7[13]	RSP7[12]	RSP7[11]	RSP7[10]	RSP7[9]	RSP7[8]	RSP7[7]	RSP7[6]	RSP7[5]	RSP7[4]	RSP7[3]	RSP7[2]	RSP7[1]	RSP7[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
150	RSP7	0	R/W	<u>Response Register 7</u>
				Note:
				R1 or R3(R4,R5 or R6) response in SD/MMC mode
				MMCCIDX [47-40]
				MMCRSP7 [39-24]
				MMCRSP6 [23-8]
				MMCRSP5-MMCRSP0
				R2 response in SD/MMC mode
				MMCCIDX [135-128]
				MMCRSP7 [127-112]
				MMCRSP6 [111-96]
				MMCRSP5 [95-80]
				MMCRSP4 [79-64]
				MMCRSP3 [63-48]
				MMCRSP2 [47-32]
				MMCRSP1 [31-16]
				MMCRSP0 [15-0]

17.12.26 MMCCIDX

MMC Command Index Register

MMCCIDX	0003:04B2				offset: 0x32				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	START	XMT	CIDX[5]	CIDX[4]	CIDX[3]	CIDX[2]	CIDX[1]	CIDX[0]
R/W	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	-	-	-	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:8	RSV			Reserved
7	START	0	R/W	Start bit in command response, stored when it is received in the native mode.
6	XMT	0	R/W	Transmission bit in command response, stored when it is received in the native mode.
5:0	CIDX	0	R/W	<u>MMC Command Index</u> Command index bits of a response, stored when it is received in the native mode. NOTE: In R2, R3 response, command index bits are reserved and '111111' is received in the CIDX field. Note: Refer note in MMCRSPx registers for response format details

17.12.27 **MMCKKC**

MMC Test Register - Clock Start Phase Control

MMCKCKC	0003:04B4			offset: 0x34												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	CPHA[7]	CPHA[6]	CPHA[5]	CPHA[4]	CPHA[3]	CPHA[2]	CPHA[1]	CPHA[0]	
R/W	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	-	-	-	-	-	-	-	-	0	0	0	0	0	0	0	0	

[illegible]

17.12.28 MMCTORC**MMC Test register – TORC**

MMCTORC	0003:04B6			offset: 0x36			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	TODCH[4]	TODCH[3]	TODCH[2]	TODCH[1]	TODCH[0]	TORC[7]	TORC[6]	TORC[5]	TORC[4]	TORC[3]	TORC[2]	TORC[1]	TORC[0]
R/W	-	-	-	R	R	R	R	R	R	R	R	R	R	R	R	R
Default	-	-	-	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:13	RSV			Reserved
12:8	TODCH	0	R	<u>Time Out</u> MMCTODC is used in order to test the counter that operates with the value of MMCTOD in a factory test or a chip evaluation. Reading is possible in order to check the number of cycles remaining before Time-Out. * Please refer TODCL filed in MMCTODC register also for calculations Note: Writing to this register is prohibited
7:0	TORC	0	R	<u>Time Out</u> MMCTORC is used in order to test the counter that operates with the value of MMCTOR in a factory test or a chip evaluation. Reading is possible in order to check the number of cycles remaining before Time-Out. Note: writing to this register is prohibited

17.12.29 MMCTODC

MMC Test register – TODC

MMCTODC	0003:04B8			offset: 0x38			default: 0xFFFF									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	TODCL[15]	TODCL[14]	TODCL[13]	TODCL[12]	TODCL[11]	TODCL[10]	TODCL[9]	TODCL[8]	TODCL[7]	TODCL[6]	TODCL[5]	TODCL[4]	TODCL[3]	TODCL[2]	TODCL[1]	TODCL[0]
R/W	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R
Default	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

Bit	Name	Reset Value	R/W	Function
150	TODCL	65535	R	<u>Time Out</u> MMCTODC is used in order to test the counter that operates with the value of MMCTOD in a factory test or a chip evaluation. Reading is possible in order to check the number of cycles remaining before Time-Out. * Please refer TODCH field in MMCTORC register also for calculations Note: Writing to this register is prohibited

17.12.31 DMATRG**DMA Trigger Register**

DMATRG	0003:04BC			offset: 0x3C			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	BRK	TRG
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	W	W
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0	0

Bit	Name	Reset Value	R/W	Function
15:2	RSV			Reserved
1	BRK	0	W	Break - DMA Transfer suspend » 0: no effect « » 1: DMA transfer break « The current DMA transfer is suspended when 1 is written. Bit is cleared to 0 immediately after writing. Note: Since the actual end of transfer is several cycles after the bit is set, please resume transfer only after confirming that transfer has completely ended; i.e., the RUNST flag of the DMASTAT1 register has been cleared to 0.
0	TRG	0	W	DMA Transfer Start Trigger » 0: no effect « » 1: DMA transfer start « The current DMA transfer is started when 1 is written. Bit is cleared to 0 immediately after writing.

17.12.32 DMAMODE

DMA Mode Register

DMAMODE	0003:04BE			offset: 0x3E			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	DT0IE	DMAE	DIR	DPRMDX	DPRMDR	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV
R/W	-	R/W	R/W	R/W	R/W	R/W	-	-	-	-	-	-	-	-	-	-
Default	-	0	0	0	0	0	-	-	-	-	-	-	-	-	-	-

Bit	Name	Reset Value	R/W	Function
15	RSV			Reserved
14	DT0IE	0	R/W	DMA Transfer Timeout Interrupt Enable » 0: DMA transfer timeout interrupt is disabled « » 1: DMA transfer timeout interrupt is enabled «
13	DMAE	0	R/W	DMA Enable » 0: Disable DMA « » 1: Enable DMA «
12	DIR	0	R/W	DMA Transfer Direction » 0: MMCIF to SDRAM « » 1: SDRAM to MMCIF «
11	DPRMDX	0	R/W	Word Swap Enable When Writing » 0: Do not word-swap 32-bit wide data when writing data from the SDRAMC to the card (SDRAMC_out[31:0] -> CARD_in[31:0]) « » 1: Word-swap 32-bit wide data when writing data from the SDRAMC to the card (SDRAMC_out[31:0] -> CARD_in[15:0,31:16]) «
10	DPRMDR	0	R/W	Word Swap Enable When Reading » 0: Do not word-swap 32-bit wide data when reading data from the card to SDRAM (CARD_out[31:0] -> SDRAM_in[31:0]) « » 1: Word-swap 32-bit wide data when reading data from the card to the SDRAMC (CARD_out[31:0] -> SDRAMC_in[15:0,31:16]) «
9:0	RSV			Reserved

17.12.33 DMAAD0

DMA Address Low

[illegible][illegible]

17.12.34 DMAAD1

DMA Address High

[illegible][illegible]

DMA Time-Out Register

[illegible]

17.12.38 SDIOCTL

SDIO Control Register

SDIOCTL	0003:04CA			offset: 0x4A													default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RDWTCR	RDWTRQ		
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W	R/W		
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0	0		

Bit	Name	Reset Value	R/W	Function
15:2	RSV			Reserved
1	RDWTCR	0	R/W	Read Wait enable for CRC error. If this bit is set, Read Wait operation automatically starts in case a CRC error is detected during multiple block read access and while it's not the last block to be transferred. In this case, RDWTRQ (Read Wait request) is automatically set. To end the Read Wait operation, write '0' to RDWTRQ. (No need to clear RDWTCR.)
0	RDWTRQ	0	R/W	Read Wait request. » 0: End Read Wait operation and release DAT[2] « » 1: Set a Read Wait request « If this bit is set, Read Wait operation starts 2 clocks after the end of the read data block. MMCIF asserts low level on DAT[2] until this bit is cleared.

17.12.39 SDIOST0**SDIO Status Register #0**

SDIOST0	0003:04CC			offset: 0x4C												default: 0x0001		
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RDWTST	INTPRD	DAT1		
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W	R/W	R/W		
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	0	0	1		

Bit	Name	Reset Value	R/W	Function
15:3	RSV			Reserved
2	RDWTST	0	R/W	Read Wait status. RDWTST becomes '1' during Read Wait operation.
1	INTPRD	0	R/W	Interrupt period. INTPRD becomes '1' during Interrupt period.
0	DAT1	1	R/W	DAT[1] status. DAT[1] can be monitored through this bit.

17.12.40 SDIOIEN**SDIO Enable Register**

SDIOIEN	0003:04CE				offset: 0x4E											default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RWSEN	IOINTEN	
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W	R/W	
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0	0	

Bit	Name	Reset Value	R/W	Function
15:2	RSV			Reserved
1	RWSEN	0	R/W	Read Wait interrupt enable » 0: Disable the interrupt « » 1: Enable the interrupt « Interrupt is generated to notify that Read Wait operation has been started.
0	IOINTEN	0	R/W	SDIO card interrupt enable » 0: Disable the interrupt « » 1: Enable the interrupt «

17.12.41 SDIOIST

SDIO Status Interrupt Status Register

SDIOIST	0003:04D0			offset: 0x50			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RWS	IOINT
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W	R/W
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0	0

Bit	Name	Reset Value	R/W	Function
15:2	RSV			Reserved
1	RWS	0	R/W	Read Wait interrupt status » 0: Not interrupted « » 1: Interrupted « This bit is set to '1' when Read Wait operation starts and Read Wait interrupt is enabled. To clear this bit, write '1'.
0	DNT	0	R/W	SDIO card interrupt status » 0: Not interrupted « » 1: Interrupted « This bit is set to '1' if the SDIO card interrupt is detected and card interrupt is enabled. To clear this bit, write '1' after the interrupt on the DAT[1] line is removed.
50	PRB4	34	R/W	Interrupt Factor with 35th highest priority

18 USB OTG Interface

18.1 Introduction

DM320 incorporates a USB 2.0 full speed dual-role OTG controller. This chapter describes the USB module present in DM320.

DM320 USB module has following features:

- Compliant with USB specification for full speed function and with the OTG supplement to the USB 2.0 specification
- Support for full speed device, limited host feature to communicate with low and full speed peripherals, OTG operations
- Built-in USB transceiver
- Built-in 4K RAM for endpoint FIFOs
- Programmable maximum packet size and transfer type for each endpoint
- DMA to EMIF/SDRAM/ARM Internal RAM through AHB master interface
- AHB Slave interface

Figure 128 shows the block diagram of USB module. DM320 has a built-in transceiver and 4K bytes RAM. The controller operates as full speed function controller or as Host/Peripheral in point-to-point communication with other USB peripheral/host.

DM320 USB controller provides encoding, decoding and checking needed in sending and receiving USB packets – interrupting ARM only when endpoint data has been successfully transferred. When acting as the host for point-to-point communications, it additionally maintains a frame counter and automatically schedules SOF, isochronous, interrupt and bulk transfers. It also includes support for the Session Request Protocol (SRP) and Host Negotiation Protocol (HNP) required for point-to-point communications, details of which are given in the *USB On-The-Go* supplement to the USB 2.0 specification.

This module has 10 endpoints (including control endpoint) and 10 endpoint FIFOs. Endpoint 0 FIFO size is fixed to 64 bytes and is shared between Rx and Tx FIFO. The other endpoints are configurable with dynamic FIFO sizing function except for the direction. Dynamic FIFO sizing function will be useful for many applications where the FIFO size needs to be modified as required by the application.

The USB controller has AHB Bridge and multi channel DMA with AHB Master port. The controller can do DMA transfers to EMIF/SDRAM/ARM Internal Memory regions.

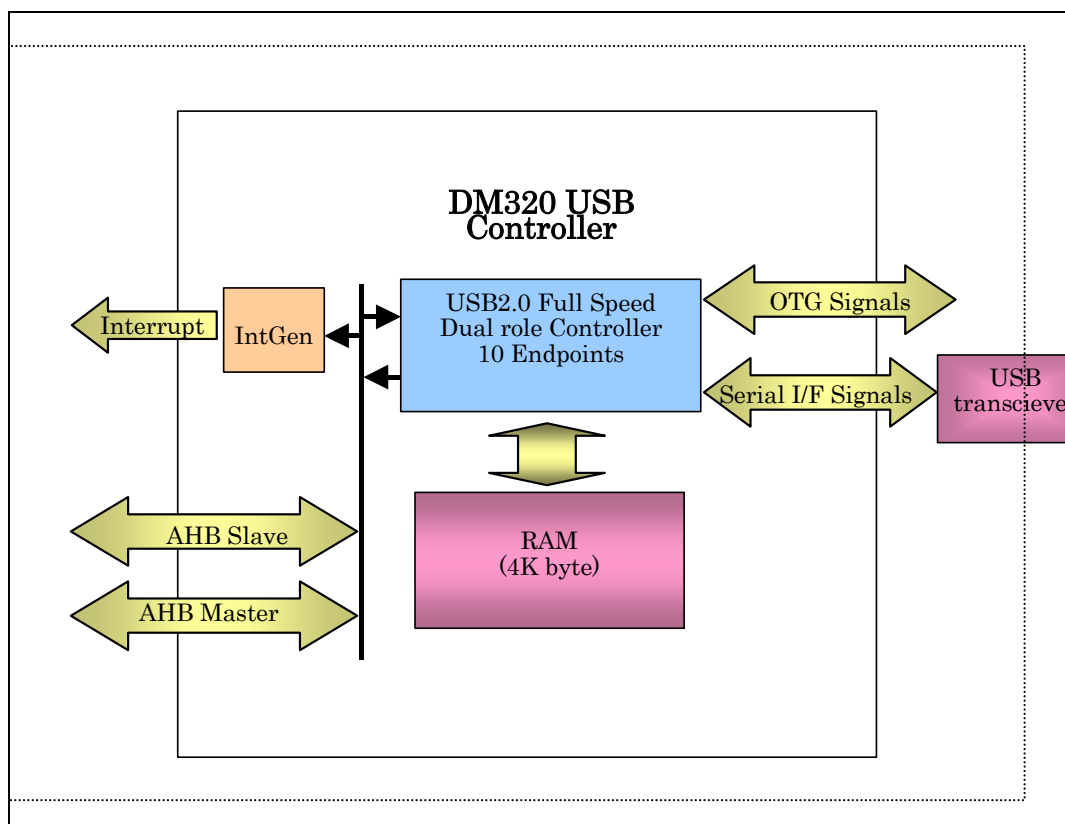


Figure 128: Block Diagram for DM320 USB Controller

18.2 Device Configuration

All endpoints except EP0 are configurable for maximum packet size and transmit type (bulk, interrupt or isochronous). The controller can be configured with regard to the size of FIFO associated with each endpoint, or alternatively the total RAM size to be assigned dynamically to the different endpoints when the controller is initialized.

The allocation of FIFO space to the different endpoints requires the following specification for each transmit and receive endpoint:

- The start address of the FIFO within the RAM block
- The maximum packet size for the endpoint (0-1023 bytes)
- Whether double-buffering is required

These details are specified through the following registers

USB Controller (Base Address-0x8000:0000)		
Address offset	NAME	DESCRIPTION
0x1C	TXFIFO1	Tx Endpoint FIFO details
0x1D	TXFIFO2	
0x1E	RXFIFO1	Rx Endpoint FIFO details
0x1F	RXFIFO2	

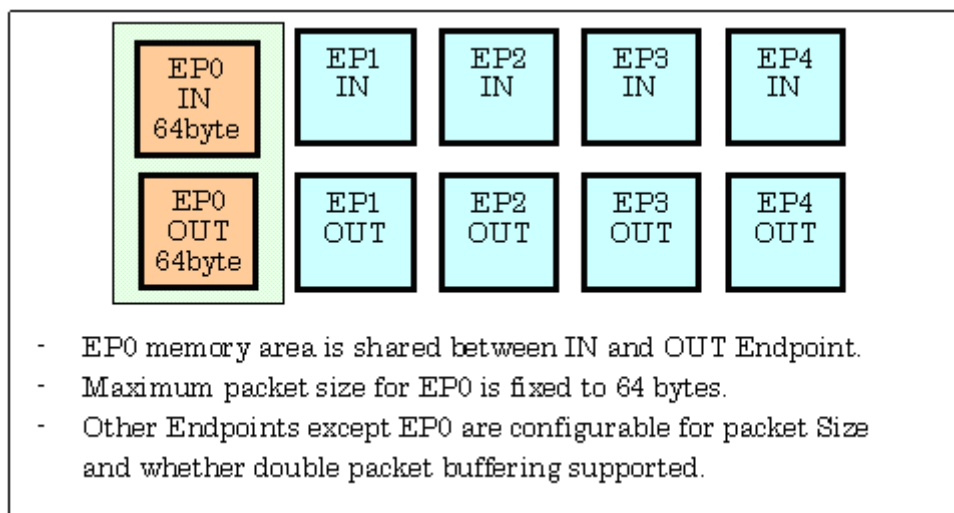


Figure 129: USB Controller RAM configuration

18.2.1 Example configurations

Different endpoint configurations can be made from the available endpoints. The programmability to configure any endpoint for any transfer provides the flexibility to use different endpoints depending upon the requirement.

As shown in Figure 130, Case 1 configuration can be used to implement USB Mass Storage Class. Similarly, Case 2 configuration can be used to implement USB Still Image Class. The different class definitions are available at www.usb.org.

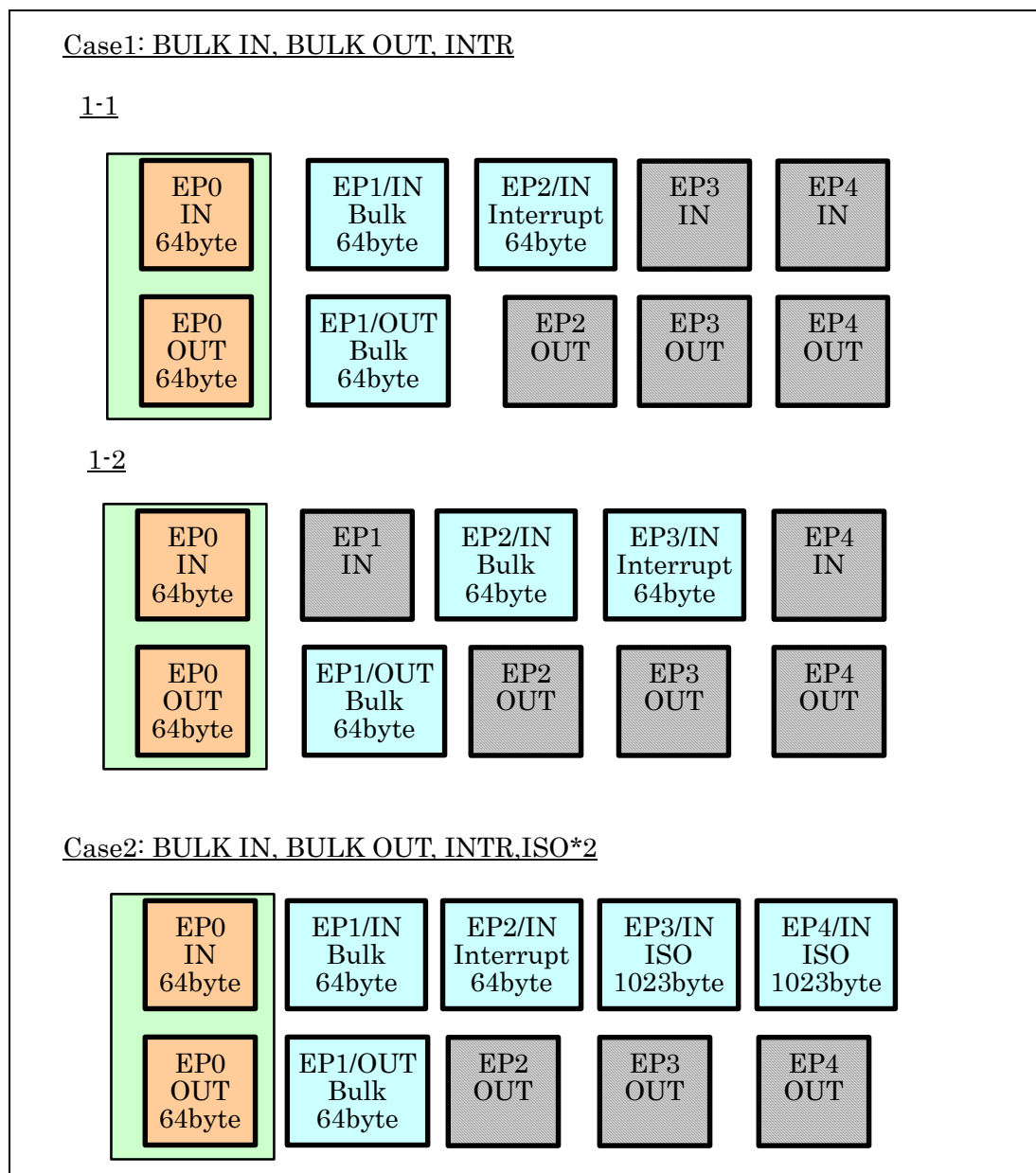


Figure 130: Configuration example

18.3 Controller Initialization

The following registers have to be configured for the proper operation of USB Controller:

Register	Setting	Operation
MOD2 register in CLKC *	Bit GIO and bit USB = 1	Enable USB & GIO clocks
LPCTL1 register in CLKC *	Bit USBPDN = 0	Disable USB Buffer Power Down
FSEL1 register in GIO *	0x5555	GIO17-25 configured to operate as USB OTG signals
FSEL2 register in GIO *	0x0001	GIO17-25 configured to operate as USB OTG signals
DEVCTL register in USB0	Bit SESSREQ = 1	Set Session Start to enable 12MHz clock out for USB

Note: It is required to set SESSREQ bit to enable 12MHz internal clock and if the clock is not enabled, some indexed register of USB will not be accessible. 12MHz clock out is also disabled when the controller goes into suspend mode. This is the feature to reduce power when no session is active.

*: Refer to “General Purpose Input Output” and “Clock Controller” chapters for more details about the registers.

18.4 OTG signals

After setting the FSEL1 and FSEL2 registers of GIO, GIO17-25 are assign to OTG function signals which controls VBUS and pull-up/down registers, or detect required VBUS state. The mapping between OTG signals and GIO pins is shown in Table 55.

External GIO Pin	OTG Signal	Direction	
GIO17	usb_vbuschg	OUT	
GIO18	usb_vbusen	OUT	
GIO19	usb_vbusval	IN	
GIO20	usb_vbusses	IN	

GIO21	usb_vbuslo	IN	
GIO22	usb_cid	IN	
GIO23	usb_pucon	OUT	
GIO24	usb_pdcon	OUT	
GIO25	usb_iden	OUT	

Table 55: OTG Signals

18.5 Register Description

USB controller registers are memory mapped on ARM memory map from 0x8000:0000 to 0x8000:0033 and USB DMA controller registers are memory mapped from 0x8000:0200 to 0x8000:023F. [USB DMA Controller](#) is discussed in later sections of the chapter.

The USB controller register map is split into three sections:

- **Common USB registers (Offset 00h–0Fh)** – These are 8 bit registers that provide control and status for the complete core.
- **Endpoint Control/Status registers (Offset 10h–1Fh)** – These are 8 bit registers that provide control and status for the endpoints. The registers mapped into this section depend on the value of the INDEX (offset 0x0E) register and on the mode of operation (host or peripheral). The host specific registers are prefixed with HST (like HST_xxx) and peripheral specific registers are prefixed with PER (like PER_xxx).

Note: INDEX register should be written with the endpoint number for which the status/control registers are to be accessed.

- **FIFOs (Offset 20h–30h)** – These are 32 bit registers that provide access to the respective FIFOs.

The register details are provided in section 18.15.

18.6 USB Interrupt Control

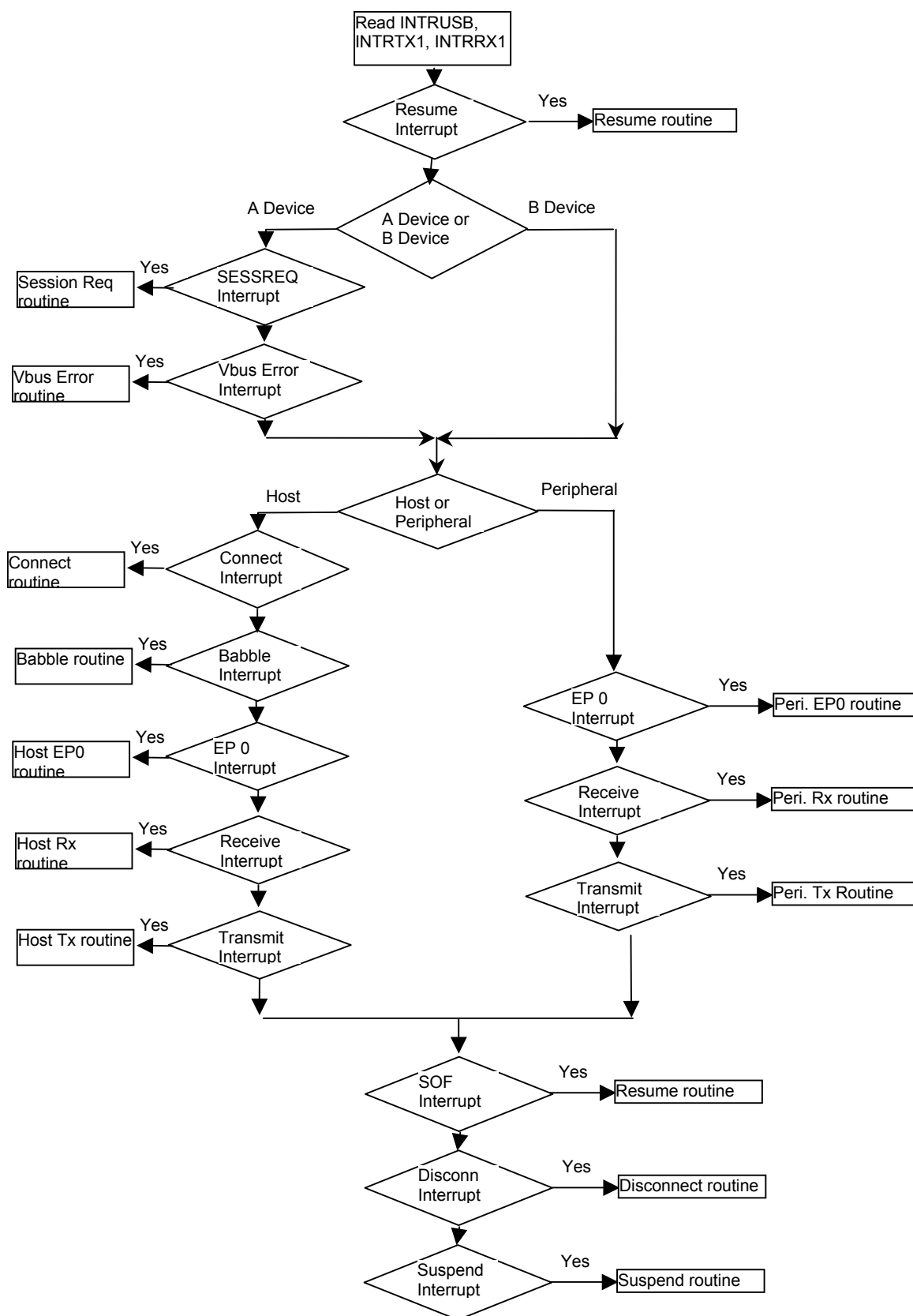
USB controller interrupts ARM on completion of the data transfer on any of the endpoints or on detecting reset, resume, suspend, connect, disconnect, SOF on the bus. Interrupts are generated for only those conditions which are enabled in the registers INTRTX1E,INTRRX1E and INTRUSBE.

When ARM is interrupted with a USB interrupt, it needs to read the interrupt status register (INTRTX1,INTRRX1 and INTRUSB) to determine which endpoints have caused the interrupt and jump to the appropriate routine. If

multiple endpoints have caused the interrupt, endpoint 0 should be serviced first, followed by the other endpoints. The Suspend interrupt should be serviced last.

The flowchart in Figure 131 describes the interrupt service routine for the USB module.

Figure 131: Flowchart for USB Interrupt Service Routine



18.7 USB Bus State

18.7.1 Suspend / Resume

When the USB controller sees no activity on the bus for 3ms, it will generate a suspend interrupt (SUSPEND bit in INTRUSB register if enabled in INTRUSBE register).

In suspend mode, the controller can be disabled.

- The USB may exit suspend mode by sending resume signaling on the bus. If the controller is left active while the USB is in suspend mode, it will monitor the bus for resume signaling.

When resume signaling appears on the bus, the controller will generate a resume interrupt (RESUME bit in INTRUSB register).

- When the suspend interrupt described above is received, the software may disable the USB controller by stopping its clock. However, because the controller is disabled, it will not then be able to detect resume signaling on the bus. As a result, some external hardware will be needed to detect resume signaling (by monitoring the D- and D+ data lines), so that the clock to the controller can be restarted.

18.7.1.1 Remote Wakeup

Remote wakeup can be initiated from the USB controller when the bus is in suspend mode. This can be done by setting the RESUME bit in the POWER register. This bit should be left set for approximately 10 ms (minimum of 2 ms, a maximum of 15 ms) then reset it to 0.

Note: No resume interrupt will be generated when the software initiates a remote wakeup.

18.7.2 USB Reset

18.7.2.1 Peripheral Mode

When a reset condition is detected on the USB, the controller performs the following actions:

- Sets FADDR register to 0
- Sets INDEX register to 0
- Flushes all endpoint FIFOs
- Clears all control/status registers.
- Enables all interrupts, except Suspend
- Generates a reset interrupt (RESET bit in INTRUSB register)

When the software receives a reset interrupt, it should close any pending transactions and wait for bus enumeration to begin.

18.7.2.2 Host Mode

To generate reset signal on the bus, set the RESET bit in the POWER register. To clear the reset signaling, RESET bit has to be cleared.

18.8 Handling Control Transactions

Control transactions are carried out through endpoint 0 in both peripheral as well as host mode.

18.8.1 Control Transactions in peripheral mode

All the standard device requests are to be handled in the software.

All the standard device requests can be classified in three categories:

- *Zero Data Requests* - in which all the information is included in the command
- *Write Requests* - in which the command will be followed by additional data
- *Read Requests* - in which the device is required to send data back to the host

18.8.1.1 Zero Data Requests

Zero data requests have all their information included in the 8-byte command and require no additional data to be transferred. Examples of such standard device requests are: SET_FEATURE, CLEAR_FEATURE, SET_ADDRESS, SET_CONFIGURATION and SET_INTERFACE.

The sequence of events will begin, as with all requests, when the software receives an endpoint 0 interrupt. The RXPKTRDY bit of PER_CSR0 will also be set. The 8-byte command should then be read from the endpoint 0 FIFO, decoded and the appropriate action taken. For example, if the command is SET_ADDRESS, the 7-bit address value contained in the command should be written to the PER_CSR0 register.

CLRRXRDY bit is to be set in PER_CSR0 register to indicate that the command has been read from the FIFO. DATAEND bit in PER_CSR0 is set to indicate that no further data is expected for this request.

When the host moves to the status stage of the request, a second endpoint 0 interrupt will be generated to indicate that the request has completed. No further action is required from the software. The second interrupt is just a confirmation that the request completed successfully.

If the command is an unrecognized command or cannot be executed, the PER_CSR0 register should be written to set the CLRRXRDY bit and SENDST bit. When the host moves to the status stage of the request, the DM320 USB controller will send a STALL to tell the host that the request was not executed. A second endpoint 0 interrupt will be generated and the SENTST bit of PER_CSR0 will be set.

If the host sends more data after the DATAEND bit has been set, then the DM320 USB controller will send a STALL. This will generate an endpoint 0 interrupt and the SENTST bit of PER_CSR0 will be set.

18.8.1.2 Write Requests

Write requests involve an additional packet (or packets) of data being sent from the host after the 8-byte command. An example of such standard device request is: SET_DESCRIPTOR.

The sequence of events will begin, when the software receives an endpoint 0 interrupt. The RXPKTRDY bit of PER_CSR0 is also set. The 8-byte command should then be read from the endpoint 0 FIFO and decoded.

Like a zero data request, the PER_CSR0 register should then be written to set the CLRRXRDY bit, indicating that the command has been read from the FIFO. Whereas the DATAEND bit should not be set, indicating that more data is expected.

When a second endpoint 0 interrupt is received, the PER_CSR0 register should be read to check the endpoint status. The RXPKTRDY bit should be set to indicate that a data packet has been received. The COUNT0 register should then be read to determine the size of this data packet. The data packet can then be read from the endpoint 0 FIFO.

If the length of the data associated with the request (indicated by the *wLength* field in the command) is greater than the maximum packet size for endpoint 0, further data packets will be sent. In this case, PER_CSR0 should be written to set the CLRRXRDY bit, but the DATAEND bit should not be set.

When all the expected data packets have been received, the PER_CSR0 register should be written to set the CLRRXRDY bit and DATAEND bit.

When the host moves to the status stage of the request, another endpoint 0 interrupt will be generated to indicate that the request has completed. The interrupt is just a confirmation that the request completed successfully.

If the host sends more data after the DATAEND has been set, then the DM320 USB controller will send a STALL. An endpoint 0 interrupt will be generated and the SENTST bit will be set.

If the command is an unrecognized command or cannot be executed, the PER_CSR0 register should be written to set the CLRRXRDY bit and SENDST bit. When the host sends more data, the DM320 USB controller will send a STALL to the host. An endpoint 0 interrupt will be generated and the SENTST bit will be set.

18.8.1.3 Read Requests

Read requests involves an additional packet (or packets) of data sent from the peripheral to the host after the 8-byte command. Examples of such standard device requests are: GET_CONFIGURATION, GET_INTERFACE, GET_DESCRIPTOR, GET_STATUS and SYNCH_FRAME.

The sequence of events will begin when the software receives an endpoint 0 interrupt. The RXPKTRDY bit of PER_CSR0 will also have been set. The 8-byte command should then be read from the endpoint 0 FIFO and decoded. The PER_CSR0 register should then be written to set the CLRRXRDY bit.

The data to be sent to the host should then be written to the endpoint 0 FIFO. If the data to be sent is greater than the maximum packet size for endpoint 0, only the maximum packet size should be written to the FIFO. The PER_CSR0

register should then be written to set the TXPKTRDY bit, indicating that there is a packet in the FIFO to be sent. When the packet has been sent to the host, another endpoint 0 interrupt will be generated and the next data packet can be written to the FIFO.

When the last data packet has been written to the FIFO, the PER_CSR0 register should be written to set the TXPKTRDY bit and to set the DATAEND bit.

When the host moves to the status stage of the request, another endpoint 0 interrupt will be generated to indicate that the request has been completed. The interrupt is just a confirmation that the request completed successfully.

If the host requests more data after DATAEND has been set, then the DM320 USB controller will send a STALL. An Endpoint 0 interrupt will be generated and the SENTST bit of PER_CSR0 will be set.

If the command is an unrecognized command or cannot be executed, the PER_CSR0 register should be written to set the CLRRXRDY bit and SENDST bit. When the host requests data, the DM320 USB controller will send a STALL to the host. An endpoint 0 interrupt will be generated and the SENTST bit will be set.

18.8.1.4 Endpoint 0 Interrupt Conditions

An endpoint 0 interrupt is generated when any of the following condition occurs:

- The core sets the RXPKTRDY bit of PER_CSR0 after a valid token has been received and data has been written to the FIFO
- The core clears the TXPKTRDY bit of PER_CSR0 after the packet of data in the FIFO has been successfully transmitted to the host
- The core sets the SENTST bit of PER_CSR0 after a control transaction is ended due to a protocol violation
- The core sets the SETEND bit of PER_CSR0 because a control transfer has ended before DATAEND bit of PER_CSR0 is set

18.8.1.5 Error handling

A control transfer may be aborted due to one of the following conditions:

- Protocol error on the USB
- Host prematurely ending the transfer
- DM320 USB controller software aborts the transfer (eg. because it cannot process the command)

The DM320 USB controller will automatically detect protocol errors and send a STALL packet to the host under the following conditions:

- *The host sends more data during the OUT Data phase of a write request than was specified in the command.* This condition is detected when the host sends an OUT token after the DATAEND bit of PER_CSR0 has been set.

- *The host request more data during the IN Data phase of a read request than was specified in the command.* This condition is detected when the host sends an IN token after the DATAEND bit in the PER_CSR0 register has been set.
- *The host sends more than Max Packet Size (RXMAXP register) data bytes in an OUT data packet.*
- *The host sends a non-zero length DATA1 packet during the STATUS phase of a read request.*

When the USB controller has sent the STALL packet, it sets the SENTST bit of PER_CSR0 and generates an interrupt. When the software receives an endpoint 0 interrupt with the SENTST bit set, it should abort the current transfer, clear the SENTST bit.

The host can prematurely end a transfer by entering the STATUS phase before all the data for the request has been transferred, or by sending a new SETUP packet before completing the current transfer. In this case, the SETEND bit of PER_CSR0 will be set and an endpoint 0 interrupt generated. When the software receives an endpoint 0 interrupt with the SETEND bit set, it should abort the current transfer, set the CLRSETEND bit of PER_CSR0. If the RXPKT RDY bit of PER_CSR0 is set indicating that the host has sent another SETUP packet, the software should process this command.

The software can abort the current transfer, in case it cannot process the command or has some other internal error, by setting the SENDST bit of PER_CSR0. The DM320 USB controller will then send a STALL packet to the host, set the SENTST bit and generate an endpoint 0 interrupt.

18.8.2 Control transactions in host mode

The following sections discuss the operation of DM320 USB controller as a host in the different phases of control transactions.

Before initiating any transactions as a host, the FADDR register needs to be set to address the peripheral device. When the device is first connected, FADDR should be set to zero. After a SET_ADDRESS command is issued, FADDR should be set the target's new address.

18.8.2.1 Setup phase

For the SETUP phase of a control transaction, the host driver needs to:

1. Load the 8 bytes of the required device request command into the endpoint 0 FIFO
2. Set SETPKT and TXPKTRDY bits of HST_CSR0. The controller then proceeds to send a SETUP token followed by the 8-byte command to endpoint 0 of the addressed device. The controller retries sending SETUP packet thrice on error conditions, before setting ERR bit of HST_CSR0 register.

Note: The SETPKT and TXPKTRDY bits should be set together.

3. At the end of the attempt to send the data packet, the DM320 USB controller will generate an endpoint 0 interrupt. The software should then read HST_CSR0 to establish whether the RXSTALL bit, the ERR bit or the NAKTO bit has been set.

If RXSTALL bit is set, it indicates that the target did not accept the command (e.g. because it is not supported by the target device) and so has issued a STALL response.

If ERR bit is set, it means that the controller has tried to send the SETUP packet and the following data packet three times without getting any valid response.

If NAKTO bit is set, it means that the USB controller has received a NAK response to each attempt to send the SETUP packet, for longer than the time set in the NAKLMT0 register. The controller can then be configured to either continue trying the transaction by clearing the NAKTO bit or abort the transaction by flushing the FIFO before clearing the NAKTO bit.

4. If none of RXSTALL, ERR or NAKTO bits are set, the SETUP phase has been correctly ACKed and the driver should proceed to the following IN Data phase, OUT Data phase or IN Status phase specified for the particular standard device request.

18.8.2.2 IN Data phase

For the IN Data phase of a control transaction, the host driver needs to:

1. Set REQPKT bit of HST_CSR0
2. Wait for the endpoint 0 interrupt while the USB controller sends the IN token and receives the required data.
3. When the USB controller generates the endpoint 0 interrupt, read HST_CSR0 to establish whether the RXSTALL bit, the ERR bit, the NAKTO bit or RXPkTRDY bit has been set.

If RXSTALL bit is set, it indicates that the target has issued a STALL response.

If ERR bit is set, it means that the USB controller has tried to send the required IN token three times without getting any valid response.

If NAKTO bit is set, it means that the USB controller has received a NAK response to each attempt to send the IN token, for longer than the time set in the NAKLMT0 register. The controller can then be configured to either continue trying the transaction by clearing the NAKTO bit or abort the transaction by clearing REQPKT bit before clearing the NAKTO bit.

4. If RXPkTRDY bit has been set, the software should read the data from the endpoint 0 FIFO, then clear RXPkTRDY bit.
5. If further data is expected, the driver should repeat steps 1 – 4.

When all the data has been successfully received, the driver should proceed to the OUT Status phase of the control transaction.

18.8.2.3 Out Data phase

For the OUT data phase of a control transaction, the host driver needs to:

1. Load the data to be sent into the endpoint 0 FIFO
2. Set the TXPKTRDY bit of HST_CSR0

3. The controller sends an OUT token followed by the data from the FIFO to endpoint 0 of the addressed device.

At the end of the attempt to send the data, the USB controller will generate an endpoint 0 interrupt. The software should then read HST_CSR0 to establish whether the RXSTALL bit, the ERR bit or the NAKTO bit has been set.

If RXSTALL bit is set, it indicates that the target has issued a STALL response.

If ERR bit is set, it means that the DM320 USB controller has tried to send the OUT token and the following data packet three times without getting any valid response.

If NAKTO bit is set, it means that the USB controller has received a NAK response to each attempt to send the OUT token, for longer than the time set in the NAKLMT0 register. The controller can be configured to either continue trying the transaction by clearing the NAKTO bit or abort the transaction by flushing the FIFO before clearing the NAKTO bit.

If none of RXSTALL, ERR or NAKTO is set, the OUT data has been correctly ACKed.

4. If further data needs to be sent, the driver should repeat steps 1 – 3

When all the data has been successfully sent, the driver should proceed to the IN Status phase of the control transaction.

18.8.2.4 IN Status phase

IN Status phase follows either the Setup phase or the Out Data phase.

For the IN Status phase of a control transaction, the host driver needs to:

1. Set STPKT and REQPKT bits of HST_CSR0.

Note: These bits should be set together.

2. Wait for endpoint 0 interrupt while the USB controller sends an IN token and receives a response from the USB peripheral.
3. When the controller generates the endpoint 0 interrupt, read HST_CSR0 to establish whether the RXSTALL bit, the ERR bit, the NAKTO bit or RXPkTRDY bit has been set.

If RXSTALL bit is set, it indicates that the target could not complete the command and so has issued a STALL response.

If ERR bit is set, it means that the USB controller has tried to send the required IN token three times without getting any valid response.

If NAKTO is set, it means that the USB controller has received a NAK response to each attempt to send the IN token, for longer than the time set in the NAKLMT0 register. The controller can be configured to either continue trying the transaction by clearing the NAKTO bit or abort the transaction by clearing REQPKT and STPKT bits before clearing the NAKTO bit.

4. If RXPkTRDY bit has been set, the driver should clear RXPkTRDY bit of HST_CSR0 register.

18.8.2.5 Out Status phase

OUT Status phase follows the IN Data phase.

For the OUT Status phase of a control transaction, the host driver needs to:

1. Set STPKT and TXPKTRDY bits of HST_CSR0.

Note: These bits should be set together.

2. Wait for endpoint 0 interrupt while the USB controller sends the OUT token and a zero-length DATA1 packet.
3. At the end of the attempt to send the data, the USB controller will generate an endpoint 0 interrupt. The software should then read HST_CSR0 to establish whether the RXSTALL bit, the ERR bit or the NAKTO bit has been set.

If RXSTALL bit is set, it indicates that the target could not complete the command and so has issued a STALL response.

If ERR bit is set, it means that the USB controller has tried to send the STATUS packet and the following data packet three times without getting any valid response.

If NAKTO bit is set, it means that the controller has received a NAK response to each attempt to send the IN token, for longer than the time set in the NAKLMT0 register. The USB controller can be configured to either continue trying the transaction by clearing the NAKTO bit or abort the transaction by flushing the FIFO before clearing the NAKTO bit.

4. If none of RXSTALL bit, ERR bit or NAKTO bit is set, the STATUS phase has been correctly ACKed.

18.9 Bulk / Interrupt / Isochronous Transactions

Each endpoint in DM320 USB controller except endpoint 0 can be configured for bulk, interrupt or isochronous transactions.

18.9.1 Bulk / Interrupt / Isochronous IN Transactions in Peripheral Mode

The following features are available which can be configured in the endpoint control registers for bulk / interrupt / isochronous transactions:

- **Double Packet Buffering:** This feature can be enabled by setting the DPB bit in the TXFIFO2 register. When enabled, up to two packets can be stored in the FIFO awaiting transmission to the host.
- **DMA:** If DMA is enabled for the endpoint, a DMA request will be generated whenever the endpoint FIFO is able to accept another packet. This feature can be used to allow the DMA controller to load packets into the FIFO without processor intervention. DMA can be enabled by setting the DMAEN bit of TXCSR2 register.
- **AutoSet:** When the AutoSet feature is enabled, the TXPKTRDY bit of PER_TXCSR1 will be automatically set when a packet of TXMAXP bytes has been loaded into the FIFO. This is particularly useful when DMA is used to load the FIFO as it avoids the need for any processor intervention when loading individual packets during a large transfer. AutoSet feature is enabled by setting

AUTOSET bit in the TXCSR2 register. AutoSet feature is useful for bulk and interrupt transfers. It is not particularly useful for isochronous transfers because usually the packet transferred in isochronous transactions is less than the MaxPktSize.

18.9.1.1 Endpoint configuration

- Set TXMAXP register with maximum packet size for the endpoint. The value should be same as the *wMaxPacketSize* field of standard endpoint descriptor for the endpoint.
- Set the FIFO address for the endpoint in TXFIFO1 and TXFIFO2 registers. Also, set the maximum packet size in the SZ field of TXFIFO2 register. The packet size in SZ should be same as written in TXMAXP register.
- Set ISO bit of TXCSR2 to 1 register for isochronous transactions. Reset ISO bit of TXCSR2 to 0 for bulk or interrupt transactions.
- Set DMAEN and AUTOSET bits of TXCSR2 depending upon the DMA requirements
- In interrupt mode, set FRDATTOG bit in TXCSR2 to force data toggle for each packet. In other modes, this bit should be 0.

During initial configuration of endpoint, following a SET_CONFIGURATION or SET_INTERFACE command on endpoint 0, set the CLRDATTOG bit in PER_TXCSR1 register. This ensures that the data toggle (which is handled automatically by the USB controller) starts in the correct state. Also, if there are any data packets in the FIFO (indicated by the FIFOEMP bit of PER_TXCSR1 being set), they should be flushed by setting the FLFIFO bit of PER_TXCSR1.

Note: If double buffering is enabled, it is necessary to set FLFIFO bit of PER_TXCSR1 twice in succession

18.9.1.2 Operation

When data is to be transferred over a Bulk / Interrupt / Isochronous IN pipe, a data packet needs to be loaded into the FIFO and set the TXPKTRDY bit in PER_TXCSR1 register. When the packet has been sent, the USB controller will clear the TXPKTRDY bit and an interrupt is generated so that the next packet can be loaded into the FIFO. If double packet buffering is enabled, then after the first packet has been set to transfer, the controller will immediately clear the TXPKTRDY bit and an interrupt is generated so that a second packet can be loaded into the FIFO.

The packet size must not exceed the size specified in the TXMAXP register. When a block of data larger than TXMAXP is to be transferred, it must be sent as multiple packets. These packets should be TXMAXP in size, except the last packet that holds the residue. The host may determine that all the data has been transferred from the total amount of data that is expected. Alternatively it may infer that all the data have been transferred when it receives a packet, which is less than TXMAXP in size. In the latter case, if the total size of the data block is a multiple of TXMAXP, it will be necessary to send a null packet after all the data has been sent. A null packet is sent by setting TXPKTRDY after the next interrupt is received, without loading any data into the FIFO.

If large blocks of data are being transferred, then the overhead of calling an interrupt service routine to load each packet can be avoided by using DMA. DMA modes of operation are discussed in section 18.14.

In the double buffer enabled mode, there is an exception for the isochronous transfers. Unless the ISO bit in POWER register is set, more than one data packet will be transferred on receipt of multiple IN tokens from the host, in the same frame. When ISO bit is set, any data packet loaded into an Isochronous Tx endpoint FIFO will not be transmitted until after the next SOF packet has been received.

18.9.1.3 Error Handling in Bulk / Interrupt Transactions

To shut down the bulk / interrupt IN pipe, set the SENDST bit PER_TXCSR1. When the USB controller receives further IN tokens, it will send a STALL to the host, set the SENTST bit of PER_TXCSR1 and generate an interrupt.

When an interrupt is received with the SENTST bit of PER_TXCSR1 set, clear the SENTST bit. SENDST bit should remain set until the IN pipe is ready to be enabled. When a pipe is enabled, the data toggle sequence should be restarted by setting the CLRDATTOG bit in the PER_TXCSR1 register.

18.9.1.4 Error Handling in Isochronous Transactions

If the endpoint has no data in its FIFO when an IN token is received, it will send a null data packet to the host and set the UNDERRUN bit in the PER_TXCSR1 register. This is an indication that data is not supplied fast enough for the host. This error condition needs to be handled by the application.

If the software is loading one packet per frame and it finds that the TXPKTRDY bit in the PER_TXCSR1 register is set when it tries to load the next packet, this indicates that the previous data packet has not been sent. It is up to the application how it handles this condition: it may choose to flush the unsent packet by setting the FLFIFO bit in the PER_TXCSR1 register, or it may choose to skip the current packet.

18.9.2 Bulk / Interrupt / Isochronous OUT Transactions in Peripheral Mode

The following features are available which can be configured in the endpoint control registers for bulk / interrupt / isochronous transactions:

- **Double Packet Buffering:** This feature can be enabled by setting the DPB bit in the

RXFIFO2 register. When enabled, up to two packets can be stored in the FIFO.

- **DMA:** If DMA is enabled for the endpoint, a DMA request will be generated whenever the endpoint has a packet in its FIFO. This feature can be used to allow the DMA controller to unload packets from the FIFO without processor intervention. DMA can be enabled by setting the DMAEN bit of PER_RXCSR2 register.
- **AutoClear:** When the AutoClear feature is enabled, the RXPKTRDY bit of PER_RXCSR1 will be automatically cleared when a packet of RXMAXP bytes has been unloaded from the FIFO. This is particularly useful when DMA is used to unload the FIFO as it avoids the need for any processor intervention when unloading individual packets during a large transfer. AutoClear feature is enabled by setting AUTOCLR bit in the PER_RXCSR2 register. AutoClear feature is useful for bulk and interrupt transfers. It is not useful for isochronous transfers because usually the packet transferred in isochronous transactions is less than the MaxPktSize.

18.9.2.1 Endpoint configuration

- Set RXMAXP register with maximum packet size for the endpoint. The value should be same as the *wMaxPacketSize* field of standard endpoint descriptor for the endpoint.
- Set the FIFO address for the endpoint in RXFIFO1 and

RXFIFO2 registers. Also, set the maximum packet size in the SZ field of

RXFIFO2 register. The packet size in SZ should be same as written in RXMAXP register.

- Set ISO bit of PER_RXCSR2 register for isochronous transactions. Reset it to 0 for bulk or interrupt transactions.
- Set DMAEN and AUTOCLR bits of PER_RXCSR2 depending upon the DMA requirements

When the endpoint is initially configured, following a SET_CONFIGURATION or SET_INTERFACE command on endpoint 0, the CLRDATTOG bit in PER_RXCSR1 register should be set. This ensures that the data toggle starts in the correct state. Also, if there are any data packets in the FIFO (indicated by the RXPBKTRDY bit of PER_RXCSR1 being set), they should be flushed by setting the FLFIFO bit of PER_RXCSR1.

Note: If double buffering is enabled, it is necessary to set FLFIFO bit of PER_RXCSR1 twice in succession.

18.9.2.2 Operation

When a data packet is received by Rx endpoint, the RXPBKTRDY bit of PER_RXCSR1 is set and an interrupt is generated. The software should read the RXCOUNT1 and RXCOUNT2 registers for the endpoint to determine the size of the data packet received. The data packet should be read from the FIFO, then the RXPBKTRDY bit should be cleared.

The packet sizes should not exceed the size specified in the RXMAXP register. When a block of data larger than RXMAXP is to be sent to the function, it will be sent as multiple packets. All the packets will be RXMAXP in size, except the last packet that contains the residue. An application specific method should be used for determining the total size of the block. Alternatively it may infer that the entire block has been received when it receives a packet, which is less than RXMAXP in size. If the total size of the data block is a multiple of RXMAXP, a null data packet will be sent after the data to signify that the transfer is complete.

If large blocks of data are being transferred, the overhead of calling an interrupt service routine to unload each packet can be avoided by using DMA. DMA modes of operation are discussed in section 18.14.

18.9.2.3 Error Handling in Bulk / Interrupt transactions

To shut down the Bulk / Interrupt OUT pipe, set the SENDST bit of PER_RXCSR1. When the controller receive further packets it will send a STALL to the host, set the SENTST bit of PER_RXCSR1 and generate an interrupt.

When an interrupt is received with the SENTST bit of PER_RXCSR1 set, clear the SENTST bit. SENDST bit should remain set until the Bulk / Interrupt OUT pipe are ready to be enabled. When a Bulk OUT pipe is enabled, the data toggle sequence should be restarted by setting the CLRDATTOG bit in the PER_RXCSR1 register.

18.9.2.4 Error Handling in Isochronous transactions

If there is no space in the FIFO to store a packet when it is received from the host, the OVERRUN bit in the PER_RXCSR1 register will be set. This is an

indication that data is not unloaded fast enough. Application can determine how this error condition is handled.

If the USB controller finds that a received packet has a CRC error, it will still store the packet in the FIFO and set the RXPKTRDY bit and the DATERR bit of PER_RXCSR1. Application can determine how this error condition is handled.

18.9.3 Bulk / Interrupt / Isochronous IN Transactions in Host Mode

The following features are available which can be configured in the endpoint control registers for bulk / interrupt / isochronous transactions:

- **Double Packet Buffering:** This feature can be enabled by setting the DPB bit in the

RXFIFO2 register. When enabled, up to two packets can be stored in the FIFO.

- **DMA:** If DMA is enabled for the endpoint, a DMA request will be generated whenever the endpoint has a received packet in its FIFO. This feature can be used to allow the DMA controller to unload packets from the FIFO without processor intervention. DMA can be enabled by setting the DMAEN bit of HST_RXCSR2 register.
- **AutoRequest:** When the Autorequest feature is enabled, the REQPKT bit in HST_RXCSR1 will be automatically set when the RXPKT_RDY bit is cleared. This feature is enabled by setting the AUTOREQ bit in the HST_RXCSR2 register.
- **AutoClear:** When the AutoClear feature is enabled, the RXPKT_RDY bit of HST_RXCSR1 will be automatically cleared when a packet of RXMAXP bytes has been unloaded from the FIFO. This is particularly useful with AutoRequest feature when DMA is used to unload the FIFO as it avoids the need for any processor intervention when unloading individual packets during a large transfer. AutoClear feature is enabled by setting AUTOCLR bit in the HST_RXCSR2 register. AutoClear feature is useful for bulk and interrupt transfers. It is not useful for isochronous transfers because usually the packets transferred in isochronous transactions are less than the MaxPktSize.

18.9.3.1 Endpoint configuration

- PROTO field in the RXTYPE register for the host endpoint should be written with type of transfer(bulk, interrupt and isochronous). EP field in RXTYPE register should be set to the value of the endpoint number of the target peripheral.
- Set RXMAXP register with maximum packet size for the endpoint. The value should be same as the *wMaxPacketSize* field of standard endpoint descriptor for the target endpoint.
- Set the RXINTVL with the required value for the NAK timeout interval or set it to zero if NAK time out is not required
- Set the FIFO address for the endpoint in RXFIFO1 and

RXFIFO2 registers. Also, set the maximum packet size in the SZ field of

RXFIFO2 register. The packet size in SZ should be same as in RXMAXP register.

- Set DMAEN, AUTOREQ and AUTOCLR bits of HST_RXCSR2 depending upon the DMA requirements

When the endpoint is initially configured, following a SET_CONFIGURATION or SET_INTERFACE command on endpoint 0, set the CLRDATTOG bit of HST_RXCSR1 register. This will ensure that the data toggle starts in the correct state. Also, if there are any data packets in the FIFO (indicated by the RXPKT RDY bit of HST_RXCSR1 being set), they should be flushed by setting the FLFIFO bit of HST_RXCSR1.

Note: If double buffering is enabled, it is necessary to set FLFIFO bit of HST_RXCSR1 twice in succession.

18.9.3.2 Operation

When data is required from the USB peripheral, set the REQPKT bit in the corresponding HST_RXCSR1 register. The USB controller will then send an IN token to the selected peripheral endpoint and waits for data.

If data is correctly received, RXPKT RDY bit of HST_RXCSR1 is set. If the USB peripheral responds with a STALL, RXSTALL bit of HST_RXCSR1 is set. If a NAK is received, the controller tries again – and continues to try until either the transaction is successful or the NAK limit set in the RXINTVL register is reached. If no valid response is received, two more attempts are made before the controller reports an error by setting ERR bit of HST_RXCSR1.

The controller then generates the appropriate endpoint interrupt. The HST_RXCSR1 register should be read to determine whether the RXPKT RDY, RXSTALL, ERR or DATERR (NAK timeout) bits are set. If the DATERR (NAK timeout) bit is set, the controller can be configured to either continue trying the transaction by clearing the DATERR bit or to abort the transaction by clearing REQPKT before clearing the DATERR bit.

If large blocks of data are being transferred, the overhead of calling an interrupt service routine to unload each packet can be avoided by using DMA. DMA modes of operation are discussed in section 18.14.

18.9.3.3 Error Handling in Bulk / Interrupt transactions

If the target peripheral wants to shut down the Bulk / Interrupt IN pipe, it will send a STALL response to the IN token. As a result, the RXSTALL bit of HST_RXCSR1 gets set and an interrupt is generated. Application needs to handle this error condition.

18.9.3.4 Error Handling in Isochronous transactions

If a CRC or bit-stuff error occurs during the reception of a packet, the packet will still be stored in the FIFO but the DATERR bit of HST_RXCSR1 is set to indicate that the data may be corrupt. An interrupt is generated and the application needs to handle this error condition.

18.9.4 Bulk / Interrupt / Isochronous OUT Transactions in Host Mode

The following features are available which can be configured in the endpoint control registers for bulk / interrupt / isochronous transactions:

- **Double Packet Buffering:** This feature can be enabled by setting the DPB bit in the TXFIFO2 register. When enabled, up to two packets can be stored in the FIFO awaiting transmission to the peripheral.
- **DMA:** If DMA is enabled for the endpoint, a DMA request will be generated whenever the endpoint FIFO is able to accept another packet. This feature can be used to allow the DMA controller to load packets into the FIFO without processor intervention. DMA can be enabled by setting the DMAEN bit of TXCSR2 register.
- **AutoSet:** When the AutoSet feature is enabled, the TXPKTRDY bit of HST_TXCSR1 will be automatically set when a packet of TXMAXP bytes has been loaded into the FIFO. This is useful when DMA is used to load the FIFO as it avoids the need for any processor intervention when loading packets during a large transfer. AutoSet feature is enabled by setting AUTOSSET bit in the TXCSR2 register. AutoSet feature is useful for bulk and interrupt transfers. It is not useful for isochronous transfers because usually the packet transferred in isochronous transactions is less than the MaxPktSize.

18.9.4.1 Endpoint configuration

- PROTO field in the TXTYPE register for the host endpoint should be written with type of transfer(bulk, interrupt and isochronous). EP field in TXTYPE register should be set to the value of the endpoint number of the target peripheral.
- Set TXMAXP register with maximum packet size for the endpoint. The value should be same as the *wMaxPacketSize* field of standard endpoint descriptor for the target endpoint.
- Set the TXINTVL register with the required value for the NAK timeout interval or set it to zero if NAK time out is not required
- Set the FIFO address for the endpoint in TXFIFO1 and TXFIFO2 registers. Also, set the maximum packet size in the SZ field of TXFIFO2 register. The packet size in SZ should be same as in TXMAXP register.
- Set DMAEN and AUTOSSET bits of TXCSR2 depending upon the DMA requirements

When the endpoint is initially configured, following a SET_CONFIGURATION or SET_INTERFACE command on endpoint 0, set the CLRDATTOG bit of the HST_TXCSR1 register. This will ensure that the data toggle starts in the correct state. Also, if there are any data packets in the FIFO (indicated by the FIFOEMP bit of HST_TXCSR1 being set), they should be flushed by setting the FLFIFO bit of HST_TXCSR1.

Note: If double buffering is enabled, it is necessary to set FLFIFO bit of HST_TXCSR1 twice in succession.

18.9.4.2 Operation

When data is to be sent to the USB peripheral, write the first packet of the data to the FIFO (or two packets if double-buffered) and set the TXPKTRDY bit in the

HST_TXCSR1 register. The controller will then send an OUT token to the selected peripheral endpoint, followed by the first data packet from the FIFO.

If data is correctly received by the peripheral, an ACK from the peripheral should be received whereupon the DM320 USB controller will clear TXPKTRDY bit of HST_TXCSR1. If the USB peripheral responds with a STALL, RXSTALL bit of HST_TXCSR1 is set. If a NAK is received, the controller tries again – and continues to try until either the transaction is successful or the NAK limit set in the TXINTVL register is reached. If no valid response is received, two more attempts are made before the controller reports an error by setting ERR bit of HST_TXCSR1.

The controller then generates the appropriate endpoint interrupt. HST_TXCSR1 register should be read to determine whether the RXSTALL, ERR or NAKTO bit is set. If the NAKTO bit is set, the controller can be configured to either continue trying the transaction by clearing the NAKTO bit or abort the transaction by flushing the FIFO before clearing the NAKTO bit.

The packet sizes should not exceed the size specified in the TXMAXP register. When a block of data larger than TXMAXP needs to be sent, it needs to be sent as multiple packets. These packets should be of TXMAXP size, except the last packet that holds the residue. If the total size of the data block is a multiple of TXMAXP, the host will need to send a null packet after all the data has been sent. A null packet is sent by setting TXPKTRDY after the next interrupt is received, without loading any data into the FIFO.

If large blocks of data are being transferred, then the overhead of calling an interrupt service routine to load each packet can be avoided by using DMA. DMA modes of operation are discussed in section 18.14.

18.9.4.3 Error Handling in Bulk / Interrupt Transactions

If the target peripheral wants to shut down the Bulk / Interrupt OUT pipe, it will send a STALL response. As a result, RXSTALL bit of HST_TXCSR1 gets set and an interrupt is generated. Application needs to handle this error condition.

18.10 Connect / Disconnect

The particular behavior related to connecting and disconnecting the USB controller depends on its use either in host mode or peripheral mode in peer-to-peer communications.

18.10.1 In Host Mode

When the DM320 USB controller is operating in host mode, the session is started by setting the SESSREQ bit of DEVCTL. Power is then applied to VBus and the controller waits for a device to be connected.

When a device connection is detected, a connect interrupt is generated and CONN bit of INTRUSB gets set. Whether a full-speed or a low-speed device has been connected can be determined by reading the DEVCTL register. The FSDEV bit will be set for a full speed device and the LSDEV bit will be set for a low-speed device.

The software should then reset the device and begin device enumeration.

If the device is disconnected while a session is in progress, a disconnect interrupt will be generated and DISCON bit of INTRUSB gets set.

18.10.2 In Peripheral Mode

Where the DM320 USB controller is operating in peripheral mode, no interrupt is generated when the device is connected to the host. However a disconnect interrupt is generated when the host terminates a session.

While a session is in progress, the peripheral can execute a 'soft' disconnect by clearing the SESSREQ bit of DEVCTL register. The USB controller just disconnects its pull-up resistor but, as far as the host is concerned, it is equivalent to the peripheral being detached from the bus. The host then ends the session.

18.11 OTG Session Request

In order to conserve power, the *USB On-The-Go* supplement allows VBus to be powered up only when required and to be turned off when there is bus overloading or no active session .

VBus is always supplied by the 'A' device on the bus. The DM320 USB controller determines whether it is the 'A' device by monitoring the CID input pin. The status of CID bit of DEVCTL register is used to determine whether DM320 is an 'A' device or a 'B' device. This bit is '0' when operating as 'A' device and '1' when operating as 'B' device.

18.11.1 USB Controller as 'A' Device

Starting a Session: When the USB controller is the 'A' device on the bus, a session is started by setting the SESSREQ bit in the DEVCTL register. The controller will then turn on VBus and wait for VBus to go above the VBus Valid threshold, as indicated by the VBUSVAL input going high. This event also causes the VBUSVAL bit in the POWER register to be set.

USB controller will then enter host mode (the 'A' device is always the default host) and wait for a peripheral to be connected.

A connect interrupt will be generated when a peripheral is detected. FSDEV or LSDEV bit of the DEVCTL register will be set depending on whether a full-speed or low-speed peripheral was detected. Then the driver should reset and enumerate the device.

To end the session, clear the SESSREQ bit of DEVCTL.

Detecting activity: If the controller detects data line or VBus pulsing on the bus when a session is not in progress, it will generate a session request interrupt and set SESSREQ bit of INTRUSB to indicate that the 'B' device is requesting a session. The controller can then start a session by setting the SESSREQ bit of DEVCTL register.

18.11.2 USB Controller as 'B' Device

Detecting activity: When the DM320 USB controller is the 'B' device on the bus and it detects that VBus goes above the Session Valid threshold, it will set the SESSREQ bit in the DEVCTL register. The valid session is indicated by the VBUSSES input being high and reflected by the VBUSSESS bit being set in the POWER register. When reset signaling is detected on the bus, a reset interrupt

will be generated and RESET bit in INTRUSB gets set. This is interpreted as the start of a session.

The controller will be in peripheral mode as the 'B' device is the default peripheral.

When VBus drops below the Session Valid threshold, the SESSREQ bit will be cleared and a disconnect interrupt will be generated to indicate that the session has ended.

Starting a Session: To request a session, set the SESSREQ bit of DEVCTL register. This will cause the USB controller to request a session using the Session Request Protocol defined in the *USB On-The-Go* supplement.

The controller will wait for the initial conditions to be met: VBus below the Session End threshold and SE0 for > 2 ms. Then the controller will pulse the data line to request a session. If the session does not start, controller will pulse the VBus to request a session.

Terminating a Session: Once started, the session will normally be terminated by the 'A' device. However, it is also possible for the 'B' device to force the session to end by clearing the SESSREQ bit of DEVCTL register. This enables the controller to execute a 'soft' disconnect from the USB bus and to disconnect its pull-up resistor. The 'A' device will then end the current session.

Note: It is possible that the controller may receive a spurious disconnect interrupt after requesting a session start. This can occur if VBus pulsing takes VBus above the Session Valid threshold, but VBus then falls below the Session Valid threshold before the 'A' device starts the session.

18.12 Host Negotiation

When the USB controller is the 'A' device (CID bit of DEVCTL = 0), it will automatically enter host mode when a session starts.

When the USB controller is the 'B' device (CID bit of DEVCTL = 1), it will automatically enter peripheral mode when a session starts. The driver can however request that the USB controller becomes the host by setting the HOSTREQ bit in the DEVCTL register. This bit can be set either when the driver requests a session start by setting the SESSREQ bit of DEVCTL or at any time after a session has started. When the controller detects next bus suspend, if the HOSTREQ bit remains set, it will enter Host mode. Controller will then disconnect the pull-up resistor to begin host negotiation (as specified in the *USB On-The-Go* supplement). This should cause the 'A' device to switch to peripheral mode by connecting its pull-up resistor. When the controller detects this, it will generate a connect interrupt. It will also set the RESET bit in the POWER register to begin resetting the 'A' device. The controller begins this reset sequence automatically to ensure that reset is started as required within 1 ms of the 'A' device connecting its pull-up resistor. The driver should wait at least 20 ms, then clear the RESET bit and enumerate the 'A' device.

When the 'B' device has finished using the bus, the driver should put bus into suspend mode by setting the SUSPEND bit of the POWER register. The 'A' device will detect this and either terminate the session or revert to host mode. If the 'A' device is based on DM320 USB controller, it will generate a disconnect interrupt.

18.13 VBus Events

The *USB On-The-Go* specification defines a series of thresholds to which the devices involved in point-to-point communications are required to respond.

- VBus Valid (required to be greater than 4.4V)
- Session Valid (required to be between 0.8V and 4.4V)
- Session End (required to be between 0.2V and 0.8V)

Which of these thresholds are critical and how the driver responds depends on whether the controller is 'A' device or 'B' device. The required actions are summarized below.

18.13.1 USB Controller actions as an 'A' Device

VBus above VBus Valid threshold with session initiated by 'A' device: When VBus becomes greater than VBus Valid, the controller selects host mode and waits for a device to be connected. It then generates a connect interrupt. The driver should reset and enumerate the connected 'B' device.

VBus above Session Valid threshold with session initiated by 'B' device: When VBus becomes greater than Session Valid, the controller will generate a session request interrupt. The driver should set the SESSREQ bit of DEVCTL. The controller will then either stay in host mode or change to peripheral mode depending on the state of the pull-up resistor on the 'B' device. The selected mode will be indicated by the state of the MODE bit of DEVCTL.

VBus below VBus Valid threshold while the SESSREQ bit remains set: This indicates a problem with the VBus power level. For example, the battery power may have dropped too low to sustain VBus Valid. Alternatively, the 'B' device may be drawing more current than the 'A' device can provide. In either case, the USB controller will automatically terminate the session and generate a Vbus Error interrupt and set VBUSERR bit of INTRUSB.

18.13.2 USB Controller actions as a 'B' Device

VBus above Session Valid threshold: This indicates activity from the 'A' device. The USB controller will set the SESSREQ bit of DEVCTL and take PUCON high in order to connect the pull-up resistor to the D+ line. It will also set the PUCON bit in the DEVCTL register.

VBus above Session Valid threshold while the SESSREQ bit remains set: This indicates that the 'A' device has lost power or has been disconnected. The USB controller will clear the SESSREQ bit of DEVCTL and generate a disconnect interrupt. The software should end the session.

VBus below Session End threshold with the HOSTREQ bit of DEVCTL set: This is the condition for a change of host. After 3ms with no activity on the bus, the USB controller will enter host mode and disconnect the pull-up resistor to begin host negotiation. When the 'A' device connects its pull-up resistor, the USB controller will generate a connect interrupt. It will also set the RESET bit of POWER register to begin resetting the 'A' device. The driver should wait at least 20ms, then clear the RESET bit and enumerate the 'A' device.

18.14 USB DMA Controller

AHB Bridge in the DM320 USB controller includes 4-channel DMA controller for efficient loading/unloading of the endpoint FIFOs. The DMA controller has 32 bit control registers and its interrupt controller. It supports two mode of operation DMA Mode0 and Mode1, and operating mode of each channel can be programmed independently.

18.14.1 DMA Bus Cycles

The maximum DMA transfer size is the maximum size of a USB packet or the maximum size of the internal DMA buffer(32bytes), whichever is smaller. The DMA controller does not support split retries and treat them as bus error. If a bus error occurs while the DMA controller is accessing memory on the AHB, the DMA controller will immediately terminate the transfer and interrupt the processor with bus error (BUSERR bit of the CNTLn register for the channel).

The start address should be word aligned. Half-word or byte aligned address is not supported. All the transfers for a packet will be word transfers (32bits), with possible half-word and/or byte transfers added at the end to handle any residue.

18.14.2 DMA Mode0 Operation

DMA Mode0 can be used with any endpoint. In this mode, the DMA controller can be programmed to load/unload only one packet. Thus processor intervention is required for each packet transferred over the USB. The below setup is required for Mode0 operation.

Mode0 for Rx Endpoints

- The relevant interrupt enable bit in the INTRRX1 register should be set.
- The DMAEN bit of the PER_RXCSR2 or HST_RXCSR2 register should be set.

When the core has received a packet in the FIFO, it will generate an interrupt. The driver should then program the DMA controller as follows:

- **ADDR:** Target memory address where the packet is to be stored
- **COUNT:** Size of packet (determined by reading the RXCOUNT1 and RXCOUNT2 register)
- **CNTL:** DMAEN = 1, DIR = 0, DMAMODE = 0, INTREN = 1, EPNUM = endpoint number for the data transfer

Then the DMA controller will start the transfer. It will generate an USB DMA interrupt when transfer is completed. The driver can then clear the RXPKTRDY bit in HST_RXCSR1 or PER_RXCSR1 register.

Mode0 for Tx Endpoints

- The relevant interrupt enable bit in the INTRTX1 register should be set.
- The DMAEN bit of the TXCSR2 register should be set.

When the FIFO becomes available for data transfer, the driver should then program DMA controller as follows:

- **ADDR:** Memory address of the packet to be sent
- **COUNT:** Size of packet to be sent
- **CNTL:** DMAEN = 1, DIR = 1, DMAMODE = 0, INTREN = 1, EPNUM = endpoint number for the data transfer

Then the DMA controller will start the transfer. It will generate an USB DMA interrupt when transfer is completed. The driver can then set the TXPKTRDY bit in PER_TXCSR1 or HST_TXCSR1 register.

18.14.3 DMA Mode1

DMA Mode1 can only be used with endpoint configured for bulk transactions. In this mode, the DMA controller can be programmed to load/unload complete bulk transfer data (which can be multiple packets). Once set up, the DMA controller loads/unloads all packets of the transfer, interrupting the processor only when all the packets have been transferred.

Mode1 for Rx Endpoints

- **ADDR:** Target memory address where the packet is to be stored
- **COUNT:** Maximum available size of data buffer.
- **CNTL:** DMAEN = 1, DIR = 0, DMAMODE = 1, INTREN = 1, MAXPKTSZ = the value set in the corresponding RXMAXP register, EPNUM = endpoint number for the data transfer

Receive endpoint of the core should be programmed as follows:

- The relevant interrupt enable bit in the INTRRX1 register should be set
- The AUTOCLR, DMAEN and DMAMODE bits of the PER_RXCSR2 or HST_RXCSR2 register should be set

When the controller receives a packet in the FIFO, the DMA controller will start the transaction. The core will automatically clear the RXPkTRDY bit in the PER_RXCSR1 or HST_RXCSR1 register. This process will continue automatically until the core receives a 'short packet' (one which is smaller than the maximum packet size for the endpoint) signifying the end of transfer. The DMA controller will not transfer this 'short packet' in Mode1, instead the controller will generate an USB DMA interrupt. The driver can then read the RXCOUNT1 and

RXCOUNT2 register to determine the size of the 'short packet' and either unload it by CPU access or program the DMA controller in Mode0 to unload the packet. The DMA controller ADDR register will be incremented as the packets were unloaded, so the processor can compare the current value of ADDR with the start address of the memory buffer to determine the size of the transfer.

Note: If the size of the transfer exceeds the data buffer size, the DMA controller will stop unloading the FIFO and generate an USB DMA interrupt.

Mode 1 for Tx Endpoints

- **ADDR:** Memory address of data block to be sent
- **COUNT:** Size of data block
- **CNTL:** DMAEN = 1, DIR = 1, DMAMODE = 1, INTREN = 1, MAXPKTSZ = the value set in the corresponding TXMAXP register, EPNUM = endpoint number for the data transfer

Transmit endpoint of the core should be programmed as follows:

- The relevant interrupt enable bit in the INTRTX1 register should be set
- The AUTOSET, DMAEN and DMAMODE bits of TXCSR2 register should be set

When the FIFO becomes available for loading the data, the DMA controller will start the transfer. The controller will automatically set the TXPKTRDY bit in PER_TXCSR1 or HST_TXCSR1 register. This process will continue until the entire data block has been transferred to the core. The DMA controller will then interrupt the processor. If the last packet to be loaded was less than the maximum packet size for the endpoint, the TXPKTRDY bit will not be set. So the processor should set the TXPKTRDY bit to allow the last 'short packet' to be sent. If the last packet to be loaded was of the maximum packet size, the processor should still set the TXPKTRDY bit in order to send a null packet signifying the end of the transfer.

Note: In this mode, the USB DMA interrupt is generated when the data has been loaded into the FIFO if the last packet is a short packet or if the last packet is a maximum packet with double packet buffering enabled and the second packet buffer is empty.

If the DMA FIFO load ends with a maximum packet and there is no empty packet buffer, then the USB DMA interrupt will not be generated until a packet has been sent on the bus and a buffer is available. The USB DMA interrupt is delayed so that when the USB DMA interrupt occurs software can write to the PER_TXCSR1 or HST_TXCSR1 register to set TXPKTRDY to send a null packet.

18.15 USB Register Map

Address	Register	Description
8000:0000	FADDR	Peripheral Address Register
8000:0001	POWER	Power Control Register
8000:0002	INTRTX1	Transmit EP Interrupt Status Register #1
8000:0003	RSV1	Reserved
8000:0004	INTRRX1	Receive EP Interrupt Status Register #1
8000:0005	RSV2	Reserved
8000:0006	INTRUSB	USB Interrupt Status Register
8000:0007	INTRTX1E	Transmit EP Interrupt Enable Register #1
8000:0008	RSV3	Reserved
8000:0009	INTRRX1E	Receive EP Interrupt Enable Register #1
8000:000A	RSV4	Reserved
8000:000B	INTRUSBE	USB Interrupt Enable Register
8000:000C	FRAME1	Lower Frame Number Register
8000:000D	FRAME2	Upper Frame Number Register
8000:000E	INDEX	Endpoint index Register
8000:000F	DEVCTL	Device Control Register
8000:0010	TXMAXP	Transmit Maximum Packet Size Register
8000:0011	PER_CSR0	Peripheral EP0 Control Register
8000:0011	PER_TXCSR1	Peripheral Tx EP Control Register #1
8000:0012	CSR02	EP0 Control Register #2
8000:0012	TXCSR2	Tx EP Control Register #2
8000:0013	RXMAXP	Receive Maximum Packet Size Register
8000:0014	PER_RXCSR1	Peripheral Rx EP Control Register #1
8000:0015	PER_RXCSR2	Peripheral Rx EP Control Register #2
8000:0016	COUNT0	Count EP0 Data Bytes Register
8000:0016	RXCOUNT1	Count EP Data bytes Register #1
8000:0017	RXCOUNT2	Count EP Data bytes Register #2
8000:0018	TXTYPE	EP Transmit Type Register
8000:0019	NAKGMT0	EP0 NAK Limit Register
8000:0019	TXINTVL	EP Transmit Interval Register
8000:001A	RXTYPE	EP Receive Type Register
8000:001B	RXINTVL	EP Receive Interval Register
8000:001C	TXFF01	EP Transmit FIFO Address Register #1
8000:001D	TXFF02	EP Transmit FIFO Address Register #2
8000:001E	RXFF01	EP Receive FIFO Address Register #1
8000:001F	RXFF02	EP Receive FIFO Address Register #2
8000:0011	HST_CSR0	Host EP0 Control Register
8000:0011	HST_TXCSR1	Host Tx EP Control Register #1
8000:0014	HST_RXCSR1	Host Rx EP Control Register #1
8000:0015	HST_RXCSR2	Host Rx EP Control Register #2
8000:0020	FF00	EP0 FIFO Access Register
8000:0024	FF01	EP1 FIFO Access Register
8000:0028	FF02	EP2 FIFO Access Register
8000:002C	FF03	EP3 FIFO Access Register
8000:0030	FF04	EP4 FIFO Access Register

18.15.1 FADDR

Peripheral Address Register

FADDR		8000:0000			offset:	0x00		default:	0x00	
Bit	7	6	5	4	3	2	1	0		
Name	RSV	ADDR[6]	ADDR[5]	ADDR[4]	ADDR[3]	ADDR[2]	ADDR[1]	ADDR[0]		
R/W	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W		
Default	-	0	0	0	0	0	0	0		

Bit	Name	Reset Value	R/W	Function
7	RSV			Reserved
6:0	ADDR	0	R/W	<u>Function Address</u> In Host Mode, value sent in SET_ADDRESS command during device enumeration In Peripheral Mode, address assigned by the host in SET_ADDRESS command

18.15.2 POWER

Power Control Register

POWER		8000:0001			offset: 0x01		default: 0x00	
Bit	7	6	5	4	3	2	1	0
Name	ISO	VBUSVAL	VBUSSESS	VBUSLO	RESET	RESUME	SUSPEND	ENSUS
R/W	R/W	R	R	R	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
7	ISO	0	R/W	<u>ISO Update field</u> 0: Send/Receive ISO packet without waiting for SOF 1: Send/Receive ISO packet only after SOF <i>Valid only in Peripheral Mode. Not valid in Host Mode</i>
6	VBUSVAL	0	R	<u>VBUSVAL input signal to USB</u> 0: Vbus below the Vbus Valid threshold 1: Vbus above the Vbus Valid threshold
5	VBUSSESS	0	R	<u>VBUSSESS input signal to USB</u> 0: Vbus below the Session Valid threshold 1: Vbus above the Session Valid threshold
4	VBUSLO	0	R	<u>VBUSLO input signal to USB</u> 0: Vbus below the Session End threshold 1: Vbus above the Session End threshold
3	RESET	0	R/W	<u>Reset Signal</u> Wr 0: In Host Mode, Clear the Reset signal on the bus Wr 1: In Host Mode, drive Reset signal on the Bus Rd 0: No Reset Signal Rd 1: In Peripheral Mode, Reset signal detected on the Bus In Host Mode, Host driving Reset <i>Read Only bit in Peripheral Mode</i>
2	RESUME	0	R/W	<u>Resume Signal</u> 0: Clear Resume signaling on the bus 1: Generate Resume signaling on the bus when device is in Suspend mode
1	SUSPEND	0	R/W	<u>Suspend Mode</u> Rd 1: In Peripheral Mode, Suspend mode is entered In Host Mode, Host driving Suspend Wr 1: In Host Mode, Set to enter Suspend mode <i>Read Only bit in Peripheral Mode.</i>
0	ENSUS	0	R/W	<u>Enable Suspend Mode</u> Rd/Wr 0: disable entry into Suspend Mode Rd/Wr 1: enable entry into Suspend Mode <i>Valid only in Peripheral Mode, Cleared when peripheral receives a RESET signal from host</i>

18.15.3 INTRTX1

Transmit EP Interrupt Status Register

INTRTX1	8000:0002			offset:	0x02		default:	0x00
Bit	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	EP4	EP3	EP2	EP1	EP0
R/W	-	-	-	R	R	R	R	R
Default	-	-	-	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
7:5	RSV			<u>Reserved</u>
4	EP4	0	R	<u>Tx Endpoint 4 interrupt</u> 0: no interrupt 1: interrupt active
3	EP3	0	R	<u>Tx Endpoint 3 interrupt</u> 0: no interrupt 1: interrupt active
2	EP2	0	R	<u>Tx Endpoint 2 interrupt</u> 0: no interrupt 1: interrupt active
1	EP1	0	R	<u>Tx Endpoint 1 interrupt</u> 0: no interrupt 1: interrupt active
0	EP0	0	R	<u>Tx / Rx Endpoint 0 interrupt</u> 0: no interrupt 1: interrupt active

18.15.4 INTRRX1

Receive EP Interrupt Status Register

INTRRX1 8000:0004 offset: 0x04 default: 0x00

Bit	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	EP4	EP3	EP2	EP1	RSV
R/W	-	-	-	R	R	R	R	-
Default	-	-	-	0	0	0	0	-

Bit	Name	Reset Value	R/W	Function
7:5	RSV			<u>Reserved</u>
4	EP4	0	R	<u>Rx Endpoint 4 interrupt</u> 0: no interrupt 1: interrupt active
3	EP3	0	R	<u>Rx Endpoint 3 interrupt</u> 0: no interrupt 1: interrupt active
2	EP2	0	R	<u>Rx Endpoint 2 interrupt</u> 0: no interrupt 1: interrupt active
1	EP1	0	R	<u>Rx Endpoint 1 interrupt</u> 0: no interrupt 1: interrupt active
0	RSV			Reserved

18.15.5 INTRUSB

USB Interrupt Status Register

INTRUSB		8000:0006			offset: 0x06		default: 0x00	
Bit	7	6	5	4	3	2	1	0
Name	VBUSERR	SESSREQ	DISCON	CONN	SOF	RESET	RESUME	SUSPEND
R/W	R	R	R	R	R	R	R	R
Default	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
7	VBUSERR	0	R	<u>Vbus Error Interrupt</u> 0: no interrupt 1: Vbus drops below VBus Valid threshold <i>Valid only for A device</i>
6	SESSREQ	0	R	<u>Session Request Signal Detection Interrupt</u> 0: no interrupt 1: Session Request Signal is detected <i>Valid only for A device</i>
5	DISCON	0	R	<u>Disconnect Interrupt</u> 0: no interrupt 1: In Host Mode, device disconnect is detected In Peripheral Mode, session end is detected
4	CONN	0	R	<u>Connection Interrupt</u> 0: no interrupt 1: Device connection is detected <i>Valid only in Host Mode</i>
3	SOF	0	R	<u>SOF Interrupt</u> 0: no interrupt 1: Start of Frame is detected
2	RESET	0	R	<u>Reset Interrupt</u> 0: no interrupt 1: In Peripheral Mode, reset signal is detected In Host Mode, Babble is detected
1	RESUME	0	R	<u>Resume Interrupt</u> 0: no interrupt 1: Resume signal is detected when USB controller is in Suspend Mode
0	SUSPEND	0	R	<u>Suspend Interrupt</u> 0: no interrupt 1: Suspend signal is detected <i>Valid only in Peripheral Mode</i>

18.15.6 INTRTX1E

Transmit EP Interrupt Enable Register

INTRTX1E	8000:0007			offset:	0x07		default:	0x1F
Bit	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	EP4	EP3	EP2	EP1	EP0
R/W	-	-	-	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	1	1	1	1	1

Bit	Name	Reset Value	R/W	Function
7:5	RSV			<u>Reserved</u>
4	EP4	1	R/W	<u>Enable Tx Endpoint 4 interrupt</u> 0: disable 1: enable
3	EP3	1	R/W	<u>Enable Tx Endpoint 3 interrupt</u> 0: disable 1: enable
2	EP2	1	R/W	<u>Enable Tx Endpoint 2 interrupt</u> 0: disable 1: enable
1	EP1	1	R/W	<u>Enable Tx Endpoint 1 interrupt</u> 0: disable 1: enable
0	EP0	1	R/W	<u>Enable Tx / Rx Endpoint 0 interrupt</u> 0: disable 1: enable

18.15.7 INTRRX1E

Receive EP Interrupt Enable Register #1

INTRRX1E	8000:0009			offset:	0x09		default:	0x1E
Bit	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	EP4	EP3	EP2	EP1	RSV
R/W	-	-	-	R/W	R/W	R/W	R/W	-
Default	-	-	-	1	1	1	1	-

Bit	Name	Reset Value	R/W	Function
75	RSV			<u>Reserved</u>
4	EP4	1	R/W	<u>Enable Rx Endpoint 4 interrupt</u> 0: disable 1: enable
3	EP3	1	R/W	<u>Enable Rx Endpoint 3 interrupt</u> 0: disable 1: enable
2	EP2	1	R/W	<u>Enable Rx Endpoint 2 interrupt</u> 0: disable 1: enable
1	EP1	1	R/W	<u>Enable Rx Endpoint 1 interrupt</u> 0: disable 1: enable
0	RSV			Reserved

18.15.8 INTRUSBE

USB Interrupt Status Enable Register

INTRUSBE		8000:000B			offset: 0x0B		default: 0x06	
Bit	7	6	5	4	3	2	1	0
Name	VBUSERR	SESSREQ	DISCON	CONN	SOF	RESET	RESUME	SUSPEND
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	1	1	0

Bit	Name	Reset Value	R/W	Function
7	VBUSERR	0	R/W	<u>Vbus Error Interrupt</u> 0: disable 1: enable
6	SESSREQ	0	R/W	<u>Session Request Signal Detection Interrupt</u> 0: disable 1: enable
5	DISCON	0	R/W	<u>Disconnect Interrupt</u> 0: disable 1: enable
4	CONN	0	R/W	<u>Connection Interrupt</u> 0: disable 1: enable
3	SOF	0	R/W	<u>SOF Interrupt</u> 0: disable 1: enable
2	RESET	1	R/W	<u>Reset Interrupt</u> 0: disable 1: enable
1	RESUME	1	R/W	<u>Resume Interrupt</u> 0: disable 1: enable
0	SUSPEND	0	R/W	<u>Suspend Interrupt</u> 0: disable 1: enable

18.15.9 FRAME1**Lower Frame Number Register**

FRAME1	8000:000C		offset: 0x0C		default: 0x00			
Bit	7	6	5	4	3	2	1	0
Name	FRNL[7]	FRNL[6]	FRNL[5]	FRNL[4]	FRNL[3]	FRNL[2]	FRNL[1]	FRNL[0]
R/W	R	R	R	R	R	R	R	R
Default	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
7:0	FRNL	0	R	<u>Frame Number</u> In Peripheral Mode, lower 8 bits of last received frame number In Host Mode, lower 8 bits of current frame number

18.15.10 FRAME2**Upper Frame Number Register**

FRAME2	8000:000D		offset: 0x0D		default: 0x00			
Bit	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	FRNH[2]	FRNH[1]	FRNH[0]
R/W	-	-	-	-	-	R	R	R
Default	-	-	-	-	-	0	0	0

Bit	Name	Reset Value	R/W	Function
7:3	RSV			<u>Reserved</u>
2:0	FRNH	0	R	<u>Frame Number</u> In Peripheral Mode, upper 3 bits of last received frame number In Host Mode, upper 3 bits of current frame number

18.15.11 INDEX**Endpoint index Register**

INDEX	8000:000E				offset:	0x0E		default:	0x00
Bit	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	EPIDX[3]	EPIDX[2]	EPIDX[1]	EPIDX[0]	
R/W	-	-	-	-	R/W	R/W	R/W	R/W	
Default	-	-	-	-	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
7:4	RSV			<u>Reserved</u>
				<u>Endpoint Index</u> The value in the field determines which endpoint control/status registers are accessed at offset USB register addresses 0x10 to 0x1F
3:0	EPDX	0	R/W	0 : Endpoint 0 1 : Endpoint 1 2 : Endpoint 2 3 : Endpoint 3 4 : Endpoint 4

18.15.12 DEVCTL

Device Control Register

DEVCTL	8000:000F			offset: 0x0F		default: 0x88		
Bit	7	6	5	4	3	2	1	0
Name	CID	FSDEV	LSDEV	PUCON	PDCON	MODE	HOSTREQ	SESSREQ
R/W	R	R	R	R	R	R	R/W	R/W
Default	1	0	0	0	1	0	0	0

Bit	Name	Reset Value	R/W	Function
7	CD	1	R	<u>CID Signal</u> Indicates ID pin of mini-A/B connector 0: A-type 1: B-type
6	FSDEV	0	R	<u>Full Speed Device Detection</u> 0: No Full Speed device detected 1: Full Speed device detected <i>Valid Only in Host Mode</i>
5	LSDEV	0	R	<u>Low Speed Device Detection</u> 0: No Low Speed device detected 1: Low Speed device detected <i>Valid Only in Host Mode</i>
4	PUCON	0	R	<u>Pull Up Connection</u> 0: No Pull up resistor required 1: Pull Up resistor required on USB D+ line to operate in peripheral mode
3	PDCON	1	R	<u>Pull Down Connection</u> 0: No Pull Down resistor required 1: Pull Down resistor required on USB D+ line to operate in host mode
2	MODE	0	R	<u>Mode of Operation</u> 0: Peripheral Mode 1: Host Mode
1	HOSTREQ	0	R/W	<u>Host Request</u> 0: Complete Host Negotiation 1: Initiate Host Negotiation
0	SESSREQ	0	R/W	<u>Session Initiation</u> 0: In 'A' device operation, End a session 1: In 'A' device operation, Start a session In 'B' device operation, initiate Session Request Protocol

18.15.13 TXMAXP**Transmit Maximum Packet Size Register**

TXMAXP		8000:0010			offset: 0x10		default: 0x00	
Bit	7	6	5	4	3	2	1	0
Name	MAXP[7]	MAXP[6]	MAXP[5]	MAXP[4]	MAXP[3]	MAXP[2]	MAXP[1]	MAXP[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
7:0	MAXP	0	R/W	<u>Maximum Packet Size</u> Maximum Packet Size for currently selected TX endpoint in bytes(See INDEX register)

18.15.14 PER_CSR0**Peripheral EP0 Control Register**

PER_CSR0	8000:0011		offset: 0x11		default: 0x00			
Bit	7	6	5	4	3	2	1	0
Name	CLRSETEND	CLRRXRDY	SENDST	SETEND	DATAEND	SENTST	TXPKTRDY	RXPKTRDY
R/W	W	W	W	R	W	R/W	R/W	R
Default	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
7	CLRSETEND	0	W	<u>Service Setup End</u> Wr 0: No effect Wr 1: Clear the SETEND (Setup END) bit Automatically cleared
6	CLRRXRDY	0	W	<u>Service Receive Packet Ready</u> Wr 0: No effect Wr 1: Clear RXPKTRDY (Receive Packet Ready) bit Automatically cleared
5	SENDST	0	W	<u>Send Stall (control)</u> Wr 0: No effect Wr 1: Transmit STALL handshake Automatically cleared
4	SETEND	0	R	<u>Setup End</u> 0: No Setup End detected 1: Control Transaction ends before DATAEND bit is set Cleared by setting CLRSETEND bit in CSR0
3	DATAEND	0	W	<u>Data End</u> Wr 0: No effect Wr 1: Last data packet in the transaction transmitted/received Automatically cleared
2	SENTST	0	R/W	<u>Sent Stall (Status)</u> Rd 0: STALL handshake NOT sent Rd 1: STALL handshake sent Wr 0: Clear SENTST bit Wr 1: No effect
1	TXPKTRDY	0	R/W	<u>Transmit Packet Ready</u> Rd 0: Data packet transmitted Rd 1: Data packet pending to be transmitted Wr 0: Not allowed Wr 1: Transmit Data packet from FIFO Automatically cleared when data packet is transmitted
0	RXPKTRDY	0	R	<u>Receive Packet Ready</u> Rd 0: No data packet received in the FIFO Rd 1: Data packet received in the FIFO Cleared by setting the bit CLRRXRDY in CSR0

18.15.15 PER_TXCSR1

Peripheral Tx EP Control Register #1

PER_TXCSR1	8000:0011	offset:		0x11	default:		0x00	
Bit	7	6	5	4	3	2	1	0
Name	RSV	CLRDATTOG	SENTST	SENDST	FLFIFO	UNDERRUN	FIFOEMP	TXPKTRDY
R/W	-	W	R/W	R/W	W	R/W	R/W	R/W
Default	-	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
7	RSV			Reserved
6	CLRDATTOG	0	W	<u>Clear Data Toggle</u> Wr 0: No effect Wr 1: Reset Data Toggle to 0
5	SENTST	0	R/W	<u>Sent Stall (Status)</u> Rd 0: STALL handshake NOT sent Rd 1: STALL handshake sent Wr 0: Clear SENTST bit Wr 1: No effect
4	SENDST	0	R/W	<u>Send Stall (Control)</u> Wr 0: Terminate STALL handshake Wr 1: Transmit STALL handshake <i>Automatically cleared, No effect in ISO mode</i>
3	FLFIFO	0	W	<u>Flush FIFO</u> Wr 0: No effect Wr 1: Flush Endpoint FIFO, Reset FIFO pointer, TXPKTRDY bit cleared <i>Note: Need to flush FIFO twice if FIFO is dual buffered, Auto Cleared, No effect if TXPKTRDY not set</i>
2	UNDERRUN	0	R/W	<u>FIFO UnderRun</u> Rd 0: No UnderRun Rd 1: IN token received when TXPKTRDY bit not set (No data in FIFO) Wr 0: Clear UNDERRUN bit Wr 1: No effect
1	FIFOEMP	0	R/W	<u>FIFO Not Empty</u> Rd 0: FIFO Empty Rd 1: FIFO Not Empty Wr : Check This
0	TXPKTRDY	0	R/W	<u>Transmit Packet Ready</u> Rd 0: Data packet transmitted Rd 1: Data packet pending to be transmitted Wr 0: Not allowed Wr 1: Transmit Data packet from FIFO Automatically cleared when data packet is transmitted

18.15.16 CSR02**Peripheral EP0 Control Register #2**

CSR02	8000:0012			offset:	0x12		default:	0x00
Bit	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	FLFIFO
R/W	-	-	-	-	-	-	-	R/W
Default	-	-	-	-	-	-	-	0

Bit	Name	Reset Value	R/W	Function
7:1	RSV			<u>Reserved</u>
0	FLFIFO	0	R/W	<u>Flush FIFO</u> Wr 0: No effect Wr 1: Flush Endpoint0 FIFO, Reset FIFO pointer, TXPKTRDY/RXPKTRDY bit of PER_CSR0 or HST_CSR0 cleared <i>Note: Need to flush FIFO twice if FIFO is dual buffered, Auto Cleared, No effect if TXPKTRDY/RXPKTRDY not set</i>

18.15.17 TXCSR2

Tx EP Control Register #2

TXCSR2	8000:0012			offset:	0x12		default:	0x00
Bit	7	6	5	4	3	2	1	0
Name	AUTOSET	ISO	MODE	DMAEN	FRDATTOG	DMAMODE	RSV	RSV
R/W	R/W	R/W	R/W	R/W	R/w	R/W	-	-
Default	0	0	0	0	0	0	-	-

Bit	Name	Reset Value	R/W	Function
7	NAKTO	0	R/W	<u>NAK Time-out</u> Wr 0: Clear NAK on EP0 Rd 1: EP0 halted, following the receipt of NAK for longer than the time set in NAKLIMIT0 register
6	STPKT	0	R/W	<u>Status Packet</u> » 0: No Status stage transaction « » 1: Status Stage Transaction « <i>Set this bit at same time as REQPKT or TXPKTRDY</i>
5	REQPKT	0	R/W	<u>Request Packet</u> Rd 0: No IN transaction requested Rd 1: IN Transaction requested Wr 0: No effect Wr 1: IN transaction requested <i>Cleared when RXPKTRDY bit is set</i>
4	ERR	0	R	<u>Error</u> » 0: No error detected « » 1: Error, No response from peripheral in three attempts to perform a transaction «
3	SETPKT	0	R/W	<u>Setup Packet</u> » 0: No SETUP token to be sent « » 1: SETUP token to be sent instead of OUT token when transmitting data (TXPKTRDY is set) «
2	RXSTALL	0	R/W	<u>Receive Stall</u> Rd 0: No STALL received Rd 1: STALL received from peripheral Wr 0: Clear RXSTALL Wr 1: No effect
1	TXPKTRDY	0	R/W	<u>Transmit Packet Ready</u> Rd 0: Data packet transmitted Rd 1: Data packet pending to be transmitted Wr 0: Not allowed Wr 1: Transmit Data packet from FIFO <i>Automatically cleared when data packet is transmitted</i>
0	RXPKTRDY	0	R/W	<u>Receive Packet Ready</u> Rd 0: No data packet received in the FIFO Rd 1: Data packet received in the FIFO Wr 0: Clear RXPKTRDY bit Wr 1: No effect

18.15.18 RXMAXP**Receive Maximum Packet Size Register**

RXMAXP	8000:0013		offset: 0x13		default: 0x00			
Bit	7	6	5	4	3	2	1	0
Name	MAXP[7]	MAXP[6]	MAXP[5]	MAXP[4]	MAXP[3]	MAXP[2]	MAXP[1]	MAXP[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
7:0	MAXP	0	R/W	Maximum Packet Size Maximum Packet Size for currently selected RX endpoint in bytes (See INDEX register)

18.15.19 PER_RXCSR1**Peripheral Rx EP Control Register #1**

PER_RXCSR1	8000:0014			offset:	0x14		default:	0x00	
Bit	7	6	5	4	3	2	1	0	
Name	CLRDATTOG	SENTST	SENDST	FLFIFO	DATERR	OVERRUN	FIFOFUL	RXPKTRDY	
R/W	W	R/W	R/W	w	R	R/W	R	R/W	
Default	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
7	CLRDATTOG	0	W	<u>Clear Data Toggle</u> Wr 0: No effect Wr 1: Reset Data Toggle to 0
6	SENTST	0	R/W	<u>Sent Stall (Status)</u> Rd 0: STALL handshake NOT sent Rd 1: STALL handshake sent Wr 0: Clear SENTST bit Wr 1: No effect
5	SENDST	0	R/W	<u>Send Stall (Control)</u> Wr 0: Terminate STALL handshake Wr 1: Transmit STALL handshake <i>No effect in ISO mode</i>
4	FLFIFO	0	w	<u>Flush FIFO</u> Wr 0: No effect Wr 1: Flush Endpoint FIFO Note: Need to flush FIFO twice if FIFO is dual buffered
3	DATERR	0	R	<u>Data Error</u> Rd 0: No Error Rd 1: CRC error or bit stuff error occurred <i>Cleared when RXPKTRDY is cleared. Effective only in ISO mode, Always zero in Bulk mode</i>
2	OVERRUN	0	R/W	<u>FIFO Over Run</u> Rd 0: No Over run Rd 1: OUT packet cannot be loaded on EP FIFO Wr 0: Clear OVERRUN bit Wr 1: No effect <i>Effective only in ISO mode, Always zero in Bulk Mode</i>
1	FIFOFUL	0	R	<u>FIFO Full</u> Rd 0: Rx FIFO is not full Rd 1: Rx FIFO full
0	RXPKTRDY	0	R/W	<u>Receive Packet Ready</u> Rd 0: No data packet received in the FIFO Rd 1: Data packet received in the FIFO Wr 0: Data packet unloaded from Rx FIFO, Clear RXPKTRDY bit Wr 1: No effect

18.15.20 PER_RXCSR2**Peripheral Rx EP Control Register #2**

PER_RXCSR2 8000:0015 offset: 0x15 default: 0x00

Bit	7	6	5	4	3	2	1	0
Name	AUTOCLR	ISO	DMAEN	DMAMODE	RSV	RSV	RSV	RSV
R/W	R/W	R/W	R/W	R/W	-	-	-	-
Default	0	0	0	0	-	-	-	-

Bit	Name	Reset Value	R/W	Function
7	AUTOCLR	0	R/W	<u>Auto Clear RXPKTRDY</u> 0: Auto Clear not enabled. Manually clear RXPKTRDY bit after unloading data packet from Rx FIFO 1: Auto Clear enabled. RXPKTRDY bit cleared automatically once data packet of RXMAXP bytes is unloaded from FIFO. Manually clear RXPKTRDY if data packet is less than RXMAXP bytes in size
6	ISO	0	R/W	<u>Isochronous Transfer</u> 0: No Isochronous transfer on this EP 1: Isochronous transfer on this EP
5	DMAEN	0	R/W	<u>DMA Enable</u> 0: DMA not enabled for Rx EP 1: DMA enabled for Rx EP
4	DMAMODE	0	R/W	<u>DMA Mode for Rx transfer</u> 0: DMA Mode 0 1: DMA Mode 1 Note: Refer to USB DMA controller register for more details
3:0	RSV			<u>Reserved</u>

18.15.21 COUNT0**Count EP0 Data Bytes Register**

COUNT0 8000:0016 offset: 0x16 default: 0x00

Bit	7	6	5	4	3	2	1	0
Name	RSV	CNT[6]	CNT[5]	CNT[4]	CNT[3]	CNT[2]	CNT[1]	CNT[0]
R/W	-	R	R	R	R	R	R	R
Default	-	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
7	RSV			<u>Reserved</u>
6:0	CNT	0	R	<u>Count Received Bytes</u> Number of received bytes in EP0 FIFO

18.15.22 RXCOUNT1**Count EP Data bytes Register #1**

RXCOUNT1		8000:0016			offset: 0x16		default: 0x00	
Bit	7	6	5	4	3	2	1	0
Name	CNTL[7]	CNTL[6]	CNTL[5]	CNTL[4]	CNTL[3]	CNTL[2]	CNTL[1]	CNTL[0]
R/W	R	R	R	R	R	R	R	R
Default	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
7:0	CNTL	0	R	<u>Count Received Bytes Low</u> Lower 8 bits of received data count in bytes in EP FIFO

18.15.23 RXCOUNT2**Count EP Data bytes Register #2**

RXCOUNT2		8000:0017			offset: 0x17		default: 0x00	
Bit	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	CNTH[2]	CNTH[1]	CNTH[0]
R/W	-	-	-	-	-	R	R	R
Default	-	-	-	-	-	0	0	0

Bit	Name	Reset Value	R/W	Function
7:3	RSV			<u>Reserved</u>
2:0	CNTH	0	R	<u>Count Received Bytes High</u> Upper 3 bits of received data count in bytes in EP FIFO

18.15.24 TXTYPE

EP Transmit Type Register

TXTYPE		8000:0018		offset: 0x18		default: 0x00		
Bit	7	6	5	4	3	2	1	0
Name	RSV	RSV	PROTO[1]	PROTO[0]	EP[3]	EP[2]	EP[1]	EP[0]
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
76	RSV			<u>Reserved</u>
				<u>Transfer Protocol</u>
				00: Illegal transfer
54	PROTO	0	R/W	01: Isochronous Transfer
				10: Bulk Transfer
				11: Interrupt Transfer
				Note: Valid only in Host mode
				<u>Endpoint Number</u>
30	EP	0	R/W	Endpoint number contained in the OUT endpoint descriptor returned by the peripheral during device enumeration
				Note: Valid only in Host mode

18.15.25 NAKLMT0

EP0 NAK Limit Register

NAKLMT0		8000:0019			offset: 0x19		default: 0x00	
Bit	7	6	5	4	3	2	1	0
Name	INTRVL[7]	INTRVL[6]	INTRVL[5]	INTRVL[4]	INTRVL[3]	INTRVL[2]	INTRVL[1]	INTRVL[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
70	INTRVL	0	R/W	<u>Tx Polling Interval</u>
				Polling interval for EP0 in number of frames.
				Note: Valid only in Host mode
				Valid values can be 2-255
				EP0 is halted NAK responses for more than INTRVL frames are received from the peripheral

18.15.26 TXINTVL

EP Transmit Interval Register

TXINTVL	8000:0019			offset:	0x19		default:	0x00
Bit	7	6	5	4	3	2	1	0
Name	INTRVL[7]	INTRVL[6]	INTRVL[5]	INTRVL[4]	INTRVL[3]	INTRVL[2]	INTRVL[1]	INTRVL[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
7:0	INTRVL	0	R/W	<u>Tx Polling Interval</u> Polling interval for the currently selected Tx EP in number of frames. Note: Valid only in Host mode Note: Interrupt valid range is 1-255 Isochronous valid range is 1-255 Bulk valid range is 2-255

18.15.27 RXTYPE

EP Receive Type Register

RXTYPE	8000:001A			offset:	0x1A		default:	0x00
Bit	7	6	5	4	3	2	1	0
Name	RSV	RSV	PROTO[1]	PROTO[0]	EP[3]	EP[2]	EP[1]	EP[0]
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
7:6	RSV			<u>Reserved</u>
5:4	PROTO	0	R/W	<u>Transfer Protocol</u> 00: Illegal transfer 01: Isochronous Transfer 10: Bulk Transfer 11: Interrupt Transfer Note: Valid only in Host mode
3:0	EP	0	R/W	<u>Endpoint Number</u> Endpoint number contained in the IN endpoint descriptor returned by the peripheral during device enumeration Note: Valid only in Host mode

18.15.28 RXINTVL**EP Receive Interval Register**

RXINTVL	8000:001B			offset:	0x1B	default:	0x00	
Bit	7	6	5	4	3	2	1	0
Name	INTRVL[7]	INTRVL[6]	INTRVL[5]	INTRVL[4]	INTRVL[3]	INTRVL[2]	INTRVL[1]	INTRVL[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
7:0	INTRVL	0	R/W	<u>Rx Polling Interval</u> Polling interval for the currently selected Rx EP in number of frames. Note: Valid only in Host mode Note: Interrupt valid range is 1-255 Isochronous valid range is 1-255 Bulk valid range is 2-255

18.15.29 TXFIFO1**EP Transmit FIFO Address Register #1**

TXFIFO 1		8000:001C			offset: 0x1C		default: 0x00	
Bit	7	6	5	4	3	2	1	0
Name	ADDRL[7]	ADDRL[6]	ADDRL[5]	ADDRL[4]	ADDRL[3]	ADDRL[2]	ADDRL[1]	ADDRL[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
7:0	ADDRL	0	R/W	<u>FIFO Address</u> Lower byte of start address for endpoint FIFO

18.15.30 TXFIFO2

EP Transmit FIFO Address Register #2

TXFIFO2	8000:001D			offset: 0x1D		default: 0x00		
Bit	7	6	5	4	3	2	1	0
Name	SZ[2]	SZ[1]	SZ[0]	DPB	ADDRH[3]	ADDRH[2]	ADDRH[1]	ADDRH[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
7:5	SZ	0	R/W	<u>Packet Size</u> Maximum Packet Size for endpoint SZ[2:0] -> Packet Size 000 -> 8 bytes 001 -> 16 bytes 111 -> 1024 bytes If DPB = 0, FIFO size is equal to SZ If DPB = 1, FIFO size is twice of SZ Note: FIFO Size = $2^{(SZ+3)}$
4	DPB	0	R/W	<u>Dual Packet Buffering</u> 0 - Single Packet buffering in FIFO 1 - Dual Packet buffering in FIFO
3:0	ADDRH	0	R/W	<u>FIFO Address</u> High bits for start Address of FIFO ADDR[11:8] = ADDRH ADDR[7:0] = ADDRL AD[11:0](hex) -> Start Address(hex) 000 -> 0000 001 -> 0008 002 -> 0010 FFF -> 7FF8 Note: Start Address of FIFO is multiples of 8 bytes

18.15.31 RXFIFO1**EP Receive FIFO Address Register #1**

RXFIFO1	8000:001E			offset: 0x1E		default: 0x00		
Bit	7	6	5	4	3	2	1	0
Name	ADDRL[7]	ADDRL[6]	ADDRL[5]	ADDRL[4]	ADDRL[3]	ADDRL[2]	ADDRL[1]	ADDRL[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Bit	Name	Reset Value	R/W	Function
7:0	ADDRL	0	R/W	<u>FIFO Address</u> Lower byte of start address for endpoint FIFO

18.15.32 RXFIFO2

EP Receive FIFO Address Register #2

RXFIFO2	8000:001F			offset: 0x1F		default: 0x00		
Bit	7	6	5	4	3	2	1	0
Name	SZ[2]	SZ[1]	SZ[0]	DPB	ADDRH[3]	ADDRH[2]	ADDRH[1]	ADDRH[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
7:5	SZ	0	R/W	<u>Packet Size</u> Maximum Packet Size for endpoint SZ[2:0] -> Packet Size 000 -> 8 bytes 001 -> 16 bytes 111 -> 1024 bytes If DPB = 0, FIFO size is equal to SZ If DPB = 1, FIFO size is twice of SZ Note: FIFO Size = $2^{(SZ+3)}$
4	DPB	0	R/W	<u>Dual Packet Buffering</u> 0 - Single Packet buffering in FIFO 1 - Dual Packet buffering in FIFO
3:0	ADDRH	0	R/W	<u>FIFO Address</u> High bits for start Address of FIFO ADDR[11:8] = ADDRH ADDR[7:0] = ADDRL AD[11:0](hex) -> Start Address(hex) 000 -> 0000 001 -> 0008 002 -> 0010 FFF -> 7FF8 Note: Start Address of FIFO is multiples of 8 bytes

18.15.33 HST_CSR0

Host EP0 Control Register

HST_CSR0	8000:0011			offset: 0x11		default: 0x00		
Bit	7	6	5	4	3	2	1	0
Name	NAKTO	STPKT	REQPKT	ERR	SETPKT	RXSTALL	TXPKTRDY	RXPKTRDY
R/W	R/W	R/W	R/W	R	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
7	NAKTO	0	R/W	<u>NAK Time-out</u> Wr 0: Clear NAK on EP0 Rd 1: EP0 halted, following the receipt of NAK for longer than the time set in NAKLIMIT0 register
6	STPKT	0	R/W	<u>Status Packet</u> 0: No Status stage transaction 1: Status Stage Transaction <i>Set this bit at same time as REQPKT or TXPKTRDY</i>
5	REQPKT	0	R/W	<u>Request Packet</u> Rd 0: No IN transaction requested Rd 1: IN Transaction requested Wr 0: No effect Wr 1: IN transaction requested <i>Cleared when RXPKTRDY bit is set</i>
4	ERR	0	R	<u>Error</u> 0: No error detected 1: Error, No response from peripheral in three attempts to perform a transaction
3	SETPKT	0	R/W	<u>Setup Packet</u> 0: No SETUP token to be sent 1: SETUP token to be sent instead of OUT token when transmitting data (TXPKTRDY is set)
2	RXSTALL	0	R/W	<u>Receive Stall</u> Rd 0: No STALL received Rd 1: STALL received from peripheral Wr 0: Clear RXSTALL Wr 1: No effect
1	TXPKTRDY	0	R/W	<u>Transmit Packet Ready</u> Rd 0: Data packet transmitted Rd 1: Data packet pending to be transmitted Wr 0: Not allowed Wr 1: Transmit Data packet from FIFO <i>Automatically cleared when data packet is transmitted</i>
0	RXPKTRDY	0	R/W	<u>Receive Packet Ready</u> Rd 0: No data packet received in the FIFO Rd 1: Data packet received in the FIFO Wr 0: Clear RXPKTRDY bit Wr 1: No effect

18.15.34 HST_TXCSR1

Host Tx EP Control Register #1

HST_TXCSR1		8000:0011		offset: 0x11		default: 0x00		
Bit	7	6	5	4	3	2	1	0
Name	NAKTO	CLRDATTOG	RXSTALL	RSV	FLFIFO	ERR	FIFOEMP	TXPKTRDY
R/W	R/W	W	R/W	-	W	R/W	R/W	R/W
Default	0	0	0	-	0	0	0	0

Bit	Name	Reset Value	R/W	Function
7	NAKTO	0	R/W	<u>NAK Time-out</u> Wr 0: Clear NAK on TX EP Rd 1: TX EP halted, following the receipt of NAK for longer than the time set in NAKLIMIT in TXINTVL register Note: Valid only for Bulk EP
6	CLRDATTOG	0	W	<u>Clear Data Toggle</u> Wr 0: No effect Wr 1: Reset Data Toggle to 0
5	RXSTALL	0	R/W	<u>Receive Stall</u> Rd 0: No STALL received Rd 1: STALL received from peripheral Wr 0: Clear RXSTALL Wr 1: No effect
4	RSV			<u>Reserved</u>
3	FLFIFO	0	W	<u>Flush FIFO</u> Wr 0: No effect Wr 1: Flush Endpoint FIFO, Reset FIFO pointer, TXPKTRDY bit cleared Note: Need to flush FIFO twice if FIFO is dual buffered, Auto Cleared, No effect if TXPKTRDY not set
2	ERR	0	R/W	<u>Error</u> Rd 0: No error detected Rd 1: No response from peripheral in three attempts to perform a transaction Wr 0: Clear ERR Wr 1: No effect Valid only in Bulk or Interrupt mode
1	FIFOEMP	0	R/W	<u>FIFO Not Empty</u> Rd 0: FIFO Empty Rd 1: FIFO Not Empty Wr 0: Clear FIFOEMP Wr 1: No effect
0	TXPKTRDY	0	R/W	<u>Transmit Packet Ready</u> Rd 0: Data packet transmitted Rd 1: Data packet pending to be transmitted Wr 0: Not allowed Wr 1: Transmit Data packet from FIFO Automatically cleared when data packet is transmitted

18.15.35 HST_RXCSR1

Host Rx EP Control Register #1

HST_RXCSR1		8000:0014			offset:	0x14		default:	0x00
Bit	7	6	5	4	3	2	1	0	
Name	CLRDATTOG	RXSTALL	REQPKT	FLFIFO	DATERR	ERR	FIFOFUL	RXPBKTRDY	
R/W	W	R/W	R/W	W	R	R/W	R	R/W	
Default	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
7	CLRDATTOG	0	W	<u>Clear Data Toggle</u> Wr 0: No effect Wr 1: Reset Data Toggle to 0
6	RXSTALL	0	R/W	<u>Receive Stall</u> Rd 0: No STALL received Rd 1: STALL received from peripheral Wr 0: Clear RXSTALL Wr 1: No effect
5	REQPKT	0	R/W	<u>Request Packet</u> Rd 0: No IN transaction requested Rd 1: IN Transaction requested Wr 0: No effect Wr 1: IN transaction requested <i>Cleared when RXPBKTRDY bit is set</i>
4	FLFIFO	0	W	<u>Flush FIFO</u> Wr 0: No effect Wr 1: Flush Endpoint FIFO Note: Need to flush FIFO twice if FIFO is dual buffered
3	DATERR	0	R	<u>Data Error / NAK time out</u> Rd 0: No Error Rd 1: CRC error / bit stuff error (valid in Isochronous) Rd 1: Time out error following the receipt of NAK responses longer than set in RXINTVL register (valid in Bulk) <i>Cleared when RXPBKTRDY is cleared.</i>
2	ERR	0	R/W	<u>Error</u> Rd 0: No error detected Rd 1: No response from peripheral in three attempts to perform a transaction Wr 0: Clear ERR Wr 1: No effect <i>Valid only in Bulk or Interrupt mode</i>
1	FIFOFUL	0	R	<u>FIFO Full</u> Rd 0: Rx FIFO is not full Rd 1: Rx FIFO full
0	RXPBKTRDY	0	R/W	<u>Receive Packet Ready</u> Rd 0: No data packet received in the FIFO Rd 1: Data packet received in the FIFO Wr 0: No effect Wr 1: Data packet unloaded from Rx FIFO, Clear RXPBKTRDY bit

18.15.36 HST_RXCSR2**Host Rx EP Control Register #2**

HST_RXCSR2		8000:0015		offset: 0x15		default: 0x00		
Bit	7	6	5	4	3	2	1	0
Name	AUTOCLR	AUTOREQ	DMAEN	DMAMODE	RSV	RSV	RSV	RSV
R/W	R/W	R/W	R/W	R/W	-	-	-	-
Default	0	0	0	0	-	-	-	-

Bit	Name	Reset Value	R/W	Function
7	AUTOCLR	0	R/W	<u>Auto Clear RXPTRDY</u> 0: Auto Clear not enabled. Manually clear RXPTRDY bit after unloading data apcket from Rx FIFO 1: Auto Clear enabled. RXPTRDY bit cleared automatically once data packet of RXMAXP bytes is unloaded from FIFO. Manually clear RXPTRDY if data packet is less than RXMAXP bytes in size
6	AUTOREQ	0	R/W	<u>Auto Request Mode</u> 0: No Auto setting of REQPKT bit 1: REQPKT bit is automatically set when RXPTRDY bit is cleared, to send next IN token
5	DMAEN	0	R/W	<u>DMA Enable</u> 0: DMA not enabled for Rx EP 1: DMA enabled for Rx EP
4	DMAMODE	0	R/W	<u>DMA Mode for Rx transfer</u> 0: DMA Mode 0 1: DMA Mode 1 <i>(Refer to Module Specifications for DMA modes in Rx EP)</i>
3:0	RSV			<u>Reserved</u>

18.15.37 **FIFO0**

EP0 FIFO Access Register

[illegible][illegible]

18.15.39 **FIFO2**

EP2 FIFO Access Register

[illegible][illegible]

18.15.40 FIFO3

EP3 FIFO Access Register

[illegible][illegible]

18.15.41 FIFO4

EP4 FIFO Access Register

[illegible][illegible]

18.16 USB DMA Controller Registers

Address	Register	Description
8000:0200	INTR	Interrupt Status Register
8000:0204	CNTL1	DMA Channel1 Control Register
8000:0208	ADDR1	DMA Channel1 Address Register
8000:020C	COUNT1	DMA Channel1 Byte Count Register
8000:0214	CNTL2	DMA Channel2 Control Register
8000:0218	ADDR2	DMA Channel2 Address Register
8000:021C	COUNT2	DMA Channel2 Byte Count Register
8000:0224	CNTL3	DMA Channel3 Control Register
8000:0228	ADDR3	DMA Channel3 Address Register
8000:022C	COUNT3	DMA Channel3 Byte Count Register
8000:0234	CNTL4	DMA Channel4 Control Register
8000:0238	ADDR4	DMA Channel4 Address Register
8000:023C	COUNT4	DMA Channel4 Byte Count Register

18.16.1 INTR

Interrupt Status Register

[illegible][illegible]

18.16.2 CNTL1

DMA Channel1 Control Register

CNTL1	8000:0204	offset: 0x04	default: 0x00000000
-------	-----------	--------------	---------------------

[illegible][illegible][illegible]

18.16.3 ADDR1

DMA Channel1 Address Register

ADDR1	8000:0208	offset: 0x08	default: 0x00000000
-------	-----------	--------------	---------------------

[illegible][illegible][illegible]

18.16.4 COUNT1

DMA Channel1 Byte Count Register

COUNT1	8000:020C	offset: 0x0C	default: 0x00000000
--------	-----------	--------------	---------------------

[illegible][illegible][illegible]

18.16.5 CNTL2

DMA Channel2 Control Register

CNTL2	8000:0214	offset: 0x14	default: 0x00000000
-------	-----------	--------------	---------------------

[illegible][illegible][illegible]

18.16.6 ADDR2

DMA Channel2 Address Register

ADDR2	8000:0218	offset: 0x18	default: 0x00000000
-------	-----------	--------------	---------------------

[illegible][illegible][illegible]

18.16.7 COUNT2

DMA Channel2 Byte Count Register

[illegible][illegible][illegible]

18.16.8 CNTL3

DMA Channel3 Control Register

CNTL3	8000:0224	offset: 0x24	default: 0x00000000
-------	-----------	--------------	---------------------

[illegible][illegible][illegible]

18.16.9 ADDR3

DMA Channel3 Address Register

ADDR3	8000:0228	offset: 0x28	default: 0x00000000
-------	-----------	--------------	---------------------

[illegible][illegible][illegible]

18.16.12 ADDR4

DMA Channel4 Address Register

ADDR4	8000:0238	offset: 0x38	default: 0x00000000
-------	-----------	--------------	---------------------

[illegible][illegible][illegible]

19 I²C

19.1 Introduction

The DM320 includes an Inter Integrated Circuit (I²C™) serial interface that can act as a single master. I²C-bus system is implemented for 100kbps and 400kbps transfer rate. The interface uses 7bit/10bit addressing. The DM320 initiates, terminates transfers and generates clock signals for I²C protocol functioning.

NOTE: I²C is a registered trademark of Philips.

19.2 Features

DM320 includes a 2-line serial interface and can perform I²C bus protocol-compliant synchronous serial communication as bus master. This module has following features.

- Supports only single bus master.
- Supports transmit, receive and mixed transfer using the slave address of 7bits or 10bits.
- Issues the start, stop and restart conditions.
- Supports transfer rate of 100kbps or 400kbps.
- Generates interrupt for each transfer. Status register is updated after interrupt generation.
- Operation of clock line from slave is inhibited.
- No support for CBUS.
- During data transfer, write access to I²C registers is not possible

19.3 I²C Bus Configuration

19.3.1 Enabling DM320 I²C Bus

I²C has the SCL pin that transfers the clock and SDA pin on which data is transmitted or received. In DM320, SCL and SDA pins are multiplexed with GIO31 and GIO32 respectively as shown in Table 56. GIO31 and GIO32 are configured for I²C using the FSEL0 register in GIO module. Refer to chapter on General Purpose Input Output (GIO) for more details.

Bus Pin	GIO port
SCL	GIO31
SDA	GIO32

Table 56: Multiplexed GIO ports for I²C bus

19.3.2 Input Clock

Clock signal input to M48XI pin of DM320 is used as clock generation source for I²C data transfer clock. Transfer rate is not as it appears in the specifications when a non-48MHz clock is input to this M48XI pin.

Clock output from PLLA or PLLB can also be used as source for I²C data transfer clock. The source clock can be selected using SEL0 register in the clock controller (CLKC) module. I²C clock is enabled by configuring MOD2 register in CLKC module. Refer to chapter on CLKC for more details.

19.3.3 Connecting Slave Devices

Figure 132 shows an example I²C bus connection. DM320 is the master and other connecting devices are slaves. Each device should be an open-drain connection. Hence each bus line should be connected with appropriate pull-up resistors. In addition, please observe the following restrictions concerning bus configuration.

Restrictions on Bus Configuration

- I²C bus of DM320 does not support the multi bus master. DM320 itself is a unique master device on bus and another device as bus master cannot exist.
- I²C bus I/O buffer of DM320 has low drive ability. Hence, when multiple slave devices are connected, it may not meet the connection requirements for each device.

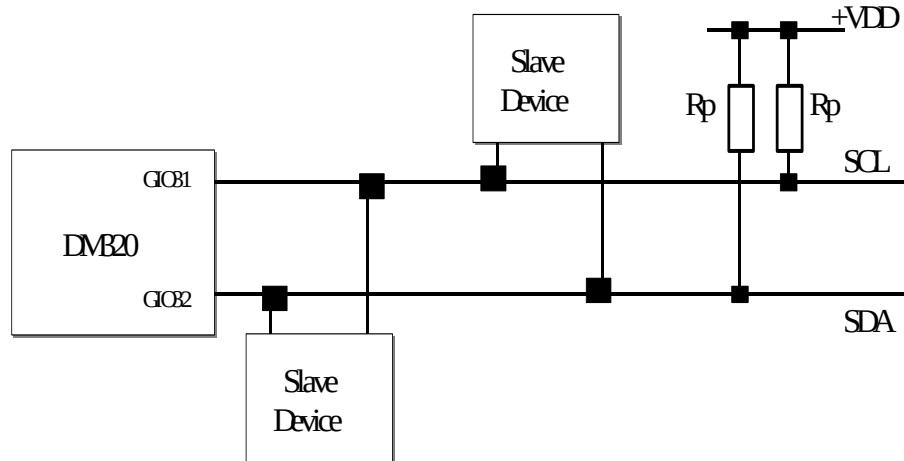


Figure 132: Example of I²C bus connection

19.4 Data Transfer Sequence

I²C module of DM320 can perform data transmit to/ receive from slave device. It also supports mixed data transfer. 7bits or 10bits address can be specified as slave address. CBUS address is not supported.

Writing to/ reading from the register in I²C module executes all procedures for data transfer. Description of actual data transfer sequence is discussed in following sections.

19.4.1 Setting data transfer rate

Data transfer rate is set prior to actual data transfer. The user can select 100kbps or 400kbps as the data transfer rate. It is 100kbps by default. Configuring CLKSEL bit in the SCS register selects the required data rate.

Note: Transfer rate is not as it appears in the specifications when a non-48MHz clock is input to I²C module.

19.5 Issuing of Start, Stop and Restart conditions

DM320 can issue “Start” condition to start data transfer. After completing the data transfer, “Stop” condition can be issued. Furthermore, “Restart” condition can also be issued.

Setting COND [1:0] bits in the SCS register to value ‘01’ issues “Start” condition. “Start” condition will be added automatically during actual data transfer start.

“Stop” condition is issued by setting COND [1:0] bits in SCS register to value ‘10’. Triggering data transfer with “Stop” condition bits set, issues actual “Stop” condition on bus. In this case only issue of “Stop” condition is performed but data transfer is not done.

“Restart” condition can be issued by starting transfer with “Start” condition and then again starting transfer with “Start” condition prior to issuing “Stop” condition.

Setting COND [1:0] bits in SCS register to value ‘00’ ensures normal transfer (See Table 57).

	Start/Restart conditions	Stop condition	Transfer without condition
Setting of COND bits [1:0]	Set ‘01’ before data transfer starts	Set ‘10’ to trigger stop condition after last byte is transferred	Set ‘00’

Table 57: Issuing Start, Stop and Restart conditions

19.6 Data Register for Transfer

There are two data transfer registers; TXDATA and RXDATA.

To transmit data to slave device, the data to be sent is written to TXD [7:0] bits of TXDATA register. After sending, the data sent is shown in RXD [7:0] bits of RXDATA register. Check the contents of RXDATA register to determine if the data was sent properly on the bus or not.

To receive data from slave device, 0xFF should be written in TXD [7:0] bits of TXDATA register. After completing the reception, the received data can be read from RXD [7:0] bits of RXDATA register. (See Table 58)

	Function of TXD [7:0]	Function of RXD [7:0]
During data transmit	Write transmit data before sending	Shows the sent data on bus after transmission is completed
During data receive	Set 0xFF before receiving	Shows the received data after receiving is completed

Table 58: Data register for transfer

19.7 Acknowledgement bit

In I²C bus, an acknowledgement bit follows transfer of every data byte.

During data transmit to slave device, the slave device can acknowledge every data byte it receives by setting the acknowledgement bit. In order

to receive acknowledgement bit properly, ACK bit in the TXDATA register should be set to “1” before sending data from DM320. After transmission is completed, acknowledgement bit from slave device can be read from ACKS bit in the RXDATA register.

During data receive from slave device, master device or DM320 must set the ACK bit in TXDATA register. Setting ACK bit in the TXDATA register is required before receiving data. In addition, after receiving is completed, Acknowledgement bit actually sent on bus can be read from ACKS bit in the RXDATA. (See Table 59)

	Function of ACK bit in TXDATA	Function of ACKS bit in RXDATA
During data transmit	Set to '1'	Indicates Acknowledgement bit from slave device after transmission is completed
During data receive	Set to '1'	Indicates Acknowledgement bit sent by the master on the bus after receiving is completed

Table 59: Acknowledgement bit

19.8 Triggering data transfer

I²C module of DM320 performs data transfer in one byte unit. Consequently, ARM must trigger data transfer after every byte is transferred.

The data transfer is triggered by XMIT bit in the SCS register. Writing '1' to this bit starts data transfer of one byte. The settings of data transfer, Start/ Stop conditions and Acknowledgement bit should be done before triggering the data transfer.

XMIT bit shows '1' during transfer. When XMIT is '1', writing '1' does not start new transfer. It is recommended to verify that this bit is set at '0' before writing '1' to start new data transfer. (See Table 60)

Note: Once a transfer starts, it cannot be suspended.

Status of XMIT bit in SCS register	Description
0	Stand-by state. Writing '1' starts data transfer
1	Transfer is in progress. Should not write to this bit when it is set to '1'.

Table 60: Triggering data transfer

19.9 I²C Interrupt

I²C module has one port to interrupt controller in DM320. An interrupt is generated in any of the following cases.

- When Acknowledgement bit is transmitted/ received
- When STOP condition is generated

Refer to the chapter on “Interrupt controller” for more details.

19.10 Transmit (7bit address)

The sequence to be followed for transmission of n-bytes of data using the 7-bit slave address is shown in Figure 133. Specific register settings required is also described.

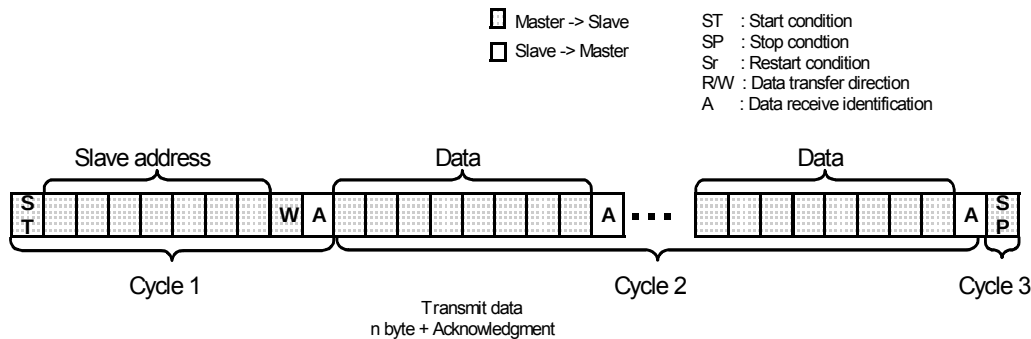


Figure 133 Transmit n-bytes using 7 bit slave address

1. Write the slave address to TXD [7:1], '0' to TXD [0] and '1' to ACK in TXDATA register
2. Write "01" (start condition) to COND [1:0] in SCS register
3. Write '1' to XMIT bit in the SCS register. Cycle 1 starts
4. Cycle 1 is terminated. An interrupt is generated
5. Write the data to TXD [7:0] and '1' to ACK in the TXDATA register
6. Write "00" (normal transfer) to COND [1:0] in the SCS register
7. Write '1' to XMIT in the SCS register. Cycle 2 starts
8. Cycle 2 is terminated. An interrupt is generated. Repeat steps (5) – (8) n times.
9. Write "10" (stop condition) to COND [1:0] in the SCS register
10. Write '1' to XMIT in the SCS register. Cycle 3 starts
11. Cycle 3 is terminated. An interrupt is generated. Transmit is completed.

19.11 Receive (7bit address)

The sequence to be followed for receiving n-bytes of data using the 7-bit slave address is shown in Figure 134. Specific register settings required is also described.

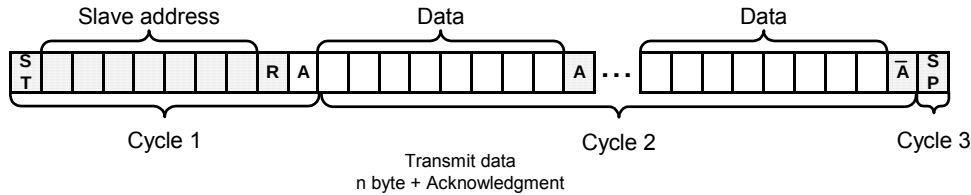


Figure 134: Receive n-bytes of data using 7 bit slave address

1. Write the slave address to TXD [7:1], '1' to TXD [0] and '1' to ACK in the TXDATA register
2. Write "01" (start condition) to COND [1:0] in the SCS register
3. Write '1' to XMIT in the SCS register. Cycle 1 starts
4. Cycle 1 is terminated. An interrupt is generated.
5. Write '1' to all TXD [7:0] and '0' to ACK in the TXDATA register (Write '1' to ACK in the nth time).
6. Write "00" (normal transfer) to COND [1:0] in the SCS register
7. Write '1' to XMIT in the SCS register. Cycle 2 starts.
8. Cycle 2 is terminated. An interrupt is generated. Repeat steps (5)-(8) n times.
9. Write "10" (stop condition) to COND [1:0] in the SCS register
10. Write "1" to XMIT in the SCS register. Cycle 3 starts.
11. Cycle 3 is terminated. An interrupt is generated. Receive is completed.

19.12 Mixed transfer (7bit address)

The sequence to be followed for mixed transfer using the 7-bit slave address is shown in Figure 135. Here, DM320 I²C master can communicate to two different slaves denoted by slave address 1 and slave address 2. Specific register settings required is also described.

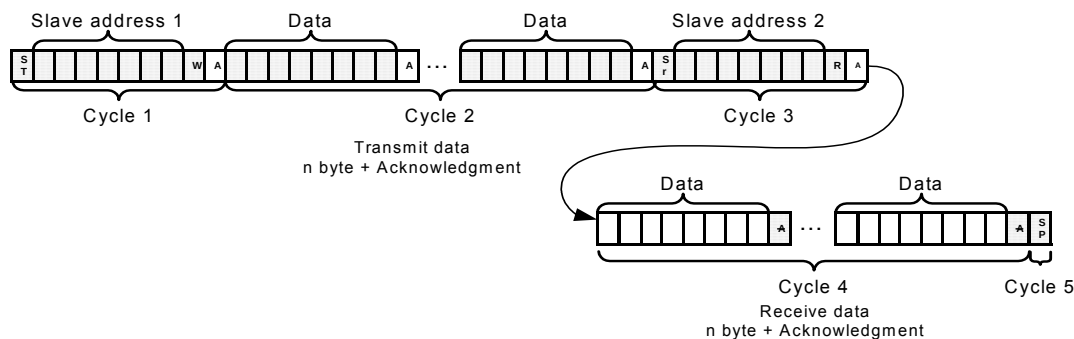


Figure 135 Mixed Transfer using 7 bit slave address

1. Write the 'slave address 1' to TXD [7:1], '0' to TXD [0] and '1' to ACK in the TXDATA register
2. Write "01" (start condition) to COND [1:0] in the SCS register
3. Write '1' to XMIT in the SCS register. Cycle 1 starts
4. Cycle 1 is terminated. An interrupt is generated.
5. Write the data to TXD [7:0] and '1' to ACK in the SCS register.
6. Write "00" (normal transfer) to COND [1:0] in the SCS register.
7. Write '1' to XMIT in the SCS register. Cycle 2 starts.
8. Cycle 2 is terminated. An interrupt is generated. Repeat steps (5) – (8) n times.
9. Write the 'slave address 2' to TXD [7:1], '1' to TXD [0] and '1' to ACK in the TXDATA.
10. Write "01" (start condition) to COND [1:0] in the SCS register.
11. Write '1' to XMIT in the SCS register. Cycle 3 starts.
12. Cycle 3 is terminated. An interrupt is generated.
13. Write '1' to all TXD [7:0] and '0' to ACK in the TXDATA. (Write '1' to ACK in the nth time).
14. Write "00" (normal transfer) to COND [1:0] in the SCS register.
15. Write '1' to XMIT in the SCS register. Cycle 4 starts.
16. Cycle 4 is terminated. An interrupt is generated. Repeat steps (13) – (16) n times.
17. Write "10" (stop condition) to COND [1:0] in the SCS register.
18. Write '1' to XMIT in the SCS register. Cycle 5 starts.
19. Cycle 5 is terminated. An interrupt is generated. Transfer is completed.

19.13 Transmit (10bit address)

The sequence to be followed for transmitting n-bytes of data using the 10-bit slave address is shown in Figure 136. Specific register settings required is also described.

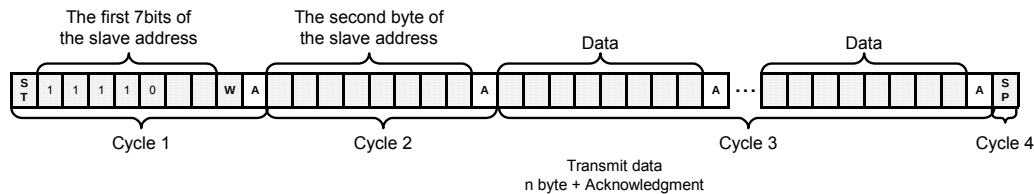


Figure 136: Transmit n-bytes of data using 10-bit slave

address

1. Write the first 7 bits of the slave address to TXD [7:1], '0' to TXD [0] and '1' to ACK in the TXDATA register.
2. Write "01" (start condition) to COND [1:0] in the SCS register.
3. Write '1' to XMIT in the SCS register. Cycle 1 starts
4. Cycle 1 is terminated. An interrupt is generated.
5. Write the second byte of the slave address to TXD [7:0] and '1' to ACK in the TXDATA register.
6. Write "00" (normal transfer) to COND [1:0] in the SCS register.
7. Write '1' to XMIT in the SCS register. Cycle 2 starts.
8. Cycle 2 is terminated. An interrupt is generated.
9. Write the data to TXD [7:0] and '1' to ACK in the TXDATA register.
10. Write "00" (normal transfer) to COND [1:0] in the SCS register.
11. Write '1' to XMIT in SCS . Cycle 3 starts.
12. Cycle 3 is terminated. An interrupt is generated. Repeat steps (9) – (12) n times.
13. Write "10" (stop condition) to COND [1:0] in the SCS register.
14. Write '1' to XMIT in the SCS. Cycle 4 starts.
15. Cycle 4 is terminated. An interrupt is generated. Transmit is completed.

19.14 Receive (10bit address)

The sequence to be followed for receiving n-bytes of data using the 10-bit slave address is shown in Figure 137. Specific register settings required is also described.

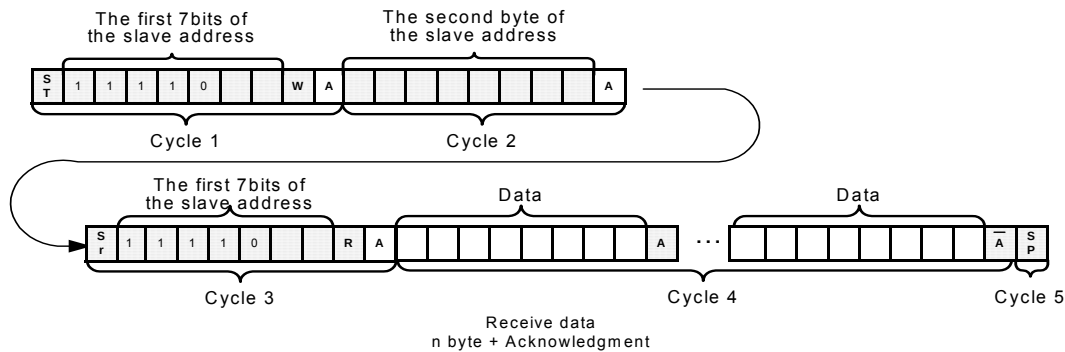


Figure 137: Receive n-bytes of data using 10-bit slave address

1. Write the first 7bits of the slave address to TXD [7:1], '0' to TXD[0] and '1' to ACK in the TXDATA register
2. Write "01" (start condition) to COND [1:0] in the SCS register
3. Write '1' to XMIT in the TXDATA register. Cycle 1 starts
4. Cycle 1 is terminated. An interrupt is generated.
5. Write the second byte of the slave address to TXD [7:0] and '1' to ACK in the TXDATA register
6. Write "00" (normal transfer) to COND [1:0] in the SCS register
7. Write '1' to XMIT in the SCS register. Cycle 2 starts.
8. Cycle 2 is terminated. An interrupt is generated.
9. Write the first 7bits of the slave address to TXD [7:1], '1' to TXD [0] and '1' to ACK in the TXDATA register.
10. Write "01" (start condition) to COND [1:0] in the SCS register.
11. Write '1' to XMIT in SCS register. Cycle 3 starts.
12. Cycle 3 is terminated. An interrupt is generated.
13. Write '1' to all TXD [7:0] and '0' to ACK in the TXDATA (Write '1' to ACK in the nth times).
14. Write "00" (normal transfer) to COND[1:0] in the SCS register.
15. Write '1' to XMIT in the SCS register. Cycle 4 starts.
16. Cycle 4 is terminated. An interrupt is generated. Repeat steps (13) –(16) n times.

17. Write "10" (stop condition) to COND[1:0] in the SCS .
18. Write '1' to XMIT in the SCS. Cycle 5 starts.
19. Cycle 5 is terminated. An interrupt is generated. Receive is completed.

19.15 Mixed transfer (10bit address)

The sequence to be followed for mixed transfer using the 10-bit slave address is shown in Figure 138. Specific register settings required is also described.

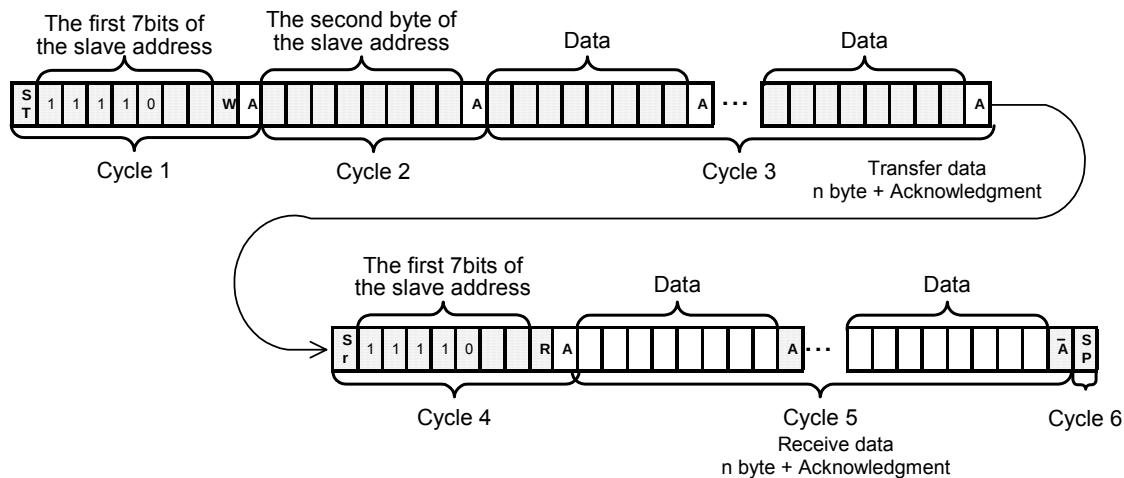


Figure 138: Mixed transfer using 10-bit slave address

1. Write the first 7bits of the slave address to TXD [7:1], '0' to TXD [0] and '1' to ACK in the TXDATA register
2. Write "01" (start condition) to COND [1:0] in the SCS register
3. Write '1' to XMIT in the SCS register. Cycle 1 starts
4. Cycle 1 is terminated. An interrupt is generated.
5. Write the second byte of the slave address to TXD [7:0] and '1' to ACK in the TXDATA register
6. Write "00" (normal transfer) to COND [1:0] in the SCS register.
7. Write '1' to XMIT in SCS . Cycle 2 starts.
8. Cycle 2 is terminated. An interrupt is generated.
9. Write the data to TXD [7:0] and '1' to ACK in the TXDATA register.
10. Write "00" (normal transfer) to COND [1:0] in the SCS register.

11. Write '1' to XMIT in the. Cycle 3 starts.
12. Cycle 3 is terminated. An interrupt is generated. Repeat steps (9) – (12) n times.
13. Write the first 7bits of the slave address to TXD [7:1], '1' to TXD[0] and '1' to ACK in the TXDATA register.
14. Write "01" (start condition) to COND [1:0] in the.
15. Write '1' to XMIT in the SCS . Cycle 4 starts.
16. Cycle 4 is terminated. An interrupt is generated.
17. Write '1' to all TXD [7:0] and "0" to ACK in the TXDATA (Write '1' to ACK in the nth times).
18. Write "00" (normal transfer) to COND [1:0] in the SCS register.
19. Write '1' to XMIT in SCS . Cycle 5 starts.
20. Cycle 5 is terminated. An interrupt is generated. Repeat 17) – 20) n times.
21. Write "10" (stop condition) to COND [1:0] in the SCS register.
22. Write '1' to XMIT in SCS . Cycle 6 starts.
23. Cycle 6 is terminated. An interrupt is generated. Transfer is completed.

19.16 I2C Register Map (I2C)

Offset	Address	Register	Description
0x00	00030D80	TXDATA	Transmit Data Register
0x01	00030D82	RXDATA	Receive Data Register
0x02	00030D84	SCS	Serial Control

19.17 I2C Registers

19.17.1 TXDATA

Transmit data register

TXDATA	0003:0D80			offset: 0x00												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	ACK	TXD[7]	TXD[6]	TXD[5]	TXD[4]	TXD[3]	TXD[2]	TXD[1]	TXD[0]	
R/W	-	-	-	-	-	-	-	W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	-	-	-	-	-	-	-	0	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:9	RSV			Reserved
8	ACK	0	W	Acknowledge control (Setting of acknowledge level) » 0: Receiver sends Acknowledge « » 1: Receiver does not Acknowledge (when sending data) «
7:0	TXD	0	R/W	Transmission address/data Note: Write "0xFF" at the time of reception

19.17.2 RXDATA

Receive data register

RXDATA	0003:0D82			offset: 0x02												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	ACKS	RXD[7]	RXD[6]	RXD[5]	RXD[4]	RXD[3]	RXD[2]	RXD[1]	RXD[0]	
R/W	-	-	-	-	-	-	-	R	R	R	R	R	R	R	R	R	
Default	-	-	-	-	-	-	-	0	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:9	RSV			Reserved
8	ACKS	0	R	<u>Acknowledge Status</u> After transfer completed (occurrence of interrupt), acknowledge value can be read.
7:0	RXD	0	R	<u>Receive Data</u> After transfer completed (occurrence of interrupt), received data can be read. Note: At the time of transmitting, after transfer completed, values written in TXDATA can be read.

19.17.3 SCS

Serial port control register

SCS	0003:0D84			offset:	0x04								default: 0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	CLKSEL	XMIT	COND[1]	COND[0]
R/W	-	-	-	-	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	-	-	-	-	-	-	-	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:4	RSV			Reserved
3	CLKSEL	0	R/W	Selection of transfer clock » 0: 100KHz « » 1: 400KHz «
2	XMIT	0	R/W	READ: Status on transfer can be read. 0: Transfer on standby 1: On transmitting WRITE: Trigger for transfer 0: No transfer 1: Transfer starts.
1:0	COND	0	R/W	Transfer Condition » 00: Normal transfer « » 01: Transfer with start condition « » 10: Transfer with stop condition « » 11: Reserved «

20 Memory Traffic Controller (MTC)

20.1 Introduction

Memory traffic controller consists of three blocks - External Memory Interface (EMIF), SDRAM controller (SDRC), and DMA controller (DMAC). MTC enables DM320 to have access to large memory spaces. SDRC controls accesses to SDRAM from all other DM320 modules. EMIF has dedicated FLASH / SRAM interface, two general purpose memory interface and two specialized interfaces to control and access Compact Flash and Smart Media cards. DMAC allows fast and efficient data transfers between different external memory regions, SDRAM and other AHB peripherals. All the three blocks of MTC are described in detail in later sections.

When processing data, it is often necessary to access large memory spaces. The internal memory of the DM320 chip is often too small to fit all the data and program. EMIF has the capability to access large amount of external memory. Since the external memory interface is configurable, many different types of memories can be connected to the external interface. In addition SDRC can support up to 256 MB of SDRAM.

DMAC consists of different DMA channels available with the MTC. These DMA channels allow fast and efficient memory transfers between the different external memory regions and the ARM.

The MTC can access memory separated into 6 regions:

- CS0, external memory region 0 (FLASH / SRAM memory interface)
- CS1, external memory region 1 (CompactFlash interface)
- CS2, external memory region 2 (SmartMedia interface)
- CS3, external memory region 3 (general purpose device)
- CS4, external memory region 4 (general purpose device)
- SDRAM memory region

Many different types of general-purpose memory can be attached to each region of the external memory interface, such as ROM, EPROM, and SRAM. There are five chip select pins for CS0 through CS4 that can be directly connected with these general-purpose memories without any external decoding circuit, and their timing can be controlled by software. For example, for a low-speed device in which output data is not immediately set to high impedance, a bus IDLE state cycle can be input between that read cycle and the next memory access cycle. Thus, data collision on the data bus can be avoided.

The interface timing characteristics of each external memory region are configurable. Furthermore, the data busses of CS0, CS1, CS2, CS3, and CS4 can be set to 8-bits or 16-bits.

An SDRAM memory region is set up within the MTC region in the same way as the other external memory areas.

External memory regions 0, 1, and 2, have special features that allow them to easily be used as a FLASH memory interface, CompactFlash interface and SmartMedia interface respectively. The CompactFlash and SmartMedia interfaces share some pins, therefore it is not possible to simultaneously access a CompactFlash and Smart Media card.

20.1.1 Features

1. Supports up to 2 Giga bit (8k Rows x 2k Column x 4 Banks) of SDRAM
2. Supports up to 16 MB flash
3. Supports 8/16bit Nand Flash
4. Supports different DMA burst (1/4/8) transfers
5. Supports 3 DMA channels to SDRAM

20.1.2 MTC Memory Map

The memory map of the DM320 MTC is illustrated in **Table 61**.

Region name	Start address	End address	Size (bytes)
External Memory (Flash + SDRAM)	0x0010: 0000	0x110F: FFFF	272M
Compact Flash	0x4000: 0000	0x40FF: FFFF	16M
SSFDC (Smart media)	0x4800: 0000	0x48FF: FFFF	16M
General purpose memory region #1	0x5000: 0000	0x50FF: FFFF	16M
General purpose memory region #2	0x6000: 0000	0x60FF: FFFF	16M
Mirror External Memory	0xFFFF: 0000	0xFFFF: FFFF	64K

Table 61: MTC memory map

20.1.3 Block Diagram

The SDRAM controller is the main communication channel between the different modules of the DM320. The CCD Controller, Preview Engine, OSD, Image Buffer, DMAC, EMIF, and ARM are connected directly to the SDRAM controller. The Image Buffer, HPIB and DMAC can be connected either to the SDRC or EMIF. Depending on the memory location accessed, the ARM will either access the SDRAM controller, or the EMIF.

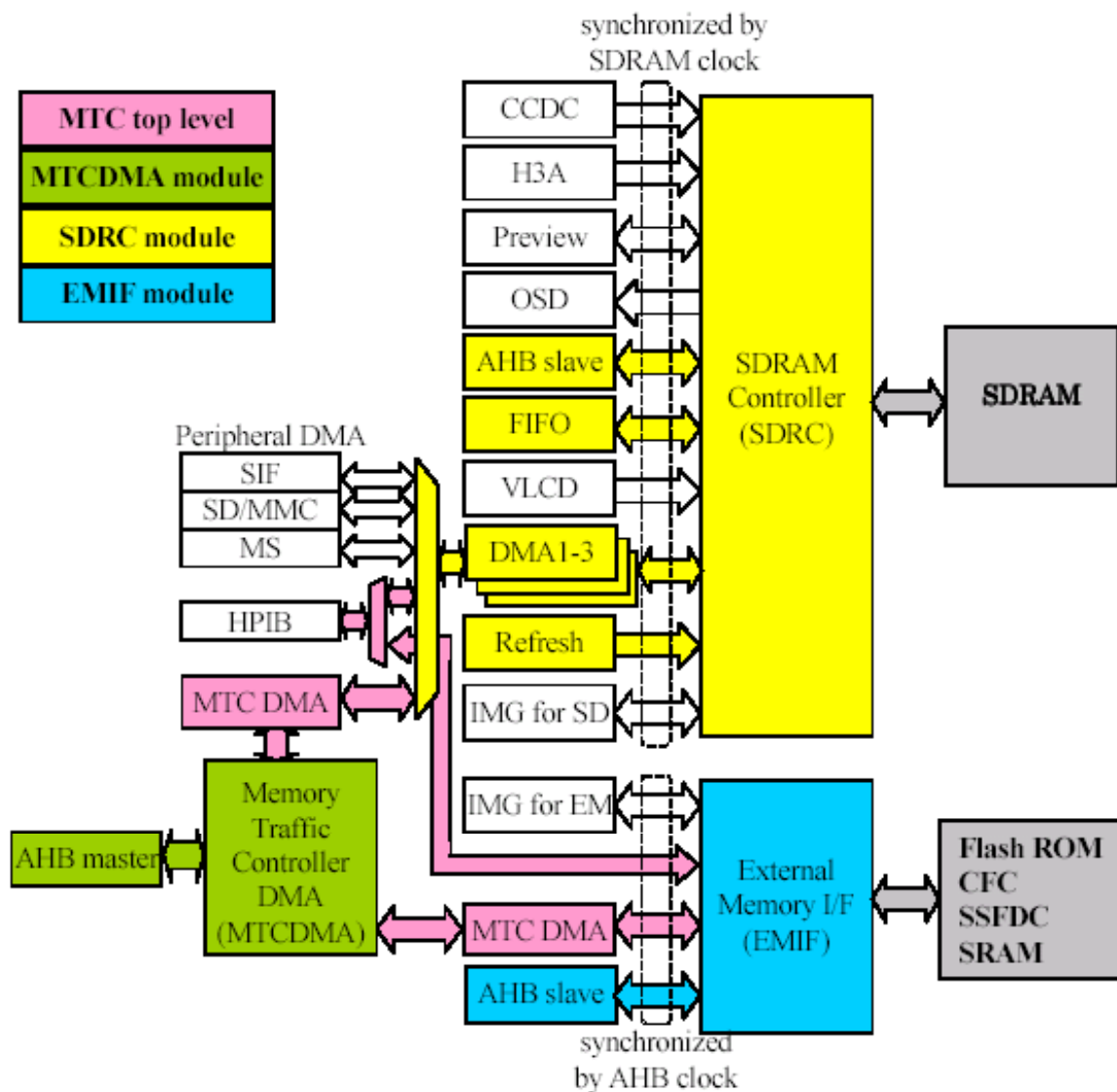


Figure 139: Memory Traffic Controller block diagram

20.1.4 End Addresses and Sizes of the External Memory Regions

The start and end address of each EMIF regions are shown **Table 62**. The end addresses are specified in units of 1 Mbytes. SDRAM start and end addresses are configurable, refer AHB Bus controller module for more details.

MTC region name	Start address	End address	Chip Select
External memory region 0	0010:0000h	0080:0000h	FLS_CE
SDRAM region	0090:0000h	1000:0000h	SDR_CS
External memory region 1	4000:0000h	0100:0000h	CF_CE1, CF_CE2
External memory region 2	4800:0000h	0100:0000h	CF_CE1
External memory region 3	5000:0000h	0100:0000h	EM_CE0
External memory region 4	6000:0000h	0100:0000h	EM_CE1

Table 62: Memory Addresses

20.1.5 Hardware interface

As external memory interfaces, DM320 has a general-purpose memory interface. The ARM can access SDRAM via the SDRAM controller module. The ARM can access general-purpose SRAM or flash ROM, a Compact Flash card, or a Smart Media card via the external memory interface module. In access from ARM to external memory, a wait is automatically inserted, and its cycle varies depending on SDRAM frequency and external memory access time. The functions of several pins in the external memory bus and SDRAM bus are shown in **Table 63** and **Table 64** respectively.

Pin Name	I/O	Explanation
ARM_A [22:0]	O	ADDRESS bus Byte address for 8 bit device and half word address for 16-bit device is outputted. ARM_A [22]: -REG for CF/CF+
ARM_D [15:0]	I/O	DATA bus
EM_WE	O	Write enable code (FLSH_CE, CFE1, CFE2, EM_CS0, EM_CS1)
EM_RDWR	O	READ/WRITE Strobe code
EM_BEH	O	Upper byte enable code
EM_BEL	O	Lower byte enable code
EM_OE	O	Read enable code (FLSH_SE, EM_CS0, EM_CS1)
EM_WAIT	I	External wait code (EM_CS0 or EM_CS1)
CF_CE1	O	CF/CF+ [-CFE1] Smart Media [-CE]
CF_CE2	O	CF/CF+ [-CFE2] Smart Media [ALE]
CF_OE	O	CF/CF+ [-OE] Smart Media [CLE]
CF_IORD	O	CF/CF+ [-IORD] Smart Media [-RE]
CF_IOWR	O	CF/CF+ [-IOWR] Smart Media [-WE]
CF_RDY	I	CF/CF+ [READY/-BUSY] Smart Media [READY/-BUSY]
CF_WAIT	I	CF/CF+ [-WAIT]
CF_IOIS16	I	CF/CF+ [-IOIS16]

Table 63: External Memory Bus Pins

Pin Name	I/O	Explanation
SDR_DQ [31-16]	I/O	SDRAM data bus (32bit mode)
SDR_DQ [15-0]	I/O	SDRAM data bus (16/32bit mode)
SDR_A [12-0]	O	SDRAM address bus
SDR_A [14-13]	O	SDRAM bank address
SDR_RAS	O	SDRAM Row address strobe
SDR_CAS	O	SDRAM Column address strobe
SDR_WE	O	SDRAM Write enable
SDR_CS0	O	SDRAM Support chip select
SDR_CKE	O	SDRAM clock enable
SDR_DQMHH	O	SDRAM mask for DQ [31:24]
SDR_DQMLH	O	SDRAM mask for DQ [23:16]
SDR_DQMLH	O	SDRAM mask for DQ [15:8]
SDR_DQMLL	O	SDRAM mask for DQ [7:0]
SDR_CLK	O	SDRAM clock

Table 64: SDRAM Bus Pins

20.1.6 Timing control

The timing for each external memory region can be configured by various register settings. This allows many different types of devices, which have different interface timing requirements, to be connected to each of the external memory regions. Also, an idle state can be inserted in order to prevent bus collision when performing continuous memory accesses. **Table 65**, **Table 66**, and **Table 67** show the configurable timing characteristics for each external memory region, and **Figure 140** shows the corresponding signal timing diagrams.

Parameter	Description	Clock variables	Variable unit	Initial value after RESET
CYCLE	Number of cycles	2-16 clocks	1 CLK (AHB_CLK)	11
CS_SU	Number of Setup of CS	0-15 clocks	1 CLK	0
WE_SU	Number of Setup of WE	"	1 CLK	1
OE_SU	Number of Setup of OE	"	1 CLK	1
CS_WIDTH	Width of CS	1-16 clocks	1 CLK	10
WE_WIDTH	Width of WE	"	1 CLK	9
OE_WIDTH	Width of OE	"	1 CLK	9
IDLE	Number of Idles	0-3 clocks	1 CLK	1

Table 65 Configurable Timing Parameters for EM_CS0/EM_CS3/EM_CS4

Parameter	Description	Clock variables	Variable unit	Initial value after RESET
CYCLE	Number of cycles	2-32 clocks	1 CLK (ARM_CLK)	22
CS_SU	Number of Setup of CS	0-15 clocks	1 CLK	0
WE_SU	Number of Setup of WE	"	1 CLK	2
OE_SU	Number of Setup of OE	"	1 CLK	2
CS_WIDTH	Width of CS	1-32 clocks	1 CLK	21
WE_WIDTH	Width of WE	"	1 CLK	10
OE_WIDTH	Width of OE	"	1 CLK	18
IDLE	Number of Idles	0-3 clocks	1 CLK	1

Table 66 Configurable Timing Parameters for EM_CE1

Parameter	Description	Clock variables	Variable unit	Initial value after RESET
CYCLE	Number of cycles	2-16 clocks	1 CLK (ARM_CLK)	8
WE_SU	Number of Setup of WE	0-15 clocks	1 CLK	2
OE_SU	Number of Setup of OE	"	1 CLK	1
CS_WIDTH	Width of CS	1-16 clocks	1 CLK	3
WE_WIDTH	Width of WE	"	1 CLK	4
READ_SU	Number of Setup of READ	3-18 clocks	1 CLK	12
IDLE	Number of Idles	0-3 clocks	1 CLK	1

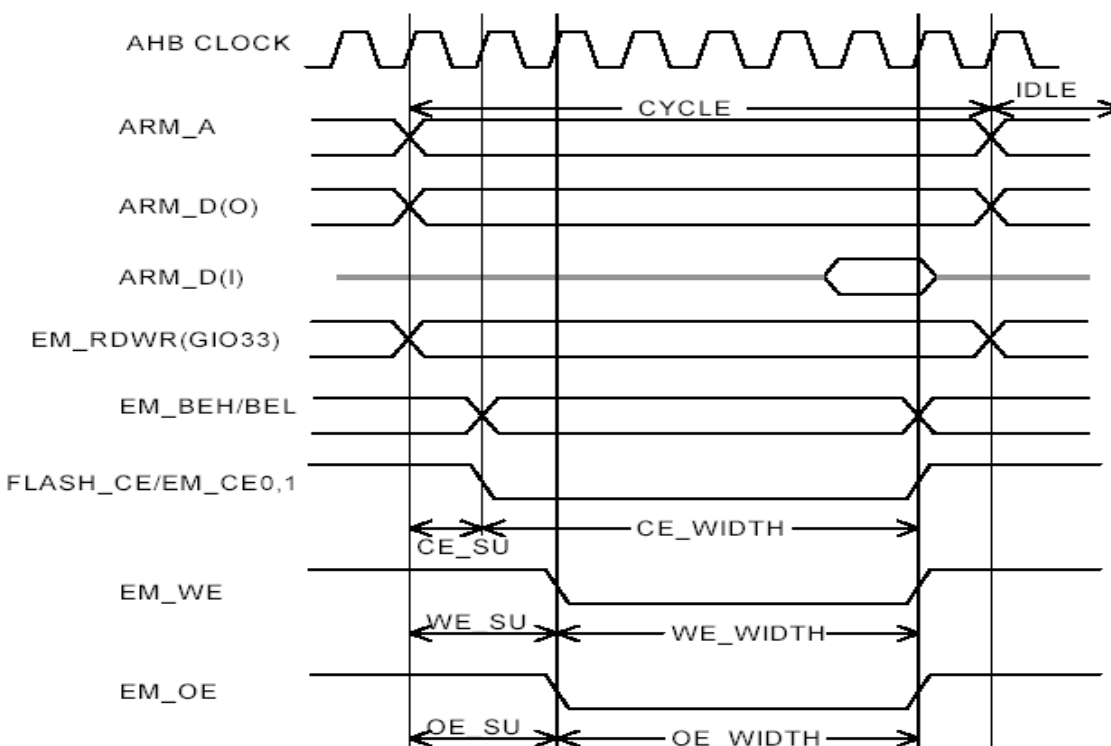
Table 67 Configurable Timing Parameters for EM_CE2

Figure 140 External Memory Bus Timings for CS0/CS3/CS4

Table 68 shows the registers, which controls the timing characteristics for different regions.

EMIF region	Registers	Description
CS0	CS0CTRL1	To set Output enable width, Write enable width, Chip enable width and cycle width for region 0.
	CS0CTRL2	To set Number of clocks of output enable setup, write enable setup and chip enable setup for region 0
	CS0CTRL3	To set Access mode and page setup width for region 0
CS1	CS1CTRL1A	To set Chip enable width and cycle width for region 1.
	CS1CTRL1B	To set Output enable width, Write enable width, for region 1.
	CS1CTRL2	To set Number of clocks of output enable setup, write enable setup and chip enable setup for region 1
CS2	CS2CTRL1	To set Output enable width, Write enable width, and cycle width for region 2.
	CS2CTRL2	To set Number of clocks of output enable setup, write enable setup, chip enable setup and the Cycle width also for region 2.
CS3	CS3CTRL1	To set Output enable width, write enable width, chip enable width and cycle width for region 3.
	CS3CTRL2	To set Number of clocks of output enable setup, write enable setup, chip enable setup and External bus width for region 3
CS4	CS4CTRL1	To set Output enable width, write enable width, Chip enable width and cycle width for region 4.
	CS4CTRL2	To set Number of clocks of output enable setup, write enable setup, chip enable setup and External bus width for region 4.

Table 68: EMIF regions and timing control registers

20.1.7 External Memory Region 0: The FLASH Memory Interface

Systems that use asynchronous FLASH ROM for program memory should attach the FLASH ROM to external memory region 0 (CS0). In External memory boot-mode, ARM resets the program counter to 0xFFFF:0000 address, which is a mirror of the external memory starting from 0x0010:0000.

Figure 141 shows the FLASH memory interface for 16 bit / 8 bit flash. Refer to the DM320 Datasheet for more information on the FLASH interface signals. When the ARM accesses the flash region, FLSH_CE becomes active. In addition to FLASH memory, it is also possible to connect SRAM or PROM devices to the CS0 interface.

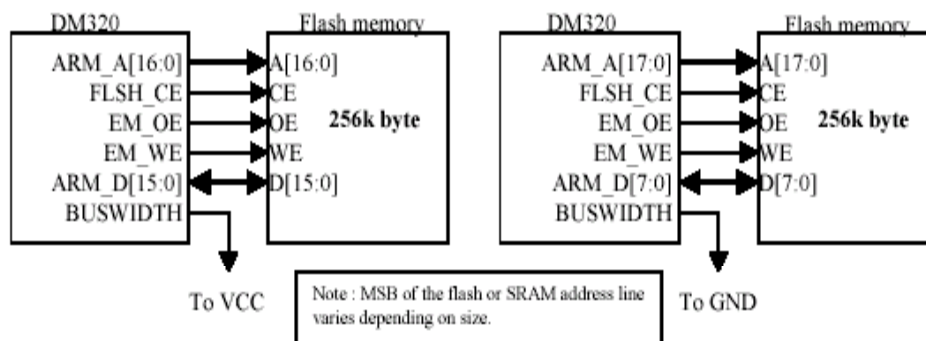


Figure 141: Flash Memory Interface

20.1.7.1 Flash Memory Interface Data Bus

External memory region 0 supports 8-bit and 16-bit wide FLASH. The BUSWIDTH pin is used to set the data bus width of the region 0 to 8 or 16 bits. When BUSWIDTH is low, 8-bit access is selected and when BUSWIDTH is high, 16-bit access is selected.

Although the FLASH data bus width can be set to 8 or 16-bits, a system may need to access 8, 16, and/or 32-bit data from the FLASH memory. For example, the LDR command will require a 32-bit read access. As shown in **Table 69** and **Figure 142**, the external memory interface will automatically perform the correct number of accesses for an 8, 16, or 32-bit accesses, given an 8 or 16-bit wide FLASH memory. When using an 8-bit wide FLASH memory, 1 access is required for an 8-bit access, 2 accesses are required for a 16-bit access, and 4 accesses are required for a 32-bit access. When using a 16-bit wide FLASH memory, 1 access is required for an 8-bit access, 1 access is required for a 16-bit access, and 2 accesses are required for a 32-bit access. To perform an 8-bit read from a 16-bit wide FLASH memory, the required 8-bit portion is loaded and the remaining 8-bits are ignored.

Access Size	Example of command from ARM	8 bits wide	16 bits wide
8-bit access	LDRB/STRB, etc.	1 time	1 time
16-bit access	LDRH/STRH, etc.	2 times	1 time
32-bit access	LDR/STR, etc.	4 times	2 times

Table 69: Relationships Between Access Size and Flash Memory Data Bus Width

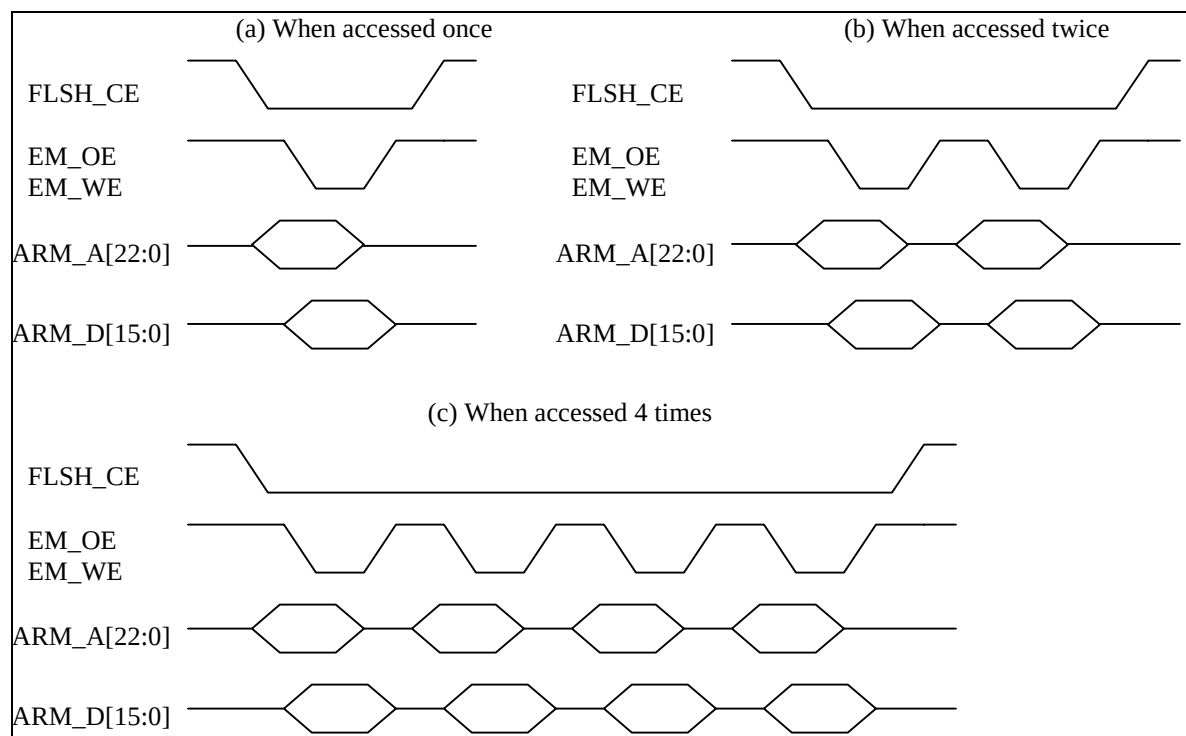


Figure 142: Flash Memory Access Timing Diagrams

20.1.8 External Memory Region 1: The CompactFlash Interface

External memory region 1 can be used to interface CompactFlash cards. The CompactFlash interface is shown in **Figure 143**. For detailed information on the CompactFlash interface signals, refer to the DM320 Datasheet. The interface supports memory mode and IO mode, however, true IDE mode is not supported. Refer to module on “Compact Flash” for more details.

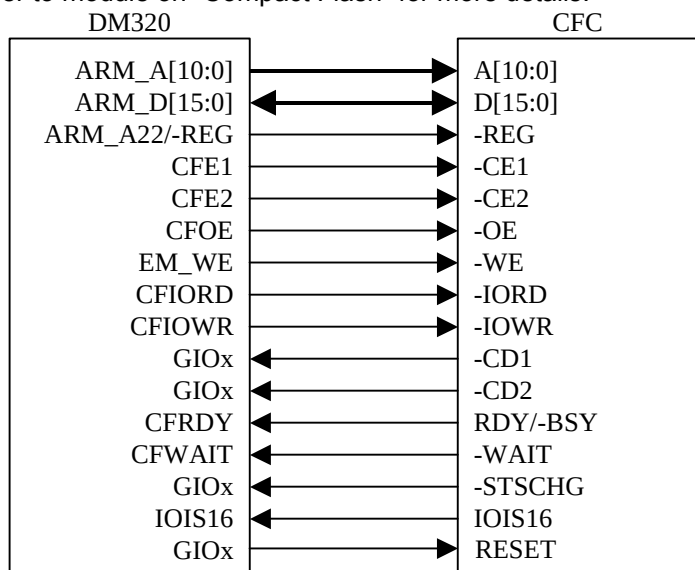


Figure 143: CompactFlash Interface

20.1.8.1 CompactFlash Data Bus

The CompactFlash interface supports dynamic bus sizing for I/O mode. Data access will be either 8-bit or 16-bit mode depending on the level of the IOIS16 pin. When IOIS16 is low, the CompactFlash interface operates with an 8-bit bus, and when IOIS16 is high, the interface operates with a 16-bit bus. Dynamic bus sizing is enabled when BUSSZ is set in CFCTRL2. When BUSSZ is cleared, the bus size is always 16-bits regardless of the IOIS16 signal. Similar to the FLASH interface, the correct number accesses are automatically performed when 32-bit, 16-bit, and 8-bit data is transferred, given that the data bus is set to be 8 or 16-bits wide.

20.1.8.2 CompactFlash Interface Memory Map

The CompactFlash Interface Memory Map is shown in **Table 70**. The table shows the offset address for the CFC Memory Space (Attribute and Common Memory Space) and the CFC IO Space. The addresses are offset from the start address of external memory region 1.

External Memory Area	Offset Address	Size	CompactFlash Interface Function
EM_CS1	00:0000 – 00:07FF	2-Kbytes	CFC Memory Space (Attribute and Common)
	40:0000 – 40:0FFF	4-Kbytes	CFC IO Space

Table 70: CompactFlash Interface Memory Map

20.1.9 External Memory Region 2: The SmartMedia Interface

External memory interface 2 has a built-in SmartMedia interface. **Figure 144** shows the SmartMedia interface. Refer to module on “Smart media” for more details.

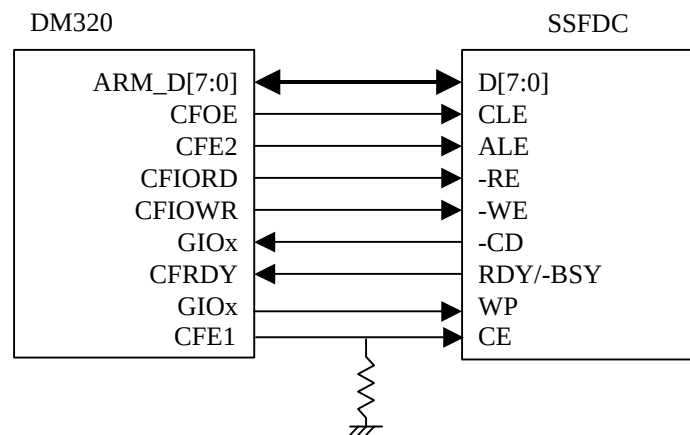


Figure 144: SmartMedia Interface

20.1.9.1 SmartMedia Data Bus

SmartMedia interface supports both 8-bit and 16-bit data bus width for command, address, and data.

Table 71 shows the offset address for command, address, and data. The ARM software should write SmartMedia commands to the address offset at 00:0000, write SmartMedia addresses to offset 00:0004, and should read and write SmartMedia data from offset 00:0008. The offset addresses are relative to the start address of external memory region 2.

External Memory Area	Offset Address	Size	Smart Media Interface Function
EM_CS2	00:0000	1 or 2 bytes	Smart Media Command Write
	00:0004	1 or 2 bytes	Smart Media Address Write
	00:0008	1 or 2 bytes	Smart Media Data Read/Write

Table 71 SmartMedia Interface Memory Map

20.1.9.2 Controlling the SmartMedia Chip Enable

The SmartMedia Chip Enable signal can be controlled by SCEH in SMCTRL . Before accessing the SmartMedia Card, enable the SmartMedia Chip Enable signal by clearing bit SCEH in the register SMCTRL . When SCEH is cleared, the chip enable signal, SCEH, is set low (active).

20.1.9.3 Controlling the SmartMedia Page Size

The SmartMedia page size can be set to 256+8-bytes per page or 512+16-bytes per page. This is set in PAGESZ register.

20.1.9.4 SmartMedia Interface Error Correction

The SmartMedia interface supports hardware error detection. During read and write, hardware error correction code (ECC) calculation is performed every 256-bytes. The generated ECC can be read from, ECC1LP, ECC8CP and ECC8LP registers. ECC1CP and ECC1LP are used for Storing Area -1 and ECC2CP and ECC2LP are used for Storing Area -2.

When the page size is configured to 256 + 8 bytes per page, Storing Area –1 corresponds to bytes 0-255 of the even numbered pages, and Storing Area – 2 corresponds to bytes 0-255 of odd numbered pages.

When the page size is configure to 512 + 16 bytes per page, Storing Area – 1 corresponds to bytes 0-255 and Storing Area – 2 corresponds to bytes 256-511.

Although the hardware calculates ECC, the ARM software must perform ECC data comparison and correction. If 512+16 byte page SmartMedia is used, the calculated ECC is stored at ECC1CP and ECC1LP after the first half (256 bytes) of data is read, and then ECC2CP and ECC2LP are updated after the second half (256-bytes) of data is read. The procedure in the case where smart media is used is shown below.

512+16bytes/page mode

- o Write “1” to ECLR in ECCCLR to clear the previous ECC results.
- o Read or Write the first half (0-255) of data -> ECC1CP & ECC1LP is updated.
- o Read or Write the second half (256-511) of data -> ECC2CP & ECC2LP is updated.
- o Check the ECC stored at ECC1CP, ECC1LP, ECC2CP and ECC2LP
- o Write “1” to ECLR in ECCCLR to clear the ECC for the next page access.

256+8bytes/page mode

- o Write “1” to ECLR in ECCCLR to clear the previous ECC results.
- o Read or Write the data area (0-255) of Even-numbered page -> ECC1CP & ECC1LP is updated.
- o Read or Write the data area (0-255) of Odd-numbered page -> ECC2CP & ECC2LP is updated.
- o Check the ECC stored at ECC1CP, ECC1LP, ECC2CP and ECC2LP
- o Write “1” to ECLR in ECCCLR to clear the ECC for the next pair of pages.

“Error Detection & Correction” can be performed by comparing the calculated ECC (ECC1LP, ECC2CP and ECC2LP) and the ECCCLR stored at the redundant area of the page.

20.1.10 CompactFlash Interface and SmartMedia Interface Interrupts and Signal Status**20.1.10.1 Interface Interrupts**

The external memory interface includes two interrupts, INT15, INT16. INT15 is generated when EMIF DMA completes, and this interrupt is described in the section “=”. INT16 is generated due to conditions specified in, and this interrupt is described in this section.

CFC Busy interrupt can be enabled or disabled by setting the bits in BUSWAITMD. The status of CFC Busy interrupt can be read from the bits in BUSWAITMD CFC Busy interrupt corresponds to INT16.

20.1.10.2 External Memory Interface Signal Status

Read the bits in BUSSTS to monitor the state of various input signals used by the External Memory Interface.

20.1.11 External Memory Regions 3 and 4

20.1.11.1 Data Bus Width

The data width of external memory areas 3 and 4 can be either 8 or 16 bits. The BUSW3 and BUSW4 bits in CS4CTRL2 and CS3CTRL1 configure the data width of region 3 and 4, respectively. By default, after reset, the data width is set to 8 bits.

20.1.11.2 External Wait

The EM_WAIT signal is available for region 3 and/or region 4, which BUSSTS enable asynchronous device interface capability. Refer to the DM320 Datasheet for detailed information on the signal interface related to the external wait feature. To enable the EM_WAIT signal for region 3 or 4, set EMWE3 or EMWE4 in BUSCTRL respectively.

When EM_WAIT is enabled and driven low during a bus cycle by an external device, an extra wait state (hardware wait-state) is added to the programmed wait states (software wait-states).

For instance, if CS_WAIT = 5 and EM_WAIT is held low for 5 clock cycles, the total number of wait states will be 10.

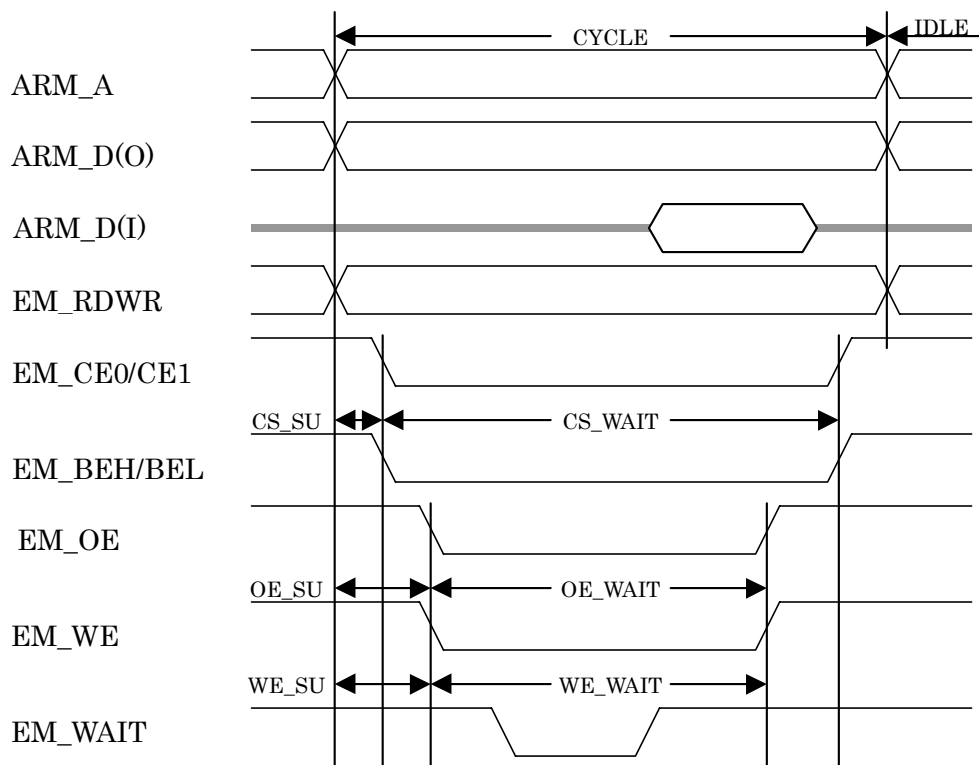


Figure 145: External memory bus timing with hardware wait-states

20.1.12 Bus Release

The EMIF bus and SDRAM bus can be released for an external device by using BUSRLS.

When EMIF bus is released, all EMIF related signal (Address/Data/Chip select) become Hi-Z. Clearing the EMIF bus release (EMREL) turns on the EMIF buses.

Additionally, the SDRAM bus can be released to an external device. All SDRAM buses (ADDRESS/DATA/RAS/CAS/CS/WE/DQM/CKE/CLK) can be released (output Hi-Z) by setting the SDRAM bus release register, SDREL in BUSRLS register, in the external memory interface module. Clearing the SDRAM bus release register turns on SDRAM buses. By using this function, SDRAM can be shared between external devices. SDRAM bus outputs Hi-Z while the SDRAM bus open register is on.

Figure below shows an example sequence of bus release, acknowledge during release of EMIF bus. Since EM_BUSREQ, EM_BUSACK signals are not available in DM320; the user must use some GIOs to implement bus request/acknowledge type of handshake. A similar approach must be taken for release of SDRAM data bus.

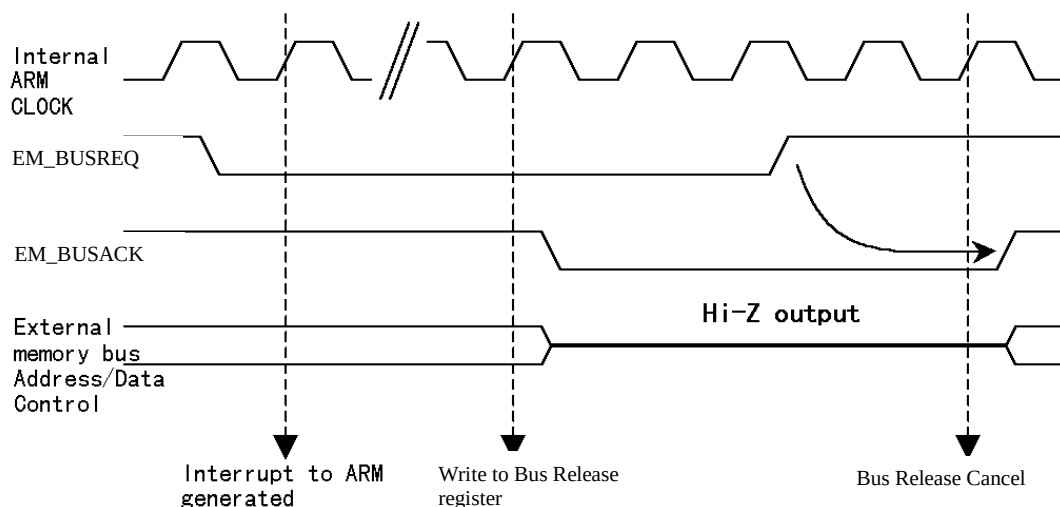


Figure 146: Bus release, acknowledgment sequence

20.1.13 EMIF access priority

The ARM, the Image Buffer's DMA controller (DSP), Host Port Interface Bus (DSP), and the MTC DMA controller can access the EMIF. PRIORCTL allows configuring EMIF access priority. By default the priority of EMIF access is as follows:

(Highest) ARM > MTC DMA > Image Buffer DMA > HPIB (DSP) (lowest)

20.2 SDRAM controller

20.2.1 Features of the SDRAM controller

The SDRAM controller, as shown in **Figure 147**, acts as the primary interface between SDRAM and all functional blocks such as ARM, DSP, CCDC, Preview Engine, OSD and DMA controller. This block supports SDRAM timing of a maximum of 100 MHz (refer to the DM320 data sheet for exact value), and provides continuous data access with low overhead. The start address and size of the SDRAM memory region are configurable within the external memory region of the ARM memory map. The SDRAM controller supports a DMA controller interface. The SDRAM access priority can be modified. The SDRAM controller supports refresh commands and can place the device in a self-refresh mode prior to going a power down mode. The refresh interval is programmable. The latency for the column access strobe (CAS) is programmable from 2 to 3.

- Supports 16/64/128/256/512/1024/2048M SDRAM
- Supports both 32-bit-wide and 16-bit-wide SDRAM
- Maximum 100 MHz operation (see DM320 datasheet for exact value)
- Access in word (32-bits), half word (16-bits), and byte (8-bits) units
- Supports mode setting, power down and self-refresh commands
- Programmable auto-refresh interval
- CAS latency of 2 or 3 cycles can be selected
- Maximum SDRAM size 256 Mbytes
- DMA transfer with peripheral modules
- Variable SDRAM access priority
- SDRAM Controller buffer access

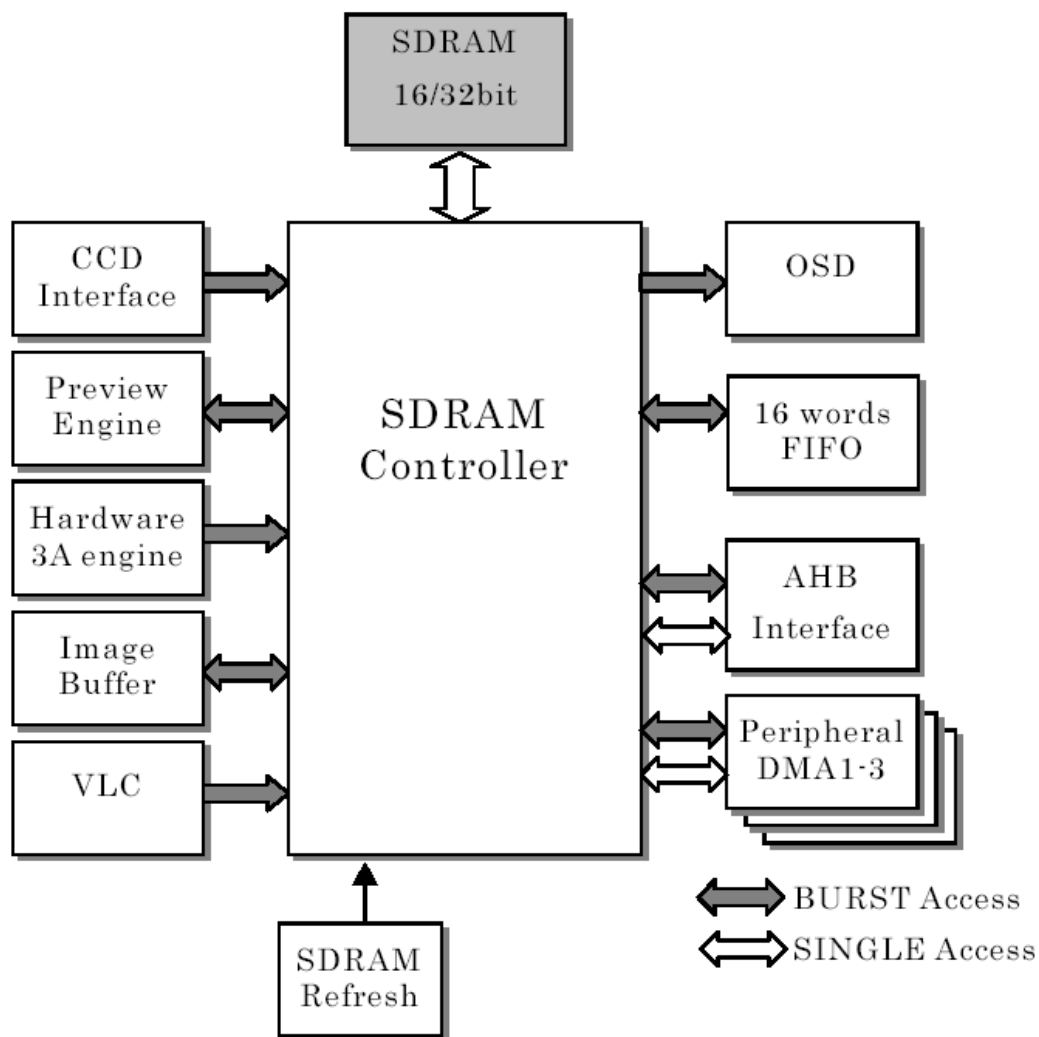


Figure 147: SDRAM Controller Block

20.2.2 Start Address and Size of the SDRAM Region

The SDRAM region IS within the external memory region. The SDRAM region is set by configuring the SDRAM start and end addresses in AHB bus controller register. The boundary address of the SDRAM region can be specified in 1-Mbyte units. The reset value of SDRAM size is 24 Mbytes.

The SDRAM controller can access up to 2048-Mbits (256-Mbytes) of SDRAM and supports both 16 and 32-bit SDRAM devices. Each access can be done in word (32-bit), half-word (16-bit) or byte units from the ARM side.

After reset, the starting address of SDRAM is 0x00900000h.

20.2.3 Hardware Connection

The functions of several pins in the SDRAM are shown in **Table 72**.

PIN Name	I/O	Explanation
SDR_D [31-16]	I/O	SDRAM Data Bus (32-bit)
SDR_D [15-0]	I/O	SDRAM Data bus (16-bit)
SDR_D [12-0]	O	SDRAM Address Bus
SDR_A [14-13]	O	SDRAM Bank Address bus
SDR_RAS	O	SDRAM Row address Strobe
SDR_CAS	O	SDRAM Column address Strobe
SDR_WE	O	SDRAM Write Enable
SDR_CS	O	SDRAM Chip Select
SDR_CKE	O	SDRAM Clock Enable
SDR_DQM3	O	SDRAM Mask for DQ [31-24]
SDR_DQM2	O	SDRAM Mask for DQ [23-16]
SDR_DQM1	O	SDRAM Mask for DQ [15-8]
SDR_DQM0	O	SDRAM Mask for DQ [7-0]
SDR_CLK	O	SDRAM Clock

Table 72: The functions of several SDRAM Controller pins

20.2.4 Memory Structure Setting

The DM320 can connect to SDRAM of a maximum of 256-Mbytes. The address that is connected differs depending on the memory structure of the SDRAM, as follows.

Memory structure	ADDRESS	Number of banks	BANK	Size in Mbyte	Size in MBits
2k x 256 WORD	SDR_A10- SDR_A0	2	SDR_A13	4	32
		4	SDR_A14- SDR_A13	8	64
4k x 256 WORD	SDR_A11- SDR_A0	2	SDR_A13	8	64
		4	SDR_A14- SDR_A13	16	128
4k x 512 WORD	SDR_A11- SDR_A0	2	SDR_A13	16	128
		4	SDR_A14- SDR_A13	32	256
8k x 512 WORD	SDR_A12- SDR_A0	2	SDR_A13	32	256
		4	SDR_A14- SDR_A13	64	512
8k x 1024 WORD	SDR_A12- SDR_A0	2	SDR_A13	64	512
		4	SDR_A14- SDR_A13	128	1024
8k x 2048 WORD	SDR_A12- SDR_A0	2	SDR_A13	128	1024
		4	SDR_A14- SDR_A13	256	2048

Table 73 Relation between memory structure and address

20.2.5 SDRAM configuration registers

Before using the SDRAM, the SDRAM controller clock must be enabled in MOD2 register in the clock controller module. After the module is on configure SDRAMC to match the SDRAM memory specifications. The SDMODE register allows changing the number of cycles to access the memory as well as the latency for the CAS. The memory can be configured to use 2 or 4 banks. The SDMODE is used to write the commands that need to be sent to the SDRAM chip. SDMODE is used to configure the auto-refresh frequency used by the controller and to select the bus width for SDRAM access.

In addition to this register REFCTL allows to specify the auto-refresh frequency used by the SDRAM Controller.

Parameter	Description	Number of CLK_SDR Cycles
TRC	Row cycle time	Min 8 + TRCD+ TRDL
TRCD	Row to Column delay	2 + TRCD
TRRD	Act to Act delay time	4
tRDL + tRP	Last data in to precharge + Row precharge time	Min 3 + TRDL

Table 74 SDRAM Burst Write with Auto-Precharge in 32-bit Mode

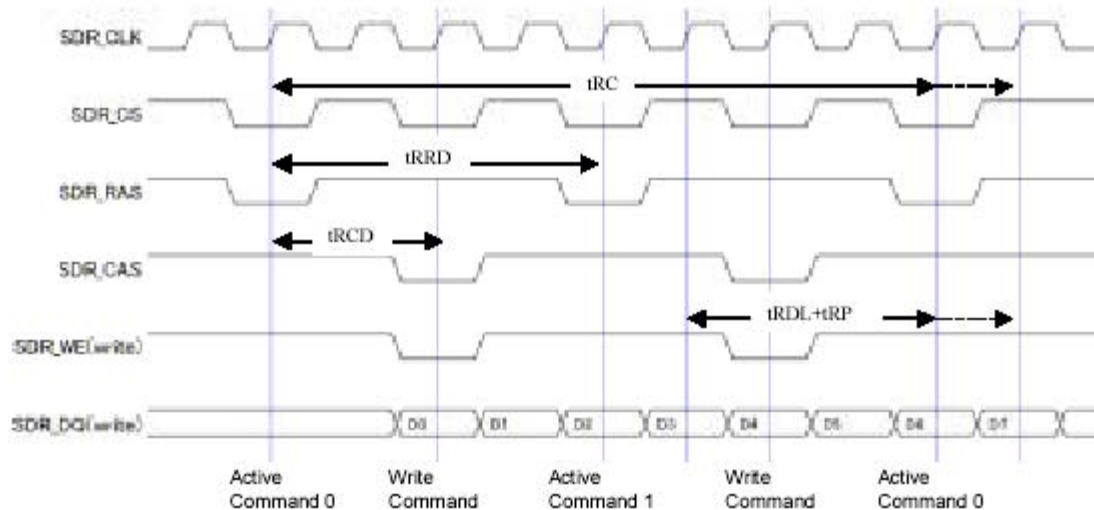


Figure 148 SDRAM Burst write with Auto-precharge in 32-bit mode (BL=0, TRCD=4)

Parameter	Description	Number of CLK_SDR Cycles
TRC	Row cycle time	Min 7 + TRCD
TRCD	Row to Column delay	2 + TRCD
TRAS	Row Active time	5
TRDL	Last data in to precharge	3
TRP	Row precharge time	2

Table 75 SDRAM Burst write interrupt by precharge(Single Access) in 32-bit Mode (BL=4)

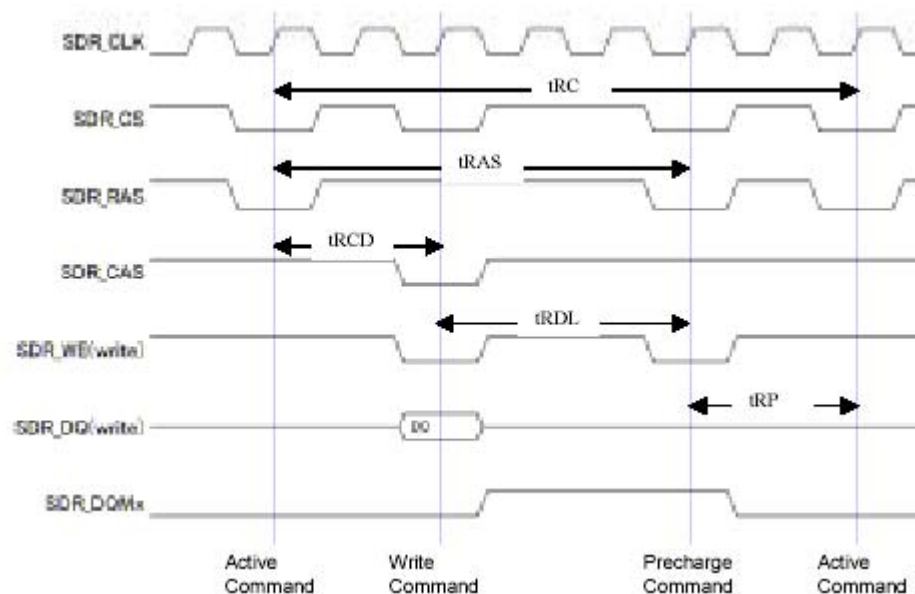


Figure 149 SDRAM Burst Write Interrupt by Precharge (Single Access) in 32-bit mode (BL=4, TRCD=0)

Parameter	Description	Number of CLK_SDR Cycles
tRC	Row cycle time	Min 16 + TRCD
tRCD	Row to Column delay	2 + TRCD
tRRD	Act to Act delay time	8
tRDL + tRP	Last data in to precharge + Row precharge time	7

Table 76: SDRAM Burst write with Auto Precharge in 16 bit mode (BL=8)

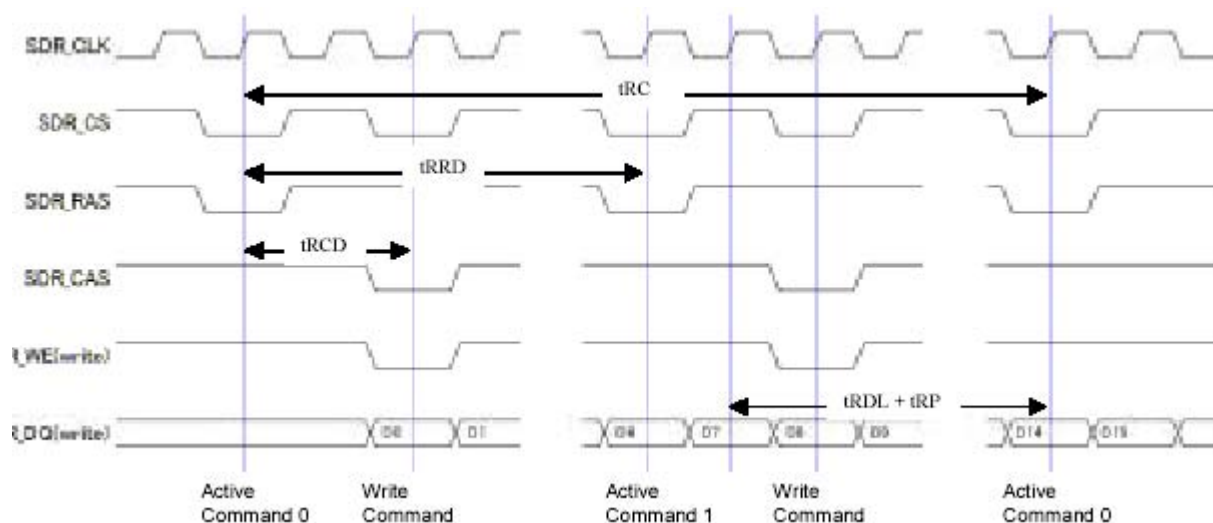


Figure 150 SDRAM Burst Write with Auto precharge in 16-Bit Mode (BL=8)

Parameter	Description	Number of CLK_SDR Cycles
TRC	Row cycle time	Min 7 + TRCD
TRCD	Row to Column delay	2 + TRCD
TRAS	Row Active time	6
TRDL	Last data in to precharge	3
TRP	Row precharge time	2

Table 77 SDRAM Burst Write Interrupt with Auto precharge (BL=8)

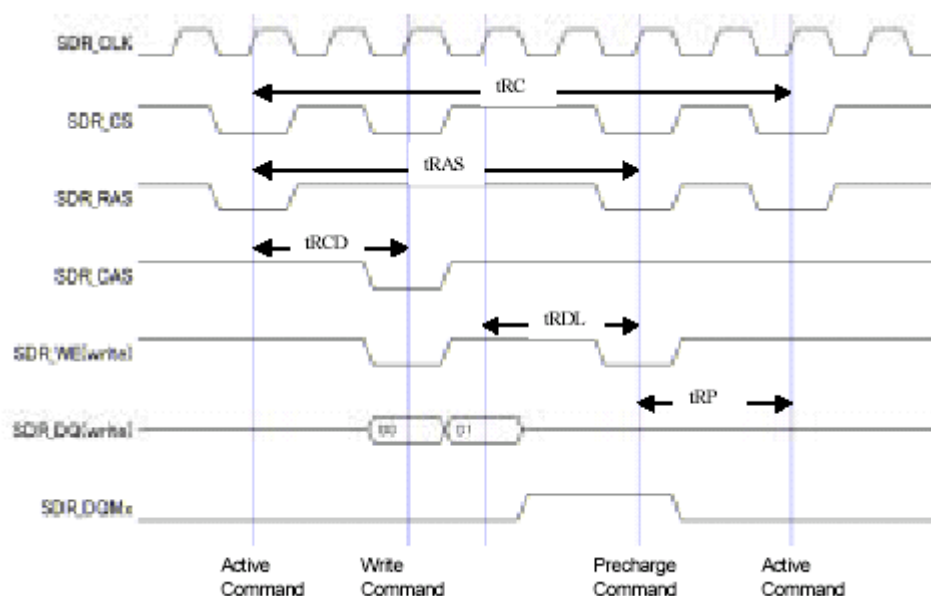


Figure 151 SDRAM Burst Write Interrupt in 16-bit Mode (BL=8)

20.2.6 SDRAM command

The ARM can issues the following commands to SDRAM by settings the registers in the SDRAM controller (SDCTL in SDMODE):

- 1) MRS (mode register set)
- 2) PREA (pre-charge all)
- 3) REF (auto refresh)
- 4) SELF (self refresh entry)
- 5) SELFC (self refresh end)
- 6) PDN (power down mode)
- 7) DPDN (Deep power down mode)

When an MRS command is issued, the following data settings are performed in the SDRAM mode register.

SDRAM Address pin	A12	A11	A10	A9	A8	A7	A6	A5	A4	A3	A2	A1	A0
Meaning	Reserved				0	0	Read Latency			S/I	Burst length		
DM320 output value	0	0	0	0	0	0	0	CL	0	0	0	BST 1	BST 0

Table 78: MRS command and SDMODE register

20.2.7 SDRAM priority access

The access priority to the SDRAM can be modified. The registers SDPRTY1 to SDPRTY10 control the priorities as shown in **Table 79**, by default the CCDC will have the highest priority and the VLNC request has the lowest.

The binary value “1” cannot be written twice or more into any one priority register. If you do, the bit 9 has highest priority, and bit 0 has lowest priority. As well, if “1” is written in the same bit position in different registers, the request of the low register address takes priority, and the request of the high register address is ignored.

When the priority configuration is ready, you need to write a ‘1’ into the bit USEPRTY of the SDPRTYON to enable it. Since SDRAM controller internal priority is changed automatically when access to SDRAM becomes IDLE (e.g. at the end of the current burst transfer), you do not need to pay attention to the change period.

If a request with a higher priority than the current transfer one occurs, the burst transfer in progress is completed prior to serving the received request. The size of a burst is 4 32-bit word in a 32 bits mode and 8 16-bit half-word in a 16 bits mode.

The SDRAM priority is changed automatically when access to SDRAM goes IDLE the user does not need to pay attention to the change period.

Register name	Default Register setting	SDRAM controller input	Priority
SDPRTY1	0x800	CCDC request	Highest
SDPRTY2	0x400	Preview Engine request	
SDPRTY3	0x200	Hardware 3A request	
SDPRTY4	0x100	OSD request	
SDPRTY5	0x080	ARM CPU random access	
SDPRTY6	0x040	MTC DMA, MS, MMCSD, DSPDMA, SIF in SDDMA channel-1	
SDPRTY7	0x020	MTC DMA, MS, MMCSD, DSPDMA, SIF in SDDMA channel-2	
SDPRTY8	0x010	MTC DMA, MS, MMCSD, DSPDMA, SIF in SDDMA channel-3	
SDPRTY9	0x008	ARM FIFO	
SDPRTY10	0x004	SDRAM AUTO REFRESH	
SDPRTY11	0x002	Image Buffer (DSP) DMA request	Lowest
SDPRTY12	0x001	VLC request	

Table 79: SDRAM access priority

20.3 SDRAM Initialization

SDRAM has a power on procedure. Since the procedure differs depending on the SDRAM that is used, SDRAM must be initialized by software using the SDRAM commands. Also, since the DQM (input/output mask) signal is high output as default, after the mode register set command (MRS) is finished, DQM

must be set to low level. Setting registers in the SDRAM controller perform these SDRAM settings.

A typical sequence is given below:

- 1) Apply power and start the clock. Attempt to maintain the NOP command at the input.
- 2) Maintain stable power, stable clock, and the NOP condition for a minimum of 200 μ s. (Minimum time depends on devices. Please refer to the datasheet of the SDRAM used)
- 3) Pre-charge all banks by the Pre-charge All (PREA) command.
- 4) Assert a minimum of 8 Auto-refresh commands. (Assert time depends on the devices. Please refer to the datasheet of the SDRAM used)
- 5) Program the mode register by the Mode Register Set command (MRS).
- 6) If required enable auto-refresh and set the appropriate refresh interval in register SDPRTY10
- 7) Put the mode back to NOP

20.3.1 SDRAM Commands

20.3.1.1 Pre-charge

The pre-charge command is used to deactivate the opened row in a particular bank or the opened row in all the banks. The banks are available for subsequent row access after a specified time (tRP) after the pre-charge command. Once a bank has been pre-charged, it is in the IDLE state and must be activated prior to read / write commands being issued to the bank. The Pre-charge command will be treated as a NOP command if there is no active row in the bank.

20.3.1.2 Auto-refresh

The Auto-Refresh is used during the normal operation of the SDRAM. The Command is non-persistent, so it must be issued each time a refresh is required. A minimum row cycle time (tRC) is required between two AUTOREFRSH commands. Internal refresh controller generates the refresh address.

20.3.1.3 Self-refresh

The self-refresh command can be used to retain the data in the SDRAM even if the rest of the system is powered down. When the SDRAM is in Self-refresh mode it retains the data without any external clocking.

20.3.1.4 Power-down

When the SDRAM is in power down mode, all the banks are in IDLE state; this mode is referred as pre-charge power-down. If power-down mode occurs when there is any active row in any bank, this mode is called active power down mode. After entering power down mode, it deactivates all the input and output buffers. However, the power down duration is limited to the refresh requirement of the device (tREF). One Clock is required for power down entry and exit.

20.3.1.5 Deep Power-down

The Deep power down is an unique function on Mobile SDRAMs with very low standby currents. All internal voltage generators inside the SDRAMs are stopped and all memory data is lost in this mode. To enter deep down power mode all banks must be precharged.

20.3.1.5.1 NOP

The no-operation command (NOP) is used to perform a NOP to the SDRAM (CS=LOW). This prevents unwanted commands being registered during IDLE states but operation already in progress will not be affected.

20.3.1.6 Mode register set

The Mode registers set are loaded via inputs (A0-A12). The MODE REGISTER SET command can only be issued when all the banks are IDLE and no burst access are in progress.

20.3.1.7 Active

Before any READ / WRITE commands can be issued to a bank with in a SDRAM, a row in a bank must be opened. This accomplished via ACTIVE Command. The minimum time interval between the successive ACTIVE commands to the same bank is defined by tRC. But a subsequent ACTIVE command can be issued while the first ACTIVE command is being accessed. The minimum time interval between successive ACTIVE commands to different banks is defined by tRRD.

20.3.1.8 Read

Subsequent to programming the MRS with CAS latency and burst length, READ burst are initiated with a READ command. The starting column and bank addresses are provided with the READ and AUTO PRECHARGE is either enabled or disabled for that burst access. During the read burst the valid data-out element from the starting column address is available the CAS latency after the READ command.

20.3.1.9 Write

The WRITE bursts are initiated with a WRITE command. The starting column addresses are provided with the WRITE command, and auto PRECHARGE is either enabled or disabled for that access. If auto pre-charge is enabled, the row is being accessed is pre-charged at the completion of the write burst.

20.3.2 ARM buffered access

In DM320, there are two types of access to SDRAM: burst access via a buffer, and random memory access. Transfer to and from SDRAM is performed in 16 16-bit word units. There is a control register (SDBUFCTL) in the SDRAM controller for burst access as well as sixteen 16-bit data registers SDBUFD0L to SDBUFD7H) to store the data from 16-word burst transfer.

To write into the SDRAM, data needs to be set in these data registers. After the write, destination SDRAM address is set in the buffer address registers (SDBUFD0L , to SDBUFD7H), a write command can be written in the SDBUFCTL register. When the proper bit (WM or WA) in this register is written, writing to the SDRAM is performed.

To read from SDRAM, the starting SDRAM address must be set. This information is stored in the buffer address registers - SDBUFAD1 and SDBUFAD2. When a read command is issued by setting the bit RSD of the control SDBUFCTL register, the data is read from SDRAM and stored in the data registers.

Check the SDBUFCTL register to know the status of a transfer.

Auto-increment of SDRAM address is possible by setting the field AAI in register SDBUFAD1.

20.3.3 SDRAM Programmable DMA Channels

The SDRAM controller has three programmable DMA channels. Each DMA channel can be connected to SIF, MS, MMCSD, HPIDMA and MTC DMA. The channel to be used for DMA to a device is configured in the SDDMASEL register.

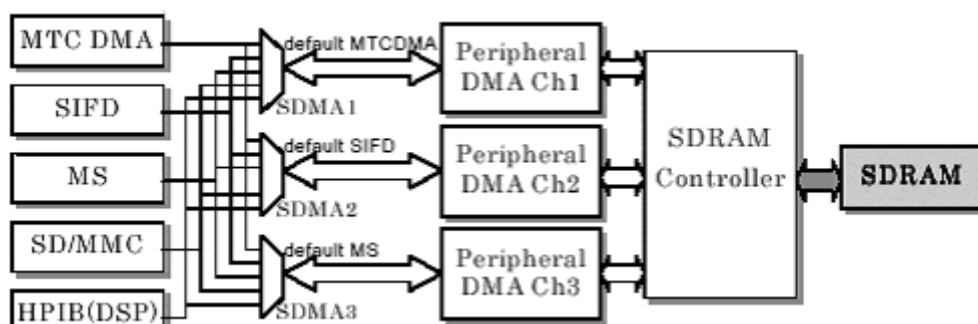


Figure 152 SDRAM DMA Channels selections

The DMA channel supports burst transfers. The burst types supported are 1-burst, 4-bursts and 8-bursts. If 8-burst is selected the DMA start address would be 32-bit aligned. If 4-burst access is selected the DMA start address should be 16-bit aligned. Configure the SDDMASEL register for the required burst size.

The DMA start address is set in each peripheral register and the address is the relative address from the SDRAM region. The transfer size is specified in words (4 bytes) in each peripheral.

BURST-3 [1-0]	DMA Burst Size	DMA Start Address
00	Single burst	Address [1-0]="00"
1	4 Burst	Address [3-0]="000"
10	8 burst	Address [4-0]="0000"

Table 80: SDRAM Burst selection

20.4 MTC DMA Controller

20.4.1 Data Transfer Between EMIF regions

The MTC has a built-in DMA controller, which can transfer data between MTC devices and AHB devices. The MTC and AHB devices are listed in *Table 81*. **Figure 153** illustrates the MTC DMA.

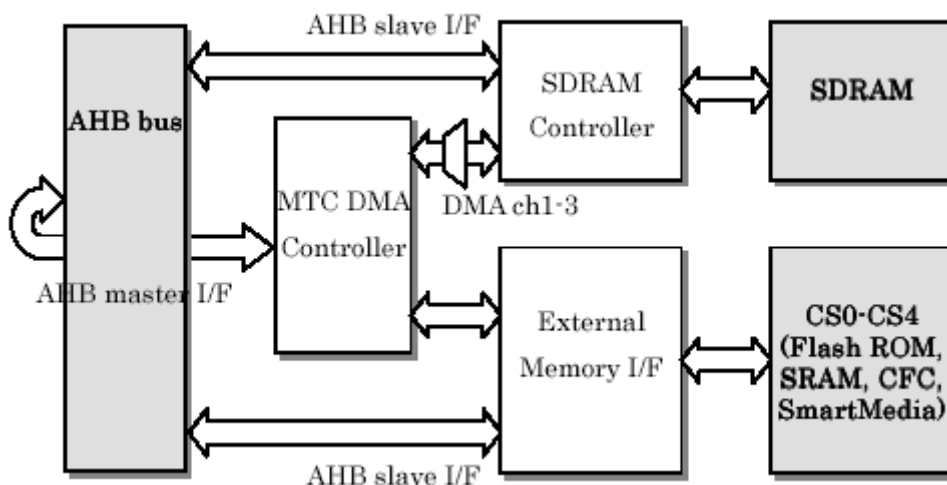


Figure 153: MTC DMA

20.4.2 MTC DMA Sources/Destinations Addresses and Transfer Size

The registers to configure the DMA start address for AHB slave and MTC's device as well as the transfer size as shown in **Table 82**. The MTC's device addresses are relative to the start of the each region used and the address are multiples of four. Also, the transfer size must always specify in a multiple of four. The AHB slave address is Absolute in the ARM memory map. The direction bit, DIR bit in the DMACTL register, determines the source and destination.

The SDRAM controller has three DMA channels, which are used to perform DMA's involving SDRAM as source or destination. A DMA channel must be selected prior to triggering DMA, when SDRAM is either a source or a destination. The DMA channel is selected using the SDDMASEL register.

The priority of MTC DMA for SDRAM access can be changed using the SDPRTYON register.

AHB Devices	MTC Devices
ARM internal Memory	External general-purpose memory CS0
External HOST interface	External general-purpose memory CS1
USB Controller	External general-purpose memory CS2
External memory Interface (EMIF in MTC)	External general-purpose memory CS3
SDRAM (SDRC in MTC)	External general-purpose memory CS4
	SDRAM

Table 81: DMA transfer source, transfer destination devices

REGISTER	DESCRIPTION
AHBADDH	AHB Slave higher Absolute address
AHBADDL	AHB Slave lower Absolute address
MTCADDH	MTC higher relative address
MTCADDL	MTC lower relative address
DMACTL	Configure Endianess, Direction and Start transfer
DMASIZE	DMA size of transfer register
DMAMTCSEL	MTC device select register

Table 82 MTC DMA Transfer registers**20.4.3 MTC DMA transfer control**

When SDRAM is selected by MTC's device select and programmable DMA channel is configured for 8 burst access, the address must start from 8--burst boundary. Also the channel is configured for 4 burst, the address must start from 4-burst boundary. The burst configuration is done using the SDDMASEL register.

By configuring the ENDI bits in DMACTL register the endianess of data transferred can be changed. The endianess changing is possible only when the MTC device is the source for the DMA.

An interrupt (INT17) is generated when DMA is finished. Also user can read GO bit in DMACTL register to identify, if DMA is on going or finished.

CS1 and CS2 are generally used to interface Compact Flash and SmartMedia cards, respectively. Data transfer to a CompactFlash or SmartMedia card is always performed through a fixed address, so unlike other memories, the access address should not auto-increment after each transfer. This has to be configured in the AHBADDH or MTCADDH register depending on whether the device is as AHB or MTC device.

Note: Endianess conversion is possible only when the MTC device is the source for the DMA.

20.4.4 COP DMA and DSP HPIB access to the EMIF

It is possible to transfer DSP internal memory data to the EMIF/SDRAM using HPIB DMA. Coprocessor DMA can transfer data to and from the Coprocessor shared memories (Image buffers, iMX Memory etc) to the EMIF/SDRAM.

The EMIF device for COP / HPIB DMA is configured using IMGDSPDEST register. When CS1 or CS2 (Compact flash or Smart Media) is the source / destination in a COP / HPIB DMA, IMGDSPADDH and IMGDSPADDL registers are used to specify the address. When transferring data to / from SDRAM, CS0, CS3, or CS4 the registers in the HPIB and COP DMA are used to specify SDRAM / EMIF transfer address.

IMPORTANT: When using HPIB DMA to SDRAM, the DMA channel must be selected using SDDMASEL register.

20.5 SDRAM & External Memory I/F Pin Sharing

The SDRAM interface signals and EMIF interface signals can share SDRAM bus on the DM320. The pin sharing is illustrated in **Figure 154**. The sharing of pins between SDRAM and external memory is carried out on bus cycle by bus cycle basis. Each of SDRAM interface and flash memory interface has a separate chip select. When the chip select of the SDRAM is asserted for a particular bus cycle, the shared pins generate the SDRAM-related timing control signals to perform the cycle. Similarly for external memory, if its chip select is asserted, the shared pins generate the signals conforming to external memory to perform the cycle. It is possible that while ARM is accessing the external memory (for example, to fetch an instruction), there may be concurrent DMA activities on the SDRAM. In such cases, external memory and the DMA will arbitrate for the control for the memory interface.

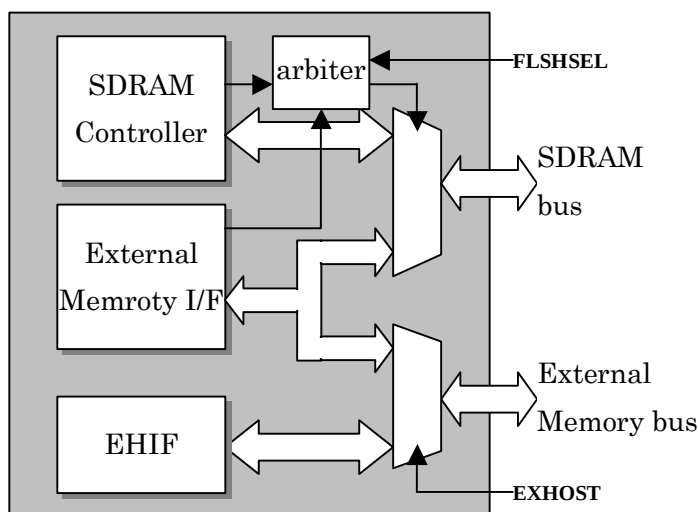


Figure 154: SDRAM and EMIF pin sharing

Table 83 shows the different modes of SDRAM and the External memory Bus pin sharing. All the logic for multiplexing SDRAM interface signals and the EMIF interface signals to the I/O pad is done by configuring BUSSEL bit in BUSRLS register and FLHSSEL/EXHOST pins. **Table 85** shows the pin list, which is shared between SDRAM bus and external memory bus. The different modes are illustrated in **Figure 155**, **Figure 156** and **Figure 157**.

In normal mode, the master of the SDRAM memory is the SDRAM controller (SDRC). In pin-sharing mode the SDRAM interface pins are not only used by the SDRAM controller but by External Memory Interface (EMIF) also and they are arbitrated.

Pin sharing is required to support the media storage card hot plug capability, within the constraints of pin count. The basic idea of hot plug is to have two separate buses for the media storage card (for example Compact Flash Card)

and the flash memory so that plugging and unplugging of the media card does not interfere with the normal operation of flash memory. Another pin sharing method is available to support external HOST CPU connection with external memory bus as shown in **Figure 158**. In Mode 3 the external CPU can be a bus master, since the external memory bus is released to it.

MODE	FLSHSEL*	BUSSEL bit	SDRAM	CS0	CS1-CS4
MODE1	Low	0	SDRAM bus	Ex. Memory bus	Ex. Memory bus
MODE2	High	0	SDRAM bus	SDRAM bus	Ex. Memory bus
MODE3	High	1	SDRAM bus	SDRAM bus	SDRAM bus

*FLSHSEL: External pin

Table 83 Memory Bus sharing

EXTHOST*	External Host CPU
Low	Not available
High	Available

Table 84 External Host CPU Configurations

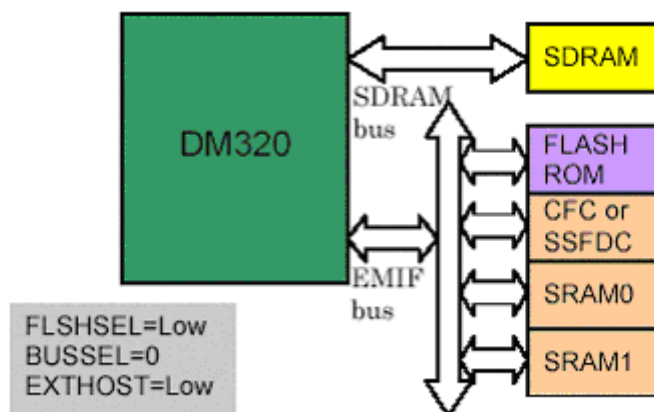


Figure 155: MODE1

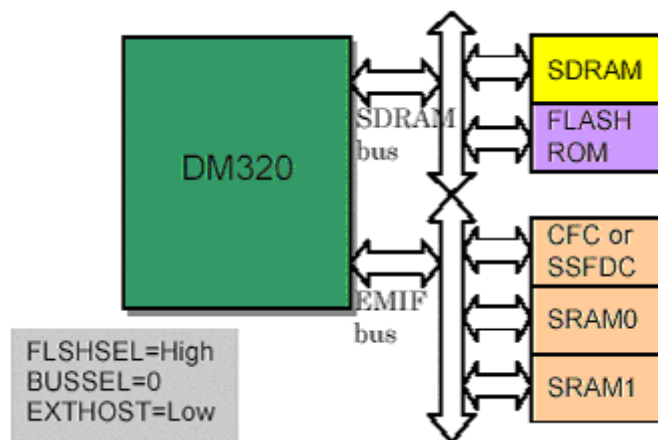


Figure 156: MODE2

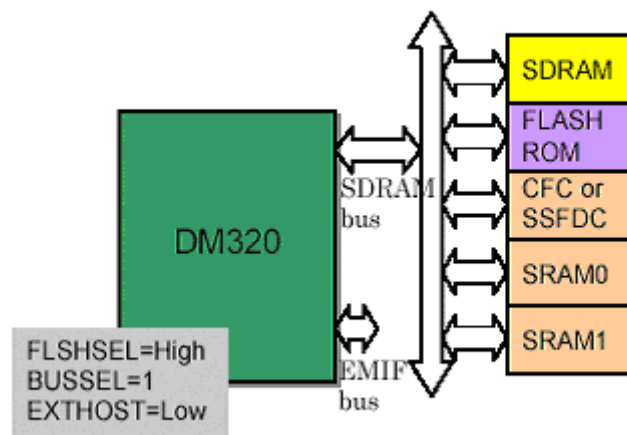


Figure 157: MODE3

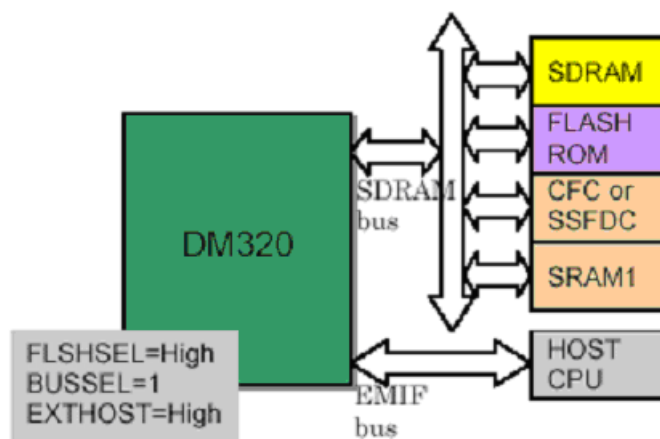


Figure 158: MODE3 + External Host CPU

Pin name	External Memory bus
SDR_DQ23	ARM_A22
SDR_DQ22	ARM_A21
SDR_DQ21	ARM_A20
SDR_DQ20	ARM_A19
SDR_DQ19	ARM_A18
SDR_DQ18	ARM_A17
SDR_DQ17	ARM_A16
SDR_DQ16	ARM_A15
SDR_A14	ARM_A14
SDR_A13	ARM_A13
SDR_A12	ARM_A12
SDR_A11	ARM_A11
SDR_A10	ARM_A10
SDR_A9	ARM_A9
SDR_A8	ARM_A8
SDR_A7	ARM_A7
SDR_A6	ARM_A6
SDR_A5	ARM_A5
SDR_A4	ARM_A4
SDR_A3	ARM_A3
SDR_A2	ARM_A2
SDR_A1	ARM_A1
SDR_A0	ARM_A0
SDR_DQ15	ARM_D15
SDR_DQ14	ARM_D14
SDR_DQ13	ARM_D13
SDR_DQ12	ARM_D12
SDR_DQ11	ARM_D11
SDR_DQ10	ARM_D10
SDR_DQ9	ARM_D9
SDR_DQ8	ARM_D8
SDR_DQ7	ARM_D7
SDR_DQ6	ARM_D6
SDR_DQ5	ARM_D5
SDR_DQ4	ARM_D4
SDR_DQ3	ARM_D3
SDR_DQ2	ARM_D2
SDR_DQ1	ARM_D1
SDR_DQ0	ARM_D0
SDR_RAS	EM_OE
SDR_CAS	EM_WE
SDR_DQ26	EM_RDWR
SDR_DQ25	EM_BEH
SDR_DQ24	EM_BEL

Table 85: Pins shared between SDRAM and EXBUS

The EMIF bus can be released to an external device by configuring EMREL bit of BUSRLS register. All EMIF signals (ADDRESS/DATA/CS*/WE/OE) are released (output high impedance). By using this it is possible to share memory among external memories.

The SDRAM bus can also be released to an external device. All SDRAM interface signals (ADDRESS/DATA/RAS/CAS/CS/WE/DQM/CKE/CLK) can be released (output high impedance) by setting the SDREL bit in BUSRLS register. By using this function, SDRAM bus can be shared between external devices or can be used to turn off the power to SDRAM.

20.6 EMIF Register Map

Address	Register	Description
0003.0A00	CS0CTRL1	CS0 Control Register #1
0003.0A02	CS0CTRL2	CS0 Control Register #2
0003.0A04	CS0CTRL3	CS0 Control Register #3
0003.0A06	CS1CTRL1A	CS1 Control Register #1A
0003.0A08	CS1CTRL1B	CS1 Control Register #1B
0003.0A0A	CS1CTRL2	CS1 Control Register #2
0003.0A0C	CS2CTRL1	CS2 Control Register #1
0003.0A0E	CS2CTRL2	CS2 Control Register #2
0003.0A10	CS3CTRL1	CS3 Control Register #1
0003.0A12	CS3CTRL2	CS3 Control Register #2
0003.0A14	CS4CTRL1	CS4 Control Register #1
0003.0A16	CS4CTRL2	CS4 Control Register #2
0003.0A18	BUSCTRL	Bus Control Register
0003.0A1A	BUSRLS	Bus Release Control Register
0003.0A1C	CFCTRL1	CFC Control Register #1
0003.0A1E	CFCTRL2	CFC Control Register #2
0003.0A20	SMCTRL	SmartMedia Control Register
0003.0A22	BUSINTEN	Bus Interrupt Enable Register
0003.0A24	BUSSTS	Bus Status Register
0003.0A26	BUSWAITMD	Bus Wait Mode Register
0003.0A28	ECC1CP	ECC Area 1 CP Register
0003.0A2A	ECC1LP	ECC Area 1 LP Register
0003.0A2C	ECC2CP	ECC Area 2 CP Register
0003.0A2E	ECC2LP	ECC Area 2 LP Register
0003.0A30	ECC3CP	ECC Area 3 CP Register
0003.0A32	ECC3LP	ECC Area 3 LP Register
0003.0A34	ECC4CP	ECC Area 4 CP Register
0003.0A36	ECC4LP	ECC Area 4 LP Register
0003.0A38	ECC5CP	ECC Area 5 CP Register
0003.0A3A	ECC5LP	ECC Area 5 LP Register
0003.0A3C	ECC6CP	ECC Area 6 CP Register
0003.0A3E	ECC6LP	ECC Area 6 LP Register
0003.0A40	ECC7CP	ECC Area 7 CP Register
0003.0A42	ECC7LP	ECC Area 7 LP Register
0003.0A44	ECC8CP	ECC Area 8 CP Register
0003.0A46	ECC8LP	ECC Area 8 LP Register
0003.0A48	ECCCLR	ECC Clear Register
0003.0A4A	PAGESZ	SmartMedia Page Size Register
0003.0A4C	PRIDRCTL	Priority control for DMA
0003.0A4E	MGDSPDEST	DSP / MGBUF DMA destination
0003.0A50	MGDSPADDH	DSP / MGBUF high address
0003.0A52	MGDSPADDL	DSP / MGBUF low address
0003.0A54	AHBADDH	AHB high address
0003.0A56	AHBADDL	AHB low address
0003.0A58	MTCADDH	MTC high address
0003.0A5A	MTCADDL	MTC low address
0003.0A5C	DMASIZE	DMA Transfer Size Register
0003.0A5E	DMAMTSEL	DMA Device Select Register
0003.0A60	DMACTL	DMA Control Register

20.7 External Memory Interface Registers

20.7.1 CS0CTRL1

Control register1 of EMIF Region zero.

CS0CTRL1	0003:0A00				offset: 0x00				default: 0x889A							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	OEC0[3]	OEC0[2]	OEC0[1]	OEC0[0]	WEC0[3]	WEC0[2]	WEC0[1]	WEC0[0]	CEC0[3]	CEC0[2]	CEC0[1]	CEC0[0]	CYCL0[3]	CYCL0[2]	CYCL0[1]	CYCL0[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	1	0	0	0	1	0	0	0	1	0	0	1	1	0	1	0

Bit	Name	Reset Value	R/W	Function
15:12	OEC0	8	R/W	<u>Output Enable Width</u> » 0000: 1 « » 0001: 2 « : » 1110: 15 « » 1111: 16 « The width is (OEC + 1) ARM clock cycles.
11:8	WEC0	8	R/W	<u>Write Enable Width</u> » 0000: 1 « » 0001: 2 « : » 1110: 15 « » 1111: 16 « The width is (WEC + 1) ARM clock cycles.
7:4	CEC0	9	R/W	<u>Chip Enable Width</u> » 0000: 1 « » 0001: 2 « : » 1110: 15 « » 1111: 16 « The width is (CEC + 1) ARM clock cycles.
3:0	CYCL0	10	R/W	<u>Cycle Width</u> » 0000: 2 « » 0001: 2 « : » 1110: 15 « » 1111: 16 « The cycle width is (CYCL + 1) ARM clock cycles, but for 0 it is 2.

20.7.2 CS0CTRL2

Control register2 of EMIF Region zero.

CS0CTRL2	0003:0A02			offset: 0x02												default: 0x1110	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	IDLE0[1]	IDLE0[0]	OESU0[3]	OESU0[2]	OESU0[1]	OESU0[0]	WESU0[3]	WESU0[2]	WESU0[1]	WESU0[0]	CESU0[3]	CESU0[2]	CESU0[1]	CESU0[0]	
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	-	-	0	1	0	0	0	1	0	0	0	1	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
13:12	IDLE0	1	R/W	<u>Number of IDLE State Clocks</u> » 00: 0 « » 01: 1 « » 10: 2 « » 11: 3 «
11:8	OESU0	1	R/W	<u>Number of Clocks of Output Enable Setup</u> » 0000: 0 « » 0001: 1 « : » 1110: 14 « » 1111: 15 «
7:4	WESU0	1	R/W	<u>Number Clocks of Write Enable Setup</u> » 0000: 0 « » 0001: 1 « : » 1110: 14 « » 1111: 15 «
3:0	CESU0	0	R/W	<u>Number of Clocks of Chip Enable Setup</u> » 0000: 0 « » 0001: 1 « : » 1110: 14 « » 1111: 15 «

20.7.3 CS0CTRL3

Control register3 of EMIF Region zero.

CS0CTRL3	0003:0A04			offset: 0x04												default: 0x0003		
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	MODE[1]	MODE[0]	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	PGCYCL[2]	PGCYCL[1]	PGCYCL[0]		
R/W	-	-	R/W	R/W	-	-	-	-	-	-	-	-	-	R/W	R/W	R/W		
Default	-	-	0	0	-	-	-	-	-	-	-	-	-	0	1	1		

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
13:12	MODE	0	R/W	<u>Access mode of CS0</u> » 0 : Normal « » 1 : Page 4 « » 2 : Page 8 « Note: Access mode must be Normal, when DM320 enters sleep mode
11:3	RSV			Reserved
2:0	PGCYCL	3	R/W	<u>Page cycle width</u> » 0 : 3 cycle « » 1 : 3 cycle « » 2 : 3 cycle « » 3 : 4 cycle « » 4 : 5 cycle « » 5 : 6 cycle « » 6 : 7 cycle « » 7 : 8 cycle « Note: Minimum width is 3.

20.7.4 CS1CTRL1A

Control register1A of EMIF Region One.

CS1CTRL1A			0003:0A06			offset: 0x06											default: 0x1415		
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	RSV	CEC1[4]	CEC1[3]	CEC1[2]	CEC1[1]	CEC1[0]	RSV	RSV	RSV	CYCL1[4]	CYCL1[3]	CYCL1[2]	CYCL1[1]	CYCL1[0]			
R/W	-	-	-	R/W	R/W	R/W	R/W	R/W	-	-	-	R/W	R/W	R/W	R/W	R/W			
Default	-	-	-	1	0	1	0	0	-	-	-	1	0	1	0	1			

Bit	Name	Reset Value	R/W	Function
15:13	RSV			
12:8	CEC1	20	R/W	<u>Chip Enable Width</u> » 00000: 1 « » 00001: 2 « : » 11110: 31 « » 11111: 32 « The width is (CEC1 + 1) ARM clock cycles.
7:5	RSV			Reserved
4:0	CYCL1	21	R/W	<u>Cycle Width</u> » 00000: 2 « » 00001: 2 « : » 11110: 31 « » 11111: 32 « The width is (CYCL1 + 1) ARM clock cycles, except when set value is 0000, the width is 2 ARM clock cycles.

20.7.5 CS1CTRL1B

Control register 1B of EMIF Region One

CS1CTRL1B	0003:0A08			offset: 0x08												default: 0x1109	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	OEC1[4]	OEC1[3]	OEC1[2]	OEC1[1]	OEC1[0]	RSV	RSV	RSV	WEC1[4]	WEC1[3]	WEC1[2]	WEC1[1]	WEC1[0]	
R/W	-	-	-	R/W	R/W	R/W	R/W	R/W	-	-	-	R/W	R/W	R/W	R/W	R/W	
Default	-	-	-	1	0	0	0	1	-	-	-	0	1	0	0	1	

Bit	Name	Reset Value	R/W	Function
15:13	RSV			
12:8	OEC1	17	R/W	Output Enable Width » 00000: 1 « » 00001: 2 « : » 11110: 31 « » 11111: 32 « The width is (set value + 1) ARM clock cycles.
7:5	RSV			Reserved
4:0	WEC1	9	R/W	Write Enable Width » 00000: 1 « » 00001: 2 « : » 11110: 31 « » 11111: 32 « The width is (set value + 1) ARM clock cycles.

20.7.6 CS1CTRL2

Control register2 of EMIF Region one.

CS1CTRL2	0003:0A0A			offset: 0x0A													default: 0x1220	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	RSV	RSV	IDLE1[1]	IDLE1[0]	OESU1[3]	OESU1[2]	OESU1[1]	OESU1[0]	WESU1[3]	WESU1[2]	WESU1[1]	WESU1[0]	CESU1[3]	CESU1[2]	CESU1[1]	CESU1[0]		
R/W	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W		
Default	-	-	0	1	0	0	1	0	0	0	1	0	0	0	0	0		

Bit	Name	Reset Value	R/W	Function
15:14	RSV			Reserved
13:12	IDLE1	1	R/W	<u>Number of IDLE State Clocks</u> » 00: 0 « » 01: 1 « » 10: 2 « » 11: 3 «
11:8	OESU1	2	R/W	<u>Number of Clocks of Output Enable Setup</u> » 0000: 0 « » 0001: 1 « : » 1110: 14 « » 1111: 15 «
7:4	WESU1	2	R/W	<u>Number Clocks of Write Enable Setup</u> » 0000: 0 « » 0001: 1 « : » 1110: 14 « » 1111: 15 «
3:0	CESU1	0	R/W	<u>Number of Clocks of Chip Enable Setup</u> » 0000: 0 « » 0001: 1 « : » 1110: 14 « » 1111: 15 «

20.7.7 CS2CTRL1

Control register1 of EMIF Region two.

CS2CTRL1	0003:0A0C			offset: 0x0C												default: 0x3207	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	OEC2[3]	OEC2[2]	OEC2[1]	OEC2[0]	WEC2[3]	WEC2[2]	WEC2[1]	WEC2[0]	RSV	RSV	RSV	RSV	CYCL2[3]	CYCL2[2]	CYCL2[1]	CYCL2[0]	
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	-	-	-	-	R/W	R/W	R/W	R/W	
Default	0	0	1	1	0	0	1	0	-	-	-	-	0	1	1	1	

Bit	Name	Reset Value	R/W	Function
-----	------	-------------	-----	----------

15:12	OEC2	3	R/W	<u>Output Enable Width</u> » 0000: 1 « » 0001: 2 « : » 1110: 15 « » 1111: 16 « The width is (OEC + 1) ARM clock cycles.
-------	------	---	-----	---

11:8	WEC2	2	R/W	<u>Write Enable Width</u> » 0000: 1 « » 0001: 2 « : » 1110: 15 « » 1111: 16 « The width is (WEC + 1) ARM clock cycles.
------	------	---	-----	--

7:4	RSV			<u>Reserved</u>
-----	-----	--	--	-----------------

3:0	CYCL2	7	R/W	<u>Cycle Width</u> » 0000: 2 « » 0001: 2 « » 0010: 3 « : » 1110: 15 « » 1111: 16 « The cycle width is (CYCL + 1) ARM clock cycles, except when set to 0, in which case the width is 2 ARM clock cycles.
-----	-------	---	-----	--

20.7.8 CS2CTRL2

Control register2 of EMIF Region two

CS2CTRL2	0003:0A0E				offset: 0x0E				default: 0x1129							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	BUSW2	IDLE2[1]	IDLE2[0]	OESU2[3]	OESU2[2]	OESU2[1]	OESU2[0]	WESU2[3]	WESU2[2]	WESU2[1]	WESU2[0]	RESU2[3]	RESU2[2]	RESU2[1]	RESU2[0]
R/W	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	0	0	1	0	0	0	1	0	0	1	0	1	0	0	1

Bit	Name	Reset Value	R/W	Function
15	RSV			Reserved
14	BUSW2	0	R/W	<u>Bus width</u> » 0 : 8 bit « » 1 : 16 bit «
13:12	IDLE2	1	R/W	<u>Number of IDLE State Clocks</u> » 00: 0 « » 01: 1 « » 10: 2 « » 11: 3 «
11:8	OESU2	1	R/W	<u>Number of Clocks of Output Enable Setup</u> » 0000: 0 « » 0001: 1 « : » 1110: 14 « » 1111: 15 «
7:4	WESU2	2	R/W	<u>Number Clocks of Write Enable Setup</u> » 0000: 0 « » 0001: 1 « : » 1110: 14 « » 1111: 15 «
3:0	RESU2	9	R/W	<u>Number of Clocks of Chip Enable Setup</u> » 0000: 3 « » 0001: 4 « » 0010: 5 « : » 1110: 17 « » 1111: 18 « The width is (RESU + 3) ARM clock cycles.

20.7.9 CS3CTRL1

Control register1 of EMIF Region three

CS3CTRL1	0003:0A10				offset: 0x10				default: 0x889A							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	OEC3[3]	OEC3[2]	OEC3[1]	OEC3[0]	WEC3[3]	WEC3[2]	WEC3[1]	WEC3[0]	CEC3[3]	CEC3[2]	CEC3[1]	CEC3[0]	CYCL3[3]	CYCL3[2]	CYCL3[1]	CYCL3[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	1	0	0	0	1	0	0	0	1	0	0	1	1	0	1	0

Bit	Name	Reset Value	R/W	Function
-----	------	-------------	-----	----------

15:12	OEC3	8	R/W	<u>Output Enable Width</u> » 0000: 1 « » 0001: 2 « : » 1110: 15 « » 1111: 16 « The width is (OEC + 1) ARM clock cycles.
-------	------	---	-----	---

11:8	WEC3	8	R/W	<u>Write Enable Width</u> » 0000: 1 « » 0001: 2 « : » 1110: 15 « » 1111: 16 « The width is (WEC + 1) ARM clock cycles.
------	------	---	-----	--

7:4	CEC3	9	R/W	<u>Chip Enable Width</u> » 0000: 1 « » 0001: 2 « : » 1110: 15 « » 1111: 16 « The width is (CEC + 1) ARM clock cycles.
-----	------	---	-----	---

3:0	CYCL3	10	R/W	<u>Cycle Width</u> » 0000: 2 « » 0001: 2 « : » 1110: 15 « » 1111: 16 « The cycle width is (CYCL + 1) ARM clock cycles, except for 0 when it is 2.
-----	-------	----	-----	---

20.7.10 CS3CTRL2

Control register2 of EMIF Region three

CS3CTRL2	0003:0A12			offset: 0x12			default: 0x1110									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	BUSW3	IDLE3[1]	IDLE3[0]	OESU3[3]	OESU3[2]	OESU3[1]	OESU3[0]	WESU3[3]	WESU3[2]	WESU3[1]	WESU3[0]	CESU3[3]	CESU3[2]	CESU3[1]	CESU3[0]
R/W	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	0	0	1	0	0	0	1	0	0	0	1	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15	RSV			Reserved
14	BUSW3	0	R/W	External Device Bus Width » 0: 8-bits wide « » 1: 16-bits wide «
13:12	IDLE3	1	R/W	Number of IDLE State Clocks » 00: 0 « » 01: 1 « » 10: 2 « » 11: 3 «
11:8	OESU3	1	R/W	Number of Clocks of Output Enable Setup » 0000: 0 « » 0001: 1 « : » 1110: 14 « » 1111: 15 «
7:4	WESU3	1	R/W	Number Clocks of Write Enable Setup » 0000: 0 « » 0001: 1 « : » 1110: 14 « » 1111: 15 «
3:0	CESU3	0	R/W	Number of Clocks of Chip Enable Setup » 0000: 0 « » 0001: 1 « : » 1110: 14 « » 1111: 15 «

20.7.11 CS4CTRL1

Control register1 of EMIF Region four

CS4CTRL1	0003:0A14				offset: 0x14				default: 0x889A							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	OEC4[3]	OEC4[2]	OEC4[1]	OEC4[0]	WEC4[3]	WEC4[2]	WEC4[1]	WEC4[0]	CEC4[3]	CEC4[2]	CEC4[1]	CEC4[0]	CYCL4[3]	CYCL4[2]	CYCL4[1]	CYCL4[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	1	0	0	0	1	0	0	0	1	0	0	1	1	0	1	0

Bit	Name	Reset Value	R/W	Function
-----	------	-------------	-----	----------

15:12	OEC4	8	R/W	<u>Output Enable Width</u> » 0000: 1 « » 0001: 2 « : » 1110: 15 « » 1111: 16 « The width is (OEC + 1) ARM clock cycles.
-------	------	---	-----	---

11:8	WEC4	8	R/W	<u>Write Enable Width</u> » 0000: 1 « » 0001: 2 « : » 1110: 15 « » 1111: 16 « The width is (WEC + 1) ARM clock cycles.
------	------	---	-----	--

7:4	CEC4	9	R/W	<u>Chip Enable Width</u> » 0000: 1 « » 0001: 2 « : » 1110: 15 « » 1111: 16 « The width is (CEC + 1) ARM clock cycles.
-----	------	---	-----	---

3:0	CYCL4	10	R/W	<u>Cycle Width</u> » 0000: 2 « » 0001: 2 « : » 1110: 15 « » 1111: 16 « The cycle width is (CYCL + 1) ARM clock cycles, except for 0 when it is 2.
-----	-------	----	-----	---

20.7.12 CS4CTRL2

Control register2 of EMIF Region four

CS4CTRL2	0003:0A16		offset: 0x16		default: 0x1110											
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	CESL4	BUSW4	IDLE4[1]	IDLE4[0]	OESU4[3]	OESU4[2]	OESU4[1]	OESU4[0]	WESU4[3]	WESU4[2]	WESU4[1]	WESU4[0]	CESU4[3]	CESU4[2]	CESU4[1]	CESU4[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	1	0	0	0	1	0	0	0	1	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15	CESL4	0	R/W	<u>Polarity of chip enable of memory area 4</u> » 0: Chip enable (CE1) is active LOW « » 1: Chip enable (CE1) is active HIGH «
14	BUSW4	0	R/W	External Device Bus Width » 0: 8-bits wide « » 1: 16-bits wide «
13:12	IDLE4	1	R/W	<u>Number of IDLE State Clocks</u> » 00: 0 « » 01: 1 « » 10: 2 « » 11: 3 «
11:8	OESU4	1	R/W	<u>Number of Clocks of Output Enable Setup</u> » 0000: 0 « » 0001: 1 « : » 1110: 14 « » 1111: 15 «
7:4	WESU4	1	R/W	<u>Number Clocks of Write Enable Setup</u> » 0000: 0 « » 0001: 1 « : » 1110: 14 « » 1111: 15 «
3:0	CESU4	0	R/W	<u>Number of Clocks of Chip Enable Setup</u> » 0000: 0 « » 0001: 1 « : » 1110: 14 « » 1111: 15 «

20.7.13 BUSCTRL

EMIF Bus control register.

BUSCTRL	0003:0A18			offset: 0x18			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	EMWE4	EMWE3
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W	R/W
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0	0

Bit	Name	Reset Value	R/W	Function
152	RSV			Reserved
1	EMWE4	0	R/W	<u>CS4 External Wait (EM WAIT) Enable</u> » 0: External wait when CS4 is accessed is invalid « » 1: External wait when CS4 is accessed is valid «
0	EMWE3	0	R/W	<u>CS4 External Wait (EM WAIT) Enable</u> » 0: External wait when CS3 is accessed is invalid « » 1: External wait when CS3 is accessed is valid «

20.7.14 BUSRLS

EMIF Bus Release Control Register

BUSRLS		0003:0A1A			offset: 0x1A												default: 0x0000		
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	BUSSEL	SDREL	EMREL			
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W	R/W	R/W			
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	0	0	0			

Bit	Name	Reset Value	R/W	Function
15:3	RSV			Reserved
2	BUSSEL	0	R/W	<u>SDRAM bus sharing mode</u> » 0: Normal Mode « » 1: SDRAM bus is shared for use (CS0-CS4) «
1	SDREL	0	R/W	<u>SDRAM Bus Open Control</u> » 0: SDRAM bus is not opened « Bus open is cleared by writing '0' » 1: SDRAM bus is opened «
0	EMREL	0	R/W	<u>External Memory Bus Open Control</u> » 0: External memory bus is not opened « Bus open is cleared when "0" is written » 1: External memory bus is opened «

20.7.15 CFCTRL1

EMIF CFC Control Register #1

CFCTRL1	0003:0A1C				offset:	0x1C				default: 0x0000						
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	CFSEL
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0

Bit	Name	Reset Value	R/W	Function
15:1	RSV			Reserved
0	CFSEL	0	R/W	Select CF card interface or Smart Media Card interface » 0: CF « » 1: Smart Media «

20.7.16 CFCTRL2

EMIF CFC Control Register #2

CFCTRL2	0003:0A1E			offset: 0x1E												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	BUSSZ	RSV	RSV	RSV	CFMOD	
R/W	-	-	-	-	-	-	-	-	-	-	-	R/W	-	-	-	R/W	
Default	-	-	-	-	-	-	-	-	-	-	-	0	-	-	-	0	

Bit	Name	Reset Value	R/W	Function
15:5	RSV			Reserved
4	BUSSZ	0	R/W	<u>CFC Dynamic Bus Sizing Enable</u> » 0: Use 16-bit wide bus to access CFC « » 1: Use dynamic bus sizing when IO space of CF card is accessed « The IOIS16 pin is used for dynamic bus sizing.
3:1	RSV			Reserved
0	CFMOD	0	R/W	<u>CFC Memory Mode REG Signal Control</u> » 0: Output low REG when accessing CFC attribute memory « » 1: Output high REG when accessing CFC common memory « When the CF card memory space is accessed (in memory mode), the REG signal controls which region is accessed. When REG is low, the attribute memory region can be accessed. When REG is high, the common memory region can be accessed. Note: When the I/O space is accessed, the REG signal is always low regardless of the CFMOD bit setting. The REG signal is output on pin ARM_A22.

20.7.17 SMCTRL

EMIF SmartMedia Control Register

SMCTRL	0003:0A20				offset: 0x20				default: 0x0001							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	SCEH
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	1

Bit	Name	Reset Value	R/W	Function
15:1	RSV			Reserved
0	SCEH	1	R/W	<u>SmartMedia Chip Enable Pin (CF CE1)</u> » 0: CF_CE1 pin is set low « » 1: CF_CE1 pin is set high «

20.7.18 BUSINTEN

EMIF Bus Interrupt Enable Register

BUSINTEN	0003:0A22			offset: 0x22		default: 0x0000										
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	CBCND	CBIEN
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W	R/W
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0	0

Bit	Name	Reset Value	R/W	Function
15:2	RSV			Reserved
1	CBCND	0	R/W	<u>CFC Busy Interrupt Condition</u> » 0: Interrupt generated when CFRDY pin changes '0'-'>'1' « » 1: Interrupt generated when CFRDY pin changes '1'-'>'0' «
0	CBIEN	0	R/W	<u>CFC Busy Interrupt Enable</u> » 0: Disable « » 1: Enable « MTC1 interrupt is generated when the CFRDY pin changes as specified by CBCND

20.7.19 BUSSTS

EMIF Bus Status Register

BUSSTS	0003:0A24			offset: 0x24												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	EM_WAIT	IOIS16	CFWAIT	CFRDY	
R/W	-	-	-	-	-	-	-	-	-	-	-	-	R	R	R	R	
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	

Bit	Name	Reset Value	R/W	Function
15:4	RSV			Reserved
3	EM_WAIT		R	EM_WAIT pin status
2	IOIS16		R	IOIS16 pin status
1	CFWAIT		R	CFWAIT pin status
0	CFRDY		R	CFRDY pin status

20.7.20 BUSWAITMD

EMIF Bus Wait Mode Register

BUSWAITMD			0003:0A26			offset: 0x26											default: 0x0000		
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	CFATTW	CFRSL	CFRW			
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W	R/W	R/W			
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	0	0	0			

Bit	Name	Reset Value	R/W	Function
15:3	RSV			Reserved
2	CFATTW	0	R/W	<u>CF Attribute wait Enable</u> » 0: Ignore CFWAIT when accessing CF attribute memory « » 1: When CFWAIT is 0, hold CF attribute memory access« Note: The controller always waits when I/O or common memory is accessed when CFWAIT is low
1	CFRSL	0	R/W	<u>CF Ready Select</u> (Specify the kind of card connected to the CFRDY terminal) » 0: Connect CF card RDY/BSYn to CFRDY terminal « » 1: Connect SmartMedia R/Bn to CFRDY terminal «
0	CFRW	0	R/W	<u>CFRDY Wait</u> Defines whether or not to control the start of bus cycle by CFRDY terminal when the card specified in CFRSL (above) is accessed » 0: Access the card regardless of CFRDY terminal state « » 1: Do not access card while The CFRDY terminal is low «

20.7.21 ECC1CP

EMIF ECC Area 1 CP Register

ECC1CP	0003:0A28				offset: 0x28				default: 0x00FF							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	1CP[5]	1CP[4]	1CP[3]	1CP[2]	1CP[1]	1CP[0]	1RD1[1]	1RD1[0]
R/W	-	-	-	-	-	-	-	-	R	R	R	R	R	R	R	R
Default	-	-	-	-	-	-	-	-	1	1	1	1	1	1	1	1

Bit	Name	Reset Value	R/W	Function
15:8	RSV			Reserved
7:2	1CP	63	R	SSFDC Area1 ECC CP The result of the ECC calculation of Storing Region-1 of SmartMedia can be read (0-255 bytes).
1:0	1RD1	3	R	Fixed at "11"

20.7.22 ECC1LP

EMIF ECC Area 1 LP Register

ECC1LP	0003:0A2A			offset: 0x2A												default: 0xFFFF	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	1LP[15]	1LP[14]	1LP[13]	1LP[12]	1LP[11]	1LP[10]	1LP[9]	1LP[8]	1LP[7]	1LP[6]	1LP[5]	1LP[4]	1LP[3]	1LP[2]	1LP[1]	1LP[0]	
R/W	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	
Default	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	

Bit	Name	Reset Value	R/W	Function
15:0	1LP	65535	R	SSFDC Area1 ECC LP The result of the ECC calculation of Storing Region-1 of SmartMedia can be read (0-255 bytes).

20.7.23 ECC2CP

EMIF ECC Area 2 CP Register

ECC2CP	0003:0A2C				offset: 0x2C				default: 0x00FF							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	2CP[5]	2CP[4]	2CP[3]	2CP[2]	2CP[1]	2CP[0]	2RD1[1]	2RD1[0]
R/W	-	-	-	-	-	-	-	-	R	R	R	R	R	R	R	R
Default	-	-	-	-	-	-	-	-	1	1	1	1	1	1	1	1

Bit	Name	Reset Value	R/W	Function
15:8	RSV			Reserved
7:2	2CP	63	R	SSFDC Area2 ECC CP The result of the ECC calculation of Storing Region-2 of SmartMedia can be read (256-511 bytes).
1:0	2RD1	3	R	Fixed at "11"

20.7.24 ECC2LP

EMIF ECC Area 2 LP Register

ECC2LP	0003:0A2E			offset: 0x2E												default: 0xFFFF	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	2LP[15]	2LP[14]	2LP[13]	2LP[12]	2LP[11]	2LP[10]	2LP[9]	2LP[8]	2LP[7]	2LP[6]	2LP[5]	2LP[4]	2LP[3]	2LP[2]	2LP[1]	2LP[0]	
R/W	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	
Default	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	

Bit	Name	Reset Value	R/W	Function
15:0	2LP	65535	R	SSFDC Area2 ECC LP The result of the ECC calculation of Storing Region-2 of SmartMedia can be read (256-511 bytes).

20.7.25 ECC3CP

EMIF ECC Area 3 CP Register

ECC3CP	0003:0A30				offset: 0x30				default: 0x00FF							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	3CP[5]	3CP[4]	3CP[3]	3CP[2]	3CP[1]	3CP[0]	3RD1[1]	3RD1[0]
R/W	-	-	-	-	-	-	-	-	R	R	R	R	R	R	R	R
Default	-	-	-	-	-	-	-	-	1	1	1	1	1	1	1	1

Bit	Name	Reset Value	R/W	Function
15:8	RSV			Reserved
7:2	3CP	63	R	SSFDC Area3 ECC CP The result of the ECC calculation of Storing Region-3 of SmartMedia can be read (512-767 bytes).
1:0	3RD1	3	R	Fixed at "11"

20.7.26 ECC3LP

EMIF ECC Area 3 LP Register

ECC3LP	0003:0A32			offset: 0x32												default: 0xFFFF	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	3LP[15]	3LP[14]	3LP[13]	3LP[12]	3LP[11]	3LP[10]	3LP[9]	3LP[8]	3LP[7]	3LP[6]	3LP[5]	3LP[4]	3LP[3]	3LP[2]	3LP[1]	3LP[0]	
R/W	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	
Default	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	

Bit	Name	Reset Value	R/W	Function
15:0	3LP	65535	R	SSFDC Area3 ECC LP The result of the ECC calculation of Storing Region-3 of SmartMedia can be read (512-767 bytes).

20.7.27 ECC4CP

EMIF ECC Area 4 CP Register

ECC4CP	0003:0A34				offset: 0x34				default: 0x00FF							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	4CP[5]	4CP[4]	4CP[3]	4CP[2]	4CP[1]	4CP[0]	4RD1[1]	4RD1[0]
R/W	-	-	-	-	-	-	-	-	R	R	R	R	R	R	R	R
Default	-	-	-	-	-	-	-	-	1	1	1	1	1	1	1	1

Bit	Name	Reset Value	R/W	Function
15:8	RSV			Reserved
7:2	4CP	63	R	SSFDC Area4 ECC CP The result of the ECC calculation of Storing Region-4 of SmartMedia can be read (768-1023 bytes).
1:0	4RD1	3	R	Fixed at "11"

20.7.28 ECC4LP

EMIF ECC Area 4 LP Register

ECC4LP	0003:0A36			offset: 0x36												default: 0xFFFF	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	4LP[15]	4LP[14]	4LP[13]	4LP[12]	4LP[11]	4LP[10]	4LP[9]	4LP[8]	4LP[7]	4LP[6]	4LP[5]	4LP[4]	4LP[3]	4LP[2]	4LP[1]	4LP[0]	
R/W	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	
Default	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	

Bit	Name	Reset Value	R/W	Function
15:0	4LP	65535	R	SSFDC Area4 ECC LP The result of the ECC calculation of Storing Region-4 of SmartMedia can be read (768-1023 bytes).

20.7.29 ECC5CP

EMIF ECC Area 5 CP Register

ECC5CP	0003:0A38				offset: 0x38				default: 0x00FF							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	5CP[5]	5CP[4]	5CP[3]	5CP[2]	5CP[1]	5CP[0]	5RD1[1]	5RD1[0]
R/W	-	-	-	-	-	-	-	-	R	R	R	R	R	R	R	R
Default	-	-	-	-	-	-	-	-	1	1	1	1	1	1	1	1

Bit	Name	Reset Value	R/W	Function
15:8	RSV			Reserved
7:2	5CP	63	R	SSFDC Area5 ECC CP The result of the ECC calculation of Storing Region-5 of SmartMedia can be read (1024-1279 bytes).
1:0	5RD1	3	R	Fixed at "11"

20.7.30 ECC5LP

EMIF ECC Area 5 LP Register

ECC5LP	0003:0A3A			offset: 0x3A												default: 0xFFFF	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	5LP[15]	5LP[14]	5LP[13]	5LP[12]	5LP[11]	5LP[10]	5LP[9]	5LP[8]	5LP[7]	5LP[6]	5LP[5]	5LP[4]	5LP[3]	5LP[2]	5LP[1]	5LP[0]	
R/W	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	
Default	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	

Bit	Name	Reset Value	R/W	Function
15:0	5LP	65535	R	SSFDC Area5 ECC LP The result of the ECC calculation of Storing Region-5 of SmartMedia can be read (1024-1279 bytes).

20.7.31 ECC6CP

EMIF ECC Area 6 CP Register

ECC6CP	0003:0A3C				offset: 0x3C				default: 0x00FF							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	6CP[5]	6CP[4]	6CP[3]	6CP[2]	6CP[1]	6CP[0]	6RD1[1]	6RD1[0]
R/W	-	-	-	-	-	-	-	-	R	R	R	R	R	R	R	R
Default	-	-	-	-	-	-	-	-	1	1	1	1	1	1	1	1

Bit	Name	Reset Value	R/W	Function
15:8	RSV			Reserved
7:2	6CP	63	R	SSFDC Area6 ECC CP The result of the ECC calculation of Storing Region-6 of SmartMedia can be read (1280-1535 bytes).
1:0	6RD1	3	R	Fixed at "11"

20.7.32 ECC6LP

EMIF ECC Area 6 LP Register

ECC6LP	0003:0A3E			offset: 0x3E												default: 0xFFFF	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	6LP[15]	6LP[14]	6LP[13]	6LP[12]	6LP[11]	6LP[10]	6LP[9]	6LP[8]	6LP[7]	6LP[6]	6LP[5]	6LP[4]	6LP[3]	6LP[2]	6LP[1]	6LP[0]	
R/W	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	
Default	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	

Bit	Name	Reset Value	R/W	Function
15:0	6LP	65535	R	SSFDC Area6 ECC LP The result of the ECC calculation of Storing Region-6 of SmartMedia can be read (1280-1535 bytes).

20.7.33 ECC7CP

EMIF ECC Area 7 CP Register

ECC7CP	0003:0A40				offset: 0x40				default: 0x00FF							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	7CP[5]	7CP[4]	7CP[3]	7CP[2]	7CP[1]	7CP[0]	7RD1[1]	7RD1[0]
R/W	-	-	-	-	-	-	-	-	R	R	R	R	R	R	R	R
Default	-	-	-	-	-	-	-	-	1	1	1	1	1	1	1	1

Bit	Name	Reset Value	R/W	Function
15:8	RSV			Reserved
7:2	7CP	63	R	SSFDC Area7 ECC CP The result of the ECC calculation of Storing Region-7 of SmartMedia can be read (1536-1791 bytes).
1:0	7RD1	3	R	Fixed at "11"

20.7.34 ECC7LP

EMIF ECC Area 7 LP Register

ECC7LP	0003:0A42			offset: 0x42												default: 0xFFFF	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	7LP[15]	7LP[14]	7LP[13]	7LP[12]	7LP[11]	7LP[10]	7LP[9]	7LP[8]	7LP[7]	7LP[6]	7LP[5]	7LP[4]	7LP[3]	7LP[2]	7LP[1]	7LP[0]	
R/W	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	
Default	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	

Bit	Name	Reset Value	R/W	Function
15:0	7LP	65535	R	SSFDC Area7 ECC LP The result of the ECC calculation of Storing Region-7 of SmartMedia can be read (1536-1791 bytes).

20.7.35 ECC8CP

EMIF ECC Area 8 CP Register

ECC8CP	0003:0A44			offset: 0x44			default: 0x00FF									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	8CP[5]	8CP[4]	8CP[3]	8CP[2]	8CP[1]	8CP[0]	8RD1[1]	8RD1[0]
R/W	-	-	-	-	-	-	-	-	R	R	R	R	R	R	R	R
Default	-	-	-	-	-	-	-	-	1	1	1	1	1	1	1	1

Bit	Name	Reset Value	R/W	Function
15:8	RSV			Reserved
7:2	8CP	63	R	SSFDC Area8 ECC CP The result of the ECC calculation of Storing Region-8 of SmartMedia can be read (1792-2047 bytes).
1:0	8RD1	3	R	Fixed at "11"

20.7.36 ECC8LP

EMIF ECC Area 8 LP Register

ECC8LP	0003:0A46			offset: 0x46			default: 0xFFFF									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	8LP[15]	8LP[14]	8LP[13]	8LP[12]	8LP[11]	8LP[10]	8LP[9]	8LP[8]	8LP[7]	8LP[6]	8LP[5]	8LP[4]	8LP[3]	8LP[2]	8LP[1]	8LP[0]
R/W	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R
Default	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

Bit	Name	Reset Value	R/W	Function
15:0	8LP	65535	R	SSFDC Area8 ECC LP The result of the ECC calculation of Storing Region-8 of SmartMedia can be read (1792-2047 bytes).

20.7.37 ECCCLR

EMIF ECC Clear Register

ECCCLR	0003:0A48			offset: 0x48												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	ECLR	
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W	
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0	

Bit	Name	Reset Value	R/W	Function
15:1	RSV			Reserved
0	ECLR	0	R/W	ECC is cleared when 1 is written to this bit This bit is automatically cleared after writing 1

20.7.38 PAGESZ

EMIF SmartMedia Page Size Register

PAGESZ	0003:0A4A			offset: 0x4A		default: 0x0001										
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	PAGESZ[1]	PAGESZ[0]
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W	R/W
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0	1

Bit	Name	Reset Value	R/W	Function
15:2	RSV			Reserved
1:0	PAGESZ	1	R/W	SmartMedia Page Size » 0: 256+8bytes per page « » 1: 512+16bytes per page or 256+8 words / page « » 2: 1024+32bytes per page or 512+16 words / page« » 3: 2048+64bytes per page or 1024+32 words / page «

20.7.39 PRIORCTL

EMIF Priority Control Register

PRIORCTL	0003:0A4C				offset: 0x4C				default: 0x001B							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	PRY1[1]	PRY1[0]	PRY2[1]	PRY2[0]	PRY3[1]	PRY3[0]	PRY4[1]	PRY4[0]
R/W	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	-	-	-	0	0	0	1	1	0	1	1

Bit	Name	Reset Value	R/W	Function
15:8	RSV			Reserved
7:6	PRY1	0	R/W	Priority of access » 0: ARM « » 1: DMA « » 2: Image Buffer « » 3: DSP (HPIB) «
5:4	PRY2	1		Priority of access » 0: ARM « » 1: DMA « » 2: Image Buffer « » 3: DSP (HPIB) «
3:2	PRY3	2		Priority of access » 0: ARM « » 1: DMA « » 2: Image Buffer « » 3: DSP (HPIB) «
1:0	PRY4	3		Priority of access » 0: ARM « » 1: DMA « » 2: Image Buffer « » 3: DSP (HPIB) «

20.7.40 IMGDSPDEST

EMIF Image Buffer/DSP (HPIB) Device Select Register

IMGDSPDEST			0003:0A4E			offset: 0x4E			default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	IDST[2]	IDST[1]	IDST[0]	RSV	DDST[2]	DDST[1]	DDST[0]
R/W	-	-	-	-	-	-	-	-	-	R/W	R/W	R/W	-	R/W	R/W	R/W
Default	-	-	-	-	-	-	-	-	-	0	0	0	-	0	0	0

Bit	Name	Reset Value	R/W	Function
15:7	RSV			Reserved
6:4	DST	0	R/W	<u>Image Buffer Data Transfer Destination</u> » 000: CS0 « » 001: CS1 « » 010: CS2 « » 011: CS3 « » 100: CS4 «
3	RSV			Reserved
2:0	DDST	0	R/W	<u>DSP (HPIB) Data Transfer Destination</u> » 000: CS0 « » 001: CS1 « » 010: CS2 « » 011: CS3 « » 100: CS4 «

20.7.41 IMGDSPADDH

EMIF Image Buffer/ DSP (HPIB) Destination Address High

IMGDSPADDH	0003:0A50				offset: 0x50				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	DSPAH[6]	DSPAH[5]	DSPAH[4]	DSPAH[3]	DSPAH[2]	DSPAH[1]	DSPAH[0]
R/W	-	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	-	-	-	-	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:7	RSV			Reserved
6:0	DSPAH	0	R/W	Upper 7 bits address of byte address of external memory when data of image buffer or DSP (HPIB) and external memory CS1 or CS2 is described. Specify the relative address from the header address of each bank. During data transfer between Image buffer or DSP (HPIB) and CS1 or CS2, the CS1 or CS2 address specified by IMGDSPADD1/2 acts as a fixed address, i.e., address is not incremented after transfer of each unit of data.

20.7.42 IMGDSPADDL

EMIF Image Buffer/ DSP (HPIB) Destination Address Low

IMGDSPADDL	0003:0A52				offset: 0x52				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	DSPAL[15]	DSPAL[14]	DSPAL[13]	DSPAL[12]	DSPAL[11]	DSPAL[10]	DSPAL[9]	DSPAL[8]	DSPAL[7]	DSPAL[6]	DSPAL[5]	DSPAL[4]	DSPAL[3]	DSPAL[2]	DSPAL[1]	DSPAL[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:0	DSPAL	0	R/W	Lower 16 bits address of byte address of external memory when data of image buffer or DSP (HPIB) and external memory CS1 or CS2 is described. Specify the relative address from the header address of each bank. When it is transferred, it becomes a fixed address.

20.7.43 AHBADDH

AHB address - high

AHBADDH	0003:0A54			offset: 0x54			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	AHBFIX	AHBAH[14]	AHBAH[13]	AHBAH[12]	AHBAH[11]	AHBAH[10]	AHBAH[9]	AHBAH[8]	AHBAH[7]	AHBAH[6]	AHBAH[5]	AHBAH[4]	AHBAH[3]	AHBAH[2]	AHBAH[1]	AHBAH[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15	AHBFIX	0	R/W	AHB fixed / Increment address 0 : Address incremented at each transfer 1 : Address is fixed for each transfer
14:0	AHBAH	0	R/W	DMA transfer AHB address - upper 15-bits Specify absolute address as in ARM memory map

20.7.44 AHBADDL

AHB address - low

AHBADDL	0003:0A56				offset: 0x56												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	AHBAL[15]	AHBAL[14]	AHBAL[13]	AHBAL[12]	AHBAL[11]	AHBAL[10]	AHBAL[9]	AHBAL[8]	AHBAL[7]	AHBAL[6]	AHBAL[5]	AHBAL[4]	AHBAL[3]	AHBAL[2]	AHBAL[1]	AHBAL[0]		
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W		
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		

Bit	Name	Reset Value	R/W	Function
15:0	AHBAL	0	R/W	DMA transfer AHB address - lower 16-bits Note: Address specified must be a multiple of 4 to align with a 32-bit address. The 2 LSBs are ignored even if non-zero values are written.

20.7.45 MTCADDH

MTC (EMIF/SDRAM) address - high

MTCADDH	0003:0A58			offset:	0x58										default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	MTCFIX	RSV	RSV	RSV	MTCAH[11]	MTCAH[10]	MTCAH[9]	MTCAH[8]	MTCAH[7]	MTCAH[6]	MTCAH[5]	MTCAH[4]	MTCAH[3]	MTCAH[2]	MTCAH[1]	MTCAH[0]
R/W	R/W	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	-	-	-	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15	MTCFIX	0	R/W	Fix DMA MTC Address » 0: Auto-increment address after each transfer « » 1: The address is fixed «
14:12	RSV			<u>Reserved</u>
11:0	MTCAH	0	R/W	DMA transfer MTC address - upper 12-bits Note : address is specified as offset from start of SDRAM/EMIF region

MTC (EMIF/SDRAM) address - low

Bit	Name	Reset Value	R/W	Function
-----	------	-------------	-----	----------

150	MTCAL	0	R/W	DMA transfer destination byte address - lower 16-bits Address specified must be a multiple of 4 to align with a 32-bit address. The 2 LSBs are ignored even if non-zero values are written. If 8-burst mode is selected ADDR[4:0] should be zero and if 4-burst mode is selected ADDR[3:0] should be zero. Note : address is specified as offset from start of SDRAM/EMIF region
-----	-------	---	-----	---

20.7.47 DMASIZE

EMIF DMA Transfer Size Register

DMASIZE	0003:0A5C			offset: 0x5C			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	TXW[15]	TXW[14]	TXW[13]	TXW[12]	TXW[11]	TXW[10]	TXW[9]	TXW[8]	TXW[7]	TXW[6]	TXW[5]	TXW[4]	TXW[3]	TXW[2]	TXW[1]	TXW[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:0	TXW	0	R/W	DMA transfer size in Words Transfer size must always be specified in a multiple of 4. Note: The amount of data trasfered will be TXW * 4

20.7.48 DMAMTCSEL

DMA device select register

DMAMTCSEL			0003:0A5E			offset: 0x5E									default: 0x0000		
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	MTCSEL[2]	MTCSEL[1]	MTCSEL[0]	
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	R/W	R/W	R/W	
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:3	RSV			Reserved
2:0	MTCSEL	0	R/W	Select MTC's DMA Transfer Device. » 000: CS0 « » 001: CS1 « » 010: CS2 « » 011: CS3 « » 100: CS4 « » 101: SDRAM «

20.7.49 DMACTL

EMIF DMA Control register

DMACTL	0003:0A60			offset: 0x60												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	ENDI[1]	ENDI[0]	RSV	RSV	RSV	RSV	RSV	RSV	DIR	GO	
R/W	-	-	-	-	-	-	R/W	R/W	-	-	-	-	-	-	R/W	R/W	
Default	-	-	-	-	-	-	0	0	-	-	-	-	-	-	0	0	

Bit	Name	Reset Value	R/W	Function
15:10	RSV			Reserved
9:8	ENDI	0	R/W	<u>Endian Switching Control</u> » 00: D3 D2 D1 D0 -> D3 D2 D1 D0 « » 01: D3 D2 D1 D0 -> D2 D3 D0 D1 « » 10: D3 D2 D1 D0 -> D0 D1 D2 D3 « » 11: D3 D2 D1 D0 -> D1 D0 D3 D2 « Note: Endian control is only applicable for when the DMA source is an MTC device
7:2	RSV			Reserved
1	DR	0	R/W	<u>DMA Transfer direction</u> » 0 : AHB -> MTC « » 1 : MTC -> AHB «
0	GO	0	R/W	DMA Transfer Start Bit » 0 : DMA Done « » 1 : DMA Start / DMA on-going « When "1" is set, DMA transfer is started. Automatically cleared to "0" after transfer is finished Note: After the transfer MTC0 interrupt will be generated

20.8 SDRAM Register Map

Address	Register	Description
0003:0980	SDBUFD0L	SDRAM Transfer Data Buffer 0H
0003:0982	SDBUFD0H	SDRAM Transfer Data Buffer 0L
0003:0984	SDBUFD1L	SDRAM Transfer Data Buffer 1H
0003:0986	SDBUFD1H	SDRAM Transfer Data Buffer 1L
0003:0988	SDBUFD2L	SDRAM Transfer Data Buffer 2H
0003:098A	SDBUFD2H	SDRAM Transfer Data Buffer 2L
0003:098C	SDBUFD3L	SDRAM Transfer Data Buffer 3H
0003:098E	SDBUFD3H	SDRAM Transfer Data Buffer 3L
0003:0990	SDBUFD4L	SDRAM Transfer Data Buffer 4H
0003:0992	SDBUFD4H	SDRAM Transfer Data Buffer 4L
0003:0994	SDBUFD5L	SDRAM Transfer Data Buffer 5H
0003:0996	SDBUFD5H	SDRAM Transfer Data Buffer 5L
0003:0998	SDBUFD6L	SDRAM Transfer Data Buffer 6H
0003:099A	SDBUFD6H	SDRAM Transfer Data Buffer 6L
0003:099C	SDBUFD7L	SDRAM Transfer Data Buffer 7H
0003:099E	SDBUFD7H	SDRAM Transfer Data Buffer 7L
0003:09A0	SDBUFAD1	SDRAM Transfer Address #1 - Low
0003:09A2	SDBUFAD2	SDRAM Transfer Address #2 - High
0003:09A4	SDBUFCTL	SDRAM Transfer Control
0003:09A6	SDMODE	SDRAM Mode Register
0003:09A8	REFCTL	Refresh Control Register
0003:09AA	SDPRTY1	SDRAM Priority Select #1 (CCD Controller)
0003:09AC	SDPRTY2	SDRAM Priority Select #2 (Preview Engine)
0003:09AE	SDPRTY3	SDRAM Priority Select #3 (Hardware 3A)
0003:09B0	SDPRTY4	SDRAM Priority Select #4 (On Screen Display)
0003:09B2	SDPRTY5	SDRAM Priority Select #5 (ARM CPU)
0003:09B4	SDPRTY6	SDRAM Priority Select #6 (MTC, MS, MMCSD #3)
0003:09B6	SDPRTY7	SDRAM Priority Select #7 (MTC, MS, MMCSD #2)
0003:09B8	SDPRTY8	SDRAM Priority Select #8 (MTC, MS, MMCSD #1)
0003:09BA	SDPRTY9	SDRAM Priority Select #9 (ARM FIFO)
0003:09BC	SDPRTY10	SDRAM Priority Select #10 (SDRAM Refresh)
0003:09BE	SDPRTY11	SDRAM Priority Select #11 (Image Buffer DSP)
0003:09C0	SDPRTY12	SDRAM Priority Select #12 (VLC)
0003:09C2	RSV	Reserved
0003:09C4	SDPRTYON	SDRAM Priority On Register
0003:09C6	SDDMASEL	SDRAM DMA select

20.9 SDRAM Controller Registers

20.9.1 SDBUFD0L

SDRAM Transfer Data Buffer 0 Low

SDBUFD0L	0003:0980			offset: 0x00												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	SD0L[15]	SD0L[14]	SD0L[13]	SD0L[12]	SD0L[11]	SD0L[10]	SD0L[9]	SD0L[8]	SD0L[7]	SD0L[6]	SD0L[5]	SD0L[4]	SD0L[3]	SD0L[2]	SD0L[1]	SD0L[0]	
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:0	SD0L	0	R/W	SDRAM data register 0L

20.9.2 SDBUFD0H

SDRAM Transfer Data Buffer 0 High

SDBUFD0H	0003:0982			offset: 0x02												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	SD0H[15]	SD0H[14]	SD0H[13]	SD0H[12]	SD0H[11]	SD0H[10]	SD0H[9]	SD0H[8]	SD0H[7]	SD0H[6]	SD0H[5]	SD0H[4]	SD0H[3]	SD0H[2]	SD0H[1]	SD0H[0]	
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:0	SD0H	0	R/W	SDRAM data register 0H

20.9.3 SDBUFD1L

SDRAM Transfer Data Buffer 1 Low

SDBUFD1L	0003:0984			offset: 0x04													default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	SD1L[15]	SD1L[14]	SD1L[13]	SD1L[12]	SD1L[11]	SD1L[10]	SD1L[9]	SD1L[8]	SD1L[7]	SD1L[6]	SD1L[5]	SD1L[4]	SD1L[3]	SD1L[2]	SD1L[1]	SD1L[0]		
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W		
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		

Bit	Name	Reset Value	R/W	Function
15:0	SD1L	0	R/W	SDRAM data register 1L

20.9.4 SDBUFD1H

SDRAM Transfer Data Buffer 1 High

SDBUFD1H	0003:0986			offset: 0x06			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	SD1H[15]	SD1H[14]	SD1H[13]	SD1H[12]	SD1H[11]	SD1H[10]	SD1H[9]	SD1H[8]	SD1H[7]	SD1H[6]	SD1H[5]	SD1H[4]	SD1H[3]	SD1H[2]	SD1H[1]	SD1H[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:0	SD1H	0	R/W	SDRAM data register 1H

20.9.5 SDBUFD2L

SDRAM Transfer Data Buffer 2 Low

SDBUFD2L	0003:0988			offset: 0x08													default: 0x0000		
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	SD2L[15]	SD2L[14]	SD2L[13]	SD2L[12]	SD2L[11]	SD2L[10]	SD2L[9]	SD2L[8]	SD2L[7]	SD2L[6]	SD2L[5]	SD2L[4]	SD2L[3]	SD2L[2]	SD2L[1]	SD2L[0]			
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W			
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			

Bit	Name	Reset Value	R/W	Function
15:0	SD2L	0	R/W	SDRAM data register 2L

20.9.6 SDBUFD2H

SDRAM Transfer Data Buffer 2 High

SDBUFD2H	0003:098A			offset: 0x0A			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	SD2H[15]	SD2H[14]	SD2H[13]	SD2H[12]	SD2H[11]	SD2H[10]	SD2H[9]	SD2H[8]	SD2H[7]	SD2H[6]	SD2H[5]	SD2H[4]	SD2H[3]	SD2H[2]	SD2H[1]	SD2H[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:0	SD2H	0	R/W	SDRAM data register 2H

20.9.7 SDBUFD3L

SDRAM Transfer Data Buffer 3 Low

SDBUFD3L	0003:098C			offset: 0x0C													default: 0x0000		
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	SD3L[15]	SD3L[14]	SD3L[13]	SD3L[12]	SD3L[11]	SD3L[10]	SD3L[9]	SD3L[8]	SD3L[7]	SD3L[6]	SD3L[5]	SD3L[4]	SD3L[3]	SD3L[2]	SD3L[1]	SD3L[0]			
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W			
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			

Bit	Name	Reset Value	R/W	Function
15:0	SD3L	0	R/W	SDRAM data register 3L

20.9.8 SDBUFD3H

SDRAM Transfer Data Buffer 3 High

SDBUFD3H	0003:098E			offset: 0x0E												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	SD3H[15]	SD3H[14]	SD3H[13]	SD3H[12]	SD3H[11]	SD3H[10]	SD3H[9]	SD3H[8]	SD3H[7]	SD3H[6]	SD3H[5]	SD3H[4]	SD3H[3]	SD3H[2]	SD3H[1]	SD3H[0]	
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:0	SD3H	0	R/W	SDRAM data register 3H

20.9.9 SDBUFD4L

- SDRAM Transfer Data Buffer 4 Low

SDBUFD4L	0003:0990			offset: 0x10												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	SD4L[15]	SD4L[14]	SD4L[13]	SD4L[12]	SD4L[11]	SD4L[10]	SD4L[9]	SD4L[8]	SD4L[7]	SD4L[6]	SD4L[5]	SD4L[4]	SD4L[3]	SD4L[2]	SD4L[1]	SD4L[0]	
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:0	SD4L	0	R/W	SDRAM data register 4L

20.9.10 SDBUFD4H

- SDRAM Transfer Data Buffer 4 High

SDBUFD4H	0003:0992			offset: 0x12												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	SD4H[15]	SD4H[14]	SD4H[13]	SD4H[12]	SD4H[11]	SD4H[10]	SD4H[9]	SD4H[8]	SD4H[7]	SD4H[6]	SD4H[5]	SD4H[4]	SD4H[3]	SD4H[2]	SD4H[1]	SD4H[0]	
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:0	SD4H	0	R/W	SDRAM data register 4H

20.9.11 SDBUFD5L

- SDRAM Transfer Data Buffer 5 Low

SDBUFD5L	0003:0994			offset: 0x14												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	SD5L[15]	SD5L[14]	SD5L[13]	SD5L[12]	SD5L[11]	SD5L[10]	SD5L[9]	SD5L[8]	SD5L[7]	SD5L[6]	SD5L[5]	SD5L[4]	SD5L[3]	SD5L[2]	SD5L[1]	SD5L[0]	
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
150	SD5L	0	R/W	SDRAM data register 5L

20.9.12 SDBUFD5H

SDRAM Transfer Data Buffer 5 High

SDBUFD5H	0003:0996			offset: 0x16													default: 0x0000		
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	SD5H[15]	SD5H[14]	SD5H[13]	SD5H[12]	SD5H[11]	SD5H[10]	SD5H[9]	SD5H[8]	SD5H[7]	SD5H[6]	SD5H[5]	SD5H[4]	SD5H[3]	SD5H[2]	SD5H[1]	SD5H[0]			
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W			
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			

Bit	Name	Reset Value	R/W	Function
150	SD5H	0	R/W	SDRAM data register 5H

20.9.13 SDBUFD6L

SDRAM Transfer Data Buffer 6 Low

SDBUFD6L	0003:0998			offset: 0x18													default: 0x0000		
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	SD6L[15]	SD6L[14]	SD6L[13]	SD6L[12]	SD6L[11]	SD6L[10]	SD6L[9]	SD6L[8]	SD6L[7]	SD6L[6]	SD6L[5]	SD6L[4]	SD6L[3]	SD6L[2]	SD6L[1]	SD6L[0]			
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W			
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			

Bit	Name	Reset Value	R/W	Function
150	SD6L	0	R/W	SDRAM data register 6L

20.9.14 SDBUFD6H

SDRAM Transfer Data Buffer 6 High

SDBUFD6H	0003:099A			offset: 0x1A													default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Name	SD6H[15]	SD6H[14]	SD6H[13]	SD6H[12]	SD6H[11]	SD6H[10]	SD6H[9]	SD6H[8]	SD6H[7]	SD6H[6]	SD6H[5]	SD6H[4]	SD6H[3]	SD6H[2]	SD6H[1]	SD6H[0]		
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W		
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		

Bit	Name	Reset Value	R/W	Function
150	SD6H	0	R/W	SDRAM data register 6H

20.9.15 SDBUFD7L

SDRAM Transfer Data Buffer 7 Low

SDBUFD7L	0003:099C			offset: 0x1C			default: 0x0000									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	SD7L[15]	SD7L[14]	SD7L[13]	SD7L[12]	SD7L[11]	SD7L[10]	SD7L[9]	SD7L[8]	SD7L[7]	SD7L[6]	SD7L[5]	SD7L[4]	SD7L[3]	SD7L[2]	SD7L[1]	SD7L[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:0	SD7L	0	R/W	SDRAM data register 7L

20.9.16 SDBUFD7H

SDRAM Transfer Data Buffer 7 High

SDBUFD7H	0003:099E			offset: 0x1E												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	SD7H[15]	SD7H[14]	SD7H[13]	SD7H[12]	SD7H[11]	SD7H[10]	SD7H[9]	SD7H[8]	SD7H[7]	SD7H[6]	SD7H[5]	SD7H[4]	SD7H[3]	SD7H[2]	SD7H[1]	SD7H[0]	
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:0	SD7H	0	R/W	SDRAM data register 7H

20.9.17 SDBUFAD1

SDRAM Transfer Address #1 - Low

SDBUFAD1	0003:09A0			offset: 0x20												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	AAI	RSV	SDAH[6]	SDAH[5]	SDAH[4]	SDAH[3]	SDAH[2]	SDAH[1]	SDAH[0]	
R/W	-	-	-	-	-	-	-	R/W	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	-	-	-	-	-	-	-	0	-	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:9	RSV			<u>Reserved</u>
8	AAI	0	R/W	<u>Automatic address increment</u> » 0: no autoincrement « » 1: After burst transfer, SDAD1H/L is automatically incremented to the next burst boundary «
7	RSV			<u>Reserved</u>
6:0	SDAH	0	R/W	SDRAM Address (High) Upper 7 bits of SDRAM address #1 Address is the offset from the SDRAM base and is specified in 32-byte (burst) units

20.9.18 SDBUFAD2

SDRAM Transfer Address #1 – High

SDBUFAD2	0003:09A2			offset: 0x22												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	SDAL[15]	SDAL[14]	SDAL[13]	SDAL[12]	SDAL[11]	SDAL[10]	SDAL[9]	SDAL[8]	SDAL[7]	SDAL[6]	SDAL[5]	SDAL[4]	SDAL[3]	SDAL[2]	SDAL[1]	SDAL[0]	
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:0	SDAL	0	R/W	<u>SDRAM Address (Low)</u> Lower 16 bits of SDRAM address #1 Address is the offset from the SDRAM base and is specified in 32-byte (burst) units Note: Since data is tranfered in 32-byte units the address specified must be 32-byte aligned (5 LSBs should be zero)

20.9.19 SDBUFCTL

SDRAM Transfer Control

SDBUFCTL	0003:09A4				offset: 0x24								default: 0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	BUFC	WM	WA	RSD
R/W	-	-	-	-	-	-	-	-	-	-	-	-	W	R/W	R/W	R/W
Default	-	-	-	-	-	-	-	-	-	-	-	-	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:4	RSV			Reserved
3	BUFC	0	W	Buffer Clear When set to '1', all data in data registers are cleared.
2	WM	0	R/W	<u>Write Modified Data</u> When set to "1", only the data in SDBUF* registers where write was performed since the last read are written into SDRAM at address specified by SDAD1L/H. Write is done in a 32-byte burst, aligned with a 32-byte addresses, so, 5 LSBs of SDAD1L/H are ignored. This register is automatically cleared when the transfer to SDRAM is finished.
1	WA	0	R/W	<u>Write All Data</u> When set to "1", all data in SDBUF* data registers is written to SDRAM at address specified by SDAD1L/H. Write is done in a 32-byte burst, aligned with a 32-byte addresses, so, 5 LSBs of SDAD1L/H are ignored. This register is automatically cleared when the transfer to SDRAM is finished.
0	RSD	0	R/W	<u>Read All Data</u> When set to "1", data is read into SDBUF* data registers from SDRAM at address specified by SDAD1L/H. Read is done in a 32-byte burst, aligned with a 32-byte addresses, so, 5 LSBs of SDAD1L/H are ignored. This register is automatically cleared when the transfer from SDRAM is finished.

20.9.20 SDMODE

SDRAM Mode Register

SDMODE	0003:09A6			offset:	0x26		default: 0x0080									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	TRDL	SDBW	TRCD	BNS	CASL	MEMT[2]	MEMT[1]	MEMT[0]	DQMC	APO	SDCTL[5]	SDCTL[4]	SDCTL[3]	SDCTL[2]	SDCTL[1]	SDCTL[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15	TRDL	0	R/W	<u>TRDL select (in SDRAM clock units)</u> » 0: 1 cycle « » 1: 2 cycles «
14	SDBW	0	R/W	<u>SDRAM bus width select</u> » 0: 32 bits « » 1: 16 bits «
13	TRCD	0	R/W	<u>RCD select (in SDRAM clock units)</u> » 0: 2 cycle « » 1: 3 cycles «
12	BNS	0	R/W	<u>Bank select</u> » 0: 2 banks « » 1: 4 banks «
11	CASL	0	R/W	<u>CAS Latency (in SDRAM clock units)</u> » 0: 2 cycles « » 1: 3 cycles «
10:8	MEMT	0	R/W	<u>Memory Type</u> » 0: 2k x 256 words « » 1: 4k x 256 words « » 2: 4k x 512 words « » 3: 8k x 512 words « » 4: 8k x 1024 words « » 5: 8k x 2048 words « » 6,7: Reserved «
7	DQMC	1	R/W	<u>DQM control</u> » 0: Normal « » 1: Forcibly sets DQM signal to '1' «
6	APO	0	R/W	<u>Automatic Power Down Mode</u> » 0: Disabled « » 1: Enabled « NOTE : When AP0=1 and there is no access to SDRAM for 8 SDRAM clock cycles then the SDR_CKE signal goes low. When SDRAM is accessed, SDR_CKE will be high automatically.
5:0	SDCTL	0	R/W	<u>SDRAM control select</u> » 000000: NOP: No operation « » 000001: MRS: SDRAM mode register setting « » 000010: PREA: Precharges all banks « » 000100: REF: Issues auto refresh command « » 001000: SELF: Enters self refresh mode « » 010000: SELFC: Cancels self refresh/power down mode « » 100000: PDN: Enters power down mode « <u>Settings for mobile SDRAM</u> » 010001: EMRS: SDRAM extended mode register set « » 110000: DPDN: Deep power down mode « In EMRS mode the following configurations can be done: MEMT[1:0] = 00 -> normal, 01 -> 1/2 strength, 10 -> 1/4 strength, 11 -> reserved Temperature compensated self refresh: [CASL:TRDC] bits = 00 -> 70 C, 01 -> 45 C, 10 -> 15 C, 11 -> 85 C Partial array self refresh: SDBW bit = 0 -> All banks, SDBW bit = 1 -> Banks A and B

20.9.21 REFCTL

Refresh Control Register

REFCTL	0003:09A8				offset: 0x28				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	REFEN	REFC[7]	REFC[6]	REFC[5]	REFC[4]	REFC[3]	REFC[2]	REFC[1]	REFC[0]
R/W	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	-	-	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:9	RSV			<u>Reserved</u>
8	REFEN	0	R/W	<u>Auto Refresh Enable</u> » 0: Disabled « » 1: Enabled «
7:0	REFC	0	R/W	<u>Refresh Cycle</u> Sets interval of auto refresh. Interval is (REFC + 1) x 8 SDRAM clocks.

20.9.22 SDPRTY1

SDRAM Priority Select #1 (CCD Controller)

SDPRTY1 0003:09AA offset: 0x2A default: 0x0800

[illegible][illegible]

20.9.23 SDPRTY2

SDRAM Priority Select #2 (Preview Engine)

SDPRTY2	0003:09AC				offset: 0x2C				default: 0x0400							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	PREV[11]	PREV[10]	PREV[9]	PREV[8]	PREV[7]	PREV[6]	PREV[5]	PREV[4]	PREV[3]	PREV[2]	PREV[1]	PREV[0]
R/W	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	0	1	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:12	RSV			Reserved
11:0	PREV	1024	R/W	<u>SDRAM Access priority for Preview Engine</u> - Priority is set by writing "1" in any one of bits [11:0] - Bit 11 is highest priority, bit 0 lowest priority. - Only one bit should be set to "1" - If "1" is written in the same bit position in different registers, the request of the low register address takes priority and the request of the high register address is ignored.

20.9.24 SDPRTY3

SDRAM Priority Select #3 (Hardware 3A)

SDPRTY3	0003:09AE				offset: 0x2E				default: 0x0200							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	H3A[11]	H3A[10]	H3A[9]	H3A[8]	H3A[7]	H3A[6]	H3A[5]	H3A[4]	H3A[3]	H3A[2]	H3A[1]	H3A[0]
R/W	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	0	0	1	0	0	0	0	0	0	0	0	0

[illegible]

20.9.25 SDPRTY4

SDRAM Priority Select #4 (On Screen Display)

SDPRTY4	0003:09B0				offset: 0x30										default: 0x0100	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	OSD[11]	OSD[10]	OSD[9]	OSD[8]	OSD[7]	OSD[6]	OSD[5]	OSD[4]	OSD[3]	OSD[2]	OSD[1]	OSD[0]
R/W	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	0	0	0	1	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:12	RSV			Reserved
11:0	OSD	256	R/W	<u>SDRAM Access priority for OSD</u> - Priority is set by writing "1" in any one of bits [11:0] - Bit 11 is highest priority, bit 0 lowest priority. - Only one bit should be set to "1" - If "1" is written in the same bit position in different registers, the request of the low register address takes priority and the request of the high register address is ignored.

20.9.26 SDPRTY5

SDRAM Priority Select #5 (ARM CPU)

SDPRTY5	0003:09B2			offset: 0x32			default: 0x0080									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	ARMCPU[11]	ARMCPU[10]	ARMCPU[9]	ARMCPU[8]	ARMCPU[7]	ARMCPU[6]	ARMCPU[5]	ARMCPU[4]	ARMCPU[3]	ARMCPU[2]	ARMCPU[1]	ARMCPU[0]
R/W	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	0	0	0	0	1	0	0	0	0	0	0	0

[illegible]

20.9.27 SDPRTY6

SDRAM Priority Select #6 (SDRC Channel 1)

SDPRTY6	0003:09B4				offset: 0x34				default: 0x0040							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	SDRCCH1[11]	SDRCCH1[10]	SDRCCH1[9]	SDRCCH1[8]	SDRCCH1[7]	SDRCCH1[6]	SDRCCH1[5]	SDRCCH1[4]	SDRCCH1[3]	SDRCCH1[2]	SDRCCH1[1]	SDRCCH1[0]
R/W	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	0	0	0	0	0	1	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:12	RSV			Reserved
11:0	SDRCCH1	64	R/W	<u>SDRAM Access priority for MTC, MS, MMCSD , DSP, DMA in SDRC Channel 1</u> - Priority is set by writing "1" in any one of bits [11:0] - Bit 11 is highest priority, bit 0 lowest priority. - Only one bit should be set to "1" - If "1" is written in the same bit position in different registers, the request of the low register address takes priority and the request of the high register address is ignored.

20.9.28 SDPRTY7

SDRAM Priority Select #7 (SDRC Channel 2)

SDPRTY7	0003:09B6				offset: 0x36				default: 0x0020							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	SDRCCH2[11]	SDRCCH2[10]	SDRCCH2[9]	SDRCCH2[8]	SDRCCH2[7]	SDRCCH2[6]	SDRCCH2[5]	SDRCCH2[4]	SDRCCH2[3]	SDRCCH2[2]	SDRCCH2[1]	SDRCCH2[0]
R/W	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	0	0	0	0	0	0	1	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:12	RSV			Reserved
11:0	SDRCCH2	32	R/W	SDRAM Access priority for MTC, MS, MMCSD, DSP, DMA in SDRC Channel 2 - Priority is set by writing "1" in any one of bits [11:0] - Bit 11 is highest priority, bit 0 lowest priority. - Only one bit should be set to "1" - If "1" is written in the same bit position in different registers, the request of the low register address takes priority and the request of the high register address is ignored.

20.9.29 SDPRTY8

SDRAM Priority Select #8 (SDRC Channel 3)

SDPRTY8	0003:09B8				offset: 0x38				default: 0x0010							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	SDRCCH3[11]	SDRCCH3[10]	SDRCCH3[9]	SDRCCH3[8]	SDRCCH3[7]	SDRCCH3[6]	SDRCCH3[5]	SDRCCH3[4]	SDRCCH3[3]	SDRCCH3[2]	SDRCCH3[1]	SDRCCH3[0]
R/W	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	0	0	0	0	0	0	0	1	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:12	RSV			Reserved
11:0	SDRCCH3	16	R/W	<u>SDRAM Access priority for MTC, MS, MMCSD , DSP, DMA in SDRC Channel 3</u> - Priority is set by writing "1" in any one of bits [11:0] - Bit 11 is highest priority, bit 0 lowest priority. - Only one bit should be set to "1" - If "1" is written in the same bit position in different registers, the request of the low register address takes priority and the request of the high register address is ignored.

20.9.32 SDPRTY11**SDRAM Priority Select #11 (Coprocessor DMA)**

SDPRTY11	0003:09BE				offset: 0x3E				default: 0x0002							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	MGBUG[11]	MGBUG[10]	MGBUG[9]	MGBUG[8]	MGBUG[7]	MGBUG[6]	MGBUG[5]	MGBUG[4]	MGBUG[3]	MGBUG[2]	MGBUG[1]	MGBUG[0]
R/W	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	0	0	0	0	0	0	0	0	0	0	1	0

Bit	Name	Reset Value	R/W	Function
15:12	RSV			Reserved
11:0	MGBUG	2	R/W	<u>SDRAM Access priority for Co-processor DMA</u> - Priority is set by writing "1" in any one of bits [11:0] - Bit 11 is highest priority, bit 0 lowest priority. - Only one bit should be set to "1" - If "1" is written in the same bit position in different registers, the request of the low register address takes priority and the request of the high register address is ignored.

20.9.33 SDPRTY12

SDRAM Priority Select #12 (VLC)

SDPRTY12	0003:09C0				offset: 0x40				default: 0x0001							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	VLC[11]	VLC[10]	VLC[9]	VLC[8]	VLC[7]	VLC[6]	VLC[5]	VLC[4]	VLC[3]	VLC[2]	VLC[1]	VLC[0]
R/W	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0	1

Bit	Name	Reset Value	R/W	Function
15:12	RSV			Reserved
11:0	VLC	1	R/W	<u>SDRAM Access priority for VLC</u> - Priority is set by writing "1" in any one of bits [11:0] - Bit 11 is highest priority, bit 0 lowest priority. - Only one bit should be set to "1" - If "1" is written in the same bit position in different registers, the request of the low register address takes priority and the request of the high register address is ignored.

20.9.34 RSV

SDRAM reserved register

RSV	0003:09C2			offset: 0x42												default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	

Bit	Name	Reset Value	R/W	Function
15:0	RSV			Reserved

20.9.36 SDDMASEL

SDRAM DMA selects register

SDDMASEL	0003:09C6			offset: 0x46			default: 0x0820									
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	BURST3[1]	BURST3[0]	SEL3[2]	SEL3[1]	SEL3[0]	BURST2[1]	BURST2[0]	SEL2[2]	SEL2[1]	SEL2[0]	BURST1[1]	BURST1[0]	SEL1[2]	SEL1[1]	SEL1[0]
R/W	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	0	0	0	1	0	0	0	0	0	1	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15	RSV			Reserved
14:13	BURST3	0	R/W	<u>Burst Mode #3</u> » 0 : 1 burst DMA addr[1:0] = '00' 4 byte alignment « » 1 : 4 burst DMA addr[3:0] = '0000' 16 byte alignment « » 2 : 8 burst DMA addr[4:0] = '00000' 32 byte alignment «
12:10	SEL3	2	R/W	<u>DMA Select #3</u> » 0 : MTC « » 1 : SIF « » 2 : MS « » 3 : MMCSD « » 4 : DSP (HPIB) « Note: Do not set the same value to SEL1, SEL2 and SEL3
9:8	BURST2	0	R/W	<u>Burst Mode #2:</u> » 0 : 1 burst DMA addr[1:0] = '00' 4 byte alignment « » 1 : 4 burst DMA addr[3:0] = '0000' 16 byte alignment « » 2 : 8 burst DMA addr[4:0] = '00000' 32 byte alignment «
7:5	SEL2	1	R/W	<u>DMA Select #2</u> » 0 : MTC « » 1 : SIF « » 2 : MS « » 3 : MMCSD « » 4 : DSP (HPIB) « Note: Do not set the same value to SEL1, SEL2 and SEL3
4:3	BURST1	0	R/W	<u>Burst Mode #1</u> » 0 : 1 burst DMA addr[1:0] = '00' 4 byte alignment « » 1 : 4 burst DMA addr[3:0] = '0000' 16 byte alignment « » 2 : 8 burst DMA addr[4:0] = '00000' 32 byte alignment «
2:0	SEL1	0	R/W	<u>DMA Select #1</u> » 0 : MTC « » 1 : SIF « » 2 : MS « » 3 : MMCSD « » 4 : DSP (HPIB) « Note: Do not set the same value to SEL1, SEL2 and SEL3

21 SmartMedia Interface

21.1 Introduction

The SmartMedia (also known as SSFDC (Solid State Floppy Disk Card)) Controller transfers data between the ARM or DMA controller on one side and a SmartMedia memory card or cards on the other side. The ARM and DMA controller can read/write the data in memory by accessing the memory mapped command, address and data registers of the SmartMedia memory card. The SmartMedia memory card is an MTC device and the SmartMedia interface is the External Memory Region 2 (CS2) in DM320. The SmartMedia interface in DM320 supports both FLASH Memory and MASK ROM types of SmartMedia cards.

Please refer chapter on Memory Traffic controller for details about controlling the smart media interface.

Note: XD cards are also supported using the smart media interface.

21.2 Features

The SmartMedia interface of DM320 has the following features.

- Supports a SmartMedia (SSFDC) card and NAND flash
- Support for all variants of 3.3V SmartMedia memory cards
- Software-oriented implementation for future extensibility.
- Supports DMA transfer between the SmartMedia card and SDRAM.
- Supports different page sizes
- Supports ECC generation for different page sizes

21.2.1 Smart Media interface

DM320 smart media interface is programmable. It supports different page sizes and has ECC calculation logic built in. The timing for the CS2 interface of DM320 is given in Figure 159.

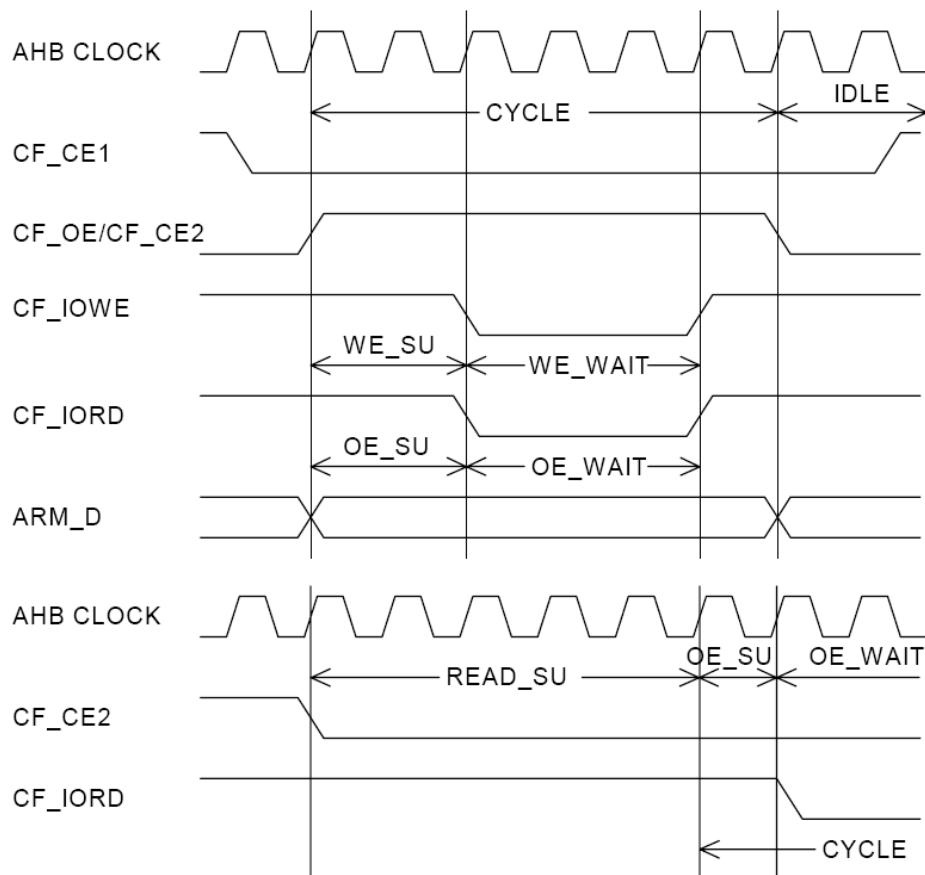


Figure 159: DM320 Smart Media (CS2) timing

21.2.2 SmartMedia Device Pins

Table 86 shows the pins of the SmartMedia memory card used in each model.

The SmartMedia card has eight bi-directional data lines for I/O, and separate read, write, address latch, command and device enable lines. In the case of FLASH memories, a separate Write Protect line is also provided. The device status is available through a Ready/Busy line.

Pin	Type	Function
GND		Ground
CLE	I	Command Latch Enable
ALE	I	Address Latch Enable
-WE	I	Write Enable
-WP	I	Write Protect (applicable only to FLASH memories)
D0	I/O	Data 0
D1	I/O	Data 1
D2	I/O	Data 2

D3	I/O	Data 3
-CD	O	Card Detect (Outputs the ground level when the SmartMedia is present)
VCC		Vcc
D4	I/O	Data 4
D5	I/O	Data 5
D6	I/O	Data 6
D7	I/O	Data 7
LVD	O	Low Voltage Detect
R/-B	O (Open Drain)	Ready or Busy (indicates the internal operating condition of the device)
-RE	I	Read Enable
-CE	I	Card Enable

*I = Input to the SmartMedia; O = Output from the SmartMedia

Table 86: SmartMedia Device Pins

21.2.3 Device Internal Configuration

The memory area is organized in blocks, with each block containing 16 or 32 pages depending on the model and size of the SmartMedia. Each page has a user data area and a redundant data area. The user data area contains the user data, and is intended to store the data that is written into it by the user. The redundant data area contains information that indicates the validity of the user data contained in that particular page. It also stores information on the logical address of the block to which the page belongs. Both the user data area and the redundant data area can be accessed by the appropriate commands.

SmartMedia device driver maintains a logical to physical mapping that converts the logical address of the block specified by the user into its physical block address. The device driver also keeps track of the bad blocks and free blocks in the device. Data is read and written (in the case of FLASH memories) in units of pages and erased (applicable only in FLASH memories) in units of blocks. SmartMedia cards, with 32 Mbytes of size or more, are organized into 16 Mbyte “zones”. Each zone contains 1024 physical blocks, of which there should be at least 1000 logical blocks.

The first usable physical block of the SmartMedia is reserved for the Card Information Structure (CIS) and the Identify Device Information (IDI) fields. This is located in the first page of the first usable physical block, in the first zone. This CIS/IDI data contains the device specific and manufacturer specific data that characterizes the memory card. The CIS/IDI field has to be located within the first few physical blocks present in the device, the exact number depends on the particular device model and capacity.

21.3 Programming the SmartMedia

SmartMedia memory cards have three memory mapped registers – Command, Address and Data registers. Programming requires the command to be placed in the Command register, the address on the address register. The data can be

read or written from/to the Data register. The status register indicates whether the SmartMedia is ready or busy and when in the ready state, whether the preceding command failed or passed.

21.3.1 SmartMedia Commands

The commands supported by SmartMedia memory cards are listed in Table 87.

Function	Opcode Sequence	Description
Serial Data Input	80h	This command is the first command in a SmartMedia page write command sequence. This command can be sent only when the SmartMedia is in the Ready state. This command is applicable to FLASH memories only.
Read (1)	00h	This command reads a page including the data area and the redundant data area. This command can be sent only when the SmartMedia is in the Ready state.
Read (2)	01h	This command is applicable to memory models with 512+16 bytes/page. This command reads the half a page of data (starting from the middle of the page). This command can be sent only when the SmartMedia is in the Ready state.
Read (3)	50h	This command reads the redundant data area of a page. This command is applicable only when the device is in the Ready state.
Reset	FFh	This command resets the SmartMedia device. This command can be sent irrespective of whether the device is in the Ready state or Busy state.
Page Program	10h	This command programs a page after the entire page data has been placed on the Data register. This is the second command in the write command sequence (the first being Serial Data Input). This command can be sent only when the device is in the Ready state. This command is applicable only to FLASH memories.
Block Erase	60h (1 st cycle) D0h (2 nd cycle)	This command erases a block of data. In an erased block, all the data values are set to "FFh". This command can be sent only when the SmartMedia is in the Ready state. This command is applicable to FLASH memories only.
Status Read	70h	This command reads the status of the SmartMedia device. The status output byte tells whether the device is Ready/Busy, and if ready, whether the preceding operation passed or failed. This also indicates whether a FLASH memory is write protected or not. This command can be sent irrespective of whether the device is Ready or Busy.
ID Read	90h	This command reads the device ID of the SmartMedia device. The output of 4 bytes contains the following information: 1 st byte: Manufacturer code 2 nd byte: Device code 3 rd byte: Whether external block present(if A5h is read, then external block is present) 4 th byte: Whether High Speed Programming is supported (if C0h is read, then high speed programming is supported).

Function	Opcode Sequence	Description
		This command can be sent only when the device is in the Ready state.
High Speed ID Read	91h	This command can be issued only to devices that support the High Speed programming mode. This command extracts the device specific information in the eight bytes of data it reads: 1st byte: Specifies the number of MLC layers, whether write cache is supported, and whether simultaneous page programming is supported. 2nd byte – 5th byte: Vendor specific data 6th byte – 8th byte: Reserved data
High Speed Status Read	71h	This command is applicable ONLY to SmartMedia devices that support High Speed programming. This command outputs a byte that indicates whether the device is ready or busy and whether it is write-protected or not. If the device is ready, this indicates whether the preceding operation passed or failed in each of the four districts.
Dummy Page Program	11h	This command is applicable only in SmartMedia FLASH devices that support high speed programming mode. This command is issued after writing a page of data into the write cache.
Cache Page Program	15h	This command is applicable only in SmartMedia FLASH devices that support high speed programming. This command is issued after writing into each of the four districts supported by high speed programming.

Table 87: SmartMedia commands

21.3.2 Configuration of SmartMedia Interface

SmartMedia interface is attached to External Memory Region 2 (CS2) of DM320 memory map. A set of registers of the EMIF interface need to be configured as indicated below, to access the SmartMedia:

- Configure the CFCTRL1 register by setting a '1' in the field "CFSEL". This selects the SmartMedia interface.
- Configure the BUSWAITMD register by setting a '1' in the fields "CFRSL" and "CFRW". Setting the CFRSL bit connects the Ready/Busy pin of the SmartMedia device to the Ready terminal of the interface. Setting the CFRW bit ensures that the device is not accessible except when the device is ready.
- Configure the PAGESZ register to select the page size of the SmartMedia device. Currently, only 256 + 8 bytes/page (set '00') and 512 + 16 bytes/page or 256 + 8 words/page (set '01'), 1024 + 32 bytes/page or 512 + 16 words/page and 2048 + 64 bytes / page or 1024 + 32 words / page are supported.
- Enable the SmartMedia device by setting a '1' in the SMCTRL register

- Configure the GIO pin which is connected to the SmartMedia Card Detect pin, to enable card insertion/removal detection
- Configure the GIO pin attached to the Write Protect pin of the SmartMedia device (applicable only in FLASH memories, if write protection is needed).

21.3.3 SmartMedia Initialization Sequence

SmartMedia cards need to be initialized before any read/write/erase operations can be performed. The initialization sequence involves testing the physical format of the device and initializing the logical to physical address mapping. The initialization sequence is as follows:

- Configure the SmartMedia interface as described in 21.3.2
- Reset the device by sending the reset command.
- Read the device ID using the ID Read command, and determine the capability information of the device, especially whether high speed programming support is present.
- Identify the device model, size and number of sectors
- Check the Card Information Structure field of the device to ascertain whether the physical format of the card is correct
- Create and initialize the logical to physical address mapping, at a block level.

21.3.4 DMA Configuration for SmartMedia Interface

SmartMedia memory cards use the EMIF DMA for effecting DMA transfers from/to SDRAM. SmartMedia, attached to External memory region 2, is a MTC device. The other endpoint SDRAM is accessed through EMIF, which is an AHB device. Hence, all DMA transfers from/to SmartMedia are MTC to AHB DMA transfers.

The MTC, SIF, MS, MMC/SD and HPiB modules in DM320 share three DMA channels. Consequently, only up to three DMAs among five modules' DMAs can be used exclusively. The setting of the modules that use the DMA channel is performed in the SELx fields (0x309C6 [12:10], [7:5] and [2:0]) of the DMASEL register of SDRG. To use this module's DMA, set 0x3 to the SEL1 or SEL2 or SEL3 field, depending on which channel is selected. The burst size of the DMA transfer can also be selected. There are three DMA burst sizes supported – 1 bit, 4 bits, and 8 bits. The DMA burst size can be selected, by setting 0 (1 bit), 1(4 bits), or 2(8 bits) in the BURSTx field (0x309C6 [14:13], [9:8] and [4:3]), corresponding to the channel number. For instance, to select a burst size of 8 bits in DMA using channel 2, set 0x3 in SEL2 and 0x2 in BURST2 of DMASEL register.

The EMIF DMA registers are configured to set the size, direction and endpoints of the DMA transfer. The selection of SmartMedia as the MTC device is done by setting 0x02 in the MTCSEL field of DMAMTCSEL register of EMIF interface. The SDRAM transfer start address is set in the EMIF's AHBADDL and AHBADDH registers. The source/destination address on the SmartMedia device for DMA transfer is set in the MTCADDL and MTCADDH registers. The size of the DMA transfer is set in DMASIZE register. The endianness control, the direction and the actual initiation of the DMA transfer is done by setting the

ENDI, DIR and GO fields respectively of the DMACTL register. The GO field is automatically cleared after the completion of the DMA.

In addition, when DMA is used, the transfer destination/ source SDRAM address and the size of data transfer can be specified only in 4bytes units.

21.3.5 SmartMedia Write Sequence

SmartMedia FLASH memories can be written one page (one sector) at a time, for all the pages in that block. In a successful write sequence of a page:

- The Controller enables the SmartMedia device and disables write protection, if any on the device.
- The Controller sends a write command (Serial Input Data – 0x80h) to the SSFDC command register. In order to write to the redundant data area alone, the following three commands are to be issued in sequence – Reset(FFh), Read(3) (50h) and Serial Input Data (80h).
- Subsequently the address of the page is placed byte by byte on the SSFDC address register.
- The Controller sends a block of data to the card, byte by byte or through EMIF DMA.
- After the data is transferred, the Controller sends a page program command (PAGE PROGRAM – 10h) to the SSFDC command register. If the redundant section alone is written, the reset command (FFh), Read (1) (00h) or Read(2) (01h) MUST be issued, prior to issuing the PAGE PROGRAM command, to reset the data pointer mode.
- The Controller waits for the write to complete and then issues a status read command by placing the status read command (70h) on the SSFDC command register.
- The status word indicates, after the device is in ready state, whether the write operation succeeded or failed.

21.3.6 SmartMedia Read Sequence

SmartMedia memory cards support three types of read commands:

1. Read of a page including the data area and the redundant data area
2. Read of half a page of data (starting from the middle of the page) in 512 bytes/page media
3. Read of the redundant data alone

The read sequence is the same for all the three types of reads, the only changes being in the command issued and the number of bytes of data read. In a successful SmartMedia read sequence:

- a. The Controller enables the SmartMedia device

- b. The Controller sends the appropriate read command (Read(1), Read(2) or Read (3) command) to the SSFDC command register.
- c. Subsequently the address of the page is placed byte by byte on the SSFDC address register.
- d. The Controller reads a block of data from the card, byte by byte or through EMIF DMA.
- e. No status read is necessary in the case of a read command

21.3.7 SmartMedia Erase Sequence

SmartMedia FLASH memories support erasure of an entire block. Erasure of a block is a necessary step before writing any page in that block. In a successful SmartMedia erase sequence:

- The Controller enables the SmartMedia device, and removes the write protection
- The Controller sends the erase command (60h) to the SSFDC command register.
- Subsequently the address of the block is placed byte by byte on the SSFDC address register.
- The Controller sends the second erase command (D0h) to the SSFDC command register.
- The Controller waits until the device is Ready and sends the Status Read command (70h) to the SSFDC command register.
- The status word indicates whether the erase was successful or otherwise.

21.4 High Speed Programming

SmartMedia FLASH memories can optionally have support for High Speed Programming. High Speed Programming enables the controller to perform a write or erase on multiple pages of the device simultaneously. Normal programming is also supported in devices that support High Speed Programming. This is an additional function, that exploits the parallelism in the device.

The presence or absence of High Speed Programming can be checked by the ID read command (90h). If the device supports high speed programming, the 4th byte of the ID read output will have the value "C0h". Further, the High Speed ID read command (91h) can be used to check whether the device supports simultaneous programming of multiple pages. If high-speed programming is supported, the three additional commands – High Speed Status Read (71h), Dummy Page Program (11h) and Cache Page Program (15h), listed in Table 2, are supported.

21.4.1 Multi-Block Write Sequence

SmartMedia devices, which support multi-block programming, execute the write operation simultaneously in all districts of the device. District refers to each independently accessible memory subunit of the device. The internal addressing maps each of the blocks in the device to one of the districts. In 64MB SmartMedia or higher capacity media, upto four districts are supported. One of each four consecutive physical blocks maps into a separate district. For instance, Blocks 0, 4, 8,map to District 0, Blocks 1, 5, 9, map to District 1 and so on. In a successful SmartMedia write sequence involving four districts:

- The Controller enables the SmartMedia device, and removes the write protection
- The Controller sends the Serial Data Input (80h) command on the SSFDC command register
- The address of the page in District 0 is placed in the SSFDC address register
- The data is placed on the SSFDC read/write register byte by byte or using DMA
- After the entire data is sent, a dummy page program command is issued. This does not actually program the page. This writes the page data in a write cache.
- The above sequence of steps is executed for all the remaining Districts
- After the data is written to the last District, instead of the Dummy page program command, the Cache page program command (15h) is sent.
- The above sequence is repeated for all the pages in the block, and after the data for the last page in the last district is sent, the Page program command (10h) is sent. This programs all the four districts simultaneously page by page.
- The Controller waits until the device is Ready and sends the High Speed Status Read command (71h) to the SSFDC command register.
- The status output word indicates whether the page program succeeded or failed in each of the four districts.

In multi-block programming, only one block can be selected from each District in one write sequence. There is no mutual address relation between the selected blocks, except in 128 Mbytes or higher capacity media. The restriction is that the multi-block programming cannot be done across the boundary of 64 Mbytes memory space.

21.4.2 Multi-Block Erase

SmartMedia devices, which support High Speed programming, can be erased four blocks at a time, if the blocks fall into separate districts. In a successful SmartMedia multi-block erase sequence:

- The Controller enables the SmartMedia device, and removes the write protection
- The Controller sends the erase command (60h) to the SSFDC command register.
- Subsequently the address of the first block is placed byte by byte on the SSFDC address register.
- The above three steps are repeated for the rest of the three blocks, each being in different districts.
- The Controller sends the second erase command (D0h) to the SSFDC command register.
- The Controller waits until the device is Ready and sends the High Speed Status Read command (71h) to the SSFDC command register.
- The status word indicates whether the erase was successful in each of the districts.

22 Compact Flash Card Controller

22.1 Introduction

This chapter describes the Compact Flash Card Controller (CFC) of DM320. The CFC is part of External Memory Interface (EMIF) of DM320. EMIF includes specialized interfaces to control and access Compact Flash. The CF controller accesses the CF card through the CS1 external memory region of ARM. Memory Traffic Controller includes a DMA Controller, which allows fast and efficient memory transfers between CF card and ARM internal memory / external memory regions including SDRAM.

22.1.1 Features

CF Controller in DM320 has the following features:

- Supports I/O and Memory mode accesses
- DMA support for data transfer to/from CF card
- Configurable wait states for data access to support different speed grade CF cards

22.2 References

- <http://www.compactflash.org/cgi-bin/specdl2.cgi>
- Refer to chapter on Memory Traffic Controller for more details.

22.3 Memory Map

The Compact Flash memory map is shown Table 88. The table shows the offset address for the CFC memory space and CFC IO space. The ARM can access the CF memory space in the range 0x0000-0x07FF and CF IO space in the range 0x400000-0x400FFF respectively. The address ranges are relative to the start address of CS1 external memory region.

By default, CF card is accessed in Memory mode. To switch from Memory mode to IO mode, write '1' to offset address 0x200.

External Memory Area	Offset address	Size	CFC Interface Function
CS1	0x0000-0x07FF	2k bytes	CFC memory Space (Common and Attribute)

	0x40000-0x400FFF	2k bytes	CFC IO Space
--	------------------	----------	--------------

Table 88: CFC memory map

22.4 Compact Flash Interface

The interconnection between DM320 and external CF card is as shown in Figure 160.

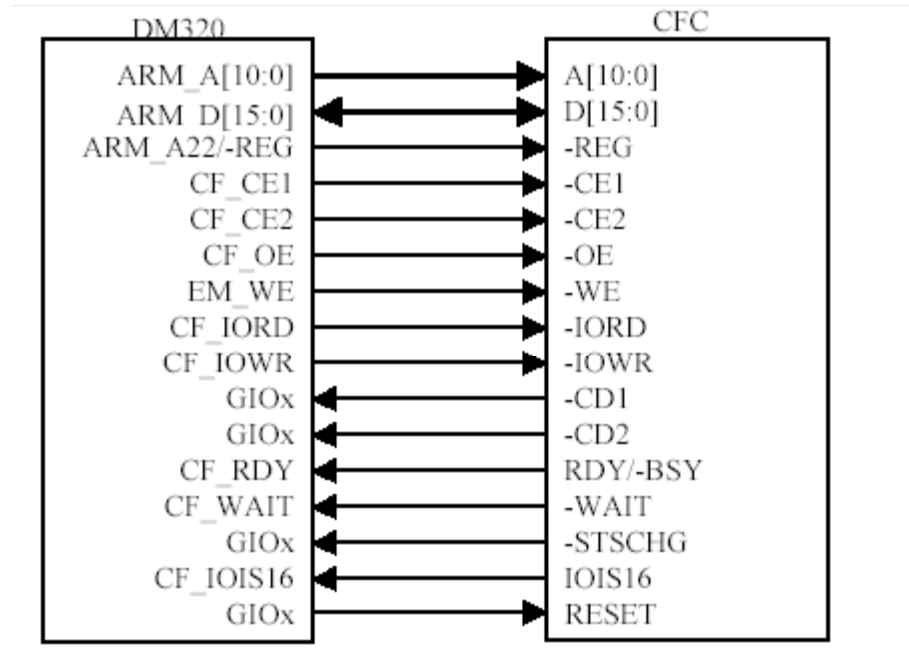


Figure 160: CFC interface block diagram

22.4.1 RDY/-BSY Signal

Compact Flash cards use the RDY/-BSY signal to indicate busy status. RDY/-BSY is connected to the CFRDY input pin of DM320. CFRDY pin is shared between Smart Media interface and CF interface. Bit CFRSL of BUSSTS register determines whether CFRDY is connected to RDY/-BSY signal of CF or Smart Media.

When CFRW in the BUSWAITMD register is set to 1, the DM320 will wait to access the card until the CFRDY signal is low. When CFRW is set to 0, the DM320 will not wait for CFRDY to access the card. The bus cycle will start independent of the state of CFRDY.

An interrupt can be generated when CFRDY transits from LOW to HIGH or HIGH to LOW depending on the status of CBCND bit in BUSINTEN register. Bit CBIEN in BUSINTEN register, enables/disables this interrupt.

22.4.2 Compact Flash Data Bus

The Compact Flash interface supports dynamic bus sizing for I/O mode. Data access will be either 8-bit or 16-bit mode depending on the level of the IOIS16 pin. When IOIS16 is low, the Compact Flash interface operates with an 8-bit bus, and when IOIS16 is high, the interface operates with a 16-bit bus. Dynamic bus sizing is enabled when BUSSZ bit is set in CFCTRL2 register. When BUSSZ is cleared, the bus size is always 16-bits regardless of the IOIS16 signal. The correct number accesses are automatically performed when 32-bit, 16-bit, and 8-bit data is transferred, given that the data bus is set to be 8 or 16-bits wide.

22.4.3 Resetting the Compact Flash

Compact Flash reset signal can be controlled via a GIO pin. Refer to the datasheet of the CF card for exact reset timings. Alternatively, a soft reset can be issued to CF card. Refer to section 22.5.5 for soft reset feature.

22.4.4 Timing control

By configuring the programmable timing parameters as shown in Table 89. The DM320 CF controller can be interfaced to various types of CF cards. Also, an idle state can be inserted in order to prevent bus collision when performing continuous memory accesses. CS1CTRL1A, CS1CTRL1B and CS1CTRL2 registers are used to configure timing characteristics. Figure 161 shows the signal timing diagrams.

Parameter	Description	Clock variables	Variable unit	Initial value after RESET
CYCLE	Number of cycles	2-32 clocks	1 CLK (AHB_CLK)	22
CS_SU	Number of Setup of CS	0-15 clocks	1 CLK	0
WE_SU	Number of Setup of WE	"	1 CLK	2
OE_SU	Number of Setup of OE	"	1 CLK	2
CS_WIDTH	Width of CS	1-32 clocks	1 CLK	21
WE_WIDTH	Width of WE	"	1 CLK	10
CS_WIDTH	Width of CS	"	1 CLK	18
IDLE	Number of Idles	0-3 clocks	1 CLK	1

Table 89: Timing parameters

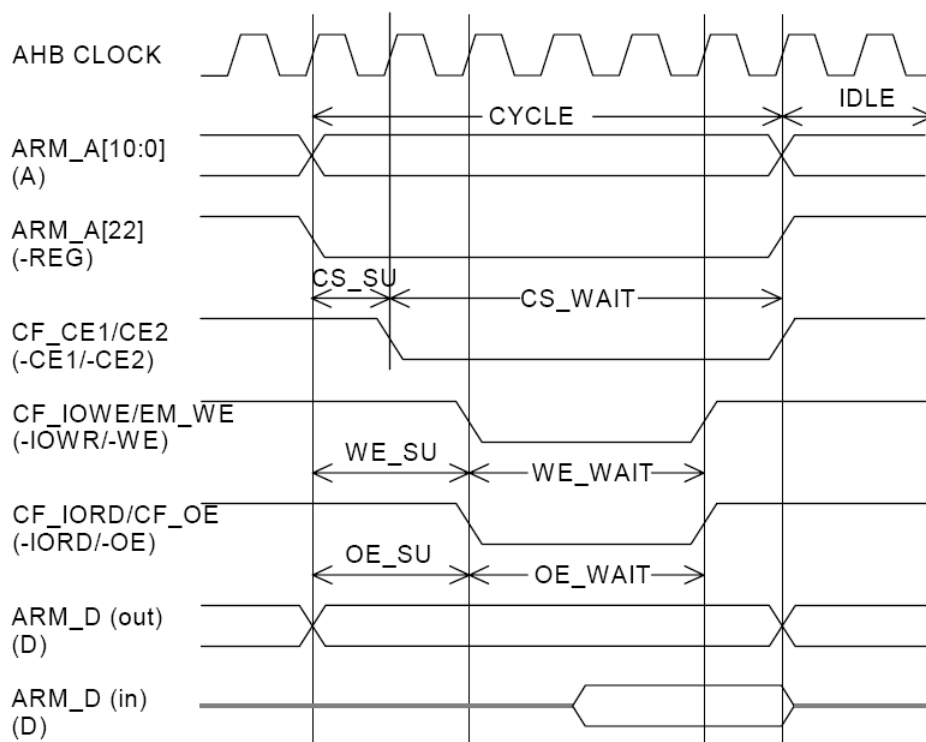


Figure 161: External Memory Bus Signals

22.5 Compact Flash Storage Card

The CF card contains a single chip controller and flash memory module in a match-book sized package with a 50-pin controller, consisting of two rows of 25 female contacts. The controller interfaces with a host system allowing data to be written to and read from the flash modules. The block diagram for the CF card is shown in Figure 162.

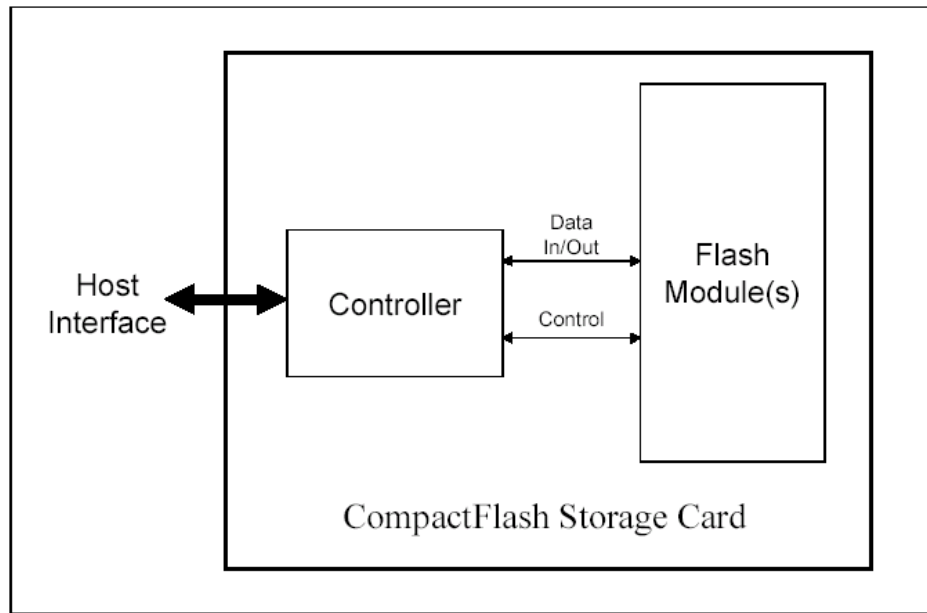


Figure 162: CF Card Block Diagram

22.5.1 Compact Flash Storage Card Operating Modes

The Compact Flash card operates in three modes:

1. Memory Mode
2. IO Mode
3. TRUE IDE mode

Note: DM320 CF Controller does not support TRUE IDE mode

The Compact Flash card is controlled using the standard PCMCIA configuration registers starting at offset address 0x200 in Attribute space when the CF card is operating on memory space.

22.5.2 Attribute Memory Space

Attribute memory space is a space where CF card identification and configuration information is stored. It is limited to 8-bit wide access and only at even addresses. Attribute memory space starts at offset 0x200.

22.5.3 Common Memory Space

The memory transfer to or from the CF card can be either 8-bit or 16-bits for the whole common memory address space.

The CF card request the host to extend the length of a memory write cycle or memory read cycle, until the data is ready, by asserting the – WAIT signal at the start of the cycle.

22.5.4 I/O Transfer Function

The I/O transfer to or from the CF card is either 8-bit or 16-bits. When a 16-bit accessible port is addressed, CF card asserts the IOIS16 signal, otherwise IOSIS16 is de-asserted. When a 16-bit transfer is attempted and the CF card does not assert IOIS16 signal, the host must generate a pair of 8-bit accesses for the 16-bit data.

The CF card request the host to extend the length of a I/O write cycle or I/O read cycle, until the data is ready, by asserting the –WAIT signal at the start of the cycle.

22.5.5 Configuring CF Card

The configuration option register is used to configure the CF card for the desired mode of operation. It can also be used to issue a soft reset to the card.

Operation	D7	D6	D5	D4	D3	D2	D1	D0
R/W	SRESET	LevlREQ	Conf5	Conf4	Conf3	Conf2	Conf1	Conf0

For CF card reset, set SRESET bit to one. The bit should be cleared after waiting for the minimum reset period to complete the reset sequence. The minimum reset period depends upon the electrical characteristics of the particular CF card.

Configuration Index (CONF5-CONF0) resets to zero after card reset. It is used to select operation mode of the CF card as shown Table 90.

Conf5	Conf4	Conf3	Conf2	Conf1	Conf0	Disk Card Mode
0	0	0	0	0	0	Memory Mapped
0	0	0	0	0	1	I/O Mapped, Any 16 byte system decoded boundary
0	0	0	0	1	0	I/O Mapped, 1F0h-1F7h/3F6h-3F7h
0	0	0	0	1	1	I/O Mapped, 170h-177h/376h-377h

Table 90: CFC Configurations

22.6 Software Interface

The CF card can be configured as high performance I/O device through:

- I/O address spaces: 1F0h-1F7h, 3F6h-3F7h (primary); 170h-177h, 376-377h(secondary)
- Memory space (Attribute and Common memory)

The registers used by the host software to issue CF commands are collectively referred to as the “Task File”. The communication to or from the CF card is done using Task File Registers that provide the necessary control and status information. The PCMCIA interface connects peripherals to the host using four register-mapping methods as shown in Table 91. All the addresses are relative offsets to the respective address spaces.

Standard Configurations			
Config Index	I/O or Memory	Address	Description
0	Memory	0h-Fh, 400h-7FFh	Memory Mapped
1	I/O	XX0h-XXFh	I/O Mapped 16 Contiguous Registers
2	I/O	1F0h-1F7h, 3F6h-3F7h	Primary I/O Mapped
3	I/O	170h-177h, 376h-377h	Secondary I/O Mapped

Table 91: CFC Standard Configurations

22.6.1 Primary and Secondary I/O Mapped Registers

When the CFC registers are accessed via I/O mode, the registers appear in the I/O address space as shown Table 92. In the table, register addresses in parenthesis are the secondary I/O address.

-REG	A9-A4	A3	A2	A1	A0	-IORD=0	-IOWR=0
0	1F(17)h	0	0	0	0	Even RD Data	Even WR Data
0	1F(17)h	0	0	0	1	Error Register	Features
0	1F(17)h	0	0	1	0	Sector Count	Sector Count
0	1F(17)h	0	0	1	1	Sector No.	Sector No.
0	1F(17)h	0	1	0	0	Cylinder Low	Cylinder Low
0	1F(17)h	0	1	0	1	Cylinder High	Cylinder High
0	1F(17)h	0	1	1	0	Select Card/Head	Select Card/Head
0	1F(17)h	0	1	1	1	Status	Command
0	3F(37)h	0	1	1	0	Alt Status	Device Control
0	3F(37)h	0	1	1	1	Drive Address	Reserved

Table 92: Primary and Secondary register addresses

22.6.2 Memory Mapped Registers

When the CF card registers are accessed in the memory mode, the registers appear in the Common memory space as shown in Table 93.

-REG	A3	A2	A1	A0	Offset	-IORD=0	-IOWR=0
0	0	0	0	0	0	Even RD Data	Even WR Data
0	0	0	0	1	1	Error	Features
0	0	0	1	0	2	Sector Count	Sector Count
0	0	0	1	1	3	Sector No.	Sector No.
0	0	1	0	0	4	Cylinder Low	Cylinder Low
0	0	1	0	1	5	Cylinder High	Cylinder High
0	0	1	1	0	6	Select Card /Head	Select Card/Head
0	0	1	1	1	7	Status	Command
0	1	0	0	0	8	Dup Even RD Data	Dup. Even WR Data
0	1	0	0	1	9	Dup. Odd RD Data	Dup. Odd WR Data
0	1	1	0	1	D	Dup. Error	Dup. Features
0	1	1	1	0	E	Alt Status	Device Ctl
0	1	1	1	1	F	Drive Address	Reserved

Table 93: Memory mapped register configurations

22.6.3 Compact Flash Card – ATA Registers

The following section describes registers used by the host software to issue CF commands.

22.6.3.1 Data register (Address –1F0h[170h];offset 0,8,9)

The data register is 16-bit register and is used to transfer the data blocks between the CF card and the HOST. The register description is shown in Table 94. This register overlaps with the Error register.

Data Register	CE2-	CE1-	A0	Offset	Data Bus
Word Data Register	0	0	X	0,8,9	D15-D0
Even Data Register	1	0	0	0,8	D7-D0
Odd Data Register	1	0	1	9	D7-D0
Odd Data Register	0	1	X	8,9	D15-D8
Error / Feature Register	1	0	1	1, Dh	D7-D0
Error / Feature Register	0	1	X	1	D15-D8
Error / Feature Register	0	0	X	Dh	D15-D8

Table 94:Data register Accesses

22.6.3.2 Error Register (Address – 1F1h [171h];Offset 1, 0Dh)

This read-only register contains additional information about the source of an error when an error is indicated on bit-0 of the status register (section 22.6.3.9).

D7	D6	D5	D4	D3	D2	D1	D0
BBK	UNC	0	IDNF	0	ABRT	0	AMNF

Bit 7 (BBK): this bit is set when a Bad Block is detected.

Bit 6 (UNC): this bit is set when an Uncorrectable Error is encountered.

Bit 5: This bit is "0".

Bit 4: (IDNF): the requested sector ID is in error or cannot be found.

Bit 3: This bit is "0".

Bit 2 (Abort): This bit is set if the command has been aborted because of a Compact Flash Card status condition: (Not Ready, Write Fault, etc.) or when an invalid command has been issued.

Bit 1: This bit is "0".

Bit 0 (AMNF): This bit is set in case of a general error.

22.6.3.3 Feature Register (Address – 1F1h[171h];Offset 1,0Dh)

This write-only register provides the information regarding features of the CF card that the host can utilize. The register is also accessed on data bits D15-D8 during write operation to offset 0 with –CE2 low and –CE1 high.

22.6.3.4 Sector Count register (Address -1F2h[172h];Offset 2)

This register contains the number of sectors of data requested to be transferred on a read or write operation between the host and the CF card. If the value in this register is zero, a count of 256 sectors is specified.

22.6.3.5 Sector Number register (Address -1F3h[173h];Offset 3)

This register contains the sector number or bits 7-0 of the logical block address (LBA) for any CF card data access, for the subsequent command.

22.6.3.6 Cylinder Low register (Address -1F4h[174h];Offset 4)

This register contains the low order 8-bits of the starting cylinder address or bits 15-8 of the LBA.

22.6.3.7 Cylinder High register (Address -1F5h[175h];Offset 5)

This register contains the high order bits of the starting cylinder address or bits 23-16 of the LBA.

22.6.3.8 Drive / head register (Address 1F6h[176h]; Offset 6)

This register is used to select the drive and head or bits 27-24 of LBA. It is also used to select the LBA addressing instead of cylinder/head/sector addressing.

22.6.3.9 Status and Alternate status registers (Address – 1F7h[177h] & 3F6h [376h]; Offsets 7 & 0Eh)

These registers return the CF card status when read by the host. Reading the status register does clear the pending interrupt while reading the auxiliary status register not clear the pending interrupt. The status bits are described as follows;

D7	D6	D5	D4	D3	D2	D1	D0
1	LBA	1	DRV	HS3	HS2	HS1	HS0

Bit 7 (BUSY): The busy bit is set when the Compact Flash Card has access to the command buffer and registers and the host is locked out from accessing the command register and buffer. No other bits in this register are valid when this bit is set to a 1.

Bit 6 (RDY): RDY indicates whether the device is capable of performing Compact Flash Card operations. This bit is cleared at power up and remains cleared until the Compact Flash card.

Storage Card is ready to accept a command.

Bit 5 (DWF): This bit, if set, indicates a write fault has occurred.

Bit 4 (DSC): This bit is set when the Compact Flash Card is ready.

Bit 3 (DRQ): The Data Request is set when the Compact Flash Card requires that information be transferred either to or from the host through the Data register.

Bit 2 (CORR): This bit is set when a Correctable data error has been encountered and the data has been corrected. This condition does not terminate a multi-sector read operation.

Bit 1 (IDX): This bit is always set to 0.

Bit 0 (ERR): This bit is set when the previous command has ended in some type of error. The bits in the Error register contain additional information describing the error. It is recommended that media access commands (such as Read Sectors and Write Sectors) that end with an error condition should have the address of the first sector in error in the command block registers.

22.6.3.10 Device Control Register (Address – 3F6h[376h]; Offset Eh)

This register is used to control the CFC interrupt request and to issue an ATA soft reset to the card. This register can be written even if the device is busy.

22.6.3.11 Card (Drive) Address Register (Address- 3F7h[377h]; Offset Fh)

The register is provided for compatibility with the ATA disk drive interface.

22.6.4 CF card-ATA Command Set

Table 95 and Table 96 summarize the CFC_ATA command set. The individual commands and the task file for them are discussed in the following sections.

This section defines the software requirements and the format of the commands the host sends to the Compact Flash Cards. Commands are issued to the Compact Flash card by loading the required registers in the command block with the supplied parameters, and then writing the command code to the Command Register. The manner in which a command is accepted varies. There are three classes of command acceptance, all dependent on the host not issuing commands unless the Compact Flash Card is not busy (BSY=0). All commands listed in this specification shall be implemented. Commands can be implemented as “no operation” to meet this requirement.

Upon receipt of a Class 1 command, the Compact Flash Card sets BSY within 400 nsec.

Upon receipt of a Class 2 command, the Compact Flash Card sets BSY within 400 nsec, sets up the sector buffer for a write operation, sets DRQ within 700 µsec, and clears BSY within 400 nsec of setting DRQ.

Upon receipt of a Class 3 command, the Compact Flash Card sets BSY within 400 nsec, sets up the sector buffer for a write operation, sets DRQ within 20 msec (assuming no re-assignments), and clears BSY within 400 nsec of setting DRQ.

Class	COMMAND	Code	FR	SC	SN	CY	DH	LBA
1	Check Power Mode	E5h or 98h	-	-	-	-	D	-
1	Execute Drive Diagnostic	90h	-	-	-	-	D	-
1	Erase Sector(s)	C0h	-	Y	Y	Y	Y	Y
1	Flush Cache	E7h	-	-	-	-	D	-
2	Format Track	50h	-	Y	-	Y	Y	Y
1	Identify Drive	ECh	-	-	-	-	D	-
1	Idle	E3h or 97h	-	Y	-	-	D	-
1	Idle Immediate	E1h or 95h	-	-	-	-	D	-
1	Initialize Drive Parameters	91h	-	Y	-	-	Y	-
1	Key Management Structure Read	B9 Feature 0-127	C	C	C	C	D C	-
1	Key Management Read Keying Material	B9 Feature 80	C	C	C	C	D C	-
2	Key Management Change Key Management Value	B9 Feature 81	C	C	C	C	D C	-
1	NOP	00h	-	-	-	-	D	-
1	Read Buffer	E4h	-	-	-	-	D	-
1	Read Long Sector	22h or 23h	-	-	Y	Y	Y	Y
1	Read Multiple	C4h	-	Y	Y	Y	Y	Y
1	Read Sector(s)	20h or 21h	-	Y	Y	Y	Y	Y
1	Read Verify Sector(s)	40h or 41h	-	Y	Y	Y	Y	Y
1	Recalibrate	1Xh	-	-	-	-	D	-
1	Request Sense	03h	-	-	-	-	D	-
1	Security Disable Password	F6h	-	-	-	-	D	-
1	Security Erase Prepare	F3h	-	-	-	-	D	-
1	Security Erase Unit	F4h	-	-	-	-	D	-
1	Security Freeze Lock	F5h	-	-	-	-	D	-
1	Security Set Password	F1h	-	-	-	-	D	-
1	Security Unlock	F2h	-	-	-	-	D	-
1	Seek	7Xh	-	-	Y	Y	Y	Y
1	Set Features	EFh	Y	-	-	-	D	-
1	Set Multiple Mode	C6h	-	Y	-	-	D	-
1	Set Sleep Mode	E6h or 99h	-	-	-	-	D	-

Table 95: ATA-Command Set

Class	COMMAND	Code	FR	SC	SN	CY	DH	LBA
1	Standby	E2h or 96h	-	-	-	-	D	-
1	Standby Immediate	E0h or 94h	-	-	-	-	D	-
1	Translate Sector	87h	-	Y	Y	Y	Y	Y
1	Wear Level	F5h	-	-	-	-	Y	-
2	Write Buffer	E8h	-	-	-	-	D	-
2	Write Long Sector	32h or 33h	-	-	Y	Y	Y	Y
3	Write Multiple	C5h	-	Y	Y	Y	Y	Y
3	Write Multiple w/o Erase	CDh	-	Y	Y	Y	Y	Y
2	Write Sector(s)	30h or 31h	-	Y	Y	Y	Y	Y
2	Write Sector(s) w/o Erase	38h	-	Y	Y	Y	Y	Y
3	Write Verify	3Ch	-	Y	Y	Y	Y	Y

Table 96: ATA-Command Set

22.6.4.1 Check Power Mode

This command checks the power mode for the card. Using this command, the host can detect if the card is in or recovering from sleep mode / idle mode.

22.6.4.2 Erase Sector

This Command used to pre-erase and condition data sectors in advance of a “write without erase” or “write multiple without erase” command. There is no data transfer associated with this command.

22.6.4.3 Identify drive

This command enables the host to receive parameter information from the CF card.

22.6.4.4 Idle

This command causes the CF card to set BSY, enter in IDLE mode, clear the BSY and generate an interrupt. The Sector Count register is used to set the time delay required before going into power down mode.

22.6.4.5 Idle Immediate

This Command causes the CF card to set BSY, enter the IDLE mode, clear the BSY and generate an interrupt.

22.6.4.6 Read Buffer

This read buffer enables the host to read the current contents of the CF card storage buffer.

22.6.4.7 Read Sector(s)

The transfer begins at the sector specified in the Sector Number Register. Up to 256 sectors can be read depending upon the value specified in the Sector Count register. An interrupt is generated after the host reads each sector.

22.6.4.8 Read Multiple

This read-multiple command performs similar to the Read Sector command. Interrupts are not generated on every sector, but on the transfer of a block, which contains the number of sectors defined in the “Set Multiple” command.

22.6.4.9 Read Long Sector

This command performs similar to the read sector command except that it return 516 bytes of data instead of 512 bytes.

22.6.4.10 Read Verify Sector(s)

This command is identical to the read sector(s) command, except that the DRQ is never set and no data is transferred to the host. When the command is accepted, the CF card sets BSY.

22.6.4.11 Security Freeze Lock

This command sets the CF card into Frozen mode. After this command completion, any other command that update the CF card lock mode is rejected.

22.6.4.12 Security Set Password

This command requests a transfer of single sector from the host. This is a set of characters to be used as password.

22.6.4.13 Security Unlock

This command requests a transfer of single sector from the host. This is a set of characters to be used as password. The same characters should have been issued with “Security Set Password” command.

22.6.4.14 Set Sleep Mode

This command causes the CF card to set BSY, enter the Sleep mode, clear BSY and generate an interrupt. The Sector Count register is used to set the time delay required before going into sleep mode.

22.6.4.15 Standby Immediate

This command causes the CF card to set BSY, enter the Sleep mode, clear BSY and generate an interrupt immediately.

22.6.4.16 Write Buffer

This command enables the host to overwrite contents of the CF card sector buffers with any pattern desired.

22.6.4.17 Write Sector(s)

The transfer begins at the sector specified in the Sector Number Register. Up to 256 sectors can be written depending upon the value specified in the Sector Count register. An interrupt is generated after each sector is written.

22.6.4.18 Write Long Sector

This command writes 516 bytes in to the CF card.

22.6.4.19 Write Multiple Command

This command writes multiple sectors in to CF card.

22.6.4.20 Write Multiple without Erase

This command writes multiple sectors in to CF card without erasing the sectors.

22.6.4.21 Write Sector(s) without erase

This command writes from 1-256 sectors as specified in the sector count register with an implied erase before the write operation is not performed. It is similar to Write Sector(s) command.

22.6.4.22 Write Verify

This command is similar to the Write Sector(s) command, except each sector is verified immediately after being written.

23 VLYNQ

23.1 Introduction

VLYNQ is a serial, memory-mapped communication interface that enables the extension of internal bus architecture to one or more external physical devices. VLYNQ accomplishes this function by serializing bus transactions in one device, transferring the serialized transaction between devices via a VLYNQ port, and de-serializing the transaction in the external device. VLYNQ is a trademark of Texas Instruments. The general features of VLYNQ are as follows:

- Low Pin Count (as few as 3 signals)
- Simple Packet Based Transfer Protocol for Memory Mapped Access
 - o Write Request Packet
 - o Write Data Packet
 - o Read Request Packet
 - o Read Response Data Packet
 - o Interrupt Request Packet
- Symmetric Operation
 - o Transmit pins on first device connect to receive pins on second device and vice versa
 - o Request packets, response packets, and flow control information are all multiplexed and sent across the same physical pins.
 - o Supports both Host/Peripheral and Peer to Peer communication models
- Simple Block Code Packet Formatting (8b/10b)
- Supports In Band Flow Control
 - o No extra pins needed
 - o Allows receiver to momentarily throttle back transmitter when overflow is about to occur
 - o Uses built in special code capability of block code to seamlessly interleave flow control information with user data.
 - o Allows system designer to balance cost of data buffering versus performance.
- Supports multiple outstanding transactions
- Automatic packet formatting optimizations
- Internal loop-back mode

23.2 References

- VLYNQ Module Specification version 2.4 from Texas Instruments

23.3 Operation

23.3.1 Overview

VLYNQ supports both host-to-peripheral and peer-to-peer communication models. Figure 163 shows the block diagram for the VLYNQ module in DM320.

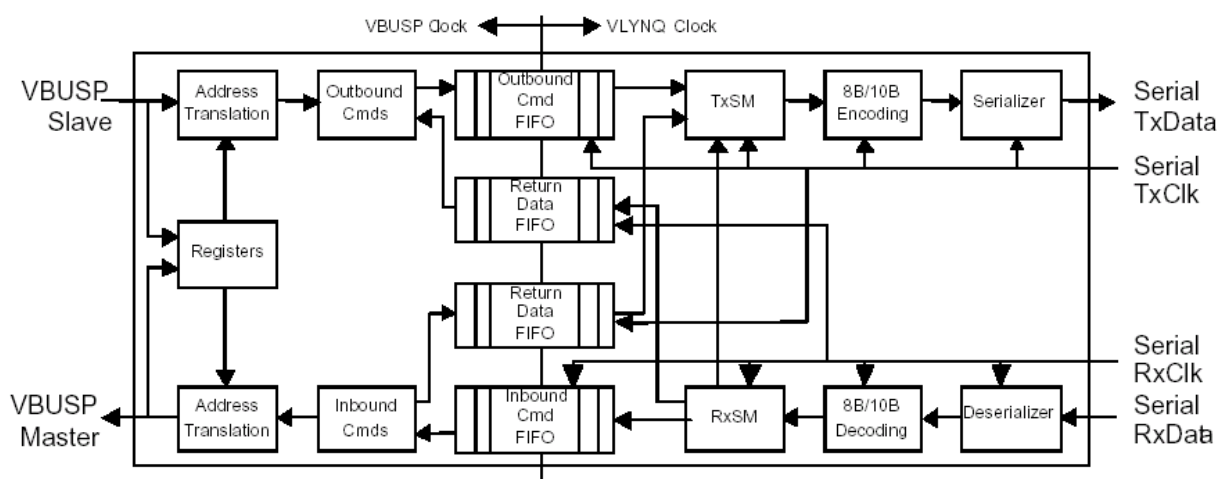


Figure 163: Block Diagram for VLYNQ Module

Each VLYNQ module implements two 32-bit, ARM system bus interfaces. A slave interface is required for transmit and control register access while one master interface is required for receive.

Each VLYNQ module can be remotely configured for appropriate address translation and operation.

Figure 164 shows a typical VLYNQ interconnection.

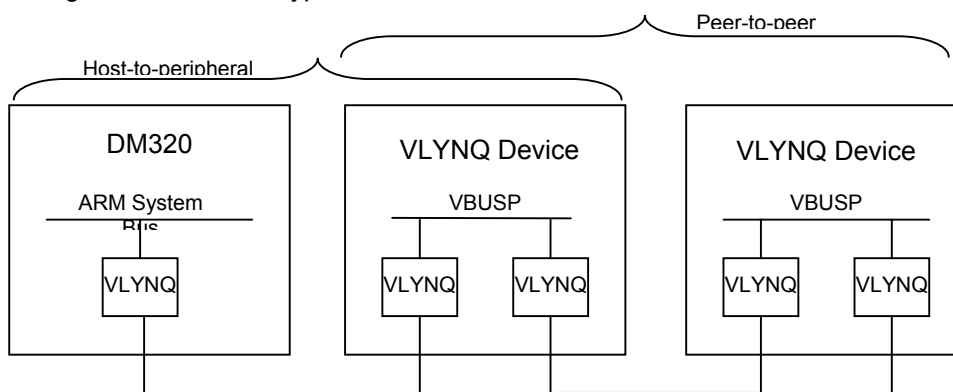


Figure 164: VLYNQ Interconnection

23.3.2 Initialization

In DM320, external pins for VLYNQ interface are multiplexed with GIOs. The following registers have to be configured for the proper operation of VLYNQ Controller:

Register	Setting	Operation
SEL0 register in CLKC *	Configure bit VLYNQ	Select VLYNQ input clock between PLLA and PLLB
DIV4 register in CLKC *	Configure the VLYNQ field	Sets the divisor for the PLLA or PLLB clock for input to VLYNQ
MOD2 register in CLKC *	Bit GIO and bit VLYNQ = 1	Enable VLYNQ & GIO clocks
FSEL1 register in GIO *	0xAAAA	GIO17-26 configured to operate as VLYNQ signals
FSEL2 register in GIO *	0x000A	GIO17-265 configured to operate as VLYNQ signals

*: Refer to “General Purpose Input Output” and “Clock Controller” chapters for more details about the registers.

Since VLYNQ devices are controlled solely over the serial interface (i.e. no local CPU exists), a reliable initialization sequence is necessary to establish a connection between two VLYNQ devices. The same sequence is used to recover from error conditions. When the VLYNQ is reset, an automatic initialization process is executed, which results in either an established VLYNQ connection or an error. The result of the initialization process is written to the LINK bit of the VL_STAT register. If the bit is set, the VLYNQ initialization sequence has completed successfully. Refer to VLYNQ spec for more details on auto-initialization.

23.3.2.1 Serial Interface Width Configuration

The width of the serial interface is automatically determined when exiting reset. Transmit and receive data pins need not be of the same width. The resulting interface width is written to the SIZEIN and SIZEOUT fields of the VL_STAT register.

23.3.3 Write Operation

- Write requests from the local ARM system bus port are written into an internal VLYNQ Tx FIFO
- Data in the FIFO is encapsulated in a Write Request packet.
- The address is translated, and the packet is transmitted to the remote device.
- The remote device subsequently receives the data in an internal Rx FIFO.
- On the remote device, write operation is then initiated after reading the address and data from the Rx FIFO.

23.3.4 Read Operation

- Read requests from the local ARM system bus port are written into an internal Tx FIFO similar to write requests.
- Data in the FIFO is encapsulated in a Read Request packet.
- The address is translated, and the packet is transmitted to the remote device.
- The remote device subsequently receives the data in an internal Rx FIFO.
- On the remote device, read operation is then initiated after reading the address from the Rx FIFO.
- Remote device puts the read data on its Tx FIFO. The data is transmitted from remote device to local device.
- On the local VLYNQ module, the received data is available on the Rx FIFO.
- Finally, the ready signal is asserted, and the read data is transferred on the ARM bus. This process is repeated for all data phases within the read burst.

In general, read operations should not be used for transfer of data packets since the serial nature of the interface results in very long latencies. The higher-level protocol should make use of write operations only. Read operations are supported, however, and may be used for less frequent register accesses since latency is not important in this case.

23.4 Address Translation

VLYNQ devices (V2USB etc) are memory mapped into the DM320 address space like other on-chip peripherals. Part of the initialization sequence is to enumerate all the VLYNQ devices into a coherent memory map for accessing each device.

VLYNQ is mapped into the DM320 address via address map registers. The transmit side has a contiguous memory map. The size of the local device address map is the sum of the 4 maps in the remote device.

VLYNQ allows each receive packet address to be translated into one of the four mapped regions. No restriction is placed on the size or offset of each mapped region except that each must be aligned to 32-bit words. This is accomplished via register pairs that define the size and offset of each mapped region (VL_RXMAPOFn and VL_RXMAPSZn registers).

DM320 can access the remote device address map by using the local device addresses. The VLYNQ module manages the translation of the local address to the remote addresses.

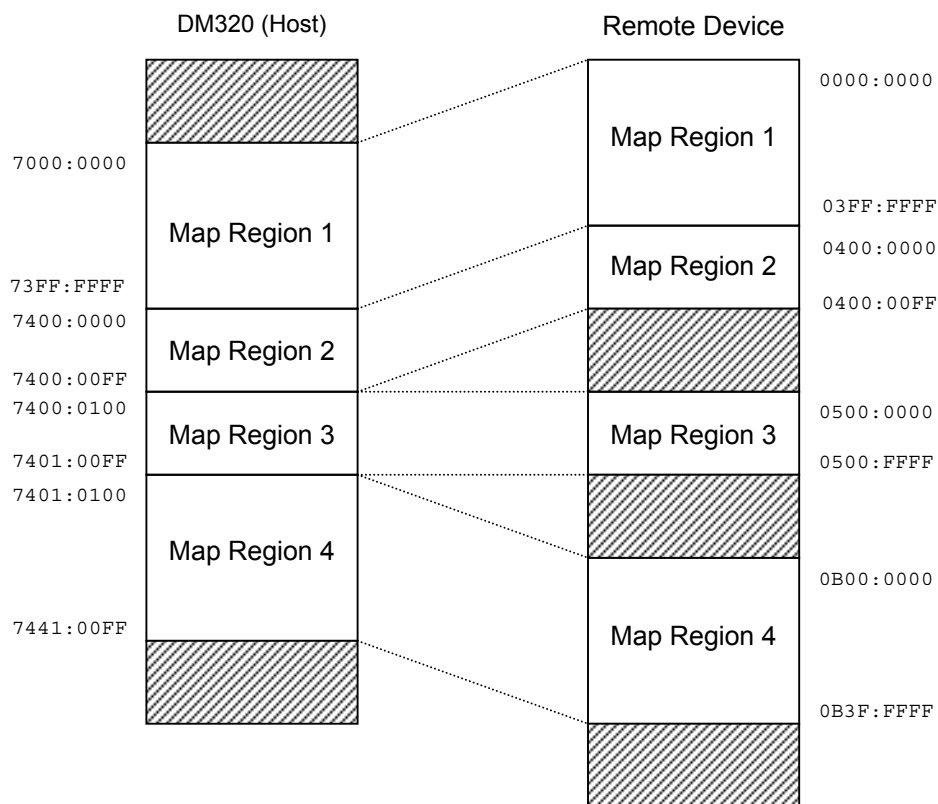


Figure 165: Address Mapping Example

When the local device transmits, the data is sent with an address offset from the VLYNQ base address.

The remote device uses the transmitted address to determine which map region is targeted. This is achieved by summing each memory size sequentially until the memory size is larger than address. The last memory size added is region targeted.

The example memory map in Figure 165 is used to illustrate the address translation used in the VLYNQ module. As shown in Table 97, VL_TXMAP register is written with the value of VLYNQ memory space on DM320. On ARM memory map, VLYNQ memory space starts from 0x70000000. VL_RXMAPSZ1 register is written with the size of the Map Region 1 on the remote device. The start offset address for the Map Region 1 is written to VL_RXMAPOF1 register.

Register	Value
VL_TXMAP	0x7000:0000
VL_RXMAPSZ1	0x0400:0000
VL_RXMAPOF1	0x0000:0000

Table 97: Address Translation Register

In this example, the local address 0x7000:0054 is translated to 0x0000:0054 on the remote VLYNQ device. Table 98 and code in Figure 166 describe the mechanism used for doing the address translation in VLYNQ module.

On Local VLYNQ Module (DM320)

	0x7000:0054	Initial address from slave interface on DM320
subtract	0x7000:0000	Tx Address Map Register (VL_TXMAP)
	0x0000:0054	Translated address to remote device via serial interface

On Remote VLYNQ Module

	0x0000:0054	Initial address from Rx serial interface
compare	0x0400:0000	Rx Address Map Size 1 Register (VL_RXMAPSZ1)
	0x0000:0054	
add	0x0000:0000	Rx Address Map Offset 1 Register (VL_RXMAPOF1)
	0x0000:0054	Translated address to remote device

Table 98: Address Translation Mechanism

```

If ((Rx Packet Address) < (Rx Address Map Size 1 Register))
{
    Translated Address = (Rx Packet Address) + (Rx Address Map Offset 1 Register)
}
else
{
    Rx Packet Address = (Rx Packet Address) - (Rx Address Map Size 1 Register)
    if ((Rx Packet Address) < (Rx Address Map Size 2 Register))
    {
        Translated Address = (Rx Packet Address) + (Rx Address Map Offset 2 Register)
    }
    else
    {
        Rx Packet Address = (Rx Packet Address) - (Rx Address Map Size 2 Register)
        if ((Rx Packet Address) < (Rx Address Map Size 3 Register))
        {
            Translated Address = (Rx Packet Address) + (Rx Address Map Offset 3 Register)
        }
        else
        {
            Rx Packet Address = (Rx Packet Address) - (Rx Address Map Size 3 Register)
            if ((Rx Packet Address) < (Rx Address Map Size 4 Register))
            {
                Translated Address = (Rx Packet Address) + (Rx Address Map Offset 4 Register)
            }
            else
            {
                Rx Packet Address = 0x0
                Generated address translation interrupt
            }
        }
        /* end of (Rx Packet Address) < (Rx Address Map Size 4 Register) */
    }
    /* end of (Rx Packet Address) < (Rx Address Map Size 3 Register) */
}
/* end of (Rx Packet Address) < (Rx Address Map Size 2 Register) */
}
/* end of (Rx Packet Address) < (Rx Address Map Size 1 Register) */

```

Figure 166: Address Translation on Remote Device

23.5 Interrupts

VLYNQ module in DM320 is designed to provide maximum flexibility for generation, detection, and forwarding interrupt information to ARM. Interrupts are generated when the VL_INTPND register is written.

23.5.1 Interrupt Interface

The physical VLYNQ interrupt interface consists of an 8-pin input interrupt vector, and an output pin. Section 23.5.2 explains the interrupt generation operation. Unlike most of the VLYNQ devices, in DM320, the 8-pin input interface is not connected to any local devices.

23.5.2 Interrupt Detection and Generation

Remote Device Interrupt:

When an interrupt packet is received on the serial interface from a remote device, the interrupt status is extracted from the packet and written to the register indicated by the VL_INTPTR register. This register may be anywhere in ARM memory mapped space. For example, the VL_INTPTR register may point to the memory-mapped location of its own VL_INTPND register. Once the VL_INTPND register is written, interrupt is generated. The interrupt status is forwarded in one of two ways depending on the state of the INTLO bit in the VL_CTRL. If INTLO bit is '0', the contents of the VL_INTPND register are inserted in an Interrupt packet that is sent over the serial interface. When packet transmission has completed, the VL_INTPND register is cleared. If INTLOCAL bit is '1', the logical OR of all of the bits in the VL_INTST register is driven onto the interrupt output pin. This pin is connected to the interrupt controller module (INT_VLYNQ). When VL_INTST register is cleared, the interrupt is de-asserted.

23.6 Clocking

VLYNQ's clock direction and clock divider are software configurable through the register interface. VLYNQ can operate on either an internally or externally generated clock. Set the CLKDIR bit in the VL_CTRL register to '0' for the clock pin to be sourced externally. When CLKDIR bit is set to '1', the clock pin is an output, sourced from a divided down version of the VLYNQ input clock. VLYNQ input clock is configured as discussed in section 23.3.2.

The CLKDIV field of the VL_CTRL register defines the division factor of the input clock. In this case the output clock frequency on VLYNQ serial bus will be **(VLYNQ input clock) / (1 + CLKDIV)**

In the case that the VLYNQ initialization process has not completed successfully (i.e., the LINK bit of the VL_STAT register is '0'), software may attempt to toggle the clock direction (CLKDIR bit in VL_CTRL register) to cause the link to occur. By default, the clock is externally sourced.

23.7 Flow Control

The VLYNQ module includes flow control features. No configuration is necessary to enable this feature, as this is inherent in the VLYNQ protocol.

23.8 Register Description

23.8.1 VLYNQ Controller Interface Register Map (VLYNQ)

Address	Register	Description
0006:0300	VL_ID	VLYNQ Local Revision/ID Register
0006:0304	VL_CTRL	VLYNQ Local Control Register
0006:0308	VL_STAT	VLYNQ Local Status Register
0006:030C	VL_INTPRI	VLYNQ Local Interrupt Priority Vector Status/Clear Register
0006:0310	VL_INTST	VLYNQ Local Interrupt Status/Clear Register
0006:0314	VL_INTPND	VLYNQ Local Interrupt Pending/Set Register
0006:0318	VL_INTPTR	VLYNQ Local Interrupt Pointer Register
0006:031C	VL_TXMAP	VLYNQ Local Tx Address Map
0006:0320	VL_RXMAPSZ1	VLYNQ Local Rx Address Map Size 1
0006:0324	VL_RXMAPOF1	VLYNQ Local Rx Address Map Offset 1
0006:0328	VL_RXMAPSZ2	VLYNQ Local Rx Address Map Size 2
0006:032C	VL_RXMAPOF2	VLYNQ Local Rx Address Map Offset 2
0006:0330	VL_RXMAPSZ3	VLYNQ Local Rx Address Map Size 3
0006:0334	VL_RXMAPOF3	VLYNQ Local Rx Address Map Offset 3
0006:0338	VL_RXMAPSZ4	VLYNQ Local Rx Address Map Size 4
0006:033C	VL_RXMAPOF4	VLYNQ Local Rx Address Map Offset 4
0006:0340	VL_CHIPVER	VLYNQ Local Chip Version Register
0006:0344	VL_AUTONEG	VLYNQ Local Auto Negotiation Register
0006:0348	VL_MANNEG	VLYNQ Local Manual Negotiation Register
0006:034C	VL_NEGSTAT	VLYNQ Local Negotiation Status Register
0006:035C	VL_ENDIAN	VLYNQ Local Endian Register
0006:0360	VL_INTVEC30	VLYNQ Local Interrupt Vector 3-0
0006:0364	VL_INTVEC74	VLYNQ Local Interrupt Vector 7-4

23.8.2 VL_ID

VLYNQ Local Revision/ID Register

VL_ID	0006:0300	offset:	0x00	default:	0x00010204
-------	-----------	---------	------	----------	------------

[illegible]

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	REVM_AJ7	REVM_AJ6	REVM_AJ5	REVM_AJ4	REVM_AJ3	REVM_AJ2	REVM_AJ1	REVM_AJ0	REVM_N7	REVM_N6	REVM_N5	REVM_N4	REVM_N3	REVM_N2	REVM_N1	REVM_N0
R/W	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R
Default	0	0	0	0	0	0	1	0	0	0	0	0	0	1	0	0

[illegible]

23.8.3 VL_CTRL

VLYNQ Local Control Register

VL_CTRL	0006:0304				offset: 0x04				default: 0x00000000							
Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Name	PMEN	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	CLKDIV[2]	CLKDIV[1]	CLKDIV[0]
R/W	R/W	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW
Default	0	-	-	-	-	-	-	-	-	-	-	-	-	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	CLKDIR	INTLO	INTEN	INTVEC[4]	INTVEC[3]	INTVEC[2]	INTVEC[1]	INTVEC[0]	INT2CFG	RSV	RSV	RSV	RSV	AOPTD	ILOOP	RESET
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	-	-	-	-	RW	RW	RW
Default	0	0	0	0	0	0	0	0	0	-	-	-	-	0	0	0

Bit	Name	Reset Value	R/W	Function
31	PMEN	0	R/W	<u>Power Management Enable</u> 0: No effect 1: If VLYNQ_CLK is set as an output, will go into idle mode with a value of 1 when there is no traffic over the serial bus
30:19	RSV			<u>Reserved</u>
18:16	CLKDIV	0	R/W	<u>Serial Clock Divider</u> Division factor for the VLYNQ_CLK pin when the pin is sourced from <i>vlynq_clk_ref</i> (CLKDIR set to 1). The output clock frequency will be equal to <i>vlynq_clk_ref</i> /(1 + CLKDIV)
15	CLKDIR	0	R/W	<u>Serial Clock Direction</u> 0: VLYNQ_CLK is sourced externally 1: VLYNQ_CLK is sourced internally from a divided down version of <i>vlynq_clk_ref</i>
14	INTLO	0	R/W	<u>Interrupt Local</u> 0: Interrupts are forwarded out of the serial interface to the remote device 1: Interrupts are posted in the Interrupt Status/Clear register and <i>vlynq_int_o</i> pin is asserted.
13	INTEN	0	R/W	<u>Interrupt Enable</u> 0: Interrupts are not posted 1: Interrupts posted to the Interrupt Pending/Set Register
12:8	INTVEC	0	R/W	<u>Interrupt Vector</u> Indicates which bit in the interrupt Pending/Set register is set for VLYNQ module interrupts
7	INT2CFG	0	R/W	<u>Interrupt to Configuration Register</u> 0: Interrupt Pending/Set register is written directly with the status contained in interrupt packets 1: Least significant 8 bits of the Interrupt Pointer Register are used to point to a local configuration register (typically Interrupt Pending/Set register)
6:3	RSV			<u>Reserved</u>
2	AOPTD	0	R/W	<u>Address Optimization Disable</u> 0: Address Optimization is Enabled 1: Address Optimization is Disabled
1	ILOOP	0	R/W	<u>Internal Loop</u> 0: No effect 1: Serial Transmit data to the serial receive data
0	RESET	0	R/W	<u>Reset</u> 0: No reset 1: Reset internal state machine, serial interface is disabled, link is lost and all output pins are disabled

23.8.4 VL_STAT

VLYNQ Local Status Register

VL_STAT	0006:0308				offset: 0x08				default: 0x00000000							
Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Name	RSV	RSV	RSV	RSV	SIZEIN[3]	SIZEIN[2]	SIZEIN[1]	SIZEIN[0]	SIZEOUT[3]	SIZEOUT[2]	SIZEOUT[1]	SIZEOUT[0]	RSV	RSV	RSV	RSV
R/W	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	-	-	-	-
Default	-	-	-	-	0	0	0	0	0	0	0	0	-	-	-	-

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	IFLOW	OFLOW	ERROR	ERROR	NFE3	NFE2	NFE1	NFE0	SPEND	MPEND	LINK
R/W	-	-	-	-	-	R	R	R/W	R/W	R	R	R	R	R	R	R
Default	-	-	-	-	-	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
31:28	RSV			<u>Reserved</u>
27:24	SIZEIN	0	R/W	<u>Inbound Serial Data Size in bits</u>
23:20	SIZEOUT	0	R/W	<u>Outbound Serial Data Size in bits</u>
19:11	RSV			<u>Reserved</u>
				<u>Inbound Flow Control</u>
				0: Disable flow control
				1: Enable flow control
10	IFLOW	0	R	Indicates that a flow control enable request has been received and has stalled transmit until a flow control disable request is received
				<u>Outbound Flow Control</u>
				0: Disable flow control
				1: Enable flow control
9	OFLOW	0	R	Indicate that the internal flow control threshold has been reached and a flow control enable request has been sent to the remote device
				<u>Remote Error</u>
				This bit indicates that a downstream VLYNQ module has detected a packet error. This bit is set when a /E/ (error) is received from the serial interface. When set, this bit will cause an interrupt if enabled in the INTEN bit of VL_CTRL register.
				Rd 0: No error
				Rd 1: Packet Error status received
				Wr 0: No effect
				Wr 1 : Clear ERROR bit
				<u>Local Error</u>
				This bit indicates that an inbound packet contains an error. When set, this bit will cause an interrupt if enabled in the INTEN bit of VL_CTRL register.
				Rd 0: No error
				Rd 1: Packet Error status received
				Wr 0: No effect
				Wr 1 : Clear LERROR bit

6	NFE3	0	R	<u>FIFO3 Not Empty</u> 0 : Slave Command FIFO empty 1 : Slave Command FIFO Not Empty
5	NFE2	0	R	<u>FIFO2 Not Empty</u> 0 : Slave Data FIFO empty 1 : Slave Data FIFO not empty
4	NFE1	0	R	<u>FIFO1 Not Empty</u> 0 : Master Command FIFO empty 1 : Master Command FIFO not empty
3	NFE0	0	R	<u>FIFO0 Not Empty</u> 0 : Master Data FIFO empty 1 : Master Data FIFO not empty
2	SPEND	0	R	<u>Pending Slave Requests</u> 0 : No pending slave requests 1 : Request has been detected on the Tx VBUSP slave interface
1	MPEND	0	R	<u>Pending Master Requests</u> 0 : No pending master requests 1 : Request has been asserted on the Rx VBUSP master interface
0	LINK	0	R	<u>Link</u> 0: Serial initialization not done 1 : Serial Interface initialization sequence has completed successfully

23.8.5 VL_INTPRI

VLINQ Local Interrupt Priority Vector Status/Clear Register

[illegible]

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	INTST[4]	INTST[3]	INTST[2]	INTST[1]	INTST[0]
R/W	-	-	-	-	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	-	-	-	-	-	-	0	0	0	0	0

[illegible]

23.8.6 VL INTST

VLYNQ Local Interrupt Status/Clear Register

VL_INTST	0006:0310	offset: 0x10	default: 0x00000000
----------	-----------	--------------	---------------------

[illegible][illegible][illegible]

23.8.7 VL_INTPND

VLYNQ Local Interrupt Pending/Set Register

[illegible][illegible]

23.8.9 VL_TXMAP

VLYNQ Local Tx Address Map

VL_TXMAP	0006:031C	offset: 0x1C	default: 0x00000000
----------	-----------	--------------	---------------------

[illegible][illegible][illegible]

23.8.10 VL_RXMAPSZ1

VLYNQ Local Rx Address Map Size 1

[illegible][illegible]

23.8.11 VL_RXMAPOF1

VLYNQ Local Rx Address Map Offset 1

[illegible][illegible]

23.8.12 VL_RXMAPSZ2

VLYNQ Local Rx Address Map Size 2

VL RXMAPSZ2	0006:0328	offset: 0x28	default: 0x00000000
-------------	-----------	--------------	---------------------

[illegible][illegible][illegible]

23.8.13 VL_RXMAPOF2

VLYNQ Local Rx Address Map Offset 2

VL RXMAPOF2	0006:032C	offset: 0x2C	default: 0x00000000
-------------	-----------	--------------	---------------------

[illegible][illegible]

23.8.14 VL_RXMAPSZ3

VLYNQ Local Rx Address Map Size 3

VL_RXMAPSZ3	0006:0330	offset: 0x30	default: 0x00000000
-------------	-----------	--------------	---------------------

[illegible][illegible][illegible]

23.8.15 VL_RXMAPOF3

VLYNQ Local Rx Address Map Offset 3

[illegible][illegible]

23.8.16 VL_RXMAPSZ4

VLYNQ Local Rx Address Map Size 4

VL_RXMAPSZ4	0006:0338	offset: 0x38	default: 0x00000000
-------------	-----------	--------------	---------------------

[illegible][illegible][illegible]

23.8.17 VL_RXMAPOF4

VLYNQ Local Rx Address Map Offset 4

[illegible][illegible]

23.8.18 VL_CHIPVER

VLYNQ Local Chip Version Register

VL_CHIPVER	0006:0340	offset: 0x40	default: 0x00100013
------------	-----------	--------------	---------------------

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Name	DEVREV[15]	DEVREV[14]	DEVREV[13]	DEVREV[12]	DEVREV[11]	DEVREV[10]	DEVREV[9]	DEVREV[8]	DEVREV[7]	DEVREV[6]	DEVREV[5]	DEVREV[4]	DEVREV[3]	DEVREV[2]	DEVREV[1]	DEVREV[0]
R/W	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R
Default	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	DEVID[15]	DEVID[14]	DEVID[13]	DEVID[12]	DEVID[11]	DEVID[10]	DEVID[9]	DEVID[8]	DEVID[7]	DEVID[6]	DEVID[5]	DEVID[4]	DEVID[3]	DEVID[2]	DEVID[1]	DEVID[0]
R/W	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R	R
Default	0	0	0	0	0	0	0	0	0	0	0	1	0	0	1	1

[illegible]

23.8.19 VL_AUTONEG

VLYNQ Local Auto Negotiation Register

VL AUTONEG	0006:0344	offset: 0x44	default: 0x00010000
------------	-----------	--------------	---------------------

[illegible][illegible][illegible]

23.8.21 VL_NEGSTAT

VLYNQ Local Negotiation Status Register

VL_NEGSTAT	0006:034C				offset: 0x4C				default: 0x00000001							
Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	MODE
R/W	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	R
Default	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	1

Bit	Name	Reset Value	R/W	Function
31:1	RSV			Reserved
0	MODE	1	R	<u>Mode Mask</u> A '1' in a particular bit position indicates that the device supports the mode corresponding with the bit number. A mode-0-only device would show 00000001b, a mode-1-only device would show 00000010b, etc. The default mode=0 is covered in the Packet Format section.

23.8.22 VL_ENDIAN

VLYNQ Local Endian Register

VL_ENDIAN	0006:035C	offset: 0x5C	default: 0x2F2F2F2F
-----------	-----------	--------------	---------------------

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Name	END[31]	END[30]	END[29]	END[28]	END[27]	END[26]	END[25]	END[24]	END[23]	END[22]	END[21]	END[20]	END[19]	END[18]	END[17]	END[16]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	1	0	1	1	1	1	0	0	1	0	1	1	1	1

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	END[15]	END[14]	END[13]	END[12]	END[11]	END[10]	END[9]	END[8]	END[7]	END[6]	END[5]	END[4]	END[3]	END[2]	END[1]	END[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	1	0	1	1	1	1	0	0	1	0	1	1	1	1

[illegible]

23.8.23 VL_INTVEC30

VLYNQ Local Interrupt Vector 3-0

VL_INTVEC30			0006:0360			offset: 0x60						default: 0x00000000				
Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Name	INTEN3	INTTYPE3	INTPOL3	INTVEC3[4]	INTVEC3[3]	INTVEC3[2]	INTVEC3[1]	INTVEC3[0]	INTEN2	INTTYPE2	INTPOL2	INTVEC2[4]	INTVEC2[3]	INTVEC2[2]	INTVEC2[1]	INTVEC2[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	INTEN1	INTTYPE1	INTPOL1	INTVEC1[4]	INTVEC1[3]	INTVEC1[2]	INTVEC1[1]	INTVEC1[0]	INTEN0	INTTYPE0	INTPOL0	INTVEC0[4]	INTVEC0[3]	INTVEC0[2]	INTVEC0[1]	INTVEC0[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
31	INTEN3	0	R/W	<u>Interrupt Enable 3</u> When set, this bit indicates that interrupts detected on the vlynq_int_i[3] pin should be written to the Interrupt Pending/Set Register which will subsequently generate an interrupt depending on the status of the intlocal bit in the Control Register
30	INTTYPE3	0	R/W	<u>Interrupt Type 3</u> When set, this bit indicates that the vlynq_int_i[3] interrupt is pulsed. When clear, this bit indicates that vlynq_int_i[3] is level sensitive
29	INTPOL3	0	R/W	<u>Interrupt Polarity 3</u> When set, this bit indicates that the vlynq_int_i[3] interrupt is active low. When clear, this bit indicates that vlynq_int_i[3] is active high
28:24	INTVEC3	0	R/W	<u>Interrupt Vector 3</u> This field maps the vlynq_int_i[3] pin to a bit in the Interrupt Pending/Set Register
23	INTEN2	0	R/W	<u>Interrupt Enable 2</u> When set, this bit indicates that interrupts detected on the vlynq_int_i[2] pin should be written to the Interrupt Pending/Set Register which will subsequently generate an interrupt depending on the status of the intlocal bit in the Control Register
22	INTTYPE2	0	R/W	<u>Interrupt Type 2</u> When set, this bit indicates that the vlynq_int_i[2] interrupt is pulsed. When clear, this bit indicates that vlynq_int_i[2] is level sensitive
21	INTPOL2	0	R/W	<u>Interrupt Polarity 2</u> When set, this bit indicates that the vlynq_int_i[2] interrupt is active low. When clear, this bit indicates that vlynq_int_i[2] is active high
20:16	INTVEC2	0	R/W	<u>Interrupt Vector 2</u> This field maps the vlynq_int_i[2] pin to a bit in the Interrupt Pending/Set Register
15	INTEN1	0	R/W	<u>Interrupt Enable 1</u> When set, this bit indicates that interrupts detected on the vlynq_int_i[1] pin should be written to the Interrupt Pending/Set Register which will subsequently generate an interrupt depending on the status of the intlocal bit in the Control Register
14	INTTYPE1	0	R/W	<u>Interrupt Type 1</u> When set, this bit indicates that the vlynq_int_i[1] interrupt is pulsed. When clear, this bit indicates that vlynq_int_i[1] is level sensitive

[illegible]

23.8.24 VL_INTVEC74

VLYNQ Local Interrupt Vector 7-4

VL_INTVEC74				0006:0364				offset: 0x64				default: 0x00000000				
Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Name	INTEN7	INTTYPE7	INTPOL7	INTVEC7[4]	INTVEC7[3]	INTVEC7[2]	INTVEC7[1]	INTVEC7[0]	INTEN6	INTTYPE6	INTPOL6	INTVEC6[4]	INTVEC6[3]	INTVEC6[2]	INTVEC6[1]	INTVEC6[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	INTEN5	INTTYPE5	INTPOL5	INTVEC5[4]	INTVEC5[3]	INTVEC5[2]	INTVEC5[1]	INTVEC5[0]	INTEN4	INTTYPE4	INTPOL4	INTVEC4[4]	INTVEC4[3]	INTVEC4[2]	INTVEC4[1]	INTVEC4[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
31	INTEN7	0	R/W	<u>Interrupt Enable 7</u> When set, this bit indicates that interrupts detected on the vlynq_int_i[7] pin should be written to the Interrupt Pending/Set Register which will subsequently generate an interrupt depending on the status of the intlocal bit in the Control Register
30	INTTYPE7	0	R/W	<u>Interrupt Type 7</u> When set, this bit indicates that the vlynq_int_i[7] interrupt is pulsed. When clear, this bit indicates that vlynq_int_i[7] is level sensitive
29	INTPOL7	0	R/W	<u>Interrupt Polarity 7</u> When set, this bit indicates that the vlynq_int_i[7] interrupt is active low. When clear, this bit indicates that vlynq_int_i[7] is active high
28:24	INTVEC7	0	R/W	<u>Interrupt Vector 7</u> This field maps the vlynq_int_i[7] pin to a bit in the Interrupt Pending/Set Register
23	INTEN6	0	R/W	<u>Interrupt Enable 6</u> When set, this bit indicates that interrupts detected on the vlynq_int_i[6] pin should be written to the Interrupt Pending/Set Register which will subsequently generate an interrupt depending on the status of the intlocal bit in the Control Register
22	INTTYPE6	0	R/W	<u>Interrupt Type 6</u> When set, this bit indicates that the vlynq_int_i[6] interrupt is pulsed. When clear, this bit indicates that vlynq_int_i[6] is level sensitive
21	INTPOL6	0	R/W	<u>Interrupt Polarity 6</u> When set, this bit indicates that the vlynq_int_i[6] interrupt is active low. When clear, this bit indicates that vlynq_int_i[6] is active high
20:16	INTVEC6	0	R/W	<u>Interrupt Vector 6</u> This field maps the vlynq_int_i[6] pin to a bit in the Interrupt Pending/Set Register
15	INTEN5	0	R/W	<u>Interrupt Enable 5</u> When set, this bit indicates that interrupts detected on the vlynq_int_i[5] pin should be written to the Interrupt Pending/Set Register which will subsequently generate an interrupt depending on the status of the intlocal bit in the Control Register
14	INTTYPE5	0	R/W	<u>Interrupt Type 5</u> When set, this bit indicates that the vlynq_int_i[5] interrupt is pulsed. When clear, this bit indicates that vlynq_int_i[5] is level sensitive

[illegible]

24 External Host Interface (EHIF)

24.1 Introduction

The external host CPU interface module (EHIF), shown in Figure 167, is a 16-bit parallel port used to interface a host CPU to the DM320. By setting the EXTHOST pin to 'High', the DM320 is set in external CPU mode and ARM_A22 - ARM_A0, EM_CE0, EM_WE and EM_OE become input pins. The EM_WAIT signal becomes an output pin. The data bus width is fixed to 16-bits and uses ARM_D15 - ARM_D0. Both read and write are half-word (16 bit) access.

The EHIF also supports bus multiplex access mode. In this mode, address and data is transferred via ARM_D15 - ARM_D0 and ARM_A22 is used to control the multiplex of data and address. ARM_A21 - ARM_A0 is not used. The function of other control signals including EM_OE, EM_WE, EM_CE0 and EM_WAIT remain unchanged.

Even in external CPU mode, the ARM9 in DM320 can access all peripheral registers, internal memories and external memories. Data can be exchanged between the host CPU and the DM320 throughout the on chip memory such as ARM internal memory and DSP internal memory via EHIF module. The SDRAM, which is connected externally, is also accessible via EHIF and SDRAM controller.

The EHIF also provides the capability to access memory during reset. Please note that this reset is only for the ARM core and does not mean the RSTN pin. All portions of the chip except the ARM core are out of reset when data or application code is loaded via the EHIF, and the application can be initialized through the boot-up option by setting the BTSEL pins appropriately. Refer chapter on "ARM Boot Up Modes" for details.

Access from a host device can be performed dynamically by enabling EM_WAIT output for the host device to monitor the status of the DM320.

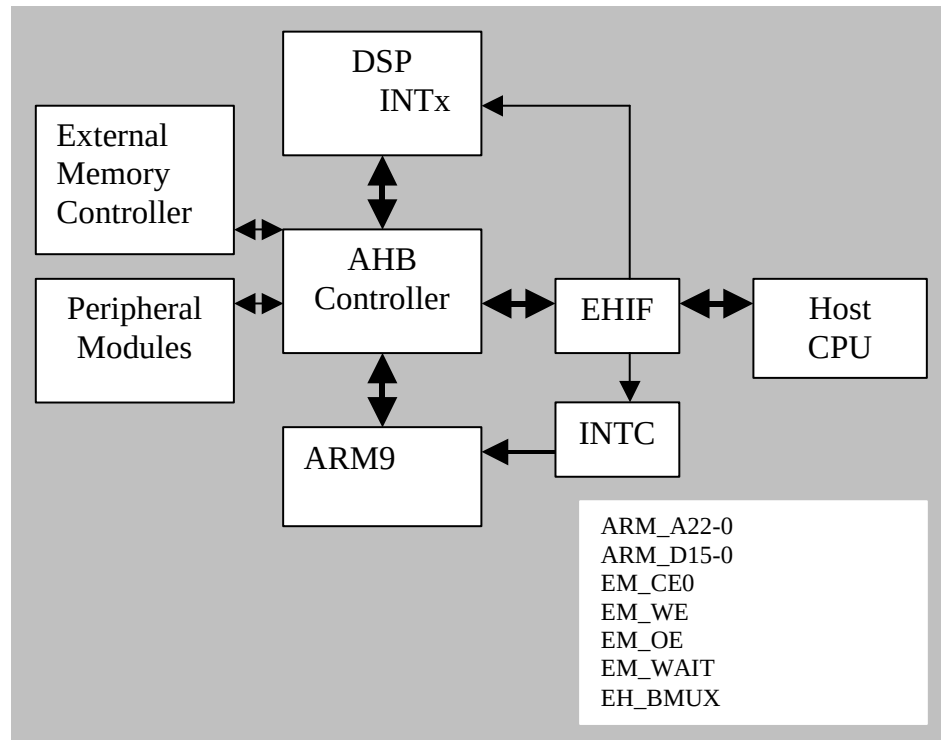


Figure 167: EHIF Block Diagram

24.2 Features

- The data bus width for both read and write operation is fixed at 16-bits (half-word) and ARM_D15 - ARM_D0 data lines are used for EHIF access.
- Access from the host device can be performed dynamically by enabling the EM_WAIT output for the host device to monitor the status of the DM320 ('0':Busy,'1':ready). WAITEN bit in EHIFCTL register of the External CPU Interface must be set in advance to enable the EM_WAIT output.
- The access time seen from the host device varies depending on ARM, DSP AND SDRAM clock frequency settings.
- 32 byte Burst / Continuous Burst access to all memory address is supported via sixteen 16bit data registers.
- Host device can issue an interrupt signal to the ARM by setting the register INTCTL
- Host device can issue an interrupt signal to the DSP by setting the register INTCTL

24.3 Memory Space

The memory areas that can be accessed from an external host controller are ARM internal memory (24 Kbytes), SDRAM (16,128Kbytes), and DSP internal memory (128 Kbytes) and Peripheral modules (4kbytes). However, the memory map addresses differ from the usual ARM memory map. Table 99 shows the memory map of DM320 as seen from the external CPU.

Note: For accessing single/random location within DM320 from an external host CPU follow the External Host Memory Map. In case of single/continuous burst transfer from External Host CPU to DM320 follow ARM Memory Map.

Access Type	Memory Area	Ext Host Memory Map		ARM Memory Map		Size (Bytes)
		Start Add	End Add	Start Add	End Add	
Direct	EHIF Registers	0x00:0000	0x00:007F	N/A	N/A	128
Direct	Peripherals	0x00:1000	0x00:1FFF	0x0003:0000	0x0003:0FFF	4K
Direct	ARM TCM	0x00:8000	0x00:DFFF	0x0002:0000	0x0002: 5FFF	24K
Direct	DSP Internal Memory	0x02:0000	0x03:FFFF	0x0004:0000	0x0005:FFFF	128K
Direct	SDRAM	0x04:0000	0xFF:FFFF	0x0090:0000	0x018B:FFFF	16,128K
Register	All (Burst Transfer)	--	--	0x0000:0000	0x1FFF:FFFF	512M

Table 99 Memory Map

ARM and DSP internal memory accesses on the DM320 are in shared-access mode, meaning both the DM320 and the host CPU can access memory. Asynchronous host accesses are resynchronized internally, and in the event that the ARM9 in DM320 and the host CPU request access to the same memory block, the host CPU has access priority. But note that the host CPU doesn't suspend an access by the ARM9 intermittently, although the host CPU has higher priority than the ARM9. At that time, the host CPU waits for completion of the access by ARM9. The priority takes effect only when both the devices start to access the same memory block at the same time. The access priority to the external SDRAM is configured by priority registers in SDRAM controller. The EM_WAIT pin can provide handshaking to the host during memory access.

Note: In case DM270 is used as host, then half-word addressing has to be used. Please note that the memory map given in Table 99 shows byte-addresses for different memory regions.

24.4 Bus Multiplex Access Mode

The EHIF supports address/data bus multiplex access. Setting the EH_BMUX pin to high configures the EHIF interface in this mode.

In this mode, both address and data are transferred via ARM_D15 - ARM_D0. ARM_A21-ARM_A0 address bus is not used in this mode. ARM_A22 pin is used

to indicate whether it is address or data transfer. The address is 23bit, so two address accesses need to be done consecutively.

Memory map is not changed even in bus multiplex mode. Refer to *Table 99* for details of memory map.

The following steps show the procedure to transmit or receive data, to/from DM320, in this mode.

1. Send the start address to/from which data is to be written/read.

To transfer the address, perform two write accesses consecutively in case of either operation, whether you want to read data from DM320 or write data to DM320. The address is transferred via ARM_D bus, and the MSBs (bit 22-16) must be transmitted first and the LSBs (bit15-0) must be transmitted next. During these accesses ARM_A22 must be high to indicate that the address is transferred. ARM_D15-ARM_D7 contents are ignored when the MSBs of the address are written.
2. Transfer Data.

Write or read 16bit data via ARM_D15-ARM_D0. ARM_A22 needs to be low to indicate that the data is transferred. After the access, the target address is linearly incremented automatically. So bytes of data can be consecutively transferred to sequential address without sending the address through the ARM_D bus between each byte of data. Note that if the address reaches to the maximum value (0x3FFFFFF), the address is reset to 0x000000 for the next 16bit transfer.
3. Repeat from step 1 onwards when address is to be changed.

24.5 Access Time

The External Host interface timings are referenced to ARM clock of the DM320. The access time seen from the host CPU varies depending on ARM, DSP and SDRAM clock frequency settings. Moreover, wait cycles will be inserted if there is other access by other AHB bus master to same memory block. The number of cycles of the ARM clock is shown below.

Memory Access		Timing
EHIF registers	Write	2 cycles + α
	Read	2 cycles + α
ARM Internal memory read	Write	3 cycles+ α
	Read	3 cycles + α
SDRAM single access/DSP memory	Write	(3 + n) cycles + α
	Read	(3 + n) cycles + α
α : Input/Output Delay in ns n: Peripheral Wait Cycles		

If a back-to-back read is performed, EM_OE must be high when RTRG bit in EHIFCTL is '1', and EM_CE0 must be high when the RTRG bit in EHIFCTL is '0'.

For back-to-back writes, EM_WE must be high.

Access from a host CPU to the DM320 can be performed dynamically by using the DM320's EM_WAIT output. The host CPU can monitor the status of the EM_WAIT signal ('0': Busy, '1': Ready) and will wait to transfer data until the

DM320 is ready. To enable the EM_WAIT output signal for dynamic access, set the bit WAITEN to '1' in the EHIFCTL register. Note that EM_WAIT is not asserted when an access takes only two cycles even if WAITEN bit is '1'. Also the host CPU can monitor the status of the AHBBSY bit in the EHIFSTS register.

24.6 Interrupts in External HOST Mode

- Host device can issue an interrupt signal to the ARM or DSP in DM320 by setting the register INTCTL appropriately.
- In case the slave DM320 needs to interrupt external HOST, an arbitrary GIO pin can be used for this purpose.

NOTE: In case a GIO pin from DM320 is used to interrupt the external host CPU, the GIO pin has to be configured to act like an interrupt pin. Refer to chapter on GIO and Interrupt Controller for more details.

24.7 SDRAM Access

The host CPU can randomly access SDRAM located in memory region 0x40000 – 0xFFFFF (byte address in external CPU memory map). When SDRAM is accessed in single access mode, extra wait cycles are added. The host CPU can check the status of SDRAM access using EM_WAIT pin as shown in Figure 168.

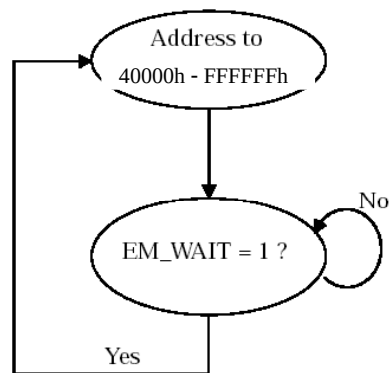


Figure 168: SDRAM Access

The control signal details for SDRAM access are shown in, Figure 169, and Figure 170.

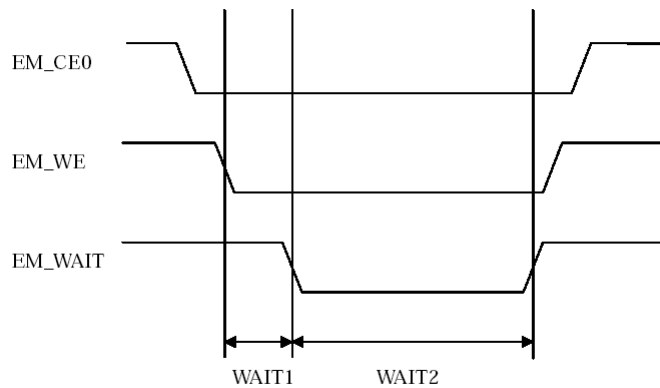


Figure 169: EHIF Control Signals for Write

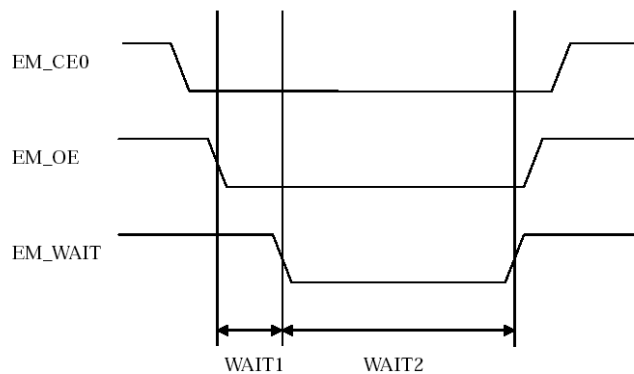


Figure 170: EHIF Control Signals for Read

24.8 Burst Access

Single Burst and Continuous Burst access to all memories is supported. Sixteen 16-bit data registers are used for Single/Continuous Burst Access. When in Continuous Burst transfer mode, data of the specified length in 32-byte unit are transferred to and from memory.

24.9 Single Burst Access

Transfer to and from memory is performed in 16-bit word units as shown in Figure 171. There is a control register in the EHIF for burst access as well as sixteen 16-bit data registers to store the data from 16-word burst transfer.

To write into the memory, data needs to be set in these data registers (BRSTBUF0 to BRSTBUF15). Destination memory address is written in the buffer address registers BRSTADR1 and BRSTADR2. Write command can be written in BRSTCTL1 register. Memory write is performed when the SBRSWR bit in BRSTCTL1 register is set.

To read from memory, the memory address from where data is to be read must be set in BRSTADR1 and BRSTADR2 registers. When a read command is issued by writing the bit SBRSRD of the BRSTCTL1 register, the data is read from memory and stored in the data registers BRSTBUF0 to BRSTBUF15. Check the status of the SBRSWR or SBRSRD bit for data transfer completion.

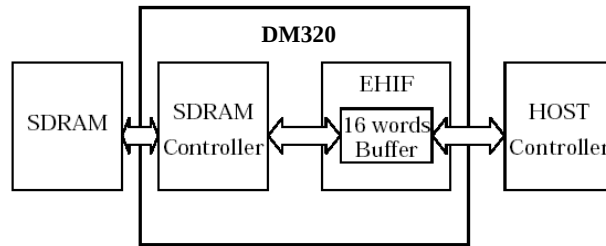


Figure 171 SDRAM Buffered Access

24.10 Continuous Burst Access (Buffered Access):

Transfer is performed using 2 sets of sixteen 16-bit word units as shown in Figure 172. To write into the memory, memory start address must be set in BRSTADR1 and BRSTADR2 registers. The total transfer size, in units of 32 bytes, must be set in CBRSLLEN register. The host CPU writes the data continuously to BRSTBUF0 register. This data is written to first set of 16-word buffer. When the 16th word is written, the burst transfer is performed automatically. The host CPU can continue writing the data to other set of 16-word buffer. In this way the continuous burst write operation is achieved. If external host writes 16-word data into a buffer before the other buffer becomes empty, WAIT signal is asserted when the WAITEN bit in EHIFCTL register is '1', and CBRSWT bit in EHIFSTS register is set. CBRSWR in BRSTCTL2 register is automatically cleared when all transfer is completed.

To read from memory, memory start address must be set in BRSTADR1 and BRSTADR2 registers. The total transfer size, in units of 32 bytes, must be set in CBRSLLEN register. After checking the CBRSWT bit in EHIFSTS register and confirming that the 16-word transfer has ended, the host CPU can read data from first set of 16-word buffer using BRSTBUF0 register. When 16th word is read, the burst transfer is performed automatically and the host CPU can read the data from other set. In this way the continuous burst read operation is achieved. If the external host reads all data in a buffer before 16-word data is transferred into the other buffer, WAIT signal is asserted if WAITEN bit in EHIFCTL register is '1', and CBRSWT bit in EHIFSTS register is set. CBRSRD bit in BRSTCTL2 register is automatically cleared when all the transfer is completed.

The flowchart in Figure 173 shows the read/write operation in continuous burst access.

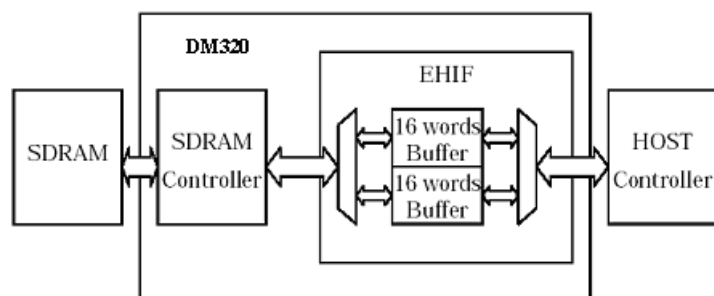


Figure 172 SDRAM Continuous Burst Access

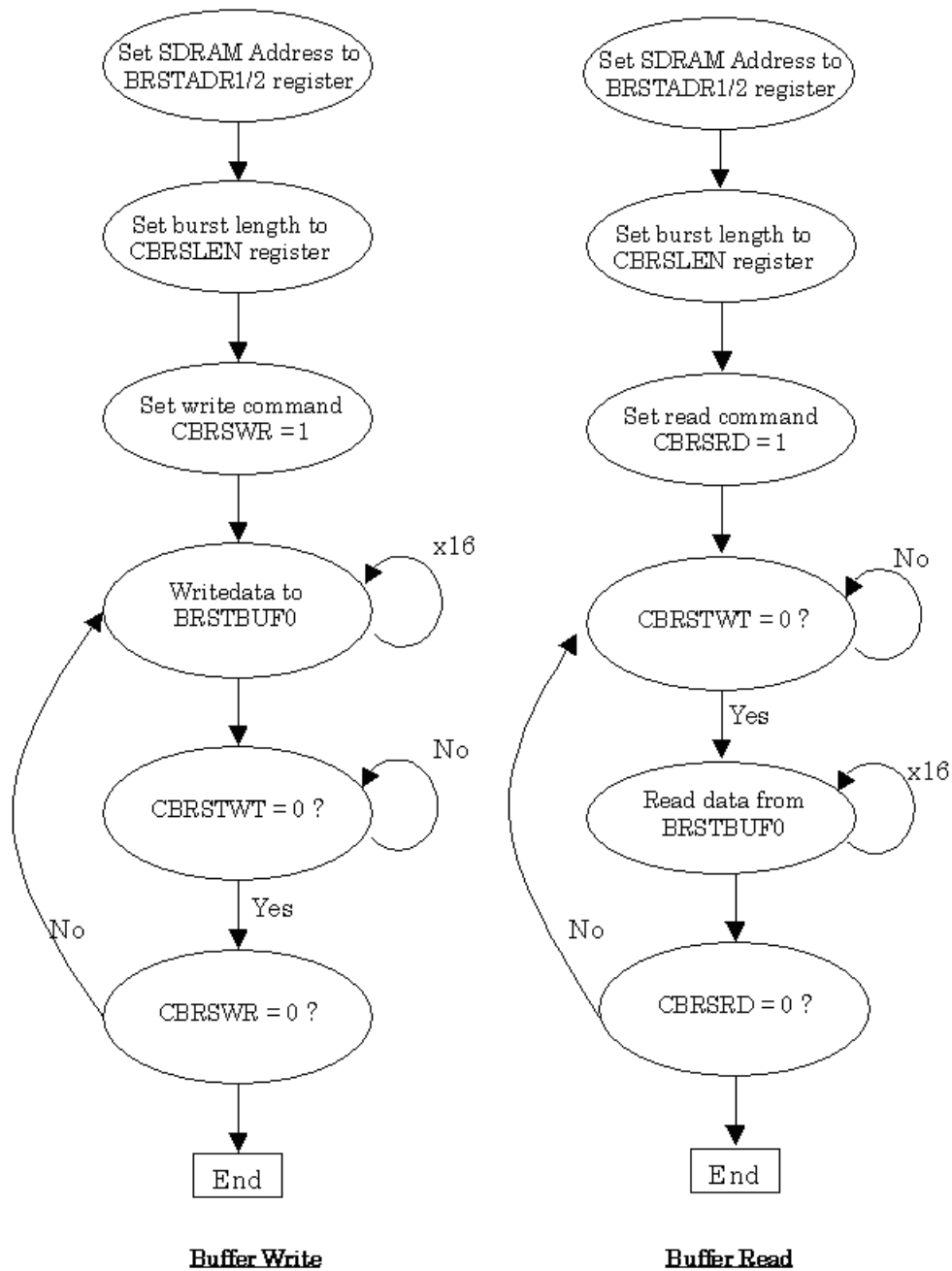


Figure 173 SDRAM Continuous Burst Read/Write Sequence

24.11 EHIF Register Map

Offset	Address	Register	Description
0x00		REVR	Revision register
0x01		CONTROL	EHIF control register
0x02		INTCTL	Interrupt control register
0x03		BRSTCTL1	Burst control register #1
0x04		BRSTCTL2	Burst control register #2
0x05		CBRSLEN	Burst length for continous burst
0x06		BRSTADR1	Burst address 1
0x07		BRSTADR0	Burst address 2
0x08		BRSTBUF0	Burst buffer 0
0x09		BRSTBUF1	Burst buffer 1
0x0A		BRSTBUF2	Burst buffer 2
0x0B		BRSTBUF3	Burst buffer 3
0x0C		BRSTBUF4	Burst buffer 4
0x0D		BRSTBUF5	Burst buffer 5
0x0E		BRSTBUF6	Burst buffer 6
0x0F		BRSTBUF7	Burst buffer 7
0x10		BRSTBUF8	Burst buffer 8
0x11		BRSTBUF9	Burst buffer 9
0x12		BRSTBUF10	Burst buffer 10
0x13		BRSTBUF11	Burst buffer 11
0x14		BRSTBUF12	Burst buffer 12
0x15		BRSTBUF13	Burst buffer 13
0x16		BRSTBUF14	Burst buffer 14
0x17		BRSTBUF15	Burst buffer 15
0x18		STATUS	EHIF status register

24.12 EHIF Registers

24.12.1 REVR

Revision Register

REVR

0

offset: 0x00

default: 0x0010

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	REV[7]	REV[6]	REV[5]	REV[4]	REV[3]	REV[2]	REV[1]	REV[0]
R/W	-	-	-	-	-	-	-	-	R	R	R	R	R	R	R	R
Default	-	-	-	-	-	-	-	-	0	0	0	1	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:8	RSV			Reserved
7:0	REV	16	R	Device Revision Id Note: ES1.0 -> 0x10

24.12.2 EHIFCTL

EHIF Control Register

CONTROL	2		offset: 0x02		default: 0x0010											
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	ENDIW[1]	ENDIW[0]	ENDIR[1]	ENDIR[0]	RSV	RSV	RSV	RTRG	RSV	RSV	RSV	WAITEN
R/W	-	-	-	-	R/W	R/W	R/W	R/W	-	-	-	R/W	-	-	-	R/W
Default	-	-	-	-	0	0	0	0	-	-	-	1	-	-	-	0

Bit	Name	Reset Value	R/W	Function
15:12	RSV			Reserved
11:10	ENDIW	0	R/W	<u>Endian select for write access</u> » 00 : No Swap « » 01 : byte swap « » 10 : word swap « » 11 : byte and word swap « Note: This bit does not affect the write to MMR registers
9:8	ENDIR	0	R/W	<u>Endian select for read access</u> » 00 : No Swap « » 01 : byte swap « » 10 : word swap « » 11 : byte and word swap « Note: This bit does not affect the read of MMR registers
7:5	RSV			Reserved
4	RTRG	1	R/W	<u>Data read trigger select</u> » 0 : CS falling edge « » 1 : OE falling edge «
3:1	RSV			Reserved
0	WAITEN	0	R/W	<u>EM_WAIT output enable</u> » 0 : High impedance « » 1 : Busy / ready state « Note: Hsot device can monitor the status of DM320 using EM_WAIT. This allows host to dynamically access DM320

24.12.3 INTCTL

EHIF Interrupt Control Register

INTCTL	4				offset: 0x04				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	DSPINT	RSV	RSV	RSV	RSV	RSV	RSV	RSV	ARMINT
R/W	-	-	-	-	-	-	-	R/W	-	-	-	-	-	-	-	R/W
Default	-	-	-	-	-	-	-	0	-	-	-	-	-	-	-	0

Bit	Name	Reset Value	R/W	Function
15:9	RSV			Reserved
8	DSPINT	0	R/W	<u>Generate interrupt to DSP</u> Write '1' to generate an interrupt, this bit will be cleared automatically
7:1	RSV			Reserved
0	ARMINT	0	R/W	<u>Generate interrupt to ARM</u> Write '1' to generate an interrupt, this bit will be cleared automatically

24.12.4 BRSTCTL1

EHIF Burst Control Register 1

BRSTCTL1	4				offset: 0x06				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	BFCLR	RSV	RSV	SBRSWR	SBRSRD
R/W	-	-	-	-	-	-	-	-	-	-	-	R/W	-	-	R/W	R/W
Default	-	-	-	-	-	-	-	-	-	-	-	0	-	-	0	0

Bit	Name	Reset Value	R/W	Function
15:5	RSV			Reserved
4	BFCLR	0	R/W	<u>Data buffer clear</u> Write '1' to clear the buffer, this bit will be cleared automatically
3:2	RSV			Reserved
1	SBRSWR	0	R/W	<u>Single burst write enable</u> Write '1' to start a 32-byte write transfer, this bit will be cleared automatically
0	SBRSRD	0	R/W	<u>Single burst read enable</u> Write '1' to start a 32-byte read transfer, this bit will be cleared automatically

24.12.5 BRSTCTL2

EHIF Burst Control Register 2

BRSTCTL2	4				offset: 0x08				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	CBRSMD	RSV	RSV	CBRSWR	CBRSRD
R/W	-	-	-	-	-	-	-	-	-	-	-	R/W	-	-	R/W	R/W
Default	-	-	-	-	-	-	-	-	-	-	-	0	-	-	0	0

Bit	Name	Reset Value	R/W	Function
15:5	RSV			Reserved
4	CBRSMD	0	R/W	<u>Continuous burst transfer mode</u> » 0 : mode-0, Burst transfer will be performed by reading / writing from data register BRSTBUF0 « » 1 : mode-1, Address decoding will not be performed and every access from a host is treated as a burst transfer request once the data transfer starts and until the transfer of the specified length finishes «
3:2	RSV			Reserved
1	CBRSWR	0	R/W	<u>Continuous burst write enable</u> Write '1' to start a continuous burst write transfer of length as specified in BUTSTLEN register, this bit will be cleared automatically, when the transfer is completed
0	CBRSRD	0	R/W	<u>Continuous burst read enable</u> Write '1' to start a continuous burst read transfer of length as specified in CBRSLLEN register, this bit will be cleared automatically, when the transfer is completed

24.12.6 CBRSLLEN

EHIF Continuous Burst Transfer Length Register

CBRSLLEN	4				offset: 0x0A				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	CBRSL[15]	CBRSL[14]	CBRSL[13]	CBRSL[12]	CBRSL[11]	CBRSL[10]	CBRSL[9]	CBRSL[8]	CBRSL[7]	CBRSL[6]	CBRSL[5]	CBRSL[4]	CBRSL[3]	CBRSL[2]	CBRSL[1]	CBRSL[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:0	CBRSL	0	R/W	Total number of 32-byte bursts

24.12.7 BRSTADR1

EHIF Burst Transfer Address Register 1

BRSTADR1	4				offset: 0x0C				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	AINC	BRSTA1[7]	BRSTA1[6]	BRSTA1[5]	BRSTA1[4]	BRSTA1[3]	BRSTA1[2]	BRSTA1[1]	BRSTA1[0]
R/W	-	-	-	-	-	-	-	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	-	-	-	-	-	-	-	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:9	RSV			Reserved
8	AINC	0	R/W	Address auto increment
7:0	BRSTA1	0	R/W	Upper 8 bits of the burst address in 32-byte units BRSTA[23:16]

24.12.8 BRSTADR2

EHIF Burst Transfer Address Register 2

BRSTADR0	4				offset: 0x0E				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	BRSTA0[15]	BRSTA0[14]	BRSTA0[13]	BRSTA0[12]	BRSTA0[11]	BRSTA0[10]	BRSTA0[9]	BRSTA0[8]	BRSTA0[7]	BRSTA0[6]	BRSTA0[5]	BRSTA0[4]	BRSTA0[3]	BRSTA0[2]	BRSTA0[1]	BRSTA0[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:0	BRSTA0	0	R/W	Lower 16 bits of burst address in 32-byte units BRSTA[15:0]

24.12.9 BRSTBUF0

EHIF Data Register for Burst Transfer

BRSTBUF0	4				offset: 0x10				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	BRB0[15]	BRB0[14]	BRB0[13]	BRB0[12]	BRB0[11]	BRB0[10]	BRB0[9]	BRB0[8]	BRB0[7]	BRB0[6]	BRB0[5]	BRB0[4]	BRB0[3]	BRB0[2]	BRB0[1]	BRB0[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:0	BRB0	0	R/W	Burst buffer 0

24.12.10 BRSTBUF1

EHIF Data Register for Burst Transfer

BRSTBUF1	4				offset: 0x12				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	BRB1[15]	BRB1[14]	BRB1[13]	BRB1[12]	BRB1[11]	BRB1[10]	BRB1[9]	BRB1[8]	BRB1[7]	BRB1[6]	BRB1[5]	BRB1[4]	BRB1[3]	BRB1[2]	BRB1[1]	BRB1[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:0	BRB1	0	R/W	Burst buffer 1

24.12.11 BRSTBUF2

EHIF Data Register for Burst Transfer

BRSTBUF2	4				offset: 0x14				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	BRB2[15]	BRB2[14]	BRB2[13]	BRB2[12]	BRB2[11]	BRB2[10]	BRB2[9]	BRB2[8]	BRB2[7]	BRB2[6]	BRB2[5]	BRB2[4]	BRB2[3]	BRB2[2]	BRB2[1]	BRB2[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:0	BRB2	0	R/W	Burst buffer 2

24.12.12 BRSTBUF3

EHIF Data Register for Burst Transfer

BRSTBUF3				4	offset: 0x16				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	BRB3[15]	BRB3[14]	BRB3[13]	BRB3[12]	BRB3[11]	BRB3[10]	BRB3[9]	BRB3[8]	BRB3[7]	BRB3[6]	BRB3[5]	BRB3[4]	BRB3[3]	BRB3[2]	BRB3[1]	BRB3[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:0	BRB3	0	R/W	Burst buffer 3

24.12.13 BRSTBUF4

EHIF Data Register for Burst Transfer

BRSTBUF4				4	offset: 0x18				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	BRB4[15]	BRB4[14]	BRB4[13]	BRB4[12]	BRB4[11]	BRB4[10]	BRB4[9]	BRB4[8]	BRB4[7]	BRB4[6]	BRB4[5]	BRB4[4]	BRB4[3]	BRB4[2]	BRB4[1]	BRB4[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:0	BRB4	0	R/W	Burst buffer 4

24.12.14 BRSTBUF5

EHIF Data Register for Burst Transfer

BRSTBUF5				4	offset: 0x1A				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	BRB5[15]	BRB5[14]	BRB5[13]	BRB5[12]	BRB5[11]	BRB5[10]	BRB5[9]	BRB5[8]	BRB5[7]	BRB5[6]	BRB5[5]	BRB5[4]	BRB5[3]	BRB5[2]	BRB5[1]	BRB5[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:0	BRB5	0	R/W	Burst buffer 5

24.12.15 BRSTBUF6

EHIF Data Register for Burst Transfer

BRSTBUF6	4				offset: 0x1C				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	BRB6[15]	BRB6[14]	BRB6[13]	BRB6[12]	BRB6[11]	BRB6[10]	BRB6[9]	BRB6[8]	BRB6[7]	BRB6[6]	BRB6[5]	BRB6[4]	BRB6[3]	BRB6[2]	BRB6[1]	BRB6[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:0	BRB6	0	R/W	Burst buffer 6

24.12.16 BRSTBUF7

EHIF Data Register for Burst Transfer

BRSTBUF7	4				offset: 0x1E				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	BRB7[15]	BRB7[14]	BRB7[13]	BRB7[12]	BRB7[11]	BRB7[10]	BRB7[9]	BRB7[8]	BRB7[7]	BRB7[6]	BRB7[5]	BRB7[4]	BRB7[3]	BRB7[2]	BRB7[1]	BRB7[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:0	BRB7	0	R/W	Burst buffer 7

24.12.17 BRSTBUF8

EHIF Data Register for Burst Transfer

BRSTBUF8	4				offset: 0x20				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	BRB8[15]	BRB8[14]	BRB8[13]	BRB8[12]	BRB8[11]	BRB8[10]	BRB8[9]	BRB8[8]	BRB8[7]	BRB8[6]	BRB8[5]	BRB8[4]	BRB8[3]	BRB8[2]	BRB8[1]	BRB8[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:0	BRB8	0	R/W	Burst buffer 8

24.12.18 BRSTBUF9

EHIF Data Register for Burst Transfer

BRSTBUF9				4				offset: 0x22								default: 0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	BRB9[15]	BRB9[14]	BRB9[13]	BRB9[12]	BRB9[11]	BRB9[10]	BRB9[9]	BRB9[8]	BRB9[7]	BRB9[6]	BRB9[5]	BRB9[4]	BRB9[3]	BRB9[2]	BRB9[1]	BRB9[0]			
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W			
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			

Bit	Name	Reset Value	R/W	Function
15:0	BRB9	0	R/W	Burst buffer 9

24.12.19 BRSTBUF10

EHIF Data Register for Burst Transfer

BRSTBUF10				4				offset: 0x24								default: 0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	BRB10[15]	BRB10[14]	BRB10[13]	BRB10[12]	BRB10[11]	BRB10[10]	BRB10[9]	BRB10[8]	BRB10[7]	BRB10[6]	BRB10[5]	BRB10[4]	BRB10[3]	BRB10[2]	BRB10[1]	BRB10[0]			
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W			
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			

Bit	Name	Reset Value	R/W	Function
15:0	BRB10	0	R/W	Burst buffer 10

24.12.20 BRSTBUF11

EHIF Data Register for Burst Transfer

BRSTBUF11		4				offset: 0x26										default: 0x0000	
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Name	BRB11[15]	BRB11[14]	BRB11[13]	BRB11[12]	BRB11[11]	BRB11[10]	BRB11[9]	BRB11[8]	BRB11[7]	BRB11[6]	BRB11[5]	BRB11[4]	BRB11[3]	BRB11[2]	BRB11[1]	BRB11[0]	
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

Bit	Name	Reset Value	R/W	Function
15:0	BRB11	0	R/W	Burst buffer 11

24.12.21 BRSTBUF12

EHIF Data Register for Burst Transfer

BRSTBUF12				4				offset: 0x28								default: 0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	BRB12[15]	BRB12[14]	BRB12[13]	BRB12[12]	BRB12[11]	BRB12[10]	BRB12[9]	BRB12[8]	BRB12[7]	BRB12[6]	BRB12[5]	BRB12[4]	BRB12[3]	BRB12[2]	BRB12[1]	BRB12[0]			
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W			
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			

Bit	Name	Reset Value	R/W	Function
15	BRB12	0	R/W	Burst buffer 12

24.12.22 BRSTBUF13

EHIF Data Register for Burst Transfer

BRSTBUF13	4				offset: 0x2A				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	BRB13[15]	BRB13[14]	BRB13[13]	BRB13[12]	BRB13[11]	BRB13[10]	BRB13[9]	BRB13[8]	BRB13[7]	BRB13[6]	BRB13[5]	BRB13[4]	BRB13[3]	BRB13[2]	BRB13[1]	BRB13[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15	BRB13	0	R/W	Burst buffer 13

24.12.23 BRSTBUF14

EHIF Data Register for Burst Transfer

BRSTBUF14				4				offset: 0x2C								default: 0x0000			
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Name	BRB14[15]	BRB14[14]	BRB14[13]	BRB14[12]	BRB14[11]	BRB14[10]	BRB14[9]	BRB14[8]	BRB14[7]	BRB14[6]	BRB14[5]	BRB14[4]	BRB14[3]	BRB14[2]	BRB14[1]	BRB14[0]			
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W			
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			

Bit	Name	Reset Value	R/W	Function
15	BRB14	0	R/W	Burst buffer 14

24.12.24 BRSTBUF15

EHIF Data Register for Burst Transfer

BRSTBUF15	4				offset: 0x2E				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	BRB15[15]	BRB15[14]	BRB15[13]	BRB15[12]	BRB15[11]	BRB15[10]	BRB15[9]	BRB15[8]	BRB15[7]	BRB15[6]	BRB15[5]	BRB15[4]	BRB15[3]	BRB15[2]	BRB15[1]	BRB15[0]
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Default	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Name	Reset Value	R/W	Function
15:0	BRB15	0	R/W	Burst buffer 15

24.12.25 EHIFSTS

EHIF Status Register

STATUS	4				offset: 0x30				default: 0x0000							
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Name	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	RSV	AHBBSY	RSV	RSV	RSV	CBRSWT
R/W	-	-	-	-	-	-	-	-	-	-	-	R	-	-	-	R
Default	-	-	-	-	-	-	-	-	-	-	-	0	-	-	-	0

Bit	Name	Reset Value	R/W	Function
15:5	RSV			Reserved
4	AHBBSY	0	R	<u>AHB Busy flag</u> » 0 : AHB Bus controller - EHIF interface is free « » 1 : AHB Bus controller - EHIF interface is busy « Note: Transfer can take place when the AHBBSY bit is '0'
3:1	RSV			Reserved
0	CBRSWT	0	R	<u>Continuous burst transfer wait flag</u> » 0 : No wait state asserted « » 1 : Wait state asserted « Note: Transfer can take place when the CBRSWT bit is '0'

25 ARM Boot-Up Modes

DM320 provides three different ARM boot-up sequences. These are

- o External memory (CS0) boot-up mode (boot-image in a linear FLASH)
- o External Host serial and EMIF boot-up mode (boot-image in external host)
- o Internal ROM boot-up mode (boot-image in NAND flash)

The BTSEL[1:0] pins define how ARM will function after power-on reset. 0 shows different pin configurations.

Pin	Status	Function
BTSEL[1:0]	00	CS0 - External Memory boot up Jump to external Flash memory at address 0xFFFF:0000 (mirror of 0x0010:0000)
	01	Serial and External host boot up Jump to ARM internal memory, after boot-code is downloaded from the external host through serial port or through external memory interface.
	10	Internal ROM boot-up Jump to ARM internal ROM, copy the user boot-loader from NAND flash to ARM internal RAM and give control to user boot-loader. The user boot-loader will load the application to SDRAM from NAND flash.
	11	CS0 - External Memory boot up Jump to external Flash memory at address 0xFFFF:0000 (mirror of 0x0010:0000)

Table 100: ARM boot-up configuration

In external memory (CS0) boot-up mode, the CPU executes directly from the linear flash.

External host serial boot-up mode and external host EMIF boot-up mode makes it possible to download an ARM boot code / application from the external host. Typically since the IIRAM has only 16KB memory, a boot-loader will be loaded, which will load the application to SDRAM, from a flash device.

Internal ROM is a boot-loader code, which will download an user boot-loader stored in NAND flash. This user boot-loader will load the final application to SDRAM from the NAND flash.

25.1 External Memory boot-up mode

Upon power up and reset, the ARM program counter is reset to 0xFFFF:0000 (ARM reset vector, when VINTH is high). When BTSEL[1:0] is “00”, the program counter is set to 0xFFFF:0000. The memory in the region 0xFFFF:0000 to 0xFFFF:FFFF is the mirror of the first 64KB location of the CS0 address space from 0x0010:0000 to 0x0010:FFFF. The ARM should execute a boot sequence from 0xFFFF:0000 region to start the DM320 system. A far jump is required for a branch from 0xFFFF:xxxx to 0x0010:xxxx.

Depending upon the state of the VINTH signal, which can be controlled through CP15 also, the interrupt vector table (IVT) location changes. If VINTH is high, the IVT is at 0xFFFF:0000 and if VINTH is low, the IVT is at 0x0000:0000.

After power-on reset, it takes about 10 ARM clock cycles to access CS0 (FLASH_CE) region. In CS0 boot mode, the external pins - BTSEL[1:0], should be tied to low (ground) level.

25.2 Serial boot-up mode

DM320 program/data can be downloaded into ARM internal memory, externally through serial interface (SIF0) and UART (UART0) pins. During reset, the external CPU can gain access to ARM internal memory by using SIF/UART. This interface consists of four signals serial clock input (SCLK0), enable signal input (SDEN0), serial data input (RXD0) and serial data output (TXD0). The external CPU can write the necessary data to ARM internal memory (AIM), for instance boot loader code, and read out the data from AIM.

The followings are the features in this mode:

- Full addressing of ARM internal memory (AIM) from 0x0 to 0x7FFF
- Access mode is a byte mode only
- Address can be incremented automatically after sending first address
- Supports both write and read operation
- Enabled during global reset.

Figure 10 and Figure 11 show the timing diagram for reading and writing data via the serial interface.

The first received bit indicates read/write command and it is followed by a 15-bit address. When the command bit is low, a write operation is to be performed and when the command bit is high, a read operation is to be performed.

Following the command and address bits, data of 8-bit length is sent to the AIM in the write mode via the RXD0 pin. In the read mode, data of 8-bit length from AIM will be transferred through data out (TXD0) pin.

The enable signal is active low during this process. The address to be accessed is incremented automatically when enable signal is low.

The minimum unit of the transfer is 8 bits. If the data transferred is less than 8 bits it will be discarded.

The interface has the below protocol:

- The address/data to be transferred is MSB first.
- The maximum frequency of SCLK0 is 10MHz.
- Performed while the power on reset signal is in assertion

IMPORTANT: When program/data is to be downloaded via the serial interface, BSTEL[1:0] pins must be in “01” state and reset signal must be asserted. After all the data is transferred to the AIM, the reset signal is deasserted and ARM program counter (PC) will get loaded with address 0x0000 (ARM reset vector) automatically. The user must make sure a branch to user initialization routine is set at this location (0x0 to 0x0004) during the downloading of program via the serial interface

Figure 12 shows the detailed timing sequence of ARM boot via serial interface

Figure 10. Serial data read timing

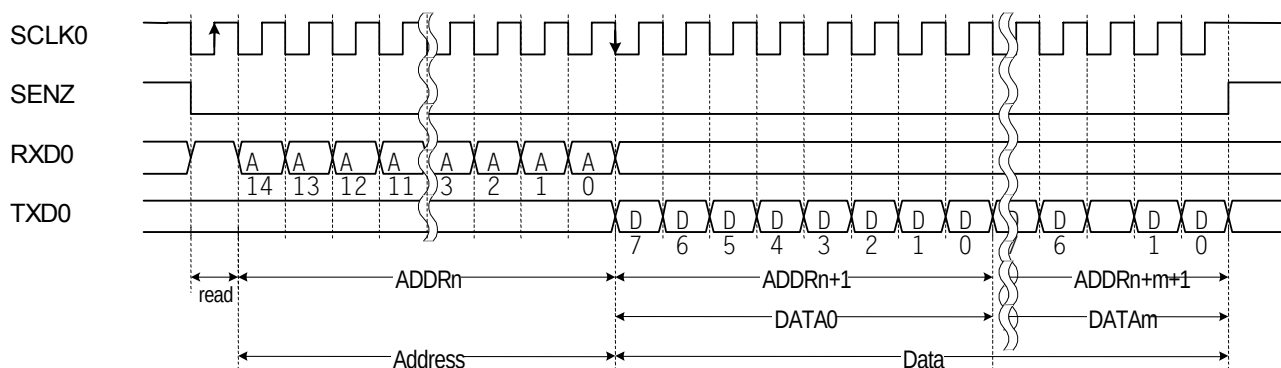


Figure 11. Serial data write timing

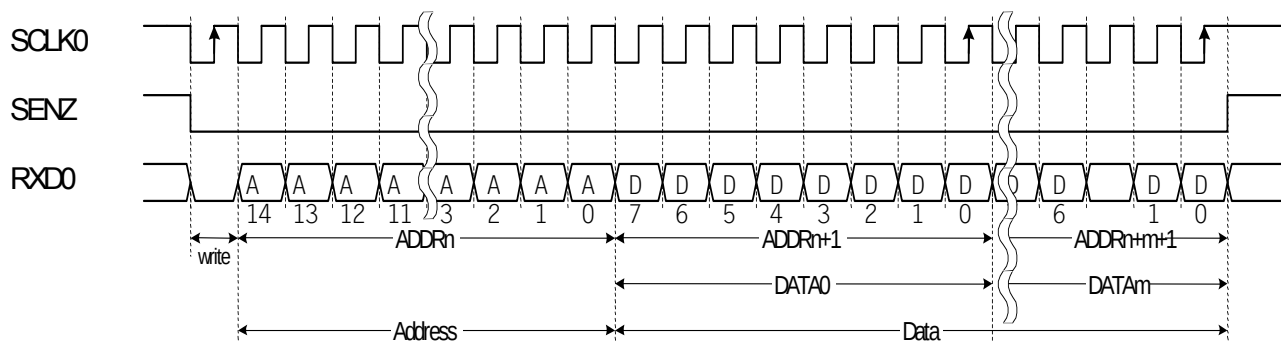
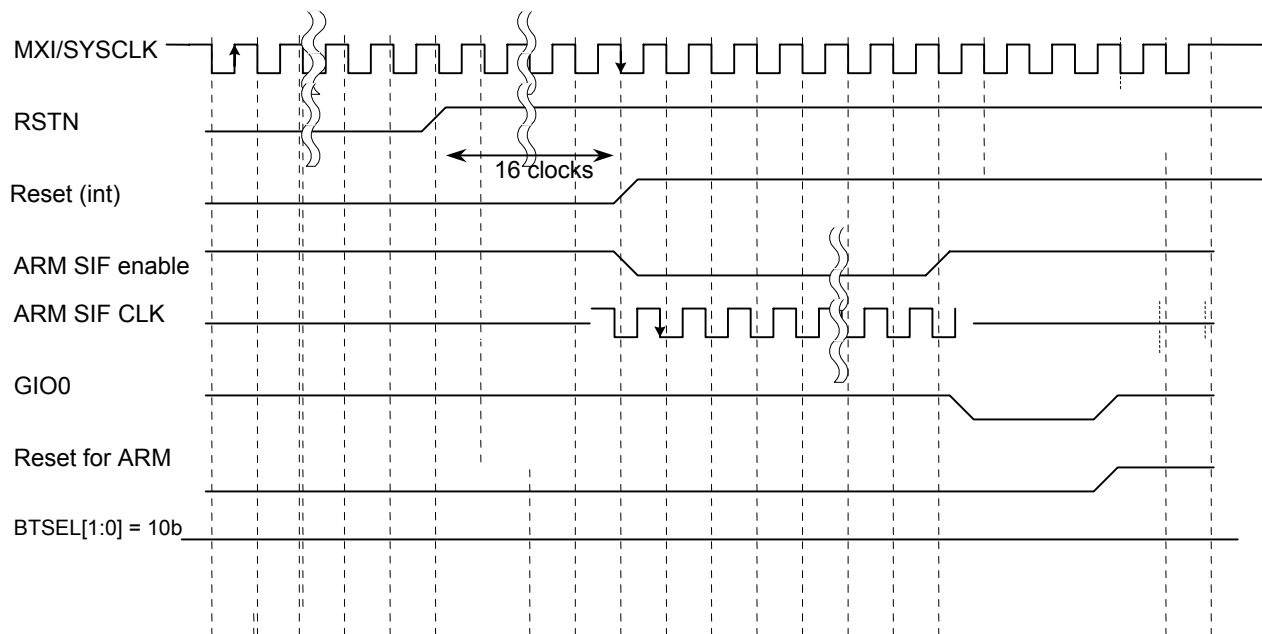


Figure 12. Timing Sequence of ARM boot via serial interface

DM320 Sequence of ARM Boot-up via AIM SIF:

1. BTSEL[1:0]="10", RSTN=L, (EXTHOST=L)
2. RSTN=H
3. Wait 16 cycles at MXI/SYSCLK
4. Load ARM code to AIM via AIMSIF
5. To deassert system reset, make GIO0=H -> L (2 cycles)



25.3 External HOST EMIF boot-up mode

External HOST can access ARM internal memory directly through EMIF bus pins. See chapter “External HOST Interface” for details about memory access in external HOST mode. This makes it possible for the external HOST to download an ARM boot code externally.

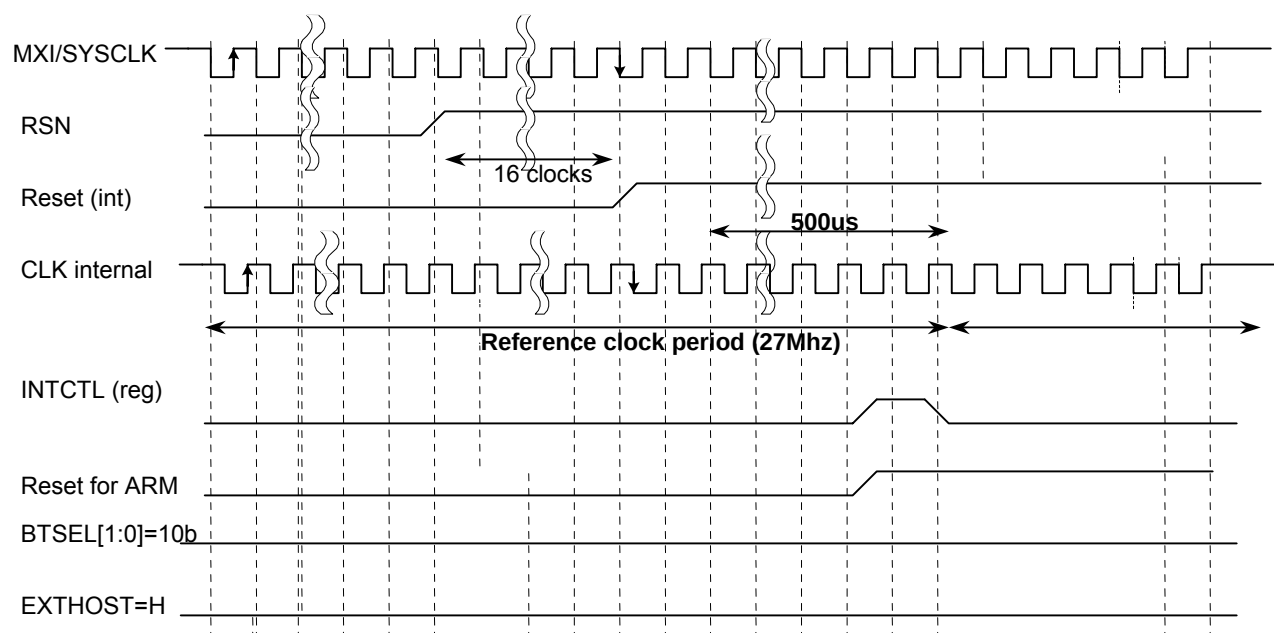
IMPORTANT: When program/data is to be downloaded via the external HOST boot mode, BSTEL[1:0] pins must be in “10” state and reset signal must be asserted. EXTHOST should also be high in external HOST mode. After the program/data is transferred to the AIM, the reset signal is deasserted and ARM program counter (PC) will be loaded with address 0x0000 (ARM reset vector) automatically. The user must make sure a branch to user initialization routine is set at this location (0x0 to 0x0004) during the downloading of program by the external HOST.

Figure 13 shows the detail timing sequence of ARM boot via external HOST boot-up mode.

Figure 13. Timing sequence of ARM boot via external HOST boot-up mode.

DM320 Sequence of ARM Boot-up via EXTHOST:

1. EXTHOST=H, BSTSEL[1:0]=“10”, RSN=L
2. RSN=H
3. Wait 16 cycles at MXI/SYSCLK
4. Load ARM code to AIM via external host
5. To deassert system reset, write 0x0001 into INTCTL(0x04) register of EHIF



25.4 Internal ROM boot-up mode

DM320 ROM boot-loader uses a two-stage boot-loader approach. The ROM boot-loader does the basic initialization of system and copy user boot-loader to internal memory and give control to user boot-loader. User boot-loader does a full system initialization (including SDRAM), and copy application code to SDRAM. The user boot-loader will then give control to the application.

This approach gives flexibility to the user to use different types of SDRAM, NAND flash etc. The functionality is divided between ROM boot-loader and user boot-loader. Detailed ROM boot-loader and user boot-loader functional requirements are given below.

The maximum size of User boot-loader is 8 Kilobytes. This is because the maximum size of IRAM1 region is 8KB and this is where the user boot-loader will be copied. Data / Code cannot be copied to IRAM0 because the address changes after a TCM remap is done.

25.4.1 Functions of ROM boot-loader

ROM boot loader initializes the DM320 system and copy the “User boot loader” from NAND flash to ARM internal memory and give control to the user boot-loader. Various functions performed by ROM boot-loader is given below:

- o Initialize the stack in RAM0 in IRAM (IRAM0 -> 0x4000-0x5FFF)
- o Initialize the clock controller (PLLs in Bypass mode, required clocks are enabled)
- o Disable all interrupts, IRQ and FIQ
- o Make VINTH = 0 in CP15 to make the vector table at address 0
- o Initialize the NAND region
- o Read the device Id of the attached NAND.
- o From the device Id find the characteristics of the device present (Max number of blocks, number of pages per block, bytes per page etc).
- o Get the details of user boot-loader from the last page of NAND (Where details about User boot loader is stored). The information stored and the sequence is given below:
 - o Magic Number (0xA1ACEDxx)
 - o Entry point (0x2000-0x3FFF)
 - o Number of pages
 - o Starting block where user boot-loader is stored
 - o Page in the block where user-boot-loader starts
- o Copy the User boot-loader from NAND flash to RAM1 in IRAM (IRAM1-> 0x2000-0x3FFF)
- o Give control to User boot loader

25.4.2 Functions of User boot-loader

User boot loader re-initializes the DM320 system including SDRAM. It then copies the application code from NAND flash to SDRAM and give control to the application. The functions done by user boot-loader is given below:

- o TCM re-map to make RAM0 from 0x0000-0x1FFF
- o Initialize the stack in RAM0 in IIRAM (RAM0 -> 0x0000-0x1FFF)
- o Enable IRQ and FIQ
- o Configure CLKC and SDRAM
- o Clock Controller settings: PLL configuration, Clock enables, etc.
- o SDRAM start/end addresses in AHB Bus controller register
- o SDRAM Controller settings: SDRAM configuration information and setup sequence
- o Copy the application code from NAND to SDRAM (Information is embedded in User boot loader)
- o Give control to the application code

26 ARM Power Saving Modes

Please refer to the chapter on “Clock Controller” for more details about clock controller features.

The DM320 consumes a low amount of power during normal operation. However, it is possible to further reduce the DM320's power consumption via the various power saving modes.

The ARM9 subsystem used in the DM320 features several power-saving modes to increase the life of battery in battery-powered systems. The content of the internal RAM, the CPU registers and the peripheral registers settings are maintained when the device is in any of the power saving modes.

Depending on the power down mode chosen, there are different ways to enter and exit the mode. Each one is described in this chapter.

Refer to the chapter on DSP subsystem for information on DSP-side power saving modes.

26.1 Disabling Module Clocks

To reduce the power consumption in the ARM Subsystem, disable the clock to any module, which is not being used. For example, clock supplied to the Preview Engine can be disabled when the system does not require the Preview Engine module.

Use the registers MOD0, MOD1 and MOD2 in the Clock Controller module to disable module clocks. Refer the chapter on “Clock Controller” for more information on these registers.

If clock supply to the ARM core, ARM internal memory and bus controller is disabled, DM320 is completely stopped. Therefore, these clocks must only be disabled at the end.

Even if all clocks in DM320 are disabled, RAM contents are saved as long as power is not lost, and it is not necessary to reload ARM and DSP programs. To save SDRAM contents, the SDRAM must be set to self-refresh mode before stopping the SDRAM clock.

Recovery from sleep is performed by WAKEUP pin. A pulse signal needs to be applied on the WAKEUP pin (active low), for at least two cycles of SYSCLK or MXI clock. Since the PLLs are still in operation, the full recovery is quick.

26.2 USB Power Saving Modes

The DM320's USB module offers the following power saving features.

26.2.1 USB I/O buffer power down

The bit USBPDN in the LPCTL1 register in Clock controller allows disabling the USB I/O buffers in DM320.

26.2.2 Disabling the Oscillation of M48XI crystal unit

The bit OSC48 in the LPCTL1 register in clock controller allows disabling the oscillation of the crystal unit connected to the M48XI input. Disable the crystal unit when the system does not require the M48XI clock input. This is applicable only when USB is using the M48XI clock.

26.3 Sleep Mode

The DM320 is in sleep mode when all module clocks are disabled. The wake up is performed using GIOs.

26.3.1 Entering Sleep Mode

The Sleep Mode bit, SLEEP, in LPCTL0 register in clock controller allows to enter the sleep mode. When SLEEP is set to "1" all module clocks are disabled.

Put CLK_ARM, CLK_DSP, CLK_SDR, and CLK_AXL in bypass state in advance by using BYP register in clock controller.

Furthermore, required configuration must be done in advance for external devices that may suffer data loss from stopping clock supply. For example for SDRAM, configure SDRC to self-refresh mode, before entering sleep mode.

Although all module clocks are disabled in sleep mode, the bits in MOD0, MOD1 and MOD2 registers are not affected. Therefore, any module clock that was enabled before entering sleep mode will be re-enabled upon wake-up from sleep mode.

26.3.2 Wake-up From Sleep Mode

GIO0 or GIO8 can be used to wake-up DM320. To use GIO for wakeup, inverse and latch settings needs to be configured in GIO module, before going into the sleep mode.

Pull the configured GIO pin low for at least the width of two CLK_AHB cycles to wake-up from sleep mode. Upon wake-up from sleep mode, the PLL's start their locking process and each clock as enabled by bits in the MOD0, MOD1 and MOD2 registers become active. Some bits of the MOD0 are set to '1' regardless of the prior state at the time of wake-up from sleep mode. Please refer to the section on the Clock Controller Registers for details.

An interrupt (INT_CLKC) is generated at the time of wake-up. Then PLLs start the locking process and will be in locked state within the PLL lock-in period. Please refer to the DM320 datasheet for the PLL lock-in period.

After PLL is in locked state, the bypass state of CLK_ARM, CLK_DSP, CLK_SDR and CLK_AXL is disabled.

NOTE: SLEEP bit is automatically cleared upon wakeup from sleep mode.

26.4 Power Down Mode

The DM320 is said to be in power down mode when the DM320 is in sleep mode, PLLs are powered down and the oscillations of the crystal unit connected to MXI and M48XI are stopped.

26.4.1 PWRDN Pin

As shown in *Table 101*, when the PWRDN pin is high (pulled-up), oscillation of the crystal unit connected to MXI and M48XI will stop, and the crystal units will oscillate when PWRDN is low (pulled-down).

Note: When PWRDN is low, the crystal units oscillate even if the RSTN signal is asserted.

PWRDN state	Oscillators
Low	Running
High	Stopped

Table 101: PWRDN pin

26.4.2 Entering Power Down Mode

Furthermore, required configurations must be done in advance for external devices that may suffer damage from stopping clock supply. For example for SDRAM, configure SDRC to self-refresh mode, before entering power-down mode.

Power Down mode can be entered by two methods:

- Normal Power Down Mode
 - Configure the GIO, which will be used for wake-up.
 - When power down mode is to be entered. Set the Sleep Mode bit, SLPMD, in LPCTL0 register of clock controller to enter sleep mode
 - After this, pull-up the PWRDN pin to stop the oscillation of the crystal units
- Auto-Power Down Mode,
 - Set PDMD bit in LPCTL1 register of clock controller
 - When power down mode is to be entered, just pull-up the PWRDN pin. This will automatically stop PLL and clock to all modules (sleep mode). After this it stops the oscillation of the crystal units connected to MXI and M48XI pins

26.4.3 Wake-up From Power Down Mode

To wake-up the device from a power down state, the following needs to be done:

- Wake-up from Normal Power Down Mode
 - Pull-down the PWRDN pin to start the oscillation of the crystal units.
 - Wait appropriate time until the oscillation of the crystal units are stable. Refer DM320 datasheet for more details.
 - Pull configured GIO pin low for at least two CLK_AHB cycles to wake-up from normal power down mode. An interrupt (INT_CLKC) is generated and the distribution of clock signal to each module is enabled.*
 - Then PLLs start the locking process and will be in locked state within the PLL lock-in period. Please refer to the DM320 datasheet for the PLL lock-in period.
 - After PLL is in locked state, the bypass state of CLK_ARM, CLK_DSP, CLK_SDR and CLK_AXL is disabled.
- Wake-up from Auto Power Down Mode
 - Pull-down the PWRDN pin to start the wake-up process from auto power down mode.
 - During wake-up an interrupt (INT_CLKC) is generated and the distribution of clock signal to each module is enabled.*
 - Then PLLs start the locking process and will be in locked state within the PLL lock-in period. Please refer to the DM320 datasheet for the PLL lock-in period.
 - After PLL has been in locked state, the bypass state of CLK_ARM, CLK_DSP, CLK_SDR and CLK_AXL is disabled.

***NOTE:** Each clock enabled by bits in MOD0, MOD1 and MOD2 become active. Some bits of MOD0 are set to '1' regardless of the former state. Please refer to the section on the Clock Controller Registers for details.

26.5 Entering the SDRAM into self refresh mode

It is possible to enter the SDRAM into self-refresh mode. This can further reduce system power.

To enter the SDRAM into self refresh:

- Disable any accesses to the SDRAM.
- Put the SDRAM in self-refresh state by configuring the SDRAM controller.
- Disable clock supply to the SDRAM Controller by CSDRC bit of the MOD0.
- Release the SDRAM bus by setting the bit SDREL in the BUSRSL register of EMIF module.

When the SDRAM bus is released, the SDRAM address, data, and control lines are all set to high impedance state. Data lines are pulled down. Please refer to the chapter on MTC (Memory Traffic Controller) for details.

To recover from self-refresh mode:

- Take the SDRAM bus out of the release state by clearing the bit SDREL in the EMIF register BUSRSL
- Enable clock supply to the SDRAM Controller.
- Perform the SDRAM recovery sequence from its self-refresh mode by configuring the SDRAM controller.
- Enable accesses to the SDRAM.

26.6 PLL power down mode

When either of two PLLs is not necessary, users can put the unused PLL into its power down mode. It is done by PWRDN bit of PLLA or PLLB. Please refer to the chapter on “Clock Controller” for details.

27 Appendix A

27.1 Chip Support Library (CSL) Macro Notation

27.1.1 Introduction

The CSL macro notation is used in many sections in the DM320 TRM.

This section introduces the ARM CSL (Chip Support Library) macros, and describes the procedure to use these macros to program the various peripheral registers present in the ARM sub-system of the DM320.

The DM320 ARM CSL includes a set of macros to manipulate registers and fields within peripheral registers present in the ARM sub-system. It uses the same standard protocol to program those peripherals as the TMS320C54x Chip Support Library (CSL).

27.1.2 ARM Peripheral Modules

The DM320 ARM CSL provides set of macros to program the registers in the following peripheral modules.

Peripheral	CSL Peripheral Prefix (PER)
Timer 0 - 3	TMR0 TMR1 TMR2 TMR3
Serial Port Interface 0, 1	SP0 SP1
UART 0, 1	UART0 UART1
Watch Dog Timer	WDT
MMC/SD Card Interface	MMCS
Interrupt Controller	INTC
General I/O	GIO
DSP Controller	DSPC
On-Screen Display	OSD
CCD Controller	CCDC
Preview Engine	PREV
Video Encoder	VENC
Clock Controller	CLKC
Bus Controller	BUSC
SDRAM Controller	SDRC
External Memory Interface	EMIF
USB 2.0 Full speed OTG	USB0
USB DMA	USBDMA
3A Function	H3A
Memory Stick Controller	MS
I ² C Bus Interface	I2C
VLYNQ interface	VLYNQ
AHB Bus controller	AHBBUSC

Table 102: ARM Peripheral list

27.1.3 Using the CSL Macros

The ARM CSL provides macros to read, write peripheral registers and its individual fields. It provides efficient and consistent interface to access registers/fields of different peripherals. The macros take care of the field masking ('and'-ing, 'or'-ing) of fields and thus keep the user code clean.

The table below provides a generic description of the ARM CSL macros used on DM320 where,

- *PER*, indicates a peripheral, as listed in Table 102(eg. CCDC, TMR0 . . .)
- *REG*, indicates, if applicable, a register. The register name is as defined in the DM320 TRM (eg. SYNCEN, MODESET . . .)
- *FIELD*, indicates, if applicable, a register field. The register field name is as defined in the DM320 TRM (eg. WEN, VDHDEN . . .)
- *regval*, indicates an integer constant or integer variable
- *fieldval*, indicates an integer constant or integer variable; all field values are right justified

Table 1. CSL Macros

Macro	Description
<i>PER_RGET</i> (<i>REG</i>)	Register GET - Returns the value in the peripheral register.
<i>PER_RSET</i> (<i>REG</i> , <i>regval</i>)	Register SET - Writes the value to the peripheral register.
<i>PER_FGET</i> (<i>REG</i> , <i>FIELD</i>)	Field Get - Returns the value of the specified <i>FIELD</i> in the peripheral register.
<i>PER_FSET</i> (<i>REG</i> , <i>FIELD</i> , <i>fieldval</i>)	Field SET - Writes <i>fieldval</i> to the specified <i>FIELD</i> in the peripheral register.

27.1.4 Example usage of CSL macros

To enable CCDC HD/VD and CCDC raw data write to SDRAM use the macros as follows:

```
CCDC_FSET(SYNCEN, WEN, 1);
CCDC_FSET(SYNCEN, HDVDEN, 1);
```

The register SYNCEN and field names WEN, HDVDEN are as defined in DM320 TRM.

To set the OSD video window 1 start address 'vwin1_start' use the macros as follows:

```
OSD_FSET(VIDWINADH, V1AH, vwin1_start >> 16 );  
OSD_RSET(VINWIN1AL, vwin1_start & 0xFFFF );
```

The register VIDWINADH, VIDWIN1ADL and field name V1AH are as defined in DM320 TRM

To wait for MMCSD DMA to complete use the macro as follows:

```
while ( MMCSD_FGET( DMASTAT1, RUNST ) == 1 );
```

The register DMASTAT1 and field name RUNST are as defined in DM320 TRM.